

NESHBOY SOC PROFESSIONAL LIBRARY

THE SOC MANAGER'S OPERATING HANDBOOK

Staffing, Shift Design, Quality, Vendors, Budget, and Crisis
Leadership — Running the Team, Not the Queue

Build date: 2026-09-16

Table of Contents

CHAPTERS

Section A — SOC Foundations & Operating Models

- Part 1 — SOC Manager Foundations & the Series Map
- Part 2 — SOC Operating Models: In-House, MSSP, Co-Managed, Hybrid
- Part 3 — Tiering Models: L1/L2/L3 and Beyond
- Part 4 — Organizational Placement & Charter

Section B — Staffing, Shift Design & Workforce Planning

- Part 5 — Headcount & Capacity Modeling
- Part 6 — Shift Pattern & Coverage Design
- Part 7 — Hiring & Sourcing Analysts
- Part 8 — Interviewing & Technical Assessment Design
- Part 9 — Queue Health & Workload Management

Section C — Analyst Development: Competency, Training & Career Progression

- Part 10 — Competency Models & Skills Matrices
- Part 11 — Onboarding & Ramp-Up Programs
- Part 12 — Ongoing Training & Skill Development
- Part 13 — Career Ladders & Promotion Criteria
- Part 14 — Mentorship & Knowledge Transfer

Section D — Quality, Performance & Culture

- Part 15 — Quality Assurance Programs
- Part 16 — Performance Management & Coaching
- Part 17 — Burnout, Fatigue & Wellbeing
- Part 18 — Attrition & Retention
- Part 19 — Team Culture & Psychological Safety

Section E — Vendor, Tooling & Budget Management

- Part 20 — Building & Defending the SOC Budget
- Part 21 — Tooling Procurement & Platform Strategy
- Part 22 — MSSP & Managed-Service Contract Management
- Part 23 — Vendor Relationship & Renewal Management

Section F — Governance, Risk & the Executive Interface

- Part 24 — Executive & Board Reporting
- Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty
- Part 26 — Cross-Team Politics & Stakeholder Alignment
- Part 27 — Legal, HR & Compliance Interfaces

Section G — Crisis Leadership & Major Incident Management

Part 28 — The Manager's Role in a Major Incident

Part 29 — Post-Incident Organizational Review

Section H — Maturity, Benchmarking & the Road Ahead

Part 30 — SOC Maturity Models

Part 31 — Benchmarking & Industry Comparison

Part 32 — AI, Automation, and the Future Shape of the SOC

Part 33 — The Next Twelve Months: Building a SOC Roadmap

APPENDICES

Appendix A1 — Staffing, Shift-Design & Capacity-Model Templates

Appendix A2 — Hiring, Interviewing & Competency Templates

Appendix A3 — Onboarding, Training & Career-Ladder Templates

Appendix A4 — QA, Performance & Retention Templates

Appendix A5 — Vendor, Contract & Procurement Templates

Appendix A6 — Budget & Business-Case Templates

Appendix A7 — Executive, Board & Risk-Acceptance Templates

Appendix A8 — Cross-Series Quick Reference: Where to Find It

CHAPTERS

Section A — SOC Foundations & Operating Models

neshbooy

Part 1 — SOC Manager Foundations & the Series Map

Why this part exists

[CONCEPT] Ask an analyst what their job is and they describe a queue: alerts arrive, get triaged, get escalated or closed. Ask a detection engineer and they describe a pipeline: telemetry arrives, gets normalized, gets matched against logic someone tuned. Ask a SOC manager what their job is and the honest answer isn't a workflow at all — it's a short list of decisions nobody else in the building is positioned to make. How many people to hire, and onto which shift. Whether this quarter's tooling renewal gets funded or a headcount requisition does instead, when both are competing for the same line item. Where the boundary sits between "the team handles this" and "this goes to the board." What to do about the analyst whose numbers are bad — a skill gap, a training gap, or a process that set them up to fail. None of those four decisions has a query behind it. Each one has a dollar figure, a person, a shift schedule, or a signature on a risk-acceptance memo behind it instead. This book is built entirely around that list, and this part exists to state the list precisely before the other 32 parts build on it.

[CONCEPT] This part also carries the book's central, falsifiable claim: a SOC fails from bad staffing or bad incentives at least as often as it fails from a bad detection rule, and the manager's chair is the one seat in the building where that failure is visible before it turns into an incident-review finding. A broken rule shows up in a false-negative count. A broken staffing model or a broken incentive shows up as attrition, as a queue that never quite clears, as an escalation that got closed early to protect someone's handle-time average — and by the time any of those show up in a metric, the damage is usually a quarter old. Parts 2 through 33 exist to make that kind of failure visible earlier and cheaper to fix than it is today.

[CONCEPT] This part deliberately does not build a headcount model (Part 5), write the mechanics of a good escalation hand-off (SOC Playbook Handbook, Part 27 — Escalation Quality), or score a detection's coverage (Detection Engineering Handbook V2, Part 41 — Detection Coverage). Its job is narrower and comes first: name the decisions that belong to this book at all, draw the boundary against the other two volumes precisely enough that a reader never has to guess which one to open, and set the thesis everything after this part is measured against.

1. The job: decisions only a manager can make

[CONCEPT] An analyst's decisions are bounded by a ticket. A detection engineer's decisions are bounded by a rule's logic and its test results. A SOC manager's decisions are bounded by none of that — they're bounded by a budget cycle, a labor market, a board's risk appetite, and a legal or HR process that runs on its own clock. Four decision domains recur constantly enough to organize the whole book around them: staffing, budget, escalation-policy design, and risk acceptance. Each one is a decision a team lead or an individual analyst can influence but cannot finally make, because each one commits the organization to a resource or a liability the manager, not the team, is accountable for.

1.1 Staffing decisions

[SENIOR MANAGER] A staffing decision answers a question like: does a SOC running three shifts around the clock need 12 analysts or 20 to actually keep one person on every shift, every day, after accounting for the people who are on PTO, out sick, or in training this week? Part 5 owns the arithmetic behind that answer. This part's only job is to name the decision itself as one that sits with the manager: a team lead can tell you the queue was short-staffed on Tuesday night, but only the manager can decide whether that's a staffing-model problem worth a new requisition or a one-off worth absorbing. The same logic applies to shift pattern design (Part 6), hiring bar and sourcing channel (Part 7), and where the line sits between "ready for Tier 2" and "not yet" (Part 10) — all decisions a manager owns even when a team lead executes the day-to-day process behind them.

1.2 Budget decisions

[SENIOR MANAGER] A budget decision answers a question like: when the SIEM renewal and a new-hire requisition are both competing for the same \$180,000 this fiscal year, which one gets funded, and what does the manager tell the CFO about the one that doesn't? Part 20 owns the budget-category structure and the cost-per-analyst framing that makes this defensible; Part 21 owns the tooling-selection process that produces the number in the first place. Neither a team lead nor a vendor's sales engineer can make this call — a team lead doesn't see the full budget, and a vendor is structurally motivated to answer "buy the tool," not "which is the better \$180,000 bet this year." Only the manager holds both sides of that tradeoff at once.

1.3 Escalation-policy design decisions

[SENIOR MANAGER] This is the decision domain most likely to get confused with a neighboring book's territory, so it's worth being exact about the boundary here, in the part whose entire job is drawing it. *Escalation-policy design* is a manager's decision: how many tiers exist at all, what response-time target backs an escalation from Tier 1 to Tier 2, and whether Tier 1 is even permitted to close a given alert category without a second set of eyes. *Escalation mechanics* — what a good hand-off actually contains, written for one ticket — is not this book's job at all.

Cross-Book Pointer *This part does not explain what a good escalation hand-off looks like on a single ticket — that's a triage-level writing and communication skill, not a staffing or policy decision. See SOC Playbook Handbook, Part 27 — Escalation Quality for the mechanics; come back here (or to Part 3, Tiering Models) once you're deciding how many tiers exist and what policy backs the hand-off in the first place.*

[SENIOR MANAGER] The distinction matters in practice, not just on paper. A manager who reads Escalation Quality (SOC Playbook Part 27) and concludes the team's escalation problem is solved has answered the wrong question — a team can write technically excellent hand-offs inside a tiering structure that adds two days of latency to every incident, or inside a policy that lets Tier 1 close account-takeover alerts unilaterally because nobody ever decided they shouldn't. Part 3 (Tiering Models) and Part 4 (Organizational Placement & Charter) pick up the policy-design side of this; the mechanics stay where SOC Playbook Handbook already owns them.

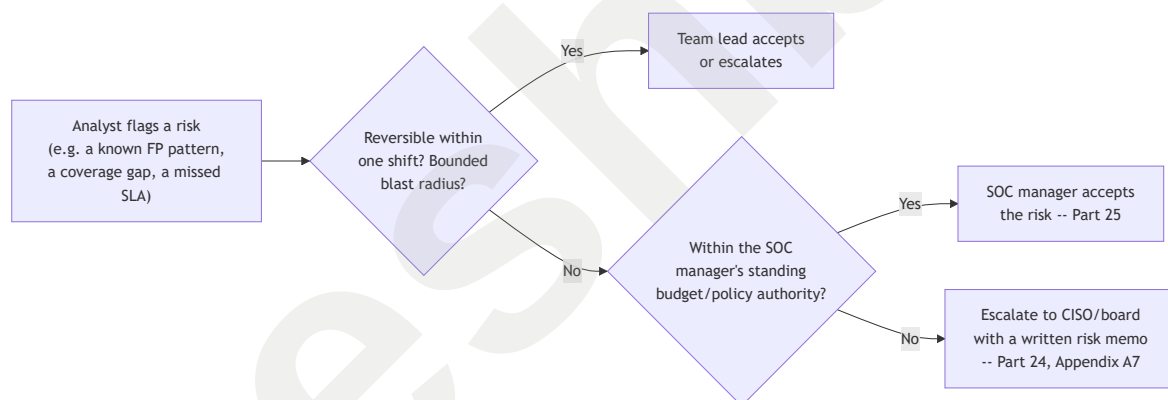
1.4 Risk-acceptance decisions

[SENIOR MANAGER] A risk-acceptance decision answers a question like: this coverage gap, this known false-positive rate, this vendor's SLA shortfall — does the manager sign off and accept it, or does it go up to the CISO or the board instead? Part 25 develops this fully as the book's central judgment-call chapter, because most risk-acceptance calls genuinely don't have a clean right answer at the time they're made. What belongs here, in the foundations chapter, is the shape of the decision: it always has an owner, that owner is never "the team" as an undifferentiated group, and the owner changes as the size and reversibility of the risk changes.

Figure 1.1 — Risk-acceptance escalation path. CONCEPTUAL. Illustrates who owns a risk-acceptance call as its scope and reversibility grow, from a bounded, shift-level call up to a board-level decision documented in a written risk memo. It is a structural decision path, not a capture of any specific organization's actual approval chain — Part 25 and Appendix A7's risk-acceptance memo template develop the real mechanics. Diagram ID **FIG-0001**.

flowchart LR

```
A["Analyst flags a risk\n(e.g. a known FP pattern,\nna coverage gap, a missed SLA)"] --> B["Reversible within\nnone shift? Bounded\nblast radius?"]
B -->|Yes| C["Team lead accepts\nor escalates"]
B -->|No| D["Within the SOC\nmanager's standing\nbudget/policy authority?"]
D -->|Yes| E["SOC manager accepts\nthe risk -- Part 25"]
D -->|No| F["Escalate to CISO/board\nwith a written risk memo\n-- Part 24, Appendix A7"]
```



[SENIOR MANAGER] The failure mode worth naming now, before Part 25 goes deep on it: a risk that never gets formally accepted by anyone still gets accepted by default, the moment nobody stops the process that's carrying it. A coverage gap a manager never signs off on is not a coverage gap the organization has decided to live with — it's a coverage gap the organization hasn't noticed yet, and "hasn't noticed yet" is not a risk posture, it's an accident waiting for a date.

The table below is a working reference, not an exhaustive one — a quick check for "whose call is this, and where in this book (or the other two) does the mechanics live."

Decision Domain	Example Decision	This Book's Part(s)	Mechanics Owned Elsewhere
Staffing/headcount	12 vs. 20 analysts for 24/7 coverage	Part 5	—
Shift & coverage design	Fixed shifts vs. follow-the-sun	Part 6	Hand-off mechanics: SOC Playbook Handbook, Part 27 — Escalation Quality
Hiring bar & sourcing	Which certifications actually predict capability	Part 7, Part 8	—
Tiering & escalation policy	How many tiers, what SLA backs a hand-off	Part 3	Hand-off writing quality: SOC Playbook Handbook, Part 27
Automation platform investment	Fund a SOAR platform, and how much automation capacity to buy	Part 21 (procurement lens)	Gating-policy design itself: SOC Playbook Handbook, Part 30 — Automation and SOAR
Budget & tooling spend	SIEM renewal vs. new headcount	Part 20, Part 21	Migration engineering cost: Detection Engineering Handbook V2, Part 6 — Normalisation
Vendor/MSSP contracting	SLA terms, right-to-audit, exit clauses	Part 22, Part 23	—
Performance management	Skill gap vs. training gap vs. bad rule	Part 16	Bad-rule diagnosis: Detection Engineering Handbook V2, Part 43 — Detection Debt
Risk acceptance	Sign off at manager level vs. escalate to board	Part 25	Severity scoring itself: SOC Playbook Handbook, Part 29 — Playbook Severity Model
Crisis/major-incident leadership	Staffing surge, board comms, retainer activation	Part 28	Incident-commander mechanics: SOC Playbook Handbook's Ransomware Master Playbook

2. Three books, one SOC: the series map

[CONCEPT] The NESHBOY SOC Professional Library is three books because a SOC manager's job, a detection engineer's job, and an analyst-facing playbook's job are three different units of analysis, not three difficulty tiers of the same subject. SOC Playbook Handbook is organized around the ticket and the client-facing operational process wrapped around it. Detection Engineering Handbook V2 is organized around the telemetry and the analytic logic that turns it into a defensible alert. This book is organized around the team and the organization that runs both of those things day after day, year after year, on a real budget with real turnover. A reader who only owns one of the three volumes can do real work with it; a reader who owns all three should never have to guess which one answers a given question, and never have to read the same explanation twice with two different vocabularies.

2.1 The governing test

[CONCEPT] Every scope decision in this book is checked against one test: if a paragraph's claim is about a rule, a query, or a playbook's internal structure, it belongs in a citation, not a rewrite — cite it and move on. If the claim is about a person, a process, a contract, or a dollar, it stays here. That test is why a chapter on queue health (Part 9) can mention that a growing backlog is sometimes a detection-

quality problem in disguise, cite Detection Engineering Handbook V2, Part 42 — Detection Quality for how to diagnose that technically, and then get straight back to the staffing question — without ever re-explaining what makes a detection noisy. Applying the test consistently is harder than it sounds, because staffing, QA, and performance content constantly brushes up against technical material: a QA review touches escalation quality, a performance conversation touches false-positive engineering. The rule holds anyway. It's the single most important thing a reviewer checks on every part in this book (see the review checklist in [STYLE-GUIDE.md](#) §11, item 10).

2.2 What SOC Playbook Handbook owns

[CONCEPT] SOC Playbook Handbook owns playbook mechanics — how a specific playbook is written and structured — and the client-facing operational chapters that wrap around a ticket's lifecycle. Concretely: escalation quality (Part 27), stakeholder communication (Part 28 — SOC-to-Stakeholder Communication), severity scoring (Part 29 — Playbook Severity Model), what to automate versus gate for human review (Part 30 — Automation and SOAR), AI-assisted operations at the point of triage (Part 31 — AI-Assisted SOC Operations), the exact definitions behind MTTA, MTTR, and false-positive rate (Part 32 — Metrics), and individual playbook-failure post-mortems (Part 35 — Playbook Failure Examples), plus a playbook-quality maturity model (Appendix 38A) and the specialty playbooks — the Ransomware Master Playbook and the Insider Threat playbooks (Category 19) chief among them. SOC Playbook Handbook also owns playbook governance and playbook testing as client-facing operational disciplines in their own right — this chapter doesn't attach part numbers to those last two here, unlike the numbered items above, because Part 1's job is the map, not the exhaustive lookup, and this book's own citation rule ([STYLE-GUIDE.md](#) §4) is explicit that a part number never gets cited without checking the target book's current index first. Appendix A8 carries the exhaustive, version-checked table; treat that appendix, not this paragraph, as the answer once it's been built.

2.3 What Detection Engineering Handbook V2 owns

[CONCEPT] Detection Engineering Handbook V2 owns everything with a technical, testable answer behind it: telemetry, detection rule logic, and query languages across Sigma, KQL, SPL, AQL, YARA-L, and Elastic (Parts 3 through 29); correlation engineering, baselining, threat intelligence as a scoring input, and risk-based detection (Parts 30 through 33); threat-hunting methodology (Parts 34 through 36); and detection quality, coverage, and debt as technical measures of the rule base itself (Parts 41 through 43). A manager reading this book will hit that territory constantly — Part 9's queue-health diagnosis, Part 16's performance-management judgment calls, and Part 25's risk-acceptance framework all touch it — and every one of those parts cites into Detection Engineering Handbook V2 by name and part number rather than re-deriving any of it.

2.4 What this book owns

[SENIOR MANAGER] This book owns the people and the operations of running the team that uses both of the other two — staffing and shift design (Section B), analyst development and career progression (Section C), quality, performance, and culture (Section D), vendor and budget management (Section E), governance and the executive interface (Section F), crisis leadership from the manager's chair (Section G), and maturity, benchmarking, and the road ahead (Section H). None of it has a query behind it. All of it has a headcount number, a dollar figure, a contract clause, or a person's career attached instead.

The table below expands the series map from a book-level split into a question-level lookup — the form a reader actually uses under time pressure.

If you're asking...	Open...
"How many analysts do we actually need for 24/7 coverage?"	This book, Part 5 — Headcount & Capacity Modeling
"How do I write or structure a phishing playbook?"	SOC Playbook Handbook, Foundations / Playbook Library
"What does a good Tier 1-to-Tier 2 hand-off look like?"	SOC Playbook Handbook, Part 27 — Escalation Quality
"How should an alert's severity be scored?"	SOC Playbook Handbook, Part 29 — Playbook Severity Model
"What should be automated versus gated for human review?"	SOC Playbook Handbook, Part 30 — Automation and SOAR
"What does MTTR/FP rate actually mean, precisely?"	SOC Playbook Handbook, Part 32 — Metrics
"What does this book mean by 'handle time' or 'backlog age'?"	This book's own operational definitions — Part 5 §2 (handle time), Part 9 §1 (backlog age)
"This playbook broke live — why, mechanically?"	SOC Playbook Handbook, Part 35 — Playbook Failure Examples
"How do I write this detection logic in KQL/Sigma/SPL?"	Detection Engineering Handbook V2, Parts 24–29
"Why is this rule so noisy, and how do I fix that technically?"	Detection Engineering Handbook V2, Part 38 — False Positive Engineering
"Do we actually have tested detection coverage for ransomware?"	Detection Engineering Handbook V2, Part 41 — Detection Coverage
"Is a growing queue a staffing problem or a bad-rule problem?"	Both — this book's Part 9 to diagnose the queue; Detection Engineering Handbook V2, Part 42 — Detection Quality to diagnose the rule
"How many analysts, organized, trained, retained, and led how?"	This book
"What's the budget case, and what does the vendor contract say?"	This book, Parts 20–23

Figure 1.2 — "Which book do I open" decision tree. *CONCEPTUAL.* Illustrates the routing logic behind the lookup table above, collapsed to three yes/no questions. It's a navigation aid for a reader holding all three volumes, not a claim that every real question sorts this cleanly — the "both" row in the table above is the honest exception this diagram simplifies away. Diagram ID [FIG-0002](#).

flowchart TD

A["You have a SOC question"] --> B{"Is it about writing or running a\nspecific playbook, an escalation,\nseverity scoring, automation gating,\nor a metric's definition?"}

B -->|Yes| C["SOC Playbook Handbook"]

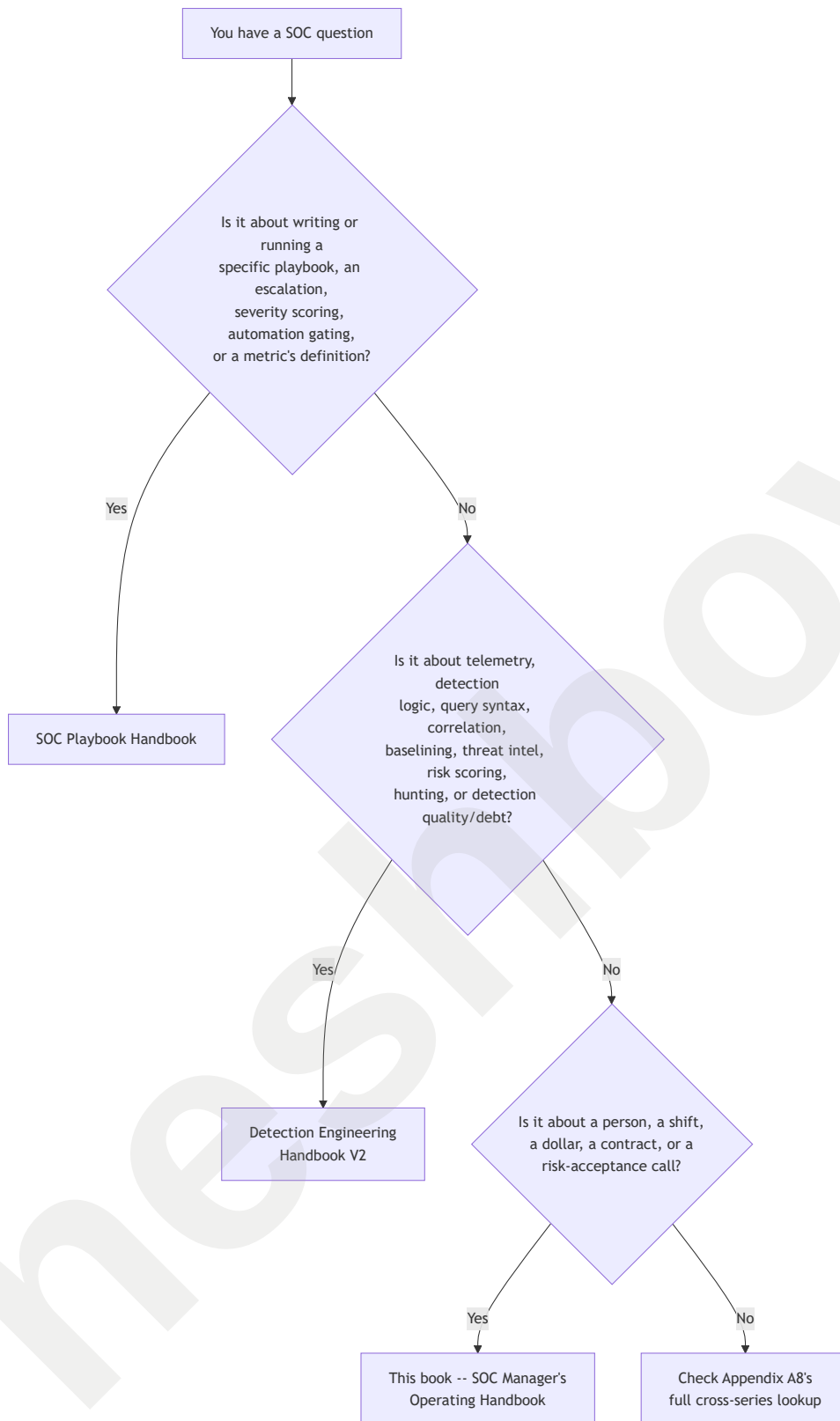
B -->|No| D{"Is it about telemetry, detection\nlogic, query syntax, correlation,\nbaselining, threat intel, risk scoring,\nhunting, or detection quality/debt?"}

D -->|Yes| E["Detection Engineering\nHandbook V2"]

D -->|No| F{"Is it about a person, a shift,\na dollar, a contract, or a\nrisk-acceptance call?"}

F -->|Yes| G["This book -- SOC Manager's\nOperating Handbook"]

F -->|No| H["Check Appendix A8's\nfull cross-series lookup"]



Cross-Book Pointer *This chapter's series map is a working summary, not the authoritative index — part numbers in either companion volume can shift on a future revision, and this book's own citation standard treats a stale number as worse than no citation. Appendix A8 — Cross-Series Quick Reference: Where to Find It is the single, build-checked place a reader confirms a topic really is (or isn't) covered elsewhere before assuming this book missed it.*

3. The central thesis: a SOC fails from bad staffing or bad incentives as often as from a bad rule

[CONCEPT] Rule failures are visible almost by design — a missed detection shows up as a false negative, gets a name, gets a post-mortem, and usually gets fixed within a review cycle or two because the failure mode is legible to anyone who can read the logic. Staffing failures and incentive failures are not legible the same way. They don't produce a false negative with a timestamp; they produce three resignations in one quarter, a QA score that's technically fine while real judgment quietly degrades, or an escalation that gets closed a little early because closing it helps someone's number. Nobody writes a post-mortem titled "our incentive structure caused this," because by the time the damage is visible, it looks like a training problem, an attrition problem, or a bad week — never the actual root cause, which was a decision made months earlier by someone in the manager's chair. The two worked examples below are composite constructions, not single traceable incidents, built to show the mechanism concretely rather than to point at any one organization.

3.1 The overtime spiral

[SENIOR MANAGER] **CASE-0001** below shows how a staffing model that looks defensible on paper — and would survive a casual budget review — creates the exact crisis it was built to prevent, the moment normal attrition happens to it.

Management Autopsy — "staff to headcount, not to shrinkage" (CASE-0001, COMPOSITE CASE EXAMPLE)

The decision: A 24/7, three-shift SOC at a mid-market financial-services company is staffed to 15 analysts — five per shift — with a flat 10% shrinkage allowance built into the budget model, covering routine PTO and sick time.

Why it seemed reasonable: Fifteen headcount, three shifts, five each is a number a CFO can audit in one glance, and 10% shrinkage matched the prior year's actual PTO usage almost exactly. The model wasn't sloppy — it was precisely wrong about what "shrinkage" needs to include.

How it failed: Real shrinkage — PTO, sick leave, training days, and one parental leave that started mid-year — ran closer to 32% of paid hours, not 10%, meaning "five per shift" was fiction on more weeks than not. When two senior analysts resigned in the same quarter — a 13% attrition event, unremarkable for a 15-person team over a year — there was no standing buffer to absorb it. Leadership's only lever on short notice was mandatory overtime: the remaining 13 analysts absorbed roughly 15 extra hours a week each for eight straight weeks. At a \$75,000 base salary (roughly \$36/hour) and a 1.5x overtime rate, that's about \$54/hour worked, versus an incremental cost of roughly \$7,800 a month per new hire (salary plus a 25% benefits load) — the overtime premium alone cost more over those eight weeks than opening the requisitions immediately would have, and it was paid on top of a 90-day hiring-freeze delay that made overtime the only lever available regardless of its cost. Three more analysts began actively job-hunting within the following two quarters, citing the overtime stretch by name in exit interviews.

The fix: Model headcount against realistic shrinkage — Part 5 covers the calculation — and treat "two analysts leave without much notice" as an expected event a standing buffer absorbs, not an exception that justifies emergency overtime as the default response. A staffing model that can't survive its own team's normal attrition rate isn't a staffing model; it's a bet that nobody quits this year.

(The dollar figures above are **CONCEPTUAL SAMPLE** illustrative arithmetic built to show the mechanism, not sourced benchmark data from a real organization's payroll.)

3.2 Rewarding speed over judgment

[SENIOR MANAGER] **CASE-0002** (COMPOSITE CASE EXAMPLE) below shows the second, quieter version of the same thesis: a metric that is correctly defined and honestly reported can still produce a dangerous incentive the moment a manager attaches an individual reward to it without asking what behavior the reward actually produces.

[HR/PEOPLE] A 40-analyst SOC rolled out a team dashboard ranking analysts by average ticket-closure time, with a \$2,000 quarterly spot bonus for the fastest quartile. Average closure time dropped 22% within two months — a result the manager reported upward as a clear win. A routine QA sampling pass

(Part 15) found the actual driver: escalation notes had gotten thinner, and two tickets closed as "false positive, user error" were confirmed months later, during a customer's own audit, to have been real account-takeover activity. The underlying metric — handle time, this book's own operational definition (Part 5 §2), not a Playbook Handbook term — was never wrong or ambiguous. The failure was pairing an individual cash incentive to a team-level operational metric without accounting for the fact that escalating a ticket keeps the clock running against the analyst who escalates it, while closing it — even wrongly — stops the clock in their favor. The dashboard rewarded speed of closure, not quality of judgment, and analysts optimized for exactly what was measured and paid, which is what incentives reliably do.

(The team size, bonus amount, and closure-time figures above are **CONCEPTUAL SAMPLE** illustrative numbers built to show the incentive mechanism, not sourced benchmark data from a real organization's dashboard or payroll.)

People Risk Trap Attaching an individual bonus to a team-level throughput metric — handle time, closure rate, tickets-per-shift — without first checking what behavior maximizes that specific number in the specific workflow it's measured against, teaches analysts to game the metric rather than do the underlying job well. The fix: model the fastest way to "win" the metric before rewarding it, not after a QA pass catches the damage. If the fastest way to win is a shortcut that degrades judgment, either redesign the incentive around a quality-adjusted measure (Part 16) or don't tie individual pay to that metric at all.

What Would Change My Mind This part's central claim is that staffing and incentive failures cause as much real SOC damage as rule failures, just less visibly. If a rigorous, controlled comparison across multiple SOCs found that incident-causing staffing or incentive failures were consistently rare relative to detection-logic failures — not merely less discussed in post-mortems, but actually rarer in root-cause data — that would undercut this book's founding premise and argue for treating people-and-process content as a supporting chapter inside a detection-focused book rather than a full third volume in its own right.

3.3 A three-question check before trusting a staffing or incentive model

[SENIOR MANAGER] Both cases above pass a casual review and fail a specific one. Before a headcount model, a bonus structure, or a KPI dashboard goes live, three questions catch most of what a casual review misses:

1. **What does this model assume never happens?** The overtime spiral's model assumed shrinkage stayed near 10% and nobody quit mid-cycle. Name the assumption in writing, because an assumption nobody wrote down is an assumption nobody gets to challenge before it breaks.
2. **What's the fastest way to win, under this exact measurement, this week?** Not the intended way — the fastest way. If the fastest way to win a closure-time bonus is to close a real account-

takeover ticket as user error, the model is already broken, whether or not anyone has done that yet.

3. **Who finds out first if the assumption in question 1 is wrong, and how long does that take?**

In CASE-0001, the answer was "the shift lead approving overtime, in week one" — fast, but with no authority to fix the underlying model. In CASE-0002, the answer was "a QA sample, two months later" — slow enough that real damage had already shipped to a customer. A model whose own failure signal arrives slower than the damage it can cause is a model that needs a faster tripwire, not just a better intention.

None of this replaces the actual models in Parts 5 and 16 — it's the five-minute sanity check a manager runs before trusting either one with a real decision.

4. How to read this book

[CONCEPT] Six content tags mark every paragraph below the part-title level: **[CONCEPT]** for foundational material, **[FRONTLINE MANAGER]** for what a team lead actually does this week, **[SENIOR MANAGER]** for headcount, budget, vendor, and risk-acceptance calls, **[HR/PEOPLE]** for hiring, performance, and culture content, **[VENDOR/PROCUREMENT]** for tooling and contract content, and **[EXECUTIVE]** for the board-facing register. A shift lead skimming for what changes their Tuesday can filter to **[FRONTLINE MANAGER]**; a CISO preparing a board deck can filter to **[EXECUTIVE]** and **[SENIOR MANAGER]**. Neither has to read past their tag to get something usable, and neither is reading a different book from the other — the multi-level content model in **BOOK-INDEX.md** is what makes that possible without three separate manuscripts.

[SENIOR MANAGER] Eight callout boxes carry recurring content across parts, defined exactly in **STYLE-GUIDE.md** §6: Management Autopsy, Manager's Note, Operational Reality, Blind Spot, People Risk Trap, Field Test, Cross-Book Pointer, and What Would Change My Mind. This part leans on the last two — Cross-Book Pointer because a foundations-and-series-map chapter's entire job is routing the reader correctly, and What Would Change My Mind because the thesis in §3 is a claim this book is willing to be wrong about, not a slogan. Worked examples and case narratives carry a permanent evidence classification — **ANONYMIZED CASE EXAMPLE**, **COMPOSITE CASE EXAMPLE**, **OFFICIAL REFERENCE**, or **CONCEPTUAL** — stated plainly rather than left to the reader to guess, per **STYLE-GUIDE.md** §9. Both **CASE-0001** and **CASE-0002** above are composite: built from recognizable, common patterns rather than one traceable real incident, because a single identifiable version of either would be either too identifying to publish or too thin to generalize from.

Manager's Note Read this book front to back once, then treat it as a reference you open to a specific part when a specific decision is in front of you. The series map in §2 exists so that the second and third time through, you're not re-reading Part 1 to remember which book owns what — you're checking Appendix A8 once it's built, or the table above, and getting straight to the part that actually answers your question.

5. Where this goes next

[CONCEPT] Section A (Parts 2 through 4) finishes the organizational-design foundation this part starts: the build/buy/blend decision for the SOC itself, tiering models, and the charter that states what the SOC does and does not own. Section B (Parts 5 through 9) turns the staffing decisions named in §1.1 into an actual headcount model, shift design, hiring pipeline, and queue-health diagnostic. Sections C and D (Parts 10 through 19) build the competency, career-ladder, QA, performance, burnout, and culture programs that make CASE-0002's failure mode diagnosable before a customer audit finds it instead. Section E (Parts 20 through 23) turns the budget decisions from §1.2 into a defensible business case and a governed vendor relationship. Section F (Parts 24 through 27) builds the executive interface and the risk-acceptance framework §1.4 previews. Section G (Parts 28 and 29) is the manager's-chair complement to the incident-commander mechanics SOC Playbook Handbook already owns. Section H (Parts 30 through 33) closes the book with maturity, benchmarking, AI's staffing implications, and a synthesis roadmap. Nothing past this point should need to re-explain what belongs to this book versus the other two — it should just cite correctly and move on.

Cross-references

[CONCEPT] This part assumes no earlier part in this book (it is the entry point). It previews Part 3 (Tiering Models), Part 4 (Organizational Placement & Charter), Part 5 (Headcount & Capacity Modeling), Part 9 (Queue Health & Workload Management), Part 15 (Quality Assurance Programs), Part 16 (Performance Management & Coaching), Part 20 (Building & Defending the SOC Budget), Part 24 (Executive & Board Reporting), Part 25 (Risk Acceptance & Manager Decision-Making Under Uncertainty), and Part 28 (The Manager's Role in a Major Incident). It cross-references SOC Playbook Handbook, Part 27 — Escalation Quality, Part 28 — SOC-to-Stakeholder Communication, Part 29 — Playbook Severity Model, Part 30 — Automation and SOAR, Part 31 — AI-Assisted SOC Operations, Part 32 — Metrics, Part 35 — Playbook Failure Examples, and the Ransomware Master Playbook; and Detection Engineering Handbook V2, Parts 3–29, Parts 30–33, Parts 34–36, and Parts 41–43. Appendix A8 is the standing artifact all of the above resolve to as the companion volumes' own part numbering evolves.

Part 2 — SOC Operating Models: In-House, MSSP, Co-Managed, Hybrid

Why this part exists

[CONCEPT] Before a manager can staff a shift roster (Part 6), build a competency matrix (Part 10), or defend a budget line (Part 20), someone has to answer a question those parts all assume is already settled: who actually does the work, and who carries the risk if it goes wrong. That's the build/buy/blend decision this part covers — fully in-house, fully outsourced to an MSSP or MDR provider, co-managed in either direction, or a hybrid model that outsources monitoring but keeps detection engineering in-house. Every other operating-model choice in this book sits downstream of this one.

This part stays on four axes: cost, control, talent retention, and the coverage-hours and telemetry-sensitivity economics that make one model cheaper or safer than another for a specific organization. It does not walk through how to write an MSSP statement of work, negotiate a service-level agreement, or build a right-to-audit clause — that's Part 22, and this part cites it rather than re-deriving it. It also does not re-teach the detection logic, playbook structure, or escalation mechanics that run inside any of these four models, because that content belongs to the other two volumes in this series regardless of who's staffing the seat that executes it. A Sigma rule written by an in-house detection engineer and the identical rule written by an MSSP analyst are the same technical artifact — Detection Engineering Handbook V2, Parts 3–29 own what makes either one correct. This part owns only the organizational question of where that seat sits, who's paid to fill it, and what that costs.

1. Four models, one underlying question: who does the work and who owns the risk

[CONCEPT] Vendors use "MSSP," "MDR," and "co-managed" inconsistently enough that the label on a contract tells you less than two direct questions do: who sees an alert first, and who can close it, tune it, or escalate it without asking permission. Answer those two questions for any given tier of work and the four models below stop being marketing categories and start being a straightforward org chart.

1.1 Fully in-house

[SENIOR MANAGER] Every analyst, engineer, and manager touching the SOC's telemetry is an employee (or a long-term contractor staffed and managed the same way), the SIEM and EDR consoles never grant a third party standing access, and the 24/7 roster is built and covered entirely from the organization's own headcount using the shift-pattern options in Part 6. This is the model regulated industries default to when a contract or examiner explicitly restricts third-party access to raw security telemetry, and it's the model a mature SOC gravitates toward once its alert volume and budget can absorb the coverage-hours cost worked through in §6 below.

1.2 Fully outsourced (MSSP / MDR)

[VENDOR/PROCUREMENT]

Monitoring, triage, and — in a well-negotiated MDR contract, though not in every legacy MSSP contract — some scoped containment actions are performed entirely by a third party against telemetry the organization forwards or grants read access to. Internal headcount shrinks to a relationship owner: someone who manages the vendor, receives the escalations the vendor decides warrant one, and handles anything the contract doesn't cover. "MSSP" historically implies log monitoring and alerting with the client doing the response; "MDR" usually implies the vendor takes some response action itself. Treat that as a spectrum to confirm line by line in the statement of work, not a label to trust — Part 22 covers exactly how to pin that down contractually.

1.3 Co-managed — two directions

[SENIOR MANAGER]

Co-managed splits the tiers themselves between the organization and a vendor, and it runs in either direction. The first direction — in-house tiers with vendor overflow — keeps Tier 1 and Tier 2 staffed internally during business hours and hands overnight, weekend, and surge volume to a vendor. The second — the reverse — has the vendor staff Tier 1 for high-volume initial triage against every alert, with the in-house team owning Tier 2 and Tier 3: deep investigation, incident response, and anything that needs the organization-specific context a vendor's rotating analyst pool doesn't have. Telling the two apart in a real contract is a matter of asking who sees the alert first and who can close it without escalating — the mechanics of what a good hand-off between those tiers actually looks like, once you've picked a direction, are Part 3's territory and SOC Playbook Handbook, Part 27 — Escalation Quality's, not this part's.

Operational Reality *A co-managed contract that looks clean on the org chart — vendor owns nights and weekends, in-house owns business hours — routinely breaks down on exactly the alerts that straddle the boundary: one that fires at 5:58 p.m. and is still open at the shift handoff, or a low-and-slow pattern that started on the vendor's watch and only becomes obviously malicious once an in-house analyst with environment context looks at it the next morning. Both sides can reasonably believe the other one is tracking it. Fix this with an explicit standing-alert handoff rule — who owns anything still open at the boundary, by name or by role, not "whoever notices" — rather than assuming the shift schedule alone resolves ownership.*

1.4 Hybrid with embedded detection engineering

[SENIOR MANAGER]

The hybrid model outsources 24/7 monitoring and triage entirely, the same as the fully outsourced model, but keeps a small in-house detection engineering function — typically two to four full-time engineers — that owns three things the vendor doesn't: the tuning direction sent back to the vendor's ruleset, custom detection content built for the organization's own high-risk applications and infrastructure, and the telemetry-sensitivity decisions about what does and doesn't get forwarded to the vendor's platform in the first place. This model exists to answer a specific complaint that shows up constantly in §3 below: a generic vendor ruleset built for 1,000 different clients has no way to know your business, and outsourcing detection authority along with monitoring means nobody with that context can act on it. The technical discipline of building and maintaining that content is Detection

Engineering Handbook V2's, cover to cover; this part owns only the staffing decision to keep that specific function in-house while everything else walks out the door to a vendor.

[SENIOR MANAGER] The table below is a quick-reference for telling the four models apart by who staffs what, not a substitute for the fuller comparison in §7.

Model	Tier 1 staffed by	Tier 2/3 staffed by	Detection engineering owned by	Typical trigger
Fully in-house	Employees	Employees	Employees	Telemetry too sensitive to export; budget and volume support full 24/7 headcount
Fully outsourced (MSSP/MDR)	Vendor	Vendor (escalates to a relationship owner)	Vendor	Coverage-hours economics don't support in-house staffing at all
Co-managed	Either (business hours)	Either (whichever direction wasn't Tier 1)	Usually whoever staffs Tier 2/3	Full in-house coverage is close but the thinnest shifts aren't worth staffing
Hybrid + embedded DE	Vendor	Vendor (escalates to in-house DE/IR for context)	In-house DE team	Alert volume justifies outsourcing monitoring, but generic vendor content misses org-specific risk

Cross-Book Pointer This part does not cover how to structure the contract that makes any of these four models work in practice — SLA definitions, right-to-audit clauses, data-ownership and offboarding terms. See Part 22 — MSSP & Managed-Service Contract Management for the mechanics once you've picked a direction here. It also does not cover the technical detection content any of these staffing models produces or consumes; see Detection Engineering Handbook V2, Parts 3–29 for the analytics themselves and Parts 41–43 for measuring their coverage, quality, and debt regardless of who wrote them.

2. Cost economics: what each model actually costs

[SENIOR MANAGER] The honest comparison isn't "in-house is expensive, outsourcing is cheap" — it's steady-state run cost against ramp cost against the cost of a coverage gap, and the headline number on a vendor's pricing sheet is only one of those three.

2.1 Worked example — a 4,000-endpoint mid-market organization comparing three paths

[SENIOR MANAGER] The comparison below prices three of the four models — fully in-house, fully outsourced, and hybrid — for the same 4,000-endpoint organization, using the coverage-hours math developed in §6 as the basis for the in-house headcount figure rather than an arbitrary round number.

Whoever is building the build-vs-buy business case for a budget defense (Part 20 develops the full version of this) uses a comparison like this as the opening exhibit, not the closing argument — it's the number that gets the conversation started before the softer costs below get factored in.

CONCEPTUAL SAMPLE — illustrative numbers, not sourced benchmark data.

Fully in-house, 24/7, two analysts per shift (see Sec. 6.1 for the FTE math):

11 analysts x \$95,000 loaded cost	=	\$1,045,000
2 team leads x \$130,000	=	\$260,000
1 SOC manager x \$160,000	=	\$160,000
Tooling (SIEM + EDR + SOAR licensing)	=	\$450,000
Total	=	\$1,915,000/year

Fully outsourced MDR, same 4,000 endpoints:

MDR contract, \$18/endpoint/month	=	\$864,000
1 relationship-owner FTE x \$110,000	=	\$110,000
Retained log-forwarding/visibility tooling	=	\$120,000
Total	=	\$1,094,000/year

Hybrid: outsourced monitoring + embedded detection engineering:

Monitoring-only MDR, \$15/endpoint/month	=	\$720,000
3 detection engineers x \$115,000	=	\$345,000
0.4 FTE senior-manager allocation	=	\$60,000
Total	=	\$1,125,000/year

Co-managed lands between the hybrid and fully in-house figures above, and exactly where depends entirely on which shifts stay internal — Part 20 builds the full total-cost-of-ownership model with that variable exposed, rather than collapsing it into one more fixed number here.

The three totals above only compare payroll and licensing. They say nothing about hiring-pipeline cost, vendor onboarding friction, or the price of a coverage gap while a new model ramps up — all three show up below, and none of them are optional line items just because a spreadsheet can be built without them.

[VENDOR/PROCUREMENT]

The \$18-per-endpoint figure above is only one of several pricing structures a vendor might actually propose, and the structure matters as much as the rate. Per-endpoint pricing is predictable but penalizes an organization for adding hosts regardless of how much telemetry each one generates. Per-gigabyte-ingested pricing does the opposite — it's cheaper for a lean fleet of quiet endpoints and expensive for exactly the organizations generating the richest telemetry: cloud control-plane logs, application-layer logging, and anything approaching the ingestion volume Detection Engineering Handbook V2, Part 6 — Normalisation assumes a mature program eventually wants. Per-user pricing sits in between and mostly tracks headcount rather than risk. None of these is categorically wrong, but comparing two vendor quotes priced on different structures without normalizing them to the same projected ingestion volume produces a number that looks like an apples-to-apples comparison and isn't one — Part 21 — Tooling Procurement & Platform Strategy covers building that normalized comparison properly.

Operational Reality *An MDR quote's headline per-endpoint price doesn't include the three to six months it takes the vendor's analysts to learn which of your alerts are normal — the quarterly batch job that looks like exfiltration, the internal vulnerability scanner that looks like a brute-force source, the admin group that legitimately has 40 members. Budget that ramp as a real cost, not a rounding error: expect a materially higher false-positive burden and slower true-positive triage for the first two full billing cycles, and confirm in writing whether the vendor's onboarding fee covers building that context or whether your own staff will spend that time doing it for them anyway.*

3. Control: what "control" actually buys, and what it doesn't

[SENIOR MANAGER] Control is usually argued as if it were a single dial that turns all the way up in-house and all the way down with a vendor. It's really two separate things — the authority to change a detection today, and the organizational context to know it needs changing at all — and a model can grant one without the other.

[SENIOR MANAGER] In-house staffing grants the authority by default: an employee can open a pull request against the detection repository this afternoon. Whether that pull request gets reviewed, tested, and shipped safely is a detection-as-code discipline question — Detection Engineering Handbook V2, Part 22 owns that mechanic — not something the org chart alone guarantees. A fully outsourced contract, by contrast, usually routes a tuning request through the vendor's own change-management queue, which can mean anywhere from same-day turnaround to a multi-day wait depending on how the contract defines it — exactly the kind of commitment Part 22 covers making explicit before signing rather than discovering during an incident.

The composite case below shows what happens when an organization gives up the context half of control, not just the authority half.

Composite case (CASE-0201, COMPOSITE CASE EXAMPLE — merges patterns observed across several MSSP engagements; not one identifiable client relationship.)

Management Autopsy — "outsource monitoring and detection-tuning authority together, with no retained review path"

The decision: A mid-market organization signs a fully outsourced MDR contract that gives the vendor sole authority over detection logic — no internal team reviews or requests changes to the ruleset, because the point of outsourcing was to remove that overhead entirely.

Why it seemed reasonable: Fewer standing meetings, and the argument that a vendor whose whole business is detection engineering does it better than a generalist internal team could. On paper, removing an internal review step looked like it removed friction, not judgment.

How it failed: The vendor's ruleset was built for its median client, not this one. A quarterly batch-processing job the organization ran against its data warehouse tripped a generic exfiltration rule every quarter, and because nobody internally had standing authority to suppress or fix it, the same false positive got triaged from scratch four times a year. More seriously, the organization's highest-risk asset — a custom-built claims-processing application with no equivalent at any other client — had no dedicated detection content at all 18 months into the contract, because no one on either side owned writing it: not the vendor, who had no visibility into an application unique to one client, and not the organization, who had signed away the function that would have done it.

The fix: Keep a small internal review and request path even inside a fully outsourced contract — at minimum one person with standing authority to file a tuning request and get a committed response time on it, contractually defined per Part 22. An organization with genuinely org-specific detection needs is describing the hybrid model in §1.4 above, not a fully outsourced one with an informal exception process bolted on afterward.

What Would Change My Mind This part treats retained tuning authority as necessary even inside a fully outsourced contract. If a vendor's standing change-request SLA reliably matched same-day in-house turnaround and its detection engineers demonstrably built org-specific content without a retained internal reviewer prompting them to, that would undercut the claim that authority and context need to be held by the same side — worth re-testing against a vendor that markets exactly that capability before assuming it's still true.

4. Telemetry sensitivity: the constraint that can override the cost argument

[SENIOR MANAGER] Some telemetry classes make outsourcing a real constraint, not just a preference — protected health information embedded in application logs, source code and build-pipeline telemetry for a company whose code is the product, data generated inside an active M&A staging environment, and anything already under a litigation hold. The mechanism is concrete: a vendor either

needs raw log forwarding into its own platform, or standing elevated read access inside yours, and both are a wider exposure surface than the data had the moment before the contract was signed.

[VENDOR/PROCUREMENT] Those two delivery mechanisms carry different risk shapes, and treating them as interchangeable during procurement is a common mistake. Raw log export means a second, permanent copy of your telemetry now exists on infrastructure you don't control, subject to that vendor's own retention and breach history rather than yours — the exposure persists even if the contract ends and the data was never fully deleted. A vendor operating inside your own tenant with a scoped, revocable role avoids creating that second copy, but trades it for a standing credential inside your environment that's only as safe as that vendor's own access-management discipline; a compromised vendor analyst account is now a compromised account with a foothold in your SIEM. Neither mechanism is strictly safer in the abstract — which one a given sensitivity constraint actually rules out depends on what the constraint is protecting against, a copy leaving your control or a foothold entering it.

[VENDOR/PROCUREMENT] Where that constraint is real — a regulator, a customer contract, or outside counsel has said this data class doesn't leave the organization's control — fully outsourced and the vendor-front direction of co-managed are off the table regardless of what the budget can absorb. That leaves fully in-house, the in-house-tiers-with-overflow direction of co-managed (with overflow scoped only to the telemetry that isn't restricted), or hybrid with the detection-engineering team explicitly deciding, source by source, what gets forwarded to the monitoring vendor and what doesn't. None of this is a legal-mechanics question this part answers itself — Part 27 — Legal, HR & Compliance Interfaces owns the manager's procedural obligations once a restriction like this is identified; this part's job is only to name sensitivity as an operating-model input.

Operational Reality *The telemetry-sensitivity review that happens at contract signing covers exactly the log sources listed in the statement of work at that moment, and nothing added afterward gets re-reviewed automatically. Six months into a contract, someone routes a new data source — a customer-support platform with unmasked PII, say — into the same SIEM the vendor already has standing access to, because it's the path of least resistance for getting it monitored at all. Nobody re-runs the sensitivity classification, because nothing in the process prompts anyone to. Treat new-data-source onboarding as a standing trigger for a sensitivity re-check, not a one-time gate cleared once at signing.*

5. Talent retention: the model shapes who stays, and why

[HR/PEOPLE] A fully in-house model gives a manager full ownership of the career ladder described in Part 13 — analyst to senior analyst to team lead or detection engineer — but that ladder is only as tall as the organization's own growth. A Tier 1 analyst at a small in-house SOC with no near-term headcount growth plateaus fast, and that plateau lands right inside the 18-to-24-month window Part 18 identifies as this industry's characteristic burnout-driven exit point — the model didn't cause that attrition pattern, but it removed one of the levers (an internal promotion path) that could have slowed it.

[HR/PEOPLE] A fully outsourced model moves that retention problem to the vendor's side of the relationship, which doesn't make it disappear — it makes it someone else's continuity risk that still costs the client. Vendor-side Tier 1 work is high-volume, low-context, and a well-known entry point into the field, which means the specific analyst who triaged your last three incidents may not be the one on shift for your next one. That churn is invisible on your own attrition dashboard and real anyway; ask a prospective MDR vendor directly what their own analyst tenure and account-continuity numbers look like, not just their SLA numbers.

[HR/PEOPLE] Co-managed and hybrid models both create a genuine retention lever the other two don't: a Tier 2/3 or detection-engineering role that's more interesting than pure Tier 1 triage, staffed internally, and reachable as a promotion rather than a lateral move to a competitor. That's a real reason some organizations choose co-managed or hybrid even when a pure cost comparison in §2 slightly favors fully outsourcing — the retention value doesn't show up on that spreadsheet, but the cost of replacing a senior analyst who left for exactly that missing growth path does, and Part 18 quantifies it. It also interacts with hiring: Part 7 covers the persistently tight mid-level analyst market this book keeps returning to, and a vendor's own Tier 1 bench is sometimes the most realistic sourcing pipeline into that mid-level role, whether or not the organization ever formally partners with that vendor as a hiring channel.

6. Coverage-hours economics: 24/7 is a headcount problem before it's a vendor problem

[SENIOR MANAGER] A continuously staffed seat needs coverage for 8,760 hours a year. A single analyst, after the shrinkage Part 5 accounts for in its own headcount model — paid time off, sick leave, training, and the ordinary administrative overhead of a job — delivers something on the order of 1,750 productive hours in that same year, not the roughly 2,080 a naive 40-hour-week calculation suggests. Divide one by the other and a single continuously staffed seat needs roughly five full-time-equivalent analysts before anyone has asked for a second person per shift for backup or quality, let alone a team lead.

6.1 Worked example — coverage math across models

[SENIOR MANAGER] A manager sizing a 24/7 desk uses this math before touching a vendor quote, because it's the number that determines whether outsourcing the thinnest shifts is a cost decision or the only realistic option at all.

CONCEPTUAL SAMPLE — illustrative numbers, not sourced benchmark data.

Hours needing continuous coverage, one year:	8,760
Productive hours per analyst after shrinkage (Part 5):	~1,750
FTE required per continuously staffed seat:	~5.0
Standard for two analysts on shift at all times (quality/backup):	
2 seats x ~5.0 FTE	= ~10 FTE base
+ 1 analyst for cross-training/surge buffer	= 11 FTE total
	(matches Sec. 2.1's in-house figure)

The 11-analyst figure used in §2.1 isn't a round number picked for a clean total — it's this 10-FTE base plus one buffer analyst, which is the same arithmetic every fully in-house 24/7 SOC in this book's examples runs through.

The number that this math makes visible is why small organizations almost never insource the last few hours of coverage even when they can afford full in-house staffing for business hours: the marginal cost of covering a 2 a.m.-to-6 a.m. Tuesday shift is close to a full FTE-equivalent commitment for four hours a night, because nothing about that math compresses just because the shift is short. That's the arithmetic behind co-managed's overflow direction — it isn't a compromise position, it's the correct answer to a specific cost curve.

A hybrid model sidesteps this math entirely for its detection-engineering function, which is the other reason that model is cheaper on a coverage-hours basis than it looks: detection engineers work standard business hours and don't need shift coverage at all, because their output — tuned detection content, not real-time triage — doesn't have to exist at 3 a.m. to be valuable the next morning.

The composite case below shows what happens when this math gets ignored on the way out of a vendor contract rather than on the way in.

Composite case (CASE-0202, COMPOSITE CASE EXAMPLE — merges patterns from more than one insourcing transition; not one identifiable organization.)

Management Autopsy — "terminate the MDR contract and insource all coverage in one quarter"

The decision: Leadership, unhappy with a vendor's performance after a bad quarter, decides to terminate the managed-detection contract at its renewal date and hire a full 24/7 in-house team — 10 analysts plus a lead — to go live the same quarter the contract ends.

Why it seemed reasonable: A three-year cost projection showed in-house staffing would be cheaper than the contract over that horizon, and leadership wanted full control after the vendor's performance issues rather than negotiating a fix with the same provider.

How it failed: Hiring 10 qualified analysts inside one quarter ran straight into the tight mid-level market Part 7 describes — the org went live with six of the 10 seats filled. Six analysts can't cover the coverage-hours math above; the organization immediately reintroduced the exact overnight and weekend gaps the original contract existed to close, except now with no vendor safety net, because the contract had already lapsed on schedule regardless of hiring progress.

The fix: Treat a full insourcing transition as itself a co-managed period, not a single cutover date. Keep the vendor on a reduced, overflow-only contract until internal headcount has held steady at the target coverage-hours number for at least one full quarter, and only let the full contract lapse once that's confirmed rather than scheduled around a renewal date chosen for budget reasons alone.

7. Choosing and re-choosing: a decision framework

[SENIOR MANAGER] The four models aren't a one-time fork in the road. Most SOC's move through more than one of them over their lifetime, usually driven by whichever constraint from §§2–6 is binding hardest right now — which is exactly what the decision path below traces.

7.1 The decision path

[SENIOR MANAGER] The path below asks the coverage-capacity question first, because it's usually the fastest constraint to hit a hard wall on, then routes through telemetry sensitivity and detection-engineering investment depending on which branch that first answer lands on.

Figure 2.1 — Choosing (and re-choosing) a SOC operating model. CONCEPTUAL. Diagram ID **FIG-0201**. Illustrates one defensible decision path through the coverage, sensitivity, and detection-engineering-investment questions developed in §§2–6 above; it is a teaching decision tree, not a validated algorithm, and §7.2's scorecard is the finer-grained tool for weighing a real decision. Notice that co-managed is reachable from two different constraints — a staffing shortfall or a telemetry-sensitivity limit — which is part of why it's the most common fallback model in practice.

flowchart TD

```
A{"Can you staff two or more analysts\nper shift, every shift, year-round, today?"}  
A -->|No| B{"Is telemetry too sensitive for third-party\nraw access -- regulatory, IP, litigation  
hold?"}  
A -->|Yes| C{"Does overnight/weekend volume justify\nfull in-house staffing for those shifts  
too?"}  
B -->|Yes| D["Co-managed:\nin-house tiers, vendor overflow\nonly on lower-sensitivity telemetry"]  
B -->|No| E{"Can you justify two or more FTE\non detection engineering, not generalist triage?"}  
E -->|Yes| F["Hybrid:\noutsource 24/7 monitoring +\nembedded detection engineering"]  
E -->|No| G["Fully outsourced:\nMSSP / MDR"]  
C -->|No| D  
C -->|Yes| H["Fully in-house"]
```

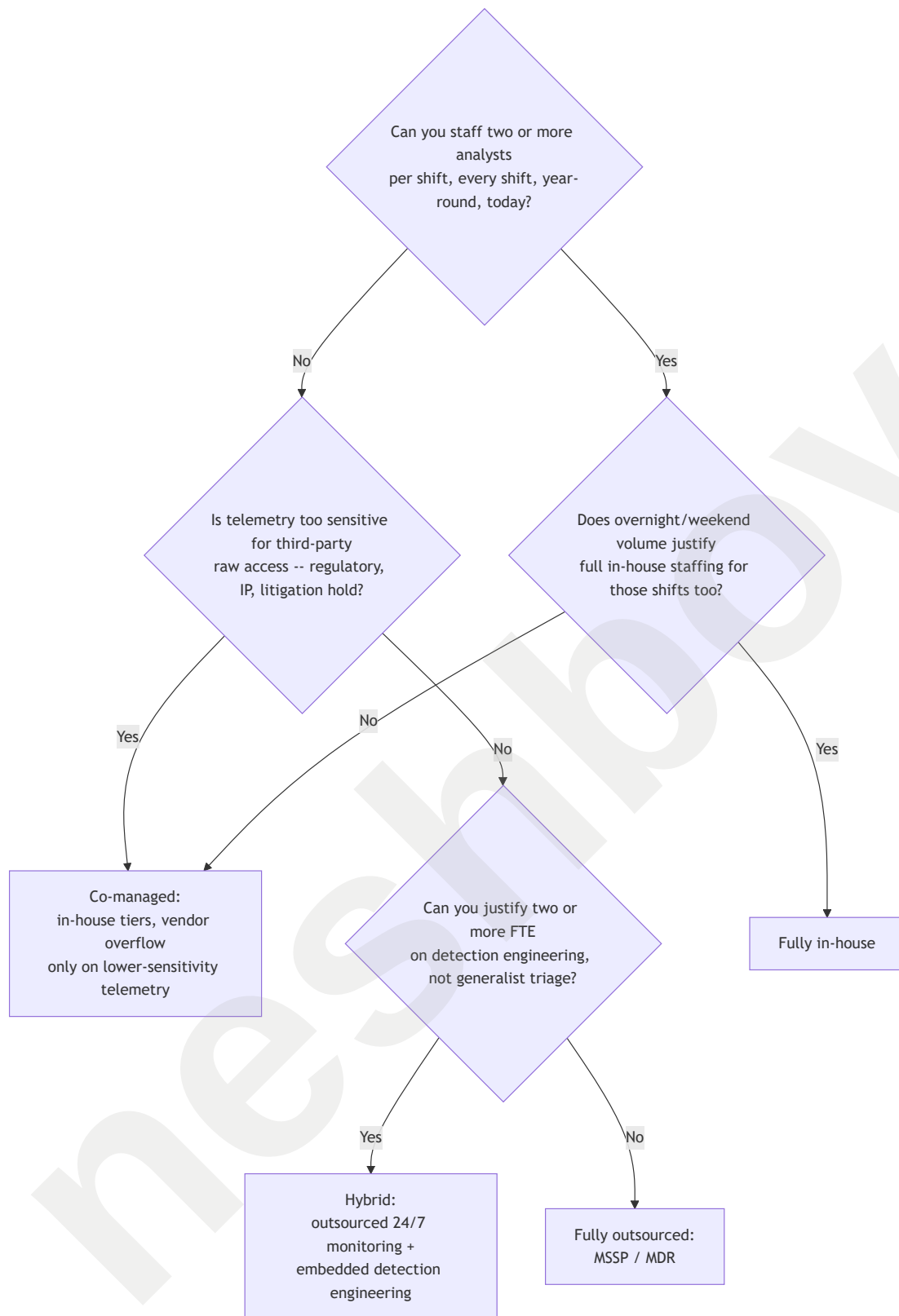


Figure 2.1 — Choosing (and re-choosing) a SOC operating model. *CONCEPTUAL*. Static decision-tree diagram rendered from the Mermaid source directly above. It supports the model-selection guidance in this section, restated in scorecard form in §7.2 immediately below. Diagram ID [FIG-0201](#).

7.2 The scorecard

[SENIOR MANAGER] Where the decision path above gives a single recommended path, the scorecard below is the tool for a manager defending that recommendation against a stakeholder who wants to see the tradeoffs made explicit rather than collapsed into one arrow. Appendix A6 develops this into a full scoring worksheet alongside Part 20's budget templates; the version here is the comparison in its simplest form.

The table below compares all four models across the dimensions this part has walked through, to support exactly that stakeholder conversation (CONCEPTUAL SAMPLE — illustrative ratings, not a scored instrument validated against real outcomes).

Dimension	Fully in-house	Fully outsourced	Co-managed	Hybrid + embedded DE
Steady-state cost, mid-market scale	Highest	Lowest	Mid, depends on split	Mid-low
Ramp time to full operation	Slowest (hiring-bound)	Fastest (contract-bound)	Moderate	Moderate
Tuning-authority latency	Same-day, if detection-as-code discipline exists	Vendor change-window bound	Fast for in-house tiers, slow for vendor tiers	Same-day for custom content, vendor-bound for generic rules
Telemetry-sensitivity fit	Best	Weakest	Good, if overflow scope is limited correctly	Good, if forwarding decisions are deliberate
Retention lever for senior talent	Strong, if growth funds the ladder	Weak (vendor's problem, still your risk)	Strong for in-house tiers	Strong (detection-engineer role is the lever)
Coverage-hours cost efficiency	Weakest for thin shifts	Strongest	Strong (thin shifts bought, not staffed)	Strong (monitoring bought; DE doesn't need shift coverage)

8. Model transitions: the MSSP-to-hybrid trajectory, and the reverse

[SENIOR MANAGER] A common trajectory starts with a small or newly formed security function choosing fully outsourced coverage first, simply because the coverage-hours math in §6 makes staffing a full 24/7 desk indefensible at 40 or 50 employees. As headcount, alert volume, and organization-specific risk grow, the next step is usually insourcing Tier 1 and Tier 2 for business hours while keeping the vendor for overnight and weekend overflow — arriving at co-managed not as a downgrade from full outsourcing, but as the next rung up. Adding a dedicated detection-engineering function once alert volume and organizational complexity justify it — without necessarily insourcing 24/7 monitoring at all — is the move into the hybrid model instead of, or in addition to, that co-managed step.

[SENIOR MANAGER] The reverse trajectory is just as real and gets treated far too often as a failure rather than a correction: a fully in-house SOC that's chronically short-staffed on its worst shifts converts overnight or weekend coverage to a vendor overflow arrangement, arriving at co-managed from the opposite direction. That move is frequently the right one precisely because the alternative — leaving the same one or two analysts covering the graveyard shift indefinitely — is the burnout-and-fatigue failure mode Part 17 covers in detail, not a superior outcome just because the org chart says "fully in-house."

[SENIOR MANAGER]

Neither trajectory is the same question as SOC maturity. Part 30 — SOC Maturity Models scores staffing, process, training, and tooling maturity as its own lens, deliberately kept separate from the operating-model choice this part covers; a fully outsourced SOC can be operating at a mature level on Part 30's terms, and a fully in-house SOC can be immature on every one of them. Which model an organization runs is a build/buy/blend decision driven by the economics in §§2–6; how well it runs whichever model it picked is a different question entirely.

What Would Change My Mind This part treats the outsource-first, insource-later trajectory in this section as the dominant pattern for growing security functions. If a meaningful share of mid-market organizations were found to insource fully from day one and never touch a vendor relationship at all — not because sensitivity forced it, but because the coverage-hours economics in §6 didn't apply the way this part assumes — that would be worth re-examining against real staffing histories rather than the pattern this part currently assumes holds.

Cross-references: Assumes Part 1 — SOC Manager Foundations & the Series Map for the book's overall scope boundary. Feeds forward into Part 3 — Tiering Models (the co-managed tier split), Part 5 — Headcount & Capacity Modeling (the shrinkage figure used in §6), Part 6 — Shift Pattern & Coverage Design, Part 7 — Hiring & Sourcing Analysts, Part 13 — Career Ladders & Promotion Criteria, Part 17 — Burnout, Fatigue & Wellbeing, Part 18 — Attrition & Retention, Part 20 — Building & Defending the SOC Budget, Part 21 — Tooling Procurement & Platform Strategy, Part 22 — MSSP & Managed-Service Contract Management, Part 27 — Legal, HR & Compliance Interfaces, and Part 30 — SOC Maturity Models. Cites SOC Playbook Handbook, Part 27 — Escalation Quality and points to Detection Engineering Handbook V2, Parts 3–29, Part 6 — Normalisation, and Parts 41–43 for the technical content this part deliberately doesn't re-cover.

Part 3 — Tiering Models: L1/L2/L3 and Beyond

Why this part exists

[CONCEPT] Tiering is an org chart, not a technology. A SOC that labels its analysts "L1," "L2," and "L3" has made exactly one decision — who is allowed to touch which alert, and who has to hand it to someone else — and that decision has consequences for cost, speed, and retention regardless of what SIEM, EDR, or SOAR sits underneath it. This part exists because that decision is made badly more often than it's made well: copied from a job-board template, inherited from whoever built the SOC five years ago, or flattened on a whim because a vendor blog said tierless was the future. The job here is to give a manager a real basis for the decision — what each tier and specialist function should own, what actually has to be true before a ticket moves between them, when the tierless alternative is the right call instead, and a framework for telling "tiering is earning its cost" apart from "tiering is just adding a queue."

This part owns the organizational-design question: who does the work, and under what structure. It does not own what a good hand-off looks like once tiers exist — SOC Playbook Handbook, Part 27 — Escalation Quality already owns that mechanic in full, and re-deriving it here would be the exact silent-duplication failure this book's provenance notes warn against. It does not own severity scoring, which drives many hand-off triggers below — that's SOC Playbook Handbook, Part 29 — Playbook Severity Model. And it does not own what a threat hunter, detection engineer, or incident responder technically does day to day — that's Detection Engineering Handbook V2 territory, cited by part number wherever it matters, never re-explained. What this part owns is narrower and, for most SOCs, never actually written down anywhere: the shape of the org chart itself, and the reasoning that should have produced it.

This part assumes the reader has already read Part 1's framing of what decisions belong to a SOC manager, and Part 2's build/buy/blend operating-model decision — tiering is a second, largely independent axis on top of that choice, and §7 below returns to how the two interact.

1. What a tier is actually buying you, and what it costs

[CONCEPT] A tier is a filter with a price tag on both sides. The classic rationale is straightforward: cheap, less-experienced staff absorb the highest-volume, lowest-complexity work, so that expensive, experienced staff spend their time on the smaller number of things that actually need their judgment. Done well, this is a real economic win — a Tier 1 analyst earning a Tier 1 wage closes the 70–85% of alerts that turn out to be routine or false-positive, and a Tier 2 or Tier 3 analyst earning two to three times that wage never sees them (CONCEPTUAL SAMPLE — illustrative closure-rate range, not sourced benchmark data). That arithmetic is the entire case for tiering, and it's a good one when the underlying assumptions hold.

[SENIOR MANAGER] The assumptions stop holding the moment the hand-off itself starts costing real time. Every escalation — even a well-written one, following every mechanic SOC Playbook Part 27 recommends — requires the receiving analyst to re-read the alert, re-establish context, and re-verify at least some of what the first analyst already checked, because trusting an escalation note blind is its own

failure mode. That re-establishment cost runs somewhere in the range of eight to fifteen minutes per hand-off even on a good day (CONCEPTUAL SAMPLE — illustrative range, not measured against this book's own data). Multiply that by hand-off volume and it stops being a rounding error: a SOC escalating 200 tickets a day is spending on the order of 25–50 analyst-hours a day just moving tickets between people, before anyone does a minute of actual investigation on them. Tiering is a bet that the filtering savings on the cheap side exceed that hand-off tax on the expensive side. The rest of this part is about how to tell, for a specific SOC, which way that bet actually runs.

Operational Reality *The tier chart on the wall says Tier 1 closes known-pattern alerts and escalates everything else on a clean threshold. What actually happens during a volume surge is that Tier 1 starts pushing ambiguous tickets up early to protect their own queue metrics, Tier 2 starts pushing tickets back down with "check X and re-escalate if still unclear" to protect theirs, and a ticket can cross the L1/L2 boundary two or three times before anyone actually works it. A tier boundary that isn't monitored for round-trips is a boundary that's quietly become a stalling mechanism, not a filter.*

2. The three-tier core model and what each tier owns

[CONCEPT] The classic three-tier structure divides work by depth of investigation required, not by seniority for its own sake — seniority is a proxy for the skill the depth demands, not the goal itself.

2.1 Tier 1 — triage and known-pattern response

[FRONTLINE MANAGER] Tier 1 owns initial alert intake: running the matching playbook, closing what the playbook's decision logic clears as benign or fully resolvable at that level, and escalating what it doesn't. Tier 1's value is speed and volume, not depth — a well-run Tier 1 function should be closing the large majority of what it sees inside a target handle time measured in minutes, and escalating a minority that needs a different skill set or more context than the playbook anticipated. This part does not define what a good playbook or a good disposition decision looks like mechanically — that library lives entirely in SOC Playbook Handbook.

[SENIOR MANAGER] What matters here is staffing: Tier 1 is where a SOC absorbs entry-level and career-changer hiring (Part 7 covers sourcing those pipelines), and it's the tier most exposed to the automation shift covered in Part 32 — a growing share of what Tier 1 used to do by hand is now dispositioned by SOAR logic before a human ever sees it.

2.2 Tier 2 — investigation, correlation, and scoping

[FRONTLINE MANAGER] Tier 2 owns what Tier 1 couldn't close on its own: correlating an alert against other activity, pulling additional context (endpoint, identity, network) the initial playbook didn't call for, and scoping whether what looked like one alert is actually part of something bigger. Tier 2 is where a case, in the sense the rest of the series uses the word, actually starts to take shape — bundling related alerts, ruling competing explanations in or out, and deciding whether what's in front of them needs Tier 3 or IR involvement at all. A common ratio in a mature three-tier SOC runs somewhere around three or

four Tier 1 analysts for every Tier 2 analyst (CONCEPTUAL SAMPLE — illustrative ratio, not a sourced benchmark); the right ratio for a specific SOC is a headcount-modeling question this part doesn't own — see Part 5.

2.3 Tier 3 — deep technical specialists and incident command

[FRONTLINE MANAGER] Tier 3 owns the work that needs depth Tier 2 doesn't carry day to day: forensic analysis, malware behavior review, and — when a Tier 2 scoping effort confirms real impact — standing up as incident command or feeding a dedicated IR function.

[SENIOR MANAGER] Tier 3 is also usually the smallest tier by headcount and the one where a SOC is most exposed to a single-point-of-failure risk if only one or two people carry that depth; Part 14's succession-planning material and Part 18's retention material both exist partly because losing a Tier 3 analyst hurts more than losing a Tier 1 analyst, disproportionate to the headcount gap between them.

2.4 A RACI view across tiers and specialist functions

[SENIOR MANAGER] The three-tier writeup above describes steady-state ownership in prose; a RACI matrix makes the same ownership auditable at a glance, and is worth building once a tier chart exists rather than trusted from memory. The table below maps common SOC workflow stages against who is Responsible, Accountable, Consulted, or Informed, across the three tiers and the specialist functions from §3.

CONCEPTUAL SAMPLE — illustrative RACI assignment for a generic three-tier SOC, not a template for any specific organization's actual authority structure.

Workflow Stage	Tier 1	Tier 2	Tier 3	Detection Engineering	Threat Hunting	IR Lead
Alert intake & known-pattern triage	R/A	I	—	—	—	—
Multi-source investigation & scoping	C	R/A	I	—	—	I
Escalation / severity call	R	A	C	—	—	—
Forensics & malware behavior review	—	C	R/A	C	—	C
Incident command once confirmed	I	I	C	I	I	R/A
Turning disposition data into a tuned or new analytic	I	C	C	R/A	C	I
Hypothesis-driven proactive hunting	—	I	I	C	R/A	—

[SENIOR MANAGER] This view is only as good as its maintenance discipline — a RACI matrix built once at tier-design time and never revisited drifts the same way a headcount model does (Part 5's warning about a model built once at budget time applies equally here). It also has a real limit worth stating plainly: it shows steady-state ownership, not who has override authority once a major incident is declared and the org chart temporarily reshuffles around a single incident commander — that authority question belongs to Part 28's treatment of the manager's role during a live incident, not to this table.

3. Specialist tiers aren't "L4": threat hunting, detection engineering, and IR

3.1 Why "L4" is the wrong mental model

[SENIOR MANAGER] The most common structural mistake in tier design is treating threat hunting, detection engineering, and incident response as a fourth rung on the same ladder — "L4" — staffed by promoting the most senior Tier 3 people and leaving them sitting at the top of the same escalation chain. That's a category error. Tier 1 through Tier 3 are reactive: work arrives as an alert, and the tier structure exists to route it. Threat hunting is proactive: a hunter goes looking for activity that never generated an alert in the first place, on a hypothesis, not a queue item. Detection engineering isn't triage work at all — it's the feedback function that turns a confirmed finding, a repeated false positive, or a hunt result into a new or tuned analytic. Incident response, in most SOCs, isn't a standing tier that sits idle waiting for work; it's an on-call rotation drawn from Tier 2 and Tier 3 staff, activated only when a scoping effort actually confirms an incident.

Cross-Book Pointer *This part does not cover what a threat hunter or detection engineer actually does — that's genuinely a different book's territory. For threat hunting methodology, see Detection Engineering Handbook V2, Parts 34–36; for the detection rule logic and telemetry work a detection engineer produces, see Detection Engineering Handbook V2, Parts 3–29. This part's only job is where those functions sit relative to the tiers and how they're staffed and protected — read those parts for what the work itself looks like, then come back here for how to organize around it.*

[SENIOR MANAGER] Once "L4" is on the org chart, it behaves like a tier whether anyone intends it to or not — unresolved Tier 3 tickets drift upward to it by the same gravity that moves tickets from Tier 1 to Tier 2, because it's the next name up the chain. The fix is structural, not a policy memo: give threat hunting and detection engineering their own backlog, entirely separate from the live-alert queue, with a named owner whose calendar time is tracked against it the same way an analyst's handle time is tracked against triage.

3.2 The specialist-tier trap: dumping ground vs. protected function

[HR/PEOPLE] A second, related mistake is using a threat-hunting or detection-engineering role as an informal reward tier — somewhere to put a burned-out senior analyst who's earned a change of pace — without building the actual skill pipeline or protecting the time the role needs to produce anything. Part 13's career-ladder material covers building a real pathway into these roles; the point here is narrower: a specialist tier with no protected calendar time isn't a specialist tier, it's a title change with the same job underneath it.

Blind Spot Standard tiering metrics — escalation rate, hand-off time, MTTR by tier — have nothing to say about whether a specialist function is actually doing specialist work. A threat-hunting program can show a fully staffed roster and zero findings for two straight quarters because every hunter's calendar is quietly absorbed by live-incident overflow, and none of the metrics a manager is already tracking will surface that. The only way to see it is to track protected time directly — hours actually spent on hunting or engineering backlog versus hours pulled into reactive work — as its own number, not inferred from anything else.

Management Autopsy — "promote the two senior L3 analysts into a Tier 4 threat-hunting function" (CASE-0301, COMPOSITE CASE EXAMPLE)

The decision: A 40-analyst in-house SOC created a "Tier 4 — Threat Hunting" role, staffed by promoting its two most senior Tier 3 analysts, with no new hires and no change to the existing escalation chain.

Why it seemed reasonable: It rewarded seniority without adding headcount cost, it gave two flight-risk analysts a title change that read as a promotion, and threat hunting had been on the roadmap for two years with no dedicated budget to fund it any other way.

How it failed: Because Tier 4 sat at the top of the same escalation chain as Tier 3, every unresolved Tier 3 ticket eventually landed there too. Within a quarter, the two "hunters" were spending on the order of 35–40 hours a week on live-incident overflow and single-digit hours on anything that looked like a hypothesis-driven hunt (CONCEPTUAL SAMPLE — illustrative hours, not measured against this book's own data). The hunting program produced no documented findings for two consecutive quarters, and both analysts privately described the promotion as a demotion with better pay.

The fix: Detach the function from the escalation chain entirely. Threat hunting needs its own backlog, a rule that live-incident work requires the SOC manager's explicit sign-off to interrupt it, and a tracked protected-time number reviewed monthly — not a title sitting at the top of a ladder it was never meant to be part of.

4. Hand-off criteria: what actually triggers a tier change

[FRONTLINE MANAGER]

A tier structure without a written hand-off trigger is a tier structure that runs on individual judgment alone, which produces exactly the round-trip problem the Operational Reality box in §1 describes. The trigger doesn't need to be complicated, but it does need to be specific enough that two different analysts would make the same call on the same ticket. The table below is a starting set of trigger categories, not an exhaustive rulebook — the specific thresholds belong in each SOC's own playbook library.

The table below maps common escalation triggers to the tier boundary they cross and the basis a manager can audit them against.

Trigger	From → To	Basis	Example Threshold
Severity	Tier 1 → Tier 2	Playbook severity score	Medium or above (SOC Playbook Handbook, Part 29 — Playbook Severity Model)
No playbook match	Tier 1 → Tier 2	Triage produced no clear disposition path	Unresolved after 15 minutes against the matched playbook
SLA risk	Tier 1 → Tier 2	Time in queue approaching breach	Unresolved at 50% of the SLA window
Confirmed impact	Tier 2 → Tier 3 / IR	Scoping confirms lateral movement, data access, or persistence	Any confirmed post-exploitation activity
Specialist technique needed	Tier 2 → Tier 3	Requires forensics or malware behavior review beyond Tier 2 scope	Disk or memory forensics required
Recurring false positive	Tier 1 or Tier 2 → detection engineering	Same pattern repeatedly closed as benign	Five or more identical dispositions in 30 days

Cross-Book Pointer *This table names when a ticket should move tiers. It says nothing about how to write the hand-off itself — what context has to transfer, how to phrase an escalation note so the receiving analyst doesn't have to redo the first analyst's work, or how a manager audits hand-off quality at scale. All of that is SOC Playbook Handbook, Part 27 — Escalation Quality, in full. Design your triggers here, then go there for the mechanics of executing them well.*

[FRONTLINE MANAGER] The last row deserves its own note: a recurring false positive isn't a Tier 2 or Tier 3 problem at all, no matter how many times it gets escalated. Escalating the same noisy alert five times in a month and resolving it as benign five times is a detection-quality problem wearing a triage costume — see Detection Engineering Handbook V2, Part 38 — False Positive Engineering for the tuning fix, and Part 9 of this book for how a manager tells that pattern apart from a genuine staffing shortfall in the first place.

Manager's Note *Write hand-off triggers as a yes/no test, not a judgment call dressed up as a rule. "Escalate if the ticket seems complicated" produces a different answer from every analyst who reads it; "escalate if no playbook step matched within 15 minutes" produces the same answer from all of them, which is the entire point of writing the trigger down in the first place.*

Field Test Setup: A written hand-off-trigger table exists (like the one above), but has never been checked against how analysts actually use it. **Action:** Pull three closed Tier 1 tickets that sat right at a trigger boundary — close but not obviously over the line — and ask two different Tier 1 analysts, independently and without seeing each other's answer, whether each one should have escalated. **Expected result:** Both analysts should reach the same call on all three tickets. A split verdict on even one means the trigger's wording is doing less work than the table implies, and it's worth rewriting before trusting it as the basis for a staffing or automation decision.

5. The tierless alternative: follow-the-alert

[CONCEPT] The tierless model removes the structural hand-off entirely. One analyst owns an alert from intake through closure, pulling in a specialist only for a bounded task — forensics, malware reverse engineering, incident command — not because a threshold on a chart says the ticket has to move. The analyst who opened it is still the owner when it closes.

[SENIOR MANAGER] The case for tierless is the mirror image of the case for tiering: it eliminates the hand-off tax described in §1, and it eliminates the redundant-investigation problem where Tier 2 re-verifies work Tier 1 already did competently. It also demands far more from every analyst on the roster — there's no cheap tier to absorb volume while someone else is trained up, so the hiring bar, and the pay band that comes with it, has to be uniformly higher than a tiered SOC's Tier 1 band. That's the tradeoff in one sentence: tierless converts a latency cost into a hiring-and-compensation cost, and whether that trade is worth making depends entirely on the four variables §6 walks through.

flowchart TB

subgraph Tiered["Tiered model"]

direction TB

A1["Alert fires"] --> A2["Tier 1: triage against\nknown playbook pattern"]

A2 -->|"Fully resolved"| A3["Closed at Tier 1"]

A2 -->|"Severity or ambiguity\nthreshold met"| A4["Hand-off to Tier 2"]

A4 --> A5["Tier 2: investigation,\ncorrelation, scoping"]

A5 -->|"Resolved"| A6["Closed at Tier 2"]

A5 -->|"Confirmed impact or\nspecialist skill needed"| A7["Hand-off to Tier 3 / IR"]

A7 --> A8["Tier 3: forensics,\nincident command"]

end

subgraph Tierless["Tierless: follow-the-alert"]

direction TB

B1["Alert fires"] --> B2["Assigned analyst owns\nintake through closure"]

B2 -->|"Resolved"| B3["Closed by owning analyst"]

B2 -->|"Needs forensics, malware\nreview, or incident command"| B4["Pull in specialist for a\nbounded task; ownership\nstays with the analyst"]

end

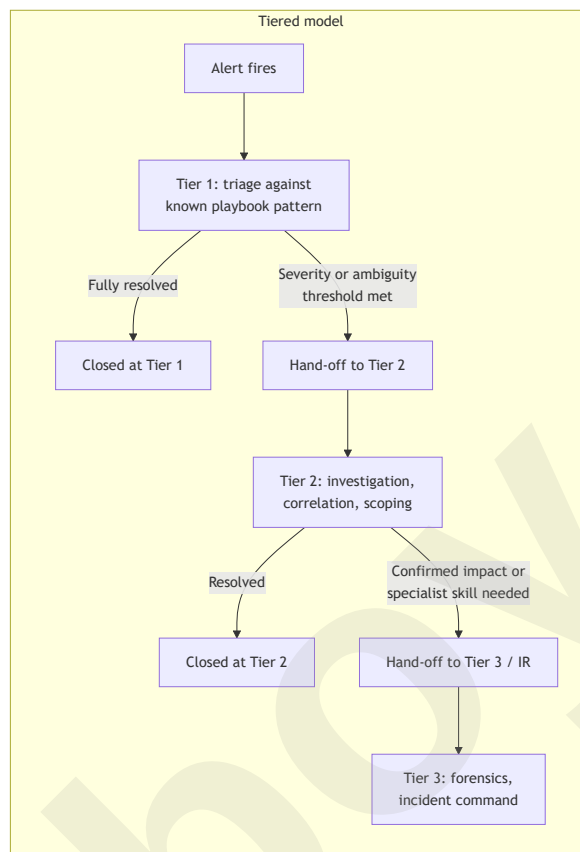
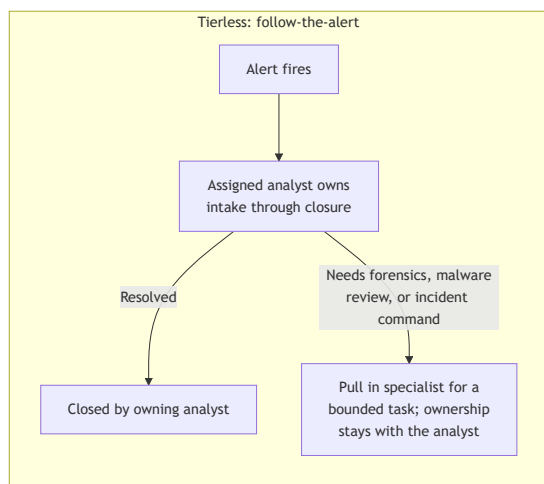


Figure 3.1 — Tiered escalation chain vs. tierless follow-the-alert ownership. *CONCEPTUAL.*

Illustrates the structural difference between a strict Tier 1→2→3 hand-off chain, where each threshold moves ownership to a new person, and a tierless model, where one analyst retains ownership from intake to closure and pulls in a specialist only for a bounded task. This is a process diagram of two design options, not a capture of either model running in a real SOC.

Management Autopsy — "flatten three tiers into one generalist pod to cut MTTR"
(CASE-0302, COMPOSITE CASE EXAMPLE)

The decision: A 22-analyst SOC eliminated its Tier 1/Tier 2 distinction in one move, converting every analyst into a generalist who was expected to triage and investigate their own alerts end to end, on the strength of industry commentary that tierless SOC's report lower MTTR.

Why it seemed reasonable: Hand-off latency had been identified, correctly, as the single largest contributor to the SOC's MTTR. Removing the hand-off looked like a direct fix, and leadership wanted a visible win without waiting for a multi-quarter hiring or training initiative.

How it failed: Roughly two-thirds of the roster had been hired into what was, until that point, an explicitly Tier 1 role, through the help-desk-to-SOC pipeline Part 7 covers — competent at known-pattern triage, not yet trained for independent investigation. Ambiguous tickets that would previously have moved cleanly to a Tier 2 specialist instead sat unresolved, because no one held explicit authority or skill to close them alone. The SOC's three most experienced analysts began fielding a steady stream of informal "can you take a look at this" pings, recreating a tier boundary in practice with none of the tracking, staffing, or hand-off discipline the formal version had.

The fix: Tierless only works once the average analyst's skill floor is high enough to close most tickets alone — that means raising the hiring bar and the pay band first (Part 7 for sourcing, Part 18 for what it costs to hold that bar against competitor poaching), not flattening the structure and hoping the skill gap closes itself. This SOC eventually reintroduced a lightweight two-tier model rather than reverting fully, once it became clear a straight revert would have looked like admitting the initiative failed rather than correcting course.

6. Decision framework: when tiering helps vs. when it only adds latency

6.1 The four variables that actually decide it

[SENIOR MANAGER] Four variables do most of the work in deciding whether tiering earns its cost for a given SOC, and none of them is "what everyone else in the industry does."

[SENIOR MANAGER] Alert volume and its variance matter first: tiering pays for itself when there's enough volume, and enough spread between routine and complex tickets, that a cheap first pass genuinely saves expensive time downstream. A SOC seeing 4,000 alerts a day with a long tail of one-minute closures and a short tail of genuinely hard investigations is exactly tiering's home turf. A SOC seeing 400 alerts a day where almost every one needs real context has much less to filter, and a hand-off adds cost without removing much.

[SENIOR MANAGER] Team size sets a hard floor: a tier needs enough people in it to cover shifts without becoming a single point of failure, and a "tier" of one or two people isn't a tier — it's a promotion ceiling and an outage risk wearing a tier's name. Below roughly ten to twelve analysts total, sustaining

three separately staffed tiers across a coverage schedule usually means either chronic understaffing in one tier or idle capacity nobody can justify; Part 6's shift-coverage math is the tool for checking this precisely for a specific schedule.

[HR/PEOPLE] The hiring pool's skill floor matters as much as headcount: if the realistic candidate pool is mostly early-career and career-changer talent, tiering is how a SOC affordably hires and trains that pool up rather than needing every seat filled by someone already senior — see Part 7 for that pipeline in detail. If the realistic pool is mostly experienced hires at a premium pay band, tiering wastes the seniority a SOC is already paying for by making a senior hire wait behind a Tier 1 filter they don't need.

[SENIOR MANAGER] Detection debt and false-positive rate are the variable managers most often miss, because it looks like a technical question and gets waved off as someone else's book's territory. It isn't, for this specific purpose: a SOC with high detection debt and a noisy rule set has a lot of low-value alerts that genuinely benefit from a cheap human filter before anything expensive touches them, which argues for tiering almost regardless of the other three variables. A SOC with a well-tuned detection set and a low false-positive rate — see Detection Engineering Handbook V2, Part 43 — Detection Debt for how that's measured — has much less cheap-to-filter noise, and the case for a filtering tier weakens correspondingly.

6.2 The tiering fit scorecard

[SENIOR MANAGER] The table below turns those four variables into a quick working scorecard. Score each row honestly for your own SOC before deciding anything.

CONCEPTUAL SAMPLE — illustrative scoring thresholds for teaching purposes, not a validated instrument.

Criterion	Favors Tiering	Favors Tierless
Alert volume per analyst per day	High (50+) with wide severity spread	Low (under 15), mostly already high-context
Team size	12 or more analysts — enough for shift coverage per tier	Under 10 analysts total
Hiring-pool skill floor	Mostly early-career / help-desk pipeline	Mostly experienced or senior hires
Detection debt / false-positive rate	High — noisy rules need cheap filtering	Low — well-tuned detections

[SENIOR MANAGER] A clean sweep in one column is a strong signal. A split result — two rows favoring tiering, two favoring tierless — usually means the honest answer isn't a forced binary pick at all; it's the hybrid model §7 covers, where only part of the workflow carries a hand-off.

6.3 Worked example: two SOC's, same alert volume, opposite answer

[SENIOR MANAGER] The scorecard's value is easiest to see when two SOC's land on opposite answers despite looking similar on the one number people default to comparing — alert volume.

CONCEPTUAL SAMPLE — illustrative comparison for teaching purposes, not a sourced benchmark.

SOC A – regional retail, in-house, 30 analysts

Alert volume: ~3,800/day, wide severity spread
Hiring pool: mostly early-career, help-desk pipeline
Detection debt: high; several rules generate 40+ FPs/week each
Pay bands: Tier 1 ~\$62K, Tier 2 ~\$92K, Tier 3 ~\$135K
Scorecard result: 4-for-4 favors tiering
Structure adopted: classic three-tier

SOC B – SaaS platform security team, in-house, 9 analysts

Alert volume: ~450/day, mostly already high-context
Hiring pool: mostly senior hires, ex-Tier 2/3 from other SOC's
Detection debt: low; detections tuned quarterly against real disposition data
Pay bands: single senior band, ~\$118K across the roster
Scorecard result: 4-for-4 favors tierless
Structure adopted: follow-the-alert, specialists pulled in as needed

[SENIOR MANAGER] Both SOC's are staffed appropriately for their own inputs. A manager who benchmarked SOC B against SOC A's tier chart because "that's the industry-standard structure" would be importing a cost structure built for a different hiring pool and a different noise floor — the single most common reason a copied org chart underperforms the one it was copied from.

What Would Change My Mind *This framework treats alert volume, team size, hiring-pool skill floor, and detection debt as the four variables that determine tiering fit. If a SOC held three of those four firmly in tiering's favor and still got measurably better MTTR and lower attrition running tierless — not anecdotally, but across more than one comparable period — that would mean a fifth variable this framework is missing is doing real work, and the model here would need to be revised rather than defended.*

7. Hybrid models and how tier boundaries move over time

[SENIOR MANAGER] Few SOC's stay purely tiered or purely tierless for long, and the honest reason is usually that the scorecard's inputs themselves change. A SOC that adopts three tiers because its hiring pool is junior-heavy and its detection debt is high will, if it invests in Part 32's automation and detection-tuning work over a couple of years, find its Tier 1 volume shrinking as SOAR logic absorbs more known-pattern dispositioning. At that point the honest options are to shrink Tier 1 headcount, or to convert the surviving Tier 1 seats into a small cross-trained pod that behaves tierlessly for what's left. Neither is a failure of the original decision — the original decision was correct for the inputs at the time, and the inputs moved.

[SENIOR MANAGER] A common hybrid worth naming explicitly is the pod model: several small, cross-trained groups, each anchored by one senior analyst, each running tierlessly inside itself but sitting alongside the others rather than in a strict escalation chain above or below them. This captures much of tierless's speed for routine work while still giving a newer analyst someone senior to lean on without a formal hand-off — in effect, tiering compressed from a chain into a per-pod relationship. It works best in the middle of the scorecard, where a SOC has decent headcount and a mixed-experience roster but

hasn't crossed the volume or detection-debt thresholds that make a full three-tier chain worth its hand-off cost.

[SENIOR MANAGER] This is also where Part 2's operating-model decision reconnects with tiering directly, rather than sitting alongside it as an independent axis. A co-managed model — in-house tiers with vendor overflow, or the reverse — effectively splits a tier boundary across an organizational and a company boundary at the same time: an escalation from an outsourced Tier 1 to an in-house Tier 2 carries the ordinary hand-off tax from §1 plus a contractual layer neither purely in-house nor purely tierless designs have to account for. Part 22's contract-governance material covers structuring that boundary's SLA commitments; the tier-design point here is narrower — decide the tier structure and the operating model together, not sequentially, because a co-managed split changes where the hand-off tax in §1 actually lands.

[SENIOR MANAGER] Because the scorecard's four inputs move, re-score them on a fixed cadence rather than trusting the org chart to flag its own obsolescence — once a year at minimum, and after any change large enough to move a whole row (a new SOAR deployment eating Tier 1 volume, a hiring-pool shift after a competitor opens a local office and starts poaching, per Part 18). Two leading indicators are worth watching between formal re-scores: a rising round-trip rate on the boundary the Operational Reality box in §1 describes, and a Tier 1 closure rate that creeps down even though the underlying alert mix hasn't changed — both usually mean the tier boundary was drawn for a version of the SOC that no longer exists.

Cross-references

This part assumes Part 1 — SOC Manager Foundations & the Series Map (the manager's-decisions frame) and Part 2 — SOC Operating Models (the build/buy/blend axis §7 reconnects to). It feeds forward into Part 4 — Organizational Placement & Charter, Part 5 — Headcount & Capacity Modeling, Part 6 — Shift Pattern & Coverage Design, Part 7 — Hiring & Sourcing Analysts, Part 9 — Queue Health & Workload Management, Part 10 — Competency Models & Skills Matrices, Part 13 — Career Ladders & Promotion Criteria, Part 14 — Mentorship & Knowledge Transfer, Part 18 — Attrition & Retention, Part 22 — MSSP & Managed-Service Contract Management, and Part 32 — AI, Automation, and the Future Shape of the SOC, all of which take a tier decision made here as a given input. Outside this book: SOC Playbook Handbook, Part 27 — Escalation Quality (hand-off mechanics), Part 29 — Playbook Severity Model (the severity thresholds §4's table cites), and Part 30 — Automation and SOAR (what's shrinking Tier 1's volume, previewed in §7); Detection Engineering Handbook V2, Parts 3–29 (detection engineering's technical content), Parts 34–36 (threat hunting methodology), Part 38 — False Positive Engineering, and Part 43 — Detection Debt (the noise-floor variable in §6's scorecard).

Part 4 — Organizational Placement & Charter

Why this part exists

[CONCEPT] Part 3 answered who does the work inside the SOC — tiers, hand-offs, specialist roles. This part answers a different question the org chart asks before any of that matters: who does the SOC answer to, and what has it actually been given permission to do? Those are not the same question, and conflating them is how a manager ends up running a well-tiered, well-staffed team that still can't get a critical patch applied on schedule because the team it needs to move reports to the same VP the SOC reports to, and that VP's other KPI is uptime, not remediation speed. Placement is the first constraint on everything else in this book — a headcount model, a QA program, a board narrative all assume a level of authority that placement either grants or withholds.

The charter is where that authority gets written down instead of assumed. A charter that only states what the SOC does is half a document; the more load-bearing half states what it deliberately does not do, so that the first time a patch doesn't ship, an insider-threat finding needs HR involvement, or a business unit wants the SOC to babysit a compliance checkbox it was never resourced to own, there's a written answer instead of a fight settled by whoever yells first. That standing division of labor — with IT operations, engineering, legal, privacy, and physical security — is also the seed of the full politics treatment in Part 26 — Cross-Team Politics & Stakeholder Alignment; this part builds the RACI, Part 26 covers what happens when someone ignores it.

This part does not cover the mechanics of a good hand-off between tiers once an alert is already in the queue — that boundary, and the escalation-quality mechanics for a single ticket, belong to SOC Playbook Handbook, Part 27 — Escalation Quality. It does not re-litigate the build/buy/blend decision from Part 2 — Placement questions apply whether the SOC is fully in-house, co-managed, or a hybrid with an embedded detection-engineering function, and this part assumes the operating-model decision is either made or being made in parallel, not that it's settled first. What follows is squarely about where the function sits, what its charter says, and who it has a standing agreement with before anything goes wrong.

1. Placement is a design decision, not an org-chart footnote

1.1 What placement actually changes

[CONCEPT] Four things move when a SOC's reporting line changes, and none of them are cosmetic. Budget authority — whose signature approves a new tool, whose budget absorbs a headcount request — sits with whoever the SOC reports through, not with the SOC manager alone, regardless of how the org chart is drawn below that line. Escalation weight — how far up the chain a SOC manager can go without asking permission first — is set by proximity to the executive layer, not by title. Independence — whether the SOC's findings about a team's negligence can survive that team's manager reading them — is compromised the moment the SOC and the team it monitors share a boss who evaluates both on the same scorecard. And tooling autonomy — whether the SOC can mandate onboarding a new log

source from a system another team owns — depends on whether that other team's manager outranks, is a peer of, or reports to the same person as the SOC manager.

[SENIOR MANAGER] None of this shows up in a job description. A SOC manager evaluating a new role, or defending the current one, should ask exactly one diagnostic question before anything else: two years from now, if this SOC needs to force a remediation deadline against a business unit that doesn't want to comply, who actually has to say yes, and how many layers separate that person from me? If the honest answer is "the same person I already report to, and they've never had to choose between us and them," that's the placement to renegotiate before it's tested for real.

1.2 The independence problem

[CONCEPT] A SOC that reports through IT operations — a common, cost-driven choice, because IT already owns the infrastructure the SOC monitors and the ticketing system it triages into — inherits a structural conflict the org chart never states out loud: the SOC's job includes flagging IT operations' own negligence, and its reporting line runs through the person accountable for IT operations' performance.

Blind Spot A SOC reporting through the same IT leadership chain it monitors cannot see its own filtering. Findings that would embarrass the reporting chain — a patch SLA blown for six months on a server the infrastructure director personally signed off on, a misconfigured firewall rule the network team's own lead wrote — don't get suppressed through any dramatic act of censorship; they get "deprioritized," "queued behind a change freeze," or quietly reclassified to a lower severity in a conversation the SOC manager has no standing to refuse, because the person asking is the SOC manager's own boss. No QA program or metrics dashboard catches this, because the finding was real and got logged — it just never escalated past the one person with both the authority and the motive to sit on it.

2. The four common reporting lines

2.1 CISO-direct

[SENIOR MANAGER] The SOC reports to a dedicated CISO (or equivalent VP/Head of Security) who reports to the CIO, COO, or CEO, with no IT-operations or engineering layer in between. This is the placement with the fewest structural conflicts, because the CISO's own performance is measured on security outcomes, not on the uptime or delivery-velocity metrics the SOC's findings sometimes threaten. It's also the most expensive and politically demanding to establish — it requires an organization willing to fund a security leadership seat as a peer to IT and engineering leadership, not a subordinate of either, and a CISO willing to spend political capital defending the SOC's escalation path the first several times it's tested.

2.2 Under the CIO or VP of IT Infrastructure

[SENIOR MANAGER] The SOC sits inside the IT organization, typically several layers below the CIO, often sharing a ticketing queue and change-management calendar with the infrastructure team it audits. This

is the cheapest placement to stand up — no new executive seat, no new budget line separate from IT's — and the most common in organizations that grew a SOC function out of an existing IT security team rather than building one from scratch. It's also where the independence problem in §1.2 shows up most reliably, because the same director who owns the SOC's budget also owns the uptime SLA the SOC's own recommendations most often threaten.

2.3 Embedded in engineering

[SENIOR MANAGER] Less common outside product-led or platform companies, this placement puts the SOC (or a detection-engineering-heavy hybrid of the kind Part 2 covers) inside the engineering organization, reporting through a VP of Engineering or a Head of Platform. It buys fast access to telemetry, code ownership, and deploy pipelines — useful when the SOC's primary detection surface is the company's own product rather than corporate IT — at the cost of the same conflict as §2.2 in a different shape: the SOC's findings about engineering's own code or infrastructure choices are being escalated through engineering's own chain of command.

2.4 Shared security services across business units

[SENIOR MANAGER] In holding companies, conglomerates, and organizations with multiple semi-autonomous business units, a central SOC serves several BUs that each have their own local IT or security lead, with the SOC reporting to a group-level CISO or security function that has no direct authority over any single BU's operations. This placement solves the independence problem structurally — the central SOC doesn't report through any BU it monitors — but introduces a different cost: local BU leaders who never agreed to the central SOC's authority in the first place, and can simply decline to act on a finding with the (accurate) observation that the SOC manager isn't in their reporting chain at all. §3 covers the matrixed variant built to soften exactly this friction.

2.5 Comparing the four placements

[SENIOR MANAGER] The table below is a decision aid, not a ranking — the right placement depends on what an organization is actually optimizing for, and every option below has been the correct choice somewhere.

Placement	Independence from teams it monitors	Typical enforcement authority	Cost to establish	Most common failure mode
CISO-direct	High — CISO's own scorecard is security outcomes	Can mandate deadlines with executive-level escalation	High — new executive seat, dedicated budget	CISO becomes a bottleneck if every escalation routes through one person
Under CIO/IT Infrastructure	Low — shares a boss with the team it audits	Recommend-only in practice, absent an explicit override clause	Low — no new leadership seat needed	Findings against IT's own priorities get deprioritized, not rejected
Embedded in engineering	Low — same conflict as IT placement, different chain	Strong for product/platform findings, weak for corporate IT	Medium — needs engineering buy-in to host a non-shipping function	Corporate-IT and vendor-risk findings get under-resourced relative to product findings
Shared security services (multi-BU)	High — sits outside any single BU's chain	Formally strong, practically dependent on group-level backing	High — needs a group-level charter every BU has actually signed	Local BU leaders treat findings as advisory since the SOC isn't in their chain

[SENIOR MANAGER] The pattern worth naming explicitly: the two cheapest placements to stand up (§2.2 and, in a different shape, §2.3) are the two most likely to produce a SOC that can detect and document a problem it cannot actually force anyone to fix. That tradeoff is sometimes the right one for a smaller organization that genuinely cannot fund a dedicated security leadership seat — but it should be a chosen tradeoff, defended with eyes open, not an accident of how the SOC happened to get built.

Composite case (CASE-0401, COMPOSITE CASE EXAMPLE — patterns merged from several mid-market placements of this kind; not a single traceable organization; cost figures below are illustrative, not audited financials.)

Management Autopsy — "fold the SOC into IT Infrastructure to save a reporting layer"

The decision: A 340-person healthcare-adjacent company with a nine-person SOC decided against creating a standalone CISO seat and instead placed the SOC under the Director of IT Infrastructure, the same director who owned the change-management calendar and the 99.9% uptime SLA for the company's core application servers.

Why it seemed reasonable: One fewer executive seat to fund, one shared ticketing queue instead of two, and faster routing for the large share of SOC findings that are genuinely IT-operations issues (unpatched software, misconfigured services) rather than security-specific ones.

How it failed: The SOC flagged a critical, actively-exploited-in-the-wild CVE on an internet-facing Exchange server, opened against a 5-day internal SLA for critical findings. The infrastructure director, accountable for the uptime SLA on that same server and mid-way through an unrelated ERP migration, deprioritized the patch three separate times over six weeks, citing change-freeze windows he controlled. The SOC manager had no escalation path that didn't run through the same director — he was, structurally, both the person blocking the fix and the person the SOC manager would have had to ask permission to escalate past. The server was compromised on day 47 of the 5-day SLA; the resulting incident response, forensics, and expedited outside-counsel work cost roughly \$180,000, on top of the ERP migration's own schedule slipping regardless.

The fix: The charter was rewritten to add a named override clause: any finding that breaches its own SLA gets an automatic escalation to the CISO-equivalent role (created as part of the same fix) or, in that role's absence, directly to the COO, with no discretion left to the manager whose own priorities the finding might conflict with. Placement didn't change immediately — budget for a standalone security leadership seat took another two budget cycles to secure — but the escalation path stopped depending on the goodwill of the person being escalated against.

3. Matrixed and dotted-line placements

3.1 The dual-boss problem

[SENIOR MANAGER] A common compromise, especially in the multi-business-unit placement from §2.4, gives a local SOC lead a dotted-line relationship to the business unit they serve and a solid-line relationship to a central security function. The intent is to get the best of both: local context and buy-in, plus central independence and consistent standards. The failure mode is equally predictable — a dotted-line and a solid-line manager who disagree about priority during a live incident leave the local lead genuinely unsure whose direction controls, and that ambiguity is worst exactly when speed matters most.

Operational Reality A matrixed reporting chart on paper shows two lines converging cleanly on one analyst. In practice, during an actual incident, one of those two managers is available and engaged and the other isn't — and the charter almost never specifies which line wins by default when only one manager is reachable. The honest fix isn't better org-chart drawing; it's a written default rule ("central security direction controls during an active incident; local BU direction controls for day-to-day prioritization") that the local lead has actually seen before the day they need it, not one they're improvising during a page from two directions at once.

3.2 When matrixing is worth the ambiguity

[SENIOR MANAGER] Matrixed placement is worth the dual-boss cost specifically when the alternative — a fully centralized SOC with no local presence — would mean the SOC has zero context on a business unit's specific risk tolerance, regulatory environment, or legacy systems, and a fully decentralized SOC per business unit would mean duplicating detection engineering, tooling, and on-call coverage three, four, or more times over. It is not worth the cost as a permanent default for a single-business-unit company that adopted it only because a reorganization briefly created two plausible reporting parents and nobody picked one — that's not a matrix, it's an unresolved decision wearing a matrix's name.

4. From placement to authority: enforce vs. recommend

4.1 The enforcement ladder

[SENIOR MANAGER] Authority isn't binary. A useful way to state what a charter actually grants is a four-rung ladder, and most real charters land on different rungs for different finding types rather than one rung for everything.

Rung	What it means in practice	What a charter needs to state to grant it
Mandate with a named escalation path	The SOC sets a deadline; if it's missed, a specific named role (not "management") is automatically notified, with no discretion to skip the notification	The SLA, who gets escalated to, and that the escalation is automatic, not requested case-by-case
Mandate with no teeth	The SOC sets a deadline; missing it has no defined consequence or automatic next step	Common default when a charter states an SLA but no escalation clause — looks like authority, behaves like a recommendation
Strong recommendation	The SOC states a required action and a risk rationale but explicitly defers the final call to the owning team	Named risk-acceptance process (Part 25 covers the manager's side of this decision) so "recommended" doesn't quietly become "ignored with no record"
Informational (FYI)	The SOC shares a finding for awareness with no expectation of action	Explicit statement that this category exists, so a team doesn't mistake a courtesy notice for a request

[SENIOR MANAGER] Most SOC's discover they're operating on the second rung — mandate with no teeth — only when a deadline is missed for the first time and nothing happens. That discovery is exactly

what §2's Management Autopsy shows: the charter said "critical findings must be remediated within five days," which reads like rung one, but named no escalation role, which meant it was actually rung two the entire time.

4.2 Where authority actually comes from

[SENIOR MANAGER] A charter's authority language only works if it's backed by placement, not instead of it. A CISO-direct SOC (§2.1) can write a real rung-one clause because the CISO has the standing to enforce it against a peer director. A SOC under IT infrastructure (§2.2) can write the identical clause, and it will function as rung two, because the escalation path the clause names loops back to the same person the finding is against. Writing stronger charter language cannot substitute for weaker placement — it can only make the gap between the two visible, which is itself useful: a charter that says "mandate" next to a reporting line that can't back it up is a documented risk-acceptance decision waiting to be noticed by whoever inherits the SOC manager's job next.

5. Writing the charter

5.1 What a charter must state

[SENIOR MANAGER] A charter that survives being read by a skeptical peer during a dispute states, at minimum: the SOC's mission in one sentence that names what it protects, not just that it "protects the organization"; the environments and asset classes actually in scope (production, corporate IT, OT/ICS if applicable, cloud accounts, third-party-managed systems); the enforcement rung (§4.1) for each major finding category, not one blanket authority level; the named escalation roles for missed SLAs, stated as roles, not people, so the clause survives a departure; a review cadence and the role accountable for keeping it current; and the sign-off chain — who actually approved this document, dated, so "the charter says" has a named authority behind it rather than functioning as an unsigned aspiration.

5.2 What the SOC is deliberately not responsible for

[SENIOR MANAGER] The scope-boundary section is the part of the charter most often left out, and the part most responsible for cross-team friction when it's missing — not because teams disagree about who should own something, but because nobody wrote down the answer before the disagreement happened live, during an incident, in front of an executive. The table below is a starting scope-boundary list, not an exhaustive one; every organization's version needs entries specific to its own structure.

Activity	Commonly assumed to be SOC's job	Who actually owns it	The SOC's actual role
Applying a patch	Yes	IT operations / infrastructure	Flags the finding, tracks the SLA, escalates per §4.1 — does not touch the system
Writing a firewall or access-control rule change	Sometimes	Network/platform engineering	Recommends the change and validates it after deployment — does not author or push it
Deciding whether a breach requires regulatory notification	Often assumed	Legal and privacy	Provides the technical timeline and scope of exposure — does not make the notification call (see Part 27 — Legal, HR & Compliance Interfaces)
Physical facility response to a badge or camera alert	Sometimes	Physical security	Correlates physical and digital signals when both exist — does not dispatch guards or manage the facility incident
HR disciplinary action following an insider-threat finding	Often assumed	HR, with legal	Documents and hands off the technical finding — does not decide or execute discipline (Part 27)
Data-classification policy authorship	Sometimes	Data governance / privacy	Enforces classification tags it's given at detection time — does not write the classification scheme
Architecture or design decisions that eliminate a finding's root cause	Sometimes	Engineering	Documents the recurring exposure and its cost — does not own the redesign
Business continuity / disaster recovery ownership	Rarely, but happens	BCP/DR function	Provides incident timeline and impact data as an input — does not own the continuity plan

Operational Reality A scope-boundary list only works if the teams named as owners have actually agreed to it — a charter that unilaterally declares "physical security owns badge-alert response" without physical security's sign-off isn't a boundary, it's a wish. The charter drafting process itself needs the RACI conversation in §6 to happen before this table is finalized, not after, or the "who actually owns it" column is the SOC manager's guess rather than a standing agreement.

5.3 A charter skeleton

[SENIOR MANAGER]

The template below is a starting skeleton for a SOC manager drafting or revising a charter from scratch — use it as a structure to fill in with an organization's real names, SLAs, and sign-off chain, not as language to adopt verbatim. Its main limitation: it cannot resolve the placement question from §1–§3 for you, and a charter drafted against the wrong placement will read well and still fail the first time it's tested, exactly as in the §2 Management Autopsy.

TEMPLATE — SOC charter skeleton, permanent ID **TMPL-0004**

```
# [Organization] Security Operations Center – Charter

## Mission
<one sentence: what this SOC protects, for whom, and the outcome it exists to produce>

## Scope
In-scope environments: <production / corporate IT / cloud / OT-ICS / third-party-managed>
Out-of-scope environments: <explicitly named, not left to be inferred>

## Authority by finding category
<table: finding category | enforcement rung (see this book's Part 4 §4.1) | named escalation role>

## What this SOC is not responsible for
<the scope-boundary table from §5.2, filled in with real named owners who have signed off>

## Standing RACI
<reference to the RACI matrix in §6 of this part, or attach it directly>

## Review cadence
This charter is reviewed <quarterly/annually> by <named role>, and immediately upon
<reorg, M&A, new CIS0/reporting-line change, or a missed SLA that reveals a gap>.

## Sign-off
<name, title, date – for every party named as an owner in the scope-boundary and RACI sections,
not only the SOC manager>
```

The most common way this skeleton fails in practice is a sign-off section signed only by the SOC manager and their own boss — a scope-boundary claim about IT operations, legal, or physical security carries no real weight until someone from those teams has actually signed the same document.

6. The standing RACI: IT operations, engineering, legal, privacy, and physical security

6.1 Why the RACI is a companion document, not a memory

[SENIOR MANAGER] The charter states the SOC's own scope; the RACI states who is Responsible, Accountable, Consulted, and Informed across the handful of activities where the SOC's work and another team's work meet — and it needs to exist as a document both sides have seen, not as an understanding one manager carries in their head. An unwritten RACI works exactly until the person who understood it changes teams, retires, or is simply unavailable during the one incident where it actually mattered.

6.2 Building the matrix

[SENIOR MANAGER] The matrix below is a starting structure across the five standing relationships this part is scoped to cover — a real organization's version will have more rows, and every cell needs an actual conversation with the named team before it's treated as settled.

Activity	SOC	IT operations	Engineering	Legal	Privacy	Physical security
Detect and validate a compromise	R/A	I	I	I	I	I
Contain at network or endpoint level	R	C	I	I	I	—
Patch or remediate root cause	C	R/A	R/A	—	—	—
Preserve evidence / apply legal hold	C	C	C	R/A	C	C
Decide external breach notification	C	I	I	R/A	R/A	—
Revoke physical access (badge, facility)	C	I	—	I	—	R/A
Insider-threat finding hand-off to HR	R	I	I	C	C	—
Chair the post-incident organizational review	C	C	C	C	C	C

[SENIOR MANAGER] Two patterns in that matrix are worth naming directly. First, the SOC is Responsible or Accountable for detection and containment and almost nothing past that point — it hands off root-cause remediation, legal decisions, and physical response to the teams that actually own the systems, statutes, or facilities involved, which is the RACI's version of the same boundary the §5.2 scope table draws in prose. Second, no single row has the SOC as the sole Accountable party for anything except detection itself — every other activity's Accountable party sits outside the SOC, which is precisely why the charter's escalation clauses in §4 matter: the SOC can be Responsible for surfacing a problem while having zero authority over whether it gets fixed on time.

6.3 Where RACIs actually break: a worked example

[SENIOR MANAGER] *COMPOSITE CASE EXAMPLE CASE-0402* — patterns merged from several regulated-industry incidents; not a single traceable organization; cost figures below are illustrative, not audited financials. A mid-size financial-services firm's SOC confirmed unauthorized access to a database containing regulated customer data, starting a contractual 72-hour notification clock the moment exposure was confirmed. The RACI on paper named Legal as Accountable for the notification decision and the SOC as Consulted, which looked complete. What the RACI didn't specify was who was Responsible for actually looping Legal in — the SOC assumed its standard incident-declaration email, sent to a distribution list that included a legal-team member who happened to be on leave, satisfied that step. Legal's on-call attorney wasn't formally engaged until 61 hours into the 72-hour clock, leaving 11 hours to complete work that normally takes days: confirming scope, drafting notifications, and briefing outside counsel. The rushed engagement of expedited outside counsel and forensics support added roughly \$40,000 in costs directly attributable to the compressed timeline, separate from the underlying incident-response cost.

Blind Spot A RACI that names who's Accountable for a decision but not who's Responsible for the specific act of notifying that person has a gap that looks filled on paper and is empty in practice. "Legal is Consulted" describes a relationship, not a trigger — it doesn't say who picks up the phone, by when, through which channel, with a receipt confirming it landed. The fix isn't a longer RACI; it's naming, for every Accountable-Consulted pair that has a clock running against it, the specific Responsible party and channel for the handoff itself, not just the eventual decision.

6.4 The escalation-authority path, end to end

[SENIOR MANAGER] The decision tree below shows how the placement decision (§1–§3), the charter's enforcement rung (§4), and the RACI (§6.2) combine into one path a SOC manager actually walks when a critical finding's SLA is at risk of breach. It is a structural model of the decision, not a capture of any single organization's actual workflow tool.

flowchart TD

```
A["Critical finding opened,\nSLA clock starts"] --> B{"Owning team acts\nwithin SLA?"}
B -->|Yes| C["Closed within SLA"]
B -->|No| D["Charter names an\nautomatic escalation role\nfor this finding type?"]
D -->|No named role| E["De facto rung 2:\nmandate with no teeth --\nfinding stalls, no automatic\nnext step"]
D -->|Named role exists| F{"Is the named role\ninside the same reporting\nchain as the blocking\nteam?"}
F -->|Yes| G["Escalation is structurally\nweak regardless of wording --\nsee Part 4 Section 1.2"]
F -->|No| H["Escalation reaches an\nindependent authority"]
H --> I{"Cross-team activity\ninvolved (legal, privacy,\nphysical security)?"}
I -->|Yes| J["RACI Responsible party\nfor handoff notifies the\nAccountable party directly,\nwith\na confirmed receipt"]
I -->|No| K["Independent authority\nmandates a deadline with\nconsequence"]
J --> K
```

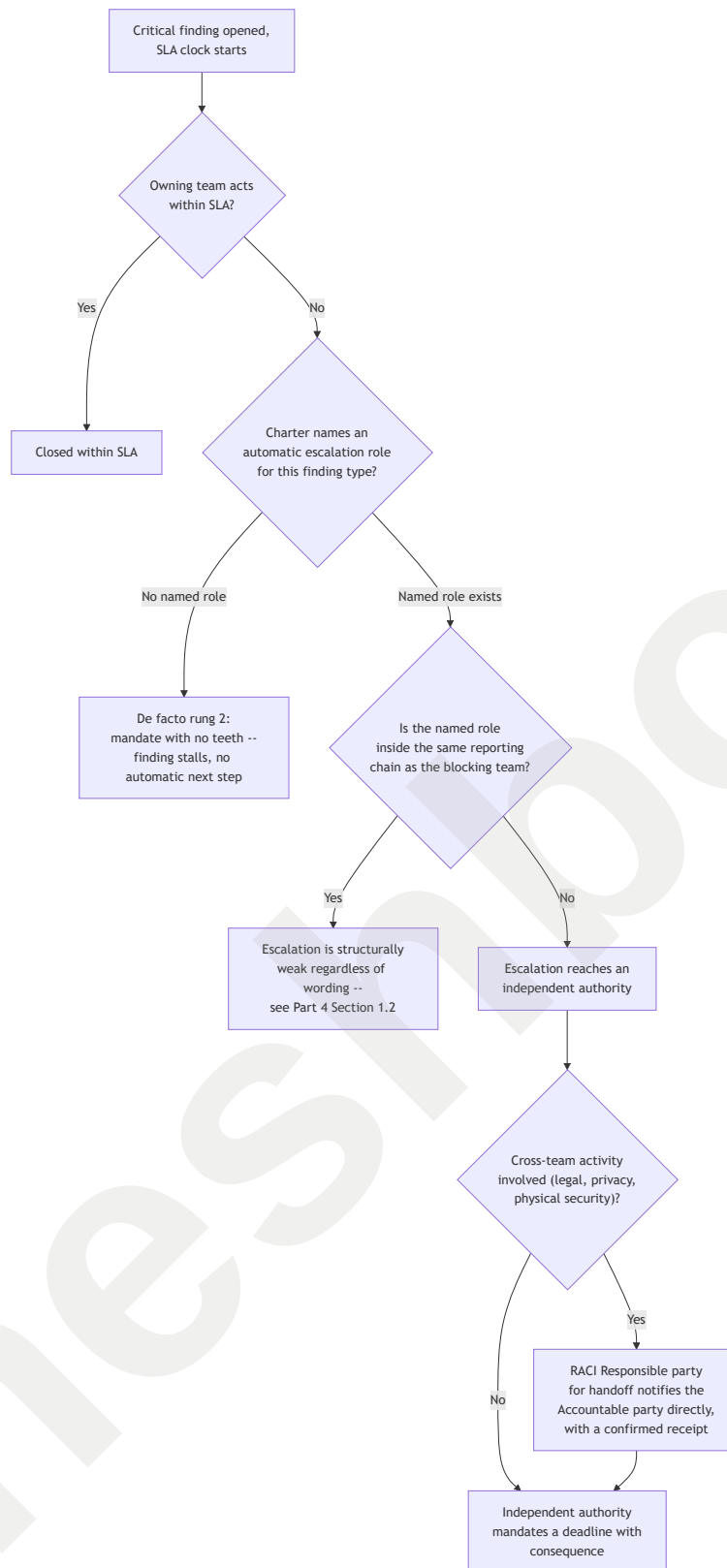


Figure 4.1 — The escalation-authority path, end to end. *CONCEPTUAL*. Shows the escalation path a critical finding follows once its SLA is at risk, tracing how placement (§1–§3), charter enforcement rung (§4), and RACI handoff (§6.2–§6.3) combine into a single walkable decision. It supports the claim, made across §4 and §6, that authority is a function of placement and named handoffs together, not of charter wording alone. Diagram ID **FIG-0401**.

Cross-Book Pointer *This part stops at the charter-level question of who has standing authority over a finding category and who must be notified when a handoff is required — it does not cover how to write a good escalation for one specific ticket once that path is already in motion. See SOC Playbook Handbook, Part 27 — Escalation Quality for the ticket-level mechanics (what a hand-off should contain, how severity is communicated) that operate inside whichever authority path this part's charter and RACI establish.*

7. Reviewing and defending the charter over time

7.1 Trigger events for revision

[SENIOR MANAGER] A charter written once and never revisited degrades the same way any other unmaintained control does. Five events should trigger an out-of-cycle charter review regardless of where the organization sits in its normal annual cadence: a reorganization that changes the SOC's reporting line or the reporting line of a team named in the RACI; a merger or acquisition that adds environments, business units, or entire IT organizations the charter never accounted for; a change of CISO, CIO, or other executive the escalation clauses name by role; any missed SLA that reveals an escalation clause didn't function as written (the §2 Management Autopsy is exactly this trigger, after the fact); and a new regulatory obligation that changes a notification timeline the RACI's Legal/Privacy rows assume.

7.2 Who audits the charter itself

[EXECUTIVE] The charter's own credibility depends on someone other than the SOC manager periodically checking whether it still matches reality — internal audit, the CISO if the SOC doesn't report there directly, or a peer SOC manager at review time are all reasonable owners of that check, but "the SOC manager reviews their own charter" produces the same blind spot as §1.2 in miniature: the person most invested in the charter looking sufficient is the person checking whether it is. A charter review that only asks "does this document still describe what we do" misses the actual failure mode; the review that matters asks "the last time an SLA was missed or a cross-team handoff failed, did this document's escalation and RACI clauses actually produce the outcome they promise, or did the real outcome depend on who happened to be reachable that day." The second question is the one worth answering, in writing, at every review — and it's the same question the board-reporting narrative in Part 24 — Executive & Board Reporting eventually needs an honest answer to, since "we have a charter" and "our charter works under pressure" are different claims a skeptical board member is entitled to tell apart.

Cross-references

This part assumes the operating-model groundwork from Part 2 — SOC Operating Models: In-House, MSSP, Co-Managed, Hybrid and the series-scope framing from Part 1 — SOC Manager Foundations & the Series Map. It hands its charter and RACI forward to Part 26 — Cross-Team Politics & Stakeholder Alignment (full politics treatment) and Part 27 — Legal, HR & Compliance Interfaces (procedural mechanics once a legal- or HR-triggering finding is confirmed), and its enforcement-rung language

forward to Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty and Part 24 — Executive & Board Reporting. Outside this book, it points to SOC Playbook Handbook, Part 27 — Escalation Quality for ticket-level hand-off mechanics that operate inside whatever authority path this part establishes.

neshbooy

Section B — Staffing, Shift Design & Workforce Planning

neshbooy

Part 5 — Headcount & Capacity Modeling

Why this part exists

[CONCEPT] Every SOC manager eventually has to answer one question in a specific, defensible number: how many analysts does this team need. Not "more than we have," not "roughly a dozen," but a number with a formula behind it that survives a CFO asking where it came from and survives a bad Tuesday six months later. This part builds that number from four inputs — alert volume, handle time, shrinkage, and target coverage hours — and walks through the arithmetic, the deductions naive models skip, and the ways even a careful model still lies to you on the day it matters most.

This part does not define what "MTTR" or "alert volume" mean, and it does not tell you how to instrument your ticketing system to produce them reliably. That is SOC Playbook Handbook, Part 32 — Metrics, and this part treats those definitions as a fixed input rather than re-deriving them. "Handle time" is not a Playbook Handbook term — Part 32 doesn't define it — so this part owns that definition itself; §2 gives it and says exactly what shape all three inputs need to arrive in. It also does not design the actual shift pattern a headcount number gets scheduled into (fixed, rotating, follow-the-sun — that's Part 6), does not build the CFO-facing budget narrative around the resulting dollar figure (Part 20), and does not tell you how to sanity-check your ratio against a published industry survey (Part 31, which is mostly a warning about doing that carelessly). What's left, and what this part actually owns, is the specific manager's job of turning four numbers into a headcount, and being honest about the three ways that headcount can still be wrong on the day it's tested.

1. What a headcount number has to survive

[CONCEPT] A headcount model has exactly one real test: does the team it describes still function on a day that isn't average. A model that produces a defensible-looking number for a typical Tuesday and collapses the first time volume spikes or two people call in sick on the same shift hasn't modeled capacity — it has modeled a best case and labeled it a plan. Everything in this part is built around treating that test as the actual bar, not the budget-approval meeting as the actual bar.

[SENIOR MANAGER] The number also has three separate audiences who will each stress it differently. The budget owner wants to know the number is the smallest defensible number, not a padded one — every unfilled seat above the real requirement is a line item someone eventually questions. The analysts living inside the number want to know it isn't quietly assuming they'll absorb the gap between the model and reality through unpaid overtime — Part 17 covers what happens when they do that for eighteen months straight. And the manager's own future self, six months into the fiscal year when a new detection doubles the volume on one alert type, wants a model built with enough structure to be re-run quickly rather than reconstructed from scratch under deadline pressure. A number that satisfies only the first audience is a number built to get approved, not a number built to work.

2. Inputs this part takes as already defined

[CONCEPT] Two of this part's four inputs — alert volume and handle time — come from the organization's instrumentation, and this part's job starts only once those numbers exist in a usable shape. **Handle time**, as this part uses it, is this book's own operational definition rather than a Playbook Handbook term: the elapsed working time an analyst spends actively triaging, investigating, and disposing of a single ticket, excluding any time the ticket simply sits untouched in a queue before someone picks it up — that's backlog age, a related but different measurement Part 9 §1 owns. A third input, target coverage hours, is also taken as a given here, but from a different source: a business or risk-tolerance decision about how much of the day the SOC commits to covering (§3.3 works through what that decision costs), not a number any metrics instrumentation produces. The fourth, shrinkage, is this part's own to build, and §4 covers it in full.

Cross-Book Pointer *This part does not define MTTR or alert volume, and it does not cover the instrumentation needed to produce them consistently across queues — that's SOC Playbook Handbook, Part 32 — Metrics. "Handle time" is this part's own definition instead (§2), not a Playbook Part 32 term. Read Part 32 first for MTTR and alert volume if your organization doesn't already have a shared, audited definition that every queue reports the same way, and make sure your organization's own "handle time" definition is applied just as consistently; every formula in this part silently assumes it is, and a headcount model built on two different definitions of handle time from two different tools will produce a number that's wrong in a way no amount of careful arithmetic here can fix.*

[SENIOR MANAGER] What this part needs from those definitions is a specific shape, not just a number. A single annual average handle time and a single annual average alert volume are close to useless for the model built in §3 through §6 — the whole argument of §7 is that the average is exactly what hides the failure. What the model actually needs is volume and handle time broken out by time block (business hours vs. overnight, weekday vs. weekend) and, ideally, by severity or queue, plus the distribution of daily volume across a full year, not just its mean. If your metrics program can produce a mean but not a distribution, that's worth flagging back to whoever owns your alert-volume instrumentation (SOC Playbook Part 32) or your handle-time tracking (this part's own definition, §2) — a headcount model is one of the clearest business cases for why a distribution matters more than an average.

[CONCEPT] One more input decision happens upstream of this part entirely: how much of the raw alert volume a human analyst ever sees in the first place. Auto-closing, auto-enrichment, and SOAR-driven auto-remediation can filter a large share of raw volume before it ever reaches a queue an analyst works — that filtering ratio is a direct multiplier on every calculation in §3, and it's a decision this book doesn't own. SOC Playbook Handbook, Part 30 — Automation and SOAR covers what should and shouldn't be automated versus gated for human review; this part assumes that decision has already been made and takes whatever volume survives it as the "volume" input to the model below.

3. From volume and handle time to raw analyst-hours

[CONCEPT]

This section turns two of §2's inputs — volume and handle time — into a raw analyst-hours requirement, before shrinkage (§4) or coverage-hours reconciliation (§5) touch the number.

3.1 The core formula

[SENIOR MANAGER]

The starting arithmetic is not complicated, which is exactly why it's easy to get wrong by skipping the steps around it rather than the multiplication itself:

CONCEPTUAL SAMPLE – illustrative formula and figures, not sourced benchmark data.

Raw analyst-hours needed (per time block)
= Alert volume (per time block) × Average handle time (per time block)

Concurrent analysts needed (per time block)
= Raw analyst-hours needed (per time block) ÷ Length of the time block

Total seat-hours needed (per year)
= Sum, across every time block in a year, of
(Concurrent analysts needed, rounded up) × (Length of the time block)

The rounding-up step is not cosmetic — a computed concurrency of 8.75 analysts is a real requirement for nine actual people; the fractional analyst doesn't exist, and rounding down to save one headcount is where an otherwise sound model quietly turns into an understaffed one.

3.2 What counts as "volume" for this calculation

[CONCEPT]

The volume this formula wants is post-automation, post-deduplication ticket volume that actually lands on a human analyst's queue — not raw log events, not raw SIEM alerts before correlation, and not a count that includes tickets a SOAR playbook closed without human involvement. Confusing "alerts generated" with "alerts a person had to work" is one of the most common ways a headcount model comes out wrong in either direction: too high if it counts volume the automation already absorbed, too low if the automation layer is newer or less reliable than the model assumes and a chunk of "auto-closed" volume is quietly coming back to the queue as re-opened tickets.

3.3 Coverage hours: what continuous coverage costs before anyone touches a ticket

[SENIOR MANAGER]

Target coverage hours set the denominator in the concurrency calculation, independent of volume. A SOC covering business hours only (roughly 50 hours a week) needs coverage across a much smaller window than one covering 24/7/365 (168 hours a week, 8,760 hours a year). The table below gives the coverage-hours floor for common coverage targets — the minimum number of concurrent seats required purely to keep the target window covered, before any volume-driven adjustment from §3.1 is added on top. This is a floor, not a headcount: it tells you the minimum shape of the schedule, and §5–§6 layer volume back on top of it to get the real requirement.

The table below supports the staffing-model decision in §5 and §6 — use it to see the coverage-hours floor before adding volume-driven concurrency on top.

Coverage Target	Weekly Hours Covered	Coverage-Floor Seats (before volume)	Typical Driver
Business hours only (5×10)	50	1	Low after-hours risk tolerance; early-stage or very small SOC
Extended hours (16 hours × 5 days)	80	1 per shift, 2 shifts	Two-shift model, no true overnight coverage
24/7, single-seat minimum	168	1 per shift, 3 shifts	Baseline always-on coverage, no redundancy built in
24/7, two-person overnight minimum	168	2 overnight, more during the day	Escalation or safety redundancy requirement — see §7.1
Follow-the-sun (regional handoff)	168	Varies by region	No shift anywhere runs the true low-staffing overnight window; model and tradeoffs owned by Part 2 and Part 6

[SENIOR MANAGER] Notice that the coverage floor and the volume-driven requirement are two separate numbers, and the larger of the two is what actually has to be staffed. A SOC with a strict 24/7 mandate but very low overnight volume is bound by the coverage floor overnight (someone has to be there even if nothing happens) and by volume-driven concurrency during the day (§3.1's math takes over once volume, not the clock, is the binding constraint). §6 works a full example where the two constraints bind at different times of day, which is exactly why a single flat number for "how many people do we need" is the wrong shape of question.

4. Shrinkage: turning nominal hours into real ones

[HR/PEOPLE] This section builds the fourth input named in this part's opening paragraph — the one this part owns outright rather than importing from SOC Playbook Handbook, Part 32 — Metrics — and it's the deduction a naive headcount model skips most often.

4.1 Building a shrinkage stack

[HR/PEOPLE] A full-time analyst nominally works 2,080 hours a year (40 hours × 52 weeks). Almost none of those hours are actually available for ticket work, and the gap between the nominal number and the real one is shrinkage — every hour an analyst is paid for but isn't sitting in the queue: PTO, sick leave, company holidays, training, coaching and 1:1 time, QA calibration sessions, team meetings and general admin overhead, paid breaks, and a fleet-wide deduction for the fact that a new hire isn't producing at full rate for their first several months. A model that skips this deduction and staffs against the full 2,080 hours is staffing against a number no real analyst ever delivers.

The table below is a worked shrinkage stack (CONCEPTUAL SAMPLE) showing how a roughly 32% shrinkage figure gets built from named categories rather than asserted as a round number — use it as a starting structure for your own team's actual categories and hours, not as a universal constant.

Category	Hours/Year (typical)	% of 2,080 Nominal Hours
PTO & vacation	120	5.8%
Sick leave	40	1.9%
Company holidays	64	3.1%
Training & certification	80	3.8%
Coaching, 1:1s, QA calibration	60	2.9%
Team meetings & admin overhead	70	3.4%
Paid breaks & shift-change overlap	100	4.8%
Fleet-wide new-hire ramp deduction	130	6.3%
Total shrinkage	664	~32%

[HR/PEOPLE] Two of these categories are easy to underweight. Fleet-wide ramp deduction is not a per-new-hire number you apply only when someone starts — it's an average drag across the whole team's tenure mix, because at any given moment some fraction of the roster is still climbing the onboarding curve Part 11 lays out, and a team with above-average attrition carries a permanently higher version of this line than a stable one. Paid breaks and shift-change overlap also get skipped in models built by someone who's never actually run a 24/7 shift schedule — a clean handoff between shifts costs real overlapping minutes, and pretending shift boundaries are instantaneous understates the hours the schedule actually consumes.

Operational Reality *The shrinkage line a headcount model treats as fixed is usually the first thing a team lead quietly cuts when the queue is running hot. Training blocks get cancelled, 1:1s get pushed to "if there's time," and QA calibration gets skipped for a month, all in the name of getting through today's volume. The model still assumes the analyst has roughly 1,414 productive hours this year (2,080 at a 32% shrinkage deduction); in practice, a team that's quietly cutting its shrinkage line this way commonly ends up with something closer to an estimated 1,600 hours of actual output, absorbed as unpaid overtime and skipped development time nobody tracks anywhere. A headcount number built on a shrinkage assumption nobody is actually protecting isn't a real number — it's a debt the team is paying down invisibly, and Part 17 covers what that debt costs once it compounds.*

4.2 A reasonable shrinkage range, and why it's a starting point, not a target

[SENIOR MANAGER] A shrinkage figure in the high-20s to mid-30s percent range is a reasonable starting assumption for a mature SOC with a normal PTO policy, a real training budget, and moderate attrition — this is a heuristic drawn from the pattern above, not a sourced industry benchmark, and it should be validated against your own team's actual time-tracking data within the first two quarters of using it. A team with unusually generous PTO, a heavier training mandate (certification-heavy roles, a new platform migration), or above-average attrition will run higher than that; a very stable, low-turnover team with minimal formal training investment can run somewhat lower. Applying an unexamined 32% to every team in every organization is the same error as applying an unexamined blended handle time

across every severity — a convenient single number standing in for something that actually varies by team.

5. Coverage models and the concurrency they actually require

[SENIOR MANAGER] §3.3's coverage-hours floor and §3.1's volume-driven concurrency have to be reconciled hour by hour, or at minimum time-block by time block, not averaged into one daily figure — averaging across time blocks is exactly the error §7.3 names as the tyranny of the average, just introduced one section early here because it's where it first becomes visible in the math. A SOC that receives most of its volume during business hours and comparatively little overnight needs more concurrent seats during the day than the clock alone would suggest, and fewer overnight than a flat 24-hour average would suggest — and a model that doesn't split the day into blocks before dividing will consistently get both numbers wrong in the same calculation.

[CONCEPT] This is deliberately not the same question as which shift pattern delivers that concurrency — fixed 8-hour shifts, 12-hour shifts with overlap, or a follow-the-sun handoff across time zones are all different ways of staffing the same concurrency requirement, and Part 6 owns choosing among them. What this section and §6 own is getting the concurrency requirement itself right before anyone starts drawing a shift calendar around it.

6. Worked example: the flat model versus the segmented model

[SENIOR MANAGER] **CASE-0501** below is a composite, illustrative regional-bank SOC — no real organization's actual figures — built to show a specific, common failure: a manager who computes headcount from a single daily average gets a number that is simultaneously more expensive and worse-distributed than one built from the same underlying data split into time blocks.

COMPOSITE CASE EXAMPLE (**CASE-0501**) — illustrative organization and figures, constructed to demonstrate the flat-vs.-segmented modeling error; not drawn from a single traceable real SOC.

The environment: A 24/7 SOC with a mandatory always-on coverage requirement, average alert volume of 600 tickets per day reaching human analysts (post-automation, per §3.2), and a blended average handle time of 15 minutes across all severities.

The flat model. A manager takes the daily average at face value: $600 \text{ tickets} \times 15 \text{ minutes} = 9,000 \text{ minutes}$, or 150 hours of analyst-hours needed per day. Spread evenly across 24 hours, that's $150 \div 24 = 6.25$ concurrent analysts, rounded up to 7 to be safe. Staffing 7 concurrent seats around the clock costs $7 \times 8,760 = 61,320$ seat-hours a year. At 1,414 productive hours per FTE after a 32% shrinkage deduction, that's $61,320 \div 1,414 \approx 43.4$, rounded up to 44 FTE. The flat model's answer: 44 analysts, staffed evenly around the clock.

The segmented model. The same organization's actual ticket data shows volume isn't flat — roughly 70% of the day's 600 tickets arrive during a 12-hour business-hours window, and the remaining 30% arrive during the 12-hour overnight window. Business hours: $420 \text{ tickets} \times 15 \text{ minutes} = 6,300 \text{ minutes}$ (105 hours) across 12 hours, or $105 \div 12 \approx 8.75$ concurrent analysts, rounded up to 9. Overnight: $180 \text{ tickets} \times 15 \text{ minutes} = 2,700 \text{ minutes}$ (45 hours) across 12 hours, or $45 \div 12 = 3.75$ concurrent analysts,

rounded up to 4. Seat-hours per day: $(9 \times 12) + (4 \times 12) = 156$, or 56,940 a year. Headcount: $56,940 \div 1,414 \approx 40.3$, rounded up to 41 FTE. The segmented model's answer: 41 analysts, staffed 9-deep during the day and 4-deep overnight.

[SENIOR MANAGER] The segmented model needs 3 fewer total FTE than the flat model (41 vs. 44) while simultaneously fixing a coverage problem the flat model created: the flat model's 7 concurrent seats overstaff the overnight window (7 scheduled against a real need of 4, three idle seats every night) and understaff the business-hours window at the exact moment 70% of the day's volume and the highest-severity escalations are actually arriving (7 scheduled against a real need of 9, two seats short during the hours that matter most). A model that only looks right in its daily-average total can be wrong in both directions at once, on the same day, and still balance on paper. The math behind both versions of this case is the same math built into the headcount calculator worksheet ([TMPL-0001](#)) in Appendix A1 — run your own volume and handle-time data through that worksheet rather than rebuilding these formulas from scratch each budget cycle.

7. The honest failure modes: queue-of-one, burst traffic, and the terrible day the average hides

[CONCEPT] §6's segmented model is a real improvement over a flat average, and it is still not the whole picture. Three specific failure modes survive even a carefully segmented, shrinkage-adjusted model, and a manager who doesn't name them explicitly will discover them the hard way, on a day the model didn't plan for.

7.1 Queue-of-one variance

[FRONTLINE MANAGER] A shift staffed by exactly one analyst has, structurally, zero reserve capacity — not "low" reserve capacity, zero. Every other failure mode in this section is a matter of degree; this one is binary. If that analyst is out sick, the shift is uncovered unless a named backup path exists and is actually reachable, not just written down. If volume on that shift runs even modestly above normal, there is no second person on the same shift to absorb the difference — the choice narrows to working faster (which has a real ceiling, per §7.2), letting the queue age past target response time, or both.

Blind Spot A queue-of-one shift design looks fully staffed on the schedule right up until the day it isn't. The gap is invisible in every normal-volume week, because a single analyst comfortably covers a single analyst's worth of expected volume, and there's no dashboard metric that shows "this shift has zero elasticity" — the metric that would show it, capacity held in reserve, is exactly zero, and zero doesn't trigger an alert on its own. The first time anyone sees the gap is the day it's tested: an unplanned absence, or a volume spike, with no second person on that shift to absorb either one.

[FRONTLINE MANAGER] The fix is not automatically "always staff at least two people per shift" — that's a real cost increase, and a small SOC covering low-volume off-hours may not be able to justify a second full seat overnight. The fix is knowing, explicitly and before the fact, whether a given shift's design has this property, and deciding on purpose whether the organization accepts that risk or funds a named

backup path (an on-call bridge to a second analyst, an MSSP overflow arrangement per this book's own Part 22 — MSSP & Managed-Service Contract Management, or a shift-lead who can step in). Accepting the risk silently, by never asking the question, is the actual failure — not the risk itself.

7.2 Burst traffic

[SENIOR MANAGER] CASE-0502 (COMPOSITE CASE EXAMPLE) illustrates the second failure mode: a small SOC with a queue-of-one day shift and a normal average volume that is already running close to its ceiling before any burst hits at all.

The setup: A small SOC covers three shifts with one analyst on each, for a baseline of three concurrent seats. Average volume is 80 tickets a day across all three shifts, or roughly 27 tickets per shift on average, at a 15-minute average handle time. An 8-hour shift has 480 available minutes; at 15 minutes a ticket with zero breaks or admin time, the theoretical ceiling is $480 \div 15 = 32$ tickets. A more realistic sustainable pace — leaving room for the shrinkage categories in §4.1 that still apply inside a single shift — puts working capacity closer to 24 tickets. The "normal" day, at 27 tickets against a realistic 24-ticket sustainable pace, is already running hot before anything unusual happens.

The burst: A mass phishing campaign hits during the day shift, pushing that shift's volume to four times normal: 108 tickets in the same 8-hour window. At best-case 15 minutes a ticket with zero slack, that shift needs $108 \times 15 = 1,620$ minutes of work — more than three full shifts' worth — compressed into 480 available minutes. The single analyst on that shift can process at most 32 tickets in the window even working flat out with no breaks; the other 76 tickets, roughly 70% of the day's volume, do not get touched by end of shift regardless of individual skill or effort. That backlog ages past whatever response-time target the organization has committed to, and it does so structurally — no amount of coaching or urgency changes the arithmetic of one person and 480 minutes.

[SENIOR MANAGER] A mean-based headcount model, even a well-segmented one, sizes concurrency against a typical day and has nothing to say about this day. The practical fix is sizing standing concurrency against a high-percentile day — something like the 90th percentile of the actual daily volume distribution, not the mean — and keeping a surge lever for the tail beyond even that: a documented MSSP overflow arrangement (this book's own Part 22, or the general co-managed model from Part 2), pre-approved mandatory overtime, or a cross-trained pool from an adjacent team that can flex in on short notice. None of those levers work if they're invented during the incident; they have to exist, tested, before the burst day arrives.

7.3 The tyranny of the average, generalized

[CONCEPT] §6 and §7.2 are both instances of the same underlying problem: an average is a single number standing in for a distribution, and a distribution has a shape the average doesn't preserve. A headcount model built entirely on averages — average daily volume, average handle time, average shrinkage — will look correct on the specific day that happens to sit at the average, and wrong on every day that doesn't, which by definition is most days in any distribution with real variance. The practical discipline is cheap relative to the risk it removes: pull the actual daily (or even hourly) volume distribution before finalizing a model, not just its mean, and check where the model's assumptions sit relative to the 90th or 95th percentile day, not just the 50th.

The table below (CONCEPTUAL SAMPLE) summarizes the failure modes this section and §6 have covered, plus two related ones this part's math doesn't automatically catch, as a working checklist for reviewing any headcount model before it ships.

Failure Mode	What It Looks Like	Why a Mean-Based Model Misses It	The Practical Fix
Queue-of-one variance	1 analyst per shift; any absence or spike means zero slack	Averages assume a pool with reserve capacity; a single-analyst pool has none	Named backup path or a 2nd seat on any shift that can't tolerate a bad day
Burst traffic (day-level)	A single day or week runs at 3 to 4 times normal volume	Annual or monthly averages smear the spike across the quiet days around it	Size concurrency to a high-percentile day (e.g., P90), plus a tested surge/overflow lever
Diurnal/intraday variance	Volume concentrated in business hours; flat 24/7 staffing over- and understaffs at once	A single daily figure hides which hours actually carry the volume	Segment concurrency by time block, not just by day (§5–§6)
Severity-mix drift	Blended handle time creeps as the ratio of quick vs. complex tickets shifts	A single blended number assumes a mix that may not hold quarter to quarter	Track handle time by severity band; re-run the model when the mix shifts materially
Shrinkage assumption gone stale	Real productive hours run below the modeled figure	Shrinkage is a snapshot, not a standing guarantee (§4.2)	Audit actual shrinkage against the model's assumption at least twice a year
Attrition/backfill lag	Approved headcount doesn't match heads actually on the floor	The model treats headcount as a step function; hiring and ramp aren't instantaneous	Model backfill lag explicitly (Part 18), not as a rounding error

What Would Change My Mind This part recommends sizing concurrency against a high-percentile volume day, not the mean, because the gap between the 50th and 90th percentile day is usually wide enough to matter operationally. If a queue's day-to-day volume were stable enough that its P50 and P90 sat within a few percent of each other consistently across a full year — a genuinely low-variance queue, not a lucky quiet quarter — the case for a percentile buffer would mostly disappear, and a flat mean-based model would be defensible for that specific queue without the correction this section argues for.

8. Keeping the model alive after budget season

[SENIOR MANAGER] A headcount model that gets built once, during annual budget planning, and never touched again is a snapshot pretending to be a policy. Alert volume shifts every time a new detection ships or an acquisition adds a new log source; handle time shifts as tooling changes or the severity mix drifts; shrinkage shifts with attrition and PTO-policy changes; and the coverage-hours requirement itself can shift if the business takes on a new regulatory or contractual obligation for faster response. Treat the model built in §3 through §6 as a living worksheet, reviewed on a fixed cadence — quarterly is a reasonable default — against actual volume, actual handle time, and actual shrinkage, not a number defended once at budget time and then defended again from memory a year later.

flowchart TD

A["Segment volume & handle time\nby time block and severity (S2-S4)"] --> B["Compute concurrency needed\nper time block (S3, S5)"]

B --> C["Apply coverage-hours target:\nbusiness hours / extended / 24x7 (S3.3)"]

C --> D["Sum to raw seat-hours per year"]

D --> E["Apply shrinkage % ->\nproductive hours per FTE (S4)"]

E --> F["Draft headcount number"]

F --> G{"Stress-test against a P90\nvolume day and a single-\nabsence scenario (S7)"}

G -->|Holds| H["Defend the number to the\nbudget owner (Part 20)"]

G -->|Breaks| I["Add buffer seats or a\nsurge/overflow lever\n(Part 6, Part 22)"]

H --> J["Re-run quarterly against\nactual volume & shrinkage (S8)"]

I --> J

J -->|drift detected| A

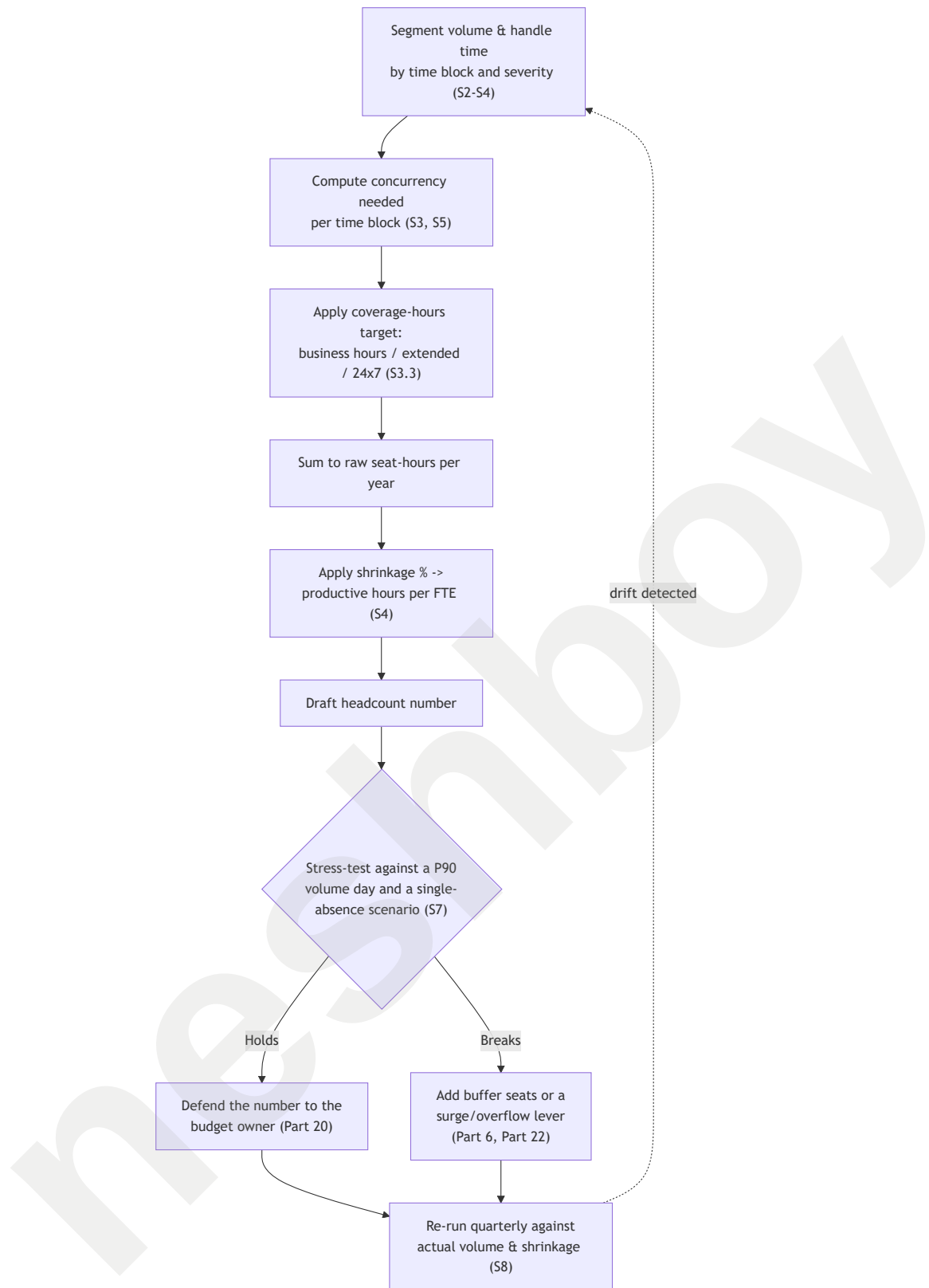


Figure 5.1 — Building and re-running a headcount model as a standing loop. *CONCEPTUAL.* Shows the headcount model as a closed loop from segmented inputs through a stress test to a quarterly re-run rather than a single output. Supports this section's claim that a headcount model is a maintained artifact, not a once-a-year deliverable, and the stress-test branch supports the failure modes named in §7. Diagram ID **FIG-0501**.

Operational Reality *An approved headcount of 41 analysts — this part's own §6 worked example (CASE-0501) — is a target, not a headcount you actually have on the floor every day. Real time-to-fill for a mid-level SOC analyst role commonly runs 60 to 90 days once a requisition is approved, and that's before a new hire is anywhere near full productivity — Part 11 covers the ramp curve that follows. After one resignation, the team operates at 40 during the search, then at an effective 39 or 40 for another two to three months while the replacement ramps. It only returns to a true 41 if nobody else leaves in the meantime. The model's buffer exists on a spreadsheet; the queue experiences the gap in real time, which is exactly the case Part 18 and Part 25 pick up when headcount is fixed and the queue keeps growing anyway.*

[EXECUTIVE] When the model changes materially between review cycles — a 10% swing in required headcount either direction — that change belongs in the next board or CISO reporting cycle, not held until the next annual budget conversation. Part 24 owns how to frame that change as a risk narrative rather than a raw number; this part's only job is making sure the underlying number is worth reporting in the first place.

9. Where this goes next

[CONCEPT] The headcount number built here is an input, not a finished product. Part 6 takes it and builds the actual shift calendar — fixed, rotating, or follow-the-sun — that delivers the concurrency this part computed. Part 9 uses the resulting queue as an ongoing health signal and teaches how to tell a genuine staffing shortfall from a detection-quality problem wearing a staffing shortfall's clothing. Part 20 turns the number into dollars a CFO will engage with, and Part 25 covers the judgment call this part can't automate: what a manager actually does when the number the model says is required and the number the budget actually approves don't match.

Cross-references. This book: Part 2 (operating models — which portion of the SOC this headcount number is even counting), Part 6 (shift pattern and coverage design, consuming this part's concurrency requirement), Part 9 (queue health as an ongoing signal distinct from this part's point-in-time model), Part 11 (onboarding ramp curve behind the shrinkage stack in §4), Part 17 (what absorbing a broken shrinkage assumption costs over time), Part 18 (attrition and backfill lag), Part 20 (budget defense), Part 22 (MSSP overflow as a burst-traffic lever), Part 24 (reporting a material model change), Part 25 (fixed headcount vs. a growing queue), Part 31 (benchmarking caution), and Appendix A1's headcount calculator worksheet (TMPL-0001). Other volumes: SOC Playbook Handbook, Part 30 — Automation and SOAR (what volume ever reaches a human) and Part 32 — Metrics (the MTTR and alert-volume definitions this part treats as a fixed input; handle time is this part's own definition, §2, not a Playbook Handbook term); Detection Engineering Handbook V2's detection-quality material, reached through this book's own Part 9 rather than cited directly here.

Part 6 — Shift Pattern & Coverage Design

Why this part exists

[CONCEPT] Part 5 gives a SOC manager a number — say, 14 analysts, after shrinkage — but a headcount number is not a schedule. Fourteen people don't cover a queue by themselves; they cover it only once someone decides how those fourteen bodies are arranged across 8,760 hours a year, who works nights, how many people are awake at 3 a.m. on a Sunday, and what happens in the ninety seconds between the analyst who worked the last eight hours and the one who's about to work the next eight. That arrangement — the shift pattern — is a distinct decision from the headcount math in Part 5, and it has its own failure modes that a correct headcount number does nothing to prevent. A SOC can be fully staffed on paper and still run an unsafe 3 a.m. shift, because the schedule that got built from the right headcount number was still a bad schedule.

This part covers the four coverage models a SOC manager actually chooses between — fixed shifts, rotating shifts, follow-the-sun, and on-call/bridge coverage — plus the two staffing problems every model eventually has to solve: holidays and weekends. It also owns shift-handoff mechanics: what information has to move from the outgoing analyst to the incoming one for the queue to survive the transition without a gap. That is deliberately not the same question as what makes one escalation from Tier 1 to Tier 2 good — SOC Playbook Handbook, Part 27 — Escalation Quality owns the mechanics of a single hand-off between tiers on one ticket. This part owns the mechanics of an entire shift's worth of state moving between two people at a fixed clock boundary, whether or not any single ticket ever gets escalated. A team can run flawless escalations all day and still lose a watch item at 6 a.m. because nobody wrote it down before going home.

This part also previews, but does not develop, the fatigue and circadian research behind why some rotation directions and shift lengths cost more in error rate than others — that's Part 17's job, and this part cites the conclusion where a scheduling decision depends on it rather than re-deriving the physiology here.

1. From headcount to a calendar: what coverage design actually decides

[CONCEPT] A year has 8,760 hours. A SOC that commits to 24/7 coverage has committed to having at least one analyst awake and watching the queue for all 8,760 of them, every year, including the ones nobody wants — December 25 at 3 a.m., the Sunday of a holiday weekend, the night before a product launch. Coverage design is the set of decisions that turns that raw hour count into a real, staffable calendar: how long is a shift, how many shifts cover a day, how do people rotate (if at all) between day and night, and what happens at the boundary where two shifts meet.

[SENIOR MANAGER] Four coverage models cover almost every real SOC schedule in production. The table below compares them on the dimensions that actually drive a manager's choice — coverage gaps, cost, and the fatigue risk this part previews and Part 17 develops (CONCEPTUAL SAMPLE — ratings are illustrative comparisons, not measured against a specific dataset).

Table 6.1 — Coverage model comparison. (*CONCEPTUAL SAMPLE.*) Use this table to narrow a coverage decision to one or two candidate models before running the full worked comparison in §8; it does not replace that comparison, because cost and fatigue risk both depend heavily on a specific team's size and geography.

Model	Typical shift length	Analysts needed for 24/7	Coverage-gap risk	Relative cost	Fatigue risk (see Part 17)
Fixed shift, no rotation	8 hours	3 dedicated crews	Low if crews are stable	Medium — night differential pay, permanently	Low — no rotation, but night-only crew carries its own chronic risk
Rotating shift	8 or 12 hours	3–4 crews, same people rotate	Low, if the rotation is planned around holidays/PTO	Medium — same differential, spread across more people	Medium-to-high — depends heavily on rotation direction and speed, see §3
Follow-the-sun	8 hours per region	3 regional crews, no one works nights	Low across the day; highest at region-boundary handoffs	High — three regional teams, travel/timezone overhead	Low per analyst — nobody works an unnatural shift, but handoff risk rises
On-call / bridge	Business hours + on-call	1 core crew + on-call rotation	Highest — depends on page response time	Low — no standing night crew	Medium — disrupted sleep on call nights, unpredictable

[SENIOR MANAGER] None of these four is inherently correct; each is a different trade of cost against coverage-gap risk against the fatigue cost this part previews. A 12-analyst regional bank's SOC and a 200-analyst global MSSP are not choosing from the same practical menu — the bank almost certainly can't staff follow-the-sun, and the MSSP almost certainly shouldn't run pure on-call for its core detection queue. The rest of this part works through each model in enough depth to make that choice defensibly, then closes with the decision framework in §8.

2. Fixed shift models

2.1 Three eight-hour shifts

[FRONTLINE MANAGER] The most common starting model: three fixed crews, each working the same eight-hour block every workday — a day shift, an evening shift, and a night (or "graveyard") shift — with the same people on the same shift indefinitely rather than rotating through all three. Its main strength is predictability: an analyst on the night shift knows they're always on nights, plans their life around it, and (if they're suited to night work) can genuinely adapt their sleep cycle to it rather than fighting it every few days the way a rotating schedule forces. Its main weakness is recruiting and retention for the night shift specifically — most candidates would rather not work permanent nights, so a manager either pays a real premium for it or ends up with a night crew skewed toward whoever has the least seniority and the least ability to say no, which is its own retention problem addressed further in Part 18.

[SENIOR MANAGER] The arithmetic matters here more than it looks. Three eight-hour shifts times 365 days is 8,760 analyst-hours of raw coverage need per single-seat queue — but "one analyst per shift"

doesn't survive contact with PTO, sick time, and training, which is exactly the shrinkage problem Part 5 quantifies. A fixed three-shift model covering one seat per shift with zero backup needs roughly three to four people per shift line once shrinkage is applied, not three exactly — at Part 5's ~32% shrinkage deduction (1,414 productive hours per FTE), one shift line's 2,920-hour annual need already rounds up to 3 heads with zero slack for the queue-of-one variance Part 5 §7.2 names as a real risk, which is why most SOC's run a 4th on that line. This part takes Part 5's shrinkage output as a given input to the calendar and does not re-derive it (see Part 5 for the shrinkage math itself).

2.2 Two twelve-hour shifts

[FRONTLINE MANAGER] The other common fixed pattern covers the day with two twelve-hour shifts instead of three eight-hour ones — a day crew and a night crew, each working twelve hours, often on a 2-2-3 pattern (two days on, two off, three on) that yields roughly 182 working days a year against 183 off. Long stretches of consecutive days off like that are consistently one of the most-cited positives when analysts talk about schedule structure in exit and stay interviews — not a benchmark this book has validated against its own data, but a pattern common enough that Part 17 treats the 2-2-3 pattern's days-off structure as a genuine retention lever worth weighing against its longer-shift fatigue cost. It also cuts the number of shift handoffs per day from three to two, which sounds like a pure win for handoff quality — fewer transitions, fewer chances to lose a watch item — but a twelve-hour shift is a materially longer stretch of sustained vigilance on a queue, and the error-rate cost of that length is exactly the tradeoff Part 17 quantifies. This part's only job here is to name the tradeoff — fewer handoffs against longer individual shifts — and point forward to where the fatigue side of that tradeoff gets a real answer.

3. Rotating shift models

3.1 Fast rotation vs. slow rotation

[FRONTLINE MANAGER] A rotating model spreads day, evening, and night coverage across the same pool of analysts instead of assigning permanent crews, on the theory that nobody should be stuck on nights forever and everyone shares the unpopular hours. Rotations vary mainly in speed: a fast rotation changes an analyst's shift every two to three days (common in "2-2-3" or "DuPont" schedules); a slow rotation holds someone on the same shift for a full week or two before rotating them to the next block. Fast rotation feels fairer in the moment — nobody works more than a few nights in a row — but it never gives an analyst's circadian rhythm enough consecutive days to adapt, so every rotation restarts the adjustment cost from zero. Slow rotation gives the body more time to adapt to each block, at the cost of a longer stretch on the block people like least.

3.2 Rotation direction

[FRONTLINE MANAGER] Less obvious than rotation speed, and more consequential: the direction a rotation moves through the clock. A forward rotation moves day → evening → night, in the same direction the day naturally advances; a backward rotation moves night → evening → day, against it. Forward rotation is measurably easier on the body — pushing bedtime later each cycle asks less of the circadian system than pulling it earlier — a finding Part 17 develops in full with the underlying sleep-science citations. This part's job is narrower: naming rotation direction as a real, low-cost design lever a

manager controls directly when building the schedule template, not a detail to leave to whoever happens to build the spreadsheet first.

Management Autopsy — "adopt a backward-rotating panama schedule to minimize shift changes" (COMPOSITE CASE EXAMPLE, [CASE-0601](#))

The decision: A 16-analyst SOC redesigning its schedule adopted a "panama" pattern — two day shifts, two night shifts, four days off, repeating — with the rotation moving backward: day block, then straight into a night block forty-eight hours later, then a long break, then back to days.

Why it seemed reasonable: The pattern minimizes the total number of shift changes per month compared to a faster rotation, and the four-day break every cycle tested well when analysts were simply asked whether they liked more days off — they did.

How it failed: The rotation direction meant analysts went from a day-shift sleep schedule straight into a night shift roughly 24 hours later, with no gradual adjustment period. QA scores on the first night shift of each cycle ran measurably lower than the second night shift of the same cycle for three consecutive quarters — the same analysts, the same queue, the same detection content, worse outcomes purely as a function of which night of the block it was. Two analysts separately raised the first-night error pattern in 1:1s before the scheduling team connected it to rotation direction.

The fix: Keep the four-days-off structure analysts liked, but reverse the rotation direction and add a 24-hour buffer between the day block and the night block instead of transitioning directly — night, then evening, then day, with a full rest day inserted at each direction change. Total days off per month didn't change; QA variance between the first and second night of a block closed to within normal range within two cycles.

[SENIOR MANAGER] The version of this that costs nothing to fix and gets ignored anyway: rotation direction is a checkbox in whatever scheduling software builds the calendar, not a negotiation with the team. If the tool defaults to backward rotation because that's how the template was built three managers ago, nobody has to accept a worse schedule to keep the parts of the pattern the team actually likes.

4. Follow-the-sun coverage

[SENIOR MANAGER] Follow-the-sun coverage staffs three (or more) regional teams — commonly Americas, EMEA, and APAC — each working ordinary daytime business hours in its own time zone, so that as one region's day ends, the next region's day is beginning and picks up the queue. Done well, no analyst anywhere ever works a night shift; the "night coverage" is just daytime coverage somewhere else on the planet. That is the model's entire appeal, and it is real: eliminating night-shift fatigue risk across the whole organization is worth a great deal, both in error rate and in retention.

[SENIOR MANAGER]

It is also the most expensive model on Table 6.1, and the expense is not just headcount. Follow-the-sun requires three regional teams each independently competent enough to run the queue alone for eight hours a day — which means three sets of on-call backups, three sets of local management overhead, and (usually) enough duplicated tooling access and detection-content familiarity across regions that a single incident spanning a shift boundary doesn't lose context crossing it. The model also concentrates all of its handoff risk into two moments a day instead of spreading it across three or four — the two region-to-region boundaries — which raises the stakes on the handoff mechanics in §7 rather than lowering them.

COMPOSITE CASE EXAMPLE, CASE-0602 — constructed from recurring patterns seen when a mid-sized organization compares single-site and follow-the-sun coverage at the same target coverage level; no single organization is identifiable, and all figures below are illustrative. A single-site model, staffed in its US headquarters with a rotating three-shift pattern per §3, needed 15 analysts after shrinkage (per the Part 5 model) at a blended fully-loaded cost of roughly \$95,000 per analyst including a 15% night-shift differential — about \$1.43 million a year in analyst cost. A three-region follow-the-sun model, with 5 analysts in each of the Americas, EMEA, and APAC (also 15 total after shrinkage, no differential since nobody works nights), cost roughly \$90,000 per analyst in base salary with no night premium anywhere — about \$1.35 million a year, a real but modest reduction from the single-site rate — but added an estimated \$180,000 a year in regional management overhead, duplicated toolchain licensing seats, and quarterly cross-region calibration travel that the single-site model never needed. The raw analyst headcount was identical; \$1.35 million in base salary plus the \$180,000 overhead put the follow-the-sun model's true cost at roughly \$1.53 million a year — about 7% higher than the single-site model's \$1.43 million, not lower, which is the opposite of what the initial single-line "no night differential" comparison suggested before anyone added up the coordination cost.

Manager's Note *When someone pitches follow-the-sun as "cheaper because you avoid the night differential," ask them to price the regional management layer and the cross-region calibration cadence before believing the comparison. The differential is the visible cost line; the coordination overhead is the one that shows up eighteen months later as three regional teams quietly drifting into three different triage standards.*

5. On-call and bridge coverage

[FRONTLINE MANAGER]

Not every SOC needs a standing overnight crew. A SOC with genuinely low off-hours alert volume — a small environment, strong automated triage, or a business with little overnight activity to generate alerts in the first place — can run business-hours core coverage with an on-call rotation covering the rest, rather than paying for a night shift that would spend most of its time watching an empty queue. The on-call model's structure is almost always two-tiered: a primary on-call analyst who gets paged first, and a secondary who gets paged if the primary doesn't acknowledge within a defined window — typically 10 to 15 minutes, set by the manager independently of whatever severity-driven response-time target governs the actual alert (that target is SOC Playbook Handbook, Part 29 — Playbook Severity Model's territory; this part only owns how the on-call rotation itself is structured, not how fast a given severity level requires a response once someone's paged).

[SENIOR MANAGER]

A "bridge" is the standing conference line or channel that a paged analyst — or, more often, a declared major incident — pulls people onto in real time, distinct from the on-call rotation that decides who gets paged in the first place. Bridge coverage matters as a coverage-design decision because it determines who is expected to be reachable and how fast, not just for the primary on-call analyst but for the specialists (a detection engineer, a manager, sometimes legal) a bridge might need on short notice. A rotation that only accounts for the primary/secondary analyst pair and never defines who else is on the hook to answer a bridge page at 2 a.m. is an incomplete on-call design, not a complete one that simply doesn't need those people.

Manager's Note *Rotate on-call by full week, not by single night, unless the team is large enough to spread single-night on-call thin. A week-long rotation means an analyst can plan around one bad week a month; a nightly rotation means every analyst carries a phone every single night of the month, which produces the same chronic low-grade disrupted sleep as a badly designed rotating shift, just spread across more nights instead of concentrated into a block.*

[HR/PEOPLE]

On-call compensation is a retention issue as much as a scheduling one — an analyst who carries a pager for a full week and never gets paged has still lost a week of unencumbered personal time, and a rotation that doesn't compensate for that (even a modest on-call stipend, separate from the pay-per-incident that a page itself might trigger) reads as free labor extraction to the person carrying it. Part 18 covers the retention math on this in more depth; the design point here is that on-call needs its own compensation line, not an assumption that base salary already covers it.

6. Holiday and weekend staffing

[HR/PEOPLE]

Every coverage model above eventually runs into the same two staffing problems that don't map cleanly onto any of them: weekends, which repeat every seven days and can be planned into a standing rotation, and holidays, which are rarer, unevenly distributed across the year, and disproportionately unpopular to work. Weekend coverage is usually just an extension of whichever model is already running — a rotating schedule already assigns weekend shifts as part of the normal cycle, and a fixed schedule needs an explicit weekend crew built in from the start rather than treated as an afterthought once the weekday shifts are locked.

[HR/PEOPLE]

Holidays are the harder problem because unlike a weekend, a holiday is not evenly distributable across a rotation — everyone wants the same handful of days off, and someone has to work them anyway if the SOC is genuinely 24/7. Three approaches cover most real holiday rosters: volunteer-first assignment (ask for volunteers, often with a pay premium, before assigning anyone by default), seniority-based rotation (the newest hires work the least popular holidays until they've earned enough tenure to trade up, which is fair by seniority but corrosive if it's the only mechanism a new analyst ever sees), and blackout-and-lottery (block out the two or three highest-demand holidays, randomly assign who works them, and guarantee no one draws the same blackout holiday two years running). Volunteer-first with a real premium — not a token one — resolves the largest share of a holiday roster with the least resentment; the remainder, after volunteers are exhausted, is where a

manager needs a pre-published, consistent tiebreaker rule rather than an ad hoc decision made under pressure two weeks before the holiday.

Manager's Note *Publish the holiday roster before the volunteer window opens, not after — analysts need to see the actual dates and premium before they decide whether to volunteer, and a vague "we'll figure out who works the holidays closer to the date" announcement reliably produces zero volunteers and a scramble.*

7. Shift-handoff mechanics: what must transfer

[CONCEPT] Every coverage model above eventually produces the same moment: one analyst's shift ends, another's begins, and the queue does not pause for the transition. What happens in that window is a genuinely different problem from escalation quality, and it is worth stating the distinction precisely rather than gesturing at it, because the two get confused constantly in practice.

Cross-Book Pointer *This part does not cover what makes a single escalation from Tier 1 to Tier 2 good — the context a handing-off analyst needs to include on one ticket so the receiving tier doesn't have to redo the triage. That's SOC Playbook Handbook, Part 27 — Escalation Quality, and it applies whether or not a shift change is happening at the same time. This part covers a different transfer: everything an entire outgoing shift knows that the entire incoming shift needs, regardless of whether any individual ticket is escalating to anyone. A shift can have zero escalations all day and still need a complete handoff; an escalation can happen at 2 p.m. with no shift change anywhere near it. Treat them as two separate mechanisms that happen to sometimes overlap in time, not one process with two names.*

7.1 The four categories of handoff content

[FRONTLINE MANAGER] A complete shift handoff transfers four distinct kinds of information, and a handoff that only covers one or two of them is incomplete even if it feels thorough:

- **Open work in progress** — tickets or cases the outgoing shift started but didn't close, with enough context (not just a ticket number) that the incoming analyst doesn't have to re-read the entire case history to know what's already been ruled out.
- **Watch items** — things that aren't yet a ticket but that the outgoing shift is deliberately keeping an eye on: an unusual-but-not-yet-alerting pattern, a system behaving oddly, a heads-up from another team about planned activity that might generate noise. Watch items are the single most commonly dropped category in an informal handoff, because there's no ticket forcing them to be written down anywhere.
- **System and tooling status** — anything not working normally: a detection source that went quiet, a SIEM ingestion delay, a maintenance window in progress, a tool the outgoing shift had to work around all shift.

- **What to expect next** — scheduled changes, planned deployments, known events (a company all-hands, a marketing campaign going live, a pen test window) likely to generate alert volume or noise the incoming shift should recognize rather than chase as if it were novel.

Table 6.2 — Shift-handoff content checklist. Use this at the handoff point itself, not as a retrospective audit tool — its purpose is to catch a missing category in the moment, before the outgoing analyst leaves, not to grade the handoff afterward. A fuller, fillable version of this checklist (the shift-handoff checklist, [TMPL-0602](#)) lives in Appendix A1's shift-pattern templates.

Category	What transfers	Where it commonly gets dropped
Open work in progress	Case/ticket IDs, current status, what's already ruled out, next action owner	Ticket number given with no context — incoming analyst has to re-triage from scratch
Watch items	Anything being informally monitored that isn't yet a ticket	Never written down at all — lives only in the outgoing analyst's head
System/tooling status	Known outages, ingestion delays, maintenance windows, workarounds in use	Assumed "everyone already knows" because it's been broken for hours
What to expect next	Scheduled changes, planned noisy events, known upcoming activity	Told verbally to one person, not logged anywhere the whole incoming shift can see

7.2 The handoff process

[FRONTLINE MANAGER]

The mechanism that actually gets all four categories to transfer reliably is less about the content and more about sequencing: the outgoing analyst has to produce the record before the incoming analyst arrives, not narrate it from memory in the ninety seconds they physically overlap. A live-only handoff — whatever the outgoing analyst remembers to say out loud as they're leaving — loses exactly the categories that don't have a ticket forcing them to exist, which is precisely the watch-items category above.

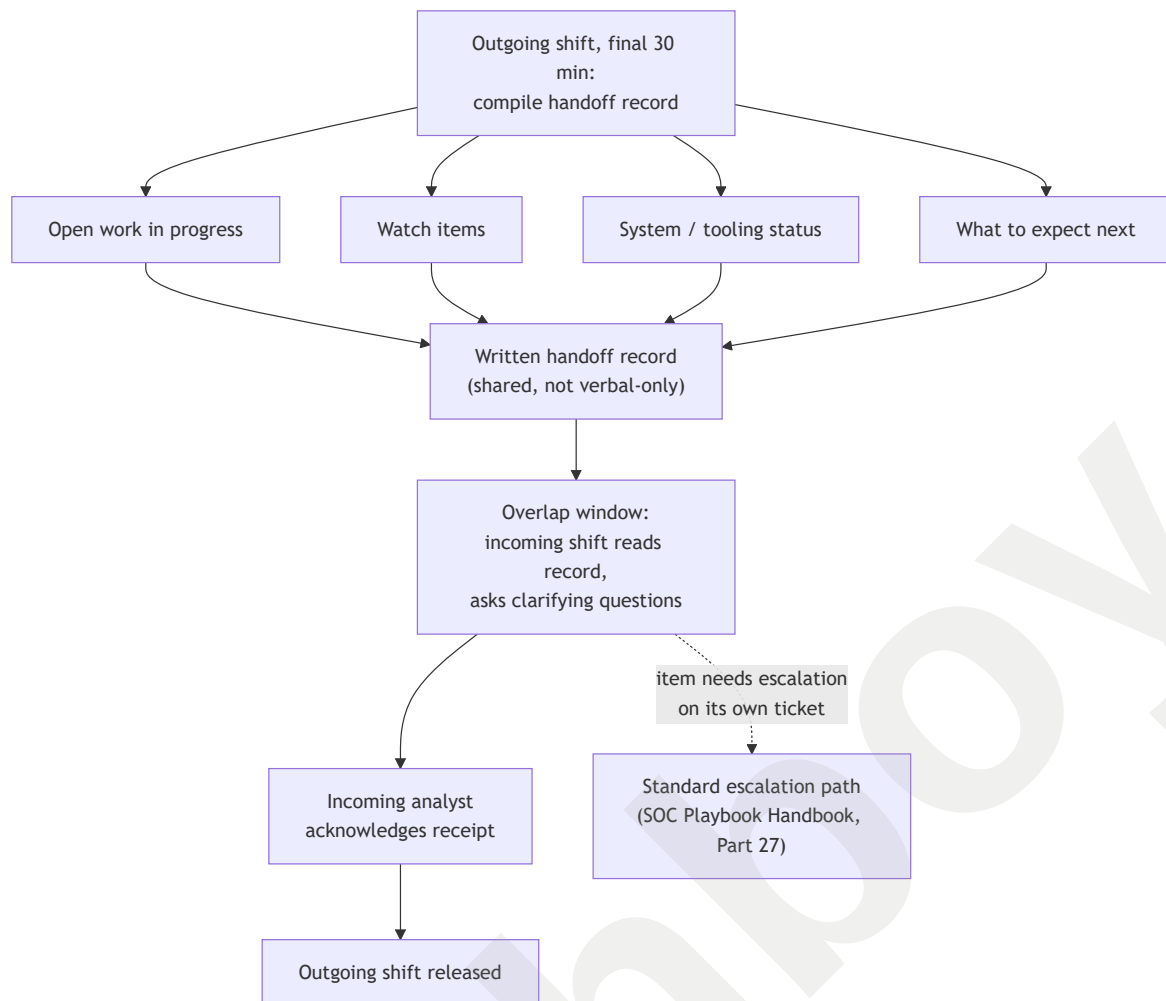
Figure 6.1 — The shift-handoff transfer flow. *CONCEPTUAL*. Illustrates the sequence a complete handoff follows across a shift boundary, independent of which of the four coverage models in §1–5 is running; it is a process diagram, not a capture of any specific team's tooling. Diagram ID [FIG-0601](#).

flowchart TD

```

A["Outgoing shift, final 30 min:\ncompile handoff record"] --> B["Open work in progress"]
A --> C["Watch items"]
A --> D["System / tooling status"]
A --> E["What to expect next"]
B --> F["Written handoff record\n(shared, not verbal-only)"]
C --> F
D --> F
E --> F
F --> G["Overlap window:\nincoming shift reads record,\nasks clarifying questions"]
G --> H["Incoming analyst\nacknowledges receipt"]
H --> I["Outgoing shift released"]
G -.-> J["item needs escalation\non its own ticket"]
J --> K["Standard escalation path\n(SOC Playbook Handbook, Part 27)"]

```



[FRONTLINE MANAGER]

Two structural details make the diagram above work in practice rather than as an aspiration. First, the written record has to exist before the overlap window starts — the overlap is for questions and clarification, not for producing the record itself, which means the outgoing shift needs protected time near the end of the shift specifically to write it, not "whenever there's a lull." Second, the incoming analyst's acknowledgment has to be a real step, not implicit — a shift that lets the outgoing analyst leave the moment the incoming one shows up, with no confirmation the record was actually read, has the same failure mode as sending an escalation and never checking whether anyone picked it up.

Management Autopsy — "cut the shift-overlap window to save payroll" (COMPOSITE CASE EXAMPLE, CASE-0603)

The decision: A 22-analyst SOC running three fixed eight-hour shifts eliminated its 30-minute paid overlap between shifts, moving to a hard clock-boundary handoff where the outgoing analyst was expected to log off at the shift's end and the incoming analyst started cold.

Why it seemed reasonable: Thirty minutes a day, across three shift boundaries, seven days a week, worked out to roughly 548 payroll-hours a year for that single seat line — multiplied across every seat in the SOC, a real, board-visible number the budget review flagged as low-hanging fruit with "no operational impact," since the plan was for outgoing analysts to leave brief written notes before logging off instead.

How it failed: Written notes, produced under no protected time and with nobody checking whether they'd actually been read, defaulted to whatever the outgoing analyst remembered to type in the last few minutes before their shift ended — which was reliably the open tickets (because those had ticket numbers forcing them to exist) and almost never the watch items or system-status context that had no forcing function. Three separate incidents over one quarter traced back to the incoming shift not knowing about a watch item or a known tooling outage the previous shift had been tracking informally. One of the three cost roughly six hours of duplicated investigation before someone on the new shift rediscovered, from scratch, a false-positive pattern the previous shift had already identified and would have mentioned in a real overlap conversation.

The fix: Restore the overlap window as protected, paid production time — not optional, not the first thing cut when the shift is short-staffed — and pair it with the written record in Figure 6.1 rather than relying on either the overlap or the notes alone. The payroll line the original decision saved was real — roughly 548 hours a year on that single seat line. But the six-hour duplicated investigation on just one of the three incidents, priced at a fully-loaded incident-response rate rather than a routine overlap-window wage, plausibly cost more by itself than a month of that saved time, without counting the other two incidents or the ongoing risk of a fourth.

[SENIOR MANAGER] The number worth carrying out of that autopsy isn't "overlap is expensive" — it's that overlap looks like pure cost on a budget line and only shows its value when something goes wrong, which is exactly the asymmetry that makes it an easy, wrong target during a budget review. If a manager can't point to a specific incident the overlap prevented, that's not evidence the overlap is unnecessary; the same is true of any other control that only pays for itself on the days it's needed.

8. Choosing a coverage model: a decision framework

[SENIOR MANAGER] The four models in §1–5 aren't a menu a manager picks from freely — they're constrained hard by three questions: is the workforce genuinely distributable across regions, does off-hours alert volume justify a standing overnight crew at all, and can the organization tolerate the cost of

the model that best fits the first two answers. The flow below sequences those questions in the order that actually eliminates options fastest.

Figure 6.2 — Coverage model decision path. *CONCEPTUAL.* Illustrates the sequence of constraints that typically narrows a coverage-model choice to one realistic candidate; it is a decision-support diagram, not a substitute for the cost and headcount arithmetic in §1–5 once a candidate model is identified. Diagram ID FIG-0602.

flowchart TD

A["Does off-hours alert volume\njustify a standing overnight crew?"] -->|No| B["On-call / bridge coverage (\$5)"]

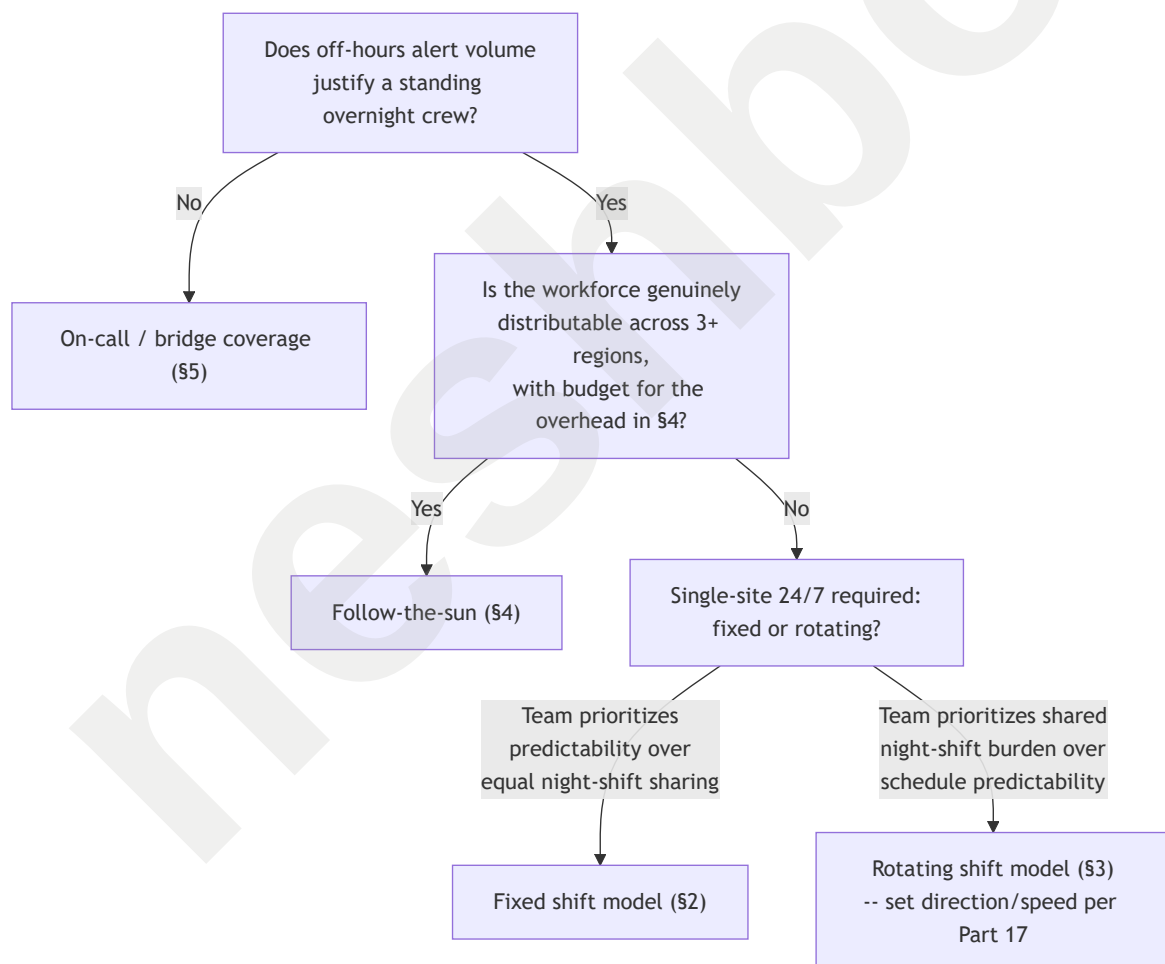
A -->|Yes| C["Is the workforce genuinely\ndistributable across 3+ regions,\nwith budget for the overhead in §4?"]

C -->|Yes| D["Follow-the-sun (§4)"]

C -->|No| E["Single-site 24/7 required:\nfixed or rotating?"]

E -->|"Team prioritizes\npredictability over\nequal night-shift sharing"| F["Fixed shift model (\$2)"]

E -->|"Team prioritizes shared\nnight-shift burden over\nschedule predictability"| G["Rotating shift model (§3)\n-- set direction/speed per Part 17"]



[SENIOR MANAGER] The honest failure mode of this framework is the same one every decision tree has: it assumes each question has a clean yes/no answer when real organizations often have a genuinely uncertain middle case — off-hours volume that's low most nights and spikes during specific business

events, or a workforce that's distributable in two regions but not a clean three. Treat a "maybe" answer at any node as a signal to model both branches with real numbers (per the worked comparison in §4) rather than forcing a premature yes or no just to keep the flowchart moving.

[EXECUTIVE] When this decision reaches a budget conversation, the framework above is not what a CFO or board wants to see — they want the dollar delta between the realistic candidate models, in the same format Part 20 uses for the broader SOC budget case. The value of walking the decision path first is narrowing to one or two models worth costing out in that detail, rather than presenting every option's full cost breakdown and asking the board to do the narrowing themselves.

9. Where this goes next

[CONCEPT] This part assumed a headcount number from Part 5 and turned it into a coverage model, a rotation design, and a handoff mechanism. It deliberately left two things for later parts to finish: the sleep-science and error-rate evidence behind rotation direction, shift length, and mandatory recovery time after a major incident belongs to Part 17, which this part cited but did not re-derive; and the retention cost of night-shift and on-call burden belongs to Part 18. Appendix A1 carries the fillable versions of the shift-pattern templates ([TMPL-0601](#)) and the handoff checklist introduced here ([TMPL-0602](#)), plus an on-call rotation calculator this part's §5 assumed but didn't build ([TMPL-0603](#)). Nothing past this point should need to re-decide what has to transfer at a shift boundary — it should assume Table 6.2's four categories and build on top of them.

Cross-references

This part assumes the headcount output of Part 5 — Headcount & Capacity Modeling, and previews the fatigue/circadian evidence developed fully in Part 17 — Burnout, Fatigue & Wellbeing and the retention cost of night/on-call burden covered in Part 18 — Attrition & Retention. It defers escalation hand-off mechanics to SOC Playbook Handbook, Part 27 — Escalation Quality, per-alert severity-driven response timing to SOC Playbook Handbook, Part 29 — Playbook Severity Model, and MSSP/overflow-coverage contracting mechanics to Part 2 — SOC Operating Models: In-House, MSSP, Co-Managed, Hybrid and Part 22 — MSSP & Managed-Service Contract Management. Appendix A1 holds the companion shift-pattern ([TMPL-0601](#)), shift-handoff checklist ([TMPL-0602](#)), and on-call-rotation ([TMPL-0603](#)) templates.

Part 7 — Hiring & Sourcing Analysts

Why this part exists

[CONCEPT] This part owns getting a correctly leveled, adequately sized candidate pool in front of a hiring panel — the job description, the channels that pool comes from, and the preview that keeps a signed offer from turning into a 90-day resignation. It deliberately stops there. What happens once a candidate is in front of the panel — structured-interview design, building a practical triage exercise, de-biasing the evaluation — is Part 8's territory, not this one's. The tier boundaries a leveled job description encodes ("this is an L1 req, this is an L2 req") are borrowed from Part 10's competency matrix rather than invented here a second time; this part uses those boundaries, it does not define them. What happens in a candidate's first 90 days after signing is Part 11's onboarding and ramp-up program, and the realistic-preview obligation in §7 exists specifically so that program starts with someone who understood the job they accepted. And whether a given certification actually predicts on-the-job capability, as opposed to padding a resume, is Part 12's question in full — this part only addresses certifications as a sourcing-stage filter, a narrower failure mode with its own specific damage: screening out a capable candidate before anyone ever evaluates their actual capability.

[SENIOR MANAGER] This part assumes three things are already settled by the time sourcing starts: a tiering model that defines what an L1, L2, or L3 analyst is actually responsible for (Part 3), a headcount plan that says how many of each level you're allowed to hire and by when (Part 5), and a shift pattern that determines what hours the role actually covers (Part 6). Sourcing without those three in hand produces the failure mode this part spends most of its pages diagnosing: a posting that describes a role nobody has actually designed yet, filled by whoever applies fastest — which is not the same thing as filled correctly.

1. The mid-level analyst market, and why sourcing is where headcount plans die

[CONCEPT] A headcount plan built in Part 5 says a SOC needs, for example, 12 analysts split across three tiers to hold 24/7 coverage. Sourcing is where that number either becomes real people or becomes a permanently open req that shows up in every staffing review for the next year. The two tiers do not source at the same difficulty, and treating them as one problem is the single most common planning error this part exists to correct.

[SENIOR MANAGER] Entry-level (L1) roles draw from a genuinely large applicant pool — bootcamp graduates, degree holders, help-desk staff looking to move, career-changers testing the field. A well-written L1 posting on a mainstream job board typically fills inside 30 to 45 days, illustrative figures for a mid-market SOC rather than a sourced industry benchmark. Mid-level (L2) roles — two to five years of hands-on triage experience, capable of running an investigation without a senior analyst checking every step — draw from a pool that is small, employed, and getting called by three other SOC's and at least one MSSP the same week. An L2 req that sits open past 90 days without a serious rethink of the sourcing strategy is not unusual; it's the default outcome of running an L1 sourcing process against an L2 role.

Manager's Note *If your L2 req has been open longer than your L1 req from the same headcount round, that's not bad luck — it's the market telling you the same sourcing channel doesn't work for both roles. Diagnose the channel before you touch the compensation band; a lot of managers reach for a pay bump first because it's the lever they control fastest, when the actual problem is that the job board posting for an L2 role never reaches anyone who's already an L2 analyst somewhere else.*

[SENIOR MANAGER] Three things make the L2 layer specifically tight, and each has a different fix rather than one shared fix:

- **Supply is genuinely thin.** Producing a competent L2 analyst takes roughly a year or two of real triage volume after an L1 hire clears onboarding — there is no fast path to manufacture more L2 candidates on demand, which is why sourcing has to reach people who are already there rather than waiting for the market to produce more.
- **Demand is correlated across employers.** Every SOC that grew its L1 bench 18 months ago is now trying to promote or backfill L2 seats at roughly the same time, because headcount plans tend to follow the same budget cycle and the same industry-wide alert-volume growth.
- **Retention pressure compounds it.** Part 18 documents burnout-driven exits clustering in the 18- to 24-month range — almost exactly the point at which an L1 analyst becomes a credible L2 candidate for a competitor. The candidate you're trying to source externally is, in a real sense, competing with the internal promotion your own L1 bench should be producing for you.

[SENIOR MANAGER] The practical implication: a sourcing strategy built entirely around external, market-facing channels (job boards, contingency recruiters) will always be fighting the tightest part of the market at the worst odds. The channels covered in §4 through §6 — internal pipelines, apprenticeships, and career-changers — exist because they draw from pools the rest of the market isn't fishing in, not because they're cheaper (though they usually are).

2. Levelled job descriptions that map to your tiering model

[HR/PEOPLE] A job description has two audiences that want different things from the same document: a candidate deciding whether to apply, and an applicant-tracking system deciding whether to surface the resume to a human at all. Most job descriptions fail the second audience by accident and the first on purpose, listing every skill a fully mature L2 analyst might eventually use as if it were a day-one requirement. The result screens out the exact candidate a mid-level market can't afford to lose — someone strong on fundamentals who hasn't touched the specific SIEM named in bullet point nine.

2.1 The levelled job description skeleton

[HR/PEOPLE] The fix starts with writing one skeleton with a level parameter, not three unrelated documents. The template below states, level by level, what changes and what doesn't — filed in Appendix A2 as `TMPL-0701`, the levelled job-description template. Use it whenever a new req opens against an already-defined tier from Part 3's model; it does not decide what belongs in each tier for your SOC specifically — that's a Part 3 and Part 10 exercise this template assumes is already done.

TEMPLATE – Leveled analyst job description skeleton, permanent ID TMPL-0701

ROLE: SOC Analyst – [Tier: L1 / L2 / L3]

SHIFT COMMITMENT: [State the actual pattern from Part 6 – do not write "flexible hours" if the role is a fixed night rotation]

SCOPE OF WORK (what this person owns, not what the team owns):

- <!-- state the specific alert categories, ticket types, or investigation depth this level handles unsupervised, not "monitors security events" -->

DECISION AUTHORITY:

- <!-- what this level can close/escalate/contain without sign-off, and what always requires a hand-off -->

MUST HAVE (screens candidates out – keep this list short and load-bearing):

- <!-- the two or three things a person genuinely cannot do the job without, stated as a capability, not a certification: "can read raw firewall logs and reconstruct a session," not "CCNA required" -->

STRONGLY PREFERRED (does not screen out; used to break ties):

- <!-- everything else that would be nice -->

GROWTH PATH: What "ready for the next tier" looks like, one sentence, pointing to Part 13's ladder rather than restating it.

ON-CALL / ESCALATION EXPECTATION: <!-- the actual frequency, stated as a number ("one week in six"), not "occasional on-call support may be required" -->

One sentence states what the template does not automate: it does not decide where the must-have/strongly-preferred line sits for a given tier at your SOC — that judgment call belongs to whoever owns the competency matrix in Part 10, and a template filled in without that input just moves the kitchen-sink problem from the "requirements" section to the "preferred" section instead of fixing it.

2.2 Common defects: title inflation and the kitchen-sink requirement list

[HR/PEOPLE] Two defects account for most leveled-JD failures, and neither is exotic.

The first is writing one job description and slapping "Senior" on the title for the L2 version without changing the content — same bullet points, same tool list, same three sentences about "a passion for security," different job title. Nothing in the document tells an L2 candidate what actually changes about their day, and nothing tells the hiring panel what to test for that an L1 candidate wouldn't also pass.

The second is the kitchen-sink list: every tool the SOC has ever deployed, every certification anyone on the team happens to hold, every "nice to have" stacked into "must have" because a manager reasoned that more requirements can only improve the applicant quality. It does the opposite. An ATS keyword filter or a self-selecting candidate reads 11 required items and one missing item the same way — as a mismatch — even when the missing item is the least important one on the list.

People Risk Trap *A kitchen-sink requirements list doesn't just shrink the applicant pool — it shrinks it non-randomly, in a direction that hurts the mid-level market specifically. A candidate with five years of strong, relevant experience but a resume that happens to use different vendor-tool names than your list will self-select out or get auto-rejected before a human ever reads it, while a candidate with none of the practical experience but a resume tuned to match every keyword sails through. The fix: cap the must-have section at three items, stated as capabilities rather than tool or certification names wherever a capability claim can substitute for a brand name, and move everything else to strongly preferred — a section explicitly documented as non-disqualifying.*

[HR/PEOPLE] The table below states what should actually differ across three tiers of the same analyst job description, as a working reference for filling in **TMPL-0701** (CONCEPTUAL SAMPLE — illustrative tier definitions; substitute your own Part 3/Part 10 boundaries).

Dimension	L1	L2	L3
Scope of autonomy	Handles defined, playbook-covered alert types unsupervised	Handles ambiguous or multi-stage alerts; builds the initial hypothesis without a playbook	Owens novel investigations and mentors triage judgment calls across tiers
Escalation authority	Escalates on any uncertainty per playbook trigger	Decides when to escalate vs. close; owns the call for its own queue	Is the escalation endpoint for L1/L2; decides when to pull in IR
Typical experience band	0–2 years, including adjacent IT roles	2–5 years of hands-on triage	5+ years, often with a specialist track (hunting, detection engineering)
Tooling depth expected	Comfortable navigating the SIEM/EDR console	Can write or tune queries, not just run saved ones	Can evaluate whether the tool itself is the problem
On-call exposure	Rare or none	Rotational, as defined in Part 6	Often carries escalation on-call across shifts

3. The certification trap in sourcing screens

[HR/PEOPLE] Certifications are useful signal at the right stage and useless, actively harmful signal at the wrong one. Whether a specific certification predicts on-the-job capability once someone is already an analyst is Part 12's question, and the honest answer there is "it depends, and mostly not as much as certification-heavy job boards imply." This part's narrower claim doesn't need Part 12's full analysis to hold: using certifications as a sourcing-stage filter — before anyone has evaluated a candidate's actual reasoning — screens out people who would pass the real assessment in Part 8 and never gets the chance to find out.

CASE-0701 — the 11-line filter. *COMPOSITE CASE EXAMPLE, illustrative numbers constructed from recurring patterns across several mid-market SOC hiring processes, not a single traceable organization.* An L2 analyst req's applicant-tracking system was configured to auto-reject any resume lacking at least two of three named certifications (Security+, CySA+, GCIH) plus three years of a job title containing the word "analyst." Over a six-month posting window, the filter auto-rejected 71% of submitted

applications before a recruiter read them. A manual audit of a sample of the rejected pool — done only after the req had sat open for 140 days — found several candidates with genuine triage-relevant backgrounds the filter had no way to see: a former help-desk lead with two years of hands-on incident-ticket ownership and no formal security title, a compliance analyst who had spent a year investigating access-log anomalies for SOX evidence, and a veteran with an intelligence-analysis background and no civilian certifications at all. None had the specific job-title string or the certification pair the filter demanded. All three, once interviewed, cleared the technical bar the req actually needed.

People Risk Trap *An ATS keyword filter configured around certifications and job titles doesn't reject weak candidates — it rejects candidates whose resume happens to describe their experience differently than the filter expects, which correlates with non-traditional background far more than it correlates with actual capability. The fix is not to remove screening; it's to move the screening criteria from proxy signals (a certification name, a job-title string) to the two or three capability statements from [TMPL-0701](#)'s must-have section, evaluated by a human on a fast pass rather than an automated keyword match. If your ATS can't screen on a capability statement, screen on nothing at automated stage and let a recruiter do a 15-minute pass instead — a slower human filter that actually reads the resume outperforms a fast filter that reads only certification names.*

[HR/PEOPLE] This doesn't mean certifications are worthless in sourcing — a cert can be a legitimate, if weak, proxy for baseline knowledge on a candidate with no other track record, which is a real use for an L1 posting drawing from a large unknown pool. The trap is specifically applying that same proxy-signal logic to an L2 or L3 req, where a candidate's actual track record — investigations closed, escalations handled, tools built — is available and is a far stronger signal than whether they hold a particular vendor exam. Part 12 covers which certifications correlate with real capability once you're deciding on a training and certification-support budget for people already on the team; this part's fix is narrower and cheaper: stop using certification possession as a pre-interview gate for any role above L1.

4. Help-desk-to-SOC pipelines

[HR/PEOPLE] A help desk or NOC produces a steady stream of people who already have three things a security-fundamentals bootcamp graduate usually doesn't: real ticketing-system discipline, real experience being the calm voice on a stressful call, and direct exposure to how this specific organization's systems actually behave day to day. What they don't have yet is security-specific pattern recognition, which is exactly what a structured onboarding program (Part 11) exists to build — the pipeline's job is to select the right people and get them into that program, not to substitute for it.

CASE-0702 — the tenure-gated shadow track. *COMPOSITE CASE EXAMPLE, illustrative numbers built from a common pattern across help-desk-to-SOC programs, not one traceable organization.* A help-desk-to-SOC pipeline nominated agents with at least 12 months of tenure and a clean escalation record for an eight-week shadow track: two weeks of security-fundamentals coursework, four weeks of supervised ride-along on real (already-triaged) tickets with a mentor analyst, and two weeks running a reduced live queue under close review before a formal L1 offer. Roughly 30% of nominated agents completed the

track and converted to an L1 analyst role in a typical year. The blended cost per hire through this channel — mentor time, coursework, the opportunity cost of a reduced live queue during the ride-along weeks — ran to roughly a third of the agency-recruiter cost per hire the same SOC was paying for externally sourced L1 hires, and 12-month retention on pipeline hires ran noticeably higher than on externally sourced hires from the general job-board channel, largely because pipeline hires already knew the organization's systems, culture, and shift realities going in.

[HR/PEOPLE] Two design choices make or break a pipeline like this. First, the tenure and record gate has to be real and enforced — a pipeline that takes any help-desk agent who asks, regardless of performance, becomes a dumping ground for people the help-desk manager wanted to move along anyway, and the SOC inherits that reputation fast. Second, the help-desk manager has to be a genuine stakeholder in the program rather than someone losing staff to it with no offsetting benefit; a pipeline that quietly drains a help desk's best people without backfill or executive sponsorship turns into a cross-team political fight covered more fully in Part 26.

Manager's Note *Run the pipeline as a named, visible program with its own intake criteria — not an informal "talk to me if you're interested" arrangement. An informal version tends to surface whoever is already friends with someone on the SOC team, which is a much narrower and less representative pool than the one a documented, open nomination process reaches, and it's much harder to defend as fair if it's ever questioned.*

5. Apprenticeships and other structured non-traditional pipelines

[HR/PEOPLE] An apprenticeship model — a fixed-length, paid, cohort-based program combining structured coursework with supervised real work, common in the U.S. under the Department of Labor's registered-apprenticeship framework and in similar forms elsewhere — solves a different sourcing problem than the help-desk pipeline. It reaches people with no existing employment relationship to the organization at all: career-changers, community-college and bootcamp graduates, veterans transitioning out of a related but non-civilian specialty. The tradeoff is structural rather than financial: an apprenticeship needs a genuine curriculum, a named program owner, and a defined evaluation gate before conversion to a full analyst role, or it becomes an unpaid-adjacent labor arrangement that damages the SOC's reputation with exactly the talent pool it's trying to reach.

[SENIOR MANAGER] Building a cohort-based apprenticeship (CONCEPTUAL SAMPLE — illustrative structure, not a specific published program) typically runs 10 to 12 weeks, mixes vendor-neutral security fundamentals with graduated, supervised live-queue work in roughly the same shape as the help-desk shadow track in §4, and ends with a documented conversion decision rather than an automatic offer. The cost profile favors this channel heavily against agency recruiting on a per-hire basis once a cohort is running at any real size, because the marginal cost of a fourth or fifth apprentice in the same cohort is far lower than the marginal cost of a fourth or fifth agency placement — but the fixed cost of designing the curriculum and staffing mentors the first time is real, and a one-off cohort of two or three people rarely justifies that fixed cost on its own. This is the channel most worth building jointly with two or three peer SOCs, a community college, or a regional workforce-development partner, spreading the

fixed curriculum cost across more apprentices than any single SOC could source alone — a build-vs-buy tradeoff structurally similar to the one Part 2 works through for the SOC's operating model itself, applied here to a training pipeline instead of the SOC function as a whole.

6. Career-changers as a deliberate sourcing category

[HR/PEOPLE] Career-changers overlap partly with the help-desk and apprenticeship channels above but deserve their own sourcing lens because the transferable skill and the resume language rarely match. A military intelligence analyst, a systems administrator, a compliance or internal-audit investigator, a QA engineer used to reproducing and documenting defects, even an experienced teacher used to real-time behavior monitoring across a room of 30 people — each brings a genuinely relevant capability (structured investigation, pattern recognition under time pressure, disciplined documentation, sustained attention across a long shift) that almost never appears as a keyword an ATS or a skimming recruiter is trained to look for.

[HR/PEOPLE] The practical fix sits in the job description itself, not just in outreach: an explicit "or equivalent experience" clause attached to each must-have item, with the equivalent stated concretely rather than left as a vague escape hatch. "Two years of SOC analyst experience, or two years in a role requiring structured investigation under time pressure (audit, compliance, help desk, military intelligence, or similar)" does real work; "or equivalent experience considered" as a bare afterthought does none, because it gives a recruiter or an ATS nothing to match against and gets ignored in practice.

Manager's Note *When you interview a career-changer, resist grading them against the vocabulary an incumbent analyst would use and instead ask them to walk through a real investigation or discrepancy they resolved in their previous role, in their own language, then map it yourself to the capability you actually need. A candidate who can describe reconstructing a fraudulent-transaction pattern from disparate audit logs is describing the same underlying skill as reconstructing an attack chain from disparate telemetry — they just don't know to call it that yet, and it isn't their job to translate it for you.*

7. The realistic job preview

[HR/PEOPLE] A realistic job preview — telling a candidate, before they accept, what the job actually feels like rather than only what it requires — is an established recruiting practice, not a novel idea invented for this book: giving candidates accurate, including unflattering, information about a role has long been shown in organizational-psychology research to reduce early turnover, because it lets people who wouldn't tolerate the actual conditions self-select out before they cost the organization a signed offer and a 90-day ramp-up investment. A SOC has an unusually strong case for using one deliberately, because the gap between how a SOC analyst role reads on paper and what it feels like on a Tuesday night shift is often wide.

7.1 What a realistic preview actually includes

[FRONTLINE MANAGER]

A useful preview is concrete, not a disclaimer. Four elements do most of the work:

- **The actual shift pattern**, stated the way Part 6 defines it — not "some evening and weekend work may be required" when the real pattern is a fixed rotating night shift with one weekend in six. If a candidate would decline the role after seeing the real pattern, better to lose them before the offer than after 90 days.
- **A sample-ticket walkthrough during the interview itself** — showing (or better, walking a candidate through) a real, already-closed alert and asking them to reason about it out loud, so the interview process itself previews the actual work rather than only discussing it abstractly.
- **A stated volume expectation** — a concrete range like "40 to 60 tickets per eight-hour shift at a steady state, with surge days well above that" tells a candidate something a bulleted list of responsibilities never will.
- **Honest disclosure of the escalation and on-call reality** — how often a shift genuinely turns into a high-stress event, not just that "incident response" appears somewhere in the job description.

[FRONTLINE MANAGER]

None of this requires new infrastructure. A 15-minute segment added to the existing interview loop, run by whoever already owns the technical portion of the assessment in Part 8, covers most of it — the preview and the assessment can share the same conversation as long as the interviewer is explicit about which parts are evaluative and which parts are informational for the candidate.

CASE-0703 — attrition after adding a shift-reality preview. *COMPOSITE CASE EXAMPLE, illustrative numbers, constructed from a recurring pattern rather than one traceable organization.* A SOC hiring for a fixed night-rotation L1 role had been describing the shift pattern only as "24/7 coverage team, flexible scheduling" in the posting and interview, relying on the offer letter to disclose the actual rotation. 90-day attrition on hires from that channel ran to roughly 35%, with exit interviews (Part 18's methodology) consistently naming the shift pattern as a surprise rather than a known tradeoff. After the SOC added a mandatory 15-minute shift-reality segment to the interview — stating the exact rotation, the weekend frequency, and asking candidates directly whether that pattern was workable for their life circumstances before proceeding further — the applicant pool narrowed (fewer candidates advanced past that stage, which was the intended effect, not a defect), and 90-day attrition among hires who did advance fell to roughly 12% over the following year.

People Risk Trap *Treating a realistic preview as something that might scare off candidates you need is the mistake, not a defensible tradeoff — every candidate it scares off is a candidate who would have accepted, discovered the reality within a few weeks, and left anyway, except now the SOC has also lost the 60 to 90 days of onboarding investment. The math almost always favors losing the candidate earlier and cheaper.*

7.2 Validating your own preview

[FRONTLINE MANAGER]

A preview that exists only as a line in the job posting ("shift work required") without being reinforced in the interview conversation tends to be technically honest and practically useless, because candidates skim past boilerplate language and remember what an actual person told them.

Field Test Setup: A written realistic-preview step exists in the interview process (the shift pattern, ticket volume, and on-call reality are supposed to be disclosed at some point before an offer). **Action:** Ask someone with no involvement in hiring for this role — a peer manager, an HR partner, or a recently onboarded analyst — to sit through the standard phone screen or read the current posting cold, then describe back, unprompted, what a typical Tuesday-night shift in this role actually feels like. **Expected result:** They should be able to name the shift pattern, a rough ticket-volume range, and the on-call frequency without having to guess or extrapolate. If they can only repeat generic language like "fast-paced environment" or "24/7 operations," the preview exists on paper but isn't actually reaching candidates in the room.

8. Choosing and scoring sourcing channels

[SENIOR MANAGER]

No single channel covers every level and every urgency, and the right answer changes with how much time pressure a given req is under. The table below compares the channels this part covers, as a working reference for a headcount round rather than a fixed ranking (CONCEPTUAL SAMPLE — illustrative comparison; substitute your own sourced cost and time-to-fill data before using this for a real budget defense in Part 20).

Channel	Typical cost per hire	Typical time-to-fill	Best-suited level(s)	Primary risk
Job boards / inbound applications	Low direct cost, high recruiter screening time	30–60 days for L1; often 90+ days for L2	L1 primarily	Highest exposure to the certification-keyword trap in §3
Employee referral	Low	20–45 days	L1–L2	Can narrow diversity of the pool if run informally, per §4's caution
Contingency / agency recruiter	High — often 15–25% of first-year salary	30–60 days once engaged	L2–L3	Fast, but expensive at volume; weak on non-traditional candidates who don't match agency search terms
Help-desk-to-SOC pipeline	Low, mostly internal staff time	8–12 weeks program length, planned in advance	L1, feeding L2 over time	Needs a genuine tenure/performance gate or it becomes a dumping ground
Apprenticeship / bootcamp partnership	Low per hire at cohort scale; high fixed setup cost	10–12 week program, cohort-scheduled	L1	Fixed curriculum cost doesn't pay off below a minimum cohort size
University / early-career program	Moderate, seasonal	Tied to academic calendar	L1	Long lead time; poor fit for an urgent open req
Veteran transition program	Low to moderate	Varies with program partner	L1–L2, occasionally L3 for intelligence-adjacent roles	Resume translation gap covered in §6 if the JD isn't adapted

9. Worked example: rebuilding a sourcing funnel under a tight mid-level market

CASE-0704 — the agency-only L2 funnel. *COMPOSITE CASE EXAMPLE, illustrative numbers constructed to demonstrate the interaction of the failure modes covered in this part; not a single traceable organization.*

[SENIOR MANAGER] A 40-analyst SOC's headcount plan (from a Part 5-style capacity model) approved three additional L2 seats for the year. The SOC sourced all three exclusively through a contingency recruiting agency, using a job description carried over from the previous year with minor edits — a description that listed 11 required certifications and tools, several inherited from a platform the SOC had since retired. Average time-to-fill across the three reqs ran to roughly 150 days, agency fees ran to about \$17,500 per hire, and of the three hires eventually made, two left within six months — exit interviews naming the actual on-call rotation and escalation-volume expectations as materially different from what they'd understood going in, echoing the pattern in CASE-0703.

[SENIOR MANAGER] The redesign touched every lever this part covers, not just one:

- The job description was rewritten against **TMPL-0701**, cutting the requirements list to three must-have capability statements and moving everything else — including the retired platform's tool name — to strongly preferred.
- A help-desk-to-SOC pipeline (§4) was stood up in parallel, drawing on internal candidates who already knew the environment; two of the eventual three hires for the next round of reqs came

from this channel rather than external sourcing.

- A mandatory shift-and-escalation-reality segment (§7) was added to every interview loop for the role, disclosing the actual on-call frequency and a real ticket-volume range before any offer was extended.

[SENIOR MANAGER] Over the following 12 months, average time-to-fill for L2 reqs fell to roughly 55 days, blended cost per hire (mixing internal-pipeline hires with the smaller number still sourced externally) fell to roughly \$4,200, and six-month attrition among new L2 hires fell to roughly 10%. None of the three changes alone would have produced that result on its own — a rewritten job description with no change to sourcing channel still competes for the same thin external pool; a pipeline with no realistic preview still risks the same early-attrition pattern from CASE-0703; a preview attached to a kitchen-sink job description never gets enough candidates through the door to preview anything to. The combination is the point.

10. Choosing a channel for a specific req: a decision aid

[SENIOR MANAGER] Figure 7.1 below turns §1 through §8 into a single decision aid for a manager opening a new req — which channel (or combination) fits, given the level and how much runway the req has before it needs to be filled.

Mermaid source retained below as the editable source of truth per this book's diagram build process.

flowchart TD

```
A["New requisition opens"] --> B["What tier?\n(Part 3 definition)"]
B -->|"L1"| C["Time pressure?"]
B -->|"L2"| D["Internal candidate pool\nexists (help-desk,\nNOC, adjacent IT)?"]
B -->|"L3 / specialist"| E["Direct or agency sourcing;\nnarrow, capability-based bar"]
C -->|"Urgent, under 30 days"| F["Job boards +\nemployee referral"]
C -->|"Runway of 8+ weeks"| G["Apprenticeship cohort or\nhelp-desk shadow track"]
D -->|"Yes, 12+ months tenure\navailable"| H["Internal transfer track (Sec. 4)"]
D -->|"No"| I["External sourcing with\ncareer-changer JD framing (Sec. 6)"]
E --> J["Mandatory realistic\njob preview (Sec. 7)"]
F --> J
G --> J
H --> J
I --> J
J --> K["Proceeds to Part 8:\nstructured interview & assessment"]
```

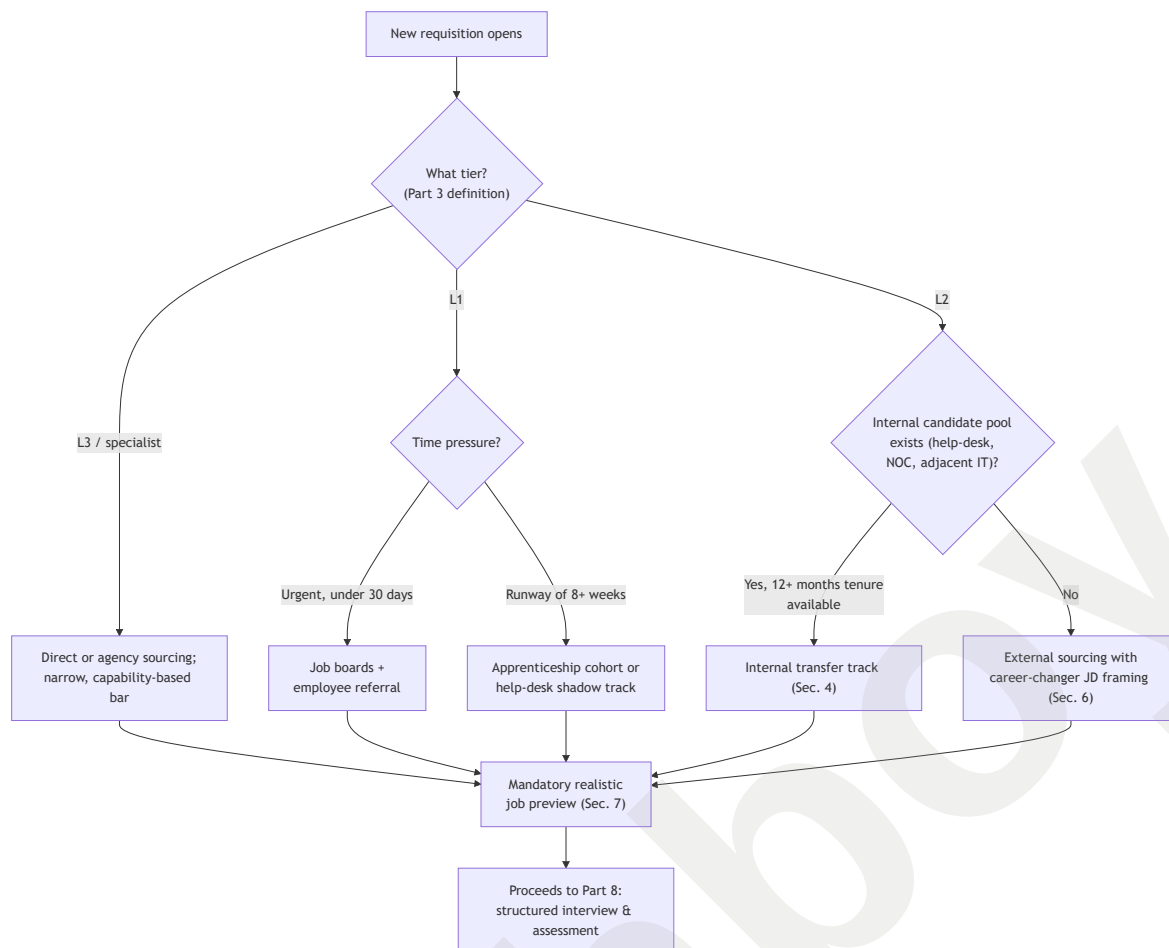


Figure 7.1 — Choosing a sourcing channel by tier and time pressure. *CONCEPTUAL.* Illustrates the decision path this part's channels feed into; it is a structural aid for a manager opening a req, not a capture of any specific SOC's actual applicant-tracking workflow. Every path converges on the realistic job preview from §7 before handing off to Part 8's assessment process — no channel is exempt from that step. Supports the channel-selection guidance in this section and the worked example in §9. Diagram ID **FIG-0701**.

Where this goes next

[CONCEPT] A correctly sourced candidate is the input Part 8's structured interview and assessment process needs to do its job — a strong assessment process cannot fix a pool that never contained the right people, and a weak one will waste a strong pool regardless. Part 10's competency matrix should be the document **TMPL-0701** pulls its tier boundaries from, and Part 13's career ladder is what the "growth path" line in that same template ultimately points to. Part 12 picks the certification question back up in full once someone is already on the team and a training budget is being decided, rather than at the sourcing gate this part covers. Part 18's attrition data is what the realistic-preview obligation in §7 exists to reduce, and Part 20's budget model is what the cost-per-hire figures in §8 and §9 ultimately feed into when a hiring channel needs to be defended in dollars rather than described in prose.

Cross-references: This part assumes Part 3 — Tiering Models: L1/L2/L3 and Beyond (tier definitions that leveled job descriptions encode), Part 5 — Headcount & Capacity Modeling (the approved req count

and timing this part sources against), and Part 6 — Shift Pattern & Coverage Design (the shift reality a realistic job preview must disclose accurately). It points forward, within this book, to Part 8 — Interviewing & Technical Assessment Design, Part 10 — Competency Models & Skills Matrices, Part 11 — Onboarding & Ramp-Up Programs, Part 12 — Ongoing Training & Skill Development, Part 13 — Career Ladders & Promotion Criteria, Part 18 — Attrition & Retention, and Part 20 — Building & Defending the SOC Budget. It does not cite SOC Playbook Handbook or Detection Engineering Handbook V2 — sourcing and hiring mechanics have no technical-mechanics counterpart in either companion volume. The leveled job-description template ([TMPL-0701](#)) is filed in Appendix A2, companion to Parts 7–8 and 10.

Part 8 — Interviewing & Technical Assessment Design

Why this part exists

[CONCEPT] Part 7 gets a candidate to the point where their resume matches a leveled job description and they've seen a realistic preview of the job. This part is what happens once that candidate is actually on a call or working through a timed exercise: how the interview loop is structured, what a practical assessment actually tests, how a panel scores it consistently, and how the whole process gets checked for bias before it quietly starts hiring for confidence instead of competence. The two parts are sequential, not overlapping. A poorly leveled job description — Part 7's problem — sinks an interview loop no matter how well the loop itself is designed, because the loop ends up screening candidates against a bar nobody actually defined.

This part does not invent new detection logic, new playbook steps, or new escalation criteria to use as assessment material. When an exercise needs a log-triage scenario, the underlying alert, query, or analytic behind it is borrowed — sanitized and adapted, not authored from scratch — from Detection Engineering Handbook V2's detection content, cited by part number where it matters. When an exercise needs a mock escalation, the standard for what a good escalation looks like comes from SOC Playbook Handbook, Part 27 — Escalation Quality, for the mechanics of a good hand-off, not from a definition invented here. This part's job is narrower, and in most hiring programs it's the part that goes wrong more often than the material does: designing the exercise format, writing the scoring rubric, running the panel process, and de-biasing the controls around whatever technical content gets dropped into them. A technically flawless exercise scored by an uncalibrated panel with no written rubric produces the same bad hire as a sloppy exercise scored well. Most SOC hiring failures trace back to the process wrapped around the material, not the material itself, which is why this chapter spends almost no time on detection or playbook correctness and a great deal of time on scorecards, calibration, and bias.

1. What a SOC technical interview loop is actually testing for

1.1 Knowledge, skill, and judgment are three different claims

[CONCEPT] Most unstructured SOC interviews test knowledge, because knowledge is the easiest thing to verify in a thirty-minute conversation. "Walk me through the MITRE ATT&CK framework" or "what's the difference between an IOC and an IOA" gets an answer a non-technical interviewer can pattern-match against a study guide, and a technical interviewer can nod along to without ever finding out whether the candidate has actually triaged anything. Knowledge is real and it matters, but it's the weakest of the three things a SOC role actually needs, because it's also the cheapest thing for a candidate to cram the week before an interview.

Skill is whether the candidate can perform the triage motion itself under realistic conditions: pull the right context, read a log line correctly, notice the field that's missing, write a clear note. Judgment is the layer above skill — knowing when the playbook doesn't cleanly apply, when a disposition should be "unable to determine" rather than a confident guess, when a pattern that matches a rule's logic still doesn't mean the activity was malicious. Detection Engineering Handbook V2, Part 1 — Detection

Engineering Foundations draws exactly this line for alert disposition: collapsing "the rule's logic matched" into "the activity was malicious" is a named triage failure mode there, and it is precisely the judgment gap a good SOC interview needs to surface and an unstructured conversation almost never does, because a candidate can describe good judgment in the abstract far more easily than they can exercise it against an actual queue item.

[HR/PEOPLE] The practical consequence for interview design is that knowledge questions belong in a short screen, not the center of the loop. If knowledge is over 60% of the total scoring weight across a loop, the loop is measuring the wrong thing — it will pass articulate candidates who can talk about triage and fail quieter candidates who are actually good at it, and it will do this consistently, cycle after cycle, until someone looks at who's getting hired and who's succeeding six months in and notices they aren't the same population.

1.2 What has to already be true before day one of interviewing

[SENIOR MANAGER] This part assumes the inputs from Part 7 are already in place: a leveled job description that states, in observable terms, what an L1 versus an L2 candidate needs to demonstrate, and a realistic job preview so a candidate walking into the loop already knows roughly what the job is. Without the leveled job description, there's no defensible answer to "what's the pass bar for this exercise" — the pass bar for an L1 hire and an L2 hire on the same log-triage exercise should differ, and it can only differ on purpose if someone already wrote down what separates the two levels before the first candidate sat down.

Cross-Book Pointer This part does not define what a good escalation looks like, mechanically — that's a per-ticket standard, not a hiring standard. See SOC Playbook Handbook, Part 27 — Escalation Quality for the criteria a real escalation is judged against (context included, severity justified, next action named); the mock escalation exercise in §3.2 borrows that standard directly as its rubric rather than inventing a separate one for hiring purposes. Come back here once you understand what "good" means mechanically — this part only covers how to observe it in a 30-minute simulation and score it consistently across candidates.

2. Structured-interview design

2.1 Why the unstructured "just have a conversation" loop fails predictably

[HR/PEOPLE] An unstructured loop — different interviewers asking whatever occurs to them, in whatever order, with no shared rubric — fails in the same handful of ways every time. Interviewers anchor on the first strong or weak answer and score the rest of the conversation to match it. Two interviewers asking different questions produce scores that aren't comparable, so a debrief becomes a negotiation over impressions rather than a comparison of evidence. And the interviewer's own specialty leaks into the bar: a detection-engineering-leaning interviewer over-weights query syntax, an incident-response-leaning interviewer over-weights containment steps, and the candidate's actual fit for the open role — which might be neither — gets lost under whichever interviewer happened to be in the room and enjoyed asking about their own specialty most.

None of this requires a bad-faith interviewer. It's what happens by default when four people are each given "assess this candidate's technical ability" with no shared definition of what that means for this specific role, at this specific level, this week.

2.2 The three-stage loop

[SENIOR MANAGER] A structured SOC loop separates the conversation from the demonstration, and separates both from the debrief. Three stages cover the ground a fourth or fifth stage rarely adds much to:

1. **Screen** (20–30 minutes) — recruiter or team lead confirms baseline fit against the leveled job description: shift availability, the realistic-preview conversation from Part 7, and a small number of scoped technical questions to rule out a clear mismatch before investing panel time.
2. **Practical assessment** (60–120 minutes, proctored or live) — the log-triage exercise and the mock escalation exercise, described in §3, which test skill and judgment directly rather than through a candidate's self-report.
3. **Panel and behavioral loop** (45–60 minutes) — a small panel (two to three interviewers, never one) reviews the assessment output with the candidate, asks behavioral questions tied to specific competencies from the leveled job description, and each interviewer scores independently before any group discussion happens.

The table below maps each stage to what it's actually designed to validate, so a manager building or auditing a loop can check whether a given stage is pulling its weight, adding time to the process for no distinct signal, or duplicating what an earlier stage already validated.

Loop Stage	Primarily Validates	Typical Duration	Owner	Common Failure Mode
Screen	Baseline fit, shift/logistics, gross mismatch	20–30 min	Recruiter or team lead	Turns into a second full technical interview, wasting panel time on candidates who were never a fit
Log-triage exercise	Skill: can the candidate actually work an alert to a defensible disposition	60–90 min	Assessment designer / SOC manager	Graded on "the right answer" instead of the reasoning that got there
Mock escalation exercise	Judgment: does the candidate know what to escalate, how, and with what context	20–30 min	Team lead or senior analyst	No written rubric — panel scores on "sounded confident"
Panel + behavioral loop	Communication, collaboration, culture add (not culture "fit")	45–60 min	Hiring manager + 2 panelists	One panelist's strong opinion overrides two written scorecards in debrief

2.3 Writing a scorecard that survives a challenge

[HR/PEOPLE] A scorecard survives a challenge — from a rejected candidate, from HR, from a lawyer — when every score traces to an observable behavior in the interview or exercise, not to an interviewer's overall impression. "Strong communicator" is not a score; "explained the triage decision in under two minutes without prompting, using the correct account and source-address fields from the log excerpt"

is a score. The difference is whether a second person, reading only the written notes with no memory of the conversation, would reach the same score.

People Risk Trap *Letting a panel debrief happen before every interviewer submits an independent written score lets whoever speaks first, speaks loudest, or outranks the room set the anchor the rest of the discussion drifts toward — a well-documented group-decision failure mode, not a SOC-specific one, but one that shows up constantly in hiring debriefs. The fix: every interviewer submits their scorecard, in writing, against the rubric, before the debrief conversation starts. The debrief resolves disagreement between already-recorded scores; it does not generate the scores in the first place.*

3. Building the practical assessment

3.1 The log-triage exercise: what to build versus what to borrow

[SENIOR MANAGER] The exercise's job is to put a candidate in front of something close to a real alert and watch how they work it, not to test whether they can solve a novel puzzle nobody in the actual role would ever face. The fastest way to build one badly is to invent a synthetic scenario with no connection to how a real analyst actually encounters ambiguity — a clean, single-cause "gotcha" that has one correct answer and one path to it, which tests puzzle-solving more than triage.

A better source is real, already-produced detection content, sanitized: a redacted alert, a de-identified log excerpt, or a described analytic behavior drawn from Detection Engineering Handbook V2's detection library (Parts 3–29 for the underlying rule logic and telemetry, or Part 1 — Detection Engineering Foundations for the vocabulary the rubric should use when scoring how the candidate talks about what they're seeing: event, signal, alert, disposition). The exercise doesn't need — and should not include — the full detection's tuning history or its false-positive allowlist; it needs enough raw material that a candidate has to do the same cognitive work a real analyst does: read the evidence, decide what's missing, and reach a disposition they can defend, including "unable to determine" when the evidence genuinely doesn't support a confident call.

[FRONTLINE MANAGER] The rubric should reward the reasoning path, not just the final label. A candidate who reaches "unable to determine, escalate for enrichment" and can name exactly what enrichment they'd request and why is demonstrating stronger judgment than a candidate who confidently calls it a false positive with no stated reasoning — even if the second candidate happens to land on the label the real detection's own disposition history eventually confirmed. Scoring the label instead of the reasoning rewards lucky guessing and punishes appropriately cautious analysts, which is exactly backward for a role where an overconfident wrong disposition is more expensive than a correctly cautious escalation.

3.2 The mock escalation exercise: what to build versus what to borrow

[SENIOR MANAGER] Where the log-triage exercise tests whether a candidate can work an alert alone, the mock escalation exercise tests whether they can hand it off — verbally, to a panelist playing a Tier 2

analyst or team lead, with a defined time limit (10–15 minutes is enough; longer starts testing stamina rather than communication). The scenario can be the same log-triage exercise the candidate just finished, or a second, shorter one, but the standard the escalation is judged against should not be invented for the hiring loop. It's the same standard a real escalation is judged against operationally: does it include the context the receiving tier actually needs, is the severity call justified rather than asserted, and does it name a concrete next action — the exact criteria in SOC Playbook Handbook, Part 27 — Escalation Quality. Reusing that standard here means a strong performer on this exercise is, by construction, someone who already escalates the way the SOC wants escalations to look on day one, not someone who happens to interview well.

3.3 Calibrating difficulty and length to the level being hired

[SENIOR MANAGER] The same exercise format should not carry the same pass bar across every level on the ladder, and it should not run the same length either — a senior candidate sitting through a 90-minute exercise built for an entry-level screen reads, correctly, as a process that doesn't know what it's hiring for. The table below is a starting calibration, not a fixed rule; adjust the specific minutes and bar language to the actual leveled job description from Part 7 (CONCEPTUAL SAMPLE — illustrative calibration bands, not sourced benchmark data).

Level	Exercise Content	Time Budget	Pass Bar Language
L1 / entry	Single alert, clear evidence trail, one missing field	45–60 min	Reaches a defensible disposition; correctly identifies the missing field without prompting
L2 / experienced analyst	Alert with a genuine ambiguity (competing plausible explanations)	60–90 min	Reaches "unable to determine" appropriately, names the exact enrichment needed, or defends a confident call with correct reasoning
Senior / team-lead track	Alert plus a coaching component (a fictional L1's incorrect disposition to review)	75–90 min	Correctly re-disposition <i>and</i> gives feedback that would actually help the fictional L1 improve, not just a corrected answer

Field Test Setup: A new or revised log-triage exercise exists on paper but has never been run against anyone. **Action:** Before offering it to a single candidate, run it, proctored, against two current analysts already known to be strong performers and one analyst who is currently on a performance-improvement track or recently struggled with a similar real ticket. **Expected result:** The two strong performers should clear the intended pass bar comfortably and within the time budget; the struggling analyst's result should visibly resemble the specific gap already documented in their real performance record. If a strong performer fails the exercise, or the struggling analyst sails through it, the exercise is measuring something other than the skill it's named for — fix the exercise before it ever touches a candidate, not after a bad hire six months later.

4. Scoring, calibration, and the pass bar

4.1 Anchoring the rubric to observable behavior

[HR/PEOPLE]

Every rubric line should answer one question: what would I actually see or hear that earns this score? "Good technical judgment — 4/5" is not a rubric line; "correctly flags the missing source-attribution field and requests the specific enrichment needed to resolve it — 4/5, one point off for not stating why the alternative explanation was ruled out" is. The difference isn't wordiness for its own sake — it's whether two different interviewers, watching the same candidate, land on the same score without discussing it first.

4.2 Running a calibration session before the first real candidate

[SENIOR MANAGER]

Calibration means multiple interviewers score the same recorded or simulated candidate performance independently, then compare scores and discuss any gap before the rubric is used on a real person. This is the hiring-side twin of the calibration practice this book develops fully for ongoing quality assurance in Part 15 — same mechanism (independent scoring, then reconciliation), different unit of analysis (one candidate's interview performance instead of one analyst's ticket).

Manager's Note *Calibrate on a recording everyone already agrees was a strong pass before calibrating on a genuinely borderline one. It's the fastest way to discover two interviewers don't actually share a definition of "clears the bar" yet — better to find that out on an easy case than in a live debrief where a real candidate's offer is riding on the disagreement.*

4.3 Setting the pass bar and defending it later

[SENIOR MANAGER]

The pass bar is a real tradeoff between two costs a manager has to name out loud, not split the difference on by instinct: a bar set too low produces bad hires that cost months of remediation, coaching time, and — if it goes badly enough — a termination and a re-hire cycle; a bar set too high produces false negatives, rejecting candidates who would have succeeded, in an analyst market Part 7 already establishes as persistently tight at the mid-level. Neither failure is visible in the moment a hiring decision is made. Bad hires reveal themselves three to six months in; false negatives never reveal themselves at all, because the rejected candidate simply goes and succeeds somewhere else, invisibly, and the hiring program never learns it made a mistake.

COMPOSITE CASE EXAMPLE (CASE-0801) — the following merges patterns from multiple structured-interview redesigns into one illustrative narrative; it is not a single traceable organization.

A 22-person SOC ran an unstructured loop for years: one 45-minute conversational interview per candidate, no shared rubric, a single interviewer's judgment call on each hire. Over an 18-month hiring window, 68% of new hires required a formal performance-improvement plan within their first six months — most for the same underlying gap, an inability to reach a defensible disposition on an ambiguous alert without heavy hand-holding, which the unstructured interview had never actually tested. Average time-to-fill was 47 days. Not every PIP ended in a loss — a few analysts improved and stayed — but 6 of the 11 new hires in that window ultimately failed the PIP and had to be backfilled. Using a rough fully-loaded cost of \$45,000 per bad hire (recruiting spend, ramp-up time, a struggling analyst's real handle-time drag on the queue, and eventual backfill), the SOC's own back-of-envelope

math put the unstructured loop's hidden cost at roughly \$275,000 across that window, against a headcount of eleven new hires.

The SOC replaced the single interview with the three-stage loop from §2.2: a screen, a 75-minute log-triage exercise borrowed and sanitized from its own detection library, a 15-minute mock escalation scored against the SOC Playbook Handbook's escalation-quality criteria, and a calibrated three-person panel scoring independently before debrief. Time-to-fill rose slightly, to 52 days — the extra stages take real calendar time — but the six-month performance-improvement-plan rate for the next hiring cohort dropped to 15%. The SOC's own estimate is that the five-day increase in time-to-fill cost roughly \$8,000 per hire in extended coverage-gap overtime, against a bad-hire-cost reduction of over \$30,000 per avoided PIP-track hire — a trade the SOC judged worth making, though it required defending a slower hiring process to a leadership team that had been measuring the recruiting function on time-to-fill alone.

What Would Change My Mind *This part treats a structured, calibrated practical assessment as a materially better predictor of on-the-job triage performance than an unstructured conversational interview. If a SOC running a genuinely unstructured loop showed a six-month performance-improvement-plan rate at or below what CASE-0801's structured loop achieved — not a self-reported "we hire good people," but a tracked rate — that would undercut this part's central claim that structure itself is doing the predictive work, rather than some other factor (a tighter labor market that quarter, a better recruiter, a smaller hiring volume) coinciding with whichever loop happened to be in place at the time.*

5. De-biasing the interview and assessment process

5.1 Where bias actually enters the loop

[HR/PEOPLE] Bias in a SOC interview loop rarely shows up as an interviewer consciously favoring one group over another. It shows up structurally, in places a well-intentioned process still lets it operate unchallenged:

- **Certification halo carried over from the resume screen.** Part 7 already names the risk of over-indexing on certifications during sourcing; the same pattern re-enters the interview when a panelist keeps steering questions back toward a candidate's certification instead of the actual exercise output, effectively re-testing the resume instead of the job.
- **Order and recency effects in the panel debrief.** The candidate discussed last, or the interviewer who speaks first in the debrief, disproportionately shapes the outcome — the exact failure the People Risk Trap in §2.3 names.
- **Unpaid, open-ended take-home exercises.** Covered in full in §5.3, because the mechanism and the fix are specific enough to deserve their own subsection.
- **Panel composition.** A panel drawn entirely from one specialty, one tenure band, or one demographic group narrows what "looks like a good hire" means without anyone deciding that on purpose.

5.2 Structural fixes that hold up under scrutiny

[HR/PEOPLE] Each bias source above has a structural fix that doesn't depend on any individual interviewer's good judgment in the moment, which is the point — a fix that relies on everyone remembering to be fair on a busy hiring week doesn't survive a busy hiring week.

Bias Source	Mechanism	Structural Fix
Certification halo	Panelist re-tests the resume instead of the exercise	Rubric has no "certifications" line past the initial screen; scorecard fields only reference exercise and behavioral evidence
Debrief order/recency	Last or loudest opinion anchors the group	Written independent scores locked before debrief opens (\$2.3)
Unpaid take-home	Filters for candidates with spare unstructured time	Proctored, time-boxed exercise or a paid stipend (\$5.3)
Narrow panel composition	"Good hire" pattern-matches to the panel's own profile	Rotate panel membership; require at least one panelist outside the hiring manager's own reporting line

People Risk Trap *A panel that always includes the same two or three senior analysts, because they're the most technically credible reviewers, quietly trains the hiring bar to reproduce whoever is already on the team — including whatever gaps that team already has. The fix: rotate panel seats across a wider pool of qualified reviewers on a fixed schedule, and treat "who's on the panel this cycle" as a deliberate roster decision, not a default to whoever's available.*

5.3 The equity cost of an unpaid take-home assessment

[HR/PEOPLE] A four-hour unpaid take-home exercise, due within a week, does not test the same thing for every candidate. A candidate with no caregiving responsibilities and no second job can carve out four uninterrupted hours on a weekday evening; a candidate working a second job, or a single parent, or someone in a household where evening hours are already committed, either can't complete it at all or has to complete it in fragments late at night, degraded by fatigue in a way that has nothing to do with their actual triage skill. The exercise ends up screening for available unstructured personal time as much as it screens for the skill it claims to measure — and the population it screens out skews toward exactly the non-traditional and career-changer candidates Part 7 identifies as the mid-level market's most under-tapped source.

Two fixes address this without weakening the exercise itself: run it proctored and time-boxed inside the loop instead of sent home unpaid, or, where a take-home format is genuinely necessary (distributed candidates, scheduling constraints), pay a stipend for the time — a modest flat rate, commonly in the \$100–\$200 range for a two-to-three-hour exercise, signals the SOC is buying the candidate's time rather than assuming it's free, and removes the "who can afford to spend an unpaid evening on this" filter entirely.

People Risk Trap Scoring a take-home exercise on how quickly it was returned — treating a same-day submission as a stronger signal than one that arrives on the deadline — punishes exactly the candidates described above, who may need the full window precisely because they're fitting the exercise around other obligations, not because they're less capable. The fix: score only the submission's content against the rubric; never let elapsed time factor into the score unless the exercise is explicitly, and fairly, timed and proctored for everyone.

6. Running the loop end to end

6.1 Sequencing and the calibration gate before a decision

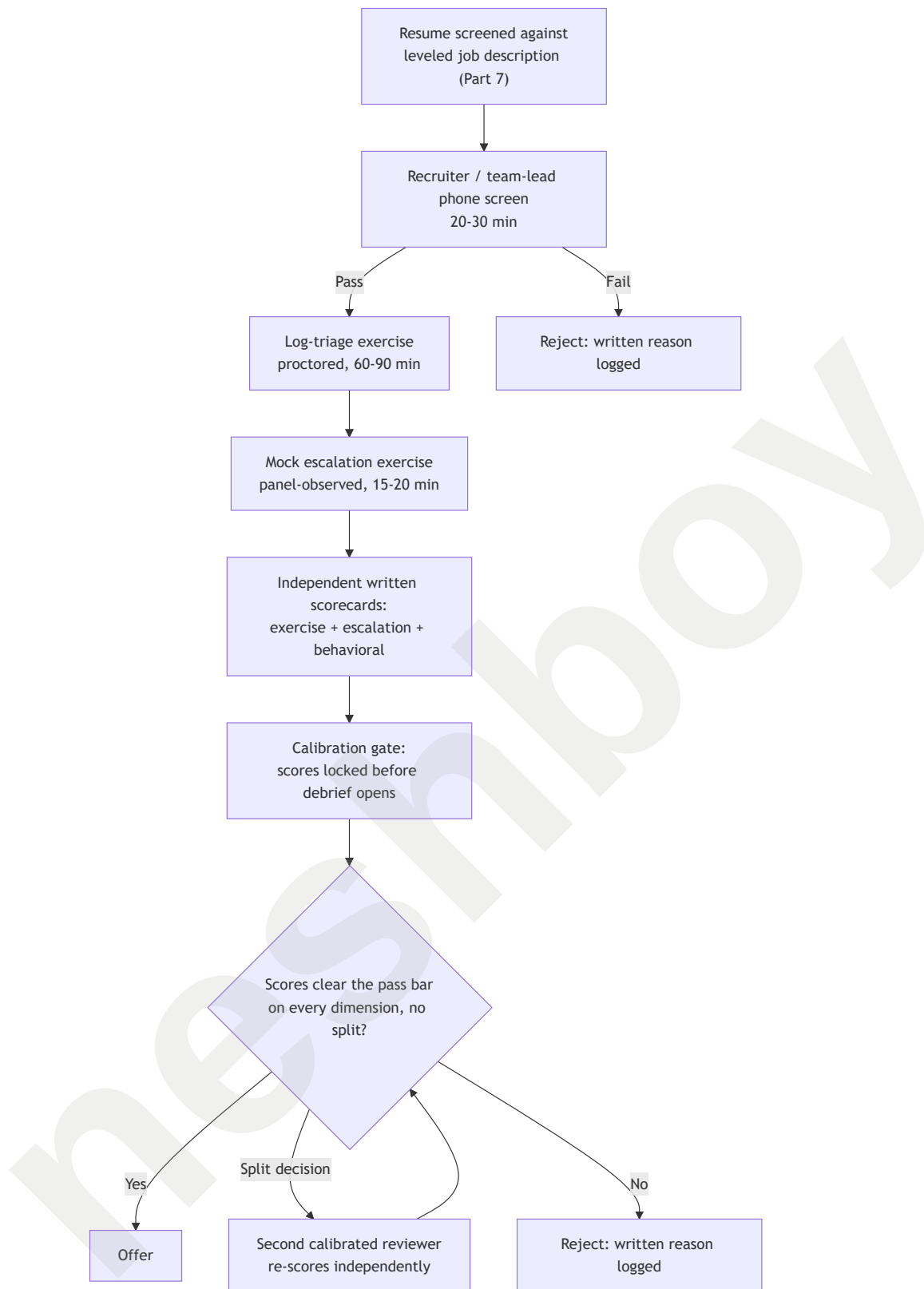
[SENIOR MANAGER] Figure 8.1 lays out the full loop from §2 through §5 as one sequence, with the calibration gate — independent written scores, locked before debrief — placed explicitly before any pass/fail decision, not folded into the debrief conversation itself.

Figure 8.1 — Structured SOC technical-interview loop, screen to offer decision. CONCEPTUAL.

Illustrates the sequencing this part recommends: independent scoring happens before group discussion at every gate, and a split decision routes to a second calibrated reviewer rather than being resolved by debate alone. This is a process model of the recommended loop, not a capture of any single organization's actual applicant-tracking workflow. Diagram ID [FIG-0801](#).

flowchart TD

```
A["Resume screened against\nleveled job description (Part 7)"] --> B["Recruiter / team-lead phone\nscreen\n20-30 min"]
B -->|Pass| C["Log-triage exercise\nproctored, 60-90 min"]
B -->|Fail| Z1["Reject: written reason logged"]
C --> D["Mock escalation exercise\npanel-observed, 15-20 min"]
D --> E["Independent written scorecards:\nexercise + escalation + behavioral"]
E --> F["Calibration gate:\nscores locked before debrief opens"]
F --> G{"Scores clear the pass bar\nnon every dimension, no split?"}
G -->|Yes| H["Offer"]
G -->|Split decision| I["Second calibrated reviewer\nre-scores independently"]
G -->|No| Z2["Reject: written reason logged"]
I --> G
```

[SENIOR MANAGER]

The split-decision path matters as much as the pass/fail paths. A loop with no defined response to disagreement quietly defaults to whoever argues hardest in the debrief winning — the same recency-and-volume problem named in §2.3 and §5.1, just relocated to the very last step instead of removed from the process. Routing a genuine split to a second, independent, calibrated reviewer costs one more scoring pass and removes the debate from the decision entirely.

6.2 What "the assessment predicted this" looks like 90 days in

[SENIOR MANAGER] The loop's real test isn't the hiring decision — it's whether the specific gaps or strengths the assessment flagged show up, unsurprised, in the new hire's first 90 days. A candidate whose log-triage exercise showed strong reasoning but a habit of under-communicating context should, 90 days in, look like an analyst whose dispositions are usually right but whose escalation notes need coaching — not a surprise, but confirmation the exercise measured something real. Part 11 — Onboarding & Ramp-Up Programs is the natural place that 90-day picture gets built out into a structured ramp plan, and Part 10 — Competency Models & Skills Matrices is where the same observable-behavior language this part uses for a hiring rubric becomes the ongoing standard a promoted-in-place analyst is measured against. If the assessment's flagged gaps and the ramp-up period's actual coaching notes routinely have nothing to do with each other, that's the signal the assessment itself needs to be redesigned, not a signal that this particular hire was simply unlucky.

7. Where this goes next

[CONCEPT] This part assumed a leveled job description and a realistic preview already exist (Part 7) and borrowed its practical-assessment material and mock-escalation rubric from SOC Playbook Handbook, Part 27 — Escalation Quality and Detection Engineering Handbook V2's detection content rather than inventing either. What it built instead is the process wrapped around that material: a three-stage loop, a scorecard anchored to observable behavior, a calibration gate that locks scores before debrief, and structural fixes for the specific ways bias enters a SOC hiring process. The structured-interview scorecard and log-triage/mock-escalation exercise templates this part describes have their reusable, fillable form in Appendix A2 ([TMPL-0801](#)), built as the companion artifact to this part and Part 7. Part 10 picks the same observable-behavior language back up to define what "ready for L2" means on an ongoing basis, and Part 13 uses it again for promotion criteria — nothing past this point should need to redefine what a calibrated, bias-checked assessment looks like; it should just use one.

Cross-references

This part assumes Part 7 — Hiring & Sourcing Analysts (leveled job description, realistic job preview, the tight mid-level market and certification-over-indexing risk) and points forward to Part 10 — Competency Models & Skills Matrices, Part 11 — Onboarding & Ramp-Up Programs, Part 13 — Career Ladders & Promotion Criteria, and Part 15 — Quality Assurance Programs (the calibration mechanism reused for ongoing QA rather than hiring). Outside this book, it cites SOC Playbook Handbook, Part 27 — Escalation Quality (the mock-escalation rubric) and Detection Engineering Handbook V2, Part 1 — Detection Engineering Foundations and Parts 3–29 (source material and vocabulary for the log-triage exercise) — this part owns assessment methodology only, never the technical correctness of either volume's underlying content.

Part 9 — Queue Health & Workload Management

Why this part exists

[CONCEPT] A queue can hold 200 open tickets and be perfectly healthy. A queue can hold 15 and be in real trouble. Severity tells you how bad any single alert is if it turns out to be true; it says nothing about whether the team working the queue as a whole is keeping pace, falling behind gradually, or quietly drowning in tickets nobody has touched in two days. Those are two different questions, answered by two different instruments, and this part is about the second one only. SOC Playbook Handbook, Part 29 — Playbook Severity Model owns how any one alert gets scored; this part does not re-score alerts, and it does not touch severity logic at all. It measures the queue those alerts sit in — depth, age, arrival rate, clearance rate — as an operational health signal that can be dangerously wrong even when every individual severity score in the queue is correct.

[CONCEPT] The other question this part exists to answer is harder and more consequential: when a queue is visibly growing, is that a staffing problem or a detection-quality problem? Those two diagnoses point at completely different fixes — hire, or tune — and a manager who reaches for the wrong one either spends real budget on headcount that won't fix anything or leaves a genuinely understaffed team convinced that "better rules" will save them from a fix that only more people can deliver. Distinguishing the two is a workload-management judgment call this book owns; the technical mechanics of what "detection quality" actually measures — coverage, false-positive rate, detection debt — belong to Detection Engineering Handbook V2, Part 42 — Detection Quality, and this part cites that material rather than re-deriving it.

[CONCEPT] Three things this part deliberately does not do, each because another part or book already owns it and re-deriving it here would be exactly the kind of duplication this book's own production rules exist to prevent: it does not define MTTR or false-positive rate — those definitions live in SOC Playbook Handbook, Part 32 — Metrics, and every number in this part's worked examples uses them as already-defined inputs. Handle time is this book's own operational term rather than a Playbook Handbook one — Part 5 §2 owns that definition, and this part's worked examples use it as already defined there. It does not do the headcount arithmetic that turns a capacity gap into a hiring plan — that model, including its honest failure modes, is Part 5 — Headcount & Capacity Modeling's job, and this part's diagnosis feeds that model rather than replacing it. And it does not resolve the cases where the right call is genuinely unclear — a persistent queue-health problem that could plausibly be either a staffing gap or a detection-debt problem, with a resourcing decision that has to be made before the ambiguity resolves itself, is exactly the judgment-call territory Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty was built for, and this part hands off to it explicitly in §3.5.

1. Queue depth and backlog age are not severity

[CONCEPT] Two SOC's can report an identical severity mix — the same proportion of critical, high, medium, and low alerts — and be in entirely different operational condition, because severity mix says nothing about *when* those alerts get worked. A critical alert triaged within 12 minutes and a critical alert of identical content sitting untouched for 9 hours carry the same severity score and represent

completely different risk exposure. Queue health is the measure of that second dimension: not what's in the queue, but how the queue as a whole is moving.

[FRONTLINE MANAGER] The mistake this produces in practice is a shift lead who checks the dashboard, sees "no criticals open," and reports the queue as healthy — while 40 medium and low-severity tickets sit unopened from the previous shift, aging past their SLA windows one at a time with nobody noticing because no single one of them is the highest-severity item in the queue at any given moment. A queue-health view has to answer "how is the backlog moving" as a question separate from "what's the worst thing currently open," because a dashboard sorted by severity will always show you the worst thing first and never show you the slow accumulation happening underneath it.

Operational Reality *A severity-sorted queue view trains analysts to work top-down by severity, which is correct for any single shift but actively hides backlog accumulation at the medium and low end — the oldest ticket in most under-strain SOC's is rarely the highest-severity one, because high-severity tickets get pulled first regardless of age, and low-severity tickets are exactly the ones left to accumulate. If your only queue view is severity-sorted, you will not see a health problem until it's already large enough to start generating SLA-breach complaints from whoever the SLA is owed to.*

[CONCEPT] Queue health, then, needs its own metrics, tracked independently of severity, and reviewed on their own cadence rather than folded into a severity-first dashboard. The next section names the four that matter and what each one actually tells you that the others don't.

2. The four numbers that describe queue health

[FRONTLINE MANAGER] Four numbers, tracked together, describe queue health well enough to act on. Tracked separately, each one alone is misleading in a specific, predictable way.

2.1 Queue depth

[FRONTLINE MANAGER] Queue depth is the simplest number and the least useful one on its own: the count of open tickets right now. It's the number most dashboards lead with because it's the easiest to compute, and it's the number most likely to make a genuinely healthy queue look alarming or a genuinely unhealthy one look fine. A depth of 150 tickets is unremarkable for a large enterprise SOC processing thousands of daily events and alarming for a four-analyst team that normally sits at 20; the number means nothing without a baseline for that specific team, and it means nothing at all about *how long* those 150 tickets have been sitting there.

2.2 Backlog age and the aging distribution

[FRONTLINE MANAGER] Backlog age — how long the oldest open tickets have been open — is the number that actually predicts trouble, and the single number worth watching if a manager can only watch one. A queue with a depth of 60 and a p90 age of 20 minutes is a team keeping up with a busy

day. A queue with a depth of 60 and a p90 age of 30 hours is a team that has lost control of the backlog, even though the depth number alone looks identical to the first case at a casual glance.

Manager's Note *If you can only watch one queue-health number on a Monday-morning check-in, watch the p90 backlog age, not the depth count. Ten tickets that are 2 minutes old is a non-event. Ten tickets that are 30 hours old is a slow-burning SLA breach sitting quietly in a queue that might report the exact same depth number either way. Depth tells you how much work is sitting there; age tells you whether anyone is actually getting to it.*

[FRONTLINE MANAGER] The full aging distribution matters more than any single percentile, because a p50 (median) age can look fine while a p90 or p99 tells a much worse story about a specific subset of the backlog that's being systematically deprioritized — usually the lower-severity tickets that never win the top-down triage fight described in §1. Track age in buckets (under 1 hour, 1–4 hours, 4–12 hours, 12–24 hours, over 24 hours) rather than a single average, and watch the proportion of the backlog sitting in the two oldest buckets specifically. That proportion, not the average, is the number that should trigger a conversation.

2.3 Arrival rate and clearance rate

[SENIOR MANAGER] Arrival rate is how many alerts land in the queue per unit time; clearance rate is how many the team closes in that same window. Depth and age are both downstream consequences of the relationship between these two — a queue only grows when arrival rate exceeds clearance rate, and it only shrinks when the reverse is true. This relationship has a formal name — Little's Law, from queuing theory — and a manager doesn't need the underlying math to use the practical version of it: **average time an item spends in the queue is proportional to the number of items in the queue, divided by the rate at which items leave.** Practically, that means a growing p90 backlog age is never a mystery once you have arrival rate and clearance rate for the same window — it's arithmetic, not a diagnosis in itself. The diagnosis is *why* arrival rate rose or clearance rate fell, which is §3's job.

Operational Reality *Most queue dashboards report arrival rate and depth but not clearance rate, because clearance rate requires tracking closures over a matching time window rather than a single snapshot, and most ticketing systems make snapshot counts easier to pull than rate-over-time counts. The result is a dashboard that can tell you the queue is growing but can't tell you whether that's because more work is arriving or because less work is getting done — the two explanations require completely different fixes, and a dashboard that only shows depth leaves that distinction to guesswork.*

3. Diagnosing a growing queue: staffing problem or detection-quality problem

[SENIOR MANAGER] A queue that's growing week over week has, in practice, one of a small number of root causes, and the fix for each is different enough that guessing wrong is expensive. The two most

consequential to distinguish are a genuine capacity gap — the team needs more people or more hours, full stop — and a detection-quality problem, where a small number of noisy or badly tuned rules are generating volume that no amount of headcount will ever fully absorb, because the rules keep generating more of it. Hiring against a detection-quality problem adds headcount that spends its days triaging the same bad rule faster; tuning a rule that isn't the actual cause of a genuine capacity shortfall leaves the real gap exactly where it was.

3.1 Decompose the backlog before diagnosing anything

[SENIOR MANAGER] The fastest, cheapest diagnostic step — and the one skipped most often under time pressure — is decomposing backlog growth by its source: which specific detection rules, alert types, or upstream systems account for the increase, rather than treating the growth as one undifferentiated number. Pull the tickets that make up the increase (not the whole backlog — just the delta over the period the growth happened) and group them by rule ID or alert type. Two outcomes are possible, and they point in opposite directions.

[SENIOR MANAGER] If a small number of rules — often one to three — account for most of the increase, that's evidence pointing toward a detection-quality explanation, and the next question is that rule's post-triage false-positive rate, which Detection Engineering Handbook V2, Part 42 — Detection Quality treats as a first-class measurement rather than an afterthought. If the increase is broad-based across dozens of unrelated rules and sources with no single one dominating, a detection-quality explanation becomes much less likely — noisy rules concentrate; genuine new attack surface or business growth spreads out.

Blind Spot A backlog decomposed by rule ID can still hide the real pattern if several rules are all reacting to the same upstream root cause — a misconfigured log source flooding duplicate events, a newly onboarded business unit whose normal activity trips five unrelated rules at once, or a single upstream API change that degrades enrichment quality across every rule that depends on it. "No single rule dominates" is not the same finding as "no single cause dominates," and a decomposition that stops at the rule level will misread a one-cause, many-symptom problem as broad-based organic growth every time.

3.2 The diagnostic decision tree

[SENIOR MANAGER] Figure 9.1 lays out the decision sequence: decompose first, then branch on concentration, then branch again on false-positive rate or on whether the broad-based growth traces to a known cause. It's a starting sequence, not a replacement for judgment on any case that sits near a branch boundary — §3.5 covers what to do when a case genuinely won't resolve cleanly to one branch.

flowchart TD

```

A["Queue depth or backlog age\n trending up for 2+ weeks"] --> B{"Decompose the increase:\nconcentrated in 1-3 rules,\nor broad-based?"}
B -->|"Concentrated"| C{"Post-triage false-positive\nrate on those rules?"}
B -->|"Broad-based"| F{"Arrival rate rising,\nor clearance rate falling\nwith arrival flat?"}
C -->|"High (roughly 80%+)"| D["Detection-quality problem.\nSee DEH Part 42 and Part 38\nbefore adding headcount."]
C -->|"Normal / low"| E["Likely genuine new threat\nvolume or a coverage gain --\nconfirm with detection\nengineering, then size the\nadded load with Part 5."]
F -->|"Arrival rising"| G{"Traces to a known change --\nonboarding, M&A, new asset\n class, new business unit?"}
F -->|"Clearance falling,\narrival flat"| J{"Tied to headcount loss,\nattribution, or a tenure/\nskill-mix shift?"}
G -->|"Yes"| H["Capacity gap tied to a known\ndriver. Size it with Part 5;\nconsider phasing the\nonboarding."]
G -->|"No known driver"| I["Unexplained volume growth --\ntreat as hunt-worthy, not\njust a staffing input."]
J -->|"Yes"| K["Staffing/skill problem.\nSee Part 16 and Part 18."]
J -->|"No"| L["Process friction -- check\nshift balance (Sec.4) and tier\nhand-off criteria (Part 3)."]

```

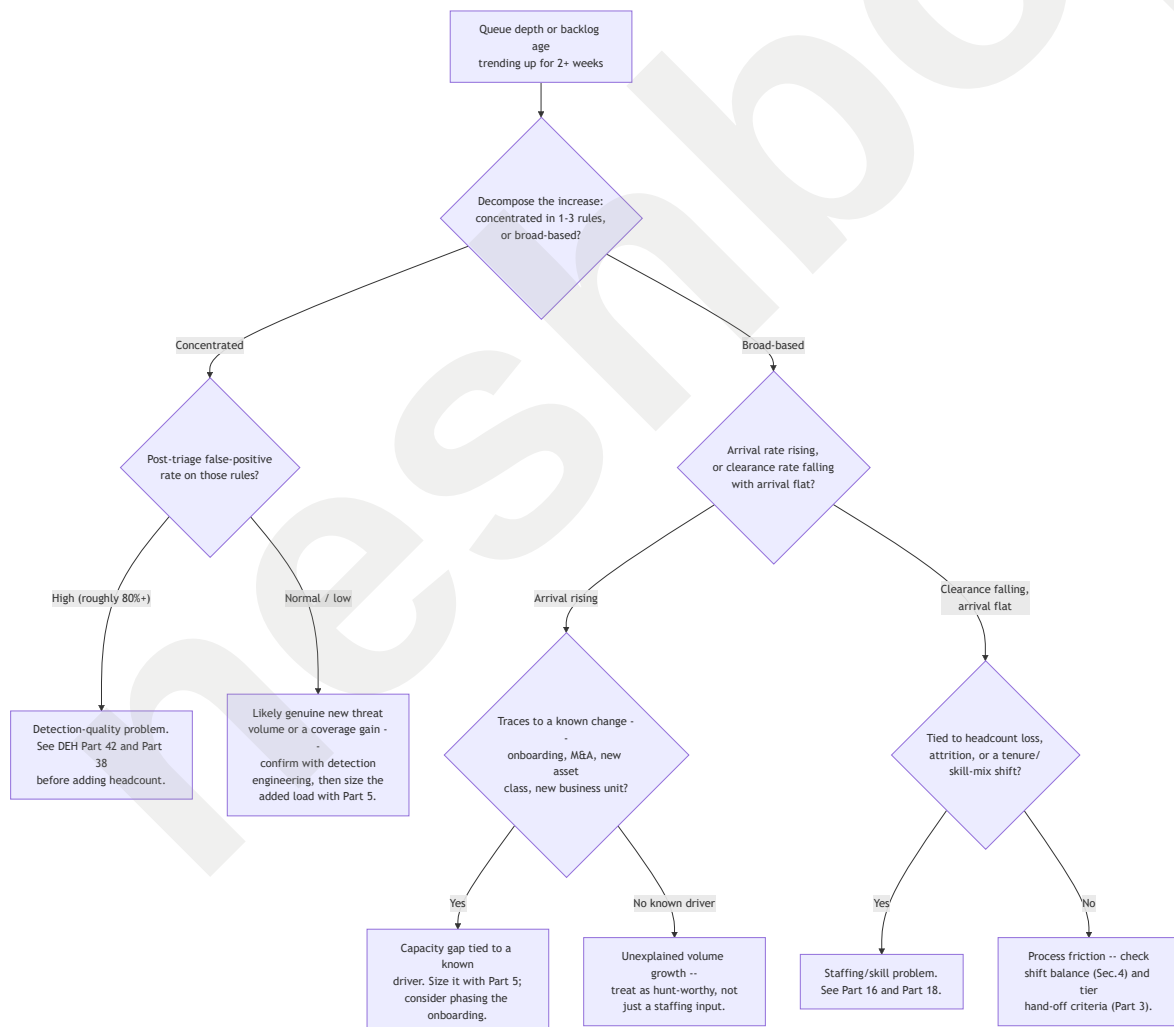


Figure 9.1 — Queue growth diagnostic decision tree. CONCEPTUAL. FIG-0901 renders the decision sequence described in §3.1–§3.2 — decompose, branch on concentration, then branch on false-positive

rate or on whether broad-based growth traces to a known cause. It supports every routing call made in §3.2's decision tree and the worked examples in §3.3–§3.5, none of which are a capture of any single organization's actual triage workflow.

Cross-Book Pointer *This part does not define false-positive rate or explain how to reduce a noisy rule's trigger volume — that's a detection-engineering fix, not a staffing one. See Detection Engineering Handbook V2, Part 38 — False Positive Engineering for the tuning mechanics, and Part 42 — Detection Quality for how coverage and quality are measured well enough to tell "this rule is genuinely catching more real activity" apart from "this rule got noisier." Come back here once you know which one you're looking at — that's the fork this part's decision tree depends on.*

3.3 Worked example — "we need more people"

COMPOSITE CASE EXAMPLE — CASE-0901. Constructed from recurring patterns seen across mid-market in-house SOC's during cloud-migration waves; no single organization is identifiable, and all figures below are illustrative.

[SENIOR MANAGER] A nine-analyst in-house SOC covering three 8-hour shifts (three analysts per shift) ran a steady-state queue depth around 40 open tickets for months, with a p90 backlog age under 3 hours. Over a 6-week window, the organization migrated roughly 40 new cloud workloads into production, and the queue changed shape: depth climbed to a sustained 165+ open tickets, and p90 backlog age rose from 3 hours to 26 hours. The medium-severity SLA target was 4 hours; the proportion of tickets breaching that SLA rose from about 4% to about 38% over the same window.

[SENIOR MANAGER] The manager's first instinct was to suspect a bad rule shipped alongside the migration. A decomposition (§3.1) ruled that out quickly: the increase was broad-based across more than 30 distinct rule IDs, with the single largest contributor accounting for only about 9% of the added volume — nowhere near the concentration §3.2's decision tree treats as a detection-quality signal. Average handle time per ticket stayed flat at roughly 14 minutes across the whole window, which also ruled out a skill or training explanation — if handle time had crept up instead, that would point at Part 16's territory (a coaching or competency gap), not this part's.

CONCEPTUAL SAMPLE – illustrative numbers, not sourced benchmark data

Existing analyst capacity: 9 analysts x 3 shifts, ~6.5 productive hours/analyst/shift
after Part 5's shrinkage deductions (PTO, training, breaks)
= ~58.5 analyst-hours/day of baseline capacity

Baseline arrival rate: ~250 alerts/day

Average handle time: 14 minutes (flat, before and after)

Baseline demand = 250 alerts/day x 14 min = 3,500 minutes/day = ~58.3 analyst-hours/day
-- essentially equal to the ~58.5-hour baseline capacity above, which is
what "steady-state, no slack" actually looks like as arithmetic

Post-migration arrival: ~310 alerts/day (+24%)

Added daily analyst-minutes needed = 60 alerts/day x 14 min = 840 minutes/day
= ~14 analyst-hours/day
= a gap the baseline's ~58.5-hour capacity has no
room to absorb without breaching age targets

[SENIOR MANAGER] Arrival rate rose 24% with handle time flat and no rule concentration — by Little's Law's practical version (§2.3), a queue under those conditions grows until either arrival rate drops or clearance capacity rises; nothing about the existing team's process was failing, it simply had no spare capacity to absorb a permanent 24% volume increase. The fix that actually worked combined two moves rather than one: two additional analysts were brought on (on a temp-to-perm basis to preserve flexibility if the migration's later phases turned out smaller than planned), adding roughly 13 analyst-hours/day — enough to close nearly all of the ~14-hour/day gap on its own — and the remaining onboarding tranches were renegotiated with the migration team to phase in over 2 months instead of 3 weeks, smoothing over the small residual and avoiding a sustained SLA breach. Backlog age returned to baseline within about 5 weeks of the headcount addition landing.

3.4 Worked example — "we need fewer, better rules"

COMPOSITE CASE EXAMPLE — CASE-0902. Constructed from recurring patterns seen when an upstream identity or network provider changes behavior underneath an existing detection; no single organization is identifiable, and all figures below are illustrative.

[SENIOR MANAGER] A different SOC saw the same surface symptom — queue depth and backlog age both climbing over roughly 3 weeks — and the manager's first instinct, reasonably, was to request two additional headcount to absorb the volume. A decomposition first: one rule, an anomalous-sign-in-location ("impossible travel") detection, accounted for 61% of the added ticket volume. A VPN provider used by a large fraction of the remote workforce had changed how it reported egress-IP geolocation, and the existing rule's travel-velocity threshold started tripping on ordinary remote logins that looked, to the rule, like physically impossible travel between sessions.

CONCEPTUAL SAMPLE – illustrative numbers, not sourced benchmark data

Rule volume before the VPN provider's change:	~40 alerts/day
Rule volume after the change:	~310 alerts/day
Average handle time for this specific rule:	3.5 minutes (fast; analysts learned the pattern quickly)
Added daily analyst-load from this rule alone:	270 extra alerts/day x 3.5 min = ~16 analyst-hours/day
Post-triage false-positive rate on this rule during the affected window:	~97% (3% escalated for further review; zero confirmed true positives among those in the sampled period)

[SENIOR MANAGER] Sixteen analyst-hours a day of added load looks, from a pure capacity-math view, almost identical to a genuine staffing gap — it's close enough in magnitude to Example 1's ~14-hour/day gap that the capacity math alone can't tell the two problems apart. The decomposition step is what kept this from becoming a headcount request: concentration in one rule (61% of the increase) plus a 97% post-triage false-positive rate is squarely the detection-quality branch of Figure 9.1, not the capacity branch. The manager routed the finding to detection engineering instead of opening a requisition. A suppression rule scoped to the VPN provider's known egress-IP ranges, plus a retuned travel-velocity threshold, went in within 4 days; volume from that rule dropped back to roughly 35 alerts/day, and queue depth normalized without any headcount change.

[SENIOR MANAGER] The cost comparison is worth stating plainly, because it's the argument that actually moves a budget conversation: two additional analysts at a fully loaded cost of roughly \$130,000 each run close to \$260,000 a year in ongoing spend; the detection-engineering fix cost approximately two engineer-days, on the order of \$1,500–\$2,000 in loaded time. Neither number is a universal constant — treat both as this composite's illustrative figures, not a benchmark — but the ratio is the point: a queue-health problem that looks like a capacity gap on the surface can be two orders of magnitude cheaper to fix as a tuning problem, and the only way to know which one you're looking at is the decomposition step in §3.1, done before the headcount request goes anywhere.

3.5 When it's genuinely both

[SENIOR MANAGER] Some queues fail this decision tree cleanly — decompose the backlog and you find real concentration in a handful of noisy rules *and* a real underlying capacity gap once those rules are tuned down, because the team was already thin before the noise arrived. Fixing the rule buys headroom; it doesn't manufacture a ninth or tenth analyst out of nowhere if the team was already running at four when workload modeling says it needs six. Treat that combination as two separate findings with two separate fixes on two separate timelines — the tuning fix is fast and should happen regardless of the headcount decision, and the headcount decision should be made on the *post-tuning* volume, not the inflated volume the noisy rule was contributing. Making a hiring decision against pre-tuning numbers routinely results in adding headcount sized to solve a problem that a same-week detection fix was about to make disappear.

What Would Change My Mind This section treats backlog concentration in a small number of rules, paired with a high post-triage false-positive rate, as reliable evidence that a queue-health problem is (at least partly) a detection-quality problem rather than a pure staffing gap. If teams applying this decomposition routinely found that concentrated, high-FP-rate rules turned out to be catching a real and growing volume of true positives once investigated properly — meaning the concentration reflected a genuine new threat pattern rather than noise — that would weaken the heuristic and argue for treating rule concentration as, at most, a prompt to investigate rather than a diagnosis in itself.

[SENIOR MANAGER] A resourcing decision made under this exact kind of ambiguity — tune first or hire first, when the honest answer is "we don't fully know yet and the queue is breaching SLA today" — is the book's central judgment-call territory. Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty works through cases where that call was genuinely unclear at the time it had to be made; this part's job stops at getting the diagnosis as clear as the decomposition can make it, not at making the call when the decomposition itself doesn't fully resolve.

4. Workload balancing across shifts and tiers

[FRONTLINE MANAGER] Queue-health numbers describe the whole queue. They don't, by themselves, tell you whether the load sitting behind those numbers is distributed evenly across the team working it — and an unhealthy queue-health trend line is exactly as often caused by one shift or one tier quietly absorbing more than its share as by a genuine aggregate shortfall.

4.1 Balancing across shifts

[FRONTLINE MANAGER] Part 6 — Shift Pattern & Coverage Design owns the design of the shift pattern itself and the mechanics of what information has to transfer at a handoff. This part's concern is narrower and sits upstream of both: the live, day-to-day decision of how much open backlog is acceptable to carry across a shift boundary before the shift pattern itself needs to change, and whether one specific shift is systematically absorbing more volume or higher-difficulty tickets than the others without anyone noticing, because the aggregate 24-hour queue-health numbers average the imbalance away.

Blind Spot A 24-hour aggregate backlog-age number can look perfectly healthy while hiding a night shift that is running 3 times over its fair share of the aging distribution and a day shift running comfortably under. Averaging across shifts is exactly the operation that erases this — three shifts each carrying a very different load can average out to a number that looks like "one team, evenly loaded," and a manager who only checks the 24-hour rollup will never see the night shift quietly drowning until someone on it burns out or quits, at which point Part 17 and Part 18's territory takes over from this part's.

[FRONTLINE MANAGER] The practical check is cheap and worth running weekly, not just after a complaint: pull backlog age and depth broken out by the shift the ticket was assigned to (or the shift

active when it aged into the oldest bucket, if ownership transfers), not just the aggregate. A gap between shifts wide enough to matter — one shift's p90 age running double another's for more than a week or two — is a workload-balance problem worth acting on directly, independent of whether the aggregate number looks fine.

4.2 Balancing across tiers

[FRONTLINE MANAGER] Part 3 — Tiering Models: L1/L2/L3 and Beyond owns what each tier is responsible for and the criteria that govern when a ticket moves from one to the next; SOC Playbook Handbook, Part 27 — Escalation Quality owns whether any single hand-off across that boundary was done well. Neither of those answers this part's question: whether the *volume* of work sitting at each tier is balanced, independent of whether every individual escalation across the boundary met quality bar. A Tier 1 queue can be executing textbook-quality escalations, one at a time, while still running structurally over capacity because Tier 2 has more slack than Tier 1 does and nobody has adjusted the escalation criteria to route more of the ambiguous middle ground upward while the imbalance lasts.

[SENIOR MANAGER] The workable lever here is temporary, not structural: loosening escalation criteria for a bounded period so more borderline tickets route to a tier with spare capacity, or pulling a Tier 2 analyst down to help triage a Tier 1 backlog surge directly, reversed once the imbalance clears. Treat this as a pressure-release valve, not a standing policy — a "temporary" loosened escalation criterion that's never tightened back up quietly becomes the new tier boundary by default, and six months later nobody remembers it was supposed to be temporary or why the boundary moved.

5. Surge handling

[SENIOR MANAGER] A surge is a sharp, usually short-lived spike in arrival rate — a mass-phishing campaign, a widely reported vulnerability driving a scan-and-triage wave, a marketing-driven false-positive storm from a new external-facing campaign — as distinct from the structural, sustained growth Example 1 in §3.3 walked through. The distinction matters because the fix is different: a structural gap needs headcount or tuning that persists; a surge needs a response that can be stood up fast and stood back down just as fast, because paying for permanent capacity to cover a one-week spike is a standing cost against a temporary problem.

Cross-Book Pointer This part covers surge handling for a routine volume spike with an identifiable, bounded cause — a phishing wave, a scan wave, a noisy campaign. It does not cover the manager's role during a live, declared major incident, where staffing surge decisions are made under a different kind of pressure and against a different decision authority. That's Part 28 — The Manager's Role in a Major Incident, which also covers executive communication cadence and retainer activation that this part's routine-surge toolkit has no reason to touch.

5.1 Recognizing a surge before committing to a response

[SENIOR MANAGER] The tell that distinguishes a surge from the start of a structural decline is the decomposition from §3.1, run early rather than after the fact: a surge is almost always concentrated (one

campaign, one CVE, one external event) and almost always has an identifiable end condition (the campaign gets blocked at the gateway, the CVE gets patched fleet-wide, the news cycle moves on). A structural gap, by contrast, usually has no end condition at all — it just persists until someone changes headcount or tuning. Committing a surge-response tactic (temporary overtime, MSSP overflow) against what turns out to be a structural gap just delays the real fix at real cost; committing a structural fix (a permanent hire) against what turns out to be a 10-day surge leaves a fully loaded headcount cost with nothing to do once the surge ends.

5.2 A surge-response toolkit

[SENIOR MANAGER]

The table below maps each standing option for covering a genuine short-term surge — not a structural gap — to how fast it can be stood up, what it costs relative to a permanent hire, and where it stops working; pick based on how long the spike is expected to last and how much lead time exists before it starts.

Option	Lead Time	Relative Cost	Best For	Where It Breaks Down
Pull from an adjacent shift or tier	Hours	Low — existing headcount, no new spend	A spike lasting under a week, concentrated in hours when another shift has slack	Runs the lending shift/tier thin; unsustainable past 1-2 weeks without creating a second surge
Voluntary overtime	Hours to 1 day	Moderate — overtime premium on existing staff	A spike with a known, short end date (days to 2 weeks)	Fatigue risk climbs fast past 2 consecutive weeks; ties directly into Part 17's burnout territory
Temporary/contract surge staff	1-2 weeks (sourcing + ramp)	Moderate-high — agency premium plus ramp-up drag on training a new person for a short engagement	A spike expected to last 3+ weeks, or a known recurring seasonal pattern	Ramp-up time can exceed the surge itself for anything under ~3 weeks; ineffective for a sudden, short spike
MSSP/vendor overflow burst capacity	Days, if a contract clause already exists; months, if not	High per-alert during burst, but zero standing cost between bursts	Organizations with an existing co-managed or hybrid model (Part 2) and a pre-negotiated burst clause (Part 22)	Useless without the contract clause already in place — negotiating one mid-surge is too slow to help this surge
Temporarily raise the triage/escalation bar	Immediate	Low direct cost, real risk cost	A true emergency spike where nothing else can be stood up fast enough	Defers real work rather than absorbing it; every ticket the raised bar defers still has to get worked eventually, usually as a backlog spike right after the surge "ends"

Operational Reality *The MSSP overflow row only works for organizations that negotiated a burst-capacity clause before the surge started — a fact most managers discover the hard way, mid-surge, when they call the MSSP account team and learn the current contract has no mechanism for temporary volume above the committed baseline without a new statement of work that takes longer to execute than the surge itself will last. Part 22 — MSSP & Managed-Service Contract Management covers negotiating that clause in advance; this part can only tell you it needs to already exist.*

[SENIOR MANAGER] The row most often reached for under real pressure — temporarily raising the triage or escalation bar — deserves one more sentence of caution, because it's the cheapest option on the table and the most tempting for exactly that reason: it doesn't absorb the surge's workload, it defers it. Every ticket that a raised bar pushes past without full triage is still sitting somewhere, aging, and it reappears as a backlog spike the moment the bar comes back down — which means this option should be reserved for the shortest, sharpest spikes, paired with an explicit plan for working through the deferred set once the surge itself clears, not treated as a general-purpose pressure release.

6. Building a queue-health scorecard your team actually watches

[FRONTLINE MANAGER] A queue-health program that lives only in an ad hoc "does this feel bad today" judgment call doesn't survive a shift-lead turnover, and a queue-health program that lives only in a dense dashboard nobody opens doesn't survive contact with a busy week either. The table below is a minimum viable scorecard, mapping four signals to a plain-language read on each and the next concrete step a bad reading should trigger — meant to be checked on a fixed cadence (daily for a shift lead, weekly for a SOC manager reviewing the trend) rather than only when someone already suspects a problem.

Signal	Healthy Pattern	Warning Pattern	Next Step If Warning Persists 2+ Weeks
Queue depth	Stable around an established baseline for this team	Sustained rise with no return to baseline within days	Run the §3.1 decomposition before anything else
Backlog age (p90)	Well inside the tightest SLA target this team owns	Approaching or exceeding the SLA target for a growing share of tickets	Check age distribution by shift (§4.1) and by tier (§4.2) before assuming it's aggregate
Arrival rate vs. clearance rate	Roughly matched, week over week	Arrival consistently outpacing clearance	Decompose by source (§3.1); this is the pairing that should trigger the diagnostic tree in Figure 9.1
Per-shift/per-tier age variance	Similar age distribution across shifts and tiers	One shift or tier's p90 running well above the others	Investigate workload balance (§4) before assuming an aggregate capacity gap

Blind Spot A scorecard checked weekly will still miss a fast surge that spikes and clears inside a 3-4 day window between check-ins — the kind of spike §5 covers. A weekly cadence catches structural drift; it is the wrong instrument for catching a sharp surge in time to respond to it while it's happening, which is why surge recognition (§5.1) has to run on a much shorter loop, closer to daily, than the structural scorecard here does.

7. Where this goes next

[CONCEPT] The diagnostic habit built in this part — decompose before diagnosing, and treat "more people" and "fewer, better rules" as genuinely different fixes rather than interchangeable responses to the same symptom — recurs through the rest of this book wherever a queue-health signal feeds into a bigger decision. Part 5 takes a confirmed capacity gap and turns it into a defensible headcount number. Part 16 picks up the case where a clearance-rate decline traces to an individual skill or coaching gap rather than a team-wide staffing shortfall. Part 25 picks up the cases where the diagnosis itself doesn't fully resolve and a resourcing decision has to be made anyway. And Detection Engineering Handbook V2, Part 38 and Part 42 pick up every case this part's decomposition routes toward a rule rather than a roster. Nothing past this point in the book should treat "the queue is growing" as a self-explanatory staffing request — it should ask which branch of Figure 9.1 it's actually looking at first.

Cross-references

Within this book: Part 3 — Tiering Models: L1/L2/L3 and Beyond (tier definitions and hand-off criteria this part's §4.2 balances load across, without redefining); Part 5 — Headcount & Capacity Modeling (the model that turns a confirmed capacity gap into a defensible headcount number); Part 6 — Shift Pattern & Coverage Design (shift-pattern design and handoff-content mechanics this part's §4.1 assumes rather than re-derives); Part 16 — Performance Management & Coaching (when a clearance-rate decline is a skill or coaching gap rather than a staffing gap); Part 17 — Burnout, Fatigue & Wellbeing (the fatigue risk behind sustained overtime as a surge-response tactic, §5.2); Part 18 — Attrition & Retention (what an unbalanced shift left unaddressed eventually costs); Part 22 — MSSP & Managed-Service Contract Management (negotiating the burst-capacity clause §5.2's MSSP row depends on); Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty (the resourcing call when §3.5's ambiguity doesn't resolve cleanly); Part 28 — The Manager's Role in a Major Incident (staffing surge under a live declared incident, distinct from this part's routine-surge toolkit).

SOC Playbook Handbook: Part 27 — Escalation Quality (the mechanics of a good single hand-off, distinct from this part's tier-level workload-balance question); Part 29 — Playbook Severity Model (per-alert severity scoring, distinct from this part's queue-level health signals per §1); Part 32 — Metrics (the definitions of MTTR and false-positive rate used as inputs throughout this part's worked examples; handle time is this book's own term, defined in Part 5 §2).

Detection Engineering Handbook V2: Part 38 — False Positive Engineering (the tuning mechanics behind every case this part's decomposition routes to a rule rather than a roster); Part 42 — Detection

Quality (measuring coverage and quality well enough to tell a genuine new threat pattern apart from a noisy rule, the fork §3.1–§3.2's decision tree depends on).

neshboy

Section C — Analyst Development: Competency, Training & Career Progression

neshbooy

Part 10 — Competency Models & Skills Matrices

Why this part exists

[CONCEPT] Every SOC manager eventually sits in a room where a shift lead says an analyst is "ready for L2" and a second reviewer says he isn't, and neither can point to anything more specific than a feeling. That argument is a symptom of a missing artifact, not a personality conflict. This part builds that artifact: a tiered competency matrix scored across four axes — technical skill, tool proficiency, communication, and judgment — that turns "ready for L2" and "ready for promotion" from an opinion into a checkable claim, with named, observable behavior standing in for each cell instead of a number pulled from nowhere. The matrix's job is narrow and specific: define what a person can reliably do, at a given tier or in a given role, in terms two independent reviewers would score the same way after watching the same evidence.

[CONCEPT] That's a different question from whether someone is doing the job well right now. A competency matrix is a capability floor, assessed periodically and expected to move slowly; performance is this month's execution against that floor, and it can dip for reasons that have nothing to do with capability — a bad rule flooding the queue, a rough personal stretch, three consecutive short-staffed shifts. Part 16 — Performance Management & Coaching owns that ongoing measurement and the coaching conversation built on it. Conflating the two is the single most common failure mode this part exists to prevent: a manager who scores competency the way they'd score performance ends up gating promotion on this month's ticket count, and a manager who scores performance the way they'd score competency ends up treating a bad month as proof someone was never ready in the first place. Keep the two questions in separate documents, scored on separate cadences, or they collapse into each other.

[CONCEPT] This part does not design the exercises used to test for these behaviors — that's Interviewing & Technical Assessment Design (Part 8), which owns assessment methodology for candidates and can reuse this matrix's tier-one row as its target bar. It does not run the committee that decides who actually gets promoted — that's Career Ladders & Promotion Criteria (Part 13), which owns the ladder structure and the fairness mechanics of a promotion decision once the matrix has supplied the evidence. And it does not define what "good" looks like technically inside a detection rule or an escalation — those standards belong to Detection Engineering Handbook V2 and SOC Playbook Handbook respectively, cited by name below wherever the technical-skill and communication axes need them. What this part owns is narrower than all three: the observable standard itself, and the discipline of writing it down so it means the same thing to every reviewer who uses it.

1. Competency and performance are not the same measurement

[CONCEPT] A competency matrix answers "could this person do L2 work if handed an L2 queue tomorrow." A performance review answers "is this person doing good work on the queue they already have." The two correlate — competency is close to a precondition for good performance — but they diverge often enough that treating them as one measurement produces bad decisions in both directions.

[SENIOR MANAGER] Run the divergence forward: an analyst can be fully L2-competent and still post a mediocre quarter, because the queue she inherited that quarter was dominated by a badly tuned rule generating hundreds of near-duplicate low-value alerts a shift — a detection-quality problem, not a capability gap, and one Detection Engineering Handbook V2, Part 43 — Detection Debt names directly as a distinct failure mode from an analyst being under-skilled. Score her competency using that quarter's numbers and the matrix reports a false negative: it says she can't do the job, when the job she was handed that quarter was unusually hard to do well regardless of who held it. Run it the other direction: an analyst can post excellent numbers for six straight months on a queue that never once handed him an ambiguous severity call, a hostile stakeholder conversation, or a multi-source pivot with no playbook to follow — and still be unready for L2, because nothing in his actual work ever tested the judgment axis this part builds out in §3.4. His performance metrics look identical to a genuinely L2-ready analyst's, right up until the first week he's handed L2-shaped ambiguity and has no track record of resolving it.

[SENIOR MANAGER] The practical fix is structural, not just conceptual: score competency against a fixed, tier-defined standard on a slow cadence — at nomination for tier change, and roughly annually otherwise — using deliberately sampled evidence chosen to test each axis, not whatever tickets happened to land in the queue that month. Score performance against the same person's actual output on a fast cadence, monthly or per sprint, using the Quality Assurance Programs (Part 15) sampling methodology built for exactly that purpose. Keep them in separate documents. A promotion packet that cites "strong Q3 metrics" as its only evidence is really citing performance data to answer a competency question, and a reviewer who knows the difference should send it back.

[SENIOR MANAGER] The conflation runs in both directions in real promotion packets, not just in theory. A packet that leads with "closed 340 tickets this quarter, zero SLA breaches" is citing performance data to answer a competency question — it says nothing about whether any of those 340 tickets ever tested judgment under real ambiguity, because a quiet quarter with no hard calls produces the same clean numbers as a quarter spent skillfully resolving difficult ones. Ask what the evidence would look like if the quarter had been harder; if the honest answer is "the same, just with a lower ticket count," the packet hasn't demonstrated competency at all, only throughput.

2. Why "I'd know it if I saw it" fails as a standard

[CONCEPT] Every experienced SOC manager can, in fact, usually tell a genuinely L2-ready analyst from one who isn't, after watching them work for a few weeks. The problem isn't that this instinct is wrong often — it's that it's expensive, slow to transfer, and impossible to defend. It's expensive because it requires the one senior manager who has the instinct to personally observe every candidate; it doesn't scale past a handful of nominations a quarter. It's slow to transfer because a new team lead promoted into a review role has no way to inherit the instinct except by watching the senior manager make calls for a year and reverse-engineering the pattern. And it's impossible to defend, in the specific sense that matters most: when a passed-over analyst asks why, "you didn't seem ready to me" is not an answer HR, a promotion-committee peer, or the analyst himself can act on. He can't go build the specific thing he's missing if nobody can name it.

Management Autopsy — "promote on tenure once the tenure clock runs out"

The decision (CASE-1001, COMPOSITE CASE EXAMPLE): *A mid-sized SOC — roughly 40 analysts across three shifts — sets its L1-to-L2 bar at 18 months of tenure with no active performance-improvement plan open, and nothing else. No matrix, no structured evidence review, no second reviewer. Tenure alone triggers the promotion packet.*

Why it seemed reasonable: *It's transparent, it's cheap to administer, nobody can accuse the manager of favoritism, and 18 months roughly matches what candidates in the local market expect before a title and pay-band change. It also feels safe against a specific accusation — that promotion is subjective — by removing subjectivity entirely.*

How it failed: *Over two years, 14 analysts crossed the 18-month mark and were promoted on schedule. Five of them — about a third — needed a shift lead to directly re-review their escalations for the next two to three months post-promotion, because they couldn't reliably resolve an ambiguous severity call alone; tenure had told the manager nothing about the judgment axis, only that time had passed. In the same window, two analysts who were demonstrably ready for L2-level ambiguity at 11 and 13 months stayed capped at L1 pay and title until the clock ran out — one of them left for a competitor offering an L2 title at 14 months, a resignation this book's Attrition & Retention (Part 18) would file under exactly the poaching pattern it tracks for mid-level analysts.*

The fix: *Tenure becomes a floor, not a bar — nobody is nominated before six months, full stop, but crossing that floor triggers a matrix-based evidence review rather than an automatic promotion. The clock stops mattering once the matrix does the actual work.*

[SENIOR MANAGER] The fix embedded in that autopsy generalizes: replace the instinct with a written standard specific enough that two different reviewers, looking at the same evidence, reach the same conclusion most of the time. That's the actual bar for "observable" as this part uses the word — not that the behavior is easy to see, but that it's specific enough for independent agreement.

[HR/PEOPLE] Tenure isn't the only proxy that quietly substitutes for observed behavior. A certification fails for the same underlying reason: it proves someone passed an exam, not that they apply the material against a live, ambiguous ticket under time pressure. Hiring & Sourcing Analysts (Part 7) already flags the sourcing-side version of this mistake — over-indexing on certifications during hiring screens out capable non-traditional candidates who never had employer-sponsored exam budget. The internal version is the mirror image: treating a certification earned mid-tenure as sufficient evidence for a tier change, without separately checking whether the analyst applies the certified material on shift. Accept a certification as one input on the technical-skill axis when it's genuinely relevant to the tier; never let it stand in for that axis's own observed-behavior evidence.

3. The four-axis framework: technical skill, tool proficiency, communication, judgment

[CONCEPT] Collapsing everything an analyst does into one competency score hides exactly the information a manager needs, for the same reason a single aggregate maturity score hides which part of a program is actually weak (a theme this book's Part 30 develops fully for organizational maturity). An analyst can be excellent on three axes and dangerously weak on the fourth, and a single blended number buries that. The four axes below aren't the only possible split, but they're the split that maps cleanly onto four genuinely different failure modes a manager needs to diagnose separately.

3.1 Technical skill

[FRONTLINE MANAGER] Technical skill is the ability to reason correctly about what's happening in the environment — recognizing a technique, forming a hypothesis, telling a true positive from a false one, designing or evaluating detection logic. It's the axis most managers default to measuring, because it's the easiest to test with a synthetic exercise, and the easiest to over-weight as a result.

Cross-Book Pointer *This part does not define what correct detection logic, a well-designed analytic, or a sound hunt hypothesis actually look like — that standard already exists, in detail, in Detection Engineering Handbook V2, Parts 3–29 (detection rule logic, telemetry, and query mechanics) and Parts 34–36 (threat hunting methodology). What this part owns is narrower: what evidence proves a specific analyst can reliably produce work that meets that already-defined standard, unsupervised, under time pressure. Come back here once you've confirmed what "good" means technically; this section is about proving someone can do it, not about what it is.*

3.2 Tool proficiency

[FRONTLINE MANAGER] Tool proficiency is a distinct axis because it's possible to reason correctly about an attack and still be too slow or clumsy in the specific SIEM, EDR, or SOAR platform in front of you to act on that reasoning inside the SLA. An analyst who can describe exactly what query she'd want to run, but takes 20 minutes to actually build it because she doesn't know the platform's query syntax or field-mapping quirks, is not tool-proficient at the tier she's being evaluated for — regardless of how sound her underlying hypothesis was. This axis is also the one most sensitive to a platform migration: Tooling Procurement & Platform Strategy (Part 21) covers the retraining cost of a SIEM switch in budget terms, and this axis is where that retraining cost actually lands, person by person, on the matrix.

3.3 Communication

[FRONTLINE MANAGER] Communication is the ability to produce an escalation, a stakeholder briefing, or an incident update that the receiving party can act on without asking a clarifying question first. SOC Playbook Handbook, Part 27 — Escalation Quality already defines what a mechanically good hand-off contains — the fields, the framing, the specific information a Tier 2 analyst needs to pick up a case cold. This axis asks a different question: does this specific analyst reliably produce that, unprompted, across a real sample of their actual tickets, not just on the one escalation a team lead happened to review this week.

3.4 Judgment

[FRONTLINE MANAGER]

Judgment is the axis most managers skip testing directly, because it's the hardest to observe in a clean exercise — and it's also, per the autopsy in §2, the axis most likely to be the real gap hiding behind a tenure-based or metrics-based promotion decision. It's the ability to correctly resolve ambiguity: applying SOC Playbook Handbook, Part 29 — Playbook Severity Model consistently when a case doesn't map cleanly onto the model's clearest examples, knowing which categories of ambiguity still require escalation regardless of tier (legal holds, suspected insider activity, anything touching an executive), and being able to explain the reasoning afterward in a way a reviewer could reconstruct and check.

Manager's Note *The fastest way to actually test judgment is to sit with an analyst on a real, already-closed ambiguous ticket and ask them to narrate their reasoning out loud, live, before you tell them how it was actually resolved. 10 minutes of real reasoning on a real case tells you more than a page of self-assessment ever will, because self-assessment measures how well someone can describe good judgment in the abstract, not whether they exercise it under the same uncertainty they'll face on shift.*

4. Building the tiered matrix

[SENIOR MANAGER]

Four axes only become a usable tool once they're anchored to specific tiers with specific, checkable behavior in each cell. The anchors below assume the L1/L2/L3 tiering structure from Tiering Models: L1/L2/L3 and Beyond (Part 3) already exists; this part doesn't redesign what each tier owns organizationally, only who's demonstrably capable of holding it.

4.1 The gate rule: a floor on every axis, not an average

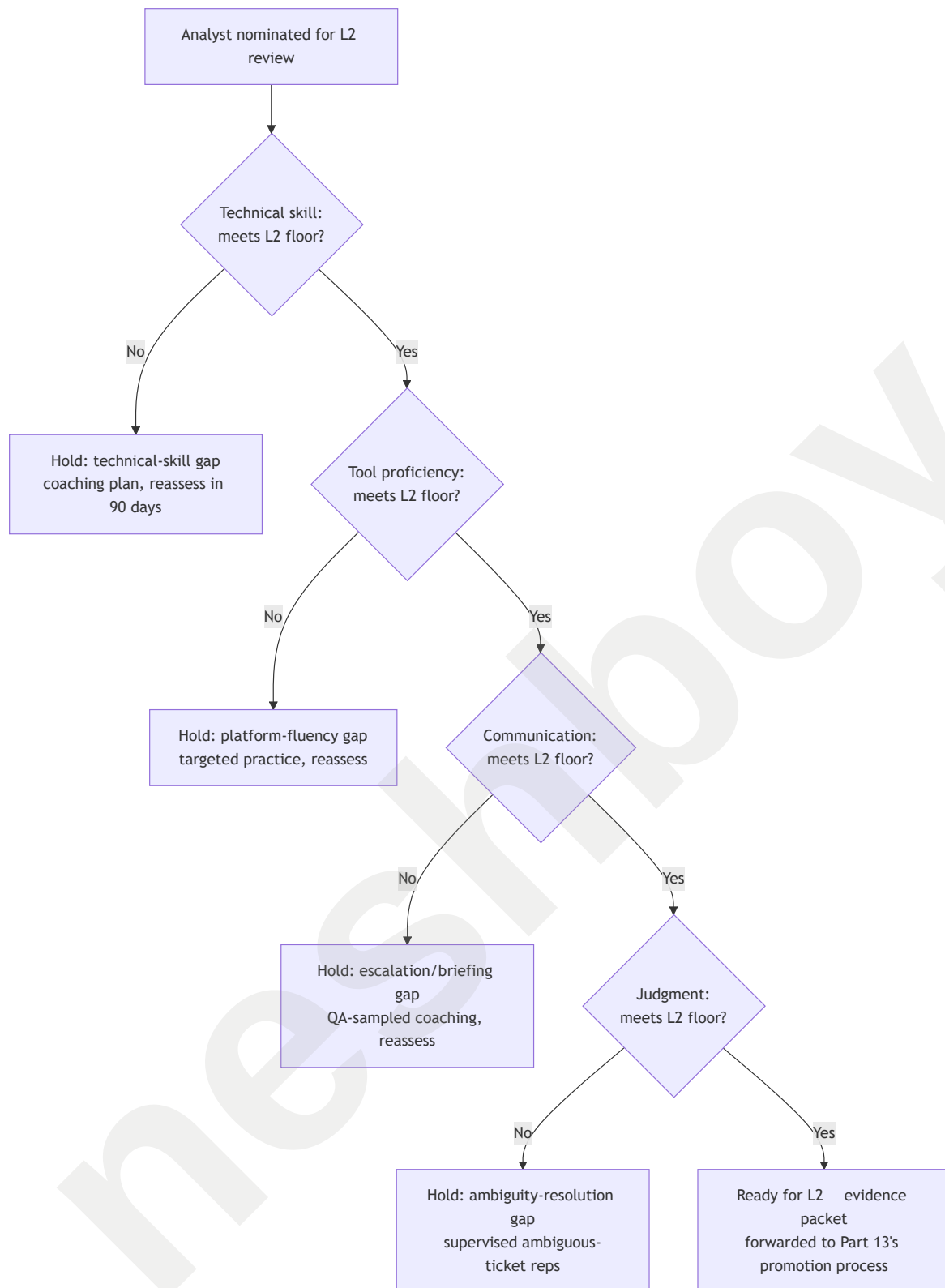
[SENIOR MANAGER]

The single most consequential design decision in a competency matrix is whether tiers gate on a floor per axis or an average across axes — and averaging is the wrong default almost every time. An analyst who scores exceptionally on technical skill and tool proficiency but is still weak on judgment is not "roughly L2 on average." He's an analyst who will make confident, fast, technically sound decisions on the wrong call, which is a more dangerous failure mode than an analyst who's slow and cautious across the board, because a fast wrong call does more damage before anyone catches it. Gate every tier transition on clearing the minimum bar on all four axes independently. A strong score on three axes does not buy down the fourth.

Figure 10.1 — The four-axis "ready for L2" gate, evaluated independently. CONCEPTUAL. Diagram ID [FIG-1001](#). Illustrates the gate-per-axis logic described above: a nomination only proceeds to promotion once every axis clears the L2 floor on its own terms, and a failure on any single axis routes to a targeted hold rather than an averaged score.

flowchart TD

```
A["Analyst nominated for L2 review"] --> B{"Technical skill:\nmeets L2 floor?"}
B -- No --> B1["Hold: technical-skill gap\ncoaching plan, reassess in 90 days"]
B -- Yes --> C{"Tool proficiency:\nmeets L2 floor?"}
C -- No --> C1["Hold: platform-fluency gap\ntargeted practice, reassess"]
C -- Yes --> D{"Communication:\nmeets L2 floor?"}
D -- No --> D1["Hold: escalation/briefing gap\nQA-sampled coaching, reassess"]
D -- Yes --> E{"Judgment:\nmeets L2 floor?"}
E -- No --> E1["Hold: ambiguity-resolution gap\nsupervised ambiguous-ticket reps"]
E -- Yes --> F["Ready for L2 – evidence packet\nforwarded to Part 13's promotion process"]
```



4.2 A worked L1-to-L2 matrix

[SENIOR MANAGER] The table below is a worked example of the anchors themselves — the specific, observable behavior a manager checks for at each tier, and the evidence source that actually lets a reviewer check it rather than guess (CONCEPTUAL SAMPLE — illustrative anchors, calibrate against your own tier definitions before use). It supports a nomination decision the way Figure 10.1's gate logic

consumes it: one row failing its L2 column is one axis on hold, regardless of how strong the other three rows look. Appendix A2's companion worksheet (per this book's index) carries a blank version of this same structure for direct use.

Axis	L1 anchor	L2 anchor	L3 / track anchor	How a manager verifies it
Technical skill	Maps an alert to the correct playbook step unaided; explains why a match is or isn't a true positive	Forms and tests a hypothesis across multiple log sources with no playbook to follow; flags when a rule's signal-to-noise ratio has gone bad	Designs or validates a new analytic to the standard defined in Detection Engineering Handbook V2, Parts 3–29; owns tuning after deployment	QA-sampled ticket review (Part 15); shadowed rule-build or hunt session
Tool proficiency	Pulls the exact fields a documented playbook calls for, within SLA, unaided	Builds an ad hoc cross-tool query to test a hypothesis with no documentation, in a comparable time budget to L1's lookup	Writes and tests platform rule content; manages false-positive suppression logic; knows where the platform's own results can't be trusted	Timed practical exercise; live shift ride-along
Communication	Escalation notes meet the bar defined in SOC Playbook Handbook, Part 27 — Escalation Quality, with no rewrite needed	Briefs a stakeholder outside the SOC, live and unscripted, translating alert detail into a business-relevant ask	Communicates directly with an executive or incident commander during a major incident, with no manager relay	QA sample of written escalations; observed live stakeholder call
Judgment	Applies the severity model consistently on clear-cut cases; escalates genuinely ambiguous ones instead of guessing	Resolves ambiguous severity calls correctly; knows which categories (legal, insider-threat-adjacent, executive-touching) still require escalation regardless	Makes risk-acceptance-adjacent calls under real uncertainty and can defend the reasoning afterward	Live ambiguous-ticket reasoning walkthrough; calibration cross-check against a second reviewer

5. Matrices fork by track, not just by tier

[SENIOR MANAGER] L1 to L2 is a linear tier transition, and the matrix above treats it that way. Career Ladders & Promotion Criteria (Part 13) covers what happens past that point — the ladder forks into senior analyst, detection engineer, threat hunter, and team-lead tracks, each of which needs its own version of these four axes rather than "more of the same L2 behavior, but better." This part's job is to flag that the fork exists and show where the axes actually diverge; Part 13 owns the ladder structure and the committee that decides who moves onto which track.

[SENIOR MANAGER] Take judgment as the clearest example of the divergence. On the detection-engineer track, an L3-level judgment anchor is about tradeoffs inside an analytic's design — accepting a specific false-positive rate in exchange for coverage against a technique the team has decided matters more, and being able to defend that tradeoff when it's questioned later. On the team-lead track, the L3-equivalent judgment anchor is almost the opposite skill: recognizing when to let another analyst make the call themselves, sit with being wrong, and learn from it, rather than stepping in to make the correct

call faster. Both are real judgment. Neither substitutes for the other, and a matrix that scores them identically is measuring the wrong thing for at least one of the two tracks.

[SENIOR MANAGER] Tool proficiency forks just as sharply. A threat hunter's L3-equivalent tool-proficiency anchor is fluency moving across raw, unstructured data with no prebuilt dashboard — building a one-off pivot against weeks of raw process-creation telemetry to test a hypothesis nobody has asked the SIEM to pre-aggregate before. A detection engineer's L3-equivalent anchor looks more like platform administration: managing rule-content version control, understanding how a specific backend's query planner behaves under load, knowing which fields silently drop out of a join when a schema changes upstream. Both are advanced tool proficiency. Neither prepares someone for the other track's daily tool use, and a manager who promotes a strong hunter directly into a detection-engineering role on the assumption that tool skill transfers is making the same category error as promoting on tenure alone — trading one unverified proxy for another.

Manager's Note *Don't build the team-lead matrix by copying the senior-analyst matrix and bolting a coaching row onto the bottom. Coaching, delegation, and running a hard conversation without personally taking over are skills a strong analyst has frequently never had reason to practice — nothing in being technically excellent teaches you to watch someone else work through a problem slower than you would and resist fixing it for them. Test for the track's actual skill directly, with its own exercise, instead of assuming it comes bundled with technical seniority.*

6. Assessing against the matrix without turning it into a review

[HR/PEOPLE] A matrix is only as good as the evidence behind each cell, and the fastest way to wreck that evidence is to let the assessment collapse into an informal performance review — a single manager's overall impression, stamped onto four boxes after the fact to make the impression look structured. Assess each axis against its own dedicated evidence: a QA sample pulled the same way Part 15's calibration methodology pulls one, a timed practical exercise, a live-observed stakeholder call, a walked-through ambiguous ticket. Evidence collected specifically to test one axis is worth more than an impression retrofitted across four.

[HR/PEOPLE] Capacity is the practical constraint that breaks this fastest. A team lead assessing more than about eight analysts against a four-axis matrix inside a single nomination cycle runs out of time to gather enough real evidence per analyst per axis — not because the axes get harder, but because there simply aren't enough observed shifts, sampled tickets, and live walkthroughs to go around. Past that point, the assessment quietly reverts to impression, even with the matrix's structure sitting on top of it. Split the cycle, add a second rater, or extend the assessment window before that ratio gets away from the team lead running it.

[HR/PEOPLE] Run the first competency calibration session on an analyst everyone already agrees is solidly L2, not on a disputed nomination. It surfaces the same problem Part 15's QA calibration surfaces on an easy ticket: reviewers who think they share a definition of "meets the judgment floor" usually don't, and it's far cheaper to find that gap on a case with no stakes than in the middle of a contested

promotion decision. Once two reviewers converge on the easy case — scoring the same four axes the same way, independently, before comparing notes — move to a genuinely marginal case and check whether the convergence holds. If it doesn't, the anchors themselves are still too vague, and the fix is rewriting the cell's language, not retraining the reviewers.

Worked example (CASE-1002, COMPOSITE CASE EXAMPLE). A 16-analyst SOC runs its first formal matrix-gated nomination cycle. Nine analysts are nominated for L2. Structured evidence — QA samples, a timed practical exercise, and a live ambiguous-ticket walkthrough per analyst — shows six clearing all four axes cleanly. The remaining three clear technical skill, tool proficiency, and communication, but all three fail the same axis: judgment, specifically the category of ambiguous severity calls that don't require escalation but that all three had been escalating anyway rather than resolving. None of the three is held indefinitely — each gets a targeted coaching plan built around supervised reps on exactly that ticket category, and the SOC's own tracking shows an average of 10 weeks before all three clear the judgment floor on a follow-up review. The pay-band step from L1 to L2 in this SOC runs \$6,500 a year (CONCEPTUAL SAMPLE); holding three analysts back 10 weeks rather than promoting them on the strength of the other three axes costs the SOC nothing but patience, and avoids handing three analysts L2-level authority over calls they'd demonstrably been routing around rather than making.

People Risk Trap *Once a matrix exists, the easiest way to quietly destroy it is grade inflation: a reviewer who finds every axis "meets bar" on every nomination to avoid an uncomfortable conversation, or to keep pace with a peer team lead who's rating generously. A matrix where every score is high stops distinguishing anyone, which means it stops doing the one thing it exists to do. The fix is the same one Part 15 uses for QA scoring: calibration sessions where two or more reviewers independently score the same evidence before comparing notes, run often enough that drift gets caught within a cycle, not discovered a year later when someone finally asks why every nomination in one team lead's queue has passed.*

7. Feeding hiring bars and promotion criteria

[SENIOR MANAGER] The matrix's value compounds once other decisions actually consume it instead of running on a separately invented standard. Interviewing & Technical Assessment Design (Part 8) should target this matrix's L1 row directly: the log-triage exercise or mock escalation that part builds is only as good as the target behavior it's testing for, and that target behavior is exactly what §4.2's L1 column already specifies. Without a shared matrix, a hiring bar and an internal promotion bar drift apart silently — a hiring manager who was burned by one bad hire tightens the interview bar informally over time, while the internal L1-to-L2 bar for existing analysts stays wherever it was set originally, and eventually an external candidate is being held to a materially higher standard than an internal analyst moving into the identical seat. That's not a hypothetical inconsistency; it's the kind of thing a rejected external candidate's LinkedIn post makes very visible very fast.

[SENIOR MANAGER] Hiring & Sourcing Analysts (Part 7) is the other direct consumer worth naming specifically: the leveled job descriptions that part builds — the actual document a candidate reads before applying — should describe this matrix's L1 row in plain language, not a separately invented list

of buzzwords assembled from a template. A job description that promises a "fast-paced, dynamic environment" and lists a certification wishlist, while the real L1 bar is "maps an alert to the correct playbook step unaided," sets a false expectation in both directions: candidates self-select against a job that isn't what's described, and the ones who do get hired discover the real bar only after starting. Write the job description from the matrix, not around it.

[SENIOR MANAGER] Career Ladders & Promotion Criteria (Part 13) consumes the matrix the other direction: it owns the promotion committee's process, its consistency mechanics across cases, and the ladder structure itself, but the committee needs something concrete to look at, and "the matrix says all four axes cleared, evidence attached" is a categorically better packet than "the manager recommends her." A promotion committee that reviews matrix evidence is reviewing a claim it can actually audit. A committee that reviews a manager's narrative recommendation is reviewing the manager's judgment about the candidate, one layer removed from the candidate's actual demonstrated behavior — which reintroduces exactly the "I'd know it if I saw it" problem this part opened with, just relocated to the committee table instead of the shift lead's desk.

[SENIOR MANAGER] One more consumer worth naming without expanding on here: AI, Automation, and the Future Shape of the SOC (Part 32) revisits this matrix later in the book to ask what happens to the technical-skill and tool-proficiency axes once AI copilots absorb a meaningful share of Tier 1 triage work. That's a real question this part deliberately leaves untouched — the matrix built here assumes today's tier definitions from Part 3, and Part 32's job is to say how those definitions, and the axes anchored to them, need to move as the work itself changes.

8. Where competency models fail

[SENIOR MANAGER] A matrix built once and never revisited drifts out of date the moment a tool changes, a new attack technique becomes routine enough to expect everyone to recognize it, or a tier's actual scope shifts under Part 3's own tiering decisions. Treat the anchors as a living document reviewed on the same cadence the headcount model in Part 5 gets reviewed, not something defended once at rollout and then left alone for three years while the environment it describes keeps changing underneath it.

[SENIOR MANAGER] A matrix also can't see a skill nobody has ever asked an analyst to demonstrate. A hunter who's never once briefed an executive isn't failing the communication axis at L3 — she's unmeasured on it, which is a different fact than "not ready," and a reviewer who conflates the two ends up blocking a genuinely qualified candidate over a gap in the evidence rather than a gap in the person. Before scoring an axis "not met," check whether the real finding is "never tested" and fix the evidence gap before treating it as a competency gap.

Management Autopsy — "build the matrix, then never touch it again"

The decision: *A SOC builds a careful four-axis matrix during a headcount-planning push, uses it for one nomination cycle, and then leaves it untouched for two years while the platform migrates from one SIEM to another and the threat model shifts toward cloud-native attack paths the original technical-skill anchors never mention.*

Why it seemed reasonable: *Building the matrix took real effort — workshops, calibration, buy-in from three shift leads — and once it shipped, it felt finished, the way a policy document feels finished once it's approved.*

How it failed: *18 months in, the technical-skill L2 anchor still describes fluency in the retired SIEM's query syntax, and two analysts who are demonstrably strong in the new platform score as "not yet met" against an anchor testing a skill nobody needs anymore, while an analyst who never adapted past the old platform still scores well against a standard that's now describing the wrong job.*

The fix: *Assign the matrix an owner and a review trigger tied to real events — a platform migration, a tiering change from Part 3, a new specialty track from Part 13 — rather than a calendar date alone, so the artifact updates when the job changes instead of drifting quietly behind it.*

[SENIOR MANAGER]

Neither failure mode is a reason to skip building the matrix — both are cheaper to fix than the tenure-based and instinct-based alternatives from §2 and §4, because both point to a specific, correctable gap in the artifact rather than an unfixable gap in the manager's judgment. That's the whole case for building one in the first place: a wrong anchor can be corrected in a review cycle; an unwritten standard can't be corrected at all, because there's nothing on paper to point at and fix.

Cross-references

This part assumes the tiering vocabulary from Part 3 — Tiering Models: L1/L2/L3 and Beyond. It feeds Part 8 — Interviewing & Technical Assessment Design and Part 13 — Career Ladders & Promotion Criteria, and hands off to Part 16 — Performance Management & Coaching, Part 15 — Quality Assurance Programs, and Part 32 — AI, Automation, and the Future Shape of the SOC for the questions this part deliberately leaves open. Outside this book, it cites SOC Playbook Handbook, Part 27 — Escalation Quality and Part 29 — Playbook Severity Model for the communication and judgment axes' technical standards, and Detection Engineering Handbook V2, Parts 3–29 and Parts 34–36 for the technical-skill axis's standard of "good," without re-deriving either.

Part 11 — Onboarding & Ramp-Up Programs

Why this part exists

[CONCEPT] A new analyst's first 90 days decide more about whether they're still on the team at month 18 than almost anything that happens afterward. A rushed ramp produces an analyst who never fully trusts their own judgment, who escalates everything or nothing because nobody ever drew the line clearly, and who either burns out chasing a standard nobody explained or gets quietly managed out for missing a bar they were never actually taught. This part owns the structure and pacing of that first-90-days program: how shadowing, supervised triage, graduated independence, and simulation-based certification sequence against each other, how much autonomy an analyst earns at each stage, who signs off on moving to the next one, and what a manager measures to know the program is actually working rather than merely running on a calendar.

It does not own the technical material a ramp program runs an analyst through. The scenarios used inside a certification exercise — a seeded alert, an injected log sample, a scripted attacker behavior — are pulled by reference from Detection Engineering Handbook V2, Part 37 — Detection Testing and SOC Playbook Handbook's Playbook Library and Part 34 — Playbook Testing, not re-authored here. It also does not own what "ready for L2" means in observable, testable terms — that's Part 10's competency matrix, consumed here as the criteria this program's gates check against — or the standing mentorship program a graduated analyst enters next, which belongs to Part 14. And it is not the ongoing-training budget and certification-strategy question, which is Part 12's: this part ends the day an analyst reaches full independent shift authority, whatever comes after that is continuing development, not ramp-up.

1. What a ramp-up program is for, and what it costs to skip

[CONCEPT] A ramp-up program is a sequence of expanding permissions with checkpoints attached, not a training curriculum. The distinction matters: a curriculum measures exposure — did the analyst sit through the material — while a ramp program measures earned autonomy — has this specific person, doing this specific work, demonstrated enough judgment to be trusted with the next increment of it. A new hire can complete every training module in a learning-management system and still not be ready to disposition a privileged-account-login alert alone at 2 a.m. with no one to ask. The four phases in this part exist because "trained" and "ready" are different claims, and only the second one should open the door to the next stage.

[SENIOR MANAGER] The reason this needs to be a manager-owned program rather than an informal "shadow someone for a couple weeks" habit is cost. Every day a new analyst isn't independently productive is a day Part 5's headcount model is quietly wrong if it counted that analyst as a full FTE from the start date. The gap doesn't disappear — it shows up as a mentor's own queue growing, or the rest of the shift absorbing the slack.

CONCEPTUAL SAMPLE — illustrative numbers, not sourced benchmark data:

New L1 analyst, fully loaded salary: \$62,000/year (~\$5,167/month)
Ramp duration: 90 days
Average productivity during ramp, blended across all 4 phases: ~30% of a fully ramped analyst's independent throughput

Senior-analyst mentor time diverted:

Weeks 1-6 (Phases 1-2): ~5 hrs/week

Weeks 7-11 (Phase 3): ~2 hrs/week

Mentor's fully loaded rate: \$68,000/year (~\$32.70/hour)

Mentor-hour cost: $(5 \text{ hrs} \times 6 \text{ weeks}) + (2 \text{ hrs} \times 5 \text{ weeks}) = 40 \text{ hours}$
 $40 \text{ hours} \times \$32.70/\text{hour} = \$1,308$

Unrealized-output cost: $3 \text{ months} \times \$5,167/\text{month} \times (1 - 0.30) = \$10,850$
(salary paid against output the team doesn't get back at full rate yet)

Total visible cost of one ramp cycle: roughly \$12,150

None of this is an argument against hiring — it's the number a headcount plan needs to hold honestly instead of assuming a new hire is a full FTE from day one. A team hiring three analysts in the same quarter is temporarily absorbing something on the order of \$36,000 in ramp cost and 120 mentor-hours pulled from senior analysts' own queues — exactly the kind of surge Part 6's shift-design options need to staff around in advance, not discover halfway through the quarter. The limitation of the number above: it counts mentor-hours and unrealized output, not the harder-to-price cost of a mentor's own queue backing up while they're diverted, which is a Part 9 queue-health problem this model doesn't capture.

[SENIOR MANAGER] The reason this cost usually stays invisible is that it never gets its own line anywhere — it's buried inside the new hire's salary line (already budgeted) and the mentor's existing headcount (already budgeted), so nothing in a standard budget review flags that the mentor is running at roughly 90% of normal queue capacity for 11 weeks. Part 20 covers building the SOC budget in full; the specific ask this part makes of that process is a visible, named ramp-cost line for every planned hire, sized against the model above, rather than an assumption that ramp-up happens inside existing capacity for free.

Operational Reality A headcount plan that lists a new hire as one FTE starting day one is already wrong by the ratio in the example above — that analyst is worth something closer to 0.3 of an FTE for the first three months, and the 0.7 gap doesn't vanish, it shows up as a mentor's queue growing and the rest of the shift absorbing the difference. Budget the ramp cost against the hiring decision itself, not against a vague assumption that onboarding is a wash.

2. The four-phase structure

[SENIOR MANAGER] The program has four phases, and the sequencing is itself the design decision a manager owns: shadowing (observe, no independent action), supervised triage (act, but nothing ships

without review), graduated independence (act independently, spot-checked at a declining rate), and simulation certification (a graduation exam under realistic, unannounced conditions). Skipping a phase or shortening it under backlog pressure is the single most common way this program breaks, covered as its own failure mode in §5 and §8 below.

[FRONTLINE MANAGER]

Table 11.1 sets illustrative day counts against a 90-day program; adjust the total and the split between phases to your own queue's alert mix and complexity, but keep the sequence and the gate logic intact. A phase with no exit criterion isn't a phase — it's a calendar entry, and calendar entries don't catch a bad disposition before it ships.

Table 11.1 (CONCEPTUAL SAMPLE) maps each phase to the decision a manager makes at its gate: advance, hold, or remediate.

Phase	Typical Duration	Supervision Model	Exit Criteria	Sign-off Authority
1. Shadowing	Days 1–10	Paired observation only; no independent actions on live tickets	Correctly names the escalation path and severity tier for 5 seeded historical alerts	Buddy + team lead
2. Supervised triage	Days 11–40	Analyst drafts a disposition; reviewer approves before it ships	3 consecutive shifts with zero reviewer-overturned dispositions	Team lead
3. Graduated independence	Days 41–75	Independent dispositions, spot-checked at a declining rate (100% → 50% → 20%)	Spot-check pass rate at or above 95% at 20% sampling, sustained for 2 consecutive weeks	Team lead + QA reviewer
4. Simulation certification	Days 76–90	One unannounced live-fire simulation under normal shift conditions	Passes the certification simulation with no unresolved critical gaps	SOC manager

The flowchart below draws Table 11.1's gate logic as a decision path, including the remediation loop at each gate — useful when explaining to a new analyst, or a skeptical HR partner, that failing a gate once is a loop back, not a dead end.

Figure 11.1 — The four-phase ramp-up gate flow, with remediation loops. *CONCEPTUAL*. Diagram ID **FIG-1101**. Illustrates Table 11.1's phase sequence and gate logic as a decision path, including the remediation loop that fires at each gate when an analyst doesn't clear it on the first attempt. It is a structural diagram of the program's own design, not a capture of any specific cohort's real ramp record.

flowchart TD

```
A["Day 0: Orientation & access provisioning"] --> B["Phase 1: Shadowing (Days 1-10)"]
B --> G1{"Gate 1: Names escalation path\nand severity tier for 5 seeded alerts?"}
G1 -- No --> B2["Extend shadowing 5 days"]
B2 --> G1
G1 -- Yes --> C["Phase 2: Supervised triage (Days 11-40)"]
C --> G2{"Gate 2: 3 consecutive shifts,\nzero reviewer-overtaken dispositions?"}
G2 -- No --> C2["Targeted coaching + repeat sampled shifts"]
C2 --> G2
G2 -- Yes --> D["Phase 3: Graduated independence (Days 41-75)"]
D --> G3{"Gate 3: Spot-check pass rate\n>= 95% at 20% sampling?"}
G3 -- No --> D2["Hold at current spot-check rate,\nadd coaching"]
D2 --> G3
G3 -- Yes --> E["Phase 4: Simulation certification (Days 76-90)"]
E --> G4{"Gate 4: Passes unannounced\nlive-fire simulation?"}
G4 -- No --> E2["Remediation plan,\nretest within 2 weeks"]
E2 --> G4
G4 -- Yes --> F["Full independent shift authority (Level 4)"]
```

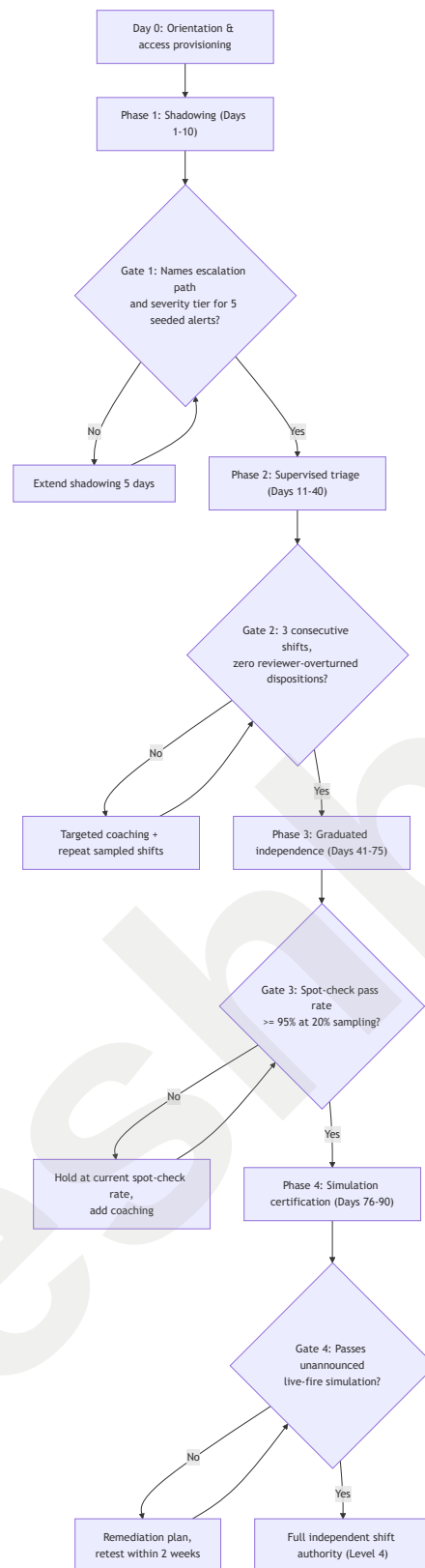


Figure 11.1 — The four-phase ramp-up gate flow, with remediation loops. *CONCEPTUAL.* Illustrates Table 11.1's phase sequence and gate logic as a decision path, including the remediation loop that fires at each gate when an analyst doesn't clear it on the first attempt. It supports the claim, made throughout §2–§6, that "graduated independence" is a loop with a remediation path built in, not a one-way progression a manager either grants or withholds once. Diagram ID **FIG-1101**.

The diagram shows the sequence and the remediation loops; it doesn't show who specifically staffs each review — that's Table 11.1's sign-off-authority column and §7's competency-matrix tie-in.

What Would Change My Mind *This part uses 90 days and the four-phase split in Table 11.1 as a default frame, calibrated to a generic mid-complexity SOC's alert mix. If a team running genuinely simple, low-variety alert traffic — a narrow tool stack, a small handful of alert types — could reliably certify analysts at Level 4 in under 60 days with spot-check pass rates still holding at the same 95% bar, that would show this part's default duration is calibrated to the alert-mix complexity most SOC's have, not a fixed property of how long judgment takes to build. The guidance should then shift from a default day count to one keyed explicitly to alert-type variety.*

3. Phase 1 — shadowing

[FRONTLINE MANAGER] Shadowing means the new analyst sits with a working analyst on a live queue, watches real tickets at real volume, and narrates back what they'd do at each decision point before being told the actual answer — not a scripted walkthrough of a handful of cherry-picked examples. Ten days is usually enough to see most of a shift's alert mix at least once; a shift that only handles 3 alert types in that window needs the schedule adjusted so shadowing spans more than one shift pattern (Part 6 covers the coverage options), not an extended shadowing phase that still misses the same gap.

The exit gate in Table 11.1 tests recognition and escalation-path naming against 5 seeded historical alerts pulled from the real queue — not judgment under live pressure, which is Phase 2's job. A new hire who can correctly say where a privileged-account-login alert routes, and why, has learned the map. Whether they can draw sound conclusions from it themselves comes next.

3.1 The shadow log

[FRONTLINE MANAGER] Shadowing produces almost nothing to check against unless the new hire keeps a record while doing it. A shadow log is a running list — one line per alert observed, noting the alert type, the disposition reached, and the one thing the new hire would have done differently if they'd been alone — kept in whatever tool the team already uses for notes, not a system built specifically for this. By day 10, the log itself is evidence for the exit gate: a log with 40 or 50 entries spanning most of the queue's alert types means the 5 seeded historical alerts in Table 11.1's exit criteria are confirming something the log already suggests. A log with 8 entries clustered on two alert types means the seeded-alert test is about to fail, and the team lead already knows why before running it.

The log's other job is surfacing this section's People Risk Trap before day 10 rather than after it. A team lead who spot-checks the log at the end of day 3 catches a stalled shadowing week — one where the new hire is watching a screen with no narration — while there's still a week left to fix it, instead of finding out on day 10 that the exit gate is failing for a reason that had nothing to do with the new hire's aptitude.

People Risk Trap Assigning the new hire to whichever senior analyst has the most tenure, with no regard for that analyst's current workload, routinely produces a shadowing week where the pairer just keeps triaging at full speed and the new hire watches a screen for six hours a day with no narration. The fix: rotate shadowing across two or three analysts who have headroom that particular week, and give the pairer explicit, counted time in their own workload plan for it — not an unstated expectation that mentoring happens for free on top of a full queue.

4. Phase 2 — supervised triage and the trust ladder

[FRONTLINE MANAGER] Supervised triage is the phase where the new analyst does the actual work — pulls the alert, gathers context, drafts a disposition and, where warranted, an escalation write-up — and a reviewer reads it before anything ships. The reviewer isn't rewriting the analyst's work; they're catching it before it goes out wrong, then explaining why, in that order. A reviewer who silently fixes and ships the draft without feedback teaches nothing, and produces an analyst who reaches Phase 3 no better calibrated than they were on day 11. The standard a reviewer holds a draft escalation against is the one already defined in SOC Playbook Handbook, Part 27 — Escalation Quality for the mechanics of a good hand-off; this phase doesn't invent a separate, softer ramp-specific bar, it applies the real one earlier, with a safety net underneath it.

[FRONTLINE MANAGER] The reviewer's actual skill in this phase is asking before telling. Faced with a draft disposition that missed a lateral-movement indicator, a reviewer who says "you need to check for follow-on authentication on the same host" fixes this one ticket and teaches nothing repeatable. A reviewer who asks "what would you expect to see next if this account really was compromised, and did you check for it" forces the analyst to build the same reasoning path they'll need alone in Phase 3. This runs slower per ticket during Phase 2, sometimes markedly slower, and that's the phase's whole point — if it moved at Phase 3 speed, it wouldn't be Phase 2.

CONCEPTUAL SAMPLE — illustrative feedback exchange, not a transcript of a real review:

Analyst's draft disposition: "Benign -- single failed-then-successful login, user likely mistyped a password."

Reviewer, asking rather than telling: "What's the time gap between the failure and the success, and does it match how long it takes a person to retype a password by hand?"

Analyst, checking: "...four seconds. That's fast for a retype."

Reviewer: "So what does that change about the disposition, and what would you check next before shipping it?"

The reviewer never states the answer outright — that credential-stuffing tooling retries near-instantly, while a human doesn't. The analyst reaches it by being asked the right question, which is the version of this finding that survives into Phase 3, when no one is there to ask it for them.

4.1 The trust ladder

[SENIOR MANAGER] The trust ladder is the axis Phases 2 and 3 actually move along — not a calendar date, but a specific, checkable expansion of what the analyst is allowed to do without someone else looking first. A manager should be able to state, for any analyst on the team, which level they're currently at and why, not just how many weeks they've been employed.

Table 11.2 (CONCEPTUAL SAMPLE) gives the five levels a manager uses to decide how much unsupervised action an analyst has earned.

Level	Autonomy Scope	Review / Spot-Check Rate	Typical Phase
0	Observation only; no actions on live tickets	100% (nothing ships without the pairer)	Shadowing
1	Drafts a full disposition; reviewer approves before it ships	100% (every disposition reviewed pre-ship)	Supervised triage
2	Independent on low-severity, well-covered alert types only	50% of dispositions spot-checked after the fact	Early graduated independence
3	Independent on all severities except a formally declared incident	20% of dispositions spot-checked after the fact	Late graduated independence
4	Full independent shift authority, including escalation and incident participation	Standard QA sampling rate, not a ramp-specific rate	Post-certification

Level 4's review rate is whatever Part 15's quality-assurance program samples for every analyst on the team. This program's job ends at getting someone onto that standard rate — it does not maintain a permanent higher-scrutiny lane for anyone just because they were once a new hire.

Field Test Setup: Phase 2 is documented as requiring 3 consecutive shifts with zero reviewer-overturned dispositions before an analyst advances to Level 2. **Action:** Pull the last five ramp analysts' Phase 2 records and check how many of them actually had a reviewer overturn at least one disposition somewhere in their Phase 2 history, versus how many gates were signed off with no overturned disposition ever logged for that analyst. **Expected result:** Some overturns should show up somewhere in most analysts' Phase 2 history — a reviewer catching a real mistake before it ships is the entire point of the phase. A pattern of zero overturns across every analyst's complete Phase 2 record means reviews are being rubber-stamped rather than performed, and the gate is protecting nothing.

Blind Spot This program tracks the analyst being ramped; it has no built-in visibility into whether the reviewer assigned to give feedback is any good at giving it. Two analysts hired in the same quarter, run through an identical Table 11.1, can graduate at meaningfully different competence levels solely because one reviewer explains reasoning out loud and the other just approves or rejects drafts silently. Rotate Phase 2 reviewers across more than one analyst per cohort where staffing allows, and add "explains reasoning, doesn't just correct" as its own line in whatever review Part 16 runs on senior analysts who take on this role.

5. Phase 3 — graduated independence

[FRONTLINE MANAGER]

Graduated independence is where the trust ladder actually moves: Level 2 to Level 3 to Level 4, each step earned by a sustained spot-check pass rate rather than a single good week. A weekly check-in against the current sampling rate — not a monthly one — catches a slipping analyst while the fix is still a coaching conversation instead of a pattern that needs a formal performance conversation under Part 16.

The single most common way this phase gets skipped is backlog pressure, and it's expensive enough to be worth a full worked case, [CASE-1101](#) below — a composite assembled from recurring patterns across several ramp-collapse situations, not one traceable incident.

Management Autopsy — "skip straight to independent queue duty because the backlog was three days deep" (CASE-1101, COMPOSITE CASE EXAMPLE)

The decision: With the queue running roughly 3 days behind and two experienced analysts out the same week, a team lead at an 18-analyst managed-detection SOC moved a Phase 2 analyst — 11 days into supervised triage — straight onto solo queue duty at full Level 3 autonomy, skipping the graduated Level 2 step entirely.

Why it seemed reasonable: The analyst's Phase 2 dispositions had looked solid in the reviews that had actually happened, the backlog was a visible and urgent problem that day, and the alternative was overtime the budget hadn't planned for.

How it failed: With no spot-check rate in place, nobody was reading the analyst's dispositions at all for 11 days. A QA sweep at the end of that stretch found that 40% of the analyst's solo dispositions from the first week needed to be reopened, and one lateral-movement-shaped alert had been dispositioned as benign and sat unescalated for six hours before an unrelated alert on the same host forced a second look.

The fix: Treat a backlog spike as a staffing problem to solve with the tools built for it — surge coverage or overtime from Part 6's shift-design options — not a reason to delete a trust-ladder level. If the ramp program has no headroom to absorb a bad week without collapsing, the headcount model in Part 5 didn't account for ramp-time shrinkage in the first place.

A spot-check "fail" during Phase 3 should mean something narrower than a general QA score: a small, fixed list of critical criteria (wrong severity, missed escalation, disposition unsupported by the evidence gathered) rather than the full stylistic rubric Part 15 runs against the whole team. Ramp spot-checks are a binary gate; Part 15's QA program is a calibrated, ongoing measurement. Treating them as the same instrument invites exactly the confusion §8 below covers.

6. Phase 4 — simulation certification

[SENIOR MANAGER] Certification is a single, deliberately realistic exercise, not a written quiz. The two credible formats are an unannounced scenario injected into a real shift and a dedicated live-fire session run against a lab environment — either way, the analyst doesn't know in advance which alert in front of them is the test.

Cross-Book Pointer *This part doesn't specify what makes a certification scenario's underlying detection logic or telemetry realistic rather than a hand-crafted toy — that's a technical-content question with two existing owners. Detection Engineering Handbook V2, Part 37 — Detection Testing documents the test-case patterns (firing for the right reason, behaving correctly against a realistic attack tool run at natural volume and timing) that separate a sound test scenario from a synthetic one; SOC Playbook Handbook, Part 34 — Playbook Testing documents the simulation and lab-based methods for producing that scenario's telemetry in the first place. Pull certification scenarios from both rather than inventing new technical content for each ramp cohort — a certification exercise built on an untested or unrealistic scenario certifies nothing.*

[FRONTLINE MANAGER] A pass means the analyst reaches a defensible disposition, follows the correct escalation path where one is warranted, and does so inside the same time window a fully independent analyst would be expected to hit on that alert type — not that they reach the exact disposition a specific reviewer happened to write. A fail routes to a two-week remediation plan targeting the specific gap the simulation exposed (a missed pivot, a wrong severity call, a skipped escalation step), followed by a retest against a different scenario in the same class. Retesting with the identical scenario only proves the analyst can remember one exercise, not that the underlying gap closed.

Field Test Setup: *An analyst is in the last week of Phase 3, spot-check pass rate already at or above 95%. **Action:** Without advance notice, inject one seeded scenario, drawn from the technical sources named in the Cross-Book Pointer above, into their queue during a normal shift, timed for a moment the queue isn't already unusually heavy. **Expected result:** A defensible disposition and correct escalation path within the alert type's normal handling-time range, produced without the analyst realizing mid-exercise that it wasn't a real ticket. If the analyst asks a reviewer for help before finishing, that's diagnostic information about how Phase 3's spot-checks have been run, not an automatic fail — note it and adjust the next cohort's Phase 3 coaching rather than only re-scoring this one analyst.*

6.1 The certification report

[SENIOR MANAGER] Certification produces one short document, not a pass/fail email. Table 11.3 (CONCEPTUAL SAMPLE) is the minimum a certification report should capture — thin enough to actually get filled out under real time pressure, complete enough to survive a promotion committee or an auditor asking why someone was certified.

Field	What Goes Here
Scenario used	Which Detection Test / playbook-testing scenario, and its source part
Observed disposition	What the analyst actually concluded, in their own words
Escalation path taken	Whether escalation was warranted, and whether the analyst took it
Time to disposition	Elapsed time, compared to the alert type's normal handling-time range
Result	Pass, or fail with the specific gap named
Remediation plan (if failed)	The 2-week plan and the retest scenario class
Sign-off	SOC manager name and date

A report with only a pass/fail checkbox and no "observed disposition" field can't support a later challenge — from the analyst, from HR, or from an auditor — about why the certification decision was made. Keep the analyst's actual words, not just the reviewer's summary of them.

7. Milestones, sign-off, and the competency tie-in

[SENIOR MANAGER] Each gate in Table 11.1 should map onto specific rows in the competency matrix from Part 10, not a separate ramp-specific rubric invented in parallel. If Part 10 defines "ready for L2" as a named set of observable technical, tool, communication, and judgment criteria, Phase 4's certification is the moment a manager checks the ramp analyst against that same matrix for the first time — not a softer, onboarding-only bar that gets quietly replaced by the real one later.

[HR/PEOPLE] Most organizations run a 90-day probationary period in parallel with exactly this program, and the two should share a calendar rather than run on separate tracks that happen to both end around day 90. A fillable version of the phase/gate structure in Table 11.1, with space for reviewer sign-off at each gate, lives in Appendix A3's 90-day onboarding plan template — use it as the shared document HR, the team lead, and the analyst all reference, rather than three informal tracking systems that quietly drift out of sync with each other.

8. Where the program actually breaks

[HR/PEOPLE] Beyond the backlog-collapse pattern in §5, the two other recurring failure modes are a checklist standing in for readiness, and a single Phase 4 result being treated as more final than it is.

Operational Reality *A completed onboarding checklist tells you the analyst was exposed to a list of topics; it doesn't tell you they can act on any of them under real pressure. A program that lets a manager mark onboarding "complete" against 100% checklist completion while the Phase 3 spot-check pass rate sits at 80% is measuring the wrong thing — the checklist is a training-exposure record, not a readiness signal. Gate advancement on the spot-check number and the certification result; keep the checklist as a compliance artifact, not the graduation criterion.*

A single Phase 4 failure is a remediation trigger, not a termination trigger — apply the same diagnostic question Part 16 develops fully for ongoing performance issues: is this a skill gap the two-week remediation plan can close, or is it evidence the hire itself was wrong for the role. A second consecutive failure on a retest against a different scenario is a materially different signal than a first failure, and the two shouldn't be handled identically.

9. Measuring whether the program is working

[SENIOR MANAGER] Four numbers tell a manager whether the program is actually working, and none of them duplicate the queue metrics SOC Playbook Handbook, Part 32 — Metrics already owns (MTTR, FP rate) or this book's own operational terms (handle time, Part 5 §2; backlog age, Part 9 §1). These are onboarding-specific and have no earlier definition anywhere in the series:

- **Time-to-Level-4** — days from start date to full independent shift authority. A rising trend across cohorts, with no change in the hiring bar, usually means Phase 2 or Phase 3 coaching capacity hasn't scaled with hiring volume.
- **Phase 3 spot-check pass-rate trajectory** — the week-by-week curve, not just the final number that clears the gate. A flat curve that jumps to 95% right before the gate is graded, rather than climbing gradually, is worth a second look at whether spot-checks are scheduled predictably enough for an analyst, or a sympathetic reviewer, to prepare for them in advance.
- **90-day and 180-day retention of each ramp cohort**, tracked separately from the team's overall attrition number in Part 18. Early-tenure attrition traceable to a rushed or badly paced ramp looks identical to burnout-driven attrition in an aggregate figure; splitting the cohort out is the only way to tell which lever in Part 18 is actually the fix.
- **Reviewer time spent per ramp analyst**, tracked against the cost model in §1. A program whose actual mentor-hour cost is running well above the illustrative 40 hours in that example has either under-resourced its reviewers or is running phases longer than Table 11.1 assumes.

10. Adapting the pacing for lateral hires and career-changers

[FRONTLINE MANAGER] The four-phase structure holds for every new hire; the duration of each phase should not. An experienced lateral hire and a career-changer from Part 7's apprenticeship pipeline are not the same ramp problem, and pacing both identically wastes the experienced hire's time or overwhelms the career-changer.

Table 11.4 (CONCEPTUAL SAMPLE) maps typical background to a pacing adjustment, using the initial assessment score from Part 8 as the input that decides how much to compress.

Candidate Background	Typical Adjustment	Rationale
Experienced lateral hire (2+ years at another SOC, strong Part 8 assessment score)	Compress Phase 1 to 3–5 days; do not compress Phase 4	Already knows how to triage generically; doesn't yet know this SOC's tools, escalation paths, or client-specific context
Help-desk-to-SOC internal transfer	Standard-length Phases 1–2; often faster through Phase 3	Knows the environment and tooling already; still building security-specific judgment
Career-changer / apprenticeship pipeline (Part 7)	Extend Phase 1 by 1–2 weeks; do not compress any later phase	Needs more exposure time before supervised triage is productive rather than overwhelming
MSSP-to-in-house transfer, same industry	Standard length across all phases	Generic triage skill transfers; this SOC's severity model and escalation paths do not

Never compress Phase 4. Certification tests calibration to this specific SOC's environment, severity model, and escalation paths, not general triage skill — a background that justifies shortening Phases 1 through 3 says nothing about whether the analyst has internalized this team's specific gates yet.

[HR/PEOPLE] Document the compression or extension decision itself, not just the adjusted dates — a one-line note in the same onboarding plan referenced in §7, stating which row of Table 11.4 applied and why. Six months later, when someone asks why one analyst's Phase 1 ran 5 days and another's ran three weeks, "the assessment score and background justified it, and here's the note from day one" is a defensible answer; "the team lead's judgment call, unrecorded" is not — particularly if the two analysts in question don't share the same background, and the difference in treatment ever gets questioned.

Cross-references: This book — Part 5 (headcount and shrinkage math behind the ramp-cost example in §1), Part 6 (shift design for shadowing pairing and surge coverage as the alternative to collapsing the ramp in §5), Part 7 (career-changer and apprenticeship sourcing referenced in §10), Part 8 (assessment scores driving the pacing decisions in §10), Part 9 (queue health as the pressure behind the Management Autopsy in §5), Part 10 (competency matrix the graduation gates check against, §7), Part 14 (the mentorship program a graduated analyst enters next), Part 15 (the standing QA sampling rate an analyst moves onto at Level 4, §4 and §5), Part 16 (diagnosing a Phase 4 failure, §8), Part 18 (splitting ramp-cohort attrition from aggregate attrition, §9). SOC Playbook Handbook — Part 27, Escalation Quality (the standard Phase 2 reviewers hold drafts against, §4) and Part 34, Playbook Testing (simulation and lab methodology for certification scenarios, §6). Detection Engineering Handbook V2 — Part 37, Detection Testing (the test-case patterns behind a realistic certification scenario, §6).

Part 12 — Ongoing Training & Skill Development

Why this part exists

[CONCEPT] Four decisions land on a SOC manager's desk every year regardless of whether a formal program exists to make them: how much continuing-education money to spend and on what, whether a specific certification request is worth funding, whether a given analyst should be cross-trained into a second specialty, and whether the team's internal knowledge-sharing sessions are actually transferring skill or just consuming an hour of everyone's Friday. This part gives a manager a way to answer all four with a number or a named criterion instead of a gut call defended after the fact.

[CONCEPT] Three adjacent parts of this book own related but distinct ground, and the boundary matters enough to state before anything else. Part 11 owns the structured first 90 days — this part starts where that one ends, once an analyst is off the ramp plan and into steady-state work. Part 13 owns the ladder an analyst climbs and the committee that decides who moves up it; this part owns the skill-building that makes someone eligible, not the promotion decision itself. Part 14 owns formal mentorship pairing and succession planning for a specific irreplaceable role before its incumbent leaves; this part owns the standing, always-on program — budget, certification policy, rotation, and internal teach-backs — that reduces how often a role becomes irreplaceable in the first place. Where the actual technical content of a certification exam, a detection technique, or a hunting method belongs to SOC Playbook Handbook or Detection Engineering Handbook V2, this part cites that content and spends its own words on the program that decides who learns it, when, and on whose budget.

[CONCEPT] All four decisions share one underlying tension a manager should expect to hold rather than resolve once: every dollar and every scheduled hour spent building a specific person's skill is also a dollar and an hour that makes that person more valuable to a competitor, and every skill concentrated in one specialist for convenience is a coverage plan that survives only as long as that one person's attendance. This part doesn't pretend that tension goes away with a better policy. It gives you the specific levers — service commitments, bench-depth targets, format selection — that make the tradeoff deliberate instead of accidental.

1. The continuing-education budget as an operating line, not a perk

1.1 What the line actually has to cover

[SENIOR MANAGER] A continuing-education budget that only funds conference tickets and certification exam fees is funding the two most visible categories and missing the three that usually matter more day to day: paid study time (the hours an analyst spends studying are a real labor cost whether or not a line item ever says so), internal lab or range access (a sandboxed environment to practice an attack technique or a new tool without touching production), and content licensing (a training-platform seat, a CTF-platform subscription, a book budget). A budget built only from exam-fee receipts systematically understates what training actually costs, because the largest cost — an analyst's paid time not spent on the queue — never generates an invoice.

[SENIOR MANAGER]

Appendix A3 carries a reusable training-budget worksheet that breaks a per-analyst allocation into these five categories explicitly, rather than one blended "training" number, specifically so a manager defending the budget can show a CFO what happened to the money instead of a single total that invites an across-the-board cut. Part 20 — Building & Defending the SOC Budget covers where this line sits inside the overall budget structure and how to frame it against headcount and tooling; this section only covers what belongs inside it.

1.2 Sizing the line and watching where it actually goes

[SENIOR MANAGER]

A workable starting point for a mid-market in-house SOC is \$2,000–\$3,500 in direct spend per analyst per year, plus 24–40 paid hours of study/lab time — roughly 1–2% of a fully loaded analyst's annual cost. That range is a starting point for building your own number, not a benchmark to defend to a board without adjustment; a SOC running mostly commodity SIEM/EDR tooling with low specialist demand sits at the low end, and a SOC actively building an internal detection-engineering or threat-hunting bench (Section 3 below) should expect to run well above it in the specific years it's closing a bench-depth gap.

Manager's Note *Track utilization of the training budget the same way you'd track a PTO balance — a line that's 40% spent by October isn't "under budget," it's a program nobody is using, and the reason is almost never "the team doesn't want training." It's usually that nobody has time carved out of the queue to use it, which makes this a scheduling problem (Part 6) wearing a budget problem's clothes.*

[FRONTLINE MANAGER]

The failure mode a team lead actually sees is simpler than a budget spreadsheet: an analyst gets approved for a training day, then gets pulled back onto the floor because the queue spiked, three times in a row, and stops asking. Protecting a scheduled training block the same way you'd protect a scheduled on-call handoff — not "whenever the queue allows it" — is the entire fix, and it costs nothing beyond the discipline to say no to pulling that person back for anything short of a declared major incident.

1.3 Defending the line when someone asks for its ROI

[EXECUTIVE]

A training budget is one of the easiest lines for a CFO to flag during a lean year, because unlike a SIEM license, nothing visibly breaks the week after it's cut. The honest response is not to invent a fabricated return-on-investment percentage — it's to name what the line is actually insuring against: the bench-depth gaps in Section 3, which have a real, demonstrated cost when they go unaddressed, and the certification-linked capability that keeps the team from needing to backfill a specialist role externally at a market premium every time one is created internally instead.

[EXECUTIVE]

A more defensible framing than an invented ROI number is a direct cost comparison the board can check: hiring a mid-level detection engineer externally in most markets costs 15–25% of first-year salary in recruiting fees alone, plus three to six months of reduced output while the new hire ramps against unfamiliar tooling and environment context an internal candidate already has. Growing that same capability internally through the cross-training program in Section 3 costs the 15–25% capacity

draw quantified in 3.5, paid by an existing team member who already knows the environment. That's a comparison a skeptical board member can interrogate line by line, which is exactly what a manufactured ROI percentage can't survive.

What Would Change My Mind *This part treats internally grown specialist capability as generally cheaper than externally hired specialist capability once ramp time and environment-familiarity loss are counted. If a manager could show that external specialist hires in a comparable market consistently ramped to full productivity faster than an internally cross-trained analyst — not just cheaper on the recruiting-fee line alone — that would undercut this section's default recommendation and shift the calculus back toward external hiring for urgent specialist gaps.*

2. Certification strategy: what predicts capability versus what pads a résumé

2.1 The question that actually needs answering

[SENIOR MANAGER] Part 7 — Hiring & Sourcing Analysts already covers the hiring-side version of this problem: over-indexing on certifications during sourcing screens out capable non-traditional candidates who never needed a credential to be good at the job. The development-side version is the mirror image and just as costly — funding a certification for an existing analyst that adds a line to their résumé without changing what they can do in the queue, while a different, cheaper credential would have closed a real capability gap. Both mistakes come from treating "certified" as a single signal instead of asking what, specifically, the exam actually tests.

[SENIOR MANAGER] The question worth asking before approving any certification request is not "is this a respected certification" but "does passing this exam require doing the thing, or knowing about the thing." A proctored practical exam that requires compromising a live host, writing a working detection, or completing a forensic timeline under time pressure tests a skill directly. A multiple-choice exam tests whether the candidate memorized the vocabulary and framework well enough to recognize the right answer among four — valuable for breadth and shared vocabulary, but a weak predictor of whether that person can do the corresponding task unsupervised on a real incident.

2.2 A certification scorecard

[HR/PEOPLE] The table below scores the certifications a SOC manager most commonly gets asked to fund against the dimension that actually predicts on-the-job value: whether the exam format forces demonstrated performance rather than recognition. Use it to route a request, not to reject one outright — a low hands-on score doesn't mean a certification is worthless, only that it shouldn't be the thing you point to as proof of readiness for independent work.

CONCEPTUAL SAMPLE — illustrative scoring based on each certification's published exam format as of this writing, not a validated study correlating certification with measured job performance.

Certification	Exam format	Hands-on skill validated	Typical cost & renewal	Best use
CompTIA Security+	Multiple choice	Low	~\$400, renews every 3 years	Baseline shared vocabulary for a new-to-security hire; not a signal of triage capability
GIAC GCIH (Certified Incident Handler)	Proctored, scenario-based multiple choice	Medium	~\$2,500 incl. training, renews every 4 years	Structured incident-handling framework knowledge for an analyst moving toward Tier 2
GIAC GCFA / GCFE (forensics)	Proctored, scenario-based multiple choice	Medium	~\$2,500 incl. training, renews every 4 years	Forensic methodology grounding ahead of a hands-on forensic rotation, not a substitute for one
Offensive Security OSCP	Proctored, live practical exam (compromise real hosts)	High	~\$1,600 (exam + 90-day lab), no renewal	Genuine hands-on offensive-technique fluency; strongest for an analyst cross-training toward threat hunting or red-team liaison work
Security Blue Team BTL1 / BTL2	Practical, scenario-based (investigate real captured data)	High	~\$400–\$700, no renewal	Directly exercises SOC-analyst triage and investigation skill at low cost — underused relative to its signal strength
(ISC) ² CISSP	Multiple choice, broad domain coverage	Low for hands-on work	~\$750, renews annually with CPE credits	Management/governance-track credential; a poor proxy for triage or engineering capability, useful for team-lead-and-above roles that touch policy and risk framing
Cloud vendor security certs (e.g., a major cloud provider's security-specialty exam)	Multiple choice / scenario, platform-specific	Medium	~\$300, renews every 1–2 years	Fast, cheap way to validate platform-specific knowledge once an analyst's environment has real cloud telemetry to triage

[HR/PEOPLE] Two patterns in that table are worth naming directly. First, cost and hands-on validity don't move together — BTL1/BTL2 costs a fraction of CISSP and tests the actual job more directly, which makes it a better default funding choice for a Tier 1 or Tier 2 analyst than the more expensive, more widely recognized credential. Second, CISSP's low hands-on score is not a knock on the certification's actual purpose — it's a mismatch between what it tests (governance, risk framework breadth) and what a SOC manager is usually trying to buy with a training budget (demonstrated triage or engineering skill). Fund CISSP for someone heading toward a team-lead or manager track where that breadth is the actual job; don't fund it as a stand-in for hands-on readiness.

2.3 Certification stacking and diminishing returns

[HR/PEOPLE] A second, quieter failure pattern shows up less often as a single bad decision and more as a slow drift: an analyst accumulates a fourth or fifth certification in adjacent domains — Security+, then GCIH, then a cloud-security cert, then a second incident-handling credential — with each one individually defensible and the combination adding almost nothing to what the analyst can actually do that the second certification hadn't already covered. Certifications that test overlapping domains (most entry- and mid-level security credentials cover the same core incident-response vocabulary) have

sharply diminishing marginal value past the second one in a given skill area, even though each individual exam still costs close to full price and full study time.

[HR/PEOPLE] The practical check before approving a third-or-later certification request in the same domain is the same question from 2.1 applied to the *marginal* exam rather than the first one: what can this specific credential validate that the analyst's existing certifications and demonstrated work don't already show? If the honest answer is "very little, but it looks good on a proposal we're bidding on" or "the analyst just likes collecting them," that's a legitimate reason to fund it out of a marketing or business-development budget rather than the skill-development line — the two purposes are different and conflating them hides how much the training budget is actually buying in capability versus credibility theater.

2.4 Worked example — the certification-and-exit pattern

CASE-1201 — the certification-and-exit pattern. *COMPOSITE CASE EXAMPLE, merging patterns from several mid-sized in-house SOC's into one illustrative case; no single organization's data is reproduced.*

[SENIOR MANAGER] A 16-analyst in-house SOC at a mid-market healthcare organization adopted a simple policy: any analyst with 12 months of tenure could request full sponsorship for the OSCP, no questions asked, as its primary visible investment in technical growth. Fully loaded, one certification came to roughly \$5,500 per analyst — the \$1,600 exam and lab fee, plus about 80 hours of paid study time at a loaded rate near \$50/hour. Over two years, seven analysts completed it. Five of the seven resigned within six months of certifying, each taking a role elsewhere with an average total-compensation increase of about 28% (roughly \$65,000 to \$83,000), most often into penetration-testing or red-team roles the OSCP is specifically built to signal readiness for. The SOC spent approximately \$38,500 across the seven certifications and retained the resulting skill increase in only two of the seven people past the six-month mark.

The program wasn't badly chosen on the merits — OSCP is a legitimately strong, hands-on-validated credential, and it genuinely improved the five departing analysts' offensive-technique fluency. The failure was structural: an unconditional, unbonded sponsorship of a certification whose primary external market value points directly at roles the SOC doesn't offer internally is a subsidized recruiting pipeline for other employers. The fix applied afterward had three parts — a 12-month service commitment (with pro-rated repayment on early departure) attached to any certification costing more than \$1,000, a rule that certification sponsorship requests get evaluated against whether the SOC has, or is building, an internal role the credential leads toward, and stacking certification funding with an actual internal track change (Part 13's ladder) so the raise the certification earns happens inside the organization instead of at the next one.

2.5 Certification sponsorship and the retention trap

People Risk Trap Funding a certification whose strongest external market value points at a role you don't have open internally trains your best analysts to leave, on your budget, at the exact moment they become most valuable. Before approving spend above roughly \$1,000, ask what internal role or track this credential is preparing the analyst for, and attach a service commitment or pro-rated repayment clause to anything that doesn't have a clear answer. This isn't a reason to stop funding strong certifications — it's a reason to stop funding them unconditionally.

[VENDOR/PROCUREMENT]

The exam voucher and renewal fees themselves are a small, predictable recurring cost worth negotiating directly with the certifying body or training vendor at volume — most GIAC- and vendor-affiliated programs offer a bulk-voucher discount once a SOC is purchasing more than four or five seats a year, typically in the 10–15% range. Track renewal dates centrally rather than per-analyst; a lapsed certification an analyst was relying on for a client-facing MSSP requirement (Part 22) is a contract-compliance problem discovered at the worst possible time, not a training-administration inconvenience.

3. Cross-training: closing single points of knowledge failure

3.1 Naming the risk specifically

[SENIOR MANAGER]

A single point of knowledge failure is a specific, checkable condition: exactly one person on the team can perform a given specialist function unsupervised, with no one else able to cover it at an acceptable quality bar if that person is unavailable for more than a day. It is distinct from a single point of *staffing* failure (not enough people to cover a shift, Part 5's territory) — the team can be fully staffed on paper and still have a knowledge concentration risk if the one person who understands the custom correlation-rule logic, the SOAR playbook internals, or the TIP integration is on vacation, on leave, or gone.

[SENIOR MANAGER]

Part 3 — Tiering Models: L1/L2/L3 and Beyond defines the specialist tiers — detection engineering, threat hunting, incident response — that sit above the generalist L1/L2 structure. Cross-training, for this part's purposes, means deliberately building a second (or third) person's competence in one of those specialist functions or in a peer generalist tier, not simply exposing someone to it in passing. Sitting next to the detection engineer for an afternoon is exposure; being able to triage, modify, and redeploy that engineer's rule set unsupervised six months later is cross-training that actually closed the gap.

3.2 Building a cross-training matrix

[SENIOR MANAGER]

The tool that makes a knowledge-concentration risk visible before it becomes a crisis is a simple matrix: list every specialist function down the rows, and for each one record how many analysts can currently perform it unsupervised, what the target bench depth is, and which format (Section 4) is doing the work of closing the gap. The table below is a conceptual template, not a specific team's real data.

CONCEPTUAL SAMPLE — illustrative bench-depth matrix, not a specific team's real staffing data.

Specialist function	Analysts currently unsupervised-capable	Target bench depth	Current gap	Closing format
Custom correlation-rule authoring and tuning	1	2	1	Paired rotation with the incumbent, 90 days
Threat hunting (hypothesis-driven)	1	2	1	Structured hunt-shadowing rotation, then solo hunt with review
SOAR playbook administration	1	2	1	Documented runbook + supervised playbook edit, 60 days
Threat-intel platform (TIP) integration and feed tuning	2	2	0	—
Incident-response lead (major incident)	2	3	1	Tabletop co-lead role, next two exercises

[SENIOR MANAGER] A matrix like this one turns "we should probably cross-train more" into a specific, funded plan: three of five specialist functions above show a one-person gap against target, which is exactly the pattern that produces the failure described in [CASE-1202](#) below. Rebuilding this matrix on a fixed cadence — quarterly is reasonable for most teams — keeps it from going stale the same way an unreviewed headcount model does (Part 5's Operational Reality on that same failure mode applies here without modification).

3.3 Running the rotation itself

[FRONTLINE MANAGER] A paired rotation needs four things decided before it starts, or it degrades into the same vague exposure described in 3.1: a fixed duration (60–120 days depending on the specialty's complexity — SOAR playbook administration is closer to the short end, custom correlation-rule authoring closer to the long end), an explicit list of tasks the trainee must perform under supervision before graduating (not "shadow the SME," but "author, test, and deploy one new correlation rule end to end with the SME reviewing every step"), a named point at which supervision formally ends, and who makes that call. The competency matrix from Part 10 is the natural authority for that last point — "ready to perform this function unsupervised" should be an observable, matrix-defined bar, not the mentor's informal comfort level, because a mentor who likes the trainee will tend to sign off early and one who's protective of the role will tend to never sign off at all.

[FRONTLINE MANAGER] The mentor's incentive in a paired rotation deserves a direct look, because it's easy to get backwards. An analyst who has been the sole owner of a specialist function for years, often precisely because it made them hard to schedule around, doesn't always experience "someone else can now do my job" as good news — some genuinely welcome the relief, and some quietly slow-walk the handoff without ever saying so. Naming this openly with the mentor before the rotation starts, and giving them credit for successful knowledge transfer as its own recognized contribution (not just a footnote to their individual output), heads off the version of this where the rotation technically ran for 90 days and nobody actually got signed off at the end of it.

Cross-Book Pointer *This part owns the decision to cross-train an analyst into detection engineering or threat hunting and the rotation program that gets them there — it does not teach the technical content of either discipline. For the actual methodology a hunter needs, see Detection Engineering Handbook V2, Parts 34–36 — Threat Hunting Methodology; for the detection-authoring and detection-as-code discipline a rotation into detection engineering should build toward, see Detection Engineering Handbook V2, Part 22 — Detection as Code. Once an analyst finishes a cross-training rotation and starts handing off escalations across a tier boundary, SOC Playbook Handbook, Part 27 — Escalation Quality defines what a good hand-off looks like mechanically — worth a refresher for someone operating in an unfamiliar tier for the first time.*

3.4 Worked example — the bench-depth audit

CASE-1202 — the bench-depth audit. *COMPOSITE CASE EXAMPLE, merged from patterns across several mid-sized SOC's; no single organization's staffing is reproduced.*

[SENIOR MANAGER] A 22-analyst SOC at a mid-market financial-services organization, running three shifts, had one analyst — call him the pattern's "Analyst A" — who had authored and personally tuned roughly 80% of the team's custom correlation rules over three years, mostly because he was good at it and it was faster for him to just do it than to teach it. When Analyst A took three weeks of parental leave, two of his rules began misfiring against a legitimate but newly changed upstream data source. Nobody remaining on any shift understood the rule logic well enough to retune it safely, so the on-call lead disabled both rules rather than risk a bad change to live detection — a defensible call in the moment that nonetheless reopened a real coverage gap for 11 days until Analyst A returned.

The post-incident review that followed ran a bench-depth audit across all five specialist functions the team relied on and found that four of the five had exactly one person who could perform them unsupervised — a bus factor of one on 80% of the team's specialist capability, discovered only because one of those four people happened to take planned, unremarkable leave. The fix was the matrix in 3.2, built for real, with a standing target of at least two unsupervised-capable people per specialist function within 12 months, funded from the training-budget line in Section 1 and staffed through the paired-rotation format in Section 4.

3.5 The real cost of cross-training, and why it's worth paying

[SENIOR MANAGER] Cross-training has a genuine, non-hidden cost: an analyst pulled into a rotation is, for the duration of that rotation, producing less of their normal output — reasonable to budget at a 15–25% capacity reduction for the person being trained during the active rotation window, plus a smaller draw on the mentor's time. A 90-day paired rotation on one analyst therefore costs something like 14–23 analyst-days of reduced throughput (90 days at that 15–25% draw), which is the real number to put next to the 11-day coverage gap in [CASE-1202](#) when someone asks whether the investment was worth it.

[SENIOR MANAGER] The comparison that actually justifies the spend isn't "cross-training costs nothing" — it costs something specific and schedulable. It's that an unplanned single point of knowledge failure

costs the same capacity hit at the worst possible time, with no lead time to schedule around it and a live coverage gap attached. Paying the cost on your own schedule, once, is cheaper than paying an unknown version of the same cost during someone's medical leave, resignation, or the exact week of a major incident.

4. Internal knowledge-sharing formats

4.1 The format menu

[FRONTLINE MANAGER] Five formats cover most of what a SOC needs, and they are not interchangeable — each transfers a different kind of knowledge and fits a different team size and shift pattern.

- **Brown-bag / lunch-and-learn.** A 30–45 minute session, one analyst presenting something they recently learned or built. Cheap, low-prep, good for surfacing what individuals already know; weak for deep skill transfer because it's a lecture, not practice.
- **Shadow rotation.** One analyst observes a specialist doing real work for a fixed period, without doing it themselves yet. Good for building situational awareness of a specialty before committing rotation budget to it; on its own, produces exposure, not competence — see 3.1's exposure-versus-competence distinction.
- **Paired rotation.** The format behind both cross-training case studies above — a less-experienced analyst does the actual work under a specialist's direct supervision for a fixed window, with explicit sign-off criteria for graduating to unsupervised. This is the format that actually closes a bench-depth gap; the other four support it or substitute for it when it isn't available.
- **Internal wiki / runbook library.** Asynchronous, written documentation of how a specific tool, rule set, or process actually works, maintained by whoever owns it. The only format that works the same way for every shift without live attendance — essential for a 24/7 team, and the first thing that goes stale the moment nobody's job explicitly includes updating it.
- **Post-incident teach-back.** A short session, distinct from the blameless postmortem covered in Part 29, where the analyst who handled a notable incident walks the team through what they saw and did, focused on transferable technique rather than root-cause analysis. Good for spreading a specific new investigative trick quickly while it's still fresh.

[FRONTLINE MANAGER] The table below lines up the five formats against what actually decides whether a manager should use one — cost, prep burden, and how strong a claim it supports about skill transfer — so a format gets picked for the job in front of it rather than out of habit (CONCEPTUAL SAMPLE — relative cost/cadence bands, not measured figures from any specific team).

Format	Prep/facilitation cost	Cadence that works	Transfer strength	Best evidence it worked
Brown-bag / lunch-and-learn	Low — one presenter, ~1 hour prep	Biweekly or monthly	Weak on its own	Attendees can restate the idea; rarely demonstrates a new capability
Shadow rotation	Low-medium — mentor's time, no formal prep	As needed, ahead of a rotation decision	Weak-medium	Trainee can describe the workflow accurately; cannot yet perform it
Paired rotation	High — dedicated mentor time over weeks	Ongoing per bench-depth gap (§3.2)	Strong	Trainee performs the task unsupervised and passes the matrix sign-off (§3.3)
Internal wiki / runbook library	Medium — ongoing maintenance, not a one-time cost	Continuous, owned by whoever maintains the relevant system	Medium, format-dependent	A different analyst follows the runbook unaided and gets the correct result
Post-incident teach-back	Low — one presenter, session tied to a real incident	Within 1–2 weeks of a notable incident	Medium	A later analyst applies the specific technique described, on an unrelated ticket

4.2 Fitting formats to a 24/7 shift-based team

[FRONTLINE MANAGER] A synchronous format — brown-bag, shadow rotation, teach-back — structurally excludes whichever shift isn't on the clock when it happens, unless it's recorded and the recording is actually watched afterward, which in practice happens rarely enough that a manager should assume it doesn't. A team running three shifts that schedules its only knowledge-sharing session at 10 a.m. is running a knowledge-sharing program for one shift and calling it a team-wide program.

[FRONTLINE MANAGER] The practical fix is not "record everything" — it's picking the right format for the shift-coverage problem specifically. Rotate the live session's time slot across a two- or three-week cycle so every shift gets a live turn eventually, and put the load-bearing content into the wiki/runbook format (asynchronous by construction) rather than treating a recording of a synchronous session as an adequate substitute. Part 6 — Shift Pattern & Coverage Design covers the handoff mechanics between shifts generally; this is the training-specific version of the same equal-access problem.

4.3 Telling real transfer from a performative meeting

Manager's Note *If you can't name one specific thing an analyst can now do that they couldn't do before a knowledge-sharing session, the session was entertainment, not training — good entertainment is fine to keep funding for morale, just don't count it against a skill-gap budget line.*

[SENIOR MANAGER] The honest test for any recurring knowledge-sharing format is whether it changes what someone can independently do afterward, checkable the same way a competency matrix (Part 10) checks readiness for a tier: pick one attendee, one session later, and ask them to perform the thing unsupervised. A brown-bag on a new attacker technique that never gets tested this way might still be worth running for morale and shared vocabulary, but it shouldn't be counted as having closed a skill

gap in the bench-depth matrix from Section 3 — only a format with an explicit graduation check, almost always the paired rotation, earns that credit.

Field Test Setup: A post-incident teach-back or brown-bag session on a specific technique has just run, and the presenter believes the technique is now understood by the team. **Action:** Two weeks later, without advance warning, hand a different attendee a ticket where the technique applies and ask them to work it without help. **Expected result:** The attendee should apply the technique correctly, or at minimum recognize where it applies and know where to look up the specifics. If most attendees can't do either, the session transferred awareness, not capability — schedule a paired-rotation follow-up instead of re-running the same format.

5. Building the annual training plan

5.1 Routing a skill gap to the right investment

[SENIOR MANAGER] Most training-plan mistakes come from applying one favorite format to every gap — funding certifications for everything, or defaulting to a brown-bag for everything — instead of routing each identified gap to the format actually built to close it. The decision tree below is the routing logic this part's sections build toward.

```
flowchart TD
    A["Skill gap identified"] --> B["Shared across<br>3+ analysts?"]
    B -->|Yes| C["Internal knowledge-share format<br>(wiki, brown-bag, or teach-back – $4)"]
    B -->|No, one analyst| D["Is this a new specialist<br>function, not just deepening<br>their current role?"]
    D -->|No| E["On-the-job coaching,<br>Part 16's performance cycle"]
    D -->|Yes| F["Internal SME available<br>to pair with?"]
    F -->|Yes| G["Paired cross-training<br>rotation ($3)"]
    F -->|No, needs external<br>validation/credential| H["Retention risk assessment<br>($2.5)"]
    H -->|Low flight risk,<br>clear internal track| I["Fund certification<br>($2)"]
    H -->|High flight risk,<br>no internal track| J["Fund training content only;<br>withhold full cert<br>sponsorship<br>nor attach service commitment"]
```

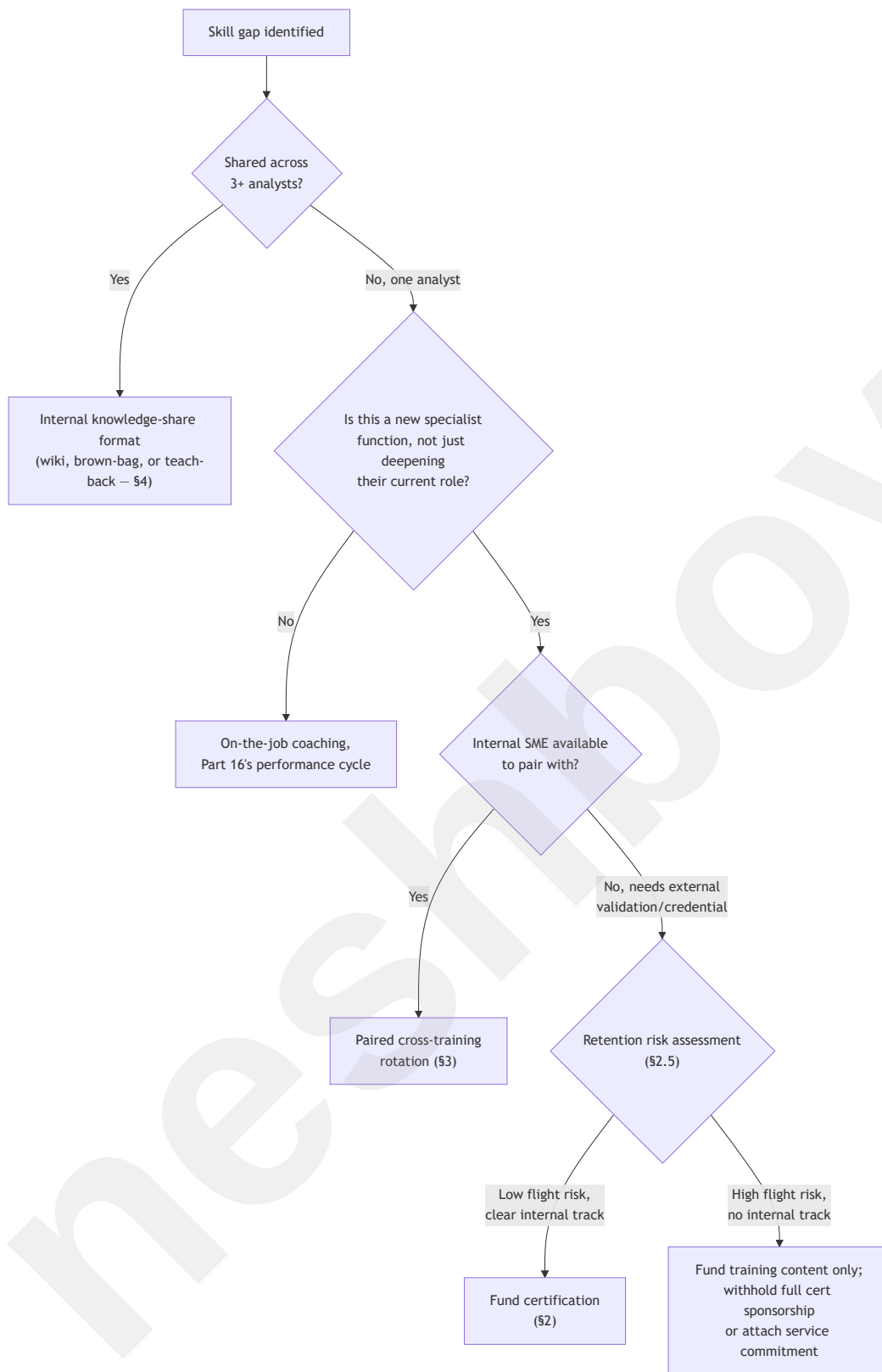


Figure 12.1 — Routing a skill gap to a training investment. *CONCEPTUAL.* Illustrates the decision sequence this part's sections build toward: whether a gap is shared or individual, whether it's a new specialty or a deepening of an existing role, and whether an external credential's retention risk is priced in before funding it. This is a structural decision aid, not a captured record of any specific team's actual approval workflow. Diagram ID **FIG-1201**.

5.2 Reviewing the plan against what actually happened

[EXECUTIVE] A training plan defended to a CISO or budget owner once a year and never revisited against outcomes is a forecast, not a program. Two numbers are worth reviewing against the plan every year, alongside the metric definitions owned by SOC Playbook Handbook, Part 32 — Metrics rather than redefined here: how many bench-depth gaps identified in the cross-training matrix (Section 3.2) actually closed on schedule, and how many certification-sponsored analysts left within 12 months of certifying, broken out separately from baseline attrition (Part 18's territory for the general number). A plan that closes gaps on schedule and doesn't lose certified analysts at a higher rate than the rest of the team is working; a plan that spends the full budget every year and can't answer either question is a spending habit wearing a program's name.

[SENIOR MANAGER] The training-budget worksheet in Appendix A3 has a column for exactly this — planned closure date against actual closure date per bench-depth gap — specifically so this review takes an afternoon against existing records instead of a reconstruction project every January.

Cross-references

Within this book: assumes Part 3 — Tiering Models (the specialist-tier definitions cross-training routes analysts into), Part 7 — Hiring & Sourcing Analysts (the hiring-side certification-screening problem this part's development-side counterpart mirrors), and Part 10 — Competency Models & Skills Matrices (the readiness criteria a training plan is built to close gaps against). Points forward to Part 6 — Shift Pattern & Coverage Design (equal shift access to synchronous training), Part 13 — Career Ladders & Promotion Criteria (what a completed certification or cross-training rotation feeds into, not decides), Part 14 — Mentorship & Knowledge Transfer (formal succession planning, distinct from this part's standing program), Part 16 — Performance Management & Coaching (on-the-job coaching versus formal training), Part 18 — Attrition & Retention (baseline attrition rate to compare certification-linked departures against), Part 20 — Building & Defending the SOC Budget (where this budget line sits in the whole), and Part 29 — Post-Incident Organizational Review (the blameless postmortem, distinct from the post-incident teach-back in §4.1).

Other volumes: Detection Engineering Handbook V2, Parts 34–36 — Threat Hunting Methodology and Part 22 — detection-as-code discipline (the technical content a cross-training rotation into those specialties actually teaches). SOC Playbook Handbook, Part 27 — Escalation Quality (hand-off mechanics for an analyst newly cross-trained across a tier boundary) and Part 32 — Metrics (the metric definitions this part's plan-review step consumes rather than redefines).

Part 13 — Career Ladders & Promotion Criteria

Why this part exists

[CONCEPT] A tiering model (Part 3) answers "who does what work today." A career ladder answers a different question: "how does a specific person move from one rung to the next, and who decides." The two get confused constantly, because the rungs of a ladder usually borrow the same labels as the tiers — Analyst I, Analyst II, detection engineer, team lead — and it is easy to mistake a staffing chart for a promotion policy. They are not the same artifact, and a SOC that only has the first one has no defensible answer when an analyst asks "what do I actually need to do to get promoted," which is the single most common question this part exists to answer in writing rather than in a hallway conversation that changes depending on who's asking and who's answering.

[CONCEPT] This part owns three things: defining the rungs from analyst through senior analyst and the branch point into detection engineer, threat hunter, or team lead; running the committee that decides who moves up those rungs, fairly and repeatably; and naming and fixing leveling drift — the specific failure mode where titles and actual demonstrated competency quietly stop matching each other. It does not own what a detection engineer or a threat hunter technically does day to day. Detection Engineering Handbook V2, Part 22 — Detection as Code and Part 34 — Threat Hunting Fundamentals already define those roles' technical content in the depth this book has no reason to duplicate; this part cites them exactly once each, for exactly what a candidate has to be able to demonstrate to earn the title, and moves on. It also does not own the competency matrix that defines "ready for the next rung" in observable terms — that is Part 10's job, and this part treats a finished competency matrix as an input it consumes rather than a thing it re-derives.

[CONCEPT] It also assumes Part 15's QA program and Part 16's performance-management process are already running, because a promotion committee that has no QA score and no performance-management record to draw on is making the decision on vibes, which is exactly the failure mode this part is written to prevent.

1. The ladder is not the tier chart

[CONCEPT] A tier chart is a snapshot: it says a 22-person SOC currently has fourteen Tier 1 analysts, five Tier 2, two detection engineers, and one team lead reporting to the SOC manager. A career ladder is a policy: it says what an Analyst I has to demonstrate to become an Analyst II, what an Analyst II has to demonstrate to be considered for the branch into detection engineer, threat hunter, or team lead, and what evidence a committee will actually look at before approving any of it. The tier chart can be redrawn every quarter as headcount and workload shift. The ladder should not move nearly that often — a ladder that gets renegotiated every time a manager wants to promote a specific person isn't a ladder, it's a title generator, and §5 covers exactly how that failure develops.

[SENIOR MANAGER] The practical reason to keep them separate: a tiering redesign (say, collapsing L1 and L2 into a single tierless "follow-the-alert" model, per Part 3's decision framework) changes who does which task on a given day. It should not, by itself, change what "Senior Analyst" means as a

demonstrated level of competency, pay band, and scope. Conflating the two means every org-design change turns into a promotion-policy renegotiation, which is slower than either change needs to be and teaches analysts that their level is a function of this quarter's org chart rather than a stable, portable credential they carry with them.

Manager's Note *If you can't answer "what would have to be different about this person's demonstrated work for them to already be at the next rung" without looking at the org chart, you're describing a tier, not a level. Levels describe a person; tiers describe a seat.*

[SENIOR MANAGER] A ladder also has a right number of rungs, and it's smaller than most first drafts. Five to seven total levels — Analyst I, Analyst II/Senior Analyst, the three specialist branches (or a combined "specialist" band if the SOC is too small to title them separately), and one senior/principal band above that — covers a SOC from roughly 10 to 60 analysts without needing a rung added every time someone asks for a bigger title. Adding a rung to solve a single person's compensation problem is the same mistake as an off-cycle promotion (§5.1 covers exactly this failure mode with a worked case): it treats the ladder as a title generator instead of a standing policy, and every extra rung is one more distinction a candidate, a peer, and eventually an auditor has to be able to explain the difference between.

2. Building the rungs: analyst through senior analyst

2.1 Analyst I to Analyst II: what actually has to change

[HR/PEOPLE] The most common defect in a first-draft ladder is defining the Analyst I → Analyst II jump entirely in tenure ("12 months in seat") with no competency bar attached. Tenure alone promotes the analyst who was merely present for a year exactly as readily as the one who got measurably better, and it gives a committee nothing to point to when a manager pushes for an early promotion or a peer asks why someone else got passed over. A defensible Analyst II bar is observable and multi-part, not a single number:

- Sustained handle time within 15% of the team median across the trailing 90 days, not a single good week.
- Independent closure — no escalation required — on at least 80% of the ticket types the L1 tier owns, measured against Part 3's own tier hand-off definitions.
- A calibrated QA score at or above 85%, sustained across two consecutive quarters under Part 15's calibration process, not a single reviewer's one-time score.
- Every "meets" rating on the Analyst II row of the Part 10 competency matrix — not an average across rows, because an analyst who is excellent at triage speed and weak at written communication has a real gap a single averaged number hides.

[HR/PEOPLE] None of these four bars is sufficient alone. Handle time alone promotes someone who's fast but escalates constantly to avoid genuinely hard tickets. QA score alone promotes someone thorough but too slow to carry a full queue. The four together are what makes the case defensible in

front of a committee that includes someone other than the candidate's own manager — the exact problem §4 exists to solve.

2.2 What "senior" should actually gate

[HR/PEOPLE] "Senior Analyst" earns its title by gating on judgment under ambiguity, not just more of the same triage at higher speed. A useful test: give the candidate a ticket type the runbook genuinely doesn't cover cleanly, and see whether they escalate immediately, guess and move on, or reason from the underlying threat model to a defensible disposition and document why. The third response is the one "senior" is supposed to certify. A ladder that promotes to Senior purely on volume and speed metrics is really just certifying "very fast Analyst I," and the title stops meaning anything the first time a genuinely ambiguous incident lands on that person's queue and they have no more judgment than they did a year earlier.

Operational Reality *In practice, most SOC's promote to Senior roughly a year to eighteen months after hire, because that's how long it takes handle time and QA scores to stabilize enough to trust — not because eighteen months is some magic threshold of judgment. Treat the timeline as a floor set by how long the evidence takes to accumulate, not a target. An analyst who hits every bar in §2.1 at month nine has met the bar at month nine; making them wait another year "because that's normally how long it takes" is exactly the kind of ungrounded rule that teaches your best people the ladder isn't really about the bar.*

3. The branch point: three roads out of senior analyst

[CONCEPT] Senior Analyst is the fork, not the destination. From there, a ladder needs at least three genuinely different paths, because detection engineering, threat hunting, and team leadership select for different aptitudes that don't reliably co-occur in the same person: detection engineering rewards someone who thinks in edge cases and false-positive mechanisms; threat hunting rewards someone comfortable pursuing a hypothesis that might resolve to nothing; team leadership rewards someone who gets more value from coaching four other people than from personally closing the hardest ticket in the queue. A ladder that only offers "stay an analyst or become a manager" forces every strong technical performer through a management funnel whether or not they want to manage anyone, which §3.4 covers as its own design failure.

Cross-Book Pointer *This part does not define what a detection engineer or a threat hunter actually does — the query languages, the detection-as-code review pipeline, the hunt methodology. Detection Engineering Handbook V2, Part 22 — Detection as Code covers the git-based rule pipeline, peer-review gates, and ownership model a detection engineer works inside every day; Part 34 — Threat Hunting Fundamentals and Part 35 — Hunt Types cover hypothesis formation, scoping, and the range of hunt types a candidate's completed-hunt record (§3.2) might draw from. Read those for the job. Come back here for how someone earns the title.*

3.1 The detection engineer branch

[SENIOR MANAGER] The entrance bar for this branch has to be a work-sample bar, not a self-declared interest. A candidate should have at least five detections merged through the actual detection-as-code review pipeline (DEH V2 Part 22) under their own authorship, with a post-deployment false-positive rate that stayed inside whatever tuning threshold Part 22's own review gate sets — not five detections drafted and abandoned, and not five detections someone else had to substantially rewrite before merge. A committee that promotes on interest alone ("they asked to move into detection engineering and seem sharp") gets a detection engineer who learns the review pipeline on the job, at the cost of every reviewer's time and every rule's early false-positive rate while that learning happens. Require the work sample before the title, not after.

3.2 The threat hunter branch

[SENIOR MANAGER] The equivalent bar here is a completed hunt record, not a completed course. A candidate should be able to point to at least two structured hunts run under DEH V2 Part 34–35's methodology, with at least one that either resolved to a documented negative finding (a real answer, not an abandoned hypothesis) or converted into a shipped detection candidate. The distinguishing failure mode for this branch is different from detection engineering's: a candidate can be technically strong at querying and still be a poor hunter if every hunt they've run started from "let's see what's weird in this data" rather than a falsifiable hypothesis. Test for hypothesis discipline specifically, not just query fluency — ask the candidate to state, before they touch the data, what a negative result would look like and what would make them abandon the hypothesis.

3.3 The team lead branch

[SENIOR MANAGER] This is the branch most SOC's get wrong, because the instinct is to hand the open team-lead seat to whoever is best at the individual-contributor job, and that instinct is measuring the wrong thing.

CASE-1301 — the fastest analyst becomes the team lead. *COMPOSITE CASE EXAMPLE* — constructed from patterns across multiple SOC's in the 12–20 analyst range, not one traceable organization.

Management Autopsy — "the fastest triage analyst becomes the team lead"

The decision: A 16-person SOC promotes its highest-QA-score, fastest-handle-time senior analyst directly into an open team-lead vacancy, with no transition period and no prior coaching assessment.

Why it seemed reasonable: She was, by every metric the team tracked, the best analyst on the floor, and promoting the visibly best performer into the open leadership seat is the promotion the rest of the team expects and rarely questions.

How it failed: Six months in, she was still personally closing the hardest 20% of the queue herself rather than delegating it, ran zero QA calibration sessions across two full quarters (Part 15 sets quarterly calibration as the baseline cadence), and two of her six direct reports had gone the full quarter with zero scheduled 1:1s. Team throughput was flat, her own workload had roughly doubled, and the two under-coached analysts both later cited "no real feedback loop" in their exit interviews (Part 18) within the year.

The fix: Separate the promotion decision from the transition. Run a structured coaching-apptitude assessment — a mock delegation scenario, a mock difficult-conversation role-play, using the same assessment-design discipline Part 8 applies to external hiring, just turned inward — before the offer, not after. Pair it with a 90-day shadow-lead period: reduced individual queue, one QA calibration session run under supervision, a mentor who is already a team lead. If the aptitude assessment comes back weak, that is real information the committee should be willing to act on, even for the team's strongest individual performer.

[SENIOR MANAGER] The bar for this branch, concretely: a passed coaching-apptitude assessment, a completed shadow-lead cycle covering at least one full stand-up rotation and one QA calibration session run with a current team lead observing, and — this is the part committees skip most often — explicit, voluntary interest. Nobody should end up in the team-lead branch because they were the obvious technical choice and nobody asked whether they actually wanted to spend their day coaching instead of triaging.

[SENIOR MANAGER] A candidate can fail any of these three branch bars on a first attempt without that failure becoming a career setback, and the ladder should say so explicitly. A detection engineer candidate whose first three merged rules ran hot on false positives, a hunter candidate whose two hunts both stalled on a hypothesis nobody could operationalize, or a team-lead candidate whose aptitude assessment came back weak on delegation specifically — each of those is diagnostic information, not a verdict. Set a fixed re-attempt window (two full quarters is a reasonable default, long enough to show a genuinely different result, short enough that the candidate isn't left in limbo) and require the committee to name, in writing, exactly which part of the bar wasn't met. A committee that says "not ready yet" with no specifics teaches the candidate nothing and reliably reads as a soft no rather than a real path forward.

3.4 The fourth road: staying a senior individual contributor

[HR/PEOPLE] A ladder with only three branches out of Senior Analyst — detection engineer, threat hunter, team lead — has a fourth, unlabeled branch by default: the analyst who is excellent at the individual-contributor job, doesn't want any of the three specialist tracks, and has nowhere to go but out. Build an explicit Principal Analyst or Staff Analyst rung that pays and titles like the specialist branches without requiring a role change — mentoring newer analysts, owning the hardest escalations personally, contributing to runbook and playbook quality without owning the QA program itself. Without this rung, a ladder quietly becomes "promote into management or your pay tops out," which is precisely the dynamic that pushes strong technical people into management roles they don't want and aren't suited for, purely because it's the only route to a bigger number on the offer letter. Part 18 covers the retention cost of this specific gap in more depth; the fix that actually closes it is a dual-ladder design decided here, not a retention program bolted on after the fact.

4. Running a fair promotion committee

[SENIOR MANAGER] A ladder with well-defined rungs still produces unfair outcomes if the decision sits with one manager evaluating their own reports against a bar only they interpret. A promotion committee exists to put more than one set of eyes, calibrated against each other, on every promotion decision before it ships.

4.1 Composition and cadence

[SENIOR MANAGER] A workable committee for a SOC in the 15–40 analyst range: the SOC manager (accountable for the final call), two team leads or senior peers from teams other than the candidate's own, and an HR partner for pay-band and equity review. Meeting quarterly, tied to the same cadence Part 15 already uses for QA calibration, keeps the promotion conversation from becoming a special, rare event that only happens when a manager pushes hard enough — a quarterly cadence normalizes it as a standing operational process instead.

flowchart TD

```
A["Team lead nominates candidate"] --> B["Evidence packet assembled\n(Part 10 competency matrix,\nPart 15 QA score, Part 16 record)"]
B --> C{"Committee review:\nbar met on all packet items?"}
C -->|"No – gap identified"| D["Written feedback to candidate\nand team lead; re-nominate\nnext cycle"]
C -->|"Yes"| E["Cross-team calibration check\n($4.3): consistent with other\nteams' recent decisions?"]
E -->|"Inconsistent"| F["Committee discussion;\nadjust or defer"]
E -->|"Consistent"| G["HR pay-band and equity review"]
G --> H["Final vote; SOC manager\naccountable for outcome"]
H --> I["Outcome communicated by\nteam lead within 5 business days"]
```

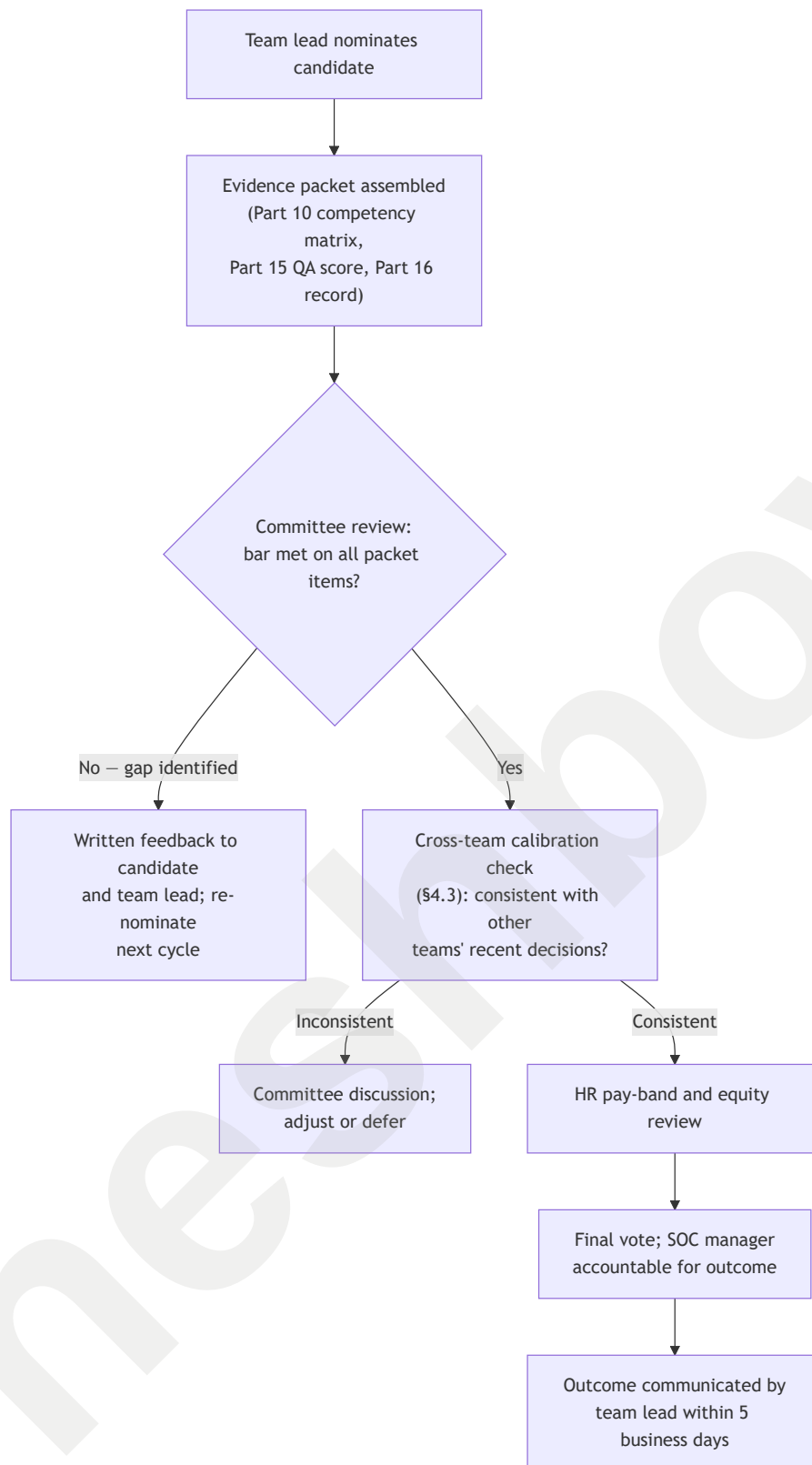


Figure 13.1 — Promotion committee decision flow, nomination through outcome. *CONCEPTUAL.* Illustrates the standing process a quarterly promotion committee runs for one candidate, from nomination through communicated outcome, including the cross-team calibration check that §4.3 exists to enforce. This is a process design, not a capture of any single organization's actual workflow. Diagram ID **FIG-1301**.

[SENIOR MANAGER]

The RACI below assigns one accountable owner — the SOC manager — across every step, which matters: a committee with no single accountable owner tends to diffuse responsibility for a bad call across everyone in the room, and nobody actually owns fixing the process when it produces an unfair outcome.

The table below maps who does what across a single promotion decision, so a new committee member (or a candidate asking "who actually decides this") can see the process at a glance.

Activity	SOC Manager	Nominating Team Lead	Promotion Committee	HR Partner	Candidate
Nominate candidate	C	R	I	I	—
Assemble evidence packet	A	R	C	I	C
Calibrate scores across teams	A	C	R	C	—
Final promotion vote	A	I	R	C	I
Communicate outcome	A	R	I	C	I

R = Responsible, A = Accountable, C = Consulted, I = Informed.

4.2 The evidence packet

[HR/PEOPLE]

A promotion case that arrives as a manager's paragraph of praise is not an evidence packet — it's an opinion with a title request attached. The table below is what a committee should expect to see before a candidate's name is even discussed.

Evidence item	Source	Minimum bar	Owner
Competency matrix rating	Part 10 skills matrix	"Meets" on every next-level row, not an average	Team lead
QA calibration score	Part 15 QA program	85% or higher, sustained across two quarters	QA reviewer
Handle time and independence	Ticket system data	Within 15% of team median; under 20% escalation rate	Team lead
Open PIP or coaching plan	Part 16 performance records	None open in the trailing 6 months	SOC manager
Peer input	Structured peer-nomination form	At least 2 peer submissions; no unaddressed concern	Committee
Branch-specific work sample	Detection PRs, hunt records, or shadow-lead log	Meets the \$3 branch bar	Team lead + specialist reviewer

Manager's Note Don't let the packet turn into a research project the team lead dreads assembling — most of this should already exist as a byproduct of running QA (Part 15) and performance management (Part 16) properly. If assembling a packet takes more than an hour of new work, the underlying record-keeping is the actual problem, not the promotion process.

4.3 Calibration across teams and managers

[SENIOR MANAGER]

The single most common way a promotion committee fails is silently, by letting a lenient manager's team get promoted at a materially higher rate than a strict manager's team for indistinguishable actual performance — and nobody notices until HR runs a pay-equity report eighteen months later.

CASE-1302 — cross-team promotion-rate gap, calibration fix. COMPOSITE CASE EXAMPLE — constructed from patterns across multiple mid-size SOCs, not one traceable organization. A 24-analyst SOC split across two teams under two different team leads promoted 22% of Team A's eligible senior-analyst pool to a specialist branch in one cycle, against 4% of Team B's — despite both teams averaging a nearly identical 3.4-out-of-5 rating on the Part 10 competency matrix. Team A's lead was simply a more generous nominator; Team B's lead held an informal, higher personal bar nobody else on the committee had agreed to. HR flagged the gap during a routine pay-equity review roughly a year and a half later, by which point Team B had two informal grievances about perceived favoritism. The committee's fix going into the next two cycles: blind the competency-matrix and QA scores from the team-lead name during the committee's first-pass review, and require a second reviewer from outside the candidate's own team on every case — the same cross-team-second-reviewer principle §4.1's RACI already assigns. Two cycles later, the promotion rates had converged to 9% (Team A) and 7% (Team B) — not identical, but no longer a gap large enough to read as favoritism rather than noise.

People Risk Trap *Letting a candidate's own direct manager be the only real decision-maker — with the "committee" rubber-stamping whatever that manager already decided — reproduces every one of that manager's individual biases at scale, and it's invisible until someone compares outcomes across managers the way CASE-1302 eventually did. The fix: require a reviewer from outside the candidate's reporting line on every case, blind the competency and QA scores from the nominating manager's name during the committee's first pass, and run an annual cross-team promotion-rate comparison as a standing check — not something HR only surfaces after a complaint.*

Field Test Setup: *A promotion committee exists on paper, with a published rung and branch bar, but has never actually voted on a case. Action:* Before the first real candidate reaches the committee, run it once against a deliberately borderline synthetic case — a fabricated evidence packet that clears three of the four §2.1 bars cleanly and misses the fourth by a small, defensible margin. **Expected result:** *The committee should split, at least initially, and the disagreement should trace back to which bar members weigh most heavily — not to who likes the fictional candidate. If every member immediately agrees with no discussion, the synthetic case wasn't ambiguous enough to test anything, and the committee hasn't actually learned whether it shares a bar yet.*

5. Leveling drift: the failure mode this part is named for

5.1 What it is and how it starts

[SENIOR MANAGER] Leveling drift is the gradual mismatch between a person's title and their actually demonstrated competency at that title's rung — in either direction, though the retention-driven direction is by far the more common one in a SOC. It rarely starts as a policy decision. It starts as a single, understandable exception.

CASE-1303 — the retention counter-promotion. *COMPOSITE CASE EXAMPLE* — constructed from patterns across multiple organizations, not one traceable incident.

Management Autopsy — "the retention counter-promotion"

The decision: *A senior analyst holding a competing offer roughly 18% above their current pay tells their manager they're leaving in two weeks. The SOC manager, unable to match the raise through base pay alone within the timeline, promotes them overnight to "Lead Detection Engineer" — bypassing the committee entirely — with no change in scope, no evidence packet, and no work sample against the \$3.1 bar.*

Why it seemed reasonable: *It was fast, it was cheaper than an equivalent base-pay counter-offer, and it kept a scarce mid-level analyst from walking out the door in a market where a replacement search realistically takes 8–12 weeks.*

How it failed: *The promotion worked exactly once, and then it became the template. Over the following year, three more retention-driven promotions happened through the identical shortcut — a manager, under time pressure, bypassing the committee to retain someone about to leave. An internal leveling audit eventually found that 45% of a 20-person team held a "Senior" or "Lead" title, but only about 20% actually cleared the Part 10 competency bar for that rung. New mid-level hires, brought in at competitive market rates, discovered they were being paid close to or above several title-inflated "seniors" with visibly less demonstrated capability. One formal pay-equity grievance was filed; exit interviews (Part 18) over the following two quarters cited perceived unfairness in the title-to-pay mapping as a contributing factor in at least two more resignations — the exact attrition the original counter-promotion was trying to prevent, now recurring for a different, self-inflicted reason.*

The fix: *One approved emergency-retention path, not an unlimited number of manager-improvised ones: a documented process where a manager can request an accelerated review, but the promotion committee still ratifies it — after the fact if timing truly requires it, within 30 days, never skipped — against the same evidence bar every other promotion uses. Publish a target ladder-distribution ceiling per level (§5.3) and run the leveling audit on a fixed schedule, not only after a grievance forces the question.*

5.2 The compounding cost

[SENIOR MANAGER] Leveling drift is expensive in a specific, calculable way, not just an abstract fairness problem. Every off-cycle title bump that isn't matched to an actual scope or pay-band change creates

one of two outcomes: either the person's pay eventually gets corrected upward to match the inflated title (a real, recurring cost the original budget never accounted for), or it doesn't, and the organization now has a "Senior" who is paid like an "Analyst II" and knows it — precisely the internal pay gap against market that Part 18's poaching-driven-exit pattern (Table 18.1) flags as the highest-risk setup for a resignation once a competing offer arrives. Either branch costs money; the drift just decides whether the organization pays it on purpose, through a corrected budget, or by surprise, through an attrition event and a re-hire at a worse market rate than the one it was trying to avoid in the first place.

[SENIOR MANAGER] There's a second cost that doesn't show up on a budget line at all: committee credibility. Once analysts learn that the fastest way to a promotion is a competing offer and a direct conversation with the manager rather than the evidence packet in §4.2, the whole apparatus in §4 becomes theater for the people who don't already know the workaround — and the people who do know it have every incentive to shop for outside offers specifically to trigger it, which is a strange incentive for a promotion system to be quietly running.

5.3 Auditing and correcting drift without a re-org bloodbath

[SENIOR MANAGER] The corrective tool is a leveling audit: compare the actual title distribution across the team against the ladder's own target distribution, run on a fixed schedule — semi-annually is a reasonable default — rather than only when a complaint forces it.

CONCEPTUAL SAMPLE — illustrative target and drifted distributions for a hypothetical 20-analyst SOC; not sourced benchmark data, and not every SOC's correct target ratio.

Level	Target % of headcount	Target headcount	Drifted actual headcount	Gap
Analyst I	30%	6	3	−3
Analyst II / Senior Analyst	40%	8	9	+1
Specialist / Lead (detection engineer, threat hunter, team lead)	25%	5	7	+2
Principal / Manager	5%	1	1	0

[SENIOR MANAGER] Read against a fully loaded pay-band delta of roughly \$14,000 a year between an Analyst II and a Specialist/Lead seat, the drifted column above represents about \$28,000 a year in unbudgeted labor cost sitting in titles the org chart the CFO approved never accounted for — on top of the fairness and credibility cost §5.2 already covers. That number is what makes a leveling audit a budget conversation (Part 20 owns defending the number to a CFO), not only an HR housekeeping exercise.

[SENIOR MANAGER] Correcting drift forward, never backward, is the rule that makes the audit survivable politically: never demote someone to fix a distribution table. Instead, freeze new off-cycle promotions outside the single ratified emergency-retention path, let the Analyst I and Analyst II rungs refill through ordinary hiring and ordinary promotion at the correct rate, and let the distribution converge over roughly two to three hiring cycles rather than a single disruptive correction. A drift audit that produces demotions will be read, correctly, as a punishment for a decision the analyst never made — the manager made it — and will do more damage to trust than the drift it was meant to fix.

What Would Change My Mind This part treats a semi-annual leveling audit as the right cadence for most SOC in the 15–40 analyst range. If a SOC of that size ran the audit only annually and still caught drift before it produced a pay-equity grievance or a drift-driven resignation — because its promotion committee's cross-team calibration (§4.3) was strong enough to prevent drift from accumulating in the first place — that would suggest the audit cadence is a backstop for weak calibration, not an independent requirement, and this part's default recommendation should soften to "audit at whatever cadence your calibration process's own track record justifies."

6. Sequencing this without breaking a team that's already running

[SENIOR MANAGER] None of §§2–5 should ship simultaneously in an organization with no existing ladder. Publish the rung definitions and the branch bars first, on their own, and let people see where they currently sit against a bar that didn't exist yesterday, before convening the first committee cycle — a committee with no published bar to work from is just the old hallway-conversation problem with a calendar invite attached. Run the first leveling audit before, not after, the first committee cycle, so any existing drift is disclosed and addressed on its own terms rather than discovered mid-cycle and treated as a reason to distrust the new process.

[SENIOR MANAGER] Expect the first two committee cycles to surface more disagreement than later ones — that's the calibration process in §4.3 doing its job, not evidence the process is broken. A committee that agrees on every candidate in its first quarter either has an unusually settled team or hasn't yet been given a case ambiguous enough to test whether its members actually share a bar.

[FRONTLINE MANAGER] A "not yet" is the outcome a new committee handles worst, because the team lead delivering it is usually the same person who nominated the candidate and now has to explain a gap in their own evidence packet. Deliver it against the specific bar item that wasn't met — "the QA score was 81% against an 85% bar, sustained for one quarter instead of two" — never as a vague "the committee felt you weren't quite ready." A specific gap is something the analyst can close on a known timeline; a vague one reads as a decision that was never really about the bar at all, which is the same trust problem §5.2 describes for leveling drift, arriving from the opposite direction.

Cross-references: This part assumes Part 3 — Tiering Models (the tier structure the branch point sits above), Part 10 — Competency Models & Skills Matrices (the observable bar every rung and branch cites), Part 15 — Quality Assurance Programs and Part 16 — Performance Management & Coaching (the QA scores and performance record the evidence packet in §4.2 draws on). It connects forward to Part 14 — Mentorship & Knowledge Transfer (the shadow-lead and mentor mechanics in §3.3), Part 18 — Attrition & Retention (the pay-compression and poaching cost leveling drift produces), and Part 20 — Building & Defending the SOC Budget (pricing a leveling audit's correction). Outside this book, it cites Detection Engineering Handbook V2, Part 22 — Detection as Code, Part 34 — Threat Hunting Fundamentals, and Part 35 — Hunt Types for what the detection-engineer and threat-hunter branches technically do, and nothing else — how someone is promoted into either role stays here.

Part 14 — Mentorship & Knowledge Transfer

Why this part exists

[CONCEPT] A resignation letter doesn't remove a person from the org chart — it starts a clock. For most analyst seats, that clock is generous: the role is documented, the queue is shared, and a reasonably competent replacement can be productive within weeks. For two seats on almost every SOC roster, the clock is short and the damage compounds fast: the team lead who's been quietly absorbing every scheduling exception and every "who do I escalate this to" question for three years, and the detection engineer who's the only person who remembers why 40 suppression rules exist and which ones are load-bearing. When either one leaves without a plan already in motion, the team doesn't lose a person — it loses a function, for however long it takes to rebuild the knowledge and judgment that person carried alone.

This part covers three things: designing a formal pairing/mentorship program instead of leaving development to whichever pairs of people happen to like each other; capturing the tribal knowledge concentrated in one person's head before that person is walking out the door, not after; and building a named-successor plan for team-lead and detection-engineer roles specifically, because those two roles fail differently than a rotating analyst seat does. It deliberately does not cover the career-ladder criteria a successor is measured against — that's Part 13 — Career Ladders & Promotion Criteria — or the ongoing, breadth-building cross-training program that reduces single-points-of-knowledge-failure across the whole team as a matter of course, which is Part 12 — Ongoing Training & Skill Development's job. This part is narrower and more urgent than either: it's what a manager does about the specific knowledge and the specific roles where a gap is already forming, whether or not the broader training program has gotten there yet. It connects forward to Part 18 — Attrition & Retention, which quantifies the cost of the coverage crisis this part exists to prevent; read that part for what attrition costs, and this one for how to make sure a single resignation doesn't turn into one.

1. Two problems wearing one name

[CONCEPT] "Mentorship program" gets used as a label for two different things that fail for different reasons, and a program that conflates them usually does neither well.

1.1 Mentorship: a career-development relationship with an owner

[HR/PEOPLE] Mentorship is a standing relationship aimed at the mentee's general growth — judgment, communication, career direction, confidence in ambiguous situations — that has no fixed deliverable and no natural end date tied to a specific piece of information. A good mentor for a Tier 1 analyst two years into their first SOC job might spend a session talking about how to push back on a manager's unrealistic timeline, not about any specific detection or tool. The relationship's success metric is the mentee's trajectory over quarters, not a checklist of topics covered.

1.2 Knowledge transfer: capturing what one person's head currently holds alone

[SENIOR MANAGER]

Knowledge transfer is narrower and has a deadline: a specific, nameable body of tacit knowledge — why this correlation rule's threshold is set where it is, which vendor contact actually picks up the phone during an outage, which three customers' environments have a quirk that breaks the standard playbook — currently exists in one person's head and needs to exist somewhere durable and shared before that person is unavailable, whether "unavailable" means resigned, on leave, or just on vacation during the one week it mattered. Unlike mentorship, knowledge transfer has a clear finish line: the knowledge is captured, or it isn't.

Manager's Note *If your "mentorship program" consists entirely of pairing a new hire with a senior analyst for their first 90 days, you don't have a mentorship program — you have an onboarding buddy system, and that's fine, but call it what it is. Onboarding's structure and pacing is Part 11 — Onboarding & Ramp-Up Programs's job; a mentorship program that only runs during someone's first 90 days on the team isn't investing in the analyst who's been there for two years and is starting to look at other postings.*

The table below separates the two by owner, trigger, and how each is judged to have worked, because a program built to solve one badly serves the other.

Dimension	Mentorship	Knowledge Transfer
Trigger	Ongoing career development, no specific event	A named departure, role change, or an identified single point of knowledge failure
Typical Duration	Months to years, informal end	Days to a few weeks, sharp end point
What's Captured	Judgment, confidence, career navigation	Specific facts, decisions, and undocumented context
Success Looks Like	Mentee's trajectory improves over quarters	A written, findable artifact exists and someone else can act on it
Primary Owner	The mentor and mentee, manager checks in	The manager, actively driving the timeline
Fails Silently When...	Pairing happens once, never followed up	The departing person's notice period runs out before the interview does

2. Designing a formal pairing/mentorship program

2.1 Why "let it happen organically" guarantees uneven coverage

[FRONTLINE MANAGER]

Informal mentorship happens to the analysts who are already easy to mentor — confident enough to ask a senior person for time, personable enough that a senior person offers it unprompted. It reliably skips the introverted analyst doing solid work in the corner of the queue, the remote hire who's never shared a physical break room with anyone senior, and the analyst whose personality doesn't naturally click with whoever's available. None of those are reasons to write someone off; they're reasons a manager can't rely on chemistry to distribute a resource this important.

Manager's Note *Ask your best analyst who their informal mentor is. If the honest answer is "nobody, I just figured it out," that's not a compliment to their independence — it's a data point that your program (if you have one) missed your strongest performer, and it'll miss the next one too unless assignment is deliberate rather than opt-in.*

2.2 Matching mentors to mentees on more than tenure

[HR/PEOPLE] The default matching logic — most senior available analyst, paired with newest hire — optimizes for the wrong variable. Tenure predicts technical knowledge, not coaching ability, and a mentor who's a poor teacher but a strong individual performer produces a mentee who feels unsupported despite being "assigned" someone qualified. Match on three things instead: demonstrated willingness to teach (ask candidates directly, don't assume), a skill or knowledge gap the mentee actually has that the mentor actually holds, and enough scheduling overlap that the pairing can meet without either person's shift making it structurally hard. A mentor two time zones and a rotating shift away from their mentee will let the relationship lapse within a month, not because either person is negligent, but because "we'll grab fifteen minutes" never has a fifteen minutes that works for both of them.

2.3 Structure, cadence, and a defined end point

[FRONTLINE MANAGER] A pairing with no structure decays into occasional hallway chat, which is better than nothing but not what a program is for. Set a fixed cadence — every two weeks is a reasonable default for a working pairing, weekly for a mentee in their first six months past onboarding — a rough agenda template (not a rigid script) covering what's going well, what's stuck, and one thing the mentee wants to get better at before the next session, and a defined program length of six to twelve months with an explicit renewal decision at the end rather than a relationship that just trails off unacknowledged. An open-ended commitment with no review point is the single most common reason a mentoring pairing that started well goes quiet by month four: neither person wants to be the one to say it isn't working, so instead it just stops, with no signal to the manager that the mentee is now unsupported again.

2.4 What the program should never be used for

[SENIOR MANAGER] A mentor is not a shadow performance manager, and mentorship notes are not performance-review input. The moment a mentee suspects that what they say in a mentoring session might reach their formal review, they stop saying anything useful, and the program's actual value — a relationship where it's safe to admit "I don't understand this escalation path" or "I think I mishandled that ticket" — disappears. Keep mentorship structurally separate from the reporting line: a mentor should never be their mentee's direct manager, and a mentor should never be asked to report on a mentee's performance, only on whether the pairing itself is functioning.

Blind Spot A program that measures itself by "number of active mentoring pairs" is measuring participation, not transfer. Two people can meet every two weeks for a year and produce nothing more durable than pleasant conversation if neither ever writes anything down or changes any observable behavior. Pair the participation count with a light mentee self-report every quarter — one question, "what's one specific thing you can do now that you couldn't six months ago" — or the program will report full health right up until the mentee who was quietly disengaged the whole time hands in their notice.

2.5 Sizing the program: how many pairs one manager can actually run

[SENIOR MANAGER] A manager personally checking in on every mentoring pair loses the ability to do that check-in honestly somewhere past eight to 10 active pairs — beyond that, "how's it going" stops being a real audit of the relationship and becomes a rubber stamp, because there isn't enough attention left to notice which pairs have quietly gone quiet. On a 30-analyst team, that arithmetic forces a choice: delegate pair oversight to team leads, each tracking their own shift's pairings and rolling up a summary rather than the manager owning every relationship directly, or run the program in cohorts — starting eight to 10 new pairs at a time, staggered a quarter apart, rather than pairing all 30 analysts simultaneously and being unable to meaningfully track any of them. Either choice is fine; running the program at a scale nobody can actually supervise is the failure mode, not a specific structure.

3. Capturing tribal knowledge before it walks out the door

3.1 Naming the risk: bus factor, not "hit by a bus"

[SENIOR MANAGER] The engineering-culture term for this is bus factor: the number of people who would need to become simultaneously unavailable before a specific piece of knowledge or capability is lost to the team. A bus factor of one on anything that matters is a standing risk, whether or not the specific person carrying it has any intention of leaving — illness, a sudden family emergency, or a competitor's unsolicited offer all produce the identical operational gap with none of the warning a two-week resignation notice at least provides. Part 12 — Ongoing Training & Skill Development owns the standing remedy for this at the skill-breadth level: cross-training analysts across specialties so competence isn't concentrated in one person in the first place. This part's job is narrower — finding the bus-factor-of-one knowledge that's already concentrated, right now, before the program-level fix has caught up to it, and doing something about the two roles (team lead, detection engineer) where that concentration is most damaging when it breaks.

3.2 The knowledge-capture audit

[SENIOR MANAGER] Most managers can't answer "what does this person know that nobody else does" with any precision until they're asked to answer it under deadline pressure during an exit interview, which is exactly the point at which the answer is hardest to extract completely. Run the audit before there's a departure: for each person in a role with meaningful bus-factor risk, list the systems they're the sole administrator or approver for, the vendor or client relationships they're the only point of contact for, the tuning decisions or exception lists with no documented rationale anyone else can point to, and

the "who do you call" answers that only exist in their head. This is a 15-to-30-minute structured conversation, not a survey — most people can't self-generate this list unprompted, because knowledge that's become invisible to its holder ("of course everyone knows that") is exactly the knowledge most likely to be genuinely undocumented. Ask about, specifically:

- Systems or accounts where this person is the sole administrator, approver, or credential holder, including anything with a shared password only they actually know.
- Vendor, client, or cross-team contacts where this person is the only one with a working relationship — the person who actually returns a call inside an hour versus the generic support line everyone else gets.
- Tuning decisions, suppression rules, or exception lists with no written rationale anyone else can point to, including anything this person would describe as "we just always do it that way."
- The specific escalation judgment calls this person makes that a written runbook doesn't capture — not "what's the process" but "when do you deviate from the process, and how do you know."

Run this against every name on the §4.2 succession scorecard at minimum, and against any analyst whose unplanned two-week absence would visibly disrupt operations, whether or not they hold a title that sounds senior.

Field Test Setup: Pick one analyst or engineer whose sudden two-week absence would meaningfully disrupt operations — not necessarily the one you'd guess first. **Action:** Without telling that person in advance, ask their manager or team lead to answer, from memory, the four questions in §3.2 (sole administration, sole points of contact, undocumented tuning rationale, "who do you call" knowledge) for that person's role. **Expected result:** If the manager can answer completely and cites a written source for each answer, bus-factor risk is genuinely low. If the manager has to guess, hedge, or say "I'd have to ask them," the audit has already found a real gap — treat that gap as the first item on the capture backlog, not as evidence the exercise failed.

3.3 Turning tacit knowledge into a durable asset

[SENIOR MANAGER] Capture only works if what comes out of the audit lands somewhere a second person will actually find and use it, which rules out a one-time knowledge-dump document nobody links to from anywhere the team actually looks. Where the captured knowledge is procedural — how to run a specific escalation, how to handle a specific recurring client request — it belongs in the same playbook library the rest of the team already checks, not a separate mentorship-program wiki nobody remembers exists.

Cross-Book Pointer This part does not cover how to structure or write a good playbook once tribal knowledge needs to become one — that's *SOC Playbook Handbook, Foundations / Playbook Library*, which owns playbook mechanics across the whole series. Use the audit in §3.2 to find what needs writing down; use that book's structure to actually write it well.

Where the captured knowledge is a detection engineer's rule-tuning history and rationale, the format is different: it belongs as metadata traveling with the rule itself, not as prose in a separate document that will drift out of sync with the rule the moment someone edits it without also updating the write-up.

Cross-Book Pointer *This part does not cover how a detection repository should structure ownership metadata, version history, and review trails — that's Detection Engineering Handbook V2, Part 22 — Detection as Code, which treats exactly this problem (a rule's tuning rationale living only in one engineer's memory) as a structural risk to be solved with metadata standards and CI-enforced review, not as a documentation nice-to-have. A rule with no ownership metadata and no tuning history is, in that book's own terms, detection debt (Detection Engineering Handbook V2, Part 43 — Detection Debt) the moment its sole author becomes unavailable — this part's knowledge-capture audit is one of the ways that debt gets found before it comes due.*

3.4 The exit-window sprint: knowledge transfer under a deadline

[SENIOR MANAGER]

Once someone has actually given notice, knowledge transfer stops being a program and becomes a sprint with a hard deadline set by their last day. Prioritize by the audit from §3.2, not by whatever the departing person happens to think of first — left to their own judgment, most people over-document the interesting parts of their job and under-document the boring parts that turn out to be exactly the parts nobody else knows, because boring and undocumented tend to be the same knowledge for a reason.

CASE-1401 — the two-week handoff. *COMPOSITE CASE EXAMPLE — merges patterns from several mid-sized SOC detection-engineer transitions rather than tracing one identifiable organization; figures below are illustrative, not audited financials.*

A 14-analyst SOC runs one dedicated detection-engineer role, responsible for roughly 340 production rules and their entire suppression-list history. That engineer resigns with two weeks' notice, the contractual minimum, with no successor previously identified and no rule-ownership documentation beyond the rules' own query text.

CONCEPTUAL SAMPLE -- illustrative figures, not sourced benchmark or audited financial data

Before departure:

False-positive rate, team-wide: 8%

Average tuning-request turnaround: 3 business days

Rules with documented suppression rationale: ~40% (engineer's own estimate, unverified)

90 days after departure (no successor identified before notice was given):

False-positive rate, team-wide: 19%

Average tuning-request turnaround: 3 weeks

Two rules silently disabled during a platform migration, unnoticed for 6 weeks

Replacement hire time-to-start: 116 days

Interim contractor cost to cover the gap: \$46,000

Estimated added analyst overtime handling elevated false-positive volume: \$22,000 over the quarter

The two weeks were spent almost entirely on the departing engineer's own priority list — the rules they considered most technically interesting — rather than the audit-driven list from §3.2. Nobody asked, in the first three days, "which rules have a suppression exception with no written reason," which is precisely the category that produced the two silently-disabled rules six weeks later: a well-meaning successor, working from the query text alone with no rationale attached, disabled what looked like dead weight during an unrelated migration and had no way to know it was covering a real gap.

Management Autopsy — "let the departing engineer run their own two-week handoff"

The decision: *The manager gave the departing detection engineer their final two weeks to "get everything documented" with no specific priority list, trusting their judgment about what mattered most to hand off.*

Why it seemed reasonable: *The engineer knew the rule set better than anyone else on the team; asking them to self-direct the handoff looked like the most efficient use of a short, fixed window, and it avoided an awkward conversation about exactly how much the team didn't already know.*

How it failed: *People documenting their own departure default to what's interesting or impressive to write about, not what's dangerous to lose — the suppression exceptions with no recorded rationale never made the list because the departing engineer had long since stopped thinking of them as knowledge at all; they were just "how it's always been."*

The fix: *Run the §3.2 audit before the notice period starts wherever possible, or on day one of it at the latest, and hand the departing person a prioritized list ranked by bus-factor risk, not a blank two weeks and good intentions. The audit turns "document what you think matters" into "document these 14 specific things, in this order," which produces a finished, ranked artifact even if the full list never gets covered before the last day.*

4. Succession planning for team-lead and detection-engineer roles

4.1 Why these two roles fail differently than a rotating analyst seat

[SENIOR MANAGER] A vacant Tier 1 analyst seat is a capacity problem: the queue gets a little slower until the seat is filled, and any reasonably competent hire restores capacity once trained. A vacant team-lead or detection-engineer seat is a judgment problem: the team lead's replacement doesn't just need to know the schedule, they need the accumulated, situational judgment about which analyst is close to burnout, which escalation actually needs the manager pulled in versus handled at the team-lead level, and which recurring client complaint is really a symptom of something else — none of which is written down anywhere, because it's the kind of knowledge that only accumulates by doing the job. The detection engineer's replacement doesn't just need the query syntax, they need the years of accumulated false-positive history that explains why 40 exceptions exist and which 10 of them are load-bearing rather than stale. Both roles concentrate exactly the kind of tacit, undocumented judgment §3 is

built to find and capture — the difference here is that succession planning tries to solve it before a vacancy exists at all, by having a specific named person already developing toward readiness.

4.2 Building the bench before there's a vacancy

[SENIOR MANAGER] A defensible succession plan names a specific person, states specifically what that person still needs before they're ready, and puts a real development activity against that gap on a real calendar — not a name scribbled on a whiteboard next to "eventually." For a team-lead successor, that development activity might be co-running one QA calibration session a month (Part 15 — Quality Assurance Programs owns the calibration mechanics themselves), owning the on-call escalation phone for one week a quarter under the current lead's supervision, and sitting in on one budget or staffing conversation a quarter so the business side of the role isn't a total surprise on day one. For a detection-engineer successor, it might mean co-owning tuning decisions on a defined subset of rules for two consecutive quarters, with the current engineer required to explain their reasoning out loud rather than just approving the change — the explaining is the actual transfer mechanism; silent approval transfers nothing.

Manager's Note *Don't wait for a successor to feel fully ready before giving them real, unsupervised reps at pieces of the role — a successor who's never run the on-call phone alone, only shadowed it, will freeze the first time they have to make a call with no one to check with. Give them the training wheels off for something low-stakes on purpose, on a schedule you control, rather than the first time being an actual emergency with no notice.*

The scorecard below turns "we have a bench" into something a manager can actually audit role by role, rather than a comforting assumption that dissolves the moment someone checks.

Table 4.1 — Succession-readiness scorecard, worked example. (CONCEPTUAL SAMPLE — illustrative role names and dates, not a specific team's actual roster.) Run this structure against every role named in §4.1, not just the one already causing visible pain.

Role	Named Successor Identified?	Development Activity on a Real Calendar?	Performed Core Duty Unsupervised, Last 90 Days?	Succession Risk Rating
Team lead, Shift A	Yes — senior analyst J.R.	Yes — monthly calibration co-lead	Yes — ran on-call week solo in Q2	Low
Team lead, Shift B	Yes — senior analyst on paper	No — no activity scheduled since naming	No	High — named but stalled
Detection engineer	No	—	—	Critical — single point of failure, no bench at all

4.3 What a defensible succession plan costs, and what skipping it costs

[SENIOR MANAGER] A working bench isn't free, but it's cheap relative to the alternative, and a manager defending the time investment to a budget-conscious director needs both numbers in the same room.

CASE-1402 — the bench that held. *COMPOSITE CASE EXAMPLE — merges patterns from several succession-planning efforts; figures below are illustrative, not sourced financial or benchmark data.*

A 22-analyst SOC running three shifts staffs two team-lead roles. A full 12 months before either lead's most likely departure window (no resignation had occurred yet — this is the planning case, not the recovery case), the manager names one senior analyst per shift as a development-track successor and commits roughly four hours a month of paired time per pairing to the activities in §4.2.

CONCEPTUAL SAMPLE -- illustrative figures, not sourced benchmark or audited financial data

Cost of running the bench, per successor, per year:

~48 hours of paired developmental time (4 hrs/month x 12)

Rough fully-loaded cost of that time at senior-analyst rates: ~\$3,000/year

Cost of an unplanned team-lead vacancy with no bench, same organization:

Time to source and onboard an external team-lead hire: 90-120 days

No interim leadership during that window -- shift runs on ad hoc peer coverage

Estimated added overtime and coverage-gap cost across the vacancy window: \$28,000-\$40,000

Team-lead-level judgment calls (schedule exceptions, escalation triage) made by whichever senior analyst is on shift, with no formal authority or documented criteria

One shift's lead resigns eight months into the plan. The named successor takes over the same week on a defined 30/60/90-day reduced-queue ramp — a lighter individual caseload for the first 30 days while they get their footing, full team-lead duties by day 60, and a formal check-in at day 90 — instead of stepping into the role at full weight on day one. No external search runs, no interim-leadership gap opens, and the shift never runs without a clear decision-maker. The other shift's lead does not resign during the period covered by this case; the point of the comparison is the avoided cost of the scenario that didn't happen, not a claim that succession planning is free insurance against every departure.

Blind Spot *"Named successor identified" is the easiest column in the §4.2 scorecard to fill in and the easiest one to be wrong about, because naming someone costs nothing and changes nothing on its own. A name with no development activity attached for six months isn't a bench — it's a name, and it will fail exactly like having no plan at all the moment it's tested, just with an extra layer of false confidence built in. Audit the middle two columns, not just the first one.*

Below is the succession-readiness pipeline this section assumes: how a role moves from "no named successor" to "succession risk: low," and where the process most commonly stalls.

flowchart TD

```
A["Critical role identified\n(team lead / detection engineer)"] --> B{"Named  
successor\nidentified?"}  
B -->|No| C["Assign a candidate against\nPart 10's competency matrix\ngaps for this role"]  
B -->|Yes| D{"Development activity\nscheduled in last 90 days?"}  
C --> D  
D -->|No| E["Bench stalled: named on\npaper only -- flag High risk"]  
D -->|Yes| F{"Performed a core duty\nunsupervised, last 90 days?"}  
F -->|No| G["Schedule a supervised\nsingle-rep trial\n(on-call week, tuning sprint)"]  
F -->|Yes| H["Succession risk: Low --\nre-audit in 90 days"]  
G --> F  
H --> I["unresolved 2+ cycles"]  
I --> J["Escalate: treat as\nCritical single point\nof failure"]
```

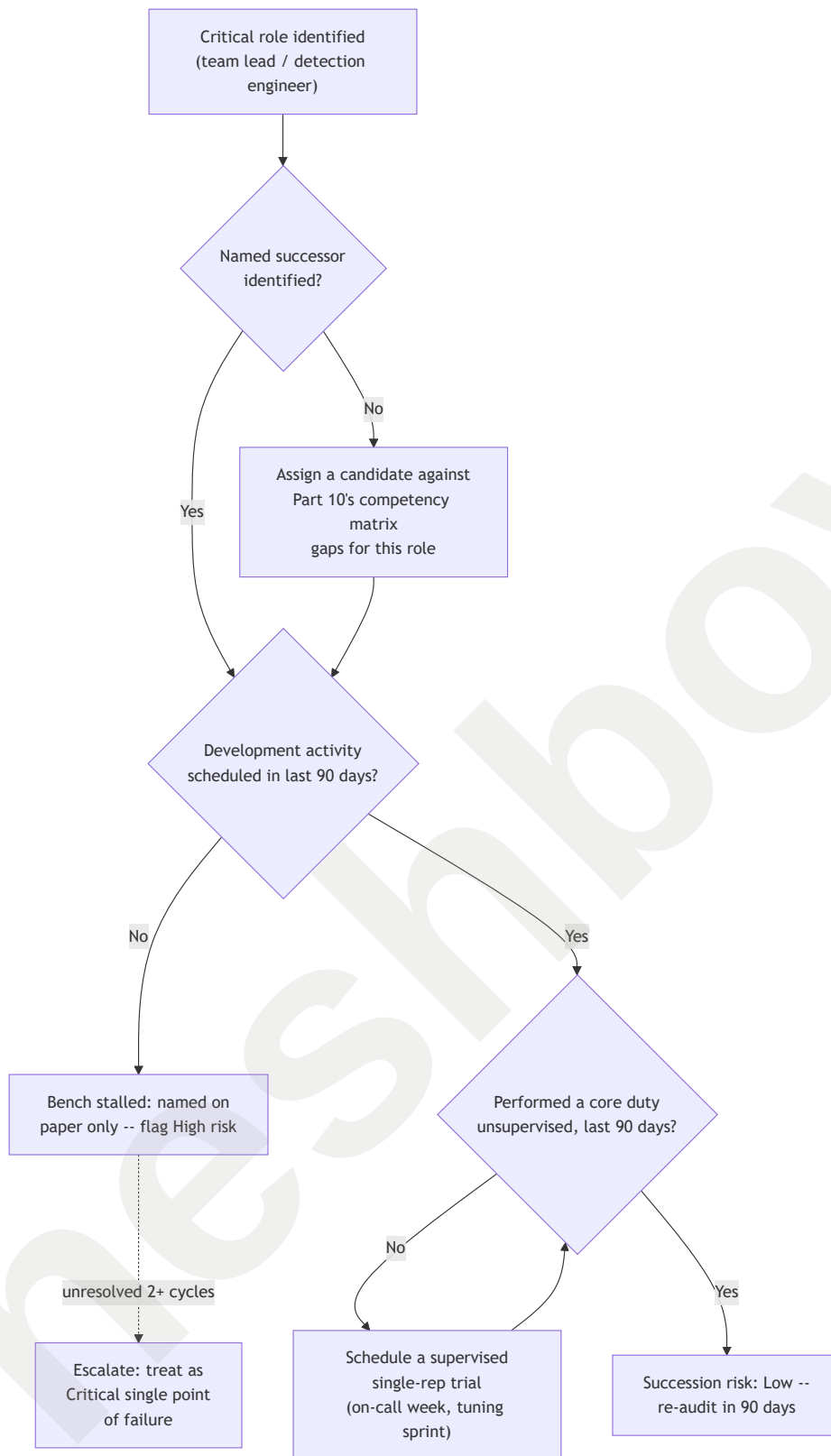


Figure 14.1 — Succession-readiness pipeline. *CONCEPTUAL.* Shows the succession-readiness pipeline as a rendered flowchart. It supports the readiness scorecard in §4.2 and the risk ratings used in Table 4.1 and CASE-1402 — a role sits at whichever node in this pipeline its scorecard row actually shows, not wherever its manager assumes it sits. FIG-1401.

4.4 Recognizing false readiness

[SENIOR MANAGER] A successor can clear every column on the §4.2 scorecard — named, scheduled development activity, one unsupervised rep on record — and still not be ready, because a single rep during a calm week tests almost nothing. Before signing off on a successor as fully ready, check for three specific gaps a clean scorecard hides: whether they've ever exercised the role's authority when something was actually going wrong, not just during routine coverage; whether they've ever had a peer or an analyst push back on a call they made and had to hold their ground or productively change their mind, rather than every decision so far going uncontested; and whether they've ever made an unpopular call — a schedule denial, a tuning decision someone disagreed with — that stuck without the current role-holder quietly overriding it afterward. A successor who's only ever been tested on easy days looks identical, on paper, to one who's actually ready, right up until the first hard day lands on their desk alone.

5. Measuring whether any of this is actually working

[SENIOR MANAGER] Mentorship and knowledge-transfer programs are easy to run in a way that looks healthy on a status slide and is quietly failing underneath it, because the easiest things to measure — pairs formed, documents written, successors named — are participation counts, not outcome measures. Track the pair below instead: an activity count and the outcome it's a weak proxy for, so a gap between the two shows up before it becomes a crisis.

Leading Indicator (Easy to Game)	What It's a Proxy For	Better Outcome Check
Number of active mentoring pairs	Career development actually happening	Quarterly mentee self-report: one concrete thing they can now do that they couldn't six months ago
Number of playbooks/runbooks written from the §3.2 audit	Tribal knowledge actually captured	A second person, cold, successfully executes the documented procedure without asking the original author for help
Number of roles with a "named successor"	Real succession readiness	The §4.2 scorecard's middle two columns — scheduled development activity and a recent unsupervised rep — not just the name field
Documentation exists for a rule/process	Documentation stays current	Last-reviewed date on the document is inside the last two review cycles, not from the year it was first written

[SENIOR MANAGER] Own the §3.2 audit and §4.2 scorecard directly rather than delegating them to an HR partner, and run both on a fixed cadence — every two quarters is a reasonable default — rather than only when a departure prompts one. An HR partner's natural view into this problem is the exit interview, which is Part 18's territory and by definition arrives after the person has already decided to leave; the audit and scorecard in this part are only useful run *before* that decision, on a schedule that doesn't wait for a resignation letter to trigger them.

What Would Change My Mind *This part treats named-successor programs and structured knowledge-capture audits as materially better than relying on informal mentorship and ad hoc documentation. If teams that skipped both — no formal succession scorecard, no audit-driven capture sprint — recovered from team-lead and detection-engineer departures just as fast and just as cleanly as teams that ran them, purely on the strength of a strong general culture of documentation and cross-training, that would undermine this part's case for the specific, formal mechanisms described here rather than trusting Part 12's broader cross-training program to eventually cover the same ground on its own.*

6. Where this connects

[CONCEPT] Everything in this part is preventive. It doesn't tell a manager what to do once a coverage crisis has already started — that's the cost model, the exit-interview analysis, and the retention levers covered in Part 18 — Attrition & Retention, which this part exists specifically to keep a manager from needing on short notice. It assumes the competency matrix from Part 10 — Competency Models & Skills Matrices as the yardstick for "ready" in §4.2's scorecard, the onboarding structure from Part 11 — Onboarding & Ramp-Up Programs as the thing formal mentorship is not a substitute for, the cross-training program from Part 12 — Ongoing Training & Skill Development as the standing, breadth-level version of the bus-factor fix this part applies narrowly and urgently, and the promotion criteria from Part 13 — Career Ladders & Promotion Criteria as what a successor is ultimately measured against once they're actually promoted into the role they've been developing toward.

Cross-references: Within this book, assumes Part 3 (role definitions for team lead and detection-engineer tiers), Part 10 (competency matrix as readiness criteria), Part 11 (onboarding, distinct from ongoing mentorship), Part 12 (cross-training as the standing breadth-level fix), and Part 13 (promotion criteria a successor is measured against); points forward to Part 18 (the cost of the coverage crisis this part prevents) and Part 15 (calibration-session mechanics referenced in §4.2). Outside this book: SOC Playbook Handbook, Foundations / Playbook Library (playbook structure for captured procedural knowledge) and Detection Engineering Handbook V2, Part 22 — Detection as Code and Part 43 — Detection Debt (rule-ownership metadata and the debt that accrues without it).

neshbooy

Part 15 — Quality Assurance Programs

Why this part exists

[CONCEPT] A SOC manager who has never built a formal QA program still has an opinion about ticket quality — it comes from whichever tickets happened to cross the manager's own desk that week, which is not a sample, it's whoever escalated loudest or broke something visibly. A QA program's entire job is to replace that accidental, unrepresentative view with a deliberate one: pull a defined slice of closed work on a schedule nobody can predict, score it against a rubric more than one person has agreed means the same thing, and route what that scoring finds into a coaching conversation before it ever becomes a number on a review form. Three separate mechanics sit inside that one sentence — sampling, calibration, and the score-to-coaching pipeline — and this part builds all three.

[CONCEPT] SOC Playbook Handbook, Part 27 — Escalation Quality already owns what a good hand-off from Tier 1 to Tier 2 looks like, mechanically, on one ticket: what context has to travel, what a receiving analyst needs to not have to re-triage from scratch. This part does not re-derive that standard. It assumes a mechanical definition of "good" already exists somewhere — Part 27's, or an internal playbook built on top of it — and answers a different question: at scale, across hundreds of tickets a month and a dozen analysts, is that standard actually being met, by whom, how consistently, and what happens once you find out it isn't. Escalation Quality is the rubric's raw material. QA is the program that applies a rubric to a sample, again and again, on a schedule, with more than one reviewer, and turns the result into something an analyst can act on. Confusing the two produces a chapter that either drifts into re-explaining escalation mechanics this book has no business owning, or a QA program with no defined standard to score against — this part is built to avoid both failure modes.

[CONCEPT] This part also stays out of two other adjacent chapters' territory on purpose. Part 16 — Performance Management & Coaching owns the 1:1 cadence, the performance-improvement-plan mechanics, and the diagnostic judgment call of whether a recurring miss is a skill gap, a training gap, or a bad process the analyst was set up to fail by. This part's job ends at producing a calibrated, trustworthy QA score and routing it toward that conversation — not running the conversation itself. And SOC Playbook Handbook, Part 32 — Metrics owns the definitions of MTTR, handle time, and false-positive rate; a QA score is a different kind of measurement entirely — a qualitative rubric judgment about one piece of work, not a queue-level throughput statistic — and this part is careful never to let the two blur into each other.

1. What a QA program measures, and what it doesn't

1.1 The unit of analysis is the sample, not the analyst

[CONCEPT] A QA program's raw unit of analysis is one reviewed ticket, scored against a rubric, by a reviewer who wasn't the analyst who worked it. An analyst's QA standing is not a single number — it's the trend line across a rolling window of that analyst's sampled tickets, because any individual ticket can be an outlier in either direction: the easiest false positive of the month scored perfectly, or a genuinely hard multi-stage incident scored a 2 out of 5 on "escalation judgment" because the analyst made one

defensible-but-wrong call under real ambiguity. A program that reacts to a single ticket's score as if it were the analyst's whole performance record has confused the sample with the population it's meant to estimate.

[SENIOR MANAGER] That distinction matters because it sets the bar for what a QA finding can responsibly justify. One low-scoring ticket is a coaching prompt — a specific thing to walk through in the next 1:1. A sustained pattern across a defined number of consecutive review cycles is the kind of evidence Part 16's performance-improvement-plan process needs before it acts. A program that skips straight from "this one ticket scored low" to a documented performance conversation is treating a single data point as if it had the statistical weight of a trend, and analysts notice the inconsistency immediately even when they can't name it that precisely.

1.2 Distinguishing QA from the metrics a manager already tracks

[SENIOR MANAGER] A queue dashboard already reports MTTR, handle time, and ticket volume per analyst — numbers that exist automatically as a byproduct of the ticketing system, with no reviewer required. A QA score is not a faster or cheaper version of that same information; it answers a question none of those numbers can. Handle time tells you an analyst closed a ticket in nine minutes. It says nothing about whether the nine-minute close correctly ruled out lateral movement or missed it. A fast, wrong disposition and a fast, right one produce identical throughput numbers and opposite QA scores — which is the entire reason a program that only watches the dashboard and never opens a ticket to read it can run for years believing it has a well-performing team.

Cross-Book Pointer *This part does not define MTTR, handle time, or false-positive rate, and a QA scorecard should never quietly redefine them either — those definitions live in SOC Playbook Handbook, Part 32 — Metrics, and Part 5 and Part 24 of this book consume them as already-settled inputs. A QA score sits next to those metrics on a manager's dashboard, not inside their definitions; treat a rising QA score alongside a flat MTTR as two independent facts about the same team, not one number restated as two.*

[CONCEPT] The two measurements are complementary rather than competing precisely because of where each one gets its evidence. A throughput metric comes from the ticketing system's own timestamps and requires no human judgment to produce, which makes it cheap and complete — every ticket contributes, all the time — but blind to correctness. A QA score comes from a human reviewer reading the actual work, which makes it expensive and necessarily partial — nobody reviews every ticket — but the only source of evidence a manager has for whether the work itself was any good. Section 2 builds the sampling methodology that decides which slice of that expensive evidence you can actually afford to collect.

2. Building a sampling methodology

2.1 Why full review is neither possible nor desirable

[SENIOR MANAGER] Reviewing every closed ticket sounds like the safest possible QA program until the arithmetic lands: a fully scored review — reading the ticket, checking the underlying telemetry the analyst worked from, scoring five or six rubric categories, and writing a note — takes a reviewer somewhere in the range of 15 to 25 minutes per ticket, even for a straightforward case. A 20-analyst SOC closing a combined 4,000 tickets a month at 20 minutes each would need roughly 1,333 reviewer-hours a month — roughly eight full-time reviewers doing nothing else, on a team that size, purely to check the work of the people actually working the queue. Full review isn't a rigorous version of QA; it's a second SOC built to watch the first one.

[SENIOR MANAGER] The real design question a sampling methodology answers is not "how do we review everything" but "what slice of the work, reviewed well, tells us the most about the team's actual quality per hour of reviewer time spent." That reframing is what makes stratified and risk-weighted sampling better than either full review or pure random sampling, covered next.

2.2 Three sampling approaches, compared

[SENIOR MANAGER] The table below compares the three sampling approaches a QA program actually chooses between, on the dimensions that drive the choice: what gets pulled, what it's good at catching, and where it leaves a gap (CONCEPTUAL SAMPLE — illustrative comparison, not validated against a specific program's measured outcomes).

Table 15.1 — Sampling approach comparison. (CONCEPTUAL SAMPLE.) Use this table to decide which approach — or, more often, which blend of approaches — fits a specific queue's risk profile before building the sampling formula in §2.3. (A note on labels: "P1/P2/P3/P4" below is this section's own illustrative shorthand for a high-to-low priority split, not the Informational/Low/Medium/High/Critical severity bands SOC Playbook Handbook, Part 29 — Playbook Severity Model defines; don't read a direct mapping between the two.)

Approach	How the Sample Is Pulled	Good At	Where It Leaves a Gap
Pure random	Every closed ticket has an equal chance of selection, regardless of severity or analyst	Giving an unbiased read on average quality across the whole queue	A rare but high-consequence ticket type (a real intrusion, a P1) may go months without ever landing in the sample by chance alone
Risk-weighted / stratified	Sampling rate varies by severity or ticket type — for example, 100% of P1/P2 escalations, a smaller fixed percentage of routine P3/P4 closures	Guaranteeing review of the tickets where a missed error costs the most	Routine, high-volume ticket types get thin coverage per analyst even at a nonzero rate, if volume is high enough
Targeted / triggered	Pulled in response to a specific signal — a customer complaint, a reopened ticket, a near-miss flagged by another analyst	Investigating a known concern quickly, with high relevance per ticket reviewed	Says nothing about baseline quality on the vast majority of tickets that never trigger a flag; can't be the <i>only</i> method a program runs

[SENIOR MANAGER] Almost no working QA program picks one row and stops — the standard structure blends risk-weighted sampling as the backbone (so nothing severe goes unreviewed) with a random baseline slice layered on top (so routine work still gets an unbiased check) and targeted pulls reserved for specific complaints or reopened tickets, which are useful precisely because they're the exception, not the method the whole program rests on.

2.3 Sizing the sample: a worked example

[SENIOR MANAGER] **CONCEPTUAL SAMPLE** — illustrative sampling math for a 20-analyst SOC; not sourced benchmark data.

A 20-analyst in-house SOC closes roughly 4,200 tickets a month across its combined queue — about 210 tickets per analyst, a mix dominated by routine P3/P4 alert triage with a much smaller volume of P1/P2 escalations, averaging around 140 P1/P2 tickets a month across the whole team. Built on the blended approach above, the program reviews 100% of the roughly 140 P1/P2 tickets (because a missed error there is the one the board eventually hears about) plus a 3% random slice of the remaining 4,060 routine tickets, which comes to about 122 additional reviews. That's 262 reviewed tickets a month out of 4,200 — a 6.2% overall sample rate — but the sample isn't evenly spread: every P1/P2 ticket gets checked, and each routine ticket has roughly a 1-in-33 chance of being pulled.

At 20 analysts and 262 reviewed tickets, that averages to about 13 reviewed tickets per analyst per month — comfortably above the rough floor of four to six reviewed tickets per analyst per month this book treats as the minimum needed for a monthly trend line to mean anything at all, rather than reacting to noise from one or two tickets. At roughly 20 minutes per full scored review, 262 tickets costs about 87 reviewer-hours a month — a bit over half of one dedicated full-time reviewer's capacity, in practice usually split across two or three team leads doing QA review alongside their other duties rather than one person doing it full time.

Blind Spot Every sampling method above only pulls from tickets that exist in the system — closed, documented, with a case ID. It has nothing to say about an alert an analyst looked at, silently judged not worth a ticket, and closed without ever creating a record, or a watch item mentioned once in a shift handoff and never followed up on. A QA program built entirely on ticket sampling can report a clean, high-scoring quarter while missing exactly the disposition an analyst never wrote down in the first place — which is also, not coincidentally, the disposition most likely to be wrong.

3. Calibration sessions: getting reviewers to agree

3.1 Why one reviewer's judgment isn't enough

[CONCEPT] A rubric with five categories and a 1-to-5 scale looks precise on paper, but "precise" and "consistent across reviewers" are different properties, and a program only has the second one if it checks for it directly. Two team leads reading the identical ticket will drift toward different scores for reasons that have nothing to do with the ticket — one reads "escalation judgment" as forgiving toward a defensible-but-wrong call, the other reads it strictly; one weighs documentation completeness heavily because that's what burned them personally on a past incident, the other barely notices it. Neither reviewer is acting in bad faith. Left unchecked, an analyst's QA trend becomes as much a measurement of which team lead happened to review their tickets that month as a measurement of the analyst's own work — and once that's true, the score has stopped measuring what the program claims it measures.

[FRONTLINE MANAGER]

Calibration is the fix, and it's a specific exercise, not a synonym for "the reviewers talked about it once." Multiple reviewers independently score the same ticket, compare results, and — critically — resolve *why* they disagree into a written anchor: a concrete example of what a 2 looks like in this category versus what a 4 looks like, attached to the rubric itself so the next reviewer doesn't have to reconstruct the same judgment call from first principles.

3.2 Running the session

[FRONTLINE MANAGER]

A calibration session needs, at minimum, every person who currently reviews QA tickets in the room, a small batch of already-closed tickets (five to 10 is enough for one session) nobody has scored yet, and the rubric itself. Each reviewer scores the batch independently and privately before anyone compares notes — comparing first and scoring second defeats the entire purpose, because it just produces polite convergence toward whoever spoke first rather than an honest read of where real disagreement lives.

Manager's Note *Tell reviewers the rubric's categories and scale in advance — that part should never be a secret, since a reviewer can't score consistently against criteria they haven't internalized. Never tell them, in advance or after the fact, which specific closed tickets will end up in a live sample; those are two different kinds of transparency, and a program that keeps both open ends up with reviewers who understand the standard and analysts who've learned exactly which tickets are safe to under-document.*

[FRONTLINE MANAGER]

Once scores are compared, the session's real work is the categories where scores diverge, not the ones where everyone already agreed — an easy convergence needs no discussion, and spending session time congratulating each other on it is the single most common way a calibration session runs out of time before reaching the categories that actually needed it. For each divergent category, the group talks through what specifically in the ticket each reviewer weighed differently, and writes down the resolution as a concrete anchor example other reviewers can apply later without re-litigating the same disagreement from scratch.

3.3 Measuring whether calibration actually worked

[SENIOR MANAGER]

CONCEPTUAL SAMPLE — illustrative inter-rater agreement figures; not sourced benchmark data.

Four reviewers — two team leads, a dedicated QA specialist, and the SOC manager — independently scored the same batch of 10 previously closed tickets against a five-category rubric, before any calibration session had been run. On the "escalation judgment" category, one ticket drew scores of 2, 4, 3, and 5 from the four reviewers — a three-point spread on a five-point scale, the widest disagreement in the batch. On "documentation completeness," the same ticket drew 4, 4, 3, and 4 — reviewers already agreed there without any intervention. Averaged across all five categories and all 10 tickets, only 58% of individual category scores landed within one point of that ticket's median score across reviewers — the working definition of "agreement" this program adopted, since a five-point rubric with reviewers routinely three points apart isn't measuring anything reliably.

A calibration session followed the structure in §3.2, producing two written anchor examples for each of the categories that showed the worst spread. Reviewers then rescored the same 10-ticket batch blind — without being shown their own earlier scores — roughly two weeks later. Agreement rose to 84%. A second session six weeks after that, prompted by a handful of new edge cases the first anchors hadn't covered, held agreement at 86%, which the program set as its working threshold: scores from a rubric with less than roughly 80% inter-rater agreement across a calibration batch aren't yet trustworthy enough to route into a coaching conversation, let alone anything Part 16's performance-management process would act on.

[SENIOR MANAGER] The 58%/84%/86% figures above are a raw agreement rate — the share of scores landing within one point of the batch median — and a manager reading them should know exactly what that rate leaves out: it never asks whether two reviewers would have landed on the same score anyway, by chance, given how each reviewer's own scores happen to be distributed. The statistic built specifically to correct for that is Cohen's kappa, developed for this exact problem — independent raters classifying the same cases — by Jacob Cohen in 1960:

$$\text{kappa} = (p_0 - p_e) / (1 - p_e)$$

- **p₀** — the observed proportion of agreement between two reviewers on the batch (the raw rate this section's percentages approximate).
- **p_e** — the proportion of agreement expected by chance alone, computed from each reviewer's own marginal scoring distribution: how often reviewer A lands on a given score, multiplied by how often reviewer B lands on that same score, summed across every score on the scale.
- **kappa** — agreement with the chance component removed from both the top and bottom of the fraction; 0 means the pair does no better than chance, 1 means perfect agreement.

The gap between the two numbers is exactly where a calibration program can fool itself. Two reviewers can land on the identical score for 90% of a batch and still produce a low kappa, if most of that 90% is both of them defaulting to the same easy, high-frequency score — most sampled tickets are routine, so "everything checked out, score it a 5" is the common answer both reviewers reach without really exercising judgment — rather than genuine shared judgment on the small number of tickets that were actually hard to call. A raw-percentage threshold like this section's 80% floor can be hit by reviewers quietly converging on leniency, not by reviewers agreeing better on the cases that matter. With four reviewers, as in the worked example above, kappa isn't one panel-wide number — it's computed for each of the six reviewer pairs separately and read as a set, because collapsing straight to an average can bury exactly the single divergent pair §5.3's People Risk Trap warns about, where one reviewer's scores run consistently apart from the other three (Cohen, Jacob. "A Coefficient of Agreement for Nominal Scales." *Educational and Psychological Measurement*, Vol. 20, No. 1, 1960, pp. 37–46:

<https://doi.org/10.1177/001316446002000104>).

Field Test Setup: A rubric and at least two active reviewers, with no calibration session run yet this quarter. **Action:** Have every active reviewer independently score the same batch of eight to 10 already-closed tickets, without comparing notes first. Run one calibration session per §3.2, then have the same reviewers blind-rescore the identical batch at least two weeks later.

Expected result: Category-level agreement (scores within one point of the batch median) should rise measurably between the first and second round — a meaningful jump is normal and expected. If agreement doesn't move, or moves and then collapses again within a quarter, the anchors from the session weren't specific enough to actually change how reviewers score, and the session needs to be rerun with sharper examples rather than assumed to have worked because it happened.

Cross-Book Pointer A calibration session inevitably surfaces disagreement about what a good escalation actually looked like on a specific ticket — that's the moment to reach for SOC Playbook Handbook, Part 27 — Escalation Quality's mechanical definition rather than let the room negotiate its own standard from scratch. Calibration should resolve how consistently reviewers apply an existing standard, not invent a competing one; if a session keeps producing disagreements that trace back to "what does a good hand-off even require," the gap is in the team's shared understanding of Part 27, not in the QA rubric.

4. The QA scorecard: what a rubric should measure

4.1 Category selection

[SENIOR MANAGER] A rubric with too many categories collapses under its own weight in a calibration session — 10 categories times a five-point scale gives reviewers 50 independent judgment calls per ticket, most of which end up correlated with each other anyway (an analyst who documents poorly usually communicates poorly too, so scoring both separately mostly adds review time without adding distinct information). Five or six categories, weighted by how much each one actually predicts downstream risk, covers what most SOC's need without diluting reviewer attention across categories that don't move independently of each other.

[SENIOR MANAGER] The categories below tie back to the competency dimensions Part 10 — Competency Models & Skills Matrices already defines for tiering and promotion decisions, deliberately reusing the same vocabulary rather than inventing a second, QA-specific taxonomy an analyst has to learn on top of the one that already governs their career progression.

4.2 A worked rubric

[SENIOR MANAGER] The table below is an illustrative starting rubric, not a fixed standard every SOC should adopt unmodified — weight distribution should reflect what a specific program's own incident history says actually predicts a missed threat, which is a question worth revisiting annually rather than

answering once and freezing (CONCEPTUAL SAMPLE — illustrative weighting, not derived from a specific program's validated outcome data).

Table 15.2 — Illustrative QA scorecard rubric. (CONCEPTUAL SAMPLE.) Use this table as a starting structure for a program's own rubric, built out in full in Appendix A4's QA scorecard template; the weights are a reasonable default, not a number to defend unmodified to a skeptical analyst who's done the math on their own score.

Category	Weight	What a 5 Looks Like	What a 1 Looks Like
Technical accuracy	30%	Correct disposition, correctly supported by the evidence actually reviewed	Wrong disposition, or right disposition reached by coincidence rather than the evidence
Escalation judgment	25%	Escalates or closes at the right point, with the context Part 27 requires already attached	Escalates everything reflexively, or sits on something that should have moved, with no defensible reasoning either way
Documentation completeness	20%	A second analyst could pick up the case cold and understand what was checked and ruled out	Case notes name a verdict with no trail showing how it was reached
Process adherence	15%	Followed the applicable playbook step order; deviations are logged with a stated reason	Skipped required steps with no record of why
Communication and tone	10%	Clear, calibrated urgency in any customer- or stakeholder-facing note; no unnecessary alarm or false reassurance	Tone mismatched to actual severity, in either direction

[HR/PEOPLE] The weighting itself is a policy statement a manager should be able to defend out loud: technical accuracy and escalation judgment together account for 55% of the score deliberately, because a beautifully documented wrong disposition is still a wrong disposition, and a program that weighted documentation as heavily as accuracy would be rewarding good paperwork over good judgment. An analyst who disputes their score is, in effect, disputing either the facts of one ticket or this weighting — and a program that can show its weighting logic survives that conversation far better than one that can only say "that's just what the form says."

5. From QA score to coaching input — not a punitive scorecard

5.1 What the pipeline has to protect against

[CONCEPT] A QA score that reaches an analyst with no defined next step invites the worst version of both possible failure modes: either it's ignored entirely because nothing ever happens as a result, or it becomes the thing an analyst fears most because the *first* time it visibly mattered was a bad surprise — tied to a bonus, a PIP, or a comment in a review the analyst never saw coming. Neither failure comes from the scoring itself; both come from never designing what happens after the score exists. Section 5.2 through 5.4 build that pipeline; the case below shows exactly what happens when it's skipped.

5.2 Management Autopsy — the bonus-linked scorecard

Management Autopsy — "tie QA score directly to the quarterly bonus" (COMPOSITE CASE EXAMPLE, CASE-1501)

The decision: A 30-analyst MSSP SOC linked its QA score directly to a quarterly performance bonus on a fixed schedule: an average score of 85 or above across the quarter's reviewed tickets earned the full 10% bonus, 70 to 84 earned half, and anything below 70 earned nothing.

Why it seemed reasonable: Leadership had heard a real, valid complaint for over a year — that QA feedback was soft, got nodded at in 1:1s, and never visibly changed anyone's behavior. Attaching real money to the score looked like the fix: give the program teeth, and analysts would finally take it seriously.

How it failed: The monthly sample was pulled from a static report that always exported closed tickets in the same fixed order, which in practice meant tickets closed in the first 10 days of the month were reviewed far more often than tickets closed in the last 10. Within two quarters, this had spread by word of mouth across the floor, and a later audit found documentation completeness scoring an average of 92% on early-month tickets against 61% on late-month tickets, for the same analysts working the same ticket types — a 31-point gap explained entirely by which window of the month a ticket happened to close in, not by any difference in the underlying work. Analysts weren't lying about their tickets; they'd correctly identified which slice of their own output was actually being watched and put their limited documentation effort there instead of spreading it evenly. Meanwhile, voluntary attrition among analysts with more than two years of tenure rose from a baseline of roughly 15% annualized to 33% annualized across the two quarters after the bonus linkage took effect; two of the four departing senior analysts named the QA-bonus scheme specifically in their exit interviews as feeling arbitrary and gameable — a complaint that damaged trust in the scores even on the tickets where they'd been scored accurately.

The fix: Sampling moved to a continuous, rolling pull across the entire month rather than a single monthly export in a fixed, learnable order, and the raw score was decoupled from direct compensation entirely. A single low-scoring ticket became a coaching prompt for the next 1:1, full stop; only a sustained pattern across several consecutive review cycles — the trend line described in §1.1 — became an input to Part 16's performance-improvement-plan process, and even then as one input among several, not an automatic trigger on its own. The program also stopped publishing the specific export mechanics of its sampling, while still publishing the rubric categories in full, so analysts understood exactly what "good" meant without being able to reverse-engineer which tickets were safe to under-document.

5.3 The score-to-consequence pipeline, designed on purpose

[SENIOR MANAGER] The pipeline that survives the failure mode above has three distinct stages, and skipping straight from scoring to consequence is what breaks it: a single reviewed ticket produces a score and, where it's low, a specific coaching note about what to do differently; a rolling window of

scored tickets (the same trend line from §1.1) produces a pattern an analyst's direct manager discusses in a normal 1:1 cadence, framed as development, not discipline; and only a documented, sustained pattern that survives that coaching conversation without improving becomes an input to Part 16's formal performance-management track. Compensation, where a QA-linked incentive exists at all, should attach to the third stage at the earliest — never the first, and case CASE-1501 above is the reason why.

People Risk Trap *Letting one team lead be both the only person who reviews their own reports' tickets and the only rater of those same reports' performance concentrates every calibration failure and every personal bias into one unchecked channel — a generous reviewer and a strict one produce different scores for identical work, and if review assignment tracks management-reporting lines exactly, an analyst's QA trend becomes partly a measurement of which manager they happen to report to. Rotate reviewer assignment away from direct-report lines wherever team size allows it, and calibrate across reviewers (§3) specifically so a score means roughly the same thing regardless of who pulled the ticket.*

5.4 Validating the pipeline actually works this way

Field Test Setup: *At least one full quarter of QA scores has been recorded, including some below the program's coaching threshold. **Action:** Pull every ticket-level score from that quarter that fell below the threshold. For each one, independently ask the analyst whose ticket it was whether a specific coaching conversation referencing that score happened within about two weeks of the score being recorded. **Expected result:** Close to all of them should say yes, with the analyst able to name at least roughly what the conversation covered. A pattern of "no" answers, or answers where the analyst didn't know the score existed at all, means the score is sitting in a spreadsheet rather than functioning as the coaching input the program claims to provide — the pipeline exists on paper but isn't actually running.*

6. Program governance: ownership, cadence, and cost

6.1 Who owns each step

[SENIOR MANAGER] QA program ownership splits cleanly across a small number of roles, and the table below states who's accountable for each step versus who's merely consulted or informed — worth pinning down explicitly, because "QA is everyone's job" in practice usually means nobody actually owns the parts that go quiet first, which is calibration cadence and rubric maintenance.

Table 15.3 — QA program RACI. (CONCEPTUAL SAMPLE.) Use this table when a program's ownership is informal or contested — most commonly when a team lead is doing QA review as an unofficial extra duty with no defined authority to change the rubric or sampling method.

Activity	SOC Manager	Team Lead / Senior Reviewer	QA Specialist (If Dedicated)	Analyst Reviewed	HR/People Partner
Own program design (rubric, sampling method, thresholds)	A	C	R	I	C
Pull the sample	I	R	R	—	—
Score a ticket	I	R	R	I	—
Run calibration sessions	C	R	R	—	—
Deliver coaching feedback from a score	I	R	C	R	—
Escalate a sustained pattern to Part 16's PIP process	A	C	I	R	C
Review and revise the rubric annually	A	C	R	I	C

[SENIOR MANAGER] Two rows are worth reading twice. The SOC manager is accountable (A) for program design and for the PIP escalation decision — the two points where a wrong call has the most downstream cost — without being the one pulling samples or scoring tickets day to day, which is exactly the separation §5.2's case study shows breaking down when one person or one incentive structure controls every stage at once. And HR/People is consulted, not merely informed, the moment a QA pattern is being considered as PIP input — a QA finding that's about to become part of a documented performance record is no longer a purely technical judgment call.

6.2 Cadence and cost, put together

[SENIOR MANAGER] Returning to the 20-analyst SOC from §2.3: 87 reviewer-hours a month at a fully loaded reviewer cost of roughly \$60 an hour comes to about \$5,220 a month, or a little over \$62,000 a year, split across two or three team leads doing review as part of a broader role rather than one dedicated headcount line. A calibration session, run quarterly with four reviewers for about two hours each, adds roughly 32 reviewer-hours a year — a small fraction of the ongoing sampling cost, and the single most common casualty when a manager needs to cut QA-program cost quickly, which is the wrong place to cut: a program that keeps sampling and scoring but stops calibrating is spending nearly the same money to produce scores that drift apart in meaning across reviewers, exactly the failure §3.1 describes.

6.3 The pipeline, end to end

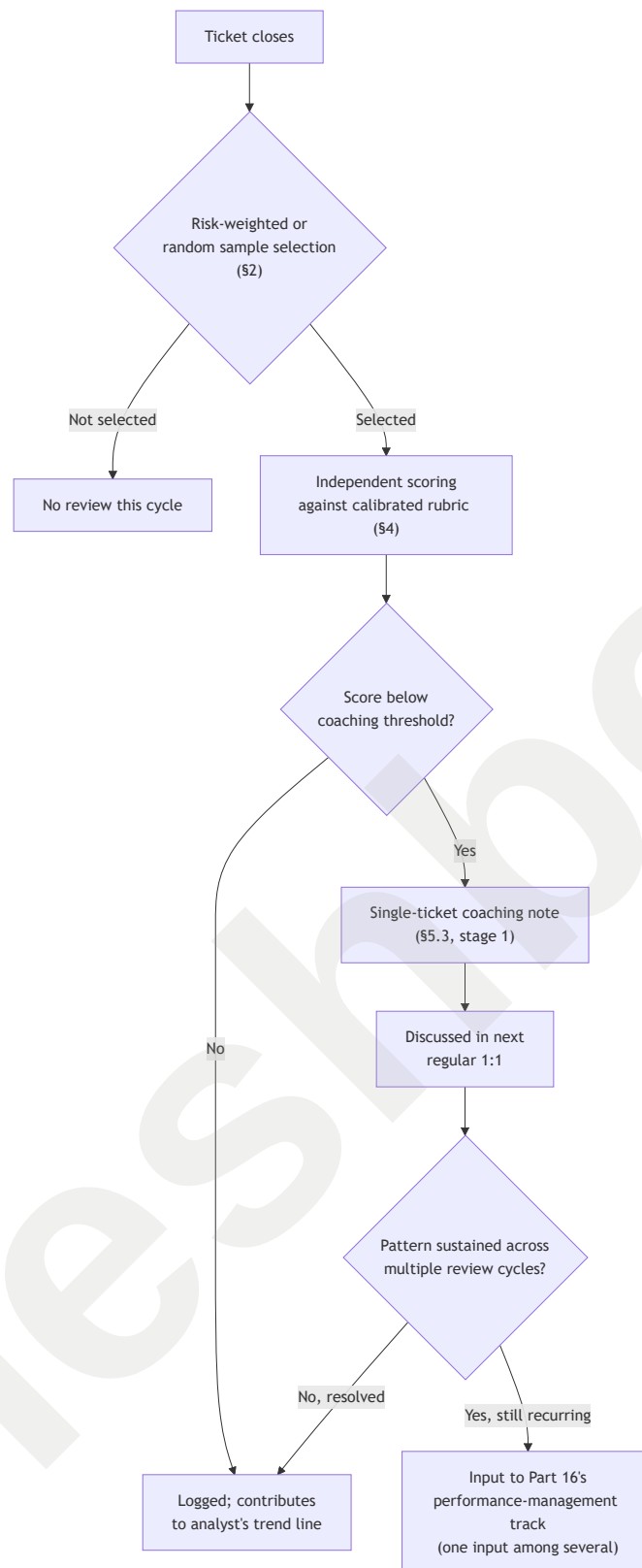
[CONCEPT] Figure 15.1 below ties Sections 2 through 5 into one process, from the moment a ticket closes to the point a sustained pattern either resolves through coaching or moves into Part 16's formal track.

Figure 15.1 — The QA program pipeline, from sample to coaching outcome. *CONCEPTUAL.*

Illustrates the sequence this part's sections build: sampling method, independent scoring against a calibrated rubric, and the two-tier routing (single-ticket coaching versus sustained-pattern escalation) developed in §5.3. It is a structural process diagram, not a capture of any specific ticketing tool's workflow. Diagram ID **FIG-1501**.

flowchart TD

```
A["Ticket closes"] --> B["Risk-weighted or\nrandom sample selection\n($2)"]
B -->|Not selected| Z["No review this cycle"]
B -->|Selected| C["Independent scoring\nagainst calibrated rubric ($4)"]
C --> D["Score below\ncoaching threshold?"]
D -->|No| E["Logged; contributes\nto analyst's trend line"]
D -->|Yes| F["Single-ticket coaching note\n($5.3, stage 1)"]
F --> G["Discussed in next\nregular 1:1"]
G --> H["Pattern sustained across\nmultiple review cycles?"]
H -->|No, resolved| E
H -->|Yes, still recurring| I["Input to Part 16's\nperformance-management track\n(one input among several)"]
```



[FRONTLINE MANAGER]

The branch worth noticing is the one most programs skip: "not selected" is a real, frequent outcome, not a gap in the diagram. Most closed tickets in any given cycle never enter the pipeline at all, which is the entire point of sampling rather than full review from §2.1 — the diagram's job is to show what happens to the minority that *is* reviewed, not to imply every ticket passes through it.

6.4 Routing a single low score versus a real pattern

[SENIOR MANAGER]

The second decision point worth diagramming on its own is the one §5.2's case study got wrong by skipping it entirely — whether a low score is noise, a coaching moment, or evidence a formal process needs to see.

Figure 15.2 — Routing a low QA score: noise, coaching, or Part 16. *CONCEPTUAL.* Illustrates the decision sequence a manager should apply before treating any single QA finding as more than a coaching prompt; it is a decision-support diagram, not a substitute for the judgment calls Part 16 develops in full once a pattern actually reaches that track. Diagram ID [FIG-1502](#).

flowchart TD

A["One ticket scores below threshold"] --> B["Is this analyst's first low score in the rolling window?"]

B -->|Yes| C["Coaching note only; no pattern yet"]

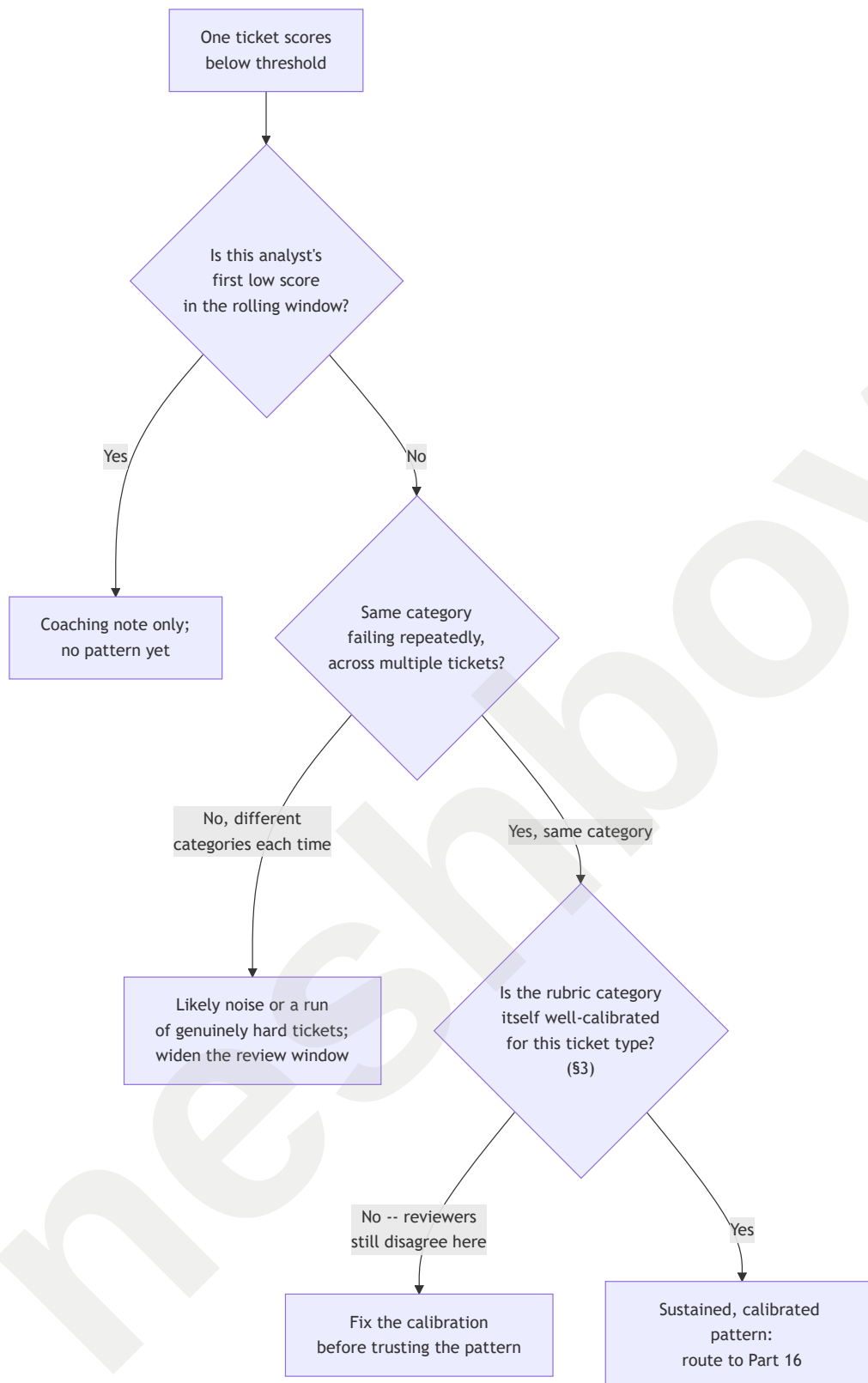
B -->|No| D["Same category failing repeatedly, across multiple tickets?"]

D -->|No, different categories each time| E["Likely noise or a run of genuinely hard tickets; widen the review window"]

D -->|Yes, same category| F["Is the rubric category itself well-calibrated for this ticket type?"]

F -->|No -- reviewers still disagree here| G["Fix the calibration before trusting the pattern"]

F -->|Yes| H["Sustained, calibrated pattern: route to Part 16"]



[SENIOR MANAGER] The branch that most protects an analyst from an unfair escalation is the one in the middle: before a repeating low score on the same category becomes evidence of anything, check whether that category is actually calibrated for the ticket type it's being applied to. A category that reviewers still disagree about isn't ready to justify a performance conversation, no matter how

consistent the low scores look on the surface — consistency in a badly calibrated measurement is not the same thing as a real finding.

7. Where this goes next

[CONCEPT] This part built the QA program as three connected mechanics — a sampling methodology sized to a real budget of reviewer time, a calibration practice that keeps more than one reviewer's judgment meaning the same thing, and a scoring-to-consequence pipeline designed so a low score's first stop is always a coaching conversation, never a bonus letter or a surprise PIP. Appendix A4 carries the fillable QA scorecard and a calibration-session guide built on the rubric and process in §3–4; nothing past this point should need to re-derive what a QA score is for — it should assume a calibrated score exists and use it as an input.

Cross-references

Within this book: Assumes Part 10 — Competency Models & Skills Matrices for the shared vocabulary this part's rubric categories reuse rather than inventing separately. Points forward to Part 16 — Performance Management & Coaching, which owns the 1:1 cadence, the performance-improvement-plan process, and the diagnostic judgment call a sustained QA pattern feeds into; Part 18 — Attrition & Retention, for the baseline attrition rate against which any QA-linked incentive's effect should be checked before assuming it's neutral; and Appendix A4 — QA, Performance & Retention Templates, which carries the fillable scorecard and calibration-session guide this part's Section 4 and Section 3 build toward.

Other volumes: SOC Playbook Handbook, Part 27 — Escalation Quality, for the mechanical definition of a good hand-off that this part's rubric scores against rather than redefines. SOC Playbook Handbook, Part 32 — Metrics, for the MTTR, handle-time, and false-positive-rate definitions this part deliberately does not restate or blur together with a QA score.

Part 16 — Performance Management & Coaching

Why this part exists

[CONCEPT] Part 15 builds the program that finds out, at scale, whether analysts are actually doing good work — sampling methodology, calibration sessions, scores that mean the same thing across reviewers. This part starts where a QA finding, a missed escalation, or a pattern a team lead has simply noticed lands next: in a 1:1, then possibly in a formal Performance Improvement Plan. The two parts have different units of analysis on purpose. Part 15 asks "is our QA program measuring the right thing, consistently, across a whole team." This part asks a narrower, harder question about one specific person and one specific recurring problem: given a finding that's real, what actually caused it, and what fixes that cause rather than just documenting it.

That second question is the load-bearing one in this part, and it's the same judgment call developed further in Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty for the cases where it stays genuinely unclear even after a manager has done the diagnostic work below. Most of the time, the ambiguity resolves cleanly once a manager actually checks: a recurring miss is a skill gap (this specific person hasn't learned this specific thing yet), a training gap (nobody was ever taught this — it's not a "this person" problem, it's a "this cohort" or "this curriculum" problem, which routes to Part 12 — Ongoing Training & Skill Development), or a broken process — most often a detection rule with a false-positive rate high enough that "getting it wrong" was close to unavoidable, which is a technical-debt problem with a technical-debt owner, not a coaching problem at all. Skipping that third possibility and defaulting straight to coaching or a PIP is the single most common and most expensive mistake this part exists to prevent.

This part covers three things: how a 1:1 should actually be run so it's a real coaching instrument rather than a status meeting with a fixed time slot, how a coaching conversation escalates into a formal Performance Improvement Plan and what makes a PIP defensible rather than a documented pretense, and the three-hypothesis diagnostic — skill, training, or process — that has to run before either of the first two decisions gets made. It does not cover QA sampling or calibration mechanics (Part 15 owns that), the termination and legal-documentation obligations once a PIP fails (Part 27 — Legal, HR & Compliance Interfaces owns what happens organizationally after that point), the cost of losing someone this process couldn't save (Part 18 — Attrition & Retention), or the technical mechanics of tuning a noisy detection or scoring detection debt as a program (Detection Engineering Handbook V2, Part 38 — False Positive Engineering and Part 43 — Detection Debt own both, and this part cites them rather than re-deriving either).

1. The 1:1 as the primary coaching instrument

1.1 Cadence: what frequency is actually buying you

[FRONTLINE MANAGER] Cadence is not a scheduling preference — it's the mechanism that determines how long a fixable problem gets to compound before anyone with the authority to fix it hears about it. An analyst struggling with one specific alert type who has a 1:1 every two weeks gets, at most, two

weeks of unaddressed drift before a team lead has a structured chance to notice; the same analyst on a monthly cadence gets up to four weeks, during which the same miss can recur eight or ten times against a live queue instead of two or three. The gap isn't just slower coaching — it's a longer window in which a fixable individual problem gets misread as a bigger pattern than it actually is, because nobody checked in early enough to catch it while it was still small.

The table below sets a starting cadence by where someone sits relative to onboarding and tenure — not a fixed rule for every team, but a defensible default a manager can adjust from rather than invent from nothing.

Analyst status	Cadence	Typical length	Why this frequency
First 90 days post-ramp (past Part 11's structured onboarding)	Weekly	20–30 min	Highest rate of new, uncorrected habit formation; small errors are cheap to fix now and expensive to unlearn later
Steady-state analyst, no open coaching item	Biweekly	20–30 min	Enough frequency to catch drift before it becomes a pattern, without turning every week into a review
Steady-state analyst, active coaching item or recent QA flag	Weekly	30 min	Matches §1.1's compounding-drift logic — a known issue needs the shorter window, not the default one
Senior analyst / informal mentor track	Monthly, plus ad hoc	30–45 min	Lower need for close-interval correction; more time spent on growth and delegation readiness than triage mechanics

[SENIOR MANAGER] A manager whose own span of control has grown past what a real weekly-or-biweekly cadence can sustain — past roughly ten to twelve direct reports for a hands-on cadence, fewer if the manager also carries operational duties — has a staffing problem, not a scheduling problem, and no amount of shortening each session fixes it. Delegating some 1:1s to team leads is a legitimate answer; quietly letting cadence slip to monthly because there's no time is not the same decision, even though it produces the same calendar.

Manager's Note *If you catch yourself repeating the identical coaching point in three consecutive 1:1s with no change in the analyst's actual behavior on the queue, stop scheduling a fourth identical conversation and run the diagnostic in §3 instead. Two clean cycles of "here's the gap, here's what to try" with no movement is the signal that repetition itself isn't the fix — either the coaching hasn't identified the real cause, or the cause isn't something coaching can fix at all.*

1.2 A fixed skeleton, an open half

[FRONTLINE MANAGER] A 1:1 with no structure drifts toward whatever's loudest that week — usually a status update on an open ticket or a scheduling question — and a 1:1 that's entirely scripted stops being a conversation at all. Split it: roughly half the time on a fixed, short skeleton (open items from the last session, any new coaching input since then, one specific thing that went well), and the other half genuinely open to whatever the analyst brings. The fixed half exists so coaching input never silently falls off a to-do list between sessions; the open half exists because the most useful thing to come out of a

1:1 is often not on the manager's agenda at all — a concern about a teammate, a question about a career path, a problem with a tool that's been quietly slowing everyone down.

[FRONTLINE MANAGER] Keep a running written note per analyst, dated, one line per open item and its status — not a full transcript, but enough that a manager looking back after two months can answer "did we actually cover this, and what did we agree to do about it" without relying on memory. This is the same discipline §2.3 needs before any coaching input becomes formal enough to matter in a PIP, and building the habit early, on ordinary 1:1s, means it's already in place the one time it needs to hold up to scrutiny.

Blind Spot *A manager who tracks whether a 1:1 happened — a checkbox in a calendar, a "yes, we met" — but not what was actually covered can't tell a real coaching program apart from a habit of meeting that happens to recur every two weeks with nothing substantive in it. Track content, not attendance: a running note with dated, specific items is the only artifact that shows the difference, and it's the artifact every later step in this part — the pattern threshold in §2.2, the PIP's evidence base in §4.2 — actually depends on existing.*

1.3 What a 1:1 is not

[SENIOR MANAGER] Three things a 1:1 quietly turns into when nobody's actively steering it, none of which is coaching: a status meeting where the analyst reports ticket counts the manager could already see in the queue tooling; a QA-scorecard readout, where the entire session is spent walking through numbers the analyst already received and had no chance to prepare a response to; or a purely social check-in with real value for morale but no coaching content at all. All three are legitimate uses of *some* meeting — they're just not this one, and letting any of them consume the slot means the actual coaching conversation this part is about never happens, while the calendar shows a 1:1 occurred right on schedule.

2. From a QA finding to a coaching conversation

2.1 The handoff Part 15 doesn't own

[HR/PEOPLE] Part 15 — Quality Assurance Programs owns the sampling methodology and the calibration session that gets multiple reviewers to the same score on the same ticket. What it deliberately doesn't own is what happens once a flagged ticket reaches the analyst who worked it — that translation, from a scored ticket to an actual conversation about what to do differently, is this part's job. A QA score with no coaching conversation behind it is a number nobody acts on; a coaching conversation with no QA data behind it is an opinion with no evidence trail. Both halves are necessary, and losing either one is a common way a QA program looks healthy on a dashboard while changing nothing in how the team actually works.

[FRONTLINE MANAGER] The translation itself is simple to describe and easy to do badly: name the specific ticket and the specific gap (not "your escalations need work" but "on ticket #4471, the escalation note didn't name which account was affected, so Tier 2 had to ask before they could start"),

ask the analyst's own account of what happened before offering a correction, and agree on one concrete thing to try differently next time — not three, not a general resolution to "be more thorough." One specific, checkable commitment is something a 1:1 two weeks later can actually verify happened; a vague resolution to improve isn't checkable at all, which means the next 1:1 has nothing to confirm and the conversation effectively resets to zero.

2.2 One miss versus a pattern: setting the threshold before you need it

[SENIOR MANAGER] Every analyst misses something occasionally, and treating a single miss as a pattern before it is one teaches an entirely different lesson than the one intended: that every QA flag is a formal event, regardless of whether it reflects one bad day or a real recurring gap. Set the threshold for "this is now a pattern worth a dedicated diagnostic conversation, not just a normal coaching note" in advance, before a specific case forces the decision under pressure — a reasonable default is three or more flags on the same underlying issue within a rolling 60- to 90-day window, adjusted down for anything safety- or scope-critical (a repeated failure to escalate a specific high-severity indicator warrants attention well before three instances) and adjusted up for genuinely low-volume alert types where three instances might represent the analyst's entire exposure to that scenario.

People Risk Trap *Reacting to every single QA flag as if it were already a pattern — pulling the analyst into a dedicated coaching conversation, complete with formal notes, for a single miss — teaches the team that every 1:1 might turn into a review at any moment. Analysts who learn this stop bringing up their own uncertainty proactively, because volunteering "I wasn't sure about this one" starts to feel indistinguishable from confessing to a performance problem. The fix: name the threshold from §2.2 out loud to the team once, apply it consistently, and treat single misses inside the threshold as ordinary coaching, covered in the fixed-skeleton half of a normal 1:1, not as a separate, heavier process.*

2.3 Documenting without turning every 1:1 into a paper trail

[HR/PEOPLE] The running note from §1.2 already carries most of what's needed, but once a pattern crosses the §2.2 threshold, the documentation standard changes: from this point on, a note needs to be specific enough that someone who wasn't in the room — a second manager during calibration, an HR partner reviewing a later PIP — could read it and understand what was raised, what was agreed, and what happened next. That's a different bar than a personal reminder, and the shift should happen deliberately at the threshold, not retroactively once a PIP is already being drafted and a manager is trying to reconstruct three months of history from memory.

3. The diagnostic call: skill gap, training gap, or a broken process

3.1 Three hypotheses, three completely different fixes

[CONCEPT] A recurring miss on the same alert type or the same kind of ticket has exactly three plausible root causes worth checking before any fix gets chosen, and each one is fixed by a different person, using a different tool, on a different timeline. A skill gap means this specific person hasn't yet

learned or practiced this specific thing — the fix is direct coaching, aimed at one person. A training gap means the gap isn't really about this person at all; it shows up in everyone who came through the same onboarding cohort or missed the same update, and the fix is a curriculum change owned by Part 12's ongoing-training program, not an individual conversation repeated once per affected analyst. A broken process — almost always, in a SOC, a detection rule with a false-positive rate high enough that a "wrong" disposition was close to the expected outcome regardless of who worked it — means the fix is a tuning change or a debt-backlog item owned by detection engineering (Detection Engineering Handbook V2, Part 38 — False Positive Engineering for the tuning mechanics, Part 43 — Detection Debt for the backlog discipline), and no amount of coaching the analyst changes the outcome, because the analyst was never the actual point of failure.

The table below is the fast version of the diagnostic — what each hypothesis looks like once you actually check, and who owns the fix once it's confirmed.

Hypothesis	Signature pattern once checked	Where to check first	Who fixes it	Risk if misdiagnosed
Skill gap	Isolated to one analyst; other analysts working the same alert type show a normal error rate	Compare this analyst's error rate on the specific alert type against team baseline for that same alert type	The analyst, coached directly by their manager or team lead	Coaching never happens; gap persists and eventually looks like a bigger problem than it is
Training gap	Shared across a hire cohort or everyone who missed a specific update; not isolated to one person	Check whether every analyst who shows the pattern shares an onboarding date, a missed update, or a common training gap	Ongoing-training program (Part 12) — curriculum fix, not individual coaching	One analyst gets coached repeatedly for a gap the whole cohort has; the others' identical gap goes uncaught until it resurfaces elsewhere
Broken process / detection debt	Error rate on this specific alert type is high for every analyst who works it, not just the flagged one	Compare the alert type's team-wide false-positive or error rate against the team's overall baseline	Detection engineering — rule tuning (DEH Part 38) or debt-backlog remediation (DEH Part 43)	Analyst is coached or performance-managed for a problem no amount of individual skill fixes; the rule keeps producing the same "miss" in whoever works it next

3.2 Why "skill gap" is the default guess, and why that default is usually wrong

[SENIOR MANAGER] A manager looking at a recurring miss almost always sees the analyst first, because the analyst is the visible, nameable actor in the story and the detection rule is an abstraction sitting several layers back in a platform the manager may not query directly. That visibility bias is exactly backward from where the check should start: the rule's team-wide baseline is cheap and fast to check — usually one query against existing QA or disposition data — while a genuine skill gap can only be confirmed by ruling out the other two explanations first. Checking the cheapest, fastest disqualifying test before assuming the expensive, slower explanation (coaching, which takes weeks to show results either way) is the right order of operations regardless of which hypothesis eventually turns out to be true.

3.3 A decision framework

[CONCEPT] Figure 16.1 lays out the check sequence as a decision tree: rule out the process explanation first, because it's the cheapest to check and the most consequential to miss, then check whether the pattern is shared across a cohort before concluding it's isolated to one person, and only open a formal coaching or PIP track once both of those have been ruled out and a documented coaching cycle has already run with no change.

Figure 16.1 — Diagnosing a recurring miss: skill gap, training gap, or broken process.

CONCEPTUAL. Illustrates the check sequence this part recommends — process first, cohort second, individual coaching last — rather than a capture of any single organization's actual triage workflow.

flowchart TD

A["Recurring miss flagged\n(crosses the Sec 2.2 pattern threshold)"] --> B{"Is this alert type's error/FP\nrate above team baseline for\nEVERY analyst who works it,\nnot just the flagged one?"}

B -->|Yes| C["Root cause: broken process /\ndetection debt.\nSee DEH Part 38 (tuning) and\nDEH Part 43 (debt backlog).\nDo not open a PIP."]

B -->|No| D{"Is the pattern shared by every\nanalyst from the same onboarding\ncohort or the same missed update?"}

D -->|Yes| E["Root cause: training gap.\nRoute to Part 12's curriculum --\ncoach the cohort, not one person."]

D -->|No| F{"Has this exact gap already been\nnamed in a documented 1:1, with\na specific practice step, and\ngiven a real cycle (2-4 weeks)\nto close before recurring?"}

F -->|No| G["Not yet PIP-eligible.\nCoach directly first (Sec 2.1);\ndocument the conversation (Sec 2.3)."]

F -->|Yes, still recurring| H["Root cause: genuine skill gap\ncoaching alone hasn't closed.\nPIP-eligible -- see Sec 4."]

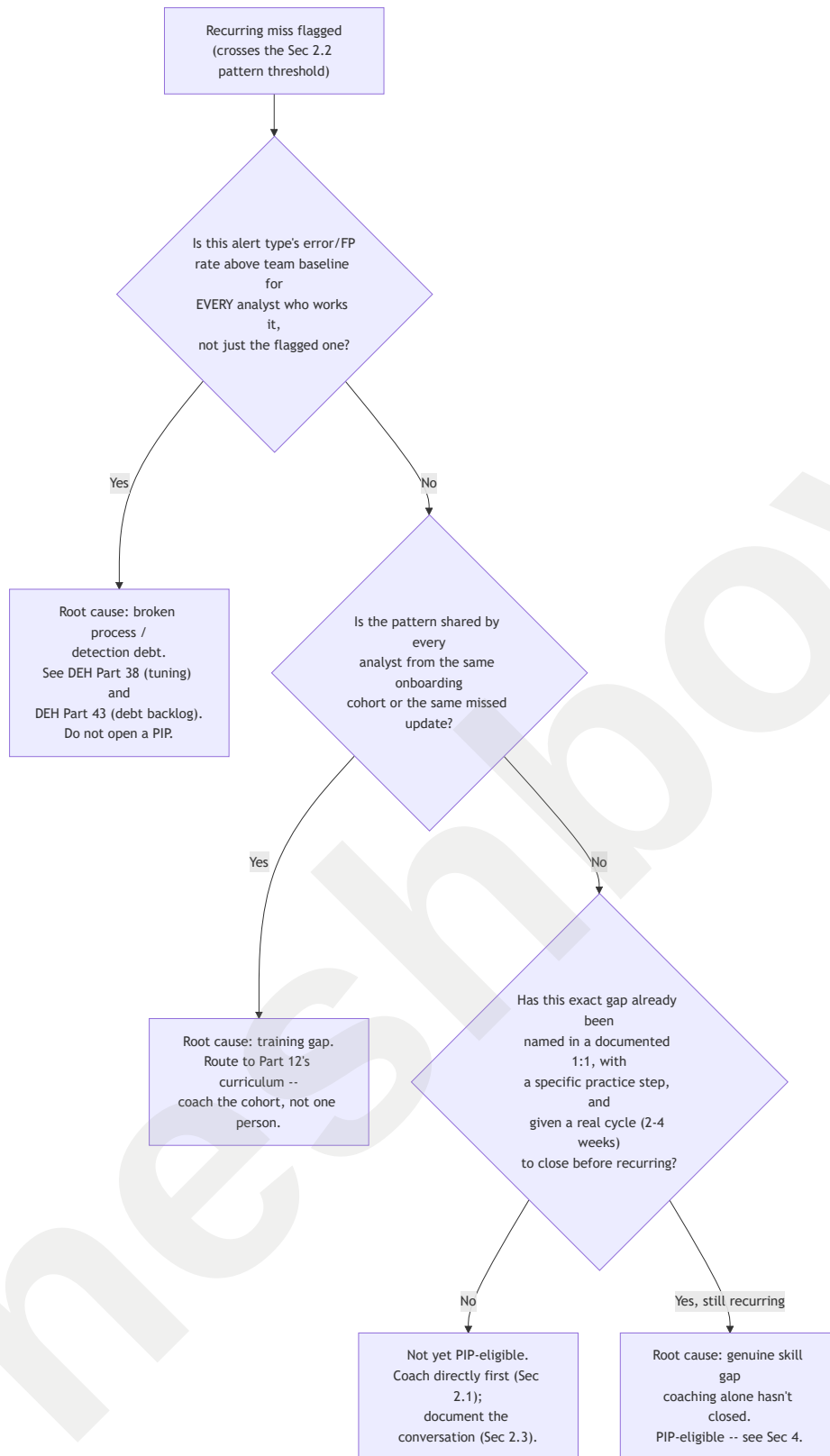


Figure 16.1 — Diagnosing a recurring miss: skill gap, training gap, or broken process.

CONCEPTUAL. Shows the three-hypothesis check sequence as a rendered flowchart. It supports the diagnostic matrix in §3.1's table and the CASE-1601 worked example below — a real recurring-miss investigation should be able to point to the exact node in this tree where it landed, not to a manager's unaided judgment call. **FIG-1601**.

3.4 Worked example: the same recurring miss, three true causes

CASE-1601 — the same alert, three analysts, three causes. *COMPOSITE CASE EXAMPLE — merges patterns from several mid-sized SOC coaching investigations into one illustrative narrative; figures below are illustrative, not sourced benchmark or audited data.*

A 16-analyst SOC's Part 15 QA sample flags a recurring problem on a single detection — a rule for suspicious base64-encoded PowerShell execution — across three analysts in the same quarterly sampling cycle. Before this investigation, the manager's working assumption was that all three needed coaching, in roughly the same way, on roughly the same gap.

CONCEPTUAL SAMPLE -- illustrative figures, not sourced benchmark or audited financial data

Team-wide baseline, this detection, trailing quarter:

False-positive / miscalled-disposition rate: 14% (roughly in line with the team's 8-16% baseline range across all detections)

Analyst A (2 months post-ramp): flagged on 6 of 9 sampled tickets for this alert type

Analyst B (14 months tenure, otherwise strong QA record): flagged on 5 of 11 sampled tickets

Analyst C (3+ years tenure, strong QA record on every other alert type): flagged on 7 of 8 sampled tickets -- but the rule's own disposition data, checked for the first time during this investigation, showed a 58% miscal rate for EVERY analyst who worked it that quarter, not just Analyst C

Running the §3.1 checks in order changed the plan for all three. Analyst A's gap was isolated and she was two months past ramp-up — a genuine skill gap: she hadn't yet learned to decode a specific PowerShell obfuscation pattern the rule's alert text didn't spell out. One coaching session plus a short practice set closed it; her error rate on this alert type matched team baseline within the next sampling cycle. Analyst B's gap looked identical on paper, but checking cohort membership showed three other analysts hired in the same onboarding window shared the exact same blind spot once anyone actually looked — the technique had been added to the detection roughly two months after that cohort's onboarding curriculum was last updated, so nobody in that cohort had ever been taught it. That routed to Part 12 as a curriculum fix, not four separate individual coaching conversations repeating the same missing content. Analyst C's case was the one the original assumption got most wrong: the rule's own team-wide rate — 58%, against a 14% baseline for every other detection this team ran — meant Analyst C's dispositions were close to what anyone would produce working that same rule that quarter. Retuning the rule's encoding-pattern match (the specific fix lived in detection engineering's backlog, not in this manager's reach) dropped Analyst C's apparent error rate back to team baseline within two sampling cycles, with zero coaching time spent on an analyst who had never actually had a performance problem.

Management Autopsy — "coach the analyst on the alert everyone was getting wrong"

The decision: Before checking the rule's own team-wide disposition rate, the manager scheduled a formal coaching conversation with Analyst C, treating the QA flag the same way as Analyst A's and Analyst B's.

Why it seemed reasonable: Analyst C was the most experienced of the three and had the strongest QA record everywhere else, which made "must be an unusual lapse, worth addressing directly" a plausible read — nobody had reason to suspect the rule itself before looking.

How it failed: A coaching conversation aimed at an analyst whose dispositions were consistent with what the rule's own logic was producing for everyone changes nothing measurable, because the analyst was never the point of failure — the miscall rate stayed at 58% for the next sampling cycle regardless of the coaching, because the rule kept producing the same ambiguous evidence for whoever worked it next.

The fix: Check the rule's team-wide baseline before scheduling any coaching conversation tied to a specific alert type — it's a single query against existing disposition data, and it's cheaper and faster than a coaching cycle that turns out to have been aimed at the wrong target from the start.

3.5 When the honest answer is detection debt, not a person

[SENIOR MANAGER] The broken-process branch of §3.1 has a name in the companion technical volume, and it's worth using precisely rather than reinventing a softer version of it here.

Cross-Book Pointer This part does not cover how to measure a detection's false-positive rate, decide whether a rule is worth tuning versus retiring, or score a detection backlog as organizational debt — those are technical-engineering questions with technical owners. See *Detection Engineering Handbook V2, Part 38 — False Positive Engineering* for the tuning mechanics behind fixing a rule like the one in CASE-1601, and Part 43 — *Detection Debt* for how a backlog of untuned, high-noise rules gets tracked and prioritized as a standing liability rather than a one-off fix. Come back here once a detection's own numbers confirm the process explanation from §3.1 — this part's job is recognizing when that's the real answer, not doing the tuning work itself.

[SENIOR MANAGER] The practical consequence for a manager is that "this alert type has bad numbers for everyone" is a finding worth escalating to detection engineering as its own item, tracked independently of whichever analyst happened to surface it. A rule sitting in DEH Part 43's debt backlog for six months while three different analysts get individually coached on its symptoms in three different quarters is a sign the finding never actually reached the team that owns the fix — the coaching kept happening because it was the visible, available lever, not because it was ever going to close the gap.

What Would Change My Mind *This part treats the process check (§3.1's cheapest, fastest test) as the correct first move before any individual coaching begins. If a manager could show that skipping the process check and going straight to coaching produced the same time-to-resolution and the same rate of correctly identifying the true cause — not just eventually arriving at the right answer, but arriving at it just as fast — that would undercut this section's ordering claim, and the recommendation would need to soften from "check process first" to "check process and skill in whichever order is locally convenient."*

4. Performance Improvement Plans

4.1 The precondition: don't open a PIP until §3 has been run

[HR/PEOPLE] A Performance Improvement Plan is the heaviest instrument in this part's toolkit, and opening one before ruling out the training-gap and broken-process branches of §3.1 means there's a real chance the plan is aimed at something the analyst was never actually in a position to fix. A PIP opened against a genuine detection-debt problem doesn't just fail to improve anything — it puts a person through a formal, stressful, career-affecting process for a defect that belonged in an engineering backlog, and it does nothing to stop the same "performance problem" from recurring in whoever inherits that alert type next.

People Risk Trap *Opening a PIP against an analyst whose recurring miss is actually a broken rule teaches everyone else who works that same rule exactly the wrong lesson: that surfacing a miss on this alert type risks a formal process, so the safer move is to quietly work around it rather than report it. That has the perverse effect of making the real defect — the rule itself — harder to find, because the QA signal that would have flagged it for detection engineering starts getting suppressed by analysts protecting themselves instead. Run §3.1's process check before a PIP, every time a recurring miss is tied to one specific alert type rather than a general pattern of judgment across many different kinds of tickets.*

[HR/PEOPLE] A second precondition, separate from the diagnostic: the specific gap named in a PIP should already have come up in at least one prior documented 1:1 (§2.3), with a real attempt at coaching and a real cycle to show improvement. A PIP is not the venue where an analyst hears about a problem for the first time — that's what a normal coaching conversation is for. If the first formal mention of a specific gap is the PIP document itself, the process has skipped a step, and the plan will read, correctly, to the analyst and to anyone reviewing it later, as a surprise rather than a documented escalation of an already-known issue.

4.2 What a defensible PIP actually contains

[HR/PEOPLE] A PIP that survives scrutiny — from the analyst, from HR, from a later reviewer trying to understand what happened — has the same handful of components every time, and each one has a specific, common way of being written too vaguely to actually do its job.

Component	What it must contain	Common failure mode
Problem statement	The specific, observable gap, tied to dated prior 1:1 notes (§2.3) or QA data (Part 15), not a general impression	"Needs to improve overall performance" — no specific behavior named, nothing to disprove
Success criteria	A measurable target tied to an existing baseline (team QA average, handle-time norm, error rate on a named alert type)	A subjective bar ("show more initiative") with no way to check objectively whether it was met
Support committed	Specific coaching time, resources, or shadowing the manager is actually providing during the plan, not just monitoring	Support is listed but never actually scheduled or delivered — the plan becomes surveillance with no assistance behind it
Timeline and checkpoints	A defined length (commonly 30, 45, or 60 days) with named checkpoint dates, not just a start and an end	No interim checkpoint — the analyst gets no signal on progress until the plan's outcome is already decided
Consequence of non-improvement	Stated plainly and honestly — what happens if the criteria aren't met by the deadline	Left vague or unstated, which reads as either a bluff or a foregone conclusion depending on who's asking

[HR/PEOPLE] The reusable PIP template this section assumes has its fillable home in Appendix A4 (TMPL-1601), built as the companion artifact to this part, Part 15, and Part 18. It doesn't automate the diagnostic judgment call in §3 — no template can — and it doesn't replace the legal review a PIP should get before it's delivered in any jurisdiction with meaningful wrongful-termination exposure, which is Part 27 — Legal, HR & Compliance Interfaces's territory, not this part's.

4.3 Cadence and checkpoints during the plan

[FRONTLINE MANAGER] A PIP with a single check-in at the very end — start the plan, disappear for 45 days, deliver a verdict — gives the analyst no chance to course-correct and gives the manager no early signal that the plan itself might be failing to produce movement. Run the same weekly cadence §1.1 already sets for an active coaching item, with each session explicitly checking progress against the specific success criteria from §4.2, not a general "how's it going." A PIP that isn't showing any measurable movement by its midpoint checkpoint rarely turns around by the deadline; that midpoint check is the honest moment to decide whether to adjust the plan's support, extend the timeline for a genuinely close case, or acknowledge that the outcome is already trending toward non-improvement.

4.4 The two ways a PIP gets used dishonestly

[SENIOR MANAGER] A PIP fails its own purpose in one of two directions, and both are common enough to name specifically. The first is using a PIP as a paper trail for a termination decision that's already been made — going through the motions of a plan with no real intention of the analyst succeeding, because the actual decision happened before the plan was written. The second, less discussed, is using a PIP as a way to avoid a termination decision that should already have been made — extending plan after plan for someone who was never going to improve, because opening a new plan feels more comfortable than closing the process with an honest outcome. Both waste the same thing: the analyst's time, the manager's credibility, and — in the first case specifically — the organization's legal position if the paper trail is ever examined.

CASE-1602 — the plan that was already decided. *COMPOSITE CASE EXAMPLE — merges patterns from several disputed-termination reviews into one illustrative narrative; figures below are illustrative, not sourced legal or financial data.*

CONCEPTUAL SAMPLE -- illustrative figures, not sourced benchmark or audited financial data

Timeline:

- Day 1: 30-day PIP opened. Problem statement: "overall performance below expectations."
No specific ticket, alert type, or dated prior 1:1 cited.
- Day 15: First check-in, 15 minutes, no documented notes beyond "discussed progress."
- Day 30: Second check-in, same format.
- Day 32: Termination. HR file contains the PIP document and two undocumented check-ins;
no prior coaching record predates the PIP by more than a few days.

Outcome:

Former analyst's attorney sends a demand letter citing the absence of any documented performance concern before the PIP, inconsistent treatment relative to a comparator who received no PIP for a similar QA record, and undocumented check-ins with no measurable criteria ever stated in writing.

Case resolved via a negotiated separation payment rather than litigation: roughly \$40,000, plus the manager's and HR's time across several weeks of document review and outside-counsel consultation that a properly documented process would not have required.

Management Autopsy — "the PIP that was really a decision already made"

The decision: *A manager who had already decided to terminate an underperforming analyst opened a 30-day PIP with a vague problem statement, intending it primarily as a documentation step rather than a genuine improvement opportunity.*

Why it seemed reasonable: *The manager believed the underlying performance problem was real and long-standing, and treated the PIP as a formality that HR policy required before a termination could proceed — a box to check on the way to a decision already settled.*

How it failed: *Because the plan was never meant to produce improvement, it was never built the way §4.2 describes: no specific problem statement, no measurable criteria, no documented check-ins, and no prior coaching record showing the issue had ever been raised before the PIP itself. When the termination was challenged, the file that was supposed to protect the decision instead read as exactly what it was: a pretense, assembled in the same window as a decision that had already been made.*

The fix: *If the decision to terminate is genuinely already made, say so honestly and route straight to Part 27's process for a documented termination — don't open a PIP to dress up a decision that isn't actually open to a different outcome. If the decision genuinely isn't made yet, build the plan to §4.2's standard from day one: a specific problem statement tied to dated prior coaching, measurable criteria, real committed support, and documented checkpoints — the version that would hold up to scrutiny regardless of which way the plan ultimately ends.*

4.5 Closing a PIP: three honest outcomes

[HR/PEOPLE]

A PIP ends in exactly one of three ways, and each has an honest next step. The analyst meets the criteria: document it clearly, close the plan, and — this matters more than it sounds — actually treat the plan as closed rather than continuing to watch that person more closely than everyone else indefinitely on the strength of a resolved plan. The analyst shows real, partial movement but hasn't fully met the criteria by the deadline: this is the genuinely judgment-call case, and a short, explicitly bounded extension can be defensible if the trend is real and documented, which is different from an open-ended series of extensions with no new criteria attached. The analyst shows no meaningful movement against clearly stated, achievable criteria: this is where the plan's stated consequence from §4.2 gets honored, and where Part 27's termination-documentation process takes over from this part's coaching-and-improvement process.

5. Measuring whether any of this is actually working

[SENIOR MANAGER]

A performance-management program can look busy — 1:1s on the calendar, PIPs opened and closed — while doing almost nothing to actually change outcomes, and the easiest way to catch that gap is to check whether the program's own diagnostic step is getting used, not just whether the paperwork exists.

Field Test Setup: Pull every PIP opened in the last 12 months, along with the prior 1:1 notes for each affected analyst. **Action:** For each PIP, check whether §3.1's three-hypothesis diagnostic — specifically, the team-wide baseline check for any alert type or ticket category named in the plan — was run and documented before the PIP was opened. **Expected result:** Every PIP tied to a specific recurring alert type or process should show a documented baseline check ruling out the process explanation. If most PIPs in the sample show no such check, the program is skipping the cheapest, most consequential step in this part's diagnostic — and some fraction of those plans were probably opened against people who were never the actual point of failure.

[SENIOR MANAGER]

Beyond that audit, three simple counts tell a manager more about program health than any survey: how many recurring-miss cases got resolved by a rule fix or curriculum change rather than a PIP (this number going up over time is a sign the diagnostic is actually catching process and training explanations before they reach an individual); how many PIPs closed successfully versus ended in termination (a program where every single PIP ends in termination is either only opening plans once the decision is already made, per §4.4, or setting criteria nobody could realistically meet); and how many PIPs show a documented prior coaching conversation predating them by more than a few days, per §4.1's second precondition.

6. Where this connects

[CONCEPT]

This part assumed Part 15's QA sampling and calibration as the source of most of the signal a 1:1 acts on, and it assumed Part 10 — Competency Models & Skills Matrices's observable-behavior standard as the yardstick for what "meets expectations" concretely means once a PIP has to state it in writing. It routes the training-gap branch of §3.1 to Part 12 — Ongoing Training & Skill

Development rather than treating a shared, cohort-wide gap as a series of individual coaching problems, and it routes the broken-process branch to Detection Engineering Handbook V2's technical-debt content rather than performance-managing a person for a rule's false-positive rate. It deliberately stops short of the legal-documentation standard a PIP needs once termination is on the table (Part 27), the cost model for what happens when this process doesn't save someone (Part 18), and the genuinely unresolved judgment calls that remain even after the diagnostic in §3 has been run correctly (Part 25) — all three pick up exactly where this part's honest limits are.

Cross-references

Within this book, this part assumes Part 10 — Competency Models & Skills Matrices (the observable-behavior standard a PIP's success criteria are written against), Part 12 — Ongoing Training & Skill Development (the routing destination for a confirmed training gap), and Part 15 — Quality Assurance Programs (the sampling and calibration mechanics that produce most of the signal a 1:1 acts on). It points forward to Part 18 — Attrition & Retention (the cost of a coaching or PIP process that doesn't succeed), Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty (the diagnostic judgment call developed further for genuinely unresolved cases), and Part 27 — Legal, HR & Compliance Interfaces (termination-documentation standards once a PIP fails). Outside this book, it cites Detection Engineering Handbook V2, Part 38 — False Positive Engineering and Part 43 — Detection Debt for the technical fix and debt-tracking discipline behind the broken-process branch of §3.1 — this part owns recognizing that a rule, not a person, is the real point of failure; it does not own tuning the rule or scoring the backlog.

Part 17 — Burnout, Fatigue & Wellbeing

Why this part exists

[CONCEPT] Part 6 — Shift Pattern & Coverage Design turns a headcount number into a calendar and previews, without developing, the fatigue and circadian research behind why some rotation directions and shift lengths cost more in error rate than others. This part is where that research actually gets developed and turned into scheduling decisions a manager can defend. It also picks up a second, related problem that Part 9 — Queue Health & Workload Management touches only at the aggregate-queue level: alert fatigue in the individual analyst who has been sitting on the highest-noise part of the queue for too many shifts in a row. Both problems share a mechanism — sustained exposure with insufficient recovery — and both have a fix a manager controls directly: the rotation, not the rule. A multi-year field study inside working SOC's reached the same conclusion from the research side: analyst burnout there was driven by the cyclic interaction of human, technical, and managerial factors rather than any single cause, and the mitigations that actually worked were changes to workflow, staffing, and rotation a manager controls, not individual coping advice (Sundaramurthy et al., "A Human Capital Model for Mitigating Security Analyst Burnout," USENIX Symposium on Usable Privacy and Security (SOUPS), 2015:

<https://www.usenix.org/conference/soups2015/proceedings/presentation/sundaramurthy>).

[CONCEPT] That last distinction is the one this part exists to hold, and it is worth stating precisely because it is the split most likely to collapse into one muddled chapter by accident. Detection Engineering Handbook V2, Part 38 — False Positive Engineering owns the fix that lowers a queue's noise floor — retuning a rule, adding suppression logic, raising a threshold, so fewer false positives are generated in the first place. This part owns the fix that bounds how much of whatever noise floor remains any single analyst absorbs, and for how long, before someone else takes a turn. A manager with only the tuning half of that picture will keep retuning a queue whose real problem is that nobody has rotated off it in three weeks; a manager with only the rotation half will rotate people through a queue that a two-day tuning fix would have made mostly disappear. Both halves are real fixes for the same symptom, aimed at different causes, and a manager needs to be able to tell which one is in front of them before reaching for either.

[CONCEPT] This part covers three things in sequence: the shift-work fatigue and circadian evidence that should inform rotation design (§1), alert fatigue treated specifically as a workload and rotation problem rather than a detection-quality one (§2), and the wellbeing program a SOC manager actually builds — leading-indicator detection before burnout becomes a resignation, and mandatory disconnected time after a major incident (§3–§4). It does not cover the retention economics of burnout-driven attrition (that cost model, and the levers that move it, belong to Part 18 — Attrition & Retention), the diagnostic judgment call between a burnout-driven performance dip and a genuine skill gap (Part 16 — Performance Management & Coaching), the manager's live staffing-surge decisions during a declared major incident (Part 28 — The Manager's Role in a Major Incident), or the standing postmortem program that would eventually audit whether a disconnect policy actually held (Part 29 — Post-Incident Organizational Review). Each gets a pointer where it's relevant and nothing more.

1. Shift-work fatigue and circadian science, applied to SOC scheduling

1.1 Why "tired" is the wrong unit of measurement

[CONCEPT] A manager who thinks about fatigue purely as "hours since last slept" is measuring the wrong thing. Sleep deprivation and circadian misalignment are two different mechanisms, and a SOC's night shift runs into the second one regardless of how well the first one is managed. The human body runs on an internal clock with a period that averages slightly longer than twenty-four hours, not exactly twenty-four — a fact established by controlled sleep-lab research on subjects living for weeks under conditions designed to isolate the body's own clock from external time cues, which found an average intrinsic period of about 24.18 hours in both younger and older adults (Czeisler et al., "Stability, Precision, and Near-24-Hour Period of the Human Circadian Pacemaker," *Science*, 1999: <https://doi.org/10.1126/science.284.5423.2177>). That internal clock does not reset itself just because an analyst's calendar says they're now working nights; it keeps running on roughly the same schedule it always has, which means a night-shift analyst is asking their body to be alert and vigilant during the hours that same body's own clock has spent a lifetime treating as the deepest part of its rest period.

[FRONTLINE MANAGER] The practical consequence is that a fully rested analyst, eight hours of sleep the day before, can still perform measurably worse at 4 a.m. than the same analyst performs at 10 a.m. on identical work, because the deficit isn't about how much they slept — it's about what their internal clock expects to be doing at that clock time. A staffing model that treats every covered hour as interchangeable, as long as a seat is filled, is quietly assuming this effect doesn't exist. It does, and it doesn't average out just because the shift is fully staffed.

1.2 The window of circadian low, and why a single-seat overnight shift sits inside it

[CONCEPT] Occupational fatigue-risk-management practice in aviation and maritime operations — industries that have spent decades building formal fatigue rules because the cost of a fatigue-driven error is measured in lives, not tickets — converges on a consistent finding: human alertness and reaction time bottom out in a nightly window, with a shallower second dip in mid-afternoon, regardless of shift history, sleep quality the night before, or how motivated the person is. The FAA calls the nighttime trough the window of circadian low (WOCL), defines it as roughly 3 a.m. to 5 a.m. for someone on a normal day-sleeping routine, and builds hard rules around it — maximum duty hours, mandatory rest, restrictions on what safety-critical tasks can be assigned inside it — precisely because willpower and good staffing intentions don't move it (Federal Aviation Administration, "Basics of Aviation Fatigue," Advisory Circular 120-100, 2010: https://www.faa.gov/documentLibrary/media/Advisory_Circular/AC%20120-100.pdf).

[SENIOR MANAGER] A SOC's overnight shift, on almost any coverage model, sits centered on that window rather than clipping its edges the way an evening shift does. An evening shift ending at midnight touches the start of the trough at most; a night shift running roughly 11 p.m. to 7 a.m. spends its entire middle stretch inside it. That has a direct, uncomfortable implication for triage quality: whatever alert happens to land at 3 a.m. gets first-pass judgment from the analyst whose cognitive performance is, structurally, at its lowest point of the entire twenty-four-hour cycle — not because that analyst is worse at the job, but because nobody's judgment is as sharp at that clock hour as it is at 10 a.m., and a coverage model that satisfies Part 6's staffing math says nothing about that.

Blind Spot A coverage model that Part 6 would certify as "fully staffed" — one analyst, every hour, no gap — answers a different question than "is the judgment in that seat as reliable at 3 a.m. as it is at 10 a.m." Staffing math counts seats; it has no variable for the window of circadian low, and a manager who only checks the staffing math will never see this gap, because the dashboard that shows a seat filled looks identical whether the person in it is at their cognitive peak or their trough. If overnight escalation accuracy or overnight false-negative rate runs measurably worse than day-shift rates on the same alert types with the same rules, check the clock before checking the rule or the person — the gap this Blind Spot describes produces exactly that signature.

1.3 Rotation direction and speed as manager-controlled levers

[FRONTLINE MANAGER] Part 6 §3.2 names rotation direction as a real, low-cost design lever without developing why it matters; here is why. Because the human circadian clock naturally runs slightly longer than twenty-four hours, delaying it — going to bed later each night — asks less of the body than advancing it — going to bed earlier. A forward-rotating schedule (day, then evening, then night) delays the clock at each transition, moving in the same direction the body's own drift already wants to go. A backward-rotating schedule (night, then evening, then day) advances it at each transition, fighting that drift every single time the rotation turns over. This isn't a theoretical preference: a field study of rotating shift workers that redesigned schedules around these same circadian principles — forward, not backward, rotation among them — found measurable improvements in schedule satisfaction, subjective health, and worker productivity after the change (Czeisler, Moore-Ede & Coleman, "Rotating Shift Work Schedules That Disrupt Sleep Are Improved by Applying Circadian Principles," *Science*, 1982: <https://doi.org/10.1126/science.7089576>). Two schedules can share an identical number of night shifts per month and differ only in which direction the rotation moves between blocks, and the backward-rotating version will reliably produce more adaptation difficulty, more subjective fatigue complaints, and — the number that actually gets a manager's attention — a measurably higher error rate on the first shift of each new block, exactly the pattern Part 6's own CASE-0601 documented.

[FRONTLINE MANAGER] Rotation speed is the second lever, and it cuts in a less intuitive direction than fairness intuition suggests. A fast rotation — changing shifts every two to three days — feels fair because nobody works more than a handful of nights in a row, but it never gives the body enough consecutive days on any one schedule to adapt at all; every rotation restarts the adjustment cost from zero, so the analyst effectively works every night shift while still circadian-adapted to a day schedule, which is its own form of chronic misalignment even though no single stretch looks extreme. A slow rotation — a full week or more per block — gives the body real time to adapt to each block before it changes again, at the cost of a longer stretch on whichever block people like least. Neither speed eliminates the problem; they trade a low-grade, constant misalignment (fast rotation) for a sharper, front-loaded one that improves as the block goes on (slow rotation). A manager choosing between them is choosing which shape of cost the team absorbs, not choosing whether to pay one.

1.4 Consecutive nights and cumulative sleep debt: a worked example

COMPOSITE CASE EXAMPLE — CASE-1701. Constructed from recurring patterns seen across mid-sized in-house SOCs that consolidated night blocks to reduce total shift transitions; no single organization is identifiable, and all figures below are illustrative.

[SENIOR MANAGER] A 20-analyst SOC running four crews of five on a slow, forward rotation decided to consolidate its night block from four consecutive nights to seven, on the reasoning that fewer, longer blocks meant fewer total rotation transitions per year and the same total number of nights worked per analyst either way. The change looked neutral on every scheduling metric the team tracked — same total hours, same total nights per quarter, same pay differential. QA sampling by night-in-sequence, run for one quarter after the change, told a different story.

CONCEPTUAL SAMPLE -- illustrative QA-sampled miss rate, not sourced benchmark data

Night 1 of block:	3.2% missed or misrouted escalation rate (QA-sampled)
Night 2 of block:	3.6%
Night 3 of block:	5.1%
Night 4 of block:	6.9% (previously the last night of the block, before consolidation)
Night 5 of block:	8.4%
Night 6 of block:	9.1%
Night 7 of block:	9.6%

[SENIOR MANAGER] The miss rate did not climb in a straight line — it accelerated after the third night, consistent with cumulative sleep debt compounding rather than simply adding up night over night. Under the old four-night block, the crew never worked past the point where the curve was still climbing gently; under the seven-night block, three additional nights were added squarely in the zone where the curve was steepest. The fix was not to abandon slow rotation — the crew still preferred fewer transitions to more — but to cap consecutive night shifts at four regardless of how the rest of the block was structured, splitting what had been one seven-night block into a four-night block and a separate three-night block with at least one recovery day between them. QA-sampled miss rate on the reduced blocks returned to the 3–5% range across every night worked.

What Would Change My Mind This part treats a roughly four-consecutive-night cap as a reasonable default before cumulative sleep debt starts accelerating rather than accumulating linearly, based on the pattern in CASE-1701 and consistent with general shift-work ergonomics guidance. If a SOC running longer consecutive-night blocks, with deliberate recovery-sleep support built into the schedule (protected sleep windows, no on-call obligation on days off immediately before a block), could show QA or error-rate data that stayed flat rather than accelerating past four nights, that would weaken this default and argue for a team-specific threshold determined by that team's own QA data rather than a fixed number carried over from this case.

1.5 Shift length: the eight-hour and twelve-hour tradeoff, resolved

[FRONTLINE MANAGER] Part 6 §2.2 names the tradeoff between three eight-hour shifts and two twelve-hour shifts — fewer handoffs against a longer individual stretch of sustained vigilance — without

resolving which side of that tradeoff wins. It depends on where the shift sits relative to the window of circadian low. An eight-hour night shift already spends its entire duration inside or adjacent to the trough; stretching that same shift to twelve hours adds four more hours of vigilance-decay on top of an already-compromised baseline, and the decay compounds with time-on-task the same way it compounds across consecutive nights in §1.4 — the last two hours of a twelve-hour overnight shift are reliably the worst two hours of anyone's week, not because the person is unmotivated, but because time-on-task fatigue and circadian trough are stacking on top of each other at exactly that point. A twelve-hour day or evening shift carries much less of this cost, because it never enters the trough at all. The practical rule this part adds to Part 6's tradeoff: twelve-hour shifts are a reasonable default for day and evening coverage, where the popular 2-2-3 pattern's long weekends are a genuine retention win with modest fatigue cost; twelve-hour night shifts should be the last pattern a manager reaches for, not the first, because they concentrate the two most expensive fatigue mechanisms in this part into the same four extra hours.

2. Alert fatigue as a rotation and workload problem

2.1 The split, stated precisely

Cross-Book Pointer This part does not explain how to lower a queue's false-positive rate — that is a detection-tuning fix, not a staffing one. See *Detection Engineering Handbook V2, Part 38 — False Positive Engineering for the tuning mechanics*, and *Part 42 — Detection Quality for how to tell a genuinely noisy rule apart from one that's catching a real, growing pattern*. This part starts downstream of that work: even a well-tuned queue almost never reaches a zero false-positive rate, and alert fatigue is what happens when one analyst absorbs whatever noise floor remains for too many consecutive shifts without relief. Tune the rule to lower the floor. Rotate the person to bound how much of that floor any one person carries alone.

[SENIOR MANAGER] The reason this split matters in practice, not just in principle, is that the two fixes are asymmetric in cost and speed. A tuning fix, once identified, can land in days — Part 9's own worked example ([CASE-0902](#)) shows a detection-engineering fix landing in four days for roughly \$1,500–\$2,000 in engineer time, against a \$260,000-a-year headcount alternative. A rotation fix costs nothing in engineering time and can be implemented as a scheduling-policy change this week, but it does not reduce total noise volume by a single alert — it only changes who is exposed to it and for how long. Reaching for the rotation fix when the real problem is an untuned rule leaves the total noise volume exactly where it was, just spread more evenly across more tired people instead of concentrated on one exhausted one. Reaching for the tuning fix when the real problem is that one analyst has been informally left on the worst queue for eleven of the last fourteen shifts leaves that analyst exactly as overexposed as before, no matter how good the resulting rule gets.

2.2 Diagnosing which problem is actually in front of you

[SENIOR MANAGER] Figure 17.1 sequences the diagnostic questions in the order that resolves fastest, building on the queue-level decomposition Part 9 §3.1 already establishes rather than repeating it. Part

9's decision tree answers whether a *growing queue* is a staffing or detection-quality problem at the aggregate level; this one answers a narrower, person-level question that Part 9's tree never reaches — whether a *stable, already-tuned* queue's residual noise is being absorbed unevenly across the team.

flowchart TD

A["An analyst's disposition accuracy\nor QA score is declining on\none specific queue"] --> B{"Has this queue's post-triage\nfalse-positive rate been checked\nagainst DEH Part 38/42 recently?"}

B -->|"No, or not in 90+ days"| C["Tuning problem, or at least\nunconfirmed -- route to detection\nengineering before anything else"]

B -->|"Yes, tuned and stable"| D{"Has one analyst carried a\ndisproportionate share of this\nqueue's shifts in the last 2-3 weeks?"}

D -->|"Yes"| E["Rotation-exposure problem --\nthis part's fix: cap consecutive\nshifts on this queue, rotate"]

D -->|"No, load already balanced"| F{"Is the decline specific to\nthis one queue, or does it\nshow up everywhere this\nanalyst works?"}

F -->|"Specific to this queue"| G["Possible skill or training\ngap on this queue's content --\nsee Part 12"]

F -->|"Shows up everywhere"| H["Broader performance or\nwellbeing question -- see\nPart 16 before assuming burnout"]

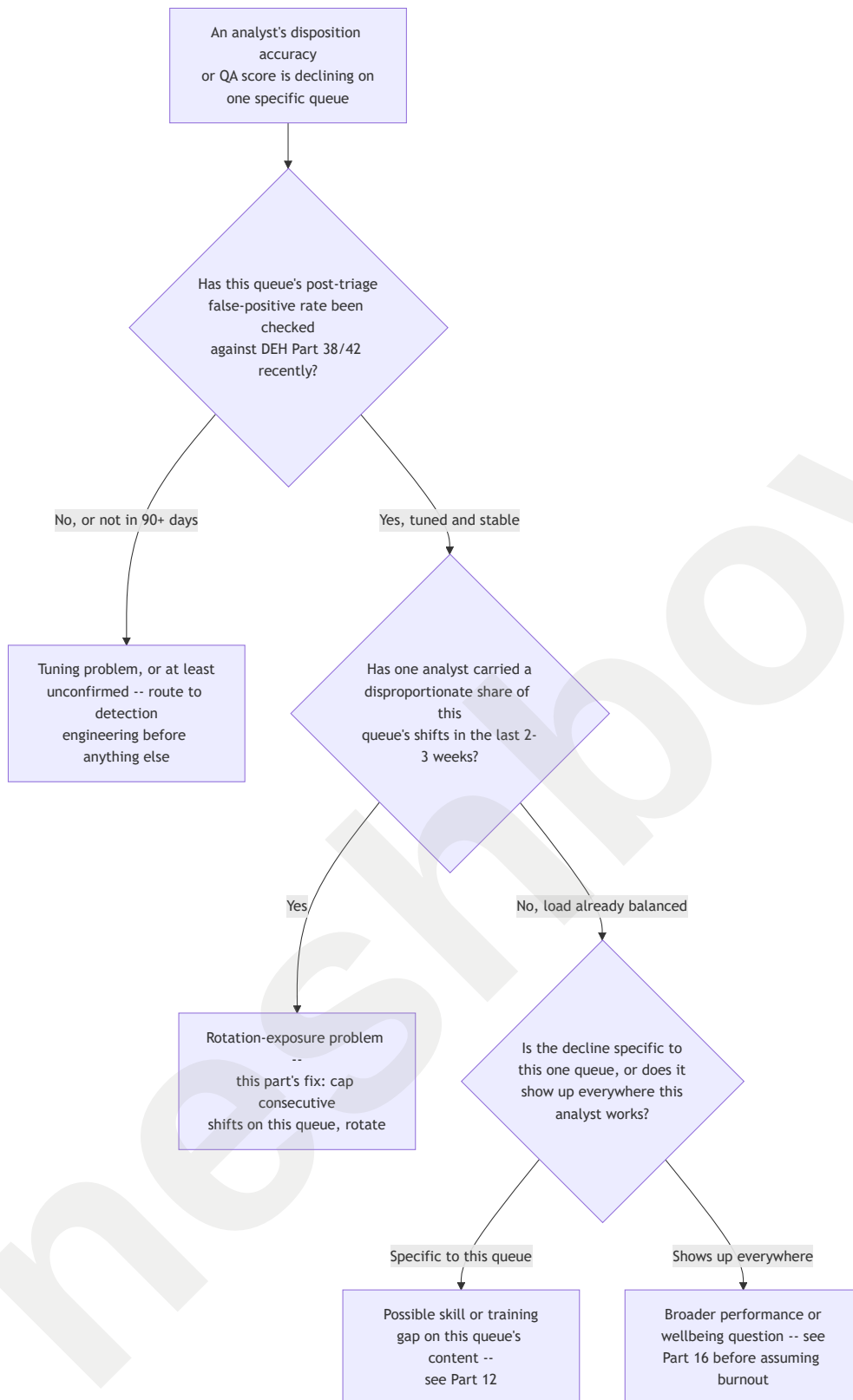


Figure 17.1 — Analyst-decline diagnostic flow. *CONCEPTUAL.* Shows the diagnostic flow as a rendered decision tree. It supports the diagnosis in §2.2 and the routing decision in CASE-1702 below — a declining analyst sits at whichever node this tree actually routes them to, not wherever a manager's first guess assumes. FIG-1701.

[SENIOR MANAGER]

The branch most often skipped under time pressure is the first one: confirming the queue is actually tuned before assuming the person is the problem. A manager who never checks whether the false-positive rate has moved in ninety days risks diagnosing a rotation problem — or worse, a performance problem — on a queue that a two-day tuning fix would have quieted regardless of who was rotated onto it.

2.3 Rotation-based mitigations, once tuning is confirmed

[FRONTLINE MANAGER]

Once a queue's noise floor is confirmed as tuned and stable, three mitigations bound how unevenly the remaining floor gets absorbed. First, a consecutive-shift cap on the specific queue carrying the highest residual noise — a number, published and enforced, not a vague expectation that "someone" will notice an imbalance. Second, mixed-queue assignment within a single shift rather than one analyst working the noisiest queue exclusively all day; splitting a shift between a high-noise and a low-noise queue gives recovery time inside the same day rather than only across days. Third, a tracked noise-exposure metric — tickets worked or minutes spent on the highest-noise queue per analyst per rolling two weeks — reviewed on the same cadence Part 9 §4.1 already recommends for shift-level backlog-age balance, just at finer, per-analyst, per-queue granularity than that check operates at.

Table 17.1 — Noise-exposure rotation policy, by queue tier. Use this table to set a consecutive-shift cap once a queue's false-positive rate is confirmed tuned per §2.1; it is a starting policy to adapt to a specific team's queue mix, not a fixed standard (CONCEPTUAL SAMPLE).

Queue Noise Tier	Illustrative Post-Tuning FP Rate	Consecutive-Shift Cap	Recovery Requirement
Low noise	Under 20%	No cap needed	—
Moderate noise	20%–50%	4 consecutive shifts	1 shift on a lower-noise queue before returning
High, structurally noisy (e.g., user-reported phishing)	Over 50%, expected to stay high even after tuning	3 consecutive shifts	2 shifts on a lower-noise queue, or a full rotation off, before returning

[FRONTLINE MANAGER]

The middle column matters more than the cap itself: a queue that is noisy because of a specific unfixed rule belongs back in §2.1's tuning conversation, not in this table at all. This table is only for queues whose noise is structural — a category, like user-submitted phishing reports, where a meaningful fraction of legitimate submissions will always look like a false positive on triage, no matter how well the surrounding automation is tuned. Applying a rotation cap to a queue that's noisy because of a bad rule treats a two-day fix as a permanent staffing policy.

2.4 Worked example: the phishing-queue rotation cap

COMPOSITE CASE EXAMPLE — CASE-1702. Constructed from recurring patterns seen in SOC's with a dedicated user-reported-phishing queue and an informal "whoever's fastest at it" assignment habit; no single organization is identifiable, and all figures below are illustrative.

[SENIOR MANAGER]

An 8-analyst Tier 1 team ran a user-reported-phishing queue that made up roughly 35% of total ticket volume, with a post-tuning false-positive rate holding steady around 55% —

high, but confirmed structural rather than rule-driven after a detection-engineering review found no single rule or filter concentration to fix. One analyst, consistently the fastest at triaging this queue, was informally left on it for eleven of the previous fourteen shifts, because rotating someone slower onto it visibly slowed the team's overall queue-clearance rate on the days it happened.

CONCEPTUAL SAMPLE -- illustrative disposition-accuracy tracking, not sourced benchmark data

Weeks 1-2 of the stretch: 96% disposition accuracy on phishing-queue tickets (QA-sampled)

Weeks 3-4 of the stretch: 89%

Week 5 of the stretch: 82%, including 2 confirmed true-positive credential-phishing reports dispositioned as benign, both caught later by QA sampling rather than at first triage

[SENIOR MANAGER]

Both missed true positives were credential-harvesting pages that matched a pattern the same analyst had correctly caught dozens of times earlier in the stretch — this was not a skill gap; the analyst had already demonstrated the skill. Figure 17.1's diagnostic path applies directly: the queue was confirmed tuned, the load was confirmed unbalanced, and the decline was specific to this one queue rather than showing up in the analyst's work elsewhere — squarely the rotation-exposure branch, not a training or broader performance question. The fix applied Table 17.1's high-noise-tier policy: a three-consecutive-shift cap on the phishing queue, with the two fastest triagers on the team each taking a rotation instead of one person carrying it continuously. Disposition accuracy across the team on this queue settled at 93–95%, below the original analyst's best two-week stretch but well above where that analyst's own accuracy had fallen by week five, and with no single person carrying the miss risk of a bad week alone. The analyst who had absorbed the heaviest stretch resigned nine weeks later, citing burnout in an exit conversation; replacing them cost the team roughly ten weeks of reduced capacity during sourcing and ramp — the retention-cost accounting for that kind of departure is Part 18's model to run, not this part's, but the fourteen-shift stretch that preceded it is exactly the pattern this section exists to catch before it gets that far.

3. Detecting burnout before it becomes attrition

3.1 Leading indicators versus lagging indicators

[HR/PEOPLE]

A resignation, a spike in sick leave, a QA failure serious enough to trigger a formal review — these are lagging indicators: real, measurable, and already too late to prevent the specific event they're measuring. A wellbeing program that only watches lagging indicators is running a smoke detector that goes off after the building has already burned partway down. Leading indicators — overtime trend, PTO non-use, on-call frequency, a declining QA trend on a previously strong performer, an uptick in unplanned absences, something an analyst says in a 1:1 that sounds like disengagement rather than a bad week — are the signals available while there's still time to act. The entire value of a wellbeing program, as distinct from an exit-interview program, is operating on the leading side of that line.

3.2 A burnout risk scorecard

Table 17.2 — Burnout risk scorecard. Use this on a monthly cadence per analyst, alongside — not instead of — a direct 1:1 conversation; it flags where to look, it does not replace the conversation itself

(CONCEPTUAL SAMPLE — illustrative thresholds, calibrate against a specific team's own baseline before using these as hard triggers).

Indicator	What It Measures	Illustrative Trigger	Manager Action
Overtime hours, trailing 30 days	Sustained work beyond scheduled hours	Over 15 hours in a month, or any 2 consecutive weeks over 5 hours	Ask why directly; check whether it's surge-driven (Part 9 §5) or chronic
Consecutive shifts on highest-noise queue	Uneven noise exposure (§2)	Exceeds Table 17.1's cap for that queue's tier	Rotate immediately; don't wait for the next scheduling cycle
PTO days used vs. accrued, trailing 6 months	Actual time off taken, not just available	Under 50% of accrued days used	Ask what's blocking use — backlog anxiety is the most common answer
Unplanned absences, trailing 90 days	Sudden pattern change from baseline	3 or more, where the analyst's prior baseline was 0-1	Direct check-in; do not treat as a discipline issue by default
On-call pages, trailing 30 days	Disrupted-sleep exposure outside scheduled hours	Above the team's median by a wide margin	Check on-call rotation fairness per Part 6 §5
QA score trend, 3-month	Quality drift on a previously stable performer	A downward trend of 2 consecutive review periods	Route through Figure 17.1 before assuming a skill gap

[HR/PEOPLE] No single row on this table is diagnostic by itself — an analyst who skips PTO because they're saving for a planned trip is not the same as one who skips it because the backlog waiting on their return makes time off feel like a cost rather than a benefit. The scorecard's job is to prompt a specific conversation, not to generate a score a manager acts on without one.

3.3 What the scorecard cannot see

Blind Spot Every row on Table 17.2 is easiest to trigger on the analyst who is already visibly struggling — someone taking unplanned days off, someone whose QA scores are already sliding. It is structurally blind to the analyst who is burned out and still performing: the one who never uses PTO, never flags anything in a 1:1, keeps QA scores flat by sheer discipline, and scores "low risk" on every single row precisely because the coping mechanism is suppression rather than visible strain. That analyst looks identical to a genuinely thriving one on this scorecard right up until they resign with no warning, or their performance drops off a cliff rather than sliding gradually. A scorecard built entirely from observable, quantifiable signals will always under-catch the person whose strategy is not showing any of them — pair it with a psychological-safety culture where disengagement can be said out loud before it's this far along, which is Part 19's territory, not a fix this scorecard can deliver on its own.

Manager's Note Ask the "never takes PTO, never complains" analyst directly, by name, in a 1:1: "what would make you actually take the time off you've been sitting on." Not "are you okay" — that question gets a reflexive "yes" from exactly the person this Blind Spot describes. A specific, practical question about a specific, observable pattern gets a more honest answer than a general check-in question does.

4. Wellbeing program design

4.1 The core components

[HR/PEOPLE] A SOC wellbeing program that's more than a poster in a break room has four working parts, and the first three of the four earn their place by directly enforcing what §2 and §3 already establish: a workload-exposure policy (Table 17.1's rotation caps, enforced, not aspirational), a leading-indicator review cadence (Table 17.2, run monthly and acted on), an enforced PTO minimum — a floor, not just an accrual balance — and access to a mental-health benefit an analyst can use without going through their own chain of command first. The fourth component, mandatory disconnected time after a major incident, is significant enough to warrant its own treatment below.

Operational Reality Most SOC's already have a generous PTO policy on paper and a real usage rate well under what that policy allows, because the backlog waiting on return makes the time off feel like it costs more than it's worth. An enforced minimum — "you must use at least this many days by this date, and your manager is notified if you haven't" — only works if it's paired with a coverage plan for the specific days requested, using the same shift-handoff and workload-balancing mechanics Part 6 §7 and Part 9 §4 already build for routine coverage gaps. A PTO mandate with no coverage plan behind it just moves the anxiety from "should I take this time" to "what's going to be waiting for me," which defeats the purpose just as effectively as no mandate at all.

[VENDOR/PROCUREMENT] Selecting and contracting an employee-assistance or mental-health benefit vendor is a procurement decision with its own evaluation criteria — utilization data privacy, network adequacy for the team's actual locations, response-time commitments — that belongs to Part 21 — Tooling Procurement & Platform Strategy and Part 23 — Vendor Relationship & Renewal Management's general vendor-governance mechanics rather than to this part. This part's job stops at establishing that the benefit needs to exist and needs to be genuinely, confidentially accessible; which vendor provides it is someone else's evaluation.

4.2 Mandatory disconnected time after a major incident

[SENIOR MANAGER] A major incident concentrates fatigue risk the way a consolidated night block does in §1.4 — sustained, high-intensity engagement with compressed or interrupted sleep, often for days, on top of whatever baseline schedule was already running. The mechanism is identical to shift-work

fatigue; the trigger is different, and it needs its own policy rather than an assumption that normal PTO or normal rotation caps will catch it, because a major incident overrides both by design while it's live.

[SENIOR MANAGER] A disconnect policy that actually works specifies four things, none of which should be left to the responder's own judgment in the moment: who it covers (the incident commander and any analyst who carried primary investigative load, not only whoever holds an official title), how long the minimum window is (48 to 72 hours is a reasonable default for a multi-day incident, scaled to the incident's actual duration and intensity rather than fixed at one number for every severity), what "disconnected" actually means operationally (removed from the incident channel, on-call phone reassigned, calendar meetings for that window cancelled or reassigned, not merely "encouraged to rest" while still reachable), and who has the authority to grant an exception if the responder genuinely wants to stay engaged through a specific handoff meeting. The person deciding who's covered and when the clock starts is doing exactly the kind of live staffing-and-backfill judgment call this book's Part 28 — The Manager's Role in a Major Incident owns; this part's job is the standing policy that Part 28's live decision executes against, not the live decision itself.

[SENIOR MANAGER] The mechanism that makes a disconnect window survivable without losing incident context is the same one Part 6 §7 already builds for an ordinary shift boundary, applied at incident-closure scale instead of a daily one: a written handoff covering open work in progress, watch items, system status, and what to expect next, produced before the responder disconnects rather than narrated from memory as they leave. A designated backup — not "whoever's around" — reads it, acknowledges receipt, and owns anything that surfaces during the window. Skipping this step and simply sending the responder home with no structured transfer produces the same failure mode Part 6's **CASE-0603** documents for a cut shift-overlap window, just at a scale where the missed watch item is a live intrusion's remaining foothold rather than a routine ticket.

4.3 Why "encouraged" time off doesn't hold under pressure

Management Autopsy — "keep the incident commander engaged because they have the context" (COMPOSITE CASE EXAMPLE, CASE-1703)

The decision: After a declared business-email-compromise incident closed on its fifth day, the incident commander and the two analysts who'd carried the bulk of the investigation were kept "available" for follow-up meetings and questions over the following week rather than given a hard disconnect window — the plan called it "light duty," a few hours a day of debrief and documentation instead of a full return to the queue.

Why it seemed reasonable: They had the freshest, most complete context on the incident, follow-up questions from legal and from the affected business unit kept arriving daily, and re-briefing someone else from scratch felt slower than just answering the questions directly.

How it failed: "Light duty" in practice meant broken sleep continued for another week on top of the five days already spent on the incident itself, with no defined end point. Three weeks after the original incident closed, a second, smaller phishing-driven compromise attempt landed in the same environment. The same incident commander, still not meaningfully rested, missed a lateral-movement indicator in the first six hours that a second analyst caught only after independently re-reviewing the same telemetry — a detail this book's own case-review discipline would flag in a Part 29 postmortem as directly traceable to accumulated fatigue, not to a gap in the commander's skill or the detection content itself.

The fix: Replace "available if needed" with a hard minimum disconnect window — in this organization's revised policy, 72 hours, full removal from the incident channel and on-call phone, with follow-up questions routed to a designated backup who received the written handoff described in §4.2 before the commander disconnected. Legal and business-unit follow-up still got answered; it went through the backup instead of through someone who badly needed to stop.

4.4 Verifying the policy actually holds

Field Test Setup: A written mandatory-disconnect policy exists after a major incident, with a defined minimum window and a named backup process. **Action:** During the next real major-incident closure, have someone outside the incident itself — the SOC manager, or a peer team lead — check, partway through the mandated window, whether the covered responder has actually been removed from the incident channel and on-call rotation, not just told to rest.

Expected result: The responder should be unreachable through the incident's normal channels for the full window, and the backup should be fielding follow-up questions without needing to escalate back to the disconnected responder for context the handoff record should already contain. If the responder is still answering pings "just to help out," or the backup can't answer a question without pulling them back in, the policy is aspirational, not operational — fix the handoff record or the backup's authority before trusting the policy in a real incident again.

5. Where this goes next

[CONCEPT] This part gave a manager three things: the circadian and shift-work evidence to make rotation-direction, rotation-speed, and shift-length decisions defensibly rather than by habit; a way to tell an alert-fatigue problem that's really a tuning problem from one that's really a rotation problem, and a rotation policy to apply once it's confirmed the latter; and a wellbeing program with leading-indicator detection and a mandatory disconnect policy that holds under real incident pressure rather than existing only on paper. It deliberately left the retention-cost math for a burnout-driven departure to Part 18, the diagnostic call between burnout and a genuine skill gap to Part 16, the live staffing-surge decision during a declared incident to Part 28, the standing postmortem program that would eventually audit this part's own disconnect policy to Part 29, and the technical false-positive-rate reduction behind every noisy queue this part assumes is already tuned to Detection Engineering Handbook V2, Part 38. Nothing past this point should need to re-derive why rotation direction matters or re-litigate whether alert fatigue is a people problem or a rule problem — it should use Figure 17.1 to find out which one it's looking at and act accordingly.

Cross-references

This part assumes Part 6 — Shift Pattern & Coverage Design (the coverage model and rotation structure this part supplies the fatigue physiology for) and Part 9 — Queue Health & Workload Management (the queue-level decomposition method §2.2's person-level diagnostic builds on rather than repeats). It points forward, within this book, to Part 12 — Ongoing Training & Skill Development (the skill-gap branch of Figure 17.1), Part 16 — Performance Management & Coaching (the broader performance-versus-wellbeing branch of the same figure), Part 18 — Attrition & Retention (the cost model for a burnout-driven departure this part only flags, never prices), Part 19 — Team Culture & Psychological Safety (the disclosure culture the §3.3 Blind Spot depends on), Part 21 — Tooling Procurement & Platform Strategy and Part 23 — Vendor Relationship & Renewal Management (EAP/benefit vendor selection, deferred from §4.1), Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty (when a rotation cap and a genuine staffing shortfall collide with no clean answer), Part 28 — The Manager's Role in a Major Incident (the live staffing-and-backfill authority §4.2's disconnect

policy executes against), and Part 29 — Post-Incident Organizational Review (where a disconnect policy's own compliance eventually gets audited, as [CASE-1703](#) illustrates). Other volumes: Detection Engineering Handbook V2, Part 38 — False Positive Engineering and Part 42 — Detection Quality (the tuning-side fix for alert fatigue this part deliberately does not re-derive, per §2.1).

neshb0y

Part 18 — Attrition & Retention

Why this part exists

[CONCEPT] Every SOC loses people. The question this part answers is narrower than "how do we stop that" — no manager stops it entirely, and a program built on that premise is already lying to itself. The real questions are: which of the departures walking out the door this year were predictable and preventable, which lever actually would have kept the preventable ones, and what does it cost to pull that lever compared to what it costs not to. This part covers four things: the two attrition patterns that show up in a SOC more often and more predictably than in most other technical teams — the burnout-driven exit clustering in the 18- to 24-month range, and competitor poaching of mid-level analysts; a structured method for turning exit interviews into something a manager can act on instead of a folder of anecdotes; a hard look at which retention levers actually move a resignation rate and which ones only feel like they should; and a cost model that puts a real dollar figure on a departure so a retention investment can be defended in the same room as a raise pool, a training budget, or a new hire.

[CONCEPT] This part sits in Section D, between Part 17 — Burnout, Fatigue & Wellbeing and Part 19 — Team Culture & Psychological Safety, and it deliberately does not re-walk either neighbor's ground. Part 17 owns the mechanism behind burnout-driven exits — the shift-fatigue physiology, the circadian research, the rotation and wellbeing fixes — and this part cites that mechanism rather than re-deriving it; this part's job starts at the point where burnout has already produced, or is about to produce, a resignation. Part 14 — Mentorship & Knowledge Transfer owns the preventive response to a departure once it's coming — succession planning, knowledge capture, the exit-window sprint — and this part cites its cost figures rather than re-running that program here. Part 13 — Career Ladders & Promotion Criteria owns the mechanics of building a ladder and running a promotion committee; this part treats a working ladder as a retention lever and prices what happens when one is missing, without re-explaining how to build it. Part 20 — Building & Defending the SOC Budget owns how a retention investment gets defended as a line item against a CFO's questions; this part builds the number Part 20 defends. And Part 5 — Headcount & Capacity Modeling owns the arithmetic of shrinkage and backfill lag as headcount inputs; this part borrows that arithmetic as one ingredient in a cost-of-departure figure, not as something to re-derive from scratch. If a paragraph below starts explaining shift physiology, a promotion committee's composition, or a headcount formula, that paragraph has drifted — the fix is a citation, not more prose.

1. The SOC attrition curve: two patterns, two mechanisms

1.1 The 18- to 24-month burnout exit

[CONCEPT] A SOC's Tier 1 tenure curve has a recognizable shape: strong early retention through the first year, while the job is still new and the learning curve is still steep, followed by a sharp resignation spike concentrated in the 18- to 24-month window, tapering off for whoever survives past it. That window isn't arbitrary. It's roughly how long sustained shift-work fatigue, alert volume, and the accumulated weight of triaging other people's emergencies takes to outweigh the novelty that carried a new analyst through their first year — the physiological and workload mechanics behind that curve are

Part 17's territory in full. What belongs here is the shape of the curve itself as a planning fact: if a SOC doesn't know where its own curve peaks, every retention conversation happens too late, aimed at analysts who are already several months past the point where a fix would have worked.

[SENIOR MANAGER] The practical use of the curve is timing, not diagnosis. Pull tenure-at-departure for the last two years of voluntary resignations and plot it as a simple histogram — most SOC's that do this for the first time are surprised by how sharply it clusters rather than spreading evenly across tenure. If the SOC's own peak sits at month 16 rather than month 20, the retention conversation — a 1:1 specifically about trajectory, workload, and what's next, not a routine check-in — needs to start by month 12, not month 18, because a conversation that starts after the disengagement has already set in is closer to an exit interview than a retention intervention.

1.2 Competitor poaching of mid-level analysts

[SENIOR MANAGER] The second pattern has a different mechanism and a different fix, and confusing the two is the single most common error in this part's territory. Poaching targets the analyst who is good enough to be immediately productive somewhere else — typically 18 months to three years in, past the burnout-exit window, with real triage judgment and no longer a flight risk on fatigue grounds alone. A competing MSSP or an enterprise standing up its own in-house SOC doesn't want to train someone; it wants someone who can carry a queue on day one, and it's willing to pay a real premium — commonly in the 15% to 20% range above the market rate this SOC is paying — to skip the ramp-up curve entirely. Part 7 — Hiring & Sourcing Analysts already documents how tight the mid-level market is from the sourcing side; this is the same tightness, experienced from the losing side of the transaction.

[SENIOR MANAGER] Part 10 — Competency Models & Skills Matrices shows the mechanism concretely in its own tenure-gating case: an analyst who was demonstrably ready for L2-level judgment months before the internal ladder's clock said so left for a competitor offering the L2 title immediately, rather than waiting out an arbitrary internal timeline. That's the poaching pattern's signature: it doesn't need to out-pay a fair internal ladder, only a slow or rigid one. A competitor doesn't have to win on total compensation if the internal path is visibly capped or visibly delayed relative to demonstrated readiness — it only has to offer the recognition faster than this SOC's own process does.

1.3 Naming the pattern before choosing the fix

[CONCEPT] A retention conversation that starts with "why are people leaving" instead of "which pattern is this" tends to reach for whatever fix is culturally comfortable — usually a culture initiative — regardless of whether that fix matches the actual mechanism. The table below is the diagnostic starting point Section 4 builds its lever recommendations from.

Table 18.1 — Attrition pattern taxonomy for a SOC. Use this to classify a departure (or a cluster of departures) before reaching for a fix; a fix aimed at the wrong pattern below wastes budget and doesn't move the number it was meant to move.

Pattern	Typical tenure window	Signature signal	Primary driver	Owning fix
Burnout-driven exit	18–24 months	Declining handle time/QA score, rising unplanned absence, disengagement in 1:1s	Sustained shift fatigue, alert volume, no recovery time	Rotation/wellbeing design (Part 17), workload rebalancing (Part 9)
Competitor poaching	18 months to 3 years	Sudden resignation, minimal notice, named competitor/MSSP as destination	Pay gap or recognition-speed gap versus market	Pay-band correction, ladder-speed fix (\$4.1–4.2)
Plateau/no-growth exit	2–4 years	"Nowhere to go" cited directly in exit interview	No dual-ladder path for a strong senior IC	Dual-ladder design (Part 13 \$3.4)
Shift-burden exit	Concentrated on night/weekend crews	Departures cluster on one shift, not team-wide	Uncompensated or unrotated undesirable shift load	Shift redesign, on-call pay (Part 6)
Performance-managed-out	Any tenure	Departure follows an open PIP	Skill/fit mismatch, not a retention failure	Part 16 — not this part's territory

[SENIOR MANAGER] The last row matters as much as the first four: not every departure is a retention failure, and a program that tries to "retain" someone already on a performance-improvement plan is solving the wrong problem with the wrong tool. Part 16 — Performance Management & Coaching owns that judgment call; this part's levers apply to the first four rows, not the fifth.

2. Reading the signal before the resignation letter

2.1 Leading indicators: metrics, not vibes

[SENIOR MANAGER] By the time someone hands in a resignation letter, the decision is usually weeks or months old, which means a manager who only reacts to the letter is always working the trailing edge of the problem. The leading indicators worth tracking are operational metrics the SOC already collects for other reasons — handle time and escalation rate, as defined once and consistently in SOC Playbook Handbook, Part 32 — Metrics rather than redefined here, plus QA score, this book's own metric defined in Part 15 — Quality Assurance Programs — read for a specific shape: a sustained decline over 60 to 90 days in an analyst who was previously stable, not a single bad week. Part 9 — Queue Health & Workload Management already treats a declining team-wide clearance rate as a signal that routes to either a staffing diagnosis or a skill diagnosis; the same logic applied to one person, rather than a whole shift, is this part's leading-indicator signal for an individual at risk of leaving.

Cross-Book Pointer This part does not define handle time or escalation rate — those definitions live in SOC Playbook Handbook, Part 32 — Metrics, and using a locally redefined version of either one here would silently create the exact same-number-two-ways problem this book's own production model is built to avoid. QA score is a different case: it's this book's own metric, defined once in Part 15 — Quality Assurance Programs and not redefined here either. What this part adds is a specific read on all three existing metrics: a 60- to 90-day downward trend in an individual's handle time or QA score, previously stable, is a flight-risk signal worth a direct conversation, distinct from the team-wide, program-level reading SOC Playbook Part 32 and this book's own Part 15 apply to the same underlying numbers.

2.2 The blind spot in every exit interview

[HR/PEOPLE] Every exit-interview program has a structural ceiling on what it can see, and a manager who forgets that ceiling exists will over-trust the data it produces.

Blind Spot An exit interview only ever talks to someone who has already decided to leave — it captures a completed decision, not the six or eight weeks of disengagement that preceded it. That's fine for identifying a pattern across many past departures, and useless for catching the specific analyst who is six weeks out from resigning right now but hasn't said anything yet. A retention program built entirely on exit-interview themes is permanently one resignation cycle behind the attrition it's trying to prevent; §2.1's leading indicators and §2.3's stay interviews exist specifically to close that gap.

2.3 Stay interviews: the complement, not a replacement

[FRONTLINE MANAGER] A stay interview is the same structured-conversation format as an exit interview, run on someone with no plans to leave, asking a narrower question: what would make you consider leaving, and what's the best part of this job that a competitor's offer couldn't replicate. It works because it removes the two biases that distort an exit interview — the departing person's incentive to give a clean, non-burning-bridges answer, and the survivor-bias gap of never hearing from the people who stayed. Run it on a rolling basis, a small handful of conversations a quarter rather than a single annual event, so it functions as an ongoing pulse check rather than a survey that goes stale the month after it runs.

Field Test Setup: Pick two or three analysts at random across different tenure bands — not just your most senior or most obviously engaged people. **Action:** Run a 20-minute stay interview using three fixed questions: "What's the one thing that would make you look at other openings?" "What's kept you here through a hard week recently?" and "If you could change one thing about how this role works day to day, what would it be?" **Expected result:** At least one answer per conversation should surprise you — something you couldn't have predicted from that person's metrics or your last 1:1. If every answer matches what you already assumed, either you already have unusually good visibility into your team, or the conversation wasn't candid; check which by asking someone else on the team what they told you.

3. Structured exit-interview analysis

3.1 Designing a question set that produces comparable data

[HR/PEOPLE] An exit interview run as an open-ended "so, what happened" conversation produces a different, incomparable narrative every time, which feels thorough and analyzes badly — six months of exit interviews conducted this way leaves a manager with six good stories and no pattern. A structured question set, asked the same way every time, trades some conversational warmth for the ability to code answers against the Table 18.1 taxonomy and actually count them.

The exit-interview question set (TMPL-1801), filed in Appendix A4 alongside the QA scorecard, calibration-session guide, and performance-improvement-plan template, is used by whoever runs the departing analyst's final conversation — ideally an HR partner or a manager one level removed from the departing person's own chain, not their direct manager, who is both an interested party and often (correctly) not trusted with a fully candid answer. Its main limitation: it structures the conversation, but it can't make a departing employee candid who has already decided that candor isn't worth the effort on their way out the door — corroborate its themes against the leading indicators in §2.1 and the stay-interview data in §2.3 rather than trusting a single exit interview's coded answer at face value.

3.2 Coding themes without overfitting to the last three departures

[SENIOR MANAGER] The most common analysis mistake is treating the most recent departure's stated reason as the organization's attrition problem, when a handful of interviews is too small a sample to generalize from reliably. Code every exit interview against the same fixed category list — ideally Table 18.1's five patterns, plus an "other" bucket for anything genuinely novel — and only treat a category as a real, actionable signal once it accounts for a clear plurality across a rolling 12-month window, not a single quarter's cluster. A SOC that loses three analysts in one bad month to three unrelated causes (one relocation, one performance exit, one genuine poach) will misread that month as a crisis in progress if it reacts to the raw count instead of the coded pattern underneath it.

3.3 Turning themes into a report a budget owner will read

[EXECUTIVE] The output of this analysis, run quarterly, should be short enough that a budget owner actually reads it: the baseline voluntary-attrition rate for the trailing 12 months, the coded breakdown

against Table 18.1's five patterns, and — this is the part that turns a report into a decision — the one or two levers from Section 4 that map to whichever pattern is currently the largest coded share. A report that lists twelve themes with no ranking and no recommended action gets filed, not acted on; a report that says "poaching accounts for more coded departures than every other pattern combined this year, and here's what closing the pay gap costs against what the departures already cost" gets a budget conversation.

4. Retention levers that move the needle, and the ones that don't

4.1 Pay-band correction against poaching-driven exits

[SENIOR MANAGER] When Table 18.1's poaching row is the dominant coded pattern, the lever that actually moves the resignation rate is a pay-band correction for the specific level being targeted — not a raise pool spread evenly across the whole team, which dilutes the fix exactly where it's needed least and does almost nothing where it's needed most. Section 5.3 works this as a full worked example with real numbers; the design point here is narrower: benchmark the specific level being poached against the specific competitor doing the poaching, not against a generic industry survey that may not reflect the local labor market a competitor just disrupted by opening an office nearby.

4.2 Career-pathing against plateau exits

[SENIOR MANAGER] When Table 18.1's plateau row dominates, the fix is structural, not financial: a dual-ladder design that gives a strong senior individual contributor a title, a pay ceiling, and real scope above Senior Analyst without forcing them into the team-lead branch — Part 13 §3.4 already owns this design in full, and this part's job is only to confirm it's the lever that actually addresses this specific pattern. Throwing a pay-band correction at a plateau exit without the structural fix just delays the same resignation; the person wasn't underpaid relative to their current title, they had no title left to grow into.

4.3 Shift and on-call redesign against shift-burden exits

[FRONTLINE MANAGER] When departures cluster on one shift rather than spreading evenly across the team, the diagnostic in Table 18.1 points at the shift itself, not the people rostered on it — a night crew that skews toward whoever has the least seniority and the least ability to say no, as Part 6 names it, will keep losing whoever's on it regardless of how good this SOC's pay band or career ladder otherwise is. The fix is Part 6's territory: rotation redesign, a real on-call stipend rather than an assumption that base salary already covers being paged, and rotating the undesirable shift's burden across more of the team instead of concentrating it on the newest hires by default.

4.4 What doesn't move the needle

[SENIOR MANAGER] Three levers get reached for constantly because they're cheap, fast, and visible, and none of them address the actual mechanism behind Table 18.1's top two rows.

Management Autopsy — "answer a poaching wave with a culture initiative"
(COMPOSITE CASE EXAMPLE, CASE-1802)

The decision: After losing several mid-level analysts to a competitor's aggressive local hiring push, a SOC manager responds by launching a team-culture initiative — a catered monthly lunch, a revamped recognition program, a redecorated break room — without adjusting the pay band the departing analysts were being outbid on.

Why it seemed reasonable: The initiative was fast to approve, visible to the whole team immediately, and framed internally as "investing in retention" without requiring a difficult compensation conversation with finance.

How it failed: The analysts still on the team liked the lunches and kept interviewing anyway, because a free lunch doesn't close a \$12,000-a-year gap (CONCEPTUAL SAMPLE — illustrative pay gap, not sourced benchmark data) between what this SOC pays and what the competitor down the road is offering for the identical job. Two more resignations followed within the same quarter, both citing pay directly in their exit interviews, and the manager had by then spent real budget on something that measurably didn't touch the pattern actually driving the losses.

The fix: Diagnose the pattern against Table 18.1 before choosing the lever — a poaching-driven exit is a pay-gap problem and gets a pay-band fix (§4.1, §5.3); a culture initiative is a real and valid investment against a completely different pattern (disengagement, weak psychological safety, the territory Part 19 owns), and spending it against the wrong pattern doesn't just fail to help, it spends the budget that could have funded the fix that would have worked.

[SENIOR MANAGER] Counter-offers are the second lever that looks like it works and mostly doesn't. Matching one departing analyst's outside offer, one time, can be the right call — but it establishes, in full view of the rest of the team, that the fastest route to a raise is an outside offer in hand, not a documented case through the normal process.

People Risk Trap A counter-offer strategy run repeatedly, case by case, without a corresponding pay-band review, trains the whole team that leverage — not performance — is what actually moves pay, and it recreates the exact leveling-drift dynamic Part 13's own case study on retention-driven counter-promotions describes, just on the compensation axis instead of the title axis. The fix: cap counter-offers at genuinely rare, individually justified exceptions, and treat two or more counter-offers on the same role within a year as a mandatory trigger for a full pay-band review under §4.1 — the review is the actual fix, and the counter-offer was only ever buying time for it to happen.

[HR/PEOPLE] The third weak lever is the exit interview itself, treated as if collecting the data were the intervention. Running exit interviews and coding them well (§3) is real, necessary infrastructure — but it changes nothing on its own unless the coded themes actually drive a §4.1–4.3 decision afterward. A SOC

that has run exit interviews faithfully for three years with no pay-band review and no ladder change in that window isn't running a retention program; it's running a well-documented attrition-observation program.

4.5 A lever scorecard

[SENIOR MANAGER] Table 18.2 consolidates §4.1 through §4.4 into a single reference for a manager choosing where to spend a limited retention budget in a given quarter.

Table 18.2 — Retention lever scorecard. Match the lever to the coded pattern from Table 18.1 before committing budget; a lever aimed at the wrong pattern below is money spent with no measurable effect on the resignation rate it was meant to move.

Lever	Targets which pattern	Moves the needle?	Typical cost	Notes
Pay-band correction	Poaching	Yes	Recurring, several thousand dollars per affected analyst per year	See §4.1, §5.3
Dual-ladder / senior-IC track	Plateau	Yes	Low direct cost; mostly a design and title decision	Part 13 §3.4 owns the design
Shift/on-call redesign	Shift-burden	Yes	Low to moderate — stipend cost, coverage-model change	Part 6 owns the mechanics
Rotation/wellbeing program	Burnout	Yes, on the burnout row specifically	Moderate — coverage cost during protected recovery time	Part 17 owns the mechanics
One-time counter-offer	Poaching (single case)	Short-term only	One-time, often less than a full pay-band fix	Escalates to §4.1 if repeated
Culture/perks initiative	Disengagement (Part 19's territory)	Not for poaching or pay-driven plateau exits	Low to moderate, recurring	Real value against the wrong pattern is a mismatch, not a benefit
Exit interviews alone, no follow-through	None	No	Low	Necessary input, not itself an intervention

5. Modeling the true cost of attrition against retention investment

5.1 What belongs in a fully loaded cost-per-departure figure

[SENIOR MANAGER] A retention investment gets rejected far more often for lack of a comparison number than for being a bad idea on its own terms — a \$9,000 pay correction sounds expensive in isolation and cheap the moment it's set next to what replacing the person it retains actually costs. Build the comparison from five components, each already covered elsewhere in this book and assembled here rather than re-derived: sourcing and recruiting cost (Part 7's channel-cost data), the coverage gap during backfill lag (Part 5's time-to-fill and shrinkage arithmetic), lost output while the replacement ramps to full productivity, the manager and team-lead time spent on the search and onboarding, and

— the component most models skip — the knowledge and judgment that leaves with the departing person, which Part 14's own succession-planning case already prices concretely for a specialist role.

5.2 Worked example: the fully loaded cost of one mid-level departure

CONCEPTUAL SAMPLE -- illustrative figures for a generalist L2 analyst departure, not sourced benchmark or audited financial data; substitute your own team's actual recruiting, ramp, and coverage costs before using this for a real budget defense.

Sourcing and recruiting (blended internal/external channel, per Part 7's cost-per-hire range):
\$7,000
Backfill-lag coverage gap (70 days at reduced capacity; overtime/contractor coverage):
\$9,500
Ramp-to-productivity shortfall (reduced output for ~90 days post-start, per Part 11's ramp curve):
\$6,500
Manager/team-lead time (search, interviews, onboarding oversight, ~40 hours fully loaded):
\$2,500
Knowledge loss / elevated error-rate cost during the gap (lower bound for a generalist L2 seat):
\$3,500

Fully loaded cost, one mid-level departure:
~\$29,000

[SENIOR MANAGER] That figure is a floor, not a ceiling — it assumes a generalist L2 seat with reasonably documented procedures. Part 14 — Mentorship & Knowledge Transfer's own case ([CASE-1401](#)) prices a specialist departure with no succession plan at a materially higher cost: roughly \$46,000 in interim contractor coverage and \$22,000 in absorbed overtime in a single quarter alone, on top of a 116-day replacement search, because a detection engineer's departure carries technical debt a generalist analyst's doesn't. Losing a detection engineer with undocumented tuning rationale also converts every rule that engineer owned into detection debt in Detection Engineering Handbook V2's own terms (DEH V2, Part 43 — Detection Debt) the moment they're unavailable to explain it — a cost this part's dollar figure doesn't fully capture and that book's does. Part 3 — Tiering Models already flags this asymmetry directly: losing a Tier 3 or lead-level analyst hurts disproportionately to the headcount share that role represents, and the \$29,000 floor above should be scaled up accordingly for any seat with real specialist depth.

5.3 Worked example: the poaching wave

CASE-1801 — the local-office poaching wave. *COMPOSITE CASE EXAMPLE — merges patterns from several mid-market SOC's facing new local competition, not one traceable organization; figures below are illustrative.*

[SENIOR MANAGER] A 28-analyst SOC runs a 10-person L2 tier at a pay band of \$70,000–\$80,000. A competing MSSP opens a local office and begins recruiting directly from the area's SOC analyst pool at a 15% to 20% premium over that band. Within four months, the SOC loses four of its 10 L2 analysts to the new competitor — a coded poaching pattern under Table 18.1, confirmed by name-the-competitor answers across all four exit interviews.

[SENIOR MANAGER] The manager's first response is the one-time counter-offer lever from §4.4: a \$5,000 retention bonus offered to the remaining six L2 analysts, with no change to the underlying pay band. One additional analyst resigns within the following 90 days despite having taken the bonus, citing the same pay gap in their exit interview — the bonus bought a quarter of retention on that individual case, not a fix to the pattern.

[SENIOR MANAGER] The root-cause response, approved for the next budget cycle through the business case Part 20 owns building, corrects the L2 band to \$80,000–\$92,000 for the remaining five L2 analysts — an average increase of roughly \$9,500 per analyst, or about \$47,500 a year in recurring cost across the corrected cohort. Applying §5.2's \$29,000 floor, avoiding just two further poaching-driven departures a year pays back the correction's entire recurring cost, before counting the knowledge, continuity, and coverage benefits a raw dollar comparison leaves out. Over the following 12 months, the corrected cohort loses one analyst, not the four-of-ten pace the SOC was on before the fix — a result consistent with §4.1's claim that a targeted pay-band correction, not a culture initiative or a repeatable bonus, is the lever that actually closes a poaching-driven gap.

5.4 A decision path from signal to lever

[SENIOR MANAGER] The flow below ties Sections 2 through 5 together as a single path a manager can actually run: from a leading indicator or a resignation, through classification against Table 18.1, to the lever and the cost comparison that justifies it.

flowchart TD

```
A["Attrition signal appears:\nresignation OR leading indicator\nfrom Section 2"] --> B{"Resignation already\ngiven, or still a\nleading indicator?"}
B -->|"Leading indicator only"| C["Run a stay interview (2.3);\nclassify against Table 18.1\nbefore it becomes a resignation"]
B -->|"Resignation given"| D["Run structured exit\ninterview, TMPL-1801 (3.1)"]
C --> E{"Classify against\nTable 18.1"}
D --> E
E -->|"Burnout-driven"| F["Route to Part 17 rotation/\nwellbeing fix"]
E -->|"Poaching"| G["Pay-band correction (4.1);\nprice against 5.2's cost floor"]
E -->|"Plateau / no growth"| H["Dual-ladder design,\nPart 13 §3.4"]
E -->|"Shift-burden"| I["Shift/on-call redesign,\nPart 6"]
E -->|"Performance-managed-out"| J["Not this part's territory --\nsee Part 16"]
F --> K["Log coded pattern in\nquarterly report (3.3)"]
G --> K
H --> K
I --> K
J --> K
K --> L{"Same pattern dominant\nacross rolling 12 months?"}
L -->|"Yes"| M["Escalate to a funded\nprogram, Part 20 budget cycle"]
L -->|"No, isolated case"| N["Address individually;\nno program-level change yet"]
```

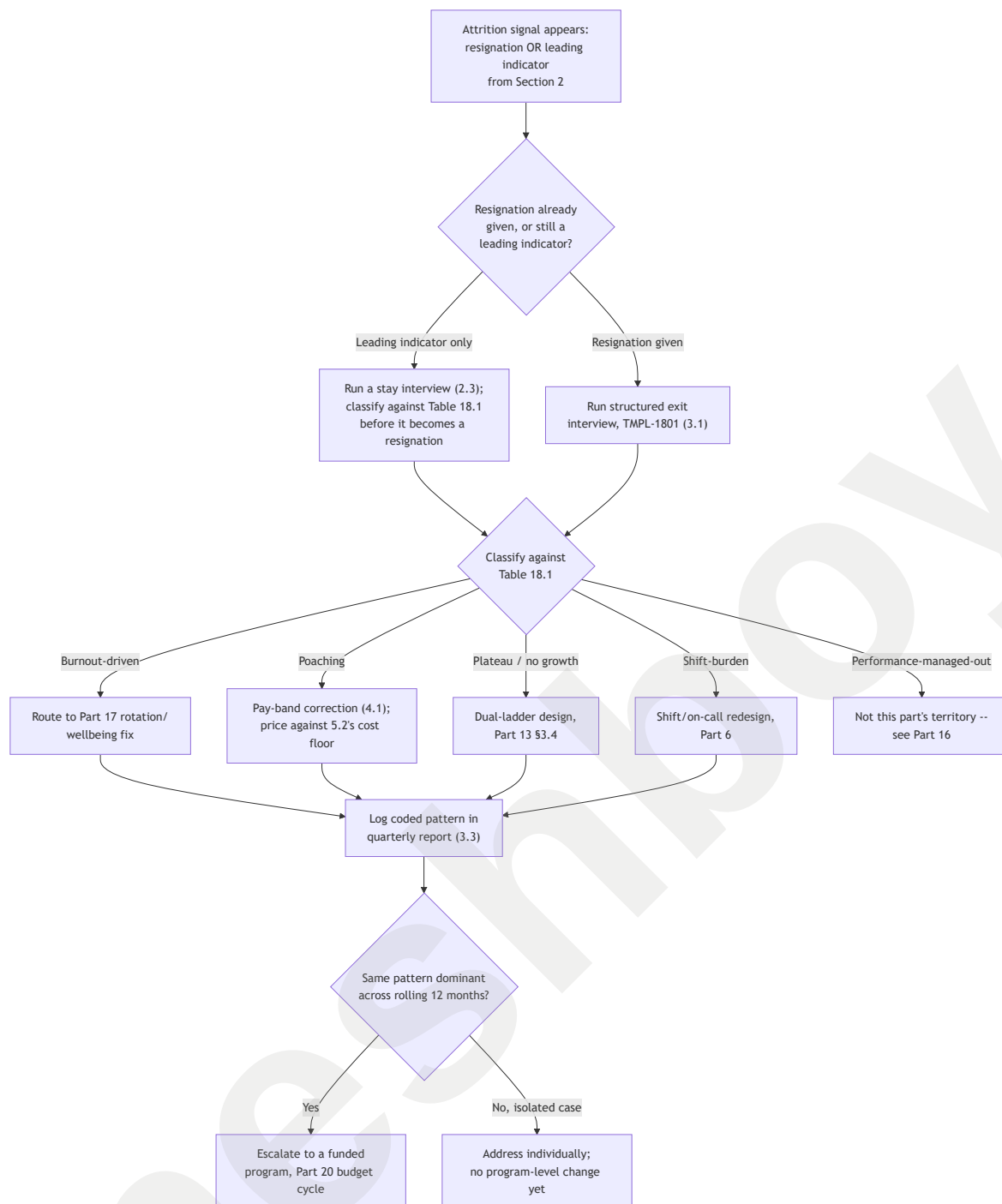


Figure 18.1 — From attrition signal to lever, classification through budget escalation.

CONCEPTUAL. Illustrates the decision path this part builds across Sections 2 through 5: a signal (leading indicator or resignation) is classified against the Table 18.1 taxonomy, routed to the matching lever, logged, and escalated to a funded program only once a pattern, not an isolated case, shows up across a rolling 12-month window. This is a process design, not a capture of any single organization's actual workflow. Supports every lever recommendation in Section 4 and the escalation trigger referenced in §3.3 and Part 20. **FIG-1801.**

6. Building the standing program

6.1 Who owns this, and on what cadence

[SENIOR MANAGER]

None of Sections 2 through 5 works as a one-time project. Own three things on a fixed, recurring cadence: the tenure-at-departure histogram from §1.1, re-pulled twice a year; the coded exit-interview theme report from §3.3, run quarterly; and a pay-band benchmark check against any competitor known to be actively hiring locally, reviewed at minimum once a year and immediately after any confirmed poaching-pattern cluster. The SOC manager is accountable for the pay-band and ladder-design decisions this data feeds; an HR partner co-owns running the exit and stay interviews themselves, for the same reason Part 14 keeps knowledge-capture audits out of HR's sole hands — the manager has to see the raw signal directly, not only a summarized version of it filtered through a partner who wasn't in the room for the day-to-day pattern.

6.2 Tying the program to budget season

[EXECUTIVE]

The quarterly report from §3.3 is only as useful as the budget cycle it feeds. A pay-band correction identified in a Q2 report and not proposed until the next annual budget cycle in Q4 gives a poaching wave two more quarters to run — Part 20 — Building & Defending the SOC Budget owns how to build and time that business case, but this part's job is making sure the case exists, with real numbers attached, before the budget conversation happens rather than being improvised in it.

What Would Change My Mind This part treats pay-band correction as the lever that actually closes a poaching-driven gap, and treats culture initiatives and repeated counter-offers as levers that don't. If a SOC facing a confirmed, sustained poaching pattern held its attrition rate flat for a year using only non-monetary levers — a stronger dual-ladder path, materially improved scheduling, and a genuine culture investment, with no pay-band change at all — while a comparable local competitor's premium stayed in the 15%-to-20% range this part assumes, that would undermine the claim that pay is the load-bearing variable in this specific pattern, and the confidence rating on §4.1 and §5.3's argument should drop from "reliable for a confirmed pay-gap pattern" to "context-dependent — verify the actual gap before assuming money is the fix."

Cross-references: Within this book, this part assumes Part 5 — Headcount & Capacity Modeling (backfill-lag and shrinkage arithmetic used in §5.2), Part 6 — Shift Pattern & Coverage Design (the shift-burden pattern and its fix in §4.3), Part 7 — Hiring & Sourcing Analysts (channel cost data used in §5.1–5.2), Part 13 — Career Ladders & Promotion Criteria (the dual-ladder fix in §4.2 and the counter-offer/leveling-drift parallel in §4.4), Part 14 — Mentorship & Knowledge Transfer (the specialist-departure cost figure cited in §5.2, [CASE-1401](#)), and Part 17 — Burnout, Fatigue & Wellbeing (the mechanism behind §1.1's tenure curve). It points forward to Part 16 — Performance Management & Coaching (the performance-managed-out row this part excludes), Part 19 — Team Culture & Psychological Safety (the pattern culture initiatives actually address), Part 20 — Building & Defending the SOC Budget (defending §5's cost figures and §6.2's business case), and Part 3 — Tiering Models (the disproportionate cost of losing a Tier 3 or lead-level analyst, scaling §5.2's floor). Outside this book, it cites SOC Playbook Handbook, Part 32 — Metrics (the handle-time, QA-score, and escalation-rate

definitions §2.1's leading indicators read) and Detection Engineering Handbook V2, Part 43 — Detection Debt (the technical-debt cost of an undocumented departure, alongside this part's dollar figure in §5.2).

neshbov

Part 19 — Team Culture & Psychological Safety

Why this part exists

[CONCEPT] A blameless postmortem policy that exists only as a paragraph in the team wiki changes nothing about what actually happens the next time an analyst realizes, alone, that they mis-triaged something several hours ago. What changes that moment is whether the analyst believes — based on what they've actually watched happen to someone else, not what a policy document promises — that saying so out loud costs them less than staying quiet does. That belief is psychological safety, and it is the precondition this part is named for: without it, a blameless postmortem program is a room full of people describing what they're comfortable admitting rather than what actually happened, and a near-miss reporting channel is a channel nobody uses. This part is about building and protecting that precondition, not about running the postmortem program itself.

[CONCEPT] Four things sit inside this part's scope, and each has a hard edge where another part or book picks up the mechanics. Blameless postmortem culture is the trust layer underneath honest incident review, with Part 29 — Post-Incident Organizational Review owning the actual program — cadence, participants, corrective-action tracking — that this culture has to make honest. Psychological safety is the specific mechanism that gets an analyst to report a near miss or a self-caught mistake early rather than late or never. Diversity and inclusion in hiring and scheduling sits downstream of Part 7 — Hiring & Sourcing Analysts and Part 8 — Interviewing & Technical Assessment Design, which already own the sourcing-channel and debiased-assessment mechanics that get a diverse candidate into the pipeline; this part owns what happens to that pipeline afterward — whether it survives conversion into a diverse, retained team, and whether the operational schedule built in Part 6 — Shift Pattern & Coverage Design quietly filters people back out after they're hired. And sustaining all of the above across a distributed or fully remote team is genuinely new ground: nothing else in this book addresses what happens to culture when the ambient, incidental contact of a shared physical space disappears.

[CONCEPT] One thing this part deliberately refuses to do: treat culture as a single soft, unmeasurable good feeling. Every section below states a specific, observable behavior culture is supposed to produce — early self-reporting, a diverse team that stays past its first year, a remote analyst who actually knows the team's real norms rather than its written ones — and a way to check whether that behavior is happening. A culture program that can't say what it's supposed to make people do differently isn't a program a manager can defend, staff, or fix.

1. Psychological safety: the mechanism behind honest self-reporting

1.1 What psychological safety actually means for a SOC

[CONCEPT] Psychological safety, in the sense this part uses it — the construct most associated with Harvard researcher Amy Edmondson's decades of team-effectiveness research — is a specific, testable belief held by a team member: that speaking up with a mistake, a question, a concern, or a piece of bad news will not get them punished, humiliated, or quietly held against them later. It is not the same thing as a comfortable team, a friendly team, or a team with no conflict. A team can be psychologically safe

and still argue hard about a disposition call, and a team can be pleasant and low-conflict while every analyst on it privately edits what they're willing to say in a postmortem. The test is behavioral, not emotional: does the belief actually change what someone says, and when they say it.

[FRONTLINE MANAGER] The sharpest version of the test for a SOC specifically: an analyst realizes, alone on an overnight shift, that a disposition they closed several hours earlier as a false positive was wrong — a fresher alert now shows the pattern they dismissed on the earlier ticket. Do they reopen it and message the on-call lead right now, admitting the earlier call was wrong? Or do they wait for the shift handoff, hoping the next shift catches it independently so the mistake never has to be attributed to them by name? Every other mechanism in this part is, in one way or another, about moving that specific decision toward the first branch.

1.2 Why this is an operating requirement, not a soft skill

[SENIOR MANAGER] The gap between "reported in minutes" and "reported in days" is not a comfort question, it's a dwell-time question, and dwell time is one of the most expensive variables in an active intrusion. A self-caught mistake that gets reported the moment it's noticed can usually be corrected inside the same shift. A self-caught mistake that gets sat on for fear of the reaction gets reported — if it gets reported at all — on whatever schedule the analyst's own courage and guilt resolve on, which has nothing to do with how fast the underlying exposure is getting worse.

CASE-1901 — the six-day gap. COMPOSITE CASE EXAMPLE — *merges patterns from several mid-size in-house SOC's post-incident reviews where a delayed self-report was identified as a contributing factor; no single organization is identifiable, and all figures are illustrative.*

A 12-analyst SOC closes a lateral-movement alert as a benign administrative script six hours after it fires, during a busy overnight shift already running short-staffed. Two days later, the same analyst notices a structurally similar alert against a different host and recognizes, this time, that the pattern matches a known post-exploitation technique rather than routine administration. They do not reopen the original ticket or tell anyone. Six months earlier, a different analyst on the same team had self-reported a comparable disposition error in a team channel and had it referenced, by name, in their next quarterly review as an example of "attention to detail concerns" — a fact the whole team knew about within a week, whether or not it was ever formally confirmed.

CONCEPTUAL SAMPLE -- illustrative figures, not sourced benchmark or audited incident data

Time from mistake to self-recognition:	~6 hours (unavoidable -- the pattern only became recognizable once a second, related alert fired)
Time from self-recognition to disclosure:	6 days
Estimated attacker dwell time added by the gap between recognition and disclosure:	~5.5 days
Additional hosts reached during the extended dwell window:	3 (versus an estimated 0-1 had containment started within hours of recognition)
Estimated incremental containment and forensic cost attributable to the extended dwell window:	\$38,000-\$55,000

[SENIOR MANAGER]

Nothing about the six-day gap required a weak analyst or a weak detection. The team's own detection worked — it eventually produced the second alert that triggered recognition. What failed was the few minutes it should have taken to send one message saying "I think I got this one wrong," weighed against six days of an analyst doing the arithmetic on what happened to a colleague six months earlier and concluding that silence was the safer bet.

Blind Spot A QA program built on ticket sampling (Part 15 — Quality Assurance Programs) can score the original disposition on the ticket it reviews, but it has no way to see the six days CASE-1901 lost between recognition and disclosure — that gap never generated a ticket, a QA sample, or an escalation for anyone to review until the analyst finally spoke up. Whatever fraction of your "average time to correct a known-bad disposition" number looks fast is, by construction, measuring only the mistakes that got reported promptly. The slow ones stay invisible to that same metric until something else — an audit, a second incident, an exit interview — surfaces them independently, usually much later and at much higher cost.

1.3 Measuring whether you actually have it

[HR/PEOPLE]

Most managers assume they know whether their team has psychological safety, and most are wrong in a specific direction: a manager who feels comfortable being challenged in a 1:1 routinely overestimates how safe the newest, most junior, or most different-from-the-manager team member feels doing the same thing. Assumption is not measurement. A short, anonymous survey — five to seven items, run separately from the standing engagement survey so it isn't buried inside dozens of other questions about pay and benefits — gives an actual baseline. The table below states five representative item types and what a low score on each specifically indicates, so a low aggregate score turns into a specific fix rather than a vague "improve culture" action item.

Item type (paraphrased)	Low score indicates	Where the fix lives
"If I make a mistake on this team, it will be held against me."	Fear-driven disposition inflation and slow self-reporting	\$1.2, \$2.2
"It is easy to ask another analyst or my lead for help on this team."	Analysts working ambiguous tickets alone rather than escalating	\$2.3
"People on this team are willing to raise problems and tough issues."	Near-misses and process gaps are noticed but not voiced	\$3
"I can bring up a mistake I made without being embarrassed in front of the team."	A public-shaming pattern in stand-ups or postmortems	\$2.2, \$2.3
"I feel as connected to this team's actual culture as someone sitting in an office would."	A distributed or remote culture gap invisible to an in-office manager	\$5

[FRONTLINE MANAGER]

Run it at the team level, not the org level, and report results back to the team that took it within a couple of weeks — a survey whose results disappear into a director's slide deck and never come back to the people who answered it teaches exactly the same lesson as a blameless-postmortem policy that turns out not to be blameless: the mechanism is theater.

Manager's Note *Don't run this survey for the first time right after a bad incident or a round of layoffs — you'll get a real number, but it'll be depressed by an acute event rather than your team's actual baseline, and comparing next quarter's score against it will look like a big improvement that's really just the acute stress wearing off. Get a baseline during a calm period if you can, and note the date of any major disruption next to every reading you take afterward.*

2. Blameless postmortem culture as precondition, not program

2.1 The precondition/program split

[CONCEPT] Part 29 — Post-Incident Organizational Review owns the blameless postmortem program as a designed artifact: who's in the room, how often it runs, how a corrective action gets tracked so the same finding doesn't recur across incidents. None of that program design determines whether the people in the room actually tell the truth. A postmortem run with a perfect cadence, the right participants, and a rigorous corrective-action tracker still produces a sanitized, defensive account of what happened if the room isn't safe — the mechanics can be flawless and the input can still be fiction. This part owns that input quality: the manager behaviors and standing norms that determine whether a postmortem's account of what happened and why is the real one.

[CONCEPT] The distinction matters because the two failure modes look identical from the outside and need opposite fixes. A postmortem program that produces thin, generic findings might have a design problem — Part 29's job to fix, by changing who attends or how corrective actions get tracked — or it might have a trust problem, where the room is designed correctly but nobody in it is willing to say what actually happened. Fixing the wrong one wastes a redesign cycle on a program that was never structurally broken.

2.2 Blameless does not mean consequence-free

[SENIOR MANAGER] "Blameless" gets misread constantly as "nothing bad ever happens to anyone, no matter what." That's not the claim. The claim is narrower and more precise: an honest, good-faith mistake, disclosed promptly, does not become the basis for a performance consequence, because punishing disclosure teaches concealment, and concealment is strictly worse for the organization than the mistake itself in almost every case. It does not mean a genuine pattern of repeated, avoidable errors is invisible to performance management, and it does not mean negligence or a deliberate policy violation gets the same treatment as an honest slip. The line runs between the disclosure and the underlying pattern: a postmortem's content should never be the trigger for a performance conversation, but an independently observed pattern — the same category of mistake recurring across multiple unrelated postmortems, spotted by whoever tracks corrective actions rather than dug out of any single blameless account — legitimately is, and Part 16 — Performance Management & Coaching owns that separate, disclosed process.

Management Autopsy — "announce blameless, track it privately anyway" (COMPOSITE CASE EXAMPLE, CASE-1903)

The decision: A SOC manager publicly rolls out a blameless-postmortem policy to the team, and privately continues logging postmortem-disclosed mistakes in each analyst's individual performance notes, reasoning that a manager needs a complete record even if it's never formally cited.

Why it seemed reasonable: It looked like a low-cost compromise — get the stated benefit of a blameless culture while still preserving a paper trail a manager might need later, and nobody had to know the trail existed unless it ever actually came up.

How it failed: It came up. An analyst saw a line referencing a specific postmortem-disclosed mistake in their annual review months later, mentioned it to a teammate, and within two weeks the whole team understood that "blameless" meant "not blamed out loud, in the room, on the day" rather than "not held against you, ever." Near-miss reports, which had briefly risen after the policy launched, fell back below their pre-policy baseline within a month — lower than before the policy existed, because now the team had direct evidence the stated policy didn't match the real one.

The fix: Draw the line in writing, and make it structural rather than a promise: postmortem content is never entered into, referenced by, or accessible to the performance-review process, full stop, enforced by keeping the two systems in genuinely separate tools with no shared owner who could cross-reference them casually. If a real pattern needs a performance conversation, it has to be built from evidence gathered independently of the postmortem record — QA scores, direct observation, ticket data — never from "I remember they mentioned this in the postmortem months ago."

2.3 The three-question test for whether "blameless" is real

[FRONTLINE MANAGER] A written policy is not evidence the policy is real. Three questions, asked honestly, tell a team lead more than the policy document ever will. Has anyone, ever, had something disclosed at a postmortem show up in a later review, a compensation conversation, or a hallway comment from someone senior — even once, even softened? Does the postmortem's written output describe a sequence of decisions and system conditions, or does it name individuals as the explanation ("Analyst X missed this") rather than describing what about the situation made the miss likely? And would a new hire, hearing about the last postmortem secondhand from a teammate rather than reading the policy, describe it as something they'd want to be involved in, or something they'd want to avoid being named in?

Manager's Note *Model the first disclosure yourself, and make it visible. The fastest way to convince a team that admitting a mistake is safe is to publicly admit one of your own — a scheduling call that backfired, a risk-acceptance decision that turned out wrong, a time you personally mis-triaged something years ago — before asking anyone junior to go first. A team watches what a manager actually does with their own mistakes far more closely than they read what a policy says should happen with everyone else's.*

3. Near-miss reporting: the leading indicator no other program can see

3.1 Why near-misses are invisible to QA and to the postmortem program

[SENIOR MANAGER] A near miss, by definition, never became an incident and often never became a ticket anyone else touched — an analyst catches their own error before it propagates, a suppression rule almost hides something real but a second, unrelated alert happens to catch it anyway, a scheduling gap almost leaves a queue uncovered but someone picks up an extra shift at the last minute. None of that generates the artifact any of this book's other quality mechanisms review: Part 15's QA program samples closed tickets, and a near miss that never became a ticket has nothing to sample; Part 29's postmortem program reviews declared incidents, and a near miss that stayed a near miss never gets declared. A near-miss reporting channel is the only mechanism in a SOC's toolkit that can see this category of information at all, and it only works if someone actually uses it.

3.2 A lightweight near-miss reporting mechanism

[FRONTLINE MANAGER] The mechanism that works is deliberately smaller and faster than a postmortem: a single-channel, low-friction log — a dedicated chat channel, a one-field form, whatever the team already checks daily — built around four fields, not a full incident write-up, because a reporting process that takes twenty minutes to fill out gets used a handful of times before people quietly stop. The table below states the four fields worth capturing and why each earns its place, rather than the longer field list a full postmortem template would use.

Field	Why it's worth capturing	What it should not require
What almost happened	The specific near-miss event, in one or two sentences	A root-cause analysis — that comes later, if at all
What caught it	The thing that actually prevented escalation (a second alert, a lucky catch, a colleague's question)	An assumption that the same catch will work next time
What it would have cost if uncaught	A rough, honest guess at the downstream impact	Precision — "probably a missed detection window on a real intrusion" is a complete answer
Reporter name	Optional, reporter's choice	Ever mandatory — see §2.2's boundary

[FRONTLINE MANAGER] The reporter-name field being optional is not a small detail — it's the field that determines whether the channel gets used by anyone below the confidence level of someone who already trusts the blameless policy completely. Early in a program's life, before trust is established,

expect most reports to arrive anonymous; a rising proportion of named reports over time is itself one of the better leading indicators that the §1.3 survey score is moving in the right direction, not just a nice-to-have.

3.3 The disclosure-to-consequence pipeline

[SENIOR MANAGER] Figure 19.1 turns §2.2's boundary — blameless for the disclosure, not consequence-free for an independently observed pattern — into an actual routing decision rather than a promise, so a team lead handling a specific report has a concrete answer for "where does this go" instead of relying on fresh judgment every time.

flowchart TD

```
A["Analyst self-catches a mistake\nor notices a near-miss"] --> B{"Did it produce real\nimpact or\ncontribute\nto a declared incident?"}
B -->|"No impact --\ncaught before\nanything happened"| C["Log to the near-miss\nchannel (S3.2) --\nreporter name optional"]
B -->|"Yes -- became or\nfed a declared incident"| D["Feeds Part 29's\nblameless\npostmortem\nprogram"]
C --> E{"Same root cause\nrecurs across 3+\nnear-miss logs?"}
D --> F["Postmortem output:\nsequence of decisions\nand conditions --\never a performance note"]
E -->|"Yes"| G["Root-cause fix routed to\nthe process/tooling owner,\nnot to any individual\nreporter"]
E -->|"No -- isolated"| H["Logged and trended;\nno further action"]
F --> I{"Independent evidence,\ngathered separately from\npostmortem content, of a\nrecurring\nindividual gap?"}
I -->|"Yes"| J["Routes to Part 16's\nperformance process --\ndisclosed, separate,\nown trigger"]
I -->|"No"| K["No individual\nconsequence"]
```

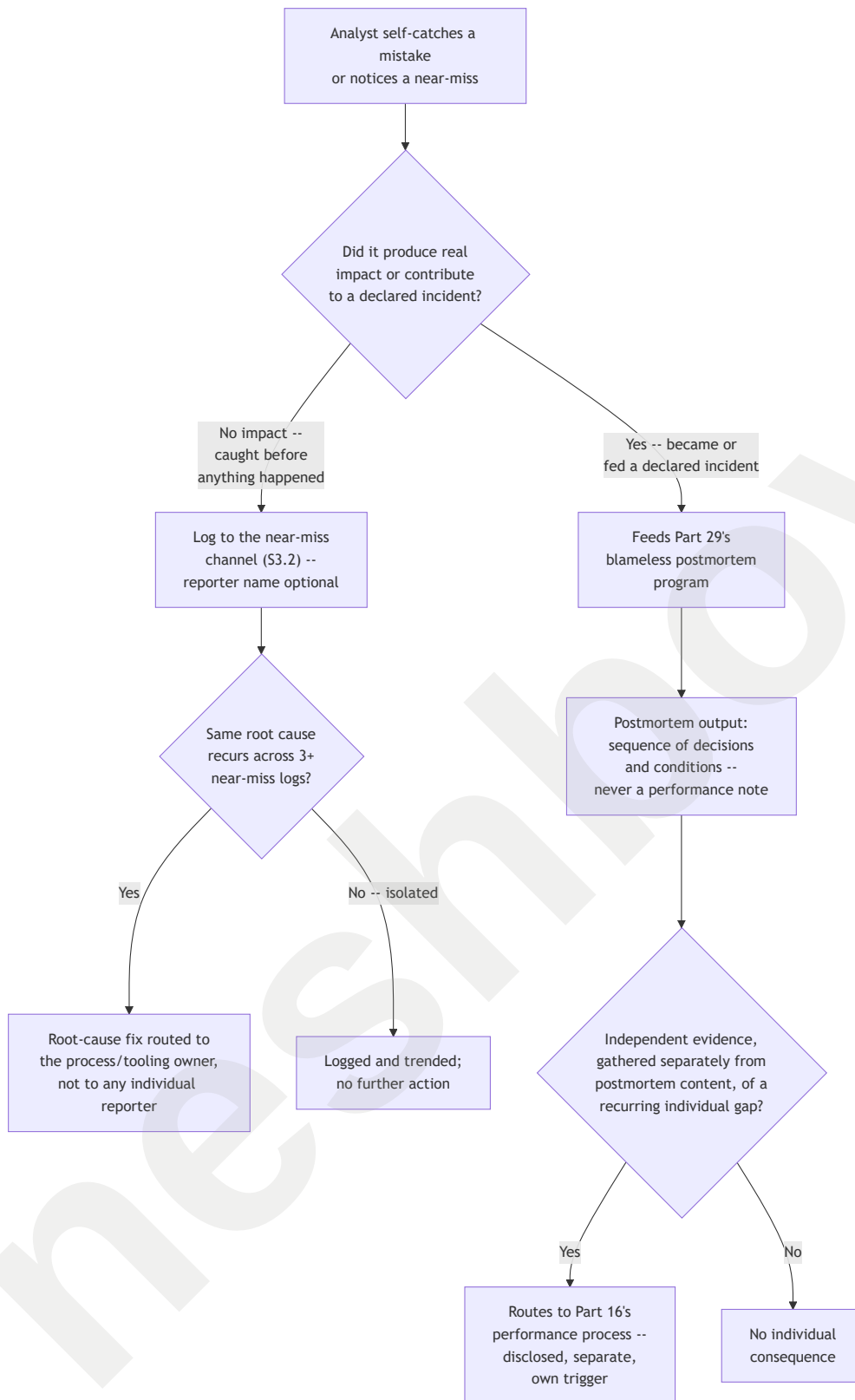


Figure 19.1 — Mistake and near-miss disclosure flow, separated from performance consequence. *CONCEPTUAL.* Illustrates how a self-caught mistake or near miss routes to either the lightweight near-miss log or Part 29's postmortem program, and how a genuine individual performance question can only enter Part 16's process through independently gathered evidence — never through the disclosure

itself. This is a structural decision aid, not a capture of any single organization's actual workflow.
Diagram ID [FIG-1901](#).

[SENIOR MANAGER] The node most managers skip in practice is the recurrence check — whether a near miss's root cause repeats — because a single near-miss report feels too small to warrant tracking over time the way a postmortem's corrective action is tracked. Route recurring near-miss root causes to whoever owns the relevant process or tool, the same way Part 29 routes a postmortem's corrective action, or the channel becomes a place where the same gap gets reported five separate times over a year with nothing ever fixed.

3.4 Worked example: near-miss reporting rate as an early-warning metric

CASE-1902 — near-miss reporting before and after a stated no-review-impact guarantee.

COMPOSITE CASE EXAMPLE — merges patterns from several SOC's that introduced a formal near-miss channel alongside explicit manager-modeled disclosure; no single organization is identifiable, and all figures are illustrative.

An 18-analyst SOC launches a near-miss channel with the fields from §3.2, an explicit written guarantee that channel content never touches performance review, and a team lead who opens the first month by publicly logging two of her own near-misses from the prior quarter.

CONCEPTUAL SAMPLE -- illustrative figures, not sourced benchmark or audited data

Near-miss reports logged, month before launch (informal count, reconstructed from hallway/chat mentions, not a real channel):	~1
Near-miss reports logged, month 1 after launch:	6
Near-miss reports logged, month 4 after launch:	14
Proportion of reports carrying a reporter name, month 1:	15%
Proportion of reports carrying a reporter name, month 4:	55%
Estimated average lag between self-recognition of a real mistake and disclosure, reconstructed from postmortems mentioning delay, before launch:	~4.2 days
Same lag, measured across postmortems in the two quarters after launch:	under 6 hours

[SENIOR MANAGER] One report in month three named a suppression exception on a rarely triggered rule that had, on inspection, been silently masking a real technique for several weeks — caught not because the rule fired again, but because an analyst updating an unrelated exception noticed the older one looked wrong and logged it as a near miss rather than quietly fixing it and moving on. The fix took under a day once flagged. Estimating what an unnoticed multi-week masking window would have cost is speculative by nature — there's no way to know what, if anything, would have exploited that gap — but this is exactly the kind of silent, undocumented suppression exception Detection Engineering Handbook V2, Part 43 — Detection Debt treats as debt accruing whether or not anyone notices it accruing, and the reporting channel is the only reason this instance surfaced before the debt came due.

4. Diversity and inclusion in hiring and scheduling

4.1 Why this is this part's territory, not Part 7's or Part 8's

[CONCEPT] Part 7 — Hiring & Sourcing Analysts already covers building sourcing channels that reach non-traditional and under-represented candidates, and why over-indexing on certifications during sourcing screens capable people out before anyone even sees a resume. Part 8 — Interviewing & Technical Assessment Design already covers de-biasing the structured-interview process itself — scorecards anchored to observable behavior, calibration gates that lock scores before debrief. Both parts do real, specific work getting a diverse candidate pool to an offer. Neither part follows that candidate past the offer to ask whether the team they're joining, and the schedule they're joining it into, actually keeps them — and that gap is exactly where a diversity effort that looks successful at the hiring-funnel stage quietly fails a year later. This part owns that gap: what happens to a fairly sourced, fairly assessed hire once they're inside a specific team's culture and a specific shift's schedule.

4.2 Closing the gap between a diverse pipeline and a diverse, retained team

[HR/PEOPLE] A hiring funnel can be diverse at every stage Part 7 and Part 8 directly control — sourced pool, interview slate, offer — and still lose that diversity at two points neither part owns: the informal "would this person fit in with the team" conversation that happens after the structured scorecard closes, and the retention curve over the candidate's first year once they're actually on the team. The first point re-introduces exactly the unstructured, gut-feel judgment Part 8's calibration gate exists to remove, just one step later in the process, where it's harder to catch. The second point is a culture question this part owns directly: a diverse hire retained for eleven months and then gone is not a hiring success by any measure that matters.

[HR/PEOPLE] The table below maps where a fairly run hiring funnel most often loses diversity after the point Part 7 and Part 8's mechanics stop controlling it, and where the actual fix lives.

Funnel stage	How diversity quietly leaks out here	Where the fix lives
Post-scorecard "team fit" conversation	An unstructured gut check re-introduces the exact bias the Part 8 calibration gate removed, one step later	Extend Part 8's calibration discipline to this conversation, or eliminate it as a separate gate entirely
Panel composition	An all-one-background interview panel signals something to a candidate before a single question is asked, regardless of how fair the questions are	Deliberately mix panel composition on every loop, not just for candidates from under-represented backgrounds
First ninety days	A new hire outside the team's existing majority demographic gets less informal mentorship by default	Part 14's deliberate mentor-matching, applied without waiting for a demographic pattern to show up in attrition data first
Year-one retention	Isolation, a lack of psychological safety specific to being the only one of a kind on the team, or a schedule that structurally excludes them	This section and §4.3, tracked against the caution in §4.4

People Risk Trap A "culture fit" question added back into the hiring process after the structured scorecard closes — even informally, even just among interviewers comparing notes over lunch — reproduces the exact unstructured bias Part 8's calibration gate was built to remove, and it's harder to catch because it happens outside any documented step. The fix: if "fit" matters, define it as specific, observable behaviors on the same scorecard everything else is scored against, evaluated by the same calibrated panel, before the loop closes — not as a separate, undocumented gut check afterward.

4.3 Scheduling as an inclusion lever, not just a coverage puzzle

[SENIOR MANAGER] Part 6 — Shift Pattern & Coverage Design owns the coverage mathematics of a shift pattern — how many people, on which rotation, covering which hours. It does not, by itself, ask whether that pattern quietly excludes categories of otherwise-qualified people before they ever apply, or burns out the people already on it faster than others on the same team. Three patterns recur often enough to name directly. Rotating night shifts fall disproportionately on analysts with caregiving responsibilities, who are still more often women and single parents than not. Rigid on-call rotations with no swap flexibility push out analysts managing a chronic condition or disability that needs a predictable schedule for treatment. And holiday coverage assigned by informal volunteering defaults to whoever is least comfortable saying no — usually the newest hires and, specifically, whichever religious minority on the team doesn't observe whatever holiday the majority of the team wants off.

[SENIOR MANAGER] The table below names each scheduling lever, the specific inclusion risk it creates if left undesigned, and where the actual shift-design fix belongs.

Scheduling lever	Inclusion risk if left undesigned	Where the fix belongs
Fixed night or rotating-night assignment	Disproportionately burdens caregivers; can silently filter them out of promotion tracks that assume night-shift reps	Build swap and bid mechanisms into Part 6's pattern design, not as an exception process
Rigid on-call with no swap path	Excludes analysts needing predictable schedules for medical treatment or disability accommodation	A documented swap/accommodation path owned jointly with HR, referenced in Part 6's pattern
Informal holiday-coverage volunteering	Defaults to newest hires and religious minorities who don't observe the majority holiday	A published, tracked holiday-coverage roster, not an ask-for-volunteers thread
Shift-swap approval left to team-lead discretion	A lead's personal comfort with a request (a Friday-evening swap for Sabbath observance, a Friday-afternoon swap for Jumma prayer) becomes an inconsistent, undocumented gate	Written swap-approval criteria applied the same way regardless of the stated reason

Management Autopsy — "ask for volunteers to cover the holiday" (COMPOSITE CASE EXAMPLE, CASE-1904)

The decision: A team lead posts an open call in the team channel asking who's willing to work the December holiday block, rather than assigning it through a tracked rotation.

Why it seemed reasonable: Asking for volunteers looked more considerate than assigning coverage top-down, and in past years someone had always stepped up without anyone having to force the issue.

How it failed: The same two analysts volunteered every year — one because she didn't observe the holiday and felt obligated to raise her hand before anyone else could be asked, the other because he was newest on the team and didn't feel he could say no to an open call his more senior colleagues were visibly not answering. Neither ever formally complained, but both mentioned the pattern, independently, in exit interviews roughly a year apart, each citing it as one example of feeling like coverage decisions fell on whoever was least able to push back rather than being distributed fairly.

The fix: Replace the open call with a published, tracked rotation that spreads major-holiday coverage across the whole team over a multi-year cycle, with an explicit, equally available swap mechanism for anyone who wants to trade in either direction — including trading toward a holiday they don't personally observe, which should be just as easy to request as trading away from one. A rotation nobody has to volunteer into removes the specific dynamic that quietly taxed the same two people every year.

4.4 Measuring D&I outcomes without creating a legal exposure

[HR/PEOPLE] The honest outcome check for everything in §4.2 and §4.3 is a retention curve broken out by whatever demographic categories matter to an organization's actual D&I goals — does a hire from an under-represented background stay at the same rate as everyone else past the first year, does a religious-minority analyst's exit-interview theme ever mention scheduling. That check requires collecting and retaining demographic data tied to individual employment outcomes, which is exactly the kind of data collection that varies sharply by jurisdiction in what's legally collectible, how it must be stored, and who inside the organization is allowed to see it tied to an individual name.

[HR/PEOPLE] Run that tracking, if it's run at all, through Part 27 — Legal, HR & Compliance Interfaces's standing relationship with HR and legal before building it, not after a well-intentioned retention dashboard turns into the first exhibit in a discovery request. This part can name the outcome worth checking; it cannot tell you what's legally collectible in a specific jurisdiction, and treating a demographic retention dashboard as a pure culture-team exercise with no legal review is its own risk, distinct from anything else in this section.

[HR/PEOPLE] Where a full cohort-retention analysis isn't feasible or isn't advisable to run internally, a lighter-weight substitute still beats nothing: track whether exit-interview themes — Part 18 — Attrition

& Retention's territory — ever cluster around scheduling fairness or feeling like "the only one," without needing to tie any single theme back to a named demographic category to notice the pattern.

5. Sustaining culture across a distributed or fully remote team

5.1 Why remote SOC culture fails silently

[CONCEPT] A co-located SOC builds trust and shared norms partly through contact nobody schedules: overhearing a stressed colleague on a call and checking in unprompted, noticing a teammate's body language sour over the course of a bad shift, watching how a lead actually reacts in the ten seconds after someone admits a mistake in person rather than reading about the policy. None of that ambient signal survives a move to distributed or fully remote work by default — it has to be deliberately rebuilt through channels that don't happen automatically, and a manager who assumes the same trust will simply persist because "we still have the same policies" is the specific failure mode this section is about.

5.2 Deliberate replacements for the office's ambient signals

[FRONTLINE MANAGER] The replacement for noticing a teammate's stress in the hallway is a standing, camera-on 1:1 cadence a lead actually protects from being the first meeting cancelled when the queue gets busy — video, not just a status message, because tone and visible fatigue carry information a text update strips out entirely. The replacement for overhearing the near miss someone mentioned in passing to a deskmate is making the near-miss channel from §3.2 genuinely the easiest way to mention something in passing on a distributed team — pinned, low-friction, checked by a lead daily, not a form buried three clicks into an intranet.

[SENIOR MANAGER] The replacement for watching how a lead reacts to a disclosed mistake in real time is harder and more deliberate: a lead on a distributed team has to narrate their own reaction in writing, in the same channel the mistake was disclosed in, because there's no hallway body language for a remote analyst to read instead. "Thanks for flagging this — here's what we're doing about the process, not about you," written visibly, every time, does the same trust-building work an in-person reaction does automatically. Skipping it because "they already know the policy" leaves a remote analyst with nothing but silence to interpret, and silence gets read as disapproval far more often than it gets read as fine.

Manager's Note *On a distributed team, write the reaction you'd give in person, every time, even when it feels repetitive to type "thank you for flagging this" for the third time this month. The repetition is the point — a remote analyst has no other signal to calibrate against, and a lead who only narrates the reaction once, the first time the policy launches, has effectively let the policy go back to being just a document again for everyone who joins after that first week.*

5.3 The follow-the-sun culture-fork risk

[SENIOR MANAGER] A follow-the-sun SOC spanning two or three regions has a specific distributed-culture risk that a single-site remote team doesn't: each region's on-the-ground lead enforces the

written blameless and near-miss norms through personal judgment, and personal judgment drifts differently in different rooms even when everyone is reading the same policy document. A region whose lead models disclosure visibly develops a real blameless culture; a region several time zones away, under a different lead who's never been told to model it explicitly, can end up with a technically identical written policy and a materially less safe actual one — and because the two regions rarely see each other's day-to-day interactions directly, that gap can run for a long time before anyone notices it isn't one culture, it's two, sharing a policy document.

Blind Spot *A single, org-wide psychological-safety survey score (§1.3) can look acceptable in aggregate while hiding exactly this fork — one region scoring comfortably safe and another scoring meaningfully lower, averaged into one number that reports "healthy" to a director who never breaks the result out by region or by lead. Always segment the §1.3 survey by team and by manager, not just by the whole distributed SOC, or a follow-the-sun program's worst region hides inside its best region's score.*

6. A culture health scorecard

[SENIOR MANAGER] Every mechanism in this part has its own local check — the §1.3 survey, the §3.2 near-miss log's volume trend, the §4.2 funnel table, the §5.3 segmented survey cut. None of them alone tells a SOC manager whether culture, as a whole, is healthy or quietly eroding, and a manager checking four different places on four different cadences will eventually stop checking one of them. The table below rolls the leading indicators from this part into a single quarterly scorecard, using the same healthy-pattern-versus-warning-pattern structure Part 9's queue-health scorecard uses, so a culture check takes the same few minutes a manager already spends on operational metrics rather than becoming its own separate project.

Signal	Healthy pattern	Warning pattern	Next step if warning persists 2+ quarters
Psychological-safety survey score, segmented by team/lead	Stable or rising, no team scoring materially below the group	One team or region scoring well below the rest	Investigate that lead's actual disclosure-response behavior (§2.3), not just the policy
Near-miss reports per analyst per quarter	Steady or rising, proportion of named reports rising over time	Flat or falling volume, or reports staying anonymous long after launch	Re-run the §2.2 boundary check — something is teaching the team disclosure isn't actually safe
Self-report lag on postmortem-identified mistakes	Hours, not days	A multi-day gap between recognizable mistake and disclosure, per postmortem records	Treat as a psychological-safety finding, not just a process-speed finding
Year-one retention, diverse hires vs. team baseline	Comparable retention curves	Diverse hires leaving materially earlier than the team baseline	Run the §4.2 funnel-stage review and the §4.4 exit-interview theme check before assuming it's unrelated to culture
Distributed/remote segment of the psychological-safety survey	Comparable to co-located or single-site scores	A region or remote cohort scoring meaningfully lower	Audit whether §5.2's deliberate replacements are actually happening in that region, not assumed

7. Where this connects

[CONCEPT] None of the mechanisms in this part work as a one-time rollout. A blameless policy that isn't tested against real disclosure for a year proves nothing; a diverse hiring funnel that isn't tracked past the offer stage proves nothing; a distributed team's culture that's never been segmented by region in a survey proves nothing. Treat every scorecard row in §6 as a standing quarterly check, not a project with an end date, and revisit §2.2's boundary explicitly any time the team goes through a leadership change — a new lead inherits the written policy automatically and inherits the team's actual trust in it not at all, and the gap between those two only shows up in the survey score or the near-miss volume, not in an org chart.

Cross-references

Within this book: Part 6 — Shift Pattern & Coverage Design (the shift-pattern mechanics §4.3's inclusion levers sit on top of, without redesigning); Part 7 — Hiring & Sourcing Analysts and Part 8 — Interviewing & Technical Assessment Design (the sourcing and debiased-assessment mechanics that get a diverse candidate to an offer, which §4 picks up from there); Part 13 — Career Ladders & Promotion Criteria (the branch and promotion tracks §4.3's night-shift row can quietly gate someone out of); Part 14 — Mentorship & Knowledge Transfer (the organic-mentorship gap §4.2 sharpens for new hires outside a team's existing majority); Part 15 — Quality Assurance Programs (the ticket-sampling mechanism §3.1 explains near-misses are invisible to); Part 16 — Performance Management & Coaching (the separate, disclosed process §2.2 and Figure 19.1 route a genuine individual pattern to); Part 18 —

Attrition & Retention (the exit-interview theme check in §4.4 and the cost of the coverage gaps this part's culture failures eventually produce); Part 27 — Legal, HR & Compliance Interfaces (the legal boundaries around demographic data collection in §4.4); Part 29 — Post-Incident Organizational Review (the operational blameless-postmortem program this part's culture work is the precondition for).

SOC Playbook Handbook: No direct citation — hiring, culture, and scheduling equity have no playbook-mechanics counterpart in that volume.

Detection Engineering Handbook V2: Part 43 — Detection Debt (the undocumented suppression-exception pattern CASE-1902's near-miss report actually caught).

neshbooy

Section E — Vendor, Tooling & Budget Management

neshbooy

Part 20 — Building & Defending the SOC Budget

Why this part exists

[CONCEPT] Every SOC manager eventually stops describing the team in headcount and shift patterns and starts describing it in dollars, because that's the language the budget owner speaks and the language a renewal or a cut actually gets decided in. This part is about building that dollar description well enough to survive the room it gets defended in: a category structure that doesn't quietly bury vendor spend inside "tooling" or drop facilities off the page entirely, two summary numbers — cost-per-analyst and cost-per-alert — a CFO can hold onto without a security background, and a full build-vs-buy-vs-outsource business case that turns Part 2's operating-model decision into a defensible multi-year number instead of one year's sticker price.

[CONCEPT] This part does not run a SIEM/EDR/SOAR selection process, score a proof-of-concept bake-off, or work through the vendor-lock-in and exit-cost analysis a platform migration specifically carries — that's Part 21 — Tooling Procurement & Platform Strategy. It does not write the service-level agreement, right-to-audit clause, or data-ownership terms that make an MSSP or MDR contract enforceable — that's Part 22. It does not cover ongoing vendor governance, renewal negotiation, or the political work of killing a redundant tool once three of them do the same job — that's Part 23. It does not turn this part's numbers into a board-ready risk narrative — Part 24 owns that translation, and this part's job stops at building the number worth translating. It does not compare your numbers against a published industry survey — Part 31, and for good reason given how easily that comparison goes wrong. And it does not define handle time, alert volume, or any other metric this part's cost-per-alert framing depends on — SOC Playbook Handbook, Part 32 — Metrics owns those definitions, used here strictly as fixed inputs.

[CONCEPT] This part assumes two things already exist: a leading candidate for Part 2's build/buy/blend operating-model decision, and a headcount number from Part 5's capacity model, including its shrinkage assumption. Everything below turns those two inputs into a budget, and then into a business case for defending — or changing — the model that produced them.

1. Five categories, and the one that swallows the other four if you let it

[CONCEPT] A SOC budget presented as a single number, or split only into "people" and "tools," doesn't survive the first serious question a finance reviewer asks: what's driving the increase this year. A budget worth defending separates five categories, each with its own growth driver and its own failure mode when it's skipped.

[SENIOR MANAGER] Headcount is salary, benefits, and payroll overhead for every tier, team lead, and manager on the roster — the number Part 5's capacity model exists to produce, run through whatever loaded-cost multiplier your finance team applies to base salary. Tooling is SIEM, EDR, SOAR, and threat-intelligence-platform licensing — the recurring software cost that scales with data volume and seat count, and the category Part 21 owns selecting and negotiating. Training is the continuing-education line Part 12 covers — certifications, conference attendance, and the internal knowledge-sharing time

that has a real opportunity cost even when it has no line-item price. Facilities and vendor services are the two categories most likely to go missing from a first-draft budget, and both deserve their own paragraph below because "missing" is exactly how they fail.

[VENDOR/PROCUREMENT] Facilities is the physical cost of running a SOC that has to be somewhere: a secure floor with badge-controlled access if the team works on-site, redundant power and network circuits so a single ISP outage doesn't blind a 24/7 desk, and — for a SOC handling classified or highly regulated telemetry — access controls beyond what a normal open-plan office needs. A remote-first SOC doesn't eliminate this category; it changes its shape into home-office stipends, hardened endpoint and VPN provisioning for remote analysts, and a smaller shared space for in-person incident bridges. Either way, treating facilities as "already covered by general corporate real estate" is how the category goes missing — a 24/7 desk's redundant-connectivity requirement is a SOC-specific cost, not a general office cost, even when it rides on the same building's lease.

[VENDOR/PROCUREMENT] Vendor services is everything that isn't a software license but also isn't payroll: an MSSP or MDR contract if Part 2's decision involves one, a standing incident-response retainer that gets activated during a major incident (Part 28 covers when a manager actually pulls that trigger), threat-intelligence feed subscriptions, and penetration-testing or red-team engagements. The distinction from tooling matters because a vendor-services contract is a relationship with an ongoing service obligation and a renewal negotiation attached to it — Part 23's territory — where a tooling license is a static grant of software rights. Collapsing the two into one "vendor spend" line makes it impossible to answer a finance reviewer's next question: is this growing because we're buying more software, or because we're buying more service hours.

The table below maps the five categories to what commonly drives their growth and what happens when the category is skipped or folded into another one — use it as the checklist for reviewing a draft budget before it goes anywhere near a stakeholder.

Category	What It Covers	Typical Growth Driver	What Happens If Skipped/Folded
Headcount	Salary, benefits, payroll overhead, all tiers and leads	Alert volume, coverage-hours target (Part 5)	Understates true cost of scaling the team; hides shrinkage and ramp lag
Tooling	SIEM, EDR, SOAR, TIP licensing	Data volume, seat count, feature-tier upgrades	Vendor renewal surprises finance as a step-function jump instead of a forecast
Training	Certifications, conferences, internal knowledge-sharing time	Headcount growth, skill-gap closure plans (Part 12)	First line cut under pressure; shows up later as retention cost (Part 18)
Facilities	Secure floor, redundant power/network, remote-work provisioning	Coverage-hours model, on-site vs. remote mix	Folded into general corporate real estate; can't be defended as a SOC-specific line when it grows
Vendor services	MSSP/MDR contract, IR retainer, threat-intel feeds, pentest	Operating-model choice (Part 2), contract renewal cycle	Folded into "tooling"; makes a service-cost increase look like a software-cost increase

[SENIOR MANAGER] Which category dominates the total is not a fixed property of running a SOC — it's a direct, mechanical consequence of Part 2's operating-model decision. A fully in-house model is headcount-heavy by construction; a fully outsourced model shifts most of that same spend into vendor

services instead, with headcount shrinking to a single relationship-owner line. Neither shape is wrong on its own. §5 of this part builds the full business case around exactly that shift, but the category table above has to exist first, because a comparison across operating models that doesn't track the same five categories consistently for every model isn't a real comparison — it's whichever category each side happened to remember to include.

2. From Part 2's payroll-and-licensing number to a true annual budget

[SENIOR MANAGER] Part 2 §2.1 priced three operating models for a composite 4,000-endpoint organization, and said so explicitly: "the three totals above only compare payroll and licensing." That was a deliberate simplification for a part about choosing a model, not defending a budget. This part's first job is completing that number using the five-category structure above, for all four models Part 2 covers — because a business case built on an admittedly incomplete number is a business case with a hole in it that a sharp finance reviewer will eventually find.

[SENIOR MANAGER] Extending Part 2's fully in-house figure (\$1,045,000 for eleven analysts, \$260,000 for two team leads, \$160,000 for one SOC manager, \$450,000 in tooling — a \$1,915,000 payroll-and-licensing total) with the three remaining categories:

CONCEPTUAL SAMPLE — illustrative numbers extending Part 2 §2.1's composite organization, not sourced benchmark data.

Fully in-house, completing Part 2's payroll-and-licensing figure:

Payroll + tooling (Part 2 §2.1)	=	\$1,915,000
Training (\$3,000/analyst x 11, \$2,500/lead or manager x 3 for leadership development)	=	\$40,500
Facilities (secure floor share, redundant network circuit, badge/access system)	=	\$60,000
Vendor services (threat-intel feed \$40,000 + standing IR retainer \$50,000, kept even with full in-house staffing)	=	\$90,000
Total	=	\$2,105,500/year

[SENIOR MANAGER] The same completion applies to the other three models, using the same category logic — a relationship-owner still needs a training budget even if it's small, an outsourced model still usually keeps an independent incident-response retainer rather than trusting the MDR vendor's own contained response to cover a genuine breach, and every model carries some facilities cost even when it's minimal.

CONCEPTUAL SAMPLE — illustrative numbers extending Part 2 §2.1, not sourced benchmark data.

Fully outsourced MDR:	\$1,094,000 (Part 2) + \$63,000 (training \$3,000, facilities \$10,000, retained IR retainer \$50,000)
	= \$1,157,000/year
Hybrid + embedded DE:	\$1,125,000 (Part 2) + \$104,000 (training \$9,000 for 3 engineers, facilities \$15,000, IR retainer \$50,000 + supplemental threat-intel feed \$30,000)
	= \$1,229,000/year

[SENIOR MANAGER]

Co-managed needs its own worked figure rather than a completion of Part 2's, because Part 2 deliberately left it as a variable: "exactly where [co-managed lands] depends entirely on which shifts stay internal — Part 20 builds the full total-cost-of-ownership model with that variable exposed." §5 below builds that model in full, including the coverage-hours arithmetic behind its headcount figure; for now, the completed co-managed total from that build-out is \$1,738,000/year, arrived at the same way as the three models above.

Operational Reality *A budget that only compares payroll and licensing across models — the exact comparison Part 2 §2.1 offers as an opening exhibit, not a finished case — routinely gets treated as the finished case anyway, because it's the number that's easiest to pull from existing quotes and offer letters. Training, facilities, and vendor services outside the primary contract get added later, if at all, usually after a finance partner asks where the incident-response retainer is hiding. Build the five-category total before the first draft goes to anyone outside the SOC, not after someone asks for it.*

3. Cost-per-analyst: the number a CFO already has a mental model for

[EXECUTIVE]

A CFO already thinks in fully loaded cost per employee for every other function in the business, so cost-per-analyst — total annual SOC budget divided by the number of analysts actually staffing a triage seat, not counting leads, managers, or engineers whose output multiplies across seats rather than being one — is the fastest number to make the SOC's economics legible in a format the rest of the org already uses.

[EXECUTIVE]

Using the completed in-house figure from §2: \$2,105,500 divided by eleven analyst seats is approximately \$191,409 per seat. That's the number worth presenting, not the \$95,000 loaded-salary line Part 2 uses for headcount modeling — the gap between the two, roughly double, is management overhead, tooling, training, facilities, and vendor services allocated across the seats that do the triage work. A CFO who only ever sees the \$95,000 figure will underestimate the true cost of adding a twelfth analyst by more than half, because every dollar of overhead this model already carries gets spread across one more seat instead of appearing as new spend.

Blind Spot *Cost-per-analyst assumes the organization is actually buying analyst seats, and that assumption breaks the moment a model stops staffing seats directly. The fully outsourced model in §2 has exactly one internal FTE — a relationship owner — and dividing \$1,157,000 by one produces a number that describes nothing real about the cost of triage; the hybrid model's three detection engineers aren't triage seats at all, and dividing its budget by three is equally meaningless. Cost-per-analyst is a genuinely useful number for comparing in-house and co-managed models against each other, or against last year's own in-house figure. It has nothing to say the moment a comparison crosses into a model that isn't staffing seats — which is exactly the comparison a build-vs-buy business case most needs a number for. §4 covers the metric that survives that crossing.*

4. Cost-per-alert: the number that survives a model change

[EXECUTIVE] Cost-per-alert — total annual budget divided by the number of alerts a human analyst actually triaged in the period, matching the same post-automation, post-deduplication definition of "volume" Part 5 §3.2 uses for headcount modeling — is comparable across every operating model, because every model produces the same denominator regardless of who's staffing the seat that clears it.

Cross-Book Pointer *This part does not define what counts as an alert reaching a human analyst, or how much of the raw signal volume automation and SOAR playbooks filter out before that point — those are SOC Playbook Handbook, Part 32 — Metrics (the definitions themselves) and Part 30 — Automation and SOAR (the decision about what gets auto-closed before a human ever sees it) territory. Every cost-per-alert figure in this section uses "alerts triaged by a human" as its denominator, exactly as Part 5 §3.2 does for headcount — confirm your own organization's automation rate before trusting a cost-per-alert comparison, because two SOCs auto-closing different shares of raw volume are not measuring the same thing even if their dashboards use the same label.*

[EXECUTIVE] Extending the composite 4,000-endpoint organization with one input Part 2 didn't need: an assumed 600 tickets a day reaching a human analyst, the same order of magnitude Part 5's own headcount worked examples use, chosen here for consistency across this book's running numbers rather than because 4,000 endpoints mechanically produces exactly that volume for any real organization. That's 219,000 alerts a year.

CONCEPTUAL SAMPLE — illustrative numbers extending Part 2 §2.1 and Part 5's volume assumption, not sourced benchmark data.

Cost-per-alert, Year 1, using each model's completed budget from Sec. 2:

Fully in-house	\$2,105,500 / 219,000 alerts	= \$9.61/alert
Fully outsourced	\$1,157,000 / 219,000 alerts	= \$5.28/alert
Hybrid + embedded DE	\$1,229,000 / 219,000 alerts	= \$5.61/alert
Co-managed	\$1,738,000 / 219,000 alerts	= \$7.94/alert

[EXECUTIVE] Unlike cost-per-analyst, all four numbers above describe the same thing: what it costs this organization to get one alert from arrival to a human disposition, regardless of whose payroll or contract that cost sits on. That portability is exactly why it's the number worth leading a CFO conversation with when the comparison spans more than one operating model — and exactly why it needs the caution below before anyone treats the lowest number as the automatic answer.

People Risk Trap *A cost-per-alert target, once a CFO has one, creates a real incentive to lower it by triaging faster and shallower rather than by getting genuinely more efficient — closing borderline dispositions without the follow-up a careful triage would have done, or discouraging analysts from escalating anything that would show up as added handle time on this quarter's number. The fix isn't refusing to report cost-per-alert; it's never reporting it without a paired quality signal from the same period — Part 15's QA sampling scores, or the escalation and re-open rate SOC Playbook Handbook, Part 27 — Escalation Quality tracks. A cost-per-alert number that improved while QA scores dropped in the same quarter isn't an efficiency win, and presenting it as one to a CFO who has no way to see the QA side is a trap this section's whole framing exists to prevent, not enable.*

What Would Change My Mind *This section treats cost-per-alert as the primary number for comparing across operating models, on the grounds that it's the one metric every model produces in comparable form. If a large share of an alert population's real cost turned out to concentrate almost entirely in a small fraction of complex, multi-hour investigations — making the blended per-alert average nearly meaningless as a predictor of what any specific model would actually cost a real organization — that would argue for a cost-per-investigation-hour metric instead, segmented by complexity, rather than a single blended cost-per-alert figure. Check your own handle-time distribution (not just its mean, per Part 5 §7.3's tyranny-of-the-average argument) before leaning on a single blended number for a decision this consequential.*

5. The build-vs-buy-vs-outsource business case: three years of dollars, not one

[SENIOR MANAGER] Part 2 makes the operating-model decision on cost, control, retention, telemetry sensitivity, and coverage-hours economics. This part's job is narrower and comes after: once a model is chosen or being reconsidered, turn it into a business case that survives more than a Year 1 comparison, because every model in §2's table carries real one-time transition cost that a steady-state number hides completely.

5.1 Building the co-managed figure Part 2 left as a variable

[SENIOR MANAGER] Co-managed's cost depends entirely on which shifts stay internal, so it needs its own worked arithmetic rather than a borrowed number. Assume the in-house side covers a twelve-hour business-hours window, seven days a week — eighty-four of the week's 168 hours — with two analysts on shift at all times, and hands the remaining eighty-four hours to a vendor overflow contract.

CONCEPTUAL SAMPLE — illustrative numbers, not sourced benchmark data.

Co-managed, business-hours in-house / overnight-and-weekend vendor overflow:

Seat-hours needed per year, two concurrent seats,

84 of the week's 168 hours: $2 \times 84 \times 52 = 8,736$

Productive hours/analyst after shrinkage (Part 2 Sec. 6) $= \sim 1,750$

FTE required: $8,736 / 1,750 = \sim 4.99$

Rounded up to 5 FTE base, + 1 buffer analyst for cross-training/surge (same buffer logic as Part 2 Sec. 6.1) $= 6 \text{ FTE}$

6 analysts $\times \$95,000 = \$570,000$

1 team lead $\times \$130,000 = \$130,000$

0.75 FTE SOC-manager allocation $\times \$160,000 = \$120,000$

Tooling (SIEM/EDR for the in-house side) $= \$350,000$

Overnight/weekend MDR overflow, $\$9/\text{endpoint/month} \times 4,000 \text{ endpoints} \times 12 \text{ months} = \$432,000$

Total $= \$1,602,000/\text{year}$

+ training $(\$3,000 \times 7 \text{ people}) \$21,000$, facilities $\$45,000$,

vendor services (IR retainer $\$50,000$ + threat-intel

supplement $\$20,000) \$70,000 = \$136,000$

True total $= \$1,738,000/\text{year}$

Operational Reality The \$9-per-endpoint overflow rate above is an illustrative assumption, not a rule — some MDR vendors price overflow-only coverage higher per endpoint than full-time coverage, precisely because unpredictable burst volume is more expensive for them to staff against than a committed 24/7 baseline. Get the overflow rate from an actual quote before building a co-managed business case around a halved version of the full-coverage price; the arithmetic above shows the method, not a number to copy.

5.2 Extending to a three-year total cost of ownership

[SENIOR MANAGER] A steady-state annual figure hides the one-time cost every model carries during its first year: hiring and onboarding for in-house seats, or vendor-onboarding friction and a real coverage-context gap for anything with a vendor in it. Adding those one-time costs to three years of the completed steady-state figures from §2 and §5.1:

CONCEPTUAL SAMPLE — illustrative numbers, not sourced benchmark data.

One-time ramp/transition cost, Year 1 only:

Fully in-house: 14 hires x \$9,000 recruiting cost = \$126,000,
+ 3-month MDR bridge contract while hiring ramps
(4,000 endpoints x \$18/mo x 3) = \$216,000
Total = \$342,000

Fully outsourced: vendor onboarding/transition fee = \$45,000
+ internal context-transfer time = \$20,000
Total = \$65,000

Hybrid: vendor onboarding fee (monitoring
scope only) = \$35,000
+ 3 detection engineers x \$9,000 = \$27,000
Total = \$62,000

Co-managed: 7 hires x \$9,000 = \$63,000
+ vendor overflow onboarding fee = \$25,000
Total = \$88,000

Three-year TCO (3 x true annual total, Sec. 2/5.1, + one-time ramp):

Fully in-house: 3 x \$2,105,500 + \$342,000 = \$6,658,500
Fully outsourced: 3 x \$1,157,000 + \$65,000 = \$3,536,000
Hybrid: 3 x \$1,229,000 + \$62,000 = \$3,749,000
Co-managed: 3 x \$1,738,000 + \$88,000 = \$5,302,000

Cost-per-alert over the same 3-year window (657,000 alerts, assuming
flat volume across the period -- a simplifying assumption, not a forecast):

Fully in-house: \$10.13/alert
Fully outsourced: \$5.38/alert
Hybrid: \$5.71/alert
Co-managed: \$8.07/alert

[SENIOR MANAGER] The ranking doesn't flip once ramp cost and a real multi-year horizon are added — fully outsourced is still cheapest, fully in-house still costs the most — but the margins move, and the in-house model's cost-per-alert rises more than any other model's once its own ramp cost is included, because a large one-time hiring cost sits entirely inside a Year 1 that a steady-state comparison never sees. A business case built on Year 1 steady-state alone understates the in-house model's true cost specifically, which matters most when in-house is the option a stakeholder already favors on control grounds and needs the full number to defend it honestly rather than optimistically.

Management Autopsy — "present the cheapest steady-state number and skip the ramp cost" (CASE-2001, COMPOSITE CASE EXAMPLE)

The decision: A SOC manager built a build-vs-buy case comparing fully in-house against a fully outsourced MDR contract using only Year 1 steady-state run cost — the same shape of comparison Part 2 §2.1 offers as an opening exhibit — and presented the outsourced model as clearly cheaper to secure budget approval for the switch.

Why it seemed reasonable: The steady-state numbers were real, pulled from an actual vendor quote and actual current payroll, and the comparison was honest as far as it went. Nobody on the finance side asked about ramp cost, because nothing in the one-page comparison prompted the question.

How it failed: The vendor's onboarding period ran three full billing cycles instead of the two the original case assumed, during which the elevated false-positive burden Part 2's own Operational Reality box describes drove real overtime cost on the reduced in-house relationship-owner team that the case never modeled. Separately, a data source the outsourcing contract's scope never anticipated — added six months later by an unrelated team — turned out to be telemetry-sensitivity-restricted, forcing an emergency partial rehire of in-house analysts to cover it outside the vendor relationship. The "cheaper" model's real first-year cost, including the emergency rehire, ended up exceeding the in-house baseline it had replaced.

The fix: Present a multi-year TCO with one-time ramp cost as the default exhibit, per §5.2, not an optional add-on requested after the fact. Treat the absence of a ramp-cost line in someone else's build-vs-buy case as a reason to ask for one before approving it, not just a reason to add one to your own.

5.3 What dollars alone still can't settle

[SENIOR MANAGER] A three-year TCO answers "which model costs less," which is a real answer but not the whole business case. Part 2's control, telemetry-sensitivity, and retention dimensions don't collapse into a dollar figure, and neither does detection quality.

Cross-Book Pointer A budget case that only compares dollars can make the cheapest model look like the right one even when it produces materially worse detection outcomes — a fully outsourced model running a generic vendor ruleset with no org-specific tuning, per Part 2 §3's CASE-0201, can win every cost comparison in this section while quietly leaving high-risk, organization-specific attack paths uncovered. This part has no way to measure that gap; Detection Engineering Handbook V2, Parts 41–43 — Detection Coverage, Quality, and Debt do. Run that assessment against whichever model wins the cost comparison before treating the cost comparison as the decision, not after.

6. Weighing the case: a decision matrix, and the process that produces one

[SENIOR MANAGER] Part 2 §7.2 offers a qualitative scorecard across the same four models; this section turns it into a weighted, numeric version — **TMPL-2002**, the build-vs-buy-vs-outsource decision matrix housed in Appendix A6 — so a dollar comparison and a set of qualitative priorities land in the same table instead of two separate conversations.

[SENIOR MANAGER] Score each model 1 through 5 (5 is best) on each dimension, using the same qualitative comparisons Part 2 §7.2 already establishes, translated to numbers:

CONCEPTUAL SAMPLE — illustrative scoring for the composite organization used throughout this part, not a universal scoring key.

Dimension	Fully in-house	Fully outsourced	Hybrid	Co-managed
3-year TCO (lower cost scores higher)	1	5	4	2
Cost-per-alert trend (Sec. 4/5.2)	1	5	4	2
Tuning-authority latency/control	5	2	4	3
Telemetry-sensitivity fit	5	1	3	4
Retention lever for senior talent	4	1	4	4
Coverage-hours cost efficiency (thin shifts)	1	5	4	4

[SENIOR MANAGER] Weight the dimensions to match what this specific organization actually cares about — a heavily cost-constrained mid-market organization with low telemetry sensitivity and no near-term retention crisis might weight TCO and cost-per-alert at 30% and 15%, control and sensitivity fit at 15% each, retention at 15%, and coverage-hours efficiency at 10%. Multiplying scores by those weights and summing (CONCEPTUAL SAMPLE — illustrative weights and resulting scores for the composite organization used throughout this part, not a universal weighting key):

Model	Weighted Score	3-Year TCO	Cost-Per-Alert (Y1)
Fully in-house	2.65	\$6,658,500	\$9.61
Fully outsourced	3.35	\$3,536,000	\$5.28
Hybrid + embedded DE	3.85	\$3,749,000	\$5.61
Co-managed	2.95	\$5,302,000	\$7.94

[SENIOR MANAGER] The weighted result is worth sitting with for a moment, because it isn't just a restatement of the cost ranking: hybrid wins the weighted comparison even though fully outsourced is cheaper on raw dollars, because hybrid's retained tuning authority and detection-engineer retention lever score high enough to overcome a modestly higher TCO. That's the entire value a weighted matrix adds over a bare cost comparison — it's the difference between "this is cheapest" and "this is the right tradeoff for what we actually value," and it's why §5.3's Cross-Book Pointer matters: the matrix still says nothing about whether hybrid's or any other model's actual detection coverage holds up, which is a separate check against a separate set of numbers this book doesn't produce.

Figure 20.1 — Process for building and defending a build-vs-buy-vs-outsource business case.

CONCEPTUAL. FIG-2001. Illustrates the sequence this part builds — completing the five-category budget, extending to a multi-year TCO, computing the two CFO-facing summary numbers, checking for a telemetry-sensitivity exclusion before scoring, and building in a re-run loop when a stakeholder challenges an assumption — rather than a one-time exhibit built once and defended unchanged.

flowchart TD

A["Complete each model's true annual\nbudget (Sec. 1-2, five categories)"] --> B["Extend to a multi-year TCO,\nincluding one-time ramp cost (Sec. 5.2)"]

B --> C["Compute cost-per-analyst (Sec. 3)\nand cost-per-alert (Sec. 4) per model"]

C --> D{"Any model excluded by telemetry\nsensitivity, Part 2 Sec. 4?"}

D -->|Yes| E["Drop excluded models\nbefore scoring"]

D -->|No| F["Score remaining models on the\nweighted decision matrix (Sec. 6)"]

E --> F

F --> G["Present the TCO comparison as the\nopening exhibit (Sec. 7)"]

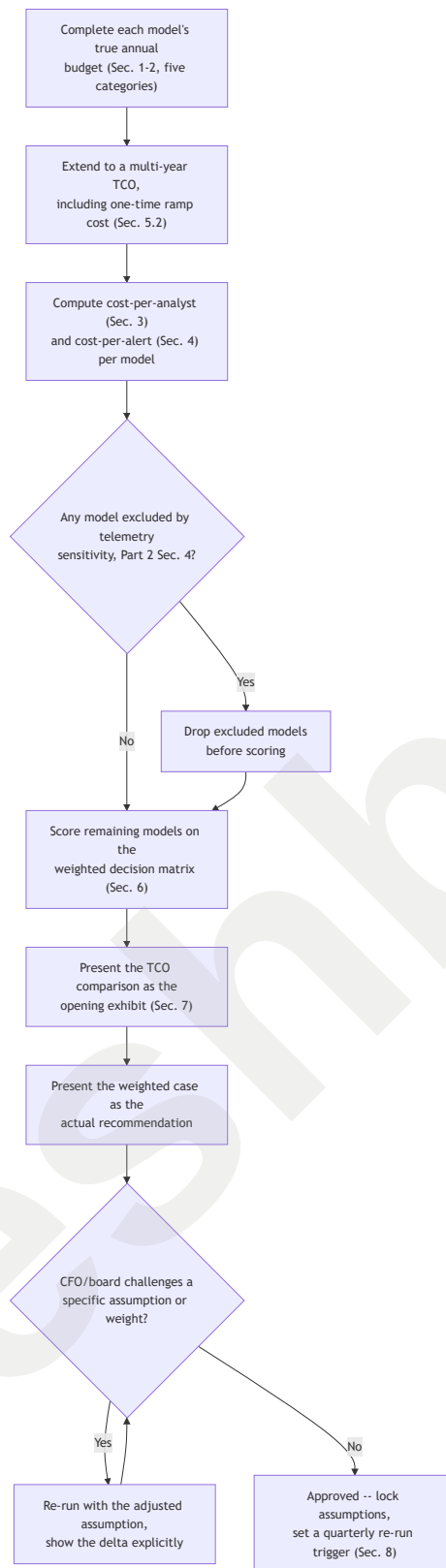
G --> H["Present the weighted case as the\nactual recommendation"]

H --> I{"CFO/board challenges a\nspecific assumption or weight?"}

I -->|Yes| J["Re-run with the adjusted assumption,\nshow the delta explicitly"]

I -->|No| K["Approved -- lock assumptions,\nset a quarterly re-run trigger (Sec. 8)"]

J --> I



7. Presenting and defending the number

[EXECUTIVE] Lead with the multi-year TCO table, not the weighted matrix — Part 2's own framing of a bare cost comparison as "the opening exhibit, not the closing argument" applies here word for word. A CFO wants to see the dollar comparison first, on its own, before being asked to accept that a more

expensive option might still be the right call once control, sensitivity, and retention are weighed in. Reversing that order — leading with a weighted score that already bakes in judgment calls the CFO hasn't seen yet — reads as burying the ball, even when the underlying analysis is sound.

[EXECUTIVE] Once the TCO table is on the table, cost-per-alert (§4) is the number to anchor the conversation to for the rest of the discussion, because it's the one figure that stays comparable no matter which model the conversation drifts toward next. Cost-per-analyst (§3) is useful only inside a single model or against that same model's own prior year — flag its limits before a CFO tries to use it to compare in-house against outsourced, or the Blind Spot in §3 becomes a live misunderstanding in the room instead of a footnote in this chapter.

Operational Reality *A budget case that lands well in the room it was built for often gets re-forwarded, months later, to a different stakeholder who wasn't there for the caveats — a board member, a new CFO, an auditor — with the weighted matrix's scores intact and the conversation that produced the weights long gone. Document the weighting rationale in the same file as the matrix itself, not just in the meeting notes from the day it was presented, or the next reader will treat the scores as objective measurements instead of the negotiated judgment calls they actually are.*

[EXECUTIVE] This part builds the number; it does not build the narrative around it. Part 24 — Executive & Board Reporting owns translating a budget defense like this one into the risk-framed language a board-level audience needs, including how to survive a skeptical follow-up question the TCO table alone doesn't answer. Bring this part's numbers there — don't rebuild the narrative logic from scratch once the audience changes from a CFO to a board.

8. Keeping the budget honest after it's approved

[SENIOR MANAGER] An approved budget is a snapshot of the assumptions that were true when it was built, not a standing guarantee that those assumptions stay true. Alert volume shifts the moment a new detection ships or an acquisition adds a log source, vendor per-endpoint pricing changes at renewal, and the shrinkage figure behind the headcount line drifts the same way Part 5 §8 describes for the capacity model itself. Review the five-category total and both summary numbers on a fixed cadence — quarterly is a reasonable default, matching Part 5's own recommendation for the headcount model this budget is built on — rather than only at the next annual budget cycle.

[SENIOR MANAGER] The specific trigger worth watching for is a cost-per-alert or cost-per-analyst figure that moves more than roughly 10% to 15% between review cycles in either direction — the same order of magnitude as (if not identical to) the flat 10% headcount swing Part 5 §8 already treats as worth escalating to the next board or CISO reporting cycle rather than holding until the next annual conversation — and for good reason: a budget number that's drifted that far from what was approved is no longer describing the model the CFO signed off on, whether the drift is good news (automation cut cost-per-alert faster than expected) or bad news (a vendor renewal came in well above the prior contract).

Cross-references. This book: Part 2 — SOC Operating Models (the build/buy/blend decision this part's business case defends, and the source of every dollar figure extended in §2 and §5); Part 5 — Headcount & Capacity Modeling (the shrinkage assumption and volume framing behind §3–§4's cost figures); Part 9 — Queue Health & Workload Management (the staffing-vs-detection-quality diagnosis a rising cost-per-alert should route through before assuming it's a budget problem); Part 12 — Ongoing Training & Skill Development (the training category in §1–§2); Part 15 — Quality Assurance Programs (the quality signal that has to accompany any cost-per-alert target, §4); Part 18 — Attrition & Retention (the retention-cost side of the weighted matrix in §6); Part 21 — Tooling Procurement & Platform Strategy, Part 22 — MSSP & Managed-Service Contract Management, and Part 23 — Vendor Relationship & Renewal Management (the tooling and vendor-services mechanics this part deliberately doesn't re-derive); Part 24 — Executive & Board Reporting (turning this part's numbers into a board-level narrative); Part 28 — The Manager's Role in a Major Incident (activating the IR retainer priced in §1–§2); Part 31 — Benchmarking & Industry Comparison (the caution against comparing this part's numbers to an external survey uncritically); Appendix A6 — Budget & Business-Case Templates ([TMPL-2001](#) budget worksheet, [TMPL-2002](#) decision matrix, [TMPL-2003](#) TCO calculator).

SOC Playbook Handbook: Part 27 — Escalation Quality (the re-open/escalation signal that should accompany a cost-per-alert figure, §4); Part 30 — Automation and SOAR (what counts as alert volume before it reaches a human, the denominator for §4's metric); Part 32 — Metrics (the handle-time and volume definitions used as fixed inputs throughout).

Detection Engineering Handbook V2: Parts 41–43 — Detection Coverage, Quality, and Detection Debt (the check this part's cost comparison in §5–§6 cannot substitute for — the cheapest model on a TCO table is not automatically the right one if it produces materially worse detection coverage).

Part 21 — Tooling Procurement & Platform Strategy

Why this part exists

[CONCEPT] Signing a SIEM, EDR, SOAR, or TIP contract is a decision made once, over a few weeks, by a handful of people in a room with a vendor's slide deck. Living with that decision is a multi-year condition that touches nearly everything else in this book: the headcount model in Part 5 assumes a stable handle-time distribution that a platform swap resets to zero while analysts relearn a new console; the QA sampling in Part 15 assumes a ticket structure that doesn't change out from under it; the coaching cadence in Part 16 assumes the tool an analyst is being coached on this month is the same tool next month. Most organizations run the selection process that produces that decision as a feature bake-off — a scored checklist of capabilities, a proof-of-concept, a reference call or two — and feature bake-offs are structurally good at answering "does this product do the thing" and structurally blind to the three questions that determine whether the decision was actually a good one: what does it cost to get our existing detection content and our analysts onto this platform, what does it cost to get back off it if we're wrong, and how does the vendor's own pricing model behave once we're no longer the size we were during the demo.

This part owns the manager's side of that selection process — scoping the requirement, structuring RFP and PoC evaluation criteria, running reference-customer calls that get past a vendor's curated list, building a total-cost-of-ownership model that includes migration and retraining rather than license price alone, and pricing vendor lock-in and exit cost as a line item in the decision rather than a surprise at the next renewal. It does not walk through how to structure the resulting contract — SLA definitions, right-to-audit clauses, data-ownership and offboarding terms are Part 22 — MSSP & Managed-Service Contract Management's territory, and this part cites that part rather than re-deriving contract mechanics. It also does not re-teach what a SIEM migration actually costs in engineering effort, field by field — that mechanical cost model belongs to Detection Engineering Handbook V2, Part 6 — Normalisation, and this part borrows its cost structure directly rather than inventing a competing one. Nor does it cover the ongoing vendor governance and renewal negotiation that happens after signing — that's Part 23 — Vendor Relationship & Renewal Management, which uses this part's exit-cost model as a standing input rather than building its own.

1. What a tooling decision actually is: a platform bet, not a purchase

[CONCEPT] A feature bake-off scores a vendor's product against a checklist of capabilities, usually two hundred rows deep, checked off against a scripted demo running on the vendor's own curated data. Mature platform vendors in the SIEM, EDR, SOAR, and TIP markets converge on broadly similar checklist coverage — correlation rules, dashboards, case management, some flavor of automation, some flavor of threat-intel enrichment — which means the checklist increasingly measures polish and breadth rather than a decisive functional gap between finalists. What the checklist has no column for at all is what happens after the contract is signed: the one-time cost of moving three hundred existing detections and years of tuning knowledge onto the new platform, the cost of an analyst team that is measurably slower for roughly six to ten weeks while it relearns query syntax and dashboard muscle memory, and

the cost of leaving, three or five years from now, once the organization has built enough proprietary content on top of the winning platform that leaving is no longer a real option.

[SENIOR MANAGER] The table below states plainly what a standard bake-off measures against what it structurally cannot see, because the two failure modes this part spends most of its length on — an underpriced total cost of ownership and an unpriced exit cost — both live in the right-hand column.

A feature bake-off measures	What it structurally can't see
Whether the product does X today, in a scripted demo	Whether it does X against your actual log volume, your actual field-naming mess, and your analysts' actual query habits
The vendor-quoted list price for the tier proposed during the sale	How that pricing model behaves once ingestion volume, endpoint count, or automation-run count grows past what the demo scoped
A curated reference customer's stated satisfaction	The dollar and month cost this organization will pay to migrate content, retrain analysts, and re-earn the coverage lost in translation
Today's stated exit terms, as described in the sales deck	The actual cost of leaving three years from now, once content and workflow are already built on top of the platform

2. Running the selection process

2.1 Scoping the requirement before the RFP goes out

[VENDOR/PROCUREMENT] An RFP written before anyone inventories the current platform's content produces a bake-off that structurally favors whichever finalist talks least about migration, because with no inventory, every finalist's migration-cost estimate defaults to zero by omission rather than by analysis. Before a requirement goes to any vendor, a manager needs three numbers already in hand: the count of standing detection rules, dashboards, and SOAR playbooks currently in production; the log sources that must ingest correctly on day one, ranked by how much of the SOC's real alert volume each one drives rather than by how recently it was onboarded; and the mandatory integration surface — ticketing, identity provider, case management, whatever downstream system a triage workflow already depends on. That inventory is the direct input to the total-cost-of-ownership model in §3 and the exit-cost model in §4; skipping it doesn't remove the cost, it just moves the discovery of that cost to a much worse moment, during the actual migration.

2.2 Structuring RFP evaluation criteria: the weighted scorecard

[VENDOR/PROCUREMENT] A weighted scorecard is the direct antidote to a checklist that rewards breadth: instead of summing yes/no answers across two hundred rows, a small number of categories get a weight that reflects how much each one actually determines whether this decision holds up in three years, and every finalist gets scored against the same categories using evidence gathered from the PoC and reference calls in §2.3–§2.4 rather than from the vendor's own pitch.

[VENDOR/PROCUREMENT] A SOC manager or the designated procurement lead fills the scorecard below in once per finalist, after the PoC and reference calls are both complete, so every row is scored against evidence the buying organization gathered itself rather than a number the vendor supplied.

TEMPLATE — Platform RFP/PoC weighted evaluation scorecard, permanent ID TMPL-2101

CRITERION	WEIGHT	SCORE (1-5)	WEIGHTED
Functional fit against the use cases named in §2.1 <!-- score only against the use cases the RFP actually named, not the vendor's full feature list -->	25%	—	—
Data ingest/normalization fit, own messiest source <!-- score from the PoC's worst-case log source (Sec. 2.3), never from the demo's cleanest one -->	15%	—	—
Integration surface (SOAR, ticketing, IdP, EDR)	10%	—	—
Three-year TCO: license + migration + retraining <!-- pull the number from the TCO worksheet in Sec. 3.2, never the vendor's list price alone -->	25%	—	—
Exit-cost / lock-in exposure <!-- pull the number from the exit-cost model in Sec. 4.2 -->	15%	—	—
Vendor viability & support model	5%	—	—
Reference-customer signal, incl. one off-list call	5%	—	—

TOTAL	100%		—

[VENDOR/PROCUREMENT] This scorecard fixes a candidate's fit and cost into one score; it does not fix the weights themselves, which a manager should adjust for a genuinely different risk profile — a heavily regulated environment might push exit-cost exposure higher still, and a SOC with no realistic near-term migration risk might legitimately push functional fit back up toward where a standard bake-off would put it.

What Would Change My Mind This scorecard weights three-year TCO and exit-cost exposure at 40% combined, above functional fit's 25%, on the claim that a standard bake-off structurally under-weights both. If a large sample of real procurement outcomes showed that platforms chosen mainly on functional fit rarely produced regret on a three-year horizon — not because TCO and exit cost didn't matter, but because they tracked functional fit closely enough that scoring them separately added no decision-useful information — that would argue for collapsing this scorecard back toward something closer to a standard bake-off after all.

2.3 Designing a PoC that tests your own data, not the vendor's demo

[VENDOR/PROCUREMENT] A proof-of-concept run on a vendor-curated data sample, on a timeline short enough that only the platform's cleanest ingest paths get exercised, tests the vendor's demo environment rather than the buying organization's actual conditions. The fix is concrete: name the single messiest log source in scope — usually an old on-prem application, a homegrown tool, or whichever source has the worst field-naming — and require it in the PoC's test plan by name, not as an optional stretch goal; require the buying organization's own analysts to drive at least half of the PoC sessions from a normal analyst seat, running real queries against real historical tickets, rather than watching the vendor's sales engineer click through a prepared walkthrough; and run the PoC long enough — a minimum of two to three weeks of real ingest, not two or three days of a scripted demo — that a field-mapping problem has time to surface at all.

COMPOSITE CASE EXAMPLE (CASE-2101) — merges patterns observed across several mid-market PoC engagements; not one identifiable organization.

Management Autopsy — "let the vendor's sales engineer run the PoC"

The decision: A finalist vendor's sales engineer configured and ran the entire proof-of-concept environment, choosing which of the buying SOC's log sources to onboard first and which sample queries to demonstrate, with the buyer's own analysts observing rather than driving.

Why it seemed reasonable: The SOC's analysts were fully committed to live queue work, the vendor's engineer could stand up a working environment in days rather than weeks, and the vendor's own product expertise seemed likely to showcase the platform's strongest capabilities rather than waste time on a clumsy self-guided setup.

How it failed: The sales engineer selected the buyer's three cleanest, best-parsed log sources and skipped a fourth — a fifteen-year-old on-prem case-management application with inconsistent field names that turned out to drive nearly a third of the SOC's highest-severity alerts. The PoC scored the platform's ingest and parsing capability as excellent because the PoC never touched the one source that would have tested it under real conditions. Six weeks after go-live, that application's logs landed in the new platform with more than half their fields unmapped, and detections built against them during the PoC's "successful" period had been silently matching against the wrong field the entire time.

The fix: Name the messiest log source before the PoC starts and require it in the test plan; have the buying organization's own analysts, not the vendor's sales engineer, drive at least half the PoC from a normal analyst seat, against tickets they already know the right answer to.

Manager's Note Ask a finalist vendor to import a real week of your own already-closed tickets — the raw logs behind five confirmed true positives and five confirmed false positives your team already has full context on — and see whether the new platform's interface gets an analyst to the same disposition in less time, not more. If a demo can't survive being pointed at data you already understand, it isn't ready to be pointed at data you don't.

2.4 Reference-customer calls: who to ask, and who not to ask

[VENDOR/PROCUREMENT] A reference list a vendor hands over during a sale is a curated sample of that vendor's most satisfied customers, selected by the same account team trying to close the deal — treating three curated conversations as representative of the vendor's typical customer experience makes the same self-selection error Part 31 — Benchmarking & Industry Comparison flags for a three-hundred-respondent industry survey, just at a much smaller and more deliberately chosen scale. The two fixes that actually move the needle: ask the vendor directly for a named reference close to the buying organization's own size, industry, and log volume who migrated off a specific, named competitor product within roughly the last two years — a request specific enough that a vendor without

a real match has to say so — and separately, before signing, spend an hour finding one reference through a peer network or professional community the vendor didn't supply, since that call is the only one guaranteed not to have been pre-briefed.

People Risk Trap *A vendor-supplied reference list is a self-selected sample of that vendor's happiest customers, curated by the team trying to close your deal. Treating it as representative is the same self-selection error a survey with an unknown, self-selected respondent pool produces, just applied to a sample of three instead of three hundred. The fix: ask for a named reference matched on size, industry, and log volume who migrated off a specific competitor product recently, and separately find one reference the vendor didn't hand you.*

A short composite example shows what an off-list call is actually worth. **COMPOSITE CASE EXAMPLE (CASE-2102)** — merges patterns from more than one mid-market procurement cycle; not one identifiable organization. A vendor's supplied reference list for an eighteen-analyst SOC's SIEM bake-off consisted of three customers, all with security teams over 150 people — a poor size match, but the only names offered. Through a peer Slack community rather than the vendor's account team, the buying manager located a former customer close to their own size who had migrated off the same product the previous year. That off-list reference reported that the vendor's quoted six-to-eight-week migration timeline for a comparable rule count had actually run to just over seven months, driven mostly by the mapping-and-parity effort Detection Engineering Handbook V2, Part 6 — Normalisation prices for exactly this kind of move, and separately flagged an egress fee for historical log export that the sales cycle had never mentioned. Because the buying organization surfaced both facts before signing rather than after, it negotiated a fixed-fee migration-support clause and a capped egress fee into the contract instead of discovering the same gap at renewal, three years later, with no leverage left.

3. Total cost of ownership: the model a bake-off never builds

3.1 The categories a bake-off skips

[SENIOR MANAGER] A vendor's quoted license price captures maybe half of what a platform actually costs over a multi-year term, and even that half grows nonlinearly as ingestion volume, endpoint count, or automation-run count crosses the tier boundaries the sales quote scoped against. The categories a standard bake-off routinely leaves at zero: the one-time engineering effort to migrate existing detection content and historical data, the retraining hours needed before analysts return to their prior handle-time baseline, the parallel-run cost of operating both platforms simultaneously during cutover, and the ongoing content-authoring FTE tail the new platform requires, which is not guaranteed to match the old platform's.

Cross-Book Pointer This part does not re-derive what a SIEM migration actually costs in engineering effort — that's *Detection Engineering Handbook V2, Part 6 — Normalisation*, which breaks real migration cost into four components (platform, mapping, parity-testing, and dual-running) and shows why the mapping and parity-testing lines, not the license line, are what actually blow a migration's budget. The worked example below pulls that component structure directly into a total-cost-of-ownership model rather than inventing a separate cost taxonomy for the same underlying work.

3.2 Worked example: three-year TCO across two SIEM finalists

[SENIOR MANAGER] The comparison below prices two finalists for a 22-analyst SOC replacing its incumbent SIEM at renewal, with 340 existing detection rules in scope. Vendor A's platform shares a schema family closely related to the incumbent's, which keeps its per-rule mapping effort low; Vendor B won the functional bake-off outright and quoted the cheaper sticker price every single year, but its schema is a genuinely different family, which raises its per-rule mapping effort under the same DEH Part 6 cost model.

CONCEPTUAL SAMPLE — illustrative numbers, not sourced benchmark data.

License, 3-year total:

Vendor A: \$410,000 (Y1) + \$430,000 (Y2) + \$450,000 (Y3) = \$1,290,000
Vendor B: \$340,000 (Y1) + \$355,000 (Y2) + \$370,000 (Y3) = \$1,065,000
License savings choosing B over A, 3-year total: = \$225,000

Migration cost (one-time, DEH Part 6 mapping + parity-testing + dual-running):

Vendor A: 340 rules x 2 hrs/rule x \$95/hr loaded eng. cost = \$64,600
+ 60-day dual-run overlap = \$35,000
Subtotal = \$99,600
Vendor B: 340 rules x 6 hrs/rule x \$95/hr loaded eng. cost = \$193,800
+ 90-day dual-run overlap = \$60,000
+ vendor migration professional-services fee = \$75,000
Subtotal = \$328,800

Retraining cost (one-time):

Vendor A: 22 analysts x 8 hrs x \$60/hr loaded cost = \$10,560
Vendor B: 22 analysts x 40 hrs x \$60/hr loaded cost = \$52,800

Temporary capacity loss during ramp (Vendor B only --
different query language, one quarter at ~20% handle-time
regression across 22 analysts, per the FTE math in Part 5): = \$23,750

One-time cost, total:

Vendor A: \$99,600 + \$10,560 = \$110,160
Vendor B: \$328,800 + \$52,800 + \$23,750 = \$405,350

3-year TCO:

Vendor A: \$1,290,000 + \$110,160 = \$1,400,160
Vendor B: \$1,065,000 + \$405,350 = \$1,470,350

[SENIOR MANAGER]

Vendor B wins the functional bake-off and quotes the cheaper license every year, saving \$225,000 in cumulative license cost over three years — and still ends up roughly \$70,000 more expensive over that same three years, because its one-time migration, retraining, and capacity-loss premium runs nearly \$295,000 higher than Vendor A's. Spread across the license savings alone, that premium takes almost four years to earn back — longer than the three-year contract term either finalist is quoting, which means a renewal negotiation arrives before the switch has paid for itself, a fact \$4 returns to as an exit-cost problem in its own right, not just a TCO one.

3.3 Retraining cost, named separately from migration cost

[HR/PEOPLE]

A migration project plan almost always budgets the vendor's own training curriculum — a two-day instructor-led course, a certification voucher — and treats retraining as finished once that course is delivered. What actually costs the SOC is the weeks after that course during which an analyst's query-writing speed on the new platform is still visibly slower than it was on the old one. That gap shows up as a real, temporary capacity loss, not a training-budget line item, and it lands directly on the metrics Part 5's headcount model and Part 9 — Queue Health & Workload Management's tracking both already watch.

Operational Reality *A migration project plan almost always budgets the vendor's own training curriculum — an instructor-led course, a certification voucher — and calls retraining done. What actually costs the SOC is the six to ten weeks after that course during which an analyst's query-writing speed on the new platform is still visibly slower than it was on the old one, showing up as a temporary bump in handle time that Part 5's headcount model and Part 9's queue-health tracking will both register as a real capacity loss, not a training-budget line item.*

[HR/PEOPLE]

A migration's retraining line is also an unplanned draw against the same annual training budget Part 12 — Ongoing Training & Skill Development governs; a manager who doesn't flag it in advance, as its own line item separate from the year's normal certification and conference spend, discovers the collision only when the training budget runs out early, with no warning and no obvious place the overrun came from.

4. Vendor lock-in and exit-cost analysis

4.1 What actually creates lock-in

[VENDOR/PROCUREMENT]

Lock-in is rarely one dramatic contract clause; it accumulates from four ordinary mechanisms, each individually reasonable and collectively expensive to escape. A proprietary query or rule language with no export path to a portable format turns every detection built on the platform into content that can only leave by being rewritten. A proprietary historical-data retention format, paired with a metered or gated bulk-export API, turns years of retained logs into an asset the vendor can charge to release. A deep integration surface — SOAR playbooks, dashboards, ticketing hooks, identity-provider connections — built on vendor-specific APIs multiplies the rebuild cost of leaving well past the core platform itself. And a bundled multi-module discount, where SIEM, EDR, and

SOAR are priced together from one vendor, means leaving any single module can unwind the discount on every module kept.

[VENDOR/PROCUREMENT] The table below names each mechanism, what it costs to escape once it's in place, and — the part that actually matters for this part's argument — where in the procurement process a manager still has real leverage to limit it, because none of the four is meaningfully negotiable after signing.

Lock-in mechanism	What creates it	What escaping it costs later	Mitigation available before signing
Proprietary query/rule language	Detection content authored directly in a vendor-specific syntax with no abstraction layer	Full re-authoring of every rule, priced the same way DEH Part 6's mapping/parity-testing cost is priced, paid a second time to leave	Negotiate a documented export format for rule logic; keep field references centralized on the new platform from day one, per DEH Part 6's own mitigation advice
Historical data retention format	Logs stored in a proprietary index behind a gated or metered bulk-export API	A one-time egress fee, sometimes large enough on its own to fund most of a migration	Negotiate a capped, included egress allowance and a defined export format in the contract, not just an unpriced right-to-export clause
Deep integration surface	SOAR playbooks, dashboards, and ticketing hooks built on vendor-specific APIs	Every downstream integration rebuilt, not just the core platform itself	Prefer a documented, versioned API; avoid a vendor-exclusive webhook format where a common alerting-webhook shape already exists
Bundled multi-module discount	SIEM, EDR, and SOAR priced together from one vendor at a combined rate	Leaving one module unwinds the discount on the modules kept	Price every module's standalone rate during the RFP, so the bundle's true size — and its unwind cost — is known before signing

4.2 Modeling exit cost as a first-class line item

[SENIOR MANAGER] Exit cost is the sum of what it would take to leave a finalist platform, estimated before signing rather than discovered at the next renewal: data egress and re-ingestion cost, content re-migration cost priced with the same four-component model used in §3.2, retraining cost for whichever platform comes next, and any contractual early-termination penalty. The mechanism that makes this analysis urgent rather than academic is that exit cost only grows over time — every quarter of accumulated custom content and every additional year of retained historical logs makes leaving more expensive than it was the quarter before — which means the only point in the relationship where an organization has real leverage to shape that cost is before it signs, not after three years of accumulation have already happened.

[EXECUTIVE] For a board or CISO conversation, express exit-cost exposure the same concrete way the TCO model in §3.2 is expressed — as a dollar figure and, more usefully, as a number of months of contract spend it would take to fund an exit today. A finalist whose exit cost equals eighteen months of contract value is carrying meaningfully more risk than one whose exit cost equals four months, even if the two quote identical license prices, and that risk deserves the same visibility in a renewal-year budget narrative that a security-risk finding gets in an incident report — Part 24 — Executive & Board Reporting covers building that narrative once this part's number exists to put in it.

4.3 When lock-in is priced only at renewal

[SENIOR MANAGER]

The composite case below shows the mechanism in §4.2 running to completion: an exit-cost estimate that was never built at procurement time, discovered instead at the exact moment it had the least leverage left to change anything.

COMPOSITE CASE EXAMPLE (CASE-2103) — merges patterns observed across several mid-market renewal negotiations; not one identifiable organization.

Management Autopsy — "the bake-off winner nobody could leave"

The decision: A mid-market, 30-analyst SOC selected a SIEM finalist with the highest functional bake-off score four years earlier — a feature-rich proprietary correlation and behavioral-analytics engine — helped along by an aggressive first-year discount that made it the cheapest option in year one specifically.

Why it seemed reasonable: The platform won the functional comparison decisively, the discounted first-year price undercut the incumbent, and the vendor's sales engineer described data portability as straightforward, available "any time," with no mention of a cost attached to it.

How it failed: Over four years, the SOC built 480 custom correlation rules directly in the vendor's proprietary rule syntax with no abstraction layer separating the logic from the platform, and its historical log retention lived entirely in the vendor's proprietary index, with bulk export gated behind a paid add-on module the organization had never purchased. At the four-year renewal, the vendor proposed a 24% list-price increase — roughly \$210,000 more per year than the prior year's spend. The SOC's own exit-cost estimate, built only at this point, totaled to roughly \$540,000: full DEH Part 6-style migration effort for 480 rules, an egress fee for three years of retained historical logs quoted at roughly \$85,000, and 320 hours of analyst retraining. At that exit cost, accepting the 24% increase was cheaper than leaving for at least the next two and a half years. According to the SOC's own account team afterward, the vendor's renewal negotiator had been aware of that exit-cost gap the entire time.

The fix: Run the exit-cost model in §4.2 before signing, not at renewal — specifically, negotiate a bulk-export/egress clause and a cap on future list-price increases into the original contract, while the buying organization still has bake-off-stage leverage. Never let custom content accumulate exclusively in a single vendor's proprietary syntax without a periodic re-measurement of what leaving would cost right now: lock-in priced early is a negotiating chip; lock-in priced only at renewal is a bill.

5. A decision framework: from trigger to signature

5.1 The procurement decision flow

[SENIOR MANAGER]

The flow below sequences everything this part has covered — scoping, the weighted scorecard, an analyst-driven PoC, off-list reference calls, and the TCO and exit-cost models — into one path a decision panel can actually follow, rather than three unconnected spreadsheets arriving at a meeting with no shared conclusion.

Figure 21.1 — The procurement decision flow, trigger to signature. *CONCEPTUAL.* Illustrates the sequence this part argues for — a weighted scorecard, an analyst-driven PoC, off-list reference calls, and a three-year TCO and exit-cost model, all resolving into one decision panel rather than a single feature-comparison score. It is a process diagram, not a capture of any specific SOC's actual procurement workflow. Diagram ID **FIG-2101**.

flowchart TD

A["Trigger: renewal, cost pressure,\nor a capability gap"] --> B["Scope requirement +\ninventory
existing content, Sec. 2.1"]

B --> C["RFP with weighted\nscorecard, Sec. 2.2"]

C --> D["Shortlist finalists"]

D --> E["PoC on own data,\nanalyst-driven, Sec. 2.3"]

E --> F["Reference calls,\nincl. one off-list, Sec. 2.4"]

F --> G["Three-year TCO model, Sec. 3"]

G --> H["Exit-cost model, Sec. 4.2"]

H --> I{"Fully-loaded TCO + exit cost\nacceptable vs. incumbent?"}

I -->|Yes| J["Decision panel signs off;\nhands to Part 22 for\ncontract structuring"]

I -->|No| K{"Is the gap in TCO,\nor in exit-cost exposure?"}

K -->|TCO| D

K -->|"Exit-cost"| L["Renegotiate egress/export\nterms before re-scoring"]

L --> H

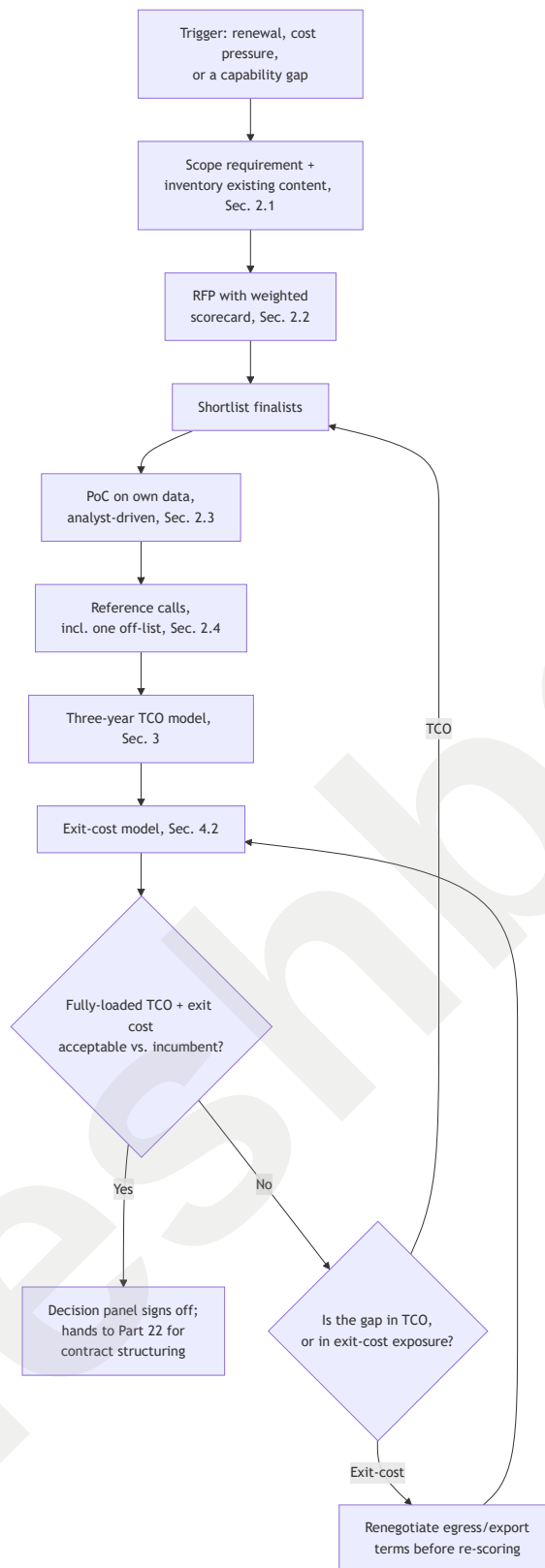


Figure 21.1 — The procurement decision flow, trigger to signature. *CONCEPTUAL.* Static flowchart rendered from the Mermaid source directly above. It supports the decision-framework guidance in §5, tying together the scorecard (§2.2), the TCO model (§3.2), and the exit-cost model (§4.2) into one process. Diagram ID **FIG-2101**.

5.2 One scorecard, not three spreadsheets

[SENIOR MANAGER] The scorecard in §2.2 already carries the TCO and exit-cost numbers as two of its seven weighted rows, so a decision panel using both isn't reconciling three separate documents in the room — it's reading one instrument whose two highest-weighted rows already have the fully loaded numbers from §3.2 and §4.2 behind them. A panel that reaches for the scorecard's total score without checking that those two rows were actually filled in from the worksheets, rather than estimated on the spot from a gut feel about the vendor's reputation, has reproduced the exact bake-off failure mode this part exists to fix, just with better formatting.

6. Platform-specific procurement wrinkles

[VENDOR/PROCUREMENT] The TCO and exit-cost mechanics in §3–§4 apply to all four platform categories this part covers, but what actually drives migration cost differs enough by platform type that a manager pricing one against the same DEH Part 6 model used for a SIEM will underprice or overprice the number without adjusting for it.

6.1 SIEM

[VENDOR/PROCUREMENT] For a SIEM, migration cost is dominated by the per-rule field-mapping-and-parity-test problem DEH Part 6 prices directly — rule count is the single largest driver of the mapping and parity-testing components in §3.2's worked example, and it scales close to linearly with however many detections the outgoing inventory in §2.1 counted.

6.2 EDR

[VENDOR/PROCUREMENT] For an EDR platform, migration cost is dominated less by query rewriting and more by fleet-wide agent redeployment: uninstalling the old agent, pushing the new one, and rebuilding prevention and detection policy across every endpoint on a patch-window schedule set by IT operations, not by the SOC. That logistics effort — and the coordination it requires with a team the SOC doesn't manage — is the dominant unbudgeted cost, and coordinating it cleanly is exactly the cross-team relationship Part 26 — Cross-Team Politics & Stakeholder Alignment covers.

6.3 SOAR

[VENDOR/PROCUREMENT] For a SOAR platform, migration cost is dominated by rewriting existing automated playbooks and re-testing every automated action path for the risk of a wrong containment or closure action, not by query syntax at all — a playbook that closes the wrong ticket or contains the wrong host silently is a materially worse failure than a detection that silently stops matching. Deciding what's safe to automate in the first place is SOC Playbook Handbook, Part 30 — Automation and SOAR's territory; this part's concern is narrower — pricing the cost of safely rewriting and re-validating playbooks that already exist, once a platform change is on the table.

6.4 TIP

[VENDOR/PROCUREMENT]

For a threat intelligence platform, migration cost is usually the smallest of the four, because a TIP rarely holds the bulk of an organization's custom detection logic — the dominant cost is re-integrating threat feeds and rewiring whatever enrichment pipeline feeds indicators into the SIEM or SOAR. Lock-in can still be real here even so: some community or paid feeds are contractually tied to a specific platform's ingestion format, and losing access to a feed a hunting program has built history against is a coverage cost, not just an integration one.

7. Where this goes next

[CONCEPT]

The TCO number this part's worked example produces is the opening exhibit for the broader budget defense Part 20 — Building & Defending the SOC Budget builds out in full; this part's job is only to make sure that number is fully loaded before it reaches that conversation. Once a decision panel signs off using the framework in §5, Part 22 turns the lock-in mitigations named in §4.1 into actual contract language — an export-format clause, a capped egress fee, a price-increase ceiling — rather than leaving them as intentions. Part 23 then picks the exit-cost model back up every renewal cycle as standing negotiating leverage, so the failure mode in §4.3's case — pricing exit cost for the first time at the moment it's least useful — never recurs on the same platform twice. Nothing past this part should treat a platform decision as a one-time purchase with a sticker price; every later reference to "the SIEM" or "the EDR" in this book assumes the fully loaded cost this part's model produces, not the number on the vendor's price sheet.

Cross-references: This part assumes Part 2 — SOC Operating Models: In-House, MSSP, Co-Managed, Hybrid (which staffing model is doing the buying, and who carries the migration effort) and Part 5 — Headcount & Capacity Modeling (the shrinkage and FTE math this part's retraining and capacity-loss figures borrow directly). It points forward, within this book, to Part 12 — Ongoing Training & Skill Development (the training-budget line a migration draws against unannounced), Part 15 — Quality Assurance Programs and Part 16 — Performance Management & Coaching (both of which assume a stable platform underneath their sampling and coaching cadence), Part 20 — Building & Defending the SOC Budget (the budget category this part's TCO model feeds), Part 22 — MSSP & Managed-Service Contract Management (the contract mechanics that implement this part's lock-in mitigations), Part 23 — Vendor Relationship & Renewal Management (the standing use of this part's exit-cost model as renewal leverage), Part 24 — Executive & Board Reporting (presenting exit-cost exposure upward), Part 26 — Cross-Team Politics & Stakeholder Alignment (coordinating an EDR fleet migration with IT operations), and Part 31 — Benchmarking & Industry Comparison (the reference-customer self-selection problem named in §2.4). It cites Detection Engineering Handbook V2, Part 6 — Normalisation for the technical migration-cost mechanics behind §3–§4, and SOC Playbook Handbook, Part 30 — Automation and SOAR for what's safe to automate once a SOAR platform is in place, which §6.3 deliberately does not re-derive. The RFP/PoC weighted evaluation scorecard ([TMPL-2101](#)) is filed in Appendix A5, companion to Parts 21–23.

Part 22 — MSSP & Managed-Service Contract Management

Why this part exists

[CONCEPT] Part 2 answers whether an organization should use an MSSP, MDR provider, or co-managed model at all. This part starts after that decision is made and a specific vendor is on the table: how the contract itself is written, because the contract — not the sales deck, not the pilot demo, not the account executive's verbal assurance — is the only thing that governs what happens the first time something goes wrong. A statement of work that reads well in a proposal and a statement of work that survives a dispute are not automatically the same document, and the gap between them is almost always in four places: how the service-level agreement is defined and measured, what the client can actually inspect and verify, who owns the data and the custom detection content once the relationship ends, and whether anyone is running a standing process to keep the contract honest between the signature and the renewal date.

[CONCEPT] Those four places are this part's entire scope, and nothing else. It does not cover choosing between operating models — that's settled ground by the time a contract is being negotiated, and Part 2 — SOC Operating Models: In-House, MSSP, Co-Managed, Hybrid owns the economics of that decision. It does not cover running a SIEM/EDR/SOAR/TIP procurement process — Part 21 — Tooling Procurement & Platform Strategy owns PoC evaluation, reference calls, and total-cost-of-ownership modeling for that kind of purchase, and while the evaluation discipline generalizes to picking an MSSP candidate in the first place, this part picks up once a specific vendor is already at the negotiating table, not before. And it does not cover the ongoing, portfolio-wide vendor governance that continues for years after signing — renewal-cycle negotiation strategy, vendor risk scoring across every tool and vendor the SOC runs, tool-sprawl rationalization, and the politics of retiring a vendor a stakeholder personally championed. That's Part 23 — Vendor Relationship & Renewal Management's territory, and the two parts are easy to blur because both use the word "governance." The distinction that keeps them separate: this part's governance cadence is the operating rhythm of one specific contract — did this MSSP hit its committed numbers this quarter, does the audit clause still work, is the offboarding language still current — while Part 23's is the multi-year, multi-vendor lens that decides whether this vendor relationship should exist at all going into the next renewal. A SOC manager who only reads this part will know how to keep a signed MSSP contract honest; Part 23 is where that manager learns how to decide, later, whether to keep it.

[CONCEPT] One more boundary matters enough to state before anything else: an SLA in an MSSP contract and the severity model a SOC uses to triage its own queue are not the same instrument, even though both use words like "critical" and "response time." SOC Playbook Handbook, Part 29 — Playbook Severity Model owns how an individual alert gets scored for internal triage purposes — a judgment call an analyst makes about one ticket. The SLA covered in this part is an external, negotiated, contractually enforceable commitment between two organizations, backed by a remedy (usually a service credit) when it's missed, and measured against clock-start and clock-stop definitions that are themselves subject to negotiation. A vendor can hit its own contractual "critical" SLA on paper while the underlying activity would score very differently against a client's internal severity rubric, and confusing

the two is one of the most common — and most expensive — mistakes a manager makes when reading a vendor's own performance dashboard at face value.

1. The SLA as a contract instrument

1.1 Why a contract SLA is not the internal severity model

[VENDOR/PROCUREMENT] A contract SLA exists to do one job: give the client a remedy when the vendor doesn't perform, and give the vendor a defensible, measurable target instead of an open-ended promise to "respond quickly." That job only gets done if every SLA metric in the contract specifies four things without ambiguity — what event starts the clock, what event stops it, which severity tier the commitment applies to, and what happens when it's missed. A metric missing any one of those four is not an SLA; it's an aspiration with a table around it.

Cross-Book Pointer *This part does not define how an alert's severity is scored for internal triage purposes — that's a per-ticket judgment call, not a contract term. See SOC Playbook Handbook, Part 29 — Playbook Severity Model for the scoring mechanics. What this part covers is different: the severity tiers named in the contract (often borrowing the same labels — critical, high, medium, low) as the buckets a negotiated response-time commitment attaches to. A vendor's contractual "critical" tier and a client's internal "critical" score can diverge the moment either side's definition changes without the other renegotiating — worth confirming they still mean the same thing at every governance review in §4, not just at signing.*

[VENDOR/PROCUREMENT] The practical test for whether an SLA clause is real: ask what document or system timestamp the clock-start event actually refers to, and whether that timestamp is one the client can independently verify, or one that only exists inside the vendor's own ticketing platform. If the answer is the second, the client has agreed to measure the vendor's performance using data only the vendor controls — a structural weakness worth naming during negotiation, not discovering during a dispute.

1.2 The clock-start problem

[VENDOR/PROCUREMENT] The single most common way an SLA looks strong on paper and performs weakly in practice is the gap between when something bad actually happened and when the vendor's contractual clock starts running. A vendor proposing "critical alerts acknowledged within 15 minutes" sounds aggressive until the contract language is read closely enough to find that the 15 minutes is measured from ticket creation inside the vendor's own platform — not from when the underlying telemetry first showed the malicious activity, and not even from when the vendor's detection logic first fired. Between those two points sits the vendor's own internal queueing and Tier 1 review, which can run anywhere from a few minutes to well over an hour depending on the vendor's current load, and none of that time counts against the SLA at all.

[VENDOR/PROCUREMENT] Negotiating the clock-start definition down to the earliest verifiable event — ideally the timestamp on the raw alert itself, logged in a system the client can also see — closes most of

this gap. Where a vendor won't agree to that (and many won't, because it exposes their own internal triage latency to the SLA for the first time), the fallback worth pushing for is a second, secondary metric: total elapsed time from alert generation to client notification, reported separately from the primary SLA, even if it carries no contractual remedy of its own. A number with no teeth is still a number a manager can track and use as leverage at the next renewal, which is more than a vendor-side clock that only starts once the vendor decides to start it.

1.3 Coverage-hours commitments: a distinct SLA class

[VENDOR/PROCUREMENT] Response-time commitments answer "how fast." Coverage commitments answer a separate question — "is anyone actually watching right now" — and the two get conflated constantly in vendor marketing material. "24/7/365 SOC monitoring" describes coverage; it says nothing about staffing depth, and a vendor can satisfy a literal reading of that phrase with one analyst covering several hundred client accounts overnight, which is a materially different service than the same phrase implied during the sales conversation.

[VENDOR/PROCUREMENT] The coverage-hours math from Part 2 — SOC Operating Models: In-House, MSSP, Co-Managed, Hybrid, §6 is the right lens for testing whether a vendor's staffing commitment is realistic, not just aspirational: a single continuously staffed seat needs roughly five full-time-equivalent analysts once shrinkage is accounted for, per that section and the underlying model in Part 5 — Headcount & Capacity Modeling. A vendor account team that can't produce a real number for how many dedicated analysts are on a given overnight shift, or that answers with the vendor's total headcount across every client rather than the per-shift figure, hasn't actually committed to a coverage level — it's committed to a phrase.

Field Test Setup: *The contract states a minimum staffing level for continuous coverage — for example, "no fewer than two dedicated SOC analysts on every shift, 24/7."* **Action:** *At an unannounced time between 2 a.m. and 4 a.m. on an ordinary weekday, place a real call to the vendor's after-hours contact line (not a scheduled test the account team knows is coming), and ask the analyst who answers two direct questions: how many analysts are on this shift right now, and how many client accounts is this shift covering.* **Expected result:** *The analyst answers both questions promptly and the numbers match what the contract commits to. If the analyst can't answer, has to check with someone else first, or names a per-analyst account load far above what a "dedicated" clause implies, the coverage commitment is functioning as marketing language, not a staffing guarantee — worth raising at the next governance review in §4, not waiting for an incident to expose it.*

[VENDOR/PROCUREMENT] The table below separates the SLA metric classes a contract typically bundles together under one heading, and names the trap specific to each — worth walking through clause by clause during negotiation rather than accepting a vendor's standard SLA table as a single unit.

SLA metric	What it commits to	Common contract trap
Notification/acknowledgment time	How fast the client is told a critical alert exists	Clock starts at the vendor's internal ticket creation, not at alert generation — elapsed real time can run well past the stated window
Time-to-triage or time-to-analysis	How fast the vendor forms an initial disposition	"Triage" often means "acknowledged," not "analyzed" — confirm the contract defines what activity must actually complete, not just that a status field changed
Time-to-escalation	How fast a confirmed finding reaches the client's own team	Only counted for alerts the vendor decides warrant escalation — a suppressed or under-triaged alert never starts this clock at all
Time-to-containment (MDR only)	How fast the vendor takes a scoped response action	Frequently the narrowest-scoped, most heavily caveated clause in the contract — confirm which actions are actually pre-authorized versus which still require client sign-off first
Coverage hours / staffing minimum	Whether committed headcount is actually on shift	"24/7 monitored" is a claim about the platform running, not about dedicated staffing depth — see the Field Test above
Platform/tooling uptime	Whether the vendor's own console and pipeline are available	Protects the vendor's infrastructure metric, not detection or response quality — a vendor can hit 99.9% platform uptime while still missing real threats

1.4 Service credits and what they actually compensate

[VENDOR/PROCUREMENT] Nearly every MSSP contract's remedy for a missed SLA is a service credit — a percentage discount off a future invoice — and nearly every one of those credit schedules is capped, both per incident and per billing period. The cap exists to protect the vendor from unlimited exposure, which is a reasonable thing for a vendor to want; the trap is a client mistaking the credit for compensation proportional to what a missed SLA actually cost, when in most contracts it is nothing close.

COMPOSITE CASE EXAMPLE **CASE-2201** — merges patterns from more than one MDR engagement; not a single traceable organization. Figures are illustrative.

Management Autopsy — "rely on the SLA credit as the real protection against a missed notification"

The decision: A mid-market retailer with roughly 2,500 endpoints signs an MDR contract with a critical-severity notification SLA of 30 minutes from the vendor's own ticket creation, backed by a service credit of 10% of that month's service fee per missed instance, capped at 20% of the monthly fee in any billing period. The monthly fee is \$28,000.

Why it seemed reasonable: A named SLA with a defined financial remedy looked like real accountability, and 30 minutes from ticket creation sounded fast enough during the sales process — nobody on the buying side ran the clock-start gap in §1.2 all the way through before signing.

How it failed: A credential-dumping alert against a domain controller fires at 02:14. The vendor's own Tier 1 queue is backed up; a ticket isn't created until 03:02 — 48 minutes later, none of which counts against the SLA. The vendor then takes 74 minutes from ticket creation to actually notify the client, missing even its own 30-minute clock. Total elapsed time from the real malicious activity to client notification: two hours and two minutes, during which the attacker moves laterally to two more hosts. Ransomware deploys at 04:40. The missed-SLA credit that lands on the next invoice is \$2,800 — 10% of the monthly fee. The incident itself, once containment, forensics, downtime, and breach-notification costs are totaled, runs approximately \$340,000.

The fix: Treat the credit schedule as a compliance signal, not a risk transfer — it tells you whether the vendor is meeting its own numbers, not whether you've been made whole for a miss. Negotiate for the credit cap to scale with incident severity rather than sitting at a flat percentage regardless of what the miss actually cost, and pair the SLA clause with the secondary elapsed-time metric from §1.2 so a repeated pattern of technically-compliant-but-slow notification is visible at governance review before it produces the incident that exposes it for real.

What Would Change My Mind This section treats service credits as compensating the vendor relationship, not the client's actual loss. If a meaningful share of MSSP contracts in practice used uncapped credit schedules that scaled with documented incident impact rather than a flat percentage of monthly fees, that would weaken the claim that credits are structurally cosmetic — worth checking a specific proposed contract's credit schedule against that standard before assuming this composite's pattern holds for it.

2. Right-to-audit clauses

2.1 What "right to audit" must specify to mean anything

[VENDOR/PROCUREMENT] "Client shall have the right to audit Vendor's security controls upon reasonable notice" is the single most common right-to-audit sentence in an MSSP contract, and it is close to unenforceable as written, because every load-bearing word in it — "reasonable," "audit," "notice" — is left for the vendor to define unilaterally the one time it actually matters. A right-to-audit clause worth having specifies five things: the audit type in scope (a documentation and access review, a technical control assessment, or an on-site visit are different commitments with different costs), the frequency the client can invoke it without cause, the notice period in days, who bears the cost of the audit itself, and the remediation timeline the vendor is bound to once a finding comes back.

[VENDOR/PROCUREMENT] A frequent, cheaper substitute worth naming explicitly rather than accepting by default: many vendors will offer an annual SOC 2 Type II report or ISO 27001 certification in place of a client-initiated audit. That substitution is a reasonable baseline for a lower-sensitivity engagement, but it answers a narrower question than a real audit does — it tells the client the vendor's controls satisfied an independent auditor's sampling on the dates covered by that report, not that the specific controls protecting this client's own telemetry are currently operating as described. Accepting the substitute clause without at least retaining the option to request a scoped, client-specific review under defined circumstances trades a real right for a document the vendor was going to produce anyway.

The table below states what each right-to-audit component should specify and why it's the piece most often left vague.

Audit clause component	What it should specify	Why it's often missing or weak
Audit type/scope	Documentation review, technical access review, or on-site — named explicitly, not left open	Vendors default to the cheapest interpretation ("reasonable" access to documentation) unless a stronger type is named
Frequency without cause	How many times per contract year the client can invoke it, e.g. once annually	Left silent, defaults to "whenever the vendor agrees it's reasonable" — functionally zero times
Notice period	A specific number of days, not "reasonable notice"	Vendors have negotiated 60- to 90-day notice periods under a "reasonable" standard, which is too slow to confirm posture before an active concern
Cost allocation	Who pays for the audit itself — client, vendor, or split above a defined scope	Silent by default, meaning the vendor can price a requested audit high enough to discourage using the clause at all
Subcontractor/fourth-party flow-down	Whether the audit right extends to any subcontracted tier, offshore desk, or platform the vendor itself relies on	Almost always missing unless negotiated — see §2.2
Remediation timeline	A committed number of days to remediate a confirmed finding, with an escalation path if missed	Left as "vendor will use commercially reasonable efforts," which sets no actual deadline
Regulator/examiner substitution	Whether the client's own regulator or outside auditor can exercise this right directly, not only the client itself	Relevant mainly in regulated industries, and rarely offered unless specifically requested

2.2 Subcontractors and the fourth-party blind spot

[VENDOR/PROCUREMENT]

An MSSP contract's right-to-audit clause is usually written against the vendor the client signed with — not against whatever subcontracted overnight desk, offshore triage tier, or platform provider that vendor itself relies on to deliver the service. That gap matters more than it looks like on paper: a vendor that markets a single brand and a single SLA can, entirely within its contractual rights, route the graveyard shift through a subcontracted staffing partner in a different jurisdiction, with different hiring standards and different access controls, and none of that shows up anywhere the client's audit clause reaches unless the flow-down was negotiated explicitly.

Blind Spot *A right-to-audit clause scoped only to the named vendor is structurally unable to see the fourth-party risk sitting underneath a subcontracted tier — the client can audit the vendor's front door and never learn who's actually staffing the 3 a.m. shift or what access controls that subcontractor's own staff operate under. This is the same telemetry-exposure question Part 2 §4 raises for raw log forwarding versus in-tenant access, one layer further removed: the client isn't just trusting the vendor's own security program, but a chain of vendors the contract may never name. Ask directly, before signing, whether any tier of the committed service is subcontracted, and negotiate flow-down audit rights or, at minimum, a disclosure obligation before assuming the audit clause covers the whole delivery chain.*

2.3 Exercising the clause before you need it

[VENDOR/PROCUREMENT]

A right-to-audit clause that has never been invoked since signing is an unknown quantity precisely when an organization most needs it to work — during an active concern about the vendor's posture, when the 60- or 90-day notice period the contract quietly specifies makes the clause useless for the situation that prompted reaching for it in the first place.

Field Test Setup: *A right-to-audit clause exists in a signed MSSP contract and has never been exercised. **Action:** Formally invoke the clause in writing, following the exact notice period and process the contract specifies, requesting the narrowest scope that would still test whether the mechanism functions — a documentation and access review, not a full on-site audit — timed to land before the next renewal decision rather than in response to an incident. **Expected result:** The vendor responds within the contracted notice window with the scoped deliverable, and the response confirms the process described in the contract actually runs the way it reads. If the vendor stalls, narrows the scope unilaterally, or leans on a "reasonable" qualifier to push past the stated window, that's the clause's real boundary — better discovered on a routine exercise than during a dispute where the clause was supposed to be the leverage.*

Operational Reality Most organizations sign a right-to-audit clause for the legal comfort of having one and never invoke it again until something has already gone wrong — at which point they discover, for the first time, exactly how long the notice period actually is and how narrowly the vendor is willing to interpret "reasonable" scope. Treat the clause the same way Part 15 — Quality Assurance Programs treats a QA rubric: a mechanism that only proves it works when it's actually exercised, on a routine cadence, not held in reserve as a theoretical deterrent.

3. Data ownership and the offboarding/transition-out clause

3.1 Telemetry vs. derived work product: two different ownership defaults

[VENDOR/PROCUREMENT] "Client owns all client data" is the boilerplate ownership line in nearly every MSSP contract, and it answers a narrower question than most managers assume it does. Raw telemetry — the logs, alerts, and events generated by the client's own infrastructure and forwarded to or processed by the vendor — is almost never seriously contested; that ownership line covers it cleanly. What it usually does not cover, unless negotiated separately, is everything the vendor's analysts and engineers build on top of that telemetry during the engagement: custom-tuned detection rules, enrichment logic, playbooks written specifically for this client's environment, and the accumulated case history and disposition notes attached to years of alerts.

[VENDOR/PROCUREMENT] Vendors have a real commercial reason to treat that second category differently, and it's not always bad faith: a detection rule written in the vendor's own proprietary rule syntax, running on the vendor's own platform, genuinely does incorporate the vendor's intellectual property alongside anything specific to the client's environment. The problem isn't that vendors claim some interest in that content — it's that most contracts never separate the two categories explicitly, leaving "client owns client data" to silently exclude the derived work product a client will need most on the way out the door.

3.2 The transition-out clause, element by element

[VENDOR/PROCUREMENT] A transition-out clause worth having specifies six things, each of which is silently absent from a generic MSSP contract unless someone negotiates it in: the format and timeline for returning historical alert and case data, stated in a portable format the client's next platform can actually ingest, not a proprietary export only the vendor's own tooling can read; how long the vendor retains that historical data after termination, and under what access terms, in case something is missed in the initial handoff; a written certification of data deletion once the retention period ends, so the client isn't relying on a verbal assurance that a former vendor actually purged its systems; a defined transition-assistance period — commonly 60 to 90 days post-termination — during which the outgoing vendor remains contractually obligated to answer questions and hand off context, not just data; an explicit statement of whether custom-tuned detection logic and the rationale behind it transfers to the client in a usable, documented form, not just as an unreadable rule file; and a cap or waiver on "reverse

transition" fees — what the vendor is allowed to charge for the act of helping the client leave, which some contracts leave entirely open-ended.

[VENDOR/PROCUREMENT] That last point deserves its own sentence: a vendor with no cap on exit-support fees has a direct financial incentive to make offboarding expensive, and a client who didn't negotiate a cap discovers that incentive exists at the worst possible moment — during the transition itself, with no leverage left, because the old contract is already ending and the new one hasn't started delivering yet.

3.3 What happens when this clause is missing

COMPOSITE CASE EXAMPLE CASE-2202 — merges patterns from more than one insourcing transition following a multi-year managed-detection engagement; not a single traceable organization. Figures are illustrative.

Management Autopsy — "let the boilerplate data-ownership line stand in for a real transition-out clause"

The decision: A 1,200-employee healthcare-services company signs a three-year co-managed MDR contract with only the standard "client owns client data" line and no negotiated transition-out terms, on the reasoning that a three-year horizon made offboarding mechanics feel like a distant, low-priority negotiating point at signing.

Why it seemed reasonable: The contract was competitively priced, the vendor's onboarding process was smooth, and nobody involved in signing expected to be thinking about an exit three years before the exit was relevant — a reasonable, common way for a low-priority-seeming clause to get traded away for a better rate elsewhere in the negotiation.

How it failed: At the end of the term, the company moves to a fully in-house model, following the same insourcing trajectory Part 2 §8 describes. The vendor confirms raw alert data is available, but only through a data-extraction service priced at \$150 per gigabyte, quoting \$61,000 to retrieve the full three-year archive. More consequentially, the vendor declines to hand over the tuning rationale behind more than 40 custom-built detection rules, citing proprietary rule syntax and internal IP — the client receives the rules' final logic but no documentation of why each threshold or exclusion was set the way it was. The incoming in-house detection-engineering team spends roughly four months rebuilding baseline detection content essentially from first principles, and two real coverage gaps surface only in hindsight, once an internal review compares the rebuilt ruleset against what the vendor's rules had actually been catching.

The fix: Negotiate the transition-out clause — data-return format, retention, deletion certification, a defined transition-assistance window, and documented custom-rule rationale — at signing, not at renewal and never at termination, when the outgoing vendor has the least remaining incentive to cooperate. Appendix A5's SLA contract-clause library exists specifically to give a negotiator standing language for this clause to start from, rather than drafting it from scratch under time pressure three years later.

Blind Spot A data-ownership clause reviewed once, at signing, only covers the log sources and rule set that existed on that date. Custom detection content built in month 14 of a three-year engagement, or a new data source onboarded in year two, was never contemplated by the original ownership language — and nothing in a typical contract prompts anyone to revisit that clause when new content gets built. Treat a material addition to the vendor's custom detection footprint as a trigger for confirming the ownership and transition terms still cover it, the same way Part 2 §4 treats a new data source as a trigger for re-running the telemetry-sensitivity review.

4. The standing governance cadence once signed

4.1 The cadence itself

[SENIOR MANAGER] A signed contract with a strong SLA, a real audit clause, and a clean transition-out provision still fails if nobody runs a standing process to check, on a fixed schedule, whether the vendor is actually performing to what was negotiated. Four cadence items belong in every MSSP governance program, run on their own schedule rather than only when something has already gone wrong: a quarterly business review (QBR) covering SLA performance, notable incidents, and upcoming changes on either side; a monthly review of the vendor's own SLA compliance report, cross-checked against the client's independent record of the same incidents rather than accepted at face value; a defined change-control process for tuning requests, so a request to suppress a noisy rule or adjust a threshold has a committed turnaround time instead of disappearing into the vendor's general queue; and tracking the renewal-notice window and the right-to-audit exercise cadence from §2.3 far enough in advance that neither deadline arrives as a surprise.

[SENIOR MANAGER] The QBR is the cadence item most likely to decay into a status-update ritual with no teeth, and the fix is specific: every QBR should walk through the SLA metrics from §1's table against the vendor's actual numbers for the quarter, not a vendor-prepared summary slide asserting compliance. A QBR that only reviews the vendor's own framing of its own performance isn't governance — it's a briefing.

4.2 Who owns which piece: a governance RACI

[SENIOR MANAGER] Governance cadence items fail most often not because nobody is willing to do them, but because no single role is named as accountable for each one, and a task with no named owner quietly stops happening the first time the person who used to do it informally changes roles. The table below assigns each standing cadence item across the roles typically involved in an MSSP relationship — who is Responsible, Accountable, Consulted, or Informed for each one, so a new SOC manager inheriting the relationship can see who actually owns keeping it honest (CONCEPTUAL SAMPLE — illustrative role assignment; substitute your own organization's actual titles).

Governance activity	SOC manager	Legal/Procurement	Vendor relationship owner	Vendor account team
Quarterly business review	C	I	R/A	R
Monthly SLA compliance cross-check	R/A	I	C	I
Tuning/change-request turnaround	R/A	—	C	R
Right-to-audit exercise (§2.3)	C	R/A	R	I
Renewal-notice-window tracking	C	R/A	R	I
Contract amendment for new data sources or added custom rules	C	R/A	R	C

[SENIOR MANAGER] Two patterns in that matrix are worth naming directly. The vendor relationship owner — a role distinct from the SOC manager in any organization large enough to separate them — carries Responsible or Accountable status on nearly every row, which is the point: someone specific has to own the relationship day to day, or every item on this list defaults to whoever notices it's overdue. And Legal/Procurement is Accountable for exactly the two items with contractual notice periods and

deadlines attached — the audit clause and the renewal window — because those are the two failure modes most likely to be discovered too late to act on, not too early.

4.3 The contract lifecycle, end to end

[SENIOR MANAGER] Figure 22.1 traces the full lifecycle this part covers, from the moment the contract's core clauses take effect through the standing governance loop, and into whichever of the two paths — renewal or termination — the governance history actually supports. Notice that the right-to-audit clause and the transition-out clause aren't separate, occasional events sitting outside the main loop; they're the two mechanisms the loop exists to feed into when performance data alone doesn't resolve the renewal question.

flowchart TD

```
A["Contract signed:\nSLA, audit rights, data-ownership\nand offboarding clauses in force"] --> B["Standing governance cadence:\nQBR, monthly SLA cross-check,\nchange-control for tuning"]
B --> C["SLA compliance and audit\nfindings over the contract term"]
C --> D["Consistently meets committed\nSLA; findings closed on time"]
D --> E["Renewal window opens\n(Part 23's territory)"]
E --> F["Repeated SLA miss or a\nfailed/blocked audit finding"]
F --> G["Right-to-audit clause\nformally invoked (Sec. 2.3)"]
G --> H["Vendor remediates within\nthe contracted window?"]
H --> I["Yes"]
I --> B
H --> J["No"]
J --> K["Termination-for-cause\nconsidered"]
K --> L["Renew"]
L --> B
K --> M["Do not renew"]
M --> N["Transition-out clause invoked:\ndata return, deletion certification,\ntransition-assistance period (Sec. 3.2)"]
N --> O["New vendor or in-house team\nnonboarded against returned data\nand documented detection logic"]
```

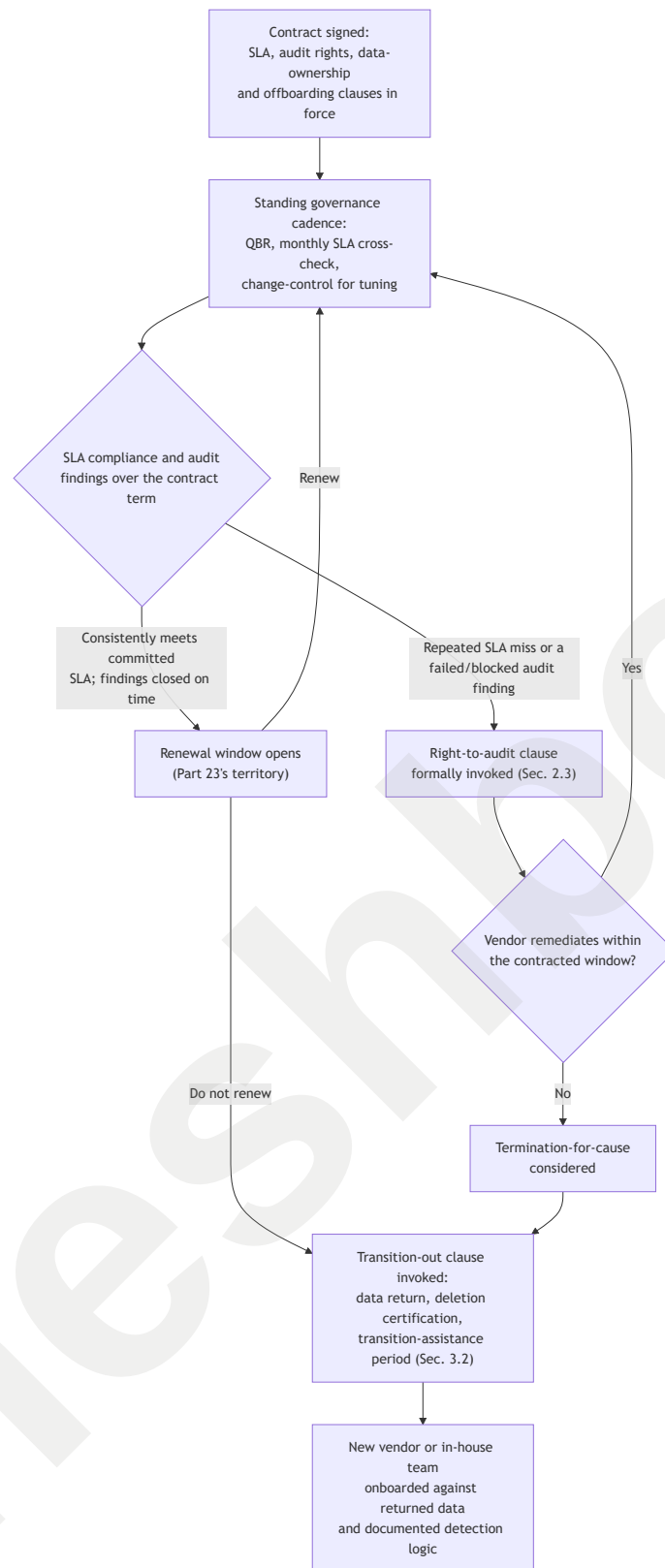


Figure 22.1 — The MSSP contract lifecycle, from signing through renewal or offboarding.

CONCEPTUAL. FIG-2201. Illustrates how the standing governance cadence (§4.1–§4.2) feeds into either a renewal decision or a formal invocation of the right-to-audit and transition-out clauses (§2–§3), and supports the claim (made across §2–§4) that the audit and transition-out clauses aren't rarely-used contract boilerplate but the two mechanisms the standing governance loop routes into whenever

ordinary performance monitoring surfaces a problem it can't resolve on its own. It is a structural model of the decision path, not a capture of any single organization's actual contract-management workflow.

5. Building the contract: a negotiation checklist

[VENDOR/PROCUREMENT] The clauses covered in §1 through §4 don't get negotiated in one pass, and a negotiator walking into a renewal or a first contract with a long legal document and no prioritization will spend the available negotiating capital on whichever clause the vendor's own draft happens to present first — not necessarily the one that matters most. The checklist below orders the four clause families by how expensive they typically are to fix after signing versus how much negotiating leverage they cost to fix before signing — a working aid for prioritizing what to push hardest on during negotiation, not a substitute for legal review of the actual contract language.

Clause family	Cost to fix after signing if missing	Typical negotiating difficulty	Priority
Clock-start definition on core SLA metrics (§1.2)	High — every subsequent SLA report is measured against the weaker definition	Moderate — vendors resist exposing internal triage latency, but usually concede a secondary metric	First
Coverage-hours staffing minimum, stated as a number, not a phrase (§1.3)	High — discovered only when coverage is actually tested, often during an incident	Moderate	First
Transition-out clause: format, retention, and custom-rule documentation (§3.2)	Very high — renegotiating this from a position of near-zero leverage, at termination, is close to impossible	Low — this clause costs a vendor little to grant and most will agree without much resistance if asked	First
Right-to-audit scope, notice period, and subcontractor flow-down (§2.1–§2.2)	High — but only realized if the client actually needed to exercise it, which is rarer than the other three	Moderate to high — subcontractor flow-down specifically draws real resistance	Second
Service-credit cap scaling with incident severity (§1.4)	Moderate — a capped credit is a known, bounded weakness, not a hidden one	High — vendors protect credit-cap terms aggressively	Third
Named-resource/key-person clauses on the account team	Moderate — mainly a continuity risk, not a data or audit risk	Low to moderate	Third

[VENDOR/PROCUREMENT] The third row is the one most negotiators underweight relative to its actual cost, precisely because it's the cheapest to fix and the most expensive to discover missing. A vendor granting clear transition-out terms at signing is giving up almost nothing it currently has — no client is transitioning out on day one — while a client who waits to negotiate that clause until the relationship is already ending has given up nearly all of its own leverage to get it. Appendix A5 — Vendor, Contract & Procurement Templates carries a fuller SLA contract-clause library built from this same prioritization; the table above is the ordering logic a negotiator should bring to that library, not a replacement for reading the specific language it contains.

Cross-references

Within this book: Assumes the operating-model decision from Part 2 — SOC Operating Models: In-House, MSSP, Co-Managed, Hybrid, and uses the coverage-hours FTE math from Part 2 §6 and Part 5 —

Headcount & Capacity Modeling to test whether a vendor's staffing commitment is realistic (§1.3). Feeds forward into Part 23 — Vendor Relationship & Renewal Management for the portfolio-wide, multi-year renewal and vendor-risk-scoring lens this part deliberately doesn't cover, and into Part 9 — Queue Health & Workload Management, which cites this part's burst-capacity contract clause as the prerequisite for MSSP overflow surge handling. Cites Part 21 — Tooling Procurement & Platform Strategy for the vendor-evaluation methodology that generalizes to selecting an MSSP candidate before this part's negotiation begins, Part 15 — Quality Assurance Programs for the same use-it-or-it's-theoretical logic applied to the right-to-audit clause, and Part 20 — Building & Defending the SOC Budget for the build-vs-buy-vs-outsource business case this part's contract terms ultimately feed a dollar figure into.

SOC Playbook Handbook: Part 29 — Playbook Severity Model (the internal, per-alert severity scoring this part's contractual SLA tiers are explicitly distinct from, per §1.1).

Detection Engineering Handbook V2: Not cited directly in this part — contract structure and clause negotiation have no technical detection content, and this part deliberately stops at the organizational and legal mechanics rather than drifting into what the vendor's detection logic itself should contain.

Part 23 — Vendor Relationship & Renewal Management

Why this part exists

[CONCEPT] Part 21 — Tooling Procurement & Platform Strategy owns the decision to bring a vendor in: the PoC, the reference calls, the total-cost-of-ownership model. Part 22 — MSSP & Managed-Service Contract Management owns writing the contract that decision produces: the SLA, the right-to-audit clause, the offboarding terms, the governance cadence that starts the day both sides sign. Neither part owns what this one does — the years between signing and either renewing or walking away, during which a vendor's product, pricing, support quality, and ownership structure all keep changing whether or not anyone is watching. A contract signed well in year one can be a bad deal by year three with no single decision anyone can point to as the moment it went wrong — it just drifted, unmonitored, until a renewal invoice or a support failure forced the question.

[CONCEPT] This part covers three things that only happen after signing: running a renewal negotiation on a cadence instead of a deadline, scoring vendor risk as a standing program rather than a one-time gate, and rationalizing tool sprawl — the accumulated state of three tools doing one job because retiring any of them requires a fight nobody wanted to start. That third topic has a specific, recurring complication this part treats as first-class rather than a footnote: the tool a senior stakeholder personally championed is, on the merits, often the easiest one to justify killing and the hardest one to actually kill, and the two facts are unrelated to each other. Part 26 — Cross-Team Politics & Stakeholder Alignment develops the general playbook for standing cross-team friction; this part covers the specific, common instance of it that shows up every time a tooling budget gets reviewed, because a vendor-retirement decision is where that friction shows up earliest and most predictably in most SOC's.

[CONCEPT] Two things this part deliberately leaves to other chapters. What a tool's renewal or replacement costs, and how that cost is defended to a CFO, is Part 20 — Building & Defending the SOC Budget's arithmetic; this part produces the renewal decision and the dollar figure behind it, not the budget-category structure that figure lands in. And whether a tool flagged for retirement is actually safe to retire from a coverage standpoint — whether some detection or telemetry source uniquely depends on it — is not a governance question this part can answer on its own; that confirmation step depends on Detection Engineering Handbook V2's coverage material, cited in §4 below rather than re-derived.

1. From procurement to governance: what changes after the signature

[CONCEPT] A vendor relationship has exactly one moment where the SOC has real, favorable leverage by default: the period before a contract is signed, when the vendor is competing for the deal and the SOC can walk away to a competitor at close to zero cost. Every day after signature erodes that leverage a little, because switching now costs migration effort, retraining, and — for anything touching detection logic — a real coverage gap during the transition. Part 22's governance cadence (quarterly business reviews, SLA-compliance checks, a named account-management contact) exists to slow that erosion, not eliminate it. A governance cadence that runs on autopilot — the quarterly call happens, the SLA numbers get reported, nobody asks a harder question than "any issues?" — recovers almost none of the leverage a genuinely adversarial renewal negotiation needs, because it never produces the two things

that create leverage after signing: a documented pattern of vendor performance drift, and a live, credible alternative.

[SENIOR MANAGER] The practical shift this part asks for is treating every signed vendor contract as an open, live risk position rather than a closed decision. A signed contract is not evidence the vendor question is settled for its term — it's evidence a decision was correct on the date it was made, with no guarantee that the vendor, the market, or the SOC's own needs will still match that decision by the next renewal. The rest of this part builds three concrete mechanisms — a renewal calendar, a risk scorecard, and a rationalization audit — that turn "we should keep an eye on this" into something that actually runs on a schedule instead of getting reprioritized every time a live incident eats a manager's week.

2. The renewal negotiation cadence

[CONCEPT] A renewal negotiation started 30 days before the contract's auto-renewal date is not a negotiation — it's a request to a vendor who already knows the SOC has no time to switch, no completed alternative evaluation, and no credible walk-away option. Leverage in a renewal conversation is a function of runway, not of how compelling the SOC's arguments are, and runway has to be built into the calendar months ahead of the date anyone starts thinking about the renewal as a live decision.

2.1 Building a renewal calendar by contract tier

[VENDOR/PROCUREMENT] Not every contract deserves the same runway. A four-figure logging add-on and a seven-figure SIEM platform contract do not carry the same switching cost, and treating them identically either wastes review time on low-stakes renewals or — far more common in practice — leaves the highest-stakes contract with the same 60-day review window as a commodity tool, because nobody built a tiered calendar and every renewal date just shows up on the same generic list. The table below sorts contracts into three tiers by annual value and technical dependency, and assigns each a review-start date measured backward from the renewal or auto-renewal date, as a working default for building a renewal calendar rather than a fixed rule for every organization.

Tier	Typical annual value	Technical-dependency signal	Review starts (before renewal date)	Owner
Tier 1	Above \$250,000, or any value with a single point of technical dependency (primary SIEM, EDR, core case-management platform)	Detection logic, playbooks, or daily triage workflow are built directly on top of this vendor's platform	180 days	SOC manager, with procurement and security architecture
Tier 2	\$25,000–\$250,000	Meaningful workflow dependency, but a credible substitute exists without a full platform migration	90 days	SOC manager or a delegated team lead
Tier 3	Below \$25,000, commodity or single-purpose tooling	Low switching cost; a comparable tool could be stood up in weeks	45 days	Team lead or whoever owns the tool day to day

[VENDOR/PROCUREMENT] The review-start date is not the negotiation-start date — it's the point by which the vendor risk scorecard (§3) and, for any tool with overlap candidates, the rationalization audit

(§4) both need a current read. A Tier 1 review that starts at 180 days but only produces a usage number and a price quote by day 30 has wasted five of its six months of runway on nothing; the calendar buys time specifically so those two inputs exist early enough to shape the negotiation itself, not so the date sits on a calendar unused until it's nearly gone.

2.2 The auto-renewal trap

[VENDOR/PROCUREMENT] Most vendor contracts of any size auto-renew by default, with a notice-to-cancel window — typically 30, 60, or 90 days before the renewal date — buried in a contract clause almost nobody rereads after signing. Missing that window doesn't just mean staying with the vendor; it locks in another full term, usually at whatever price-escalation clause the contract specifies, with zero further negotiating room until the next cycle. This is the single most avoidable failure this part covers, because the fix costs nothing but a calendar entry, and it is also the single most common one, because the notice window is easy to forget once the person who negotiated the original deal has moved on.

CASE-2301 — the missed 60-day window. *COMPOSITE CASE EXAMPLE, illustrative figures constructed from a recurring pattern across mid-market SOC tooling contracts, not one traceable organization.* A threat-intelligence platform contract, originally negotiated by a detection-engineering lead who left the organization four months before the next renewal date, carried a 60-day notice-to-cancel clause and an 8% annual price escalator on auto-renewal, both stated in the contract's Section 14.3 and never flagged anywhere else. The departing lead's own calendar reminder for the renewal review left with them; nobody else had it. Six months earlier, the SOC had migrated off the on-premises SIEM this platform's feed was originally built to enrich, and actual usage of the platform had fallen by roughly half — fewer analysts touched it, and the integration that once justified the full feed tier was no longer running. The 60-day window passed with no review. The contract auto-renewed at \$194,400 for another 12 months (up from \$180,000, the 8% escalator applied automatically), of which roughly \$97,000 — close to half the renewed value — represented capacity and integration scope the SOC was no longer using at all.

Manager's Note *Put every vendor contract's auto-renewal notice date on a calendar the day the contract is signed, owned by a role rather than a person — "SOC manager" or "vendor governance lead," never a named individual who might leave before the date arrives. The calendar entry costs five minutes. The 8% escalator on a contract nobody was tracking anymore cost this SOC roughly \$97,000 it never got a chance to negotiate away.*

[VENDOR/PROCUREMENT] The fix generalizes past this one case: every Tier 1 and Tier 2 contract's notice-to-cancel date and price-escalation terms should be extracted into the renewal calendar at signing time — the exact clause language, not a paraphrase — as one of Part 22's standing governance-cadence deliverables, so this part's calendar has something real to work from rather than having to reread every contract cold at review time.

2.3 Leverage decays with time

[VENDOR/PROCUREMENT] Leverage in a renewal conversation is highest early and collapses fast as the renewal date approaches, and the mechanism is simple: a vendor's account team can tell, from the

calendar alone, whether the SOC has enough runway left to credibly switch. A renewal conversation opened at 150 days out, backed by a live competitive evaluation already underway, is a different conversation than the same numbers presented at 20 days out with no alternative lined up — the vendor knows the second one has no real walk-away option behind it, and prices accordingly.

Time to renewal	Available leverage	What's realistically achievable
150+ days	High — a credible competitive process is still possible	Price renegotiation, scope changes, a genuine rebid if the vendor won't move
60–150 days	Moderate — too late to run a full competitive PoC, not too late to threaten one	Price and term concessions; vendor account teams still move to avoid a documented at-risk renewal
30–60 days	Low — vendor knows switching in this window is operationally disruptive	Minor concessions at best; usually a like-for-like renewal with cosmetic changes
Under 30 days	Near zero	Accept current terms, or accept a short-term (month-to-month) extension while a real process starts for next cycle

[VENDOR/PROCUREMENT] The practical implication of this curve is that the renewal calendar in §2.1 isn't primarily an administrative safeguard against missing a deadline — it's what preserves the top two rows of this table as live options instead of foreclosed ones. A SOC that starts every renewal conversation inside the bottom two rows, however good its negotiators are, is negotiating from a position the calendar gave away months earlier.

3. Vendor risk scoring as a standing program

[CONCEPT] Part 21's PoC evaluation scorecard answers one question, at one point in time: does this vendor's product fit the requirement well enough to sign. That score is a snapshot, and snapshots go stale — a vendor's financial health, security posture, support quality, and product roadmap all keep moving after the ink dries, and nothing about a strong PoC score two years ago tells a manager anything about where those dimensions sit today. Vendor risk scoring, as this part uses the term, is a different instrument: a small set of dimensions, rescored on a fixed cadence, built specifically to catch drift rather than to justify an initial decision.

3.1 What a one-time PoC score can't tell you

[VENDOR/PROCUREMENT] A PoC scorecard is built to compare competing vendors against a requirement at a single moment, and it does that job well. It has no mechanism for catching what happens next: a vendor gets acquired by a private-equity-backed roll-up and support staffing quietly thins out over the following year; a key product line goes into maintenance mode while the vendor's roadmap attention shifts to a newer, unrelated product; a vendor's own security posture degrades — a breach, a lapsed compliance certification, a leadership departure in the security function — in ways that matter enormously to a SOC buying a security tool and that a PoC run three years ago had no way to anticipate. None of these show up in an invoice. All of them show up, eventually, in a renewal conversation that goes worse than expected, or in a live incident where the vendor's own support turns out to be the bottleneck.

3.2 A standing vendor risk scorecard

[VENDOR/PROCUREMENT] The table below names six dimensions worth tracking on a fixed cadence — quarterly for Tier 1 vendors per §2.1's tiers, at minimum annually for everyone else — each scored 1 (healthy) to 5 (severe concern), with a documented reason for the score, not just a number. The scoring itself is a judgment call every quarter; the value of the program comes from forcing that judgment to happen on a schedule and get written down, so a slow decline shows up as a trend line instead of staying invisible until it becomes a crisis.

Dimension	What it measures	A 4 or 5 typically looks like
Financial stability	Public financial signals, funding rounds, layoff news, or (for private vendors) payment/renewal behavior suggesting distress	Reported layoffs in the product's own division, a down-round, or a sudden push for multi-year prepayment
Ownership/M&A exposure	Recent or rumored acquisition, private-equity ownership, leadership churn	Acquired within the last 12–18 months with no public integration roadmap yet
Support quality trend	Median ticket response/resolution time, account-team turnover, escalation-path clarity, measured against the prior review	Response time materially worse than the prior review, or a named technical contact who left with no replacement assigned
Product/roadmap health	Release cadence, whether the specific features the SOC relies on are still receiving investment	No meaningful release addressing the SOC's use case in over a year; feature deprecation notices with no clear replacement
Security posture	The vendor's own breach history, certification status (SOC 2, ISO 27001 renewal currency), and how transparently they disclose incidents	A lapsed certification, an undisclosed or slowly disclosed incident affecting customer data
Concentration risk	How many critical SOC functions depend on this one vendor with no fallback	A single vendor covers detection, case management, and reporting with no documented failover plan

[VENDOR/PROCUREMENT] Use the scorecard below as the standing worksheet a vendor-governance owner fills in each review cycle, filed alongside the renewal calendar entry for the same vendor; it belongs in Appendix A5 as **TMPL-2301**, the vendor risk-scoring template. It does not automate the judgment call behind any single score — two reviewers can legitimately land on different numbers for the same evidence, especially on support-quality trend, which is why the "evidence/reason" field is not optional filler: a score with no stated reason behind it can't be compared meaningfully to the next quarter's score, and the trend line is the entire point of running this more than once.

VENDOR: _____ REVIEW PERIOD: Q__ 20__ REVIEWER: _____

Dimension	Score (1-5)	Evidence / reason for this score	Change vs. last review
Financial stability		<!-- cite the specific signal, not a gut feel -->	
Ownership / M&A exposure			
Support quality trend		<!-- compare to a specific prior-period number, e.g. median response time -->	
Product / roadmap health			
Security posture			
Concentration risk		<!-- name the specific functions that depend on this vendor with no fallback -->	

OVERALL FLAG: ☐ Green (no action) ☐ Yellow (monitor, revisit next cycle)
☐ Red (any single dimension at 4-5, or 2+ dimensions worsened since last review -- build a contingency plan before the next renewal date, regardless of how far out it is)

[VENDOR/PROCUREMENT] A vendor that scores red on this worksheet is not necessarily a vendor to drop — a red flag means the contingency-planning clock has started, not that the decision is already made. The judgment call the scorecard doesn't make for you is what to do once a vendor is flagged; that call, and the negotiation leverage a documented red flag actually buys, is what §3.3's worked example turns on.

3.3 Worked example: acquisition, and the score that would have caught it

CASE-2302 — the private-equity acquisition nobody scored. *COMPOSITE CASE EXAMPLE, illustrative figures constructed from a recurring pattern following vendor private-equity roll-up acquisitions in the security-tooling market, not one traceable organization.*

[VENDOR/PROCUREMENT] An EDR vendor a mid-market SOC had used for three years was acquired by a private-equity-backed platform consolidator. No formal risk review existed at the time — the relationship had been running on Part 22's standard quarterly business-review cadence, which covered SLA-compliance numbers and upcoming feature requests, not ownership or support-trend questions. Over the following nine months, median support-ticket response time rose from roughly 4 business hours to roughly 3 business days; two named technical account managers left within the same window and were never replaced with a dedicated point of contact; and the product's release notes, which had shipped meaningful updates roughly every quarter for the prior two years, showed no material feature release for 14 straight months. None of this was hidden — it was all visible in ordinary support tickets and public release notes — but nothing in the SOC's existing governance cadence was built to notice a trend across nine months of otherwise-routine interactions.

[VENDOR/PROCUREMENT] The consequence arrived at renewal: a quote 22% above the prior year's price, with no corresponding improvement in the roadmap to justify it, arriving with roughly 45 days of runway left on the existing contract — deep in the low-leverage band of §2.3's table, because nobody had started a review earlier. The SOC had no completed alternative evaluation in hand and no time to run one, and renewed on largely the vendor's terms.

[VENDOR/PROCUREMENT] Run against **TMPL-2301** retroactively, the same nine months would have produced a clear early signal: support quality trend alone would have dropped from a 1 or 2 to a 4 by the third month, on the strength of the response-time change alone, well before the roadmap stagnation or the renewal quote made the problem undeniable. A quarterly scorecard catching that drop at month three — rather than discovering it live at month nine — buys six additional months of runway, moving the eventual renewal conversation from the "under 30 days" row of §2.3's leverage table into the "150+ days" row, with a credible competitive alternative evaluation actually possible in that window instead of foreclosed by the calendar before the negotiation even started.

Blind Spot *A quarterly scorecard catches a slow, visible drift — support response time, release cadence — reasonably well. It does not catch a sudden, non-public event between review cycles: an acquisition announced the week after a review just ran, a breach disclosed on a delay, a key engineering team laid off with no public statement. Treat the scorecard as a floor, not a guarantee, and pair it with a standing habit of watching public M&A and security-disclosure news for every Tier 1 vendor between formal review cycles — the review cadence tells you when to look deliberately; it doesn't replace paying attention in between.*

4. Tool sprawl rationalization

[CONCEPT] Tool sprawl is not usually one bad decision — it's the accumulated residue of several individually reasonable ones, none of which included a corresponding decision to retire anything. A SOC rarely wakes up one day having decided to run three overlapping threat-intelligence platforms; it ends up there because each one was added to solve a real problem at the time it was added, and nobody ever revisited whether the problem the first one solved still needed a dedicated tool once the third one existed.

4.1 How sprawl actually happens

[VENDOR/PROCUREMENT] Three mechanisms account for most tool sprawl this part sees, and each has a different early-warning sign worth watching for specifically, because the fix differs slightly by cause even though the eventual rationalization audit (§4.2) is the same regardless.

- **Inherited overlap from a merger or reorganization.** Two SOC's — or a SOC absorbing a business unit's informal monitoring function — combine, and each side's toolset comes along, fully licensed and fully justified on its own terms, with no forcing function that ever required choosing one over the other.
- **Shadow procurement for an urgent, narrow need.** A team lead signs up for a free-tier or low-cost trial to solve one specific problem fast, outside the normal procurement process Part 21

governs, and the trial becomes a permanent fixture because it worked well enough that nobody wanted to be the one to turn it off, even after the normal procurement process later adds a platform that does the same thing properly.

- **Bundled capability that duplicates a standalone purchase.** An EDR, SIEM, or SOAR platform upgrade adds a bundled feature — threat intel, case management, basic SOAR playbooks — that overlaps a tool already purchased standalone for the same function, and the standalone tool's renewal simply continues because nobody connected the new bundle to the old contract at the time either happened.

People Risk Trap *A shadow-procured trial has no natural expiration, which is exactly why it's the mechanism most likely to survive years past the "urgent, narrow need" that justified it — nobody owns a formal decision to keep it, so nobody ever has to make a formal decision to kill it either. The fix: any tool brought in outside Part 21's normal procurement process gets an automatic 90-day expiration written down at the moment it's adopted, with a named owner who has to actively request continuation through the normal process to keep it past that date. Silence should default to the trial ending, not to it becoming permanent.*

4.2 Auditing overlap: usage data, not seat counts

[VENDOR/PROCUREMENT]

The single most common analytical mistake in a rationalization audit is measuring adoption by license seats provisioned instead of actual usage, because seat counts are the number every vendor's own dashboard surfaces first and the number every stakeholder defending a tool will cite. Forty provisioned seats on a platform that eight people have logged into in the last 90 days is not "widely adopted" — it's eight users and thirty-two unused licenses, and the difference between those two framings is exactly the difference between a tool that's earning its renewal and one that isn't. Pull real usage telemetry wherever the platform exposes it: active users in the trailing 90 days, actual query or API call volume, and — the number that matters most for a security tool specifically — how many escalated true positives actually trace back to something this tool uniquely surfaced, as opposed to something every other tool in the stack would have caught anyway.

Cross-Book Pointer *This part's usage audit tells you whether a tool is being used. It does not tell you, on its own, whether retiring it would open a real detection gap — that's a coverage question, not a usage question, and a low-usage tool can still be the only source for one specific, rarely-triggered but important detection. See Detection Engineering Handbook V2, Part 41 — Detection Coverage for the six-tier model that answers whether a given technique's coverage genuinely depends on this tool alone, and Part 42 — Detection Quality for confirming that coverage is actually validated rather than assumed. Run that check before finalizing any retirement decision this section's audit points toward — a tool with low usage and a real, unique coverage dependency is a retuning-and-migration project, not a straightforward cut.*

4.3 Worked example: three threat-intel platforms, one job

CASE-2303 — the triplicate TIP. COMPOSITE CASE EXAMPLE, illustrative figures constructed from a recurring pattern across SOC's that accumulated threat-intelligence tooling incrementally, not one traceable organization.

[VENDOR/PROCUREMENT] A SOC found itself running three separate threat-intelligence platforms feeding IOC and enrichment data into its detection stack. TIP-A was a standalone paid platform, procured three years earlier at \$62,000 a year, before the SOC's current EDR contract existed. TIP-B arrived bundled at no additional license cost inside a later EDR enterprise-tier upgrade, but consumed roughly 15 analyst-hours a month reconciling its feed against TIP-A's, because the two platforms tagged and scored the same indicators differently and analysts had learned to cross-check both rather than trust either alone. TIP-C was a lower-cost feed with a paid enrichment add-on running about \$40,000 a year, originally wired up by a threat hunter who has since moved to a different team, now maintained by whoever happens to own the integration that quarter.

CONCEPTUAL SAMPLE -- illustrative arithmetic, not sourced benchmark data

Overlap analysis across a two-quarter sample:

Indicators duplicated across 2+ of the 3 platforms:	~85% of total volume
Escalated true positives in the sample window:	340
Of those, uniquely traceable to a TIP-A-sourced indicator:	14 (~4% of total)

Annual cost:

TIP-A license:	\$62,000/year
TIP-B:	\$0 license (bundled), ~15 analyst-hrs/month reconciliation
TIP-C + enrichment add-on:	\$40,000/year

Reconciliation labor: 15 analyst-hrs/month x 12 = ~180 analyst-hours/year spent
cross-checking duplicate indicators across three consoles instead of one

[VENDOR/PROCUREMENT] The overlap analysis, not a subjective sense that "we probably have too many tools," is what made the decision defensible: 85% indicator duplication meant the three platforms were mostly restating the same intelligence in three formats, and TIP-A — the most expensive of the three — contributed uniquely to only about 4% of actual escalated true positives over the sample window. The SOC retired TIP-A, kept TIP-B (already paid for as part of the EDR bundle) as the primary feed, and kept TIP-C's enrichment add-on for the specific indicator categories TIP-B's bundle didn't cover well. The result: \$62,000 a year in license cost eliminated outright, and roughly 180 analyst-hours a year reclaimed that had been going into reconciling duplicate alerts across three consoles instead of working a single queue.

4.4 A rationalization decision path

[VENDOR/PROCUREMENT] Figure 23.1 turns §4.2's usage audit and §4.3's overlap analysis into a repeatable decision path for any tool a rationalization review flags as a candidate, folding in the renewal-timing question from §2 and the coverage check from §4.2's Cross-Book Pointer so usage, uniqueness, and timing all get checked before a retirement decision is finalized.

flowchart TD

A["Tool flagged as a possible\nsprawl/overlap candidate"] --> B{"Usage audit (Sec 4.2):\nactive users & escalations\ntraceable to this tool alone?"}

B -->|"Low usage,\nlow unique contribution"| C{"Coverage check --\nDEH Part 41/42:\nany technique uniquely\ndepends on this tool?"}

B -->|"High usage or\nhigh unique contribution"| D["Keep -- renew per\nSec 2's normal cadence"]

C -->|"No unique dependency"| E["Retire candidate --\nproceed to Sec 5's\nstakeholder-alignment\nsequence before announcing"]

C -->|"Yes, unique dependency\nexists"| F["Not a simple retirement --\nscope as a migration project:\nrebuild that coverage\nelsewhere first"]

E --> G{"Renewal date\nwithin 60 days?"}

G -->|"Yes"| H["Use as leverage: renegotiate\ndown to a short extension\nwhile retirement completes"]

G -->|"No"| I["Time the announcement\nto the next natural budget\nor contract-review cycle"]

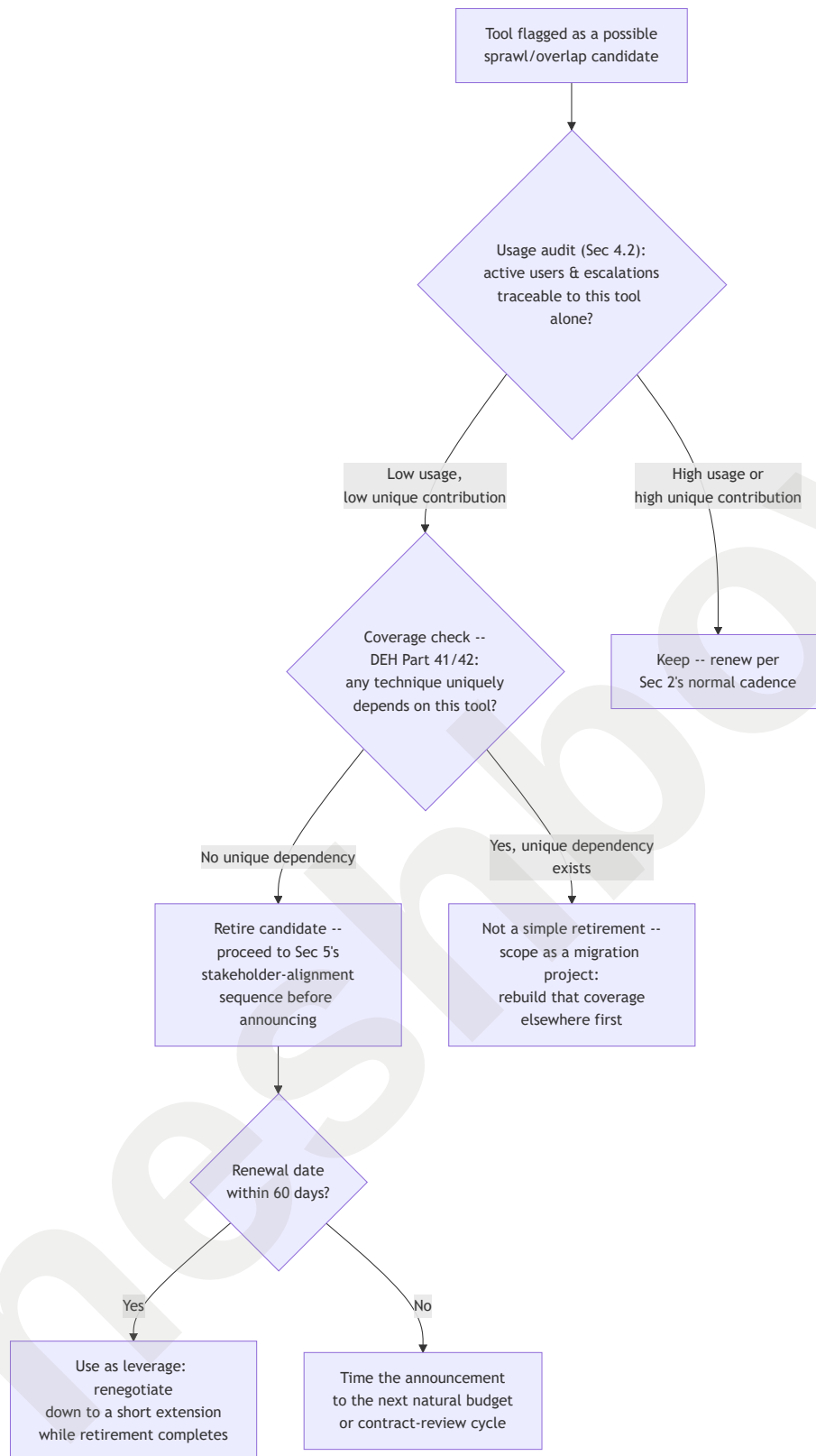


Figure 23.1 — Tool rationalization decision path. CONCEPTUAL. FIG-2301. Illustrates how a usage audit, a detection-coverage check, and renewal timing combine into a single retirement decision; it is a structural decision aid, not a capture of any specific SOC's actual tooling-review workflow. It deliberately routes every "retire candidate" outcome into §5's stakeholder-alignment sequence rather than straight to an announcement — the data-driven decision and the politically survivable announcement are two

different problems, and this diagram exists partly to keep a reviewer from treating "the numbers say retire it" as the end of the process.

5. The political difficulty of killing a champion's tool

[CONCEPT] Every rationalization audit eventually flags a tool someone senior chose, championed to their own leadership, and has a real stake in seeing succeed — not out of irrationality, but because recommending a platform, defending its budget in front of peers, and having it later quietly retired reads, to most people in that position, as a public reversal of their own judgment. The data in §4 can be completely correct and still lose this fight, because the fight isn't about the data. It's about whether the retirement can happen in a way that doesn't require the champion to publicly concede they were wrong two years ago.

5.1 Why this is a people problem, not a data problem

[SENIOR MANAGER] A manager who treats a champion's objection to a retirement recommendation as a data disagreement — "let me show you the usage numbers again, more clearly this time" — is answering a question the champion isn't actually asking. The champion already knows, in most cases, that usage is low; the objection is rarely a genuine dispute about the eight active users on a forty-seat license. What's actually being protected is standing: the narrative that the original decision was sound, the relationship with a vendor executive that might be personal as well as professional, and the risk that a visible reversal becomes the story other leaders remember about that person's judgment. No usage chart resolves any of that, because none of it is about the chart.

People Risk Trap *Presenting a data-backed retirement recommendation to a champion for the first time in a group setting — a monthly ops review, a leadership meeting, anywhere with an audience — is close to guaranteed to produce a defensive, public reaction regardless of how solid the data is, because the champion now has to respond in front of an audience whose opinion of their judgment matters to them. The fix: the champion sees every number privately, first, with enough time to react before anyone else does. A senior stakeholder blindsided in public will very often find a reason to object on the spot and win, purely on the strength of having more positional authority in that room than the person presenting — not because the objection was actually stronger than the data.*

5.2 Tactics that actually work

[SENIOR MANAGER] Four moves consistently make this conversation survivable, none of them a substitute for having the data right in the first place.

- **Brief privately, well before any public forum.** Give the champion the usage and cost data one-on-one, framed as "here's what the current numbers show, what am I missing" rather than "here's the recommendation." That framing invites correction instead of forcing a defense, and it surfaces a genuine objection — a real coverage dependency, a contract term nobody else knew about — before it becomes a public standoff.

- **Reframe the decision as evolution, not indictment.** "This was the right call for where we were two years ago; we've outgrown what any single tool needed to cover back then" is a true statement in most sprawl cases and a survivable one, because it doesn't require anyone to have been wrong — only for circumstances to have changed, which they usually genuinely have.
- **Bring a scoped alternative, not a bare cut.** A recommendation that ends at "retire this" reads as pure loss to the person who championed it. A recommendation that reads "consolidate into the platform we already have, migrate these three remaining use cases over an eight-week window" gives the champion something to co-own instead of something to lose.
- **Let the champion be the one who announces it, at least in part.** A retirement the champion frames to their own leadership — "we've evolved past needing this as a standalone tool" — lands completely differently than the same retirement announced about their decision by someone else. Ceding the announcement costs the SOC manager almost nothing and buys most of the political goodwill needed to get the decision through cleanly.

[SENIOR MANAGER] When none of that resolves the disagreement — the private brief happens, the alternative is on the table, and the champion still blocks it — the next move is escalation to a shared superior, not a public argument the SOC manager has no positional authority to win outright against someone more senior. Framed as "we have a clear cost and usage case and a stakeholder objection that hasn't been resolved after a good-faith private conversation; we need a decision," that escalation is a legitimate use of the chain of command, not a failure to have handled the politics well enough on the manager's own.

5.3 Management Autopsy: the ambush retirement

[SENIOR MANAGER] **CASE-2304** below shows what §5.1's warning looks like when it isn't heeded — a technically correct recommendation, delivered in exactly the wrong sequence, turning a straightforward cost decision into a 14-month political stalemate.

Management Autopsy — "present the retirement in the monthly ops review" (CASE-2304, COMPOSITE CASE EXAMPLE)

The decision: A SOC manager brought a fully data-backed recommendation to retire a legacy case-management console — 8 active users out of 40 analysts, averaging roughly two logins a month each — directly to the monthly ops review attended by the CISO's full leadership team, reasoning that a leadership forum was the right venue for visibility on a real cost decision.

Why it seemed reasonable: The usage numbers were clean, the cost case was strong, and the manager assumed presenting to the full leadership group would move the decision faster than a series of one-on-one conversations — more visibility, one meeting, done.

How it failed: The console's original champion — a director who had personally sold the platform to the CISO two years earlier as a signature initiative — was in the room, unbriefed, and challenged the usage numbers on the spot rather than accept them in front of peers. Rather than get corrected with a follow-up data point, the challenge escalated into "let's take this offline and revisit," which quietly became a 14-month stay of execution with no further review scheduled. The SOC manager's credibility also took a real hit outside this one decision — colleagues who witnessed what looked like a public ambush of a fellow director became noticeably more cautious about what the manager brought to shared forums afterward.

The fix: The same recommendation, delivered to the champion privately a week ahead of the ops review, with the framing from §5.2 and a scoped migration alternative already attached, would very likely have reached the same ops-review meeting as a jointly presented, already-agreed change instead of a live confrontation — the data didn't need to change at all; only the sequence in which people saw it did.

6. Running this as a standing quarterly program

[CONCEPT] A renewal calendar, a risk scorecard, and a rationalization audit each work as described above in isolation, but they compound when run together on the same standing cadence, because the inputs to one are frequently the trigger for another: a red-flagged risk score (§3) is exactly the condition that should pull a vendor's renewal review forward regardless of where it sits on the §2.1 calendar, and a rationalization audit (§4) is most useful when it's run against the same vendor list the risk scorecard already covers, rather than as a separate, occasional special project.

6.1 Who's in the room

[SENIOR MANAGER] A vendor governance review that's only ever attended by the SOC manager tends to produce decisions that stall the moment they touch budget, legal terms, or a stakeholder outside the SOC's own reporting line — exactly the failure mode §5 covers. The table below states who needs a standing seat and why, as a minimum viable roster rather than an exhaustive committee.

Role	Why they're in the room
SOC manager	Owns the overall recommendation and the tradeoffs across vendors
Procurement/legal contact	Confirms contract terms, notice windows, and negotiating room accurately, rather than from memory
Finance/budget owner	Confirms the dollar impact lands correctly in Part 20's budget structure and flags multi-year commitment implications
Security architecture representative	Speaks to the coverage-dependency question from §4.2's Cross-Book Pointer before any retirement is finalized
Relevant team lead or stakeholder (rotating)	Represents the tool's actual day-to-day users; invited specifically for vendors under review that quarter, including any champion per §5

6.2 The quarterly cadence, assembled

[SENIOR MANAGER] A minimum viable version of this program runs on two nested loops: a quarterly refresh of the risk scorecard (§3.2) for every Tier 1 vendor and a usage-data pull for any vendor already flagged as a possible overlap candidate, plus an annual pass that runs the full rationalization audit (§4) across the entire tool inventory and rebuilds the next 12 months of the renewal calendar (§2.1) from scratch rather than incrementally patching last year's dates. The quarterly loop is what catches drift early enough to matter, the way §3.3's worked example shows; the annual loop is what catches sprawl that accumulated too gradually for any single quarter's review to notice on its own.

Manager's Note *Calendar the quarterly scorecard refresh the same week every quarter — tied to a fixed date, not to "whenever things are quiet enough." A vendor governance program that only runs when the SOC has spare capacity runs during exactly the quarters when nothing urgent happened, which are the quarters least likely to contain the drift worth catching in the first place.*

7. Where this goes next

[CONCEPT] The renewal calendar, the risk scorecard, and the rationalization audit built in this part feed directly into Part 20's budget cycle every time a renewal decision produces a number that needs defending, and into Part 26's broader cross-team politics treatment every time a rationalization decision runs into a stakeholder whose objection has nothing to do with the tool itself. Nothing here replaces the initial selection discipline in Part 21 or the contract-drafting mechanics in Part 22 — this part assumes both already happened and picks up the moment the ink is dry, which is also the moment most SOC's stop paying attention until the next crisis forces the question back onto someone's desk.

Cross-references

Within this book: Part 20 — Building & Defending the SOC Budget (the budget-category structure a renewal decision's dollar figure ultimately has to fit into); Part 21 — Tooling Procurement & Platform Strategy (the PoC and TCO process this part's ongoing risk scorecard picks up from at signing); Part 22

— MSSP & Managed-Service Contract Management (the contract terms, SLA, and standing governance cadence this part's renewal calendar and risk scorecard build on rather than re-derive); Part 26 — Cross-Team Politics & Stakeholder Alignment (the general treatment of standing cross-team friction that this part's §5 covers as one specific, recurring instance).

Detection Engineering Handbook V2: Part 41 — Detection Coverage (confirming whether a rationalization candidate is the sole source of validated coverage for a technique before it's retired, per §4.2's Cross-Book Pointer); Part 42 — Detection Quality (confirming that coverage is actually tested and current, not just assumed, before the same retirement decision is finalized).

neshbooy

Section F — Governance, Risk & the Executive Interface

neshbooy

Part 24 — Executive & Board Reporting

Why this part exists

[EXECUTIVE] A SOC can have a clean, well-instrumented metrics program — mean time to respond and false-positive rate defined consistently per SOC Playbook Handbook, Part 32 — Metrics, plus mean time to detect and backlog age defined just as consistently as this book's own operational terms (§3 below, Part 9 §1) — and still fail completely the first time someone has to stand in front of a board and explain what those numbers mean for the company's risk. A board member doesn't act on "MTTR improved from 42 minutes to 31 minutes this quarter." They act on "the average window an attacker has to do damage before we notice shrank by about a quarter, and here's what that bought us on the incident we actually had." Those are the same fact, stated two different ways, and only one of them survives contact with a room full of people whose job is fiduciary oversight, not incident response.

This part covers the mechanics of getting from the first sentence to the second, reliably, on a fixed cadence, in a form that holds up when someone asks a hard question back. It does not define what MTTR, MTTA, or false-positive rate actually measure — that ownership sits with SOC Playbook Handbook, Part 32 — Metrics. Backlog age is Part 9 §1's own operational term. Mean time to detect (MTTD) is defined in §3 below, since it belongs to neither companion volume — every number this part touches beyond that is a finished input from whichever definitional source actually owns it, not something re-derived here. It does not build the CFO-facing budget defense — cost-per-analyst, cost-per-alert, the build-vs-buy business case — that's Part 20's material, and a board deck borrows that model's headline number without rebuilding the model itself. It does not cover the manager's own judgment about which risks get escalated into this reporting cycle in the first place, versus accepted quietly at the SOC-manager level — that call, and the cases where it's genuinely unclear, belongs to Part 25. And it does not cover the compressed, high-frequency communication cadence a manager runs during a live major incident, which is a different problem with a different clock — Part 28 owns that. What's left, and what this part actually owns, is the steady-state reporting relationship: cadence, format, translation, and the specific discipline of building material that survives a skeptical follow-up question.

1. What the board is actually asking

[CONCEPT] Board members overseeing a SOC's output are rarely security practitioners, and treating them as junior analysts who just need the jargon translated is its own category error — they aren't trying to learn what a SOC does, they're trying to discharge a specific fiduciary obligation with limited time on an agenda that includes this topic alongside a dozen others that have nothing to do with security. Underneath the specific questions any given board asks, three recur across almost every board relationship a SOC manager will report into.

Are we exposed, right now, in a way that matters? Not "how many alerts did we see" — whether the organization's actual risk posture changed this quarter, and in which direction. **Are we prepared for the thing most likely to hurt us?** Not "do we have an incident response plan" as a yes/no — whether the plan has been tested against something resembling the organization's real threat profile, and what that test revealed. **Is the money we're spending on this buying a proportionate reduction in risk?**

Not the budget defense itself — Part 20 owns that mechanically — but whether the board can credibly tell a regulator, an auditor, or a shareholder that security spend was overseen, not just approved.

[EXECUTIVE] That third question has gotten sharper in most jurisdictions in recent years. Both disclosure obligations below come from the same SEC final rule on cybersecurity risk management, strategy, governance, and incident disclosure, adopted July 2023, but they took effect on two different clocks worth keeping straight before citing either one to a board: the Form 8-K requirement — disclosing a material cybersecurity incident within four business days of a materiality determination — carried its own compliance date of December 18, 2023, for most registrants (smaller reporting companies got until June 15, 2024); the annual disclosure (Form 10-K, Item 1C), describing the board's cybersecurity risk oversight process itself and not just its outcome, applies starting with annual reports for fiscal years ending on or after December 15, 2023. A board that can't describe how it oversees cyber risk is exposed on that fact alone, independent of whether an incident ever happens, which is a direct board-level reason the SOC's reporting has to hold up as a documented process, not just as a set of accurate numbers. This isn't universal — plenty of organizations this book's readers run a SOC for aren't SEC registrants — but it's the sharpest current version of a pressure that shows up in some form in most regulated industries and most audit-committee relationships.

2. Reporting cadence: who gets what, how often

[SENIOR MANAGER] A single reporting cadence trying to serve the CISO, the board, and an audit committee at once produces material that's too shallow for the CISO's actual working relationship with the SOC manager and too deep for a board member juggling eleven other agenda items in the same meeting. Match cadence and depth to the audience's actual decision rights, not to whatever cadence is administratively convenient to produce. The table below matches audience to cadence and depth, and names who actually owns preparing each one.

Audience	Typical Cadence	Content Depth	Primary Question Answered	Who Prepares
SOC manager's own leadership review	Weekly	Full operational detail — queue health, staffing gaps, tuning backlog	Are this week's numbers within expected range	SOC manager / team leads
CISO	Monthly	Trend-level metrics, named exceptions, resourcing flags	Is anything drifting that needs intervention before it reaches the board	SOC manager
Audit / risk committee	Quarterly	Risk-narrative summary, control-effectiveness statements, material changes	Is the control environment operating as represented	CISO, with SOC manager input
Full board	Quarterly (or per bylaws)	High-level risk posture, incidents of note, forward risk items	Are we exposed, prepared, and spending proportionately	CISO, presenting the manager's material
Ad hoc — material incident	Hours to days of a materiality determination	Incident-specific, fact-based, legal-reviewed	What happened, what's the exposure, what's next	CISO plus legal, SOC manager as input (Part 28)

[SENIOR MANAGER]

Content should narrow, not just shrink, as it moves up this table — the weekly review is a full instrument panel, the CISO's monthly pass filters for anything that needs a decision above the SOC manager's authority, and the board's quarterly slot filters again for anything that changes the answer to one of §1's three questions. A deck that just compresses the weekly review's font size for the board isn't narrowing content, it's hiding the same volume of noise in less space.

Manager's Note *If you're the SOC manager personally standing in front of the full board every quarter, check who actually owns that relationship on paper. In most organizations the CISO holds the board seat and the SOC manager feeds material through them — confusing "I built the numbers" with "I own the board relationship" is a fast way to say something in a board room that legal or the CISO needed to see first.*

3. The translation problem: from operational metric to risk narrative

[CONCEPT]

A metric without a stated consequence is inert in a board room. "MTTD is 3.1 hours" is a fact a board member can neither dispute nor act on — it has no direction, no comparison point, and no attached decision. The translation this part builds is a fixed three-part recipe: state the metric and its trend, name the mechanism in plain business terms (what changing that number actually does or doesn't protect), and close with the specific thing the board should do, ask, or simply note as a result. Skip any one of the three and the sentence reverts to a number with a period after it.

Cross-Book Pointer *This part does not define MTTR, MTTA, or false-positive rate — those definitions, and the instrumentation discipline that makes them comparable quarter over quarter, are SOC Playbook Handbook, Part 32 — Metrics. Backlog age is Part 9 §1's own operational term. Mean time to detect (MTTD) is defined here, in this part, because neither companion volume owns it: it is the elapsed time between a security-relevant event first appearing in raw telemetry and the point a detection fires or an analyst first identifies it — a measure of dwell time before notice, distinct from MTTA (Playbook Part 32), which measures how fast an analyst acknowledges an alert that has already fired. Read Part 32 first for MTTR, MTTA, and false-positive rate, and make sure this part's own definition of MTTD (and Part 9 §1's definition of backlog age) is applied just as consistently, if your organization doesn't already have one shared, audited definition per metric — a board narrative built on a metric that means something different in Q1 than in Q3 will eventually get caught by exactly the kind of follow-up question §7 covers, and it won't survive it.*

[EXECUTIVE]

The table below is a translation pattern to reuse for any metric your program tracks — take the raw number, name the mechanism in one sentence, and close with the specific thing the board should do or ask next. The specific figures in the "Raw Statement" column are illustrative placeholders standing in for whatever your own instrumentation reports (**CONCEPTUAL SAMPLE — illustrative numbers, not sourced benchmark data**); the translation pattern in the other two columns is the reusable part.

Operational Metric (definition source)	Raw Statement	Risk Translation for the Board	What It Should Prompt the Board to Do
Mean time to detect (MTTD) — this part's own term, §3 above	MTTD dropped from 6.2 hours to 3.9 hours this quarter	An intruder now has roughly 2 hours less time inside our environment before we notice, on average — that's the window we closed	Note the trend; ask what closed it, and whether it holds under a worse scenario than an average one
Mean time to respond (MTTR) — Playbook Part 32	MTTR held flat at 31 minutes	Once we notice something, our response speed hasn't changed — flat is only reassuring if detection or exposure hasn't gotten worse elsewhere	Ask whether "flat" is masking a shift in the mix of what's coming in (see §7's pattern for this exact question)
False-positive rate — Playbook Part 32	FP rate is 68% this quarter, down from 74%	Roughly 2 out of 3 alerts our team investigates turn out to be nothing, down from 3 out of 4 — fewer wasted analyst-hours chasing noise, which is capacity we can put against real threats	Treat as a capacity and efficiency signal, not a safety signal — Detection Engineering Handbook V2, Part 42 — Detection Quality owns whether detection quality itself actually improved
Backlog age (p90) — this book's own term, Part 9 §1	P90 backlog age is 4.5 hours, up from 2 hours	1 in 10 open items now sits untouched for over 4.5 hours before anyone looks at it — double last quarter's figure, and where a real incident could sit unnoticed the longest	Ask whether this is a volume spike, a staffing gap, or both, before approving any fix
Detection coverage (validated)	Coverage against our top 5 mapped threat scenarios is 3 of 5 fully validated	2 of the 5 attack patterns we've identified as most likely to hit us don't yet have a tested detection behind them — a known, named gap, not an unknown one	Ask for the remediation timeline and whether it needs budget (Part 20) or just engineering time — Detection Engineering Handbook V2, Part 41 — Detection Coverage owns how "validated" is actually defined

Cross-Book Pointer *The false-positive-rate and detection-coverage rows above translate a technical measurement into a board-facing consequence; they don't explain how false-positive rate gets reduced or how "validated coverage" gets established in the first place. For the tuning mechanics behind a falling false-positive rate, see Detection Engineering Handbook V2, Part 38 — False Positive Engineering; for what "validated" means on a coverage matrix, see DEH Part 41 and DEH Part 42. This part only owns turning whatever those parts produce into a sentence a board member can act on.*

Manager's Note *Draft the risk-translation sentence before you finalize the chart, not after. If you can't write the one sentence a board member should take away from a metric before you've picked its axis labels and colors, the chart is decoration around a number nobody's translated yet — and that gap shows the moment someone in the room asks what it means and the honest answer is a longer, less confident version of the sentence you skipped writing.*

4. Worked example: turning a quarter's numbers into a board narrative

[SENIOR MANAGER]

COMPOSITE CASE EXAMPLE (CASE-2401) — illustrative organization and figures, constructed to show how one quarter's operational numbers become one board slide; not drawn from a single traceable real SOC.

The organization: A 28-analyst in-house SOC at a mid-market insurer (roughly 4,000 employees), reporting through a CISO who presents to the full board once a quarter and to the audit committee in the intervening months.

The quarter's raw numbers: MTTD improved from 5.8 hours to 3.1 hours following a new EDR deployment (Part 21's tooling-procurement territory, not re-covered here). One event during the quarter matched a known ransomware-precursor pattern — credential dumping followed by an attempt to disable backup agents — and was contained at the credential-dumping stage, before any encryption or exfiltration occurred. FP rate held at 71%, roughly flat quarter over quarter. P90 backlog age rose from 2.1 to 3.4 hours, driven by a two-week staffing gap while a departing analyst's backfill was still in the pipeline (Part 18's attrition territory).

The manager's first draft, too raw to present: "MTTD down from 5.8h to 3.1h. FP rate flat at 71%. One High-severity ransomware-precursor incident, contained. P90 backlog age up from 2.1h to 3.4h due to a staffing gap." Accurate, and useless to a board member reading it cold — four disconnected facts with no ranking and no consequence attached to any of them.

The board-ready version: "This quarter, we cut our average detection time nearly in half, which is what let us stop a ransomware-precursor attack at the credential-theft stage — before it reached backup systems or encryption. That's the incident this deck's next slide covers in more detail. The one number moving in the wrong direction is how long a lower-priority ticket now sits before anyone looks at it, which tracks directly to a two-week staffing gap we've since closed; we're watching it for one more quarter before treating it as a trend." Same four facts, ranked by consequence, with the one bad number named honestly instead of buried in a footnote, and a specific, bounded commitment instead of a vague reassurance.

Why the rewrite works: it leads with the fact that actually answers §1's fiduciary questions, uses the ransomware-precursor incident as concrete proof behind the MTTD number instead of leaving MTTD as an abstract improvement, and doesn't let the one negative number disappear. A board that later discovers a manager quietly dropped a worsening metric from the deck stops trusting every other number in it, which costs far more than one uncomfortable quarter.

Operational Reality *The instinct to drop or soften the one metric moving the wrong direction is strongest exactly when the rest of the deck looks good — a manager riding a genuinely strong MTTD improvement has the most to lose by also flagging the backlog-age miss, and the most cover to quietly leave it out. Boards that catch this once, even on a metric that turns out to be minor, downgrade their trust in every number that manager presents afterward, not just the one that was hidden.*

5. Building the board-ready deck

[EXECUTIVE] The table below is a five-section deck skeleton that fits inside a 20–30 minute board slot including questions — use it as a default structure, not a mandatory template. A quarter with nothing forcing a resourcing ask should simply drop that slide rather than padding it to fill the slot.

Deck Section	Purpose	Typical Length	Content Source
Risk posture summary	One-glance answer to "are we exposed"	1 slide, 3–5 lines	This part's translation layer (§3)
Incidents of note	Concrete proof behind the summary	1 slide per incident, at most two or three incidents	Post-incident organizational review (Part 29)
Trend metrics	Direction of travel on the metrics that matter this cycle	1 slide	The scorecard in §6, sourced from Playbook Part 32 (MTTR, FP rate) and this book's own definitions (MTTD — §3 above; backlog age — Part 9 §1)
Resourcing / spend ask	Ties risk posture to a specific budget or headcount decision	1 slide, only when there's an actual ask	Part 20's budget model, Part 5's headcount model
Forward risk items	What the board should expect to hear about next cycle	1 slide	Part 25's risk-acceptance log, escalated items

[SENIOR MANAGER] This structure is built out as a reusable board deck template ([TMPL-2401](#)) in Appendix A7, alongside the risk-acceptance memo template Part 25 uses and the post-incident organizational review template Part 29 uses — the three share a lot of sourcing logic, which is why they're grouped in the same appendix. Use the template as a starting skeleton and expect to cut slides more often than you add them; a deck that grows every quarter because nothing ever gets retired is the same failure mode as a metrics dashboard nobody prunes.

[SENIOR MANAGER] The pipeline from a raw metric to a board slide involves more review hops than most first-time preparers expect, and skipping one of them is a common way risky language reaches a board unreviewed.

Figure 24.1 — From raw metric to board slide, as a repeating pipeline. *CONCEPTUAL.* Illustrates the review hops between a defined operational metric and a presented board slide — CISO materiality review, a conditional legal/comms pass for disclosure-sensitive language, and a rehearsal step — feeding into a follow-up log that carries into the next cycle rather than a one-way, one-time production process. Diagram ID [FIG-2401](#).

flowchart TD

A["Raw metric, defined per\nPlaybook Part 32 or this\nbook's own §3 terms"] --> B["SOC manager drafts\nrisk translation (S3)"]

B --> C["CISO review: materiality,\nranking, forward items"]

C --> D{"Language touches legal /\nregulatory exposure?"}

D -->|Yes| E["Legal / comms review\n(disclosure-sensitive wording)"]

D -->|No| F["Slot into deck skeleton\n(TMPL-2401)"]

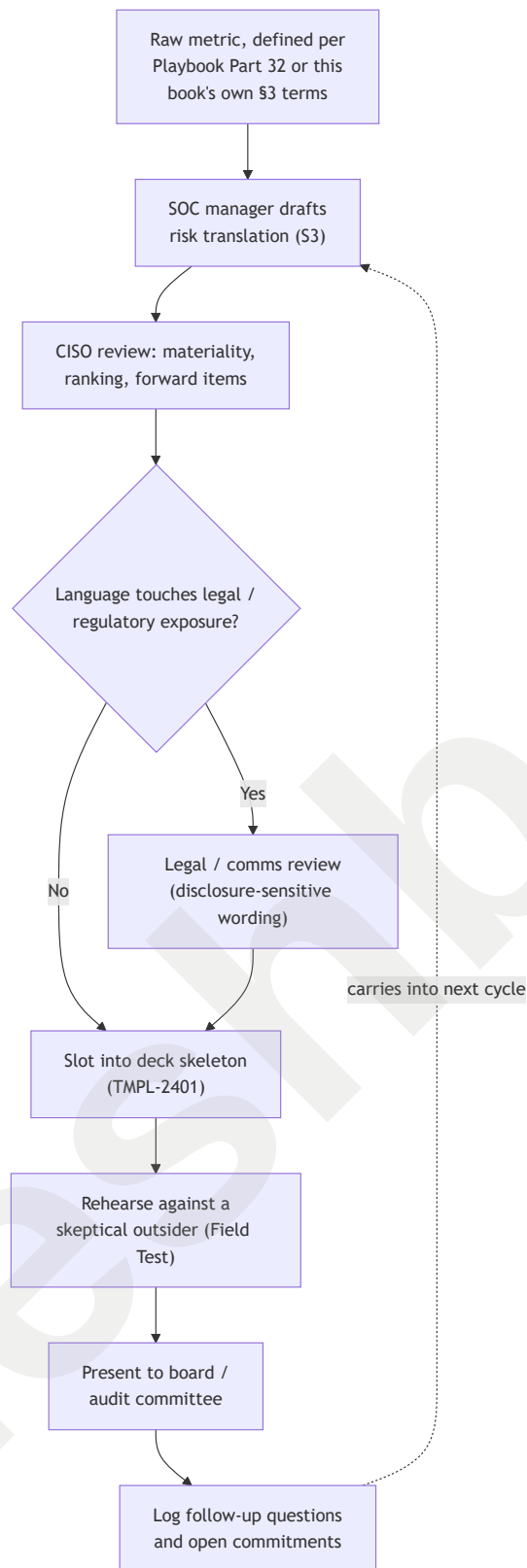
E --> F

F --> G["Rehearse against a\nskeptical outsider (Field Test)"]

G --> H["Present to board /\naudit committee"]

H --> I["Log follow-up questions\nand open commitments"]

I -->|carries into next cycle| B



Field Test Setup: A draft board deck exists, one week before the actual meeting. **Action:** Hand it to someone outside the SOC — a peer manager, the CISO's chief of staff, legal — with no context beyond what's on the slides, and ask them to raise the single hardest question they can think of after reading it once. **Expected result:** You should be able to answer that question in under 30 seconds without flipping to a slide that doesn't exist yet. If the honest answer is "let me get back to you," that's a gap to close before the real meeting, not during it.

6. Choosing what makes the cut: a metric-selection scorecard

[SENIOR MANAGER] Not every metric the weekly operational review tracks earns a board slide, and picking by instinct produces a deck that either drowns the board in detail or quietly omits the number that mattered most. Score each candidate metric against the five criteria below before it earns a slot.

Criterion	Score 0	Score 1	Score 2
Materiality	Change under 5% quarter over quarter	Change of 5–15%	Change over 15%, or any incident-linked shift
Actionability	Board has no lever that changes this	Board can influence indirectly (budget, risk acceptance)	A board decision this cycle directly changes this next cycle
Comparability	No consistent baseline yet	Baseline exists but is under two quarters old	Two or more quarters of a consistent, audited definition
Audience relevance	Purely operational (queue mechanics)	Ties to a named risk or incident	Directly answers one of \$1's three fiduciary questions
Narrative-readiness	No translation sentence written yet	Draft translation exists	Translation has survived a skeptical read (\$5's Field Test)

A metric scoring 6 or higher out of 10 belongs in the board deck; a metric scoring 3–5 belongs in the CISO's monthly review instead; anything under 3 stays in the SOC manager's own weekly operational review until it moves.

[SENIOR MANAGER] A metric's comparability score should specifically exclude comparison against an external industry benchmark, even when a board member asks for one directly. "How do we stack up against peers" is one of the single most common questions this chapter's material has to survive, and Part 31 covers, in detail, why most published benchmark reports answer that question less reliably than they look like they do — self-selected respondent pools, no shared metric definition across contributors. Have the honest, bounded version of that answer ready — what you're genuinely comparable on, what you're not, and why — rather than either refusing the question or handing over a benchmark number with no caveat.

7. Surviving the skeptical follow-up question

[EXECUTIVE] A board-ready answer to a hard question has three layers, in a fixed order: a **headline** that directly answers the literal question in one sentence, with no hedging in front of it; **evidence** — one concrete fact backing the headline, ready before the question is asked, not assembled while

answering it; and a **bound** — what you don't know yet or what's out of scope right now, plus a specific commitment to close that gap, with a name and a date attached. Skipping the bound and hoping nobody asks is how a confident answer turns into a credibility problem the moment a second question follows the first.

The table below maps four questions that come up in almost every board Q&A to this headline/evidence/bound pattern, worth rehearsing before the meeting rather than improvising during it.

Skeptical Question	Headline Answer Pattern	Evidence to Have Ready	The Honest Bound
How do we compare to peers?	State what's genuinely comparable and what isn't	Internal trend data, not a raw benchmark number (Part 31)	Name the benchmark's own limitation before the board finds it
Why should I trust this number?	State who audits or validates it, and how	The metric's definition source (SOC Playbook Part 32 for MTTR/FP rate; this part's own §3 for MTTD, or Part 9 §1 for backlog age) and its sampling or QA method (Part 15)	Flag if a definition changed recently, and whether the trend still holds under the old one
What would it take for this to happen to us?	Name the specific scenario tested, not a generic reassurance	The most recent tabletop or Field Test result	State which scenario hasn't been tested yet, and when it will be
Is spend keeping pace with the threat?	Tie spend directly to a named risk reduction, not a budget total	Part 20's cost-per-risk-reduction framing	Name what a flat or cut budget would specifically stop covering

Management Autopsy — "answer the peer-comparison question with a vendor's own benchmark number" (COMPOSITE CASE EXAMPLE, CASE-2402)

The decision: Asked by a board member how the SOC's mean time to respond compared to similar organizations, a SOC manager cited a number directly from a security vendor's annual benchmark report — "the report says the industry average is 24 hours; we're at 6" — with no further qualification.

Why it seemed reasonable: The number was favorable, it came from a named, recognizable industry report, and it directly answered the literal question in the room without requiring a follow-up meeting.

How it failed: A board member who happened to sit on another company's audit committee asked, on the spot, how the report defined "respond" and how many organizations the average was drawn from. The manager didn't know either answer. The report, like most vendor benchmark surveys, drew its respondent pool from the vendor's own customer base and left "respond" loosely defined across contributors — exactly the self-selection problem Part 31 names in detail. The favorable comparison became the least trusted number in the deck for the rest of that board's tenure, and every subsequent metric got a harder round of questions than it had before.

The fix: Answer a peer-comparison question with the organization's own trend data first, name the benchmark's sampling limitation before anyone else has to point it out, and offer the vendor number only as a loosely bounded reference point, never as the headline evidence. A comparison you can defend under one follow-up question is worth more in the room than a flattering one you can't.

What Would Change My Mind This section assumes a headline-first, three-layer answer serves a skeptical board question better than a fuller, evidence-first answer given up front. If a specific board consistently responded better to a slower walkthrough — asking for the data before the conclusion, and visibly trusting the conclusion more when it arrived last — that would be a reason to invert the pattern for that specific board, not evidence the general pattern is wrong. Boards vary enough in composition and culture that this is a default, not a law.

8. Folding in vendor and MSSP performance, and reporting an honestly bad quarter

[VENDOR/PROCUREMENT]

When part of detection or response is contracted out, a board's exposure question doesn't stop at the in-house team's numbers — a missed MSSP SLA is the organization's exposure regardless of whose payroll the analyst is on. Fold vendor performance into the same translation layer §3 builds for internal metrics rather than presenting it as a separate scorecard the

board has to reconcile on its own. "Vendor performance: 94% SLA compliance" and "in six incidents this quarter, our contracted overnight coverage was 20 to 90 minutes slower than the contract requires, and here's what we did about the two that mattered" describe the same underlying fact, and only the second is useful to a board deciding whether the contract needs renegotiating. Part 22 owns the SLA and contract-governance mechanics that produce this data; Part 23 owns the renewal-negotiation response to a pattern like this one. This part's only job is making sure the pattern reaches the board in the same risk-narrative shape as everything else in the deck, not buried on an appendix slide nobody reads.

[EXECUTIVE] Every reporting relationship eventually hits a quarter where the honest translation is unfavorable — coverage slipped, an incident wasn't caught as fast as the plan promised, a metric moved the wrong way with no good explanation yet. The three-layer pattern from §7 still applies, with one adjustment: lead with what changed and what's already in motion to fix it, not with context that reads as an excuse before the board has even heard the headline. "MTTD got worse this quarter, driven mostly by two weeks of a staffing gap we've now closed; here's what we're watching to confirm it's back on track" survives a board meeting. "A number of factors, including seasonal staffing variance and an unusually complex threat environment, contributed to some softening in our detection metrics" reads as evasive even when every clause in it happens to be true, because it delays the actual fact behind two sentences of hedging first — and a board room punishes that pattern in speech faster than a careful editor would catch it in prose.

Blind Spot A quarterly board cadence, by construction, can't see a risk that emerges and gets resolved entirely inside one quarter — the two-week staffing gap in [CASE-2401](#) above is already closed by the time the board hears about it, which is the right operational outcome and a real limit on the board's actual oversight visibility. A board relying only on the quarterly cadence to catch emerging risk is relying on the SOC manager and CISO to self-report anything that resolves faster than the reporting cycle, with no independent check on what didn't make the cut. That's a structural gap in the cadence itself, not a failure of any one report, and the only mitigation is the CISO's monthly review (§2's table) actually functioning as a real check, not a lighter copy of the quarterly deck.

9. Where this goes next

[CONCEPT] The reporting discipline built in this part feeds three later parts more than it stands alone. Part 25 covers the manager's own judgment about which risks get escalated into this reporting cycle at all, versus accepted quietly at the SOC-manager level — this part assumes that decision has already been made and only covers how the escalated item gets communicated once it arrives. Part 28 covers the compressed cadence that takes over the moment a major incident is live, replacing this part's quarterly rhythm with an hours-to-days one until the incident closes and its numbers re-enter the cycle described here. Part 29's post-incident organizational review is the actual source material behind every "incidents of note" slide this part's deck skeleton calls for — write that review well, and this part's hardest slide writes itself. And Part 30's maturity model gives a longer-horizon version of the same

board story: where this part covers one quarter's translation, Part 30 covers what "improving" looks like measured in years rather than quarters.

Cross-references. This book: Part 5 (headcount model outputs entering a board narrative when they shift materially), Part 9 (queue-health metrics, including backlog age, as board-narrative input), Part 15 (QA and calibration as part of a metric's trustworthiness), Part 18 (attrition and backfill lag behind a staffing-driven metric miss), Part 20 (the budget model a resourcing slide borrows its headline number from), Part 21 (tooling decisions behind a detection-time improvement), Part 22 and Part 23 (MSSP SLA data folded into the same risk narrative), Part 25 (the risk-acceptance judgment call upstream of what reaches this reporting cycle), Part 28 (the compressed cadence that replaces this part's rhythm during a live major incident), Part 29 (post-incident organizational review as the source of the deck's incidents-of-note slide), Part 30 (maturity narrative, a longer-horizon version of this part's board story), and Part 31 (the benchmarking caution this part leans on directly in §6 and §7). Other volumes: SOC Playbook Handbook, Part 32 — Metrics (the fixed definitional input behind this part's MTTR and false-positive-rate numbers; MTTR is defined in this part's own §3, and backlog age is Part 9 §1's definition); Detection Engineering Handbook V2, Part 38 — False Positive Engineering, Part 41 — Detection Coverage, and Part 42 — Detection Quality (the technical basis behind the false-positive and coverage rows in §3's translation table).

Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty

Why this part exists

[CONCEPT] Most of this book gives a manager a model that resolves a decision if it's run correctly: Part 5's headcount arithmetic, Part 9's decomposition of a growing queue into a staffing branch or a detection-quality branch, Part 16's three-hypothesis check for a recurring miss. Run any of those honestly, on real data, and the ambiguity usually collapses into a clear answer. This part exists for the residual cases where it doesn't — where the data is too thin, too mixed, or too slow to arrive before a decision has to be made anyway, and a manager is left holding a call with no clean model behind it. Three specific calls recur often enough to organize this part around: whether to accept a known risk at the SOC-manager level or send it up to the CISO or the board, what to do when a persistent miss won't sort cleanly into "fire the rule" or "coach the analyst," and how to make a defensible resourcing tradeoff when headcount is fixed and the queue keeps growing anyway. Every case study in this part is deliberately one where the right answer was genuinely unclear at the time it had to be made — not a diagnostic problem in disguise, and not a case where the manager was careless and a clean model would have caught it.

[CONCEPT] This part does not re-run any of the diagnostics that already exist elsewhere in this book or its companion volumes, and it's worth being exact about the boundary before anything else, because this is the part of the book most likely to drift into re-explaining material another chapter already owns. It does not re-decompose a growing queue into a staffing gap versus a detection-quality problem — Part 9 — Queue Health & Workload Management already builds that decomposition and explicitly hands off the cases where it doesn't resolve cleanly to this part's §4. It does not re-run the three-hypothesis check for a recurring miss — Part 16 — Performance Management & Coaching already builds skill-gap-versus-training-gap-versus-broken-process as a decision tree, and hands off the cases where that check stays genuinely unresolved to this part's §3. It does not teach how to tune a noisy detection rule or score a backlog of untuned rules as organizational debt — Detection Engineering Handbook V2, Part 38 — False Positive Engineering and Part 43 — Detection Debt own both, and this part cites them rather than re-deriving either. And it does not score how severe any single alert is — SOC Playbook Handbook, Part 29 — Playbook Severity Model owns that, at the level of one ticket; this part's risk classification in §2 operates one level up, at the level of a standing decision about an accepted gap, not a single alert's score.

[CONCEPT] What's left, once those boundaries are drawn, is the actual judgment call: what a manager does once the cheapest, fastest diagnostic test has already been run and the answer still isn't clean. That's a different skill than running the diagnostic well, and it's the skill this part is built to teach.

1. Why this chapter is different: judgment calls a checklist can't finish

[CONCEPT] A diagnostic works when the signal is strong enough to separate competing explanations cleanly. Part 9's decomposition works when a queue's growth concentrates in one or two rules with an

obviously abnormal false-positive rate, or spreads broadly enough that no single rule dominates — either result points somewhere specific. Part 16's three-hypothesis check works when a rule's team-wide error rate is either clearly at baseline (pointing at the individual) or clearly elevated for everyone who touches it (pointing at the rule). Both diagnostics fail quietly, not loudly, when the underlying signal doesn't cooperate: a sample too small to trust, a rule that's moderately noisy for everyone and additionally worse for one specific analyst, a queue that's growing for reasons that trace to a real capacity gap the organization has no near-term ability to fund. None of those are diagnostic failures in the sense of a broken model — they're cases where a correctly run diagnostic returns "both, partially, with real uncertainty remaining" instead of a clean branch.

[SENIOR MANAGER]

It's worth naming the difference between two things that feel identical in the moment but call for opposite responses. The first is decision paralysis dressed up as diligence: a manager who could get a clean answer by waiting two more weeks for a larger sample, or by running a query that already exists but hasn't been pulled yet, and instead treats the ambiguity as permanent because acting now is uncomfortable. The second is genuine irreducible ambiguity: the data that would resolve the question doesn't exist yet at any price the situation can afford, or won't exist until well after a decision has to be made regardless. The first case has a cheap fix — go get the data, per Part 9's and Part 16's own diagnostics — and treating it as this chapter's territory is itself a mistake worth catching early. The second case is what this part actually addresses, and the table below is the fast check for telling them apart before assuming either.

The table below is a working filter — run it before treating any specific decision as this part's territory rather than a diagnostic that just hasn't been run yet.

Signal	Decision paralysis (a diagnostic that hasn't been run)	Genuine irreducible ambiguity (this part's territory)
The missing evidence	Exists somewhere already — a query away, a report someone already has	Doesn't exist yet at any price this decision can afford, or won't exist for longer than the decision can wait
Time to resolve	Days, if someone just runs the check	Weeks to quarters, or never, if the underlying signal is genuinely this noisy
What waiting costs	Nothing beyond mild discomfort — the answer is sitting there	A real, accruing cost — an aging queue, a live risk, a person sitting under an unresolved cloud
The honest fix	Run Part 9's decomposition or Part 16's three-hypothesis check properly, then decide	Make the best defensible call now, with an explicit review trigger — §5 builds this discipline

Blind Spot A manager under real time pressure can mistake the second row for the first in either direction — either treating a genuinely thin, irreducible signal as if one more query would resolve it (and delaying a decision that's actually ready to be made), or treating a diagnostic that was simply never run as if it were this chapter's territory (and skipping a cheap check that would have resolved things cleanly). The check above only works if it's applied honestly before the decision, not used after the fact to justify whichever choice already felt easier.

2. Risk acceptance: whose signature actually belongs on this

2.1 The dimensions that decide who signs

[SENIOR MANAGER] Part 1 — SOC Manager Foundations & the Series Map previews the shape of a risk-acceptance call and the escalation path it follows as scope and reversibility grow, without building the mechanics — that's this section's job. A risk-acceptance decision is not a severity score. SOC Playbook Handbook, Part 29 — Playbook Severity Model scores how bad a single alert is if it turns out to be real; a risk-acceptance call is a standing decision about a known, ongoing gap — a coverage hole, an unpatched system running longer than policy allows, a vendor's SLA shortfall — that doesn't resolve when one ticket closes, and stays open until someone with the authority to do so either fixes it, accepts it explicitly, or escalates it to someone whose authority is bigger than theirs.

[SENIOR MANAGER] Four dimensions decide, in practice, whether a specific risk belongs at the SOC-manager's own level or needs to go up: how big the exposure is in dollar terms if it materializes; how reversible the situation is once it does (can it be undone within days, or does it become a permanent fact, like a data exposure that's already happened); how bounded the blast radius actually is, not just how bounded it looks on a network diagram; and whether the acceptance is a one-time, short-lived call or a standing policy the organization will be living with for months. None of the four is sufficient alone — a small-dollar risk with an unbounded, unpredictable blast radius can outrank a large-dollar risk that's cleanly contained, and a manager who scores only the dollar figure will systematically under-escalate the second kind.

2.2 A risk-acceptance scorecard

[SENIOR MANAGER] The table below is a working scorecard for sorting a specific risk-acceptance decision toward the right owner before it gets accepted, delayed, or escalated by default — use it at the moment a known gap is first identified, not months into living with it.

Table 25.1 — Risk-acceptance scorecard. Run a specific, real risk against all four rows before deciding who signs off on accepting it; a risk that lands in the right-hand column on any single row is a strong candidate for escalation even if the other three rows look manager-level.

Dimension	Manager-level signal	Board/CISO-level signal
Dollar exposure if it materializes	Below the organization's own defined manager-authority threshold (a specific number set in advance, not judged case by case)	At or above that threshold, or genuinely unknown and unbounded
Reversibility	Contained and correctable within the current budget cycle if it goes wrong	Permanent or slow to correct — a data exposure, a regulatory finding, a client contract breach
Blast radius, validated not assumed	Confirmed bounded by an active, currently verified compensating control	Assumed bounded by a control nobody has re-checked recently, or genuinely broad
Duration and standing nature	A short, named window with a scheduled end (a migration, a vendor fix already in flight)	An indefinite acceptance with no real end date, or a repeat of a risk already accepted before

[EXECUTIVE] The third row is the one most manager-level risk-acceptance decisions get wrong not at the moment of the decision but months later, and §2.4 covers exactly that failure mode in the context of a real case. A compensating control that was genuinely verified and bounded on the day a risk was accepted doesn't stay that way automatically — it stays that way only if something checks it again, on a schedule, independent of whether the original risk-acceptance decision ever gets revisited.

2.3 Worked example: the legacy file-cluster acceptance

COMPOSITE CASE EXAMPLE — CASE-2501. Constructed from recurring patterns in how mid-market organizations carry a legacy-system coverage gap through an extended decommission timeline; no single organization is identifiable, and all figures below are illustrative.

[SENIOR MANAGER] A 22-analyst in-house SOC at a mid-market insurer ran a legacy on-premises file-share cluster still hosting roughly 35% of policy-processing documents, originally scheduled for decommission in six months but delayed to 14 months by a vendor migration slip outside the SOC's control. The cluster's operating system was too old for the organization's standard EDR agent, leaving only network-level telemetry (NetFlow and firewall logs) and forwarded Windows Security event-log data — no process-level visibility, meaning staged ransomware activity or lateral movement on that specific cluster would not trip any of the SOC's primary host-based detections. Building an interim compensating capability — a temporary jump-box architecture with its own logging appliance — was estimated at \$85,000 and roughly two months of engineering time, for an asset with 14 months of remaining life. A worst-case exposure estimate for a full compromise of the cluster, including regulatory notification and remediation cost, ran to roughly \$1.2 million — a figure well above this organization's own manager-authority threshold, set at \$250,000 for a bounded, correctable risk.

[SENIOR MANAGER] The manager accepted the risk at the SOC-manager level rather than escalating it, reasoning that the \$1.2 million worst-case figure assumed no compensating control at all, and the cluster sat behind a network segmentation boundary that had been specifically verified — a firewall-rule review confirming only two narrow, named services could reach it from the rest of the environment — at the time the decision was made. Against that verified, bounded blast radius, the manager judged the realistic exposure closer to a contained incident than the unmitigated worst case, and treated the \$85,000 acceleration cost as not worth spending on an asset with a defined, short remaining life. That call was defensible on the day it was made: it used Table 25.1's dimensions correctly, it named its own reasoning in writing, and it treated the segmentation boundary as the load-bearing fact rather than an assumption.

[SENIOR MANAGER] Six months into the 14-month window, a contractor's compromised laptop attempted to pivot toward the file-cluster's segment and was stopped cleanly by the segmentation boundary — on its face, validation that the manager-level acceptance had been the right call. The post-incident review surfaced a second fact that had nothing to do with the incident itself: two months after the original acceptance, a vendor-integration request had opened a new firewall exception into that same segment, approved through a routine change-control process that had no linkage back to the original risk-acceptance decision and no requirement to notify whoever owned it. The exception happened to be narrow enough that this particular incident's vector didn't use it. Nothing about the original decision was wrong — the dimensions were read correctly, the compensating control was real and verified at the time. What was missing was a mechanism that would have caught the fact that the

control's own boundary had quietly moved two months into a 14-month acceptance window, independent of whether anything went wrong through it.

What Would Change My Mind This section treats CASE-2501's outcome as evidence that a verified compensating control needs a standing re-check, not evidence that the original manager-level acceptance was itself the wrong call. If a rigorous review of similar cases found that firewall or segmentation exceptions opened after a risk acceptance rarely widened blast radius in practice — that the near-miss in CASE-2501 was a rare coincidence rather than a common pattern — that would weaken the case for a mandatory re-check cadence and argue for treating this as an acceptable, low-probability residual risk instead of a standing gap worth a recurring process.

2.4 The default-acceptance trap

[SENIOR MANAGER] Part 1 names the failure mode this section builds on directly: a risk that never gets formally accepted by anyone still gets accepted by default the moment nobody stops the process carrying it. CASE-2501 shows the same failure mode one layer deeper — a risk that *was* formally accepted, correctly, at the moment of acceptance, can still slide back into the same default-acceptance trap afterward if nothing checks whether the specific fact the acceptance depended on is still true. The signature on a risk-acceptance memo is a snapshot of a judgment made against a specific set of facts on a specific day; treating that signature as durable proof the risk remains correctly sized months or years later is the mistake, not the original signature itself.

[EXECUTIVE] The practical fix costs little relative to the exposure it protects: every risk accepted under Table 25.1's dimensions gets a named re-check date, not just a named owner, and the re-check specifically re-verifies the fact the acceptance depended on — the control's boundary, the exposure estimate, the remaining duration — rather than simply confirming the memo still exists on file. The risk-acceptance memo template ([TMPL-2501](#)), filed in Appendix A7 alongside Part 24's board-reporting material, carries a dedicated re-check-date field for exactly this reason; it does not, on its own, make anyone actually run the re-check on schedule, which is a standing-cadence discipline this section's §5.3 returns to rather than something a template field can enforce by itself.

3. When Part 16's diagnostic doesn't resolve cleanly

3.1 Where the three-hypothesis check stalls

[SENIOR MANAGER] Part 16 §3 builds a fast, cheap check for a recurring miss: compare the flagged analyst's error rate on a specific alert type against the team-wide baseline for that same alert type. A clearly elevated team-wide rate points at the rule; a rate at baseline for everyone except the flagged analyst points at the person. That check stalls in three specific, recurring ways, and naming them is what separates "the diagnostic hasn't been run yet" from "the diagnostic was run correctly and the signal is genuinely mixed."

[SENIOR MANAGER]

The first stall is sample size: a low-volume alert type might give one analyst only a dozen or so sampled tickets in a quarter, which isn't enough to distinguish a real elevated error rate from ordinary noise with any real confidence. The second is a genuinely mixed signal: the rule's team-wide rate is elevated over the team's overall baseline — real evidence the rule deserves attention — while one specific analyst's rate is elevated again over that already-elevated team rate, meaning both explanations are true at once rather than one ruling out the other. The third is time pressure: the queue is bleeding SLA on this exact alert type today, and waiting for a full additional sampling cycle to get a cleaner read, the fix Part 9 and Part 16 both recommend when the data is thin, isn't available because the decision can't wait that long.

Cross-Book Pointer *This section does not cover how a detection's false-positive rate gets measured, tuned, or prioritized in a debt backlog once it's confirmed to need attention — that's Detection Engineering Handbook V2, Part 38 — False Positive Engineering for the tuning mechanics and Part 43 — Detection Debt for how a backlog of noisy, untuned rules gets tracked and prioritized as a standing liability. This part's job stops at recognizing that a mixed signal needs a decision now, not at doing the tuning work — file the rule to detection engineering regardless of how the individual-coaching question resolves, per §3.3 below.*

3.2 Worked example: the genuinely-both case

COMPOSITE CASE EXAMPLE — CASE-2502. Constructed from recurring patterns across MSSP and in-house SOC's where a rule's own noise and one analyst's individual gap compound on the same alert type; no single organization is identifiable, and all figures below are illustrative.

[SENIOR MANAGER]

A 14-analyst MSSP SOC's quarterly QA sample flagged a cloud-identity impossible-travel detection with a team-wide false-positive rate of 46% against a team baseline of 18% for other detections — clearly elevated, and squarely the kind of finding Part 16 §3.5 routes to detection engineering as its own item. It was also true, in the same sampling cycle, that one specific analyst with two years of tenure and a previously strong QA record showed a personal error rate of 64% on that same alert type, worked over 22 sampled tickets that quarter, against teammates whose rates on the identical rule ranged from 40% to 51% across sample sizes of 12 to 19 tickets each. Both facts were real at once: the rule genuinely deserved a tuning ticket, and this one analyst's rate sat meaningfully above every teammate working the identical noisy rule — evidence of a second, separate contribution stacked on top of the rule's own baseline noise, not proof that either explanation alone accounted for the gap.

Management Autopsy — "wait for a clean signal before touching the coaching question"

The decision: *Rather than open any coaching conversation with the flagged analyst, the manager chose to wait for a second full quarterly sampling cycle, reasoning that a larger sample would separate the rule's contribution from the individual's more reliably than the current, smaller one could.*

Why it seemed reasonable: *Part 16 §3.2 is explicit that a manager's instinct defaults to blaming the person, and that the process explanation should be checked first and taken seriously. Waiting for cleaner data before coaching someone whose numbers might be entirely explained by a broken rule looked like exactly the discipline that section recommends, not an avoidable delay.*

How it failed: *The extra quarter cost three more months during which the same avoidable error recurred on live, high-severity tickets, not just the ambiguous ones the sample happened to catch. One of them was a real account-takeover attempt miscalled as a false positive, caught only 11 days later when the client's own fraud team flagged suspicious account activity independently — a genuine near-miss that cost roughly 40 hours of incident-response time at a blended \$150 an hour, plus the reputational exposure of a client-side team catching what the SOC's own sampling had missed. When the rule was finally retuned three months on, the team's rates dropped in line with the new baseline — and the flagged analyst's rate stayed elevated at roughly 50%, confirming the second, individual contribution the manager had waited a full quarter to investigate at all.*

The fix: *Detection debt and an individual coaching question are not mutually exclusive hypotheses that have to be resolved in sequence — file the rule to detection engineering's backlog immediately, regardless of how confident the individual read is, and open a bounded, explicitly non-disciplinary coaching cycle with the flagged analyst in parallel, with a shared review date once a larger sample exists to reassess both signals together. §3.3 builds this as a standing pattern rather than a one-off improvisation.*

3.3 The dual-track holding pattern

[SENIOR MANAGER] The fix CASE-2502 arrived at late is reusable earlier, as a default response the moment a signal reads as genuinely mixed rather than waiting for it to resolve into one clean branch. Running both tracks costs little relative to the cost of running neither, and the two tracks don't compete for the same resource or the same person's attention.

[SENIOR MANAGER] The rule-side track starts immediately and unconditionally: file a debt-backlog item with detection engineering the moment the team-wide rate reads elevated, independent of whatever the individual signal eventually shows. A tuning fix that turns out to have been unnecessary once the individual explanation resolves costs a small amount of wasted engineering triage time; a tuning fix that never gets filed because the individual question was still open costs however long that rule keeps producing the same miscalls in whoever works it next. The coaching-side track opens in parallel, not

after, but explicitly framed to the analyst as provisional and non-disciplinary — this is not a Performance Improvement Plan, it's a bounded check-in specifically because the rule is already known to be part of the picture, with a stated shared review date (commonly four to six weeks, or whatever the next reasonable sampling window actually is) at which both tracks get reassessed together against a larger sample.

Figure 25.1 — The dual-track holding pattern for a genuinely mixed diagnostic signal.

CONCEPTUAL. Illustrates the parallel-track sequence described above — a rule-side debt filing that starts unconditionally and a coaching-side track that opens provisionally, both converging on a shared review date rather than being sequenced one after the other. It is a process diagram of the decision pattern, not a capture of any specific case-management tool's workflow. [FIG-2501](#).

flowchart TD

A["Mixed signal confirmed:\nteam-wide rate elevated AND\nnone analyst's rate elevated\nabove that team rate"] --> B["Rule-side track (starts now,\nunconditionally)"]

A --> C["Coaching-side track (starts now,\nframed explicitly as provisional\nand non-disciplinary)"]

B --> D["File debt-backlog item with\ndetection engineering --\nDEH Part 38 / Part 43"]

C --> E["Bounded check-in with the analyst;\nno PIP language, no formal\ndocumentation escalation yet"]

D --> F["Shared review date\n(commonly 4-6 weeks, or the next\nreal sampling window)"]

E --> F

F --> G{"Larger sample now available --\ndoos each track's signal hold?"}

G -->| "Rule retuned, analyst's\nrate normalizes with it" | H["Close both tracks;\nno individual gap confirmed"]

G -->| "Rule retuned, analyst's\nrate stays elevated" | I["Individual gap confirmed --\nroute to Part 16 Sec 4 (PIP path)\nwith the now-larger sample as evidence"]

G -->| "Rule still not retuned" | J["Escalate the debt-backlog item;\nhold the coaching track open\nand provisional, not escalate it"]

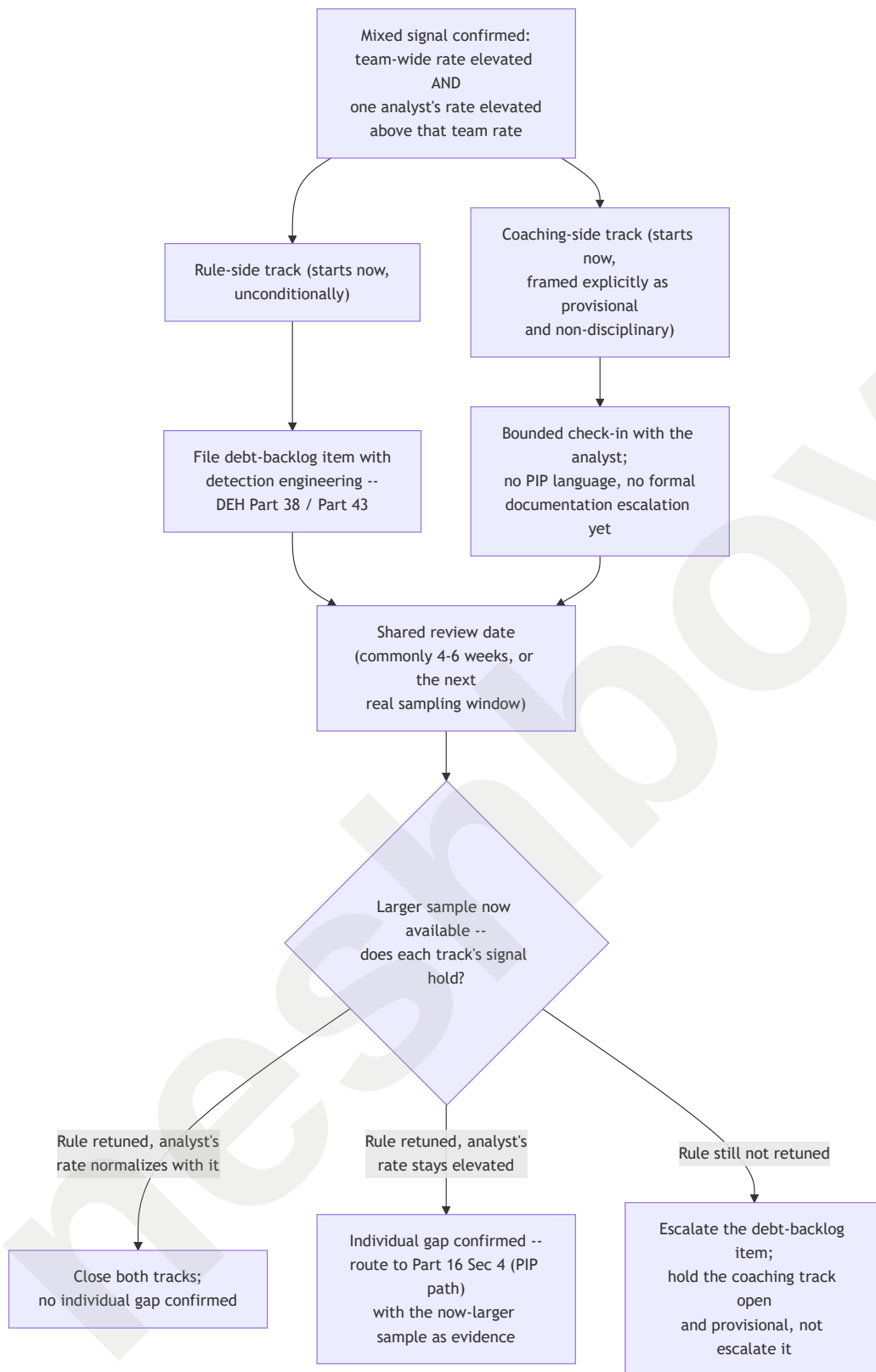


Figure 25.1 — The dual-track holding pattern for a genuinely mixed diagnostic signal. CONCEPTUAL. Shows the dual-track sequence above as a rendered flowchart. It supports this section's claim that the rule-side and coaching-side tracks should run in parallel rather than in sequence, and gives the shared review date in CASE-2502's fix a concrete decision point to resolve against once a larger sample exists. FIG-2501.

What Would Change My Mind This section recommends running the rule-side and coaching-side tracks in parallel by default whenever a signal reads as genuinely mixed, rather than sequencing detection engineering's fix first and coaching second. If analysts in a specific team culture routinely reported that a parallel, explicitly non-disciplinary coaching track still felt indistinguishable from a PIP regardless of how it was framed — that the framing failed to land, not just occasionally but as a consistent pattern — that would undercut the parallel-tracks recommendation for that team specifically, and sequencing (rule fix first, coaching only once the rule's contribution is fully known) would be the more defensible default there instead.

4. Resourcing tradeoffs when headcount is fixed and the queue keeps growing

4.1 What "fixed headcount" actually means on a manager's calendar

[SENIOR MANAGER] Part 9 §3.5 covers the case where a queue's growth is genuinely both a detection-quality problem and a real underlying capacity gap, and hands off a specific question to this part: a resourcing decision made under exactly that kind of ambiguity, when the honest answer is "we don't fully know yet and the queue is breaching SLA today." This section covers a related but distinct case, where the diagnosis itself has already resolved cleanly — Part 9's decomposition confirms a genuine, broad-based capacity gap, not a rule problem — and the ambiguity sits entirely on the resourcing side: the organization has a hiring freeze, a budget cycle that won't reopen for months, or a headcount ceiling set by a decision several layers above the SOC manager, and none of that changes what the queue is actually doing in the meantime.

[SENIOR MANAGER] This is a genuinely different kind of uncertainty than §3's mixed diagnostic signal. Nothing about the underlying cause is unclear here — the model in Part 5 and the decomposition in Part 9 both point the same direction. What's unclear is which of several imperfect options is the least-bad way to absorb a confirmed gap for as long as the organizational constraint holds, and reasonable managers looking at the same numbers can defensibly pick different options.

4.2 A tradeoff menu, and what each option actually costs

[SENIOR MANAGER] The table below compares the standing options for absorbing a confirmed capacity gap when the headcount lever itself is unavailable — not a menu of equally good choices, but a working comparison of what each one buys, what it quietly costs, and the condition that makes it defensible rather than just convenient.

Table 25.2 — Resourcing-tradeoff options when headcount is frozen. Use this table once Part 9's decomposition has already confirmed a genuine capacity gap; it does not apply to a detection-quality problem wearing a staffing shortfall's clothing, which Part 9 §3 should have already ruled out before this table is relevant.

Option	What it buys	What it actually costs	Defensible when...
Reduce QA sampling rate, freeing reviewer hours for direct triage	Immediate capacity, no new spend	Slower detection of a real quality regression — Part 15's pattern-threshold math simply takes longer to accumulate at a lower sample rate	A hard floor per alert type keeps any single detection from going fully dark, and there's an explicit trigger to restore the rate once the constraint lifts
Temporarily relax the SLA target for the lowest severities	Breathing room without touching anyone's actual workload	Aged low-severity backlog can mask an early-stage signal that only looks low-severity in isolation, per Part 9 §1's warning about severity-sorted views	The aging distribution is still tracked and reported even while the target itself is relaxed, not quietly stopped watching
Reallocate training and coaching time to direct queue work	Immediate capacity, no new spend	Ramp curves slow and skill development stalls — Part 11 and Part 12's territory, with a lagged cost that shows up later, not now	The reallocation is short (weeks, not a standing policy) and reversed the moment volume normalizes
Fund temporary contractor or MSSP overflow from a non-headcount budget line	Capacity without touching the frozen headcount line at all	Usually a higher per-unit cost, real ramp-up drag on a short engagement, and a risk of quietly becoming a permanent workaround that masks the headcount case that should be reopened at the next budget cycle	A burst-capacity contract clause already exists (Part 22) and everyone involved treats it as explicitly temporary, not a substitute for the requisition
Let backlog age and SLA breaches accumulate as the visible, honest signal	No immediate cost or reallocation at all	Real, accruing risk sitting in the queue, and a real customer or regulatory consequence if a breach lands on the wrong ticket	The breach pattern is actively tracked and reported upward (Part 24) as the evidence for reopening headcount, not silently absorbed without anyone above the SOC manager seeing it

Operational Reality A budget cycle and a queue do not run on the same clock, and a headcount freeze doesn't pause the arrival rate while it's in effect. The gap between "the org can't fund this until next quarter" and "the queue needs this today" is exactly the space every row in Table 25.2 lives in — none of the five options actually closes the gap Part 5's model says exists; each one is a different way of deciding which specific cost the organization absorbs while it waits for the lever it actually needs.

4.3 Worked example: cutting QA sampling to save the queue

COMPOSITE CASE EXAMPLE — CASE-2503. Constructed from recurring patterns in how a fixed-headcount SOC absorbs a confirmed, unfunded capacity gap; no single organization is identifiable, and all figures below are illustrative.

[SENIOR MANAGER]

A 12-analyst in-house SOC at a health-tech company saw arrival rate rise 15% over two quarters, broad-based across dozens of alert types with no single rule concentrating the increase — Part 9's decomposition ruled out a detection-quality explanation cleanly, confirming a genuine capacity gap. A company-wide hiring freeze, unrelated to the strength of the SOC's own headcount case, meant the requisition Part 5's model would have justified had no path to approval for at least two quarters.

CONCEPTUAL SAMPLE -- illustrative arithmetic, not sourced benchmark data

Baseline ticket volume:	~2,400 tickets/month
QA sample rate before the cut:	6% (~144 reviewed tickets/month), low/medium severity
QA sample rate after the cut:	3% (~72 reviewed tickets/month), low/medium severity only (High/Critical sampling held at 100%, untouched)
Reviewer time freed:	~72 fewer reviews x 20 min/review = ~1,440 minutes = ~24 reviewer-hours/month reallocated to direct triage

[SENIOR MANAGER]

The manager picked a blend from Table 25.2: cutting the QA sample rate on low- and medium-severity tickets from 6% to 3%, freeing roughly 24 reviewer-hours a month for direct queue work, while holding High/Critical sampling at 100% specifically because that's where a missed error costs the most. The reasoning at the time was defensible — the cut fell only on the tail the organization could most afford to see less of, not on the highest-consequence tickets.

Management Autopsy — "cut the QA sample rate, not the SLA target"

The decision: Reduce QA sampling on low/medium-severity tickets from 6% to 3%, reallocating the freed reviewer time to direct triage, while leaving High/Critical sampling untouched.

Why it seemed reasonable: Of every option on Table 25.2, this one appeared to concentrate its cost on the lowest-consequence slice of the queue, protected the highest-severity tickets completely, and required no new spend during a freeze that made new spend impossible anyway.

How it failed: A real, under-tuned but medium-classified detection was already producing a quiet quality regression before the cut. At the 6% rate, Part 16 §2.2's pattern threshold of three flags in a rolling window would likely have been crossed in roughly four to five weeks; at the 3% rate, the smaller sample took an estimated 11 weeks to accumulate the same three flags. The regression itself wasn't caused by the sampling cut — but the cut genuinely widened the detection window on exactly the kind of finding QA sampling exists to catch. No controlled counterfactual exists to prove the alternative would have gone better: it's equally plausible that without the reallocated hours, the queue itself would have missed something directly through sheer backlog aging. What's true either way is that this specific downside — slower QA detection latency — is the one that materialized, and the manager can defend the decision's process without being able to claim the outcome was risk-free.

The fix: Rather than treating the delayed detection as proof the sampling cut was wrong, the manager added a floor beneath it — a minimum sample size per alert type per month, regardless of the overall rate, so no single detection can go fully dark under the reduced rate — and set an explicit trigger to restore full sampling the moment the freeze lifted or headcount arrived. The cut became a named, time-boxed acceptance of a specific risk (slower QA detection latency on the low/medium tail) rather than a silent, indefinite policy change nobody had agreed to in those terms.

5. A decision discipline for calls that stay genuinely unclear

5.1 Four questions before the decision, not after

[SENIOR MANAGER] Part 1 §3.3 builds a three-question check for auditing a staffing or incentive *model* before trusting it with a decision. This section's job is narrower and comes later: not auditing a model in advance, but making a live, uncertain call — a risk acceptance, a dual-track holding pattern, a resourcing tradeoff — in the moment, when no more model-building is going to remove the remaining uncertainty before the decision has to be made. Four questions, asked at the moment of the decision rather than reconstructed afterward, do most of the work.

1. **What is the smallest, most reversible version of this decision available right now?** CASE-2503's manager chose a cut with a built-in floor and an explicit restore trigger rather than an

open-ended policy change — the same underlying tradeoff, made reversible by design rather than by luck.

2. **What specific, checkable evidence would resolve the ambiguity, and by when can it realistically arrive?** This is the question that separates §1's two rows — if the honest answer is "a query away, this week," the decision isn't ready to be made yet; if the honest answer is "not for a full sampling cycle, and the queue can't wait that long," the decision belongs in this part's territory.
3. **Has the reasoning been written down now, not reconstructed later?** CASE-2501's manager could show, months after the fact, exactly what Table 25.1 dimensions the original acceptance had weighed and why — which is precisely what let the post-incident review separate "the original call was reasonable" from "something changed afterward that nobody caught," instead of collapsing both into a single verdict on the decision itself.
4. **Who reviews this on a calendar date, regardless of outcome — not "if it becomes a problem"?** CASE-2501's near-miss traces directly to the absence of this question at the moment of acceptance; §5.3 builds the standing version of this discipline rather than leaving it to each individual decision to remember on its own.

5.2 Separating decision quality from outcome luck

[SENIOR MANAGER] CASE-2501 and CASE-2503 share a structure worth naming directly: in both, the decision was reasonable and documented against what was knowable at the time, and the outcome still carried a real downside that the manager couldn't have fully prevented without spending resources the situation didn't justify spending. Grading either decision by its outcome alone — "the segmentation nearly failed to hold, so the original acceptance was wrong" or "QA missed something for eleven weeks, so the sampling cut was wrong" — collapses two separate questions into one and produces the wrong lesson both times. The question that should drive whether a manager repeats a decision next time isn't "did this specific instance turn out fine," it's "was this the best call available given what was actually knowable at the time, and does the same call still look right applying the same standard today."

[SENIOR MANAGER] That distinction cuts in a direction managers under pressure to look decisive often resist: a bad outcome from a genuinely good decision doesn't automatically indict the decision, and a good outcome from a genuinely reckless decision doesn't retroactively vindicate it. What a bad outcome should always do is update the standing process around the decision — the re-check cadence in §2.4, the dual-track default in §3.3, the sampling floor in §4.3 — even when the original call itself holds up under review. Confusing "the decision was sound" with "nothing needs to change" is its own failure mode, distinct from blaming a sound decision for an outcome it couldn't fully control.

Manager's Note *When a risk-acceptance call goes wrong, resist writing the post-incident narrative around whoever signed the original memo. Write it around what changed between the day of the signature and the day it mattered, and whether anything was watching for that change. In most cases with CASE-2501's shape, the honest answer isn't "the wrong person accepted this" — it's "nobody was still checking."*

5.3 Reviewing accepted risk on a cadence

[EXECUTIVE] A single accepted risk with a named re-check date is a good decision. A standing register of every currently accepted risk, reviewed together on a fixed cadence rather than only when an individual memo's own date happens to come up, is what actually catches the CASE-2501 pattern before a near-miss does it instead. A quarterly pass through the open risk-acceptance register — every item still open, its original Table 25.1 dimensions, and whether the specific fact it depended on is still true — belongs on the same reporting cycle Part 24 — Executive & Board Reporting already uses for material operational changes, not as a separate, easily skipped exercise nobody's calendar protects.

[EXECUTIVE] The material outcome of that quarterly pass is usually short: most accepted risks are still exactly what they were when accepted, and confirming that costs little. The value is in the exceptions the pass catches — a compensating control that's quietly changed, a "temporary" resourcing tradeoff from §4 that's drifted past its own restore trigger with nobody noticing, a risk whose dollar-exposure estimate is stale because the underlying system's footprint grew. None of those exceptions are dramatic on the day they're caught. Each one is exactly the kind of fact CASE-2501's near-miss shows going unwatched for months at a time when nothing forces a review.

6. Where this goes next

[CONCEPT] This part picked up three specific handoffs — Part 1's risk-acceptance shape, Part 9's diagnosed-but-unresourceable capacity gap, and Part 16's genuinely mixed diagnostic signal — and built the discipline each one needed once its own model or decomposition had done everything a model can do. None of the three cases in this part resolve into a formula the way Part 5's headcount arithmetic or Part 15's sampling math do, and that's the honest point of the chapter: some of a SOC manager's real decisions never get a formula, and pretending otherwise produces false confidence rather than a better decision. What does transfer from case to case is the discipline in §5 — bias toward the reversible option, name what evidence would resolve the ambiguity and by when, write the reasoning down now, and put a real date on the calendar to check again regardless of how things look today. Nothing past this point in the book should treat "the diagnostic didn't fully resolve" as a dead end; it's the exact moment this part's discipline is built to take over.

Cross-references

Within this book: Part 1 — SOC Manager Foundations & the Series Map (the risk-acceptance escalation shape this part's §2 builds out into a full scorecard); Part 5 — Headcount & Capacity Modeling (the fixed-headcount constraint §4's tradeoff menu operates against); Part 9 — Queue Health & Workload Management (the queue-growth decomposition that hands off both the genuinely-both case and the resourced-but-unresourceable case to this part's §4); Part 15 — Quality Assurance Programs (the sampling math §4.3's tradeoff and CASE-2503's floor are built on); Part 16 — Performance Management & Coaching (the three-hypothesis diagnostic this part's §3 picks up once it stalls, and the pattern-threshold math CASE-2502 and CASE-2503 both reference); Part 18 — Attrition & Retention (the cost of a coaching track that goes on too long without resolving, relevant to §3.3's dual-track framing); Part 20 — Building & Defending the SOC Budget (the budget-cycle constraint behind §4's frozen-headcount scenario); Part 22 — MSSP & Managed-Service Contract Management (the burst-capacity contract clause Table 25.2's contractor-overflow row depends on already existing); Part 24 —

Executive & Board Reporting (the reporting cadence §2.4 and §5.3's standing risk-register review rides on); Appendix A7 — Executive, Board & Risk-Acceptance Templates (the risk-acceptance memo template, [TMPL-2501](#), this part's §2.4 assumes as the durable record of a decision made under this chapter's discipline).

SOC Playbook Handbook: Part 29 — Playbook Severity Model, for the per-alert severity score this part's decision-level risk classification in §2.1 sits one level above and does not redefine.

Detection Engineering Handbook V2: Part 38 — False Positive Engineering, for the tuning mechanics behind the rule-side track in §3.3's dual-track holding pattern; Part 43 — Detection Debt, for how a confirmed noisy rule gets tracked and prioritized as a standing backlog item once §3's diagnostic points that direction, independent of whatever the individual coaching question resolves to.

Part 26 — Cross-Team Politics & Stakeholder Alignment

Why this part exists

[CONCEPT] Part 4 — Organizational Placement & Charter built the charter and the standing RACI: a written answer, signed by every named party, to who is Responsible, Accountable, Consulted, and Informed across the handful of activities where the SOC's work and another team's work meet. That document is necessary and not sufficient. A RACI cell that names IT operations as Responsible and Accountable for patching doesn't make an infrastructure director's Friday change freeze disappear the week a critical finding needs a Tuesday patch. A charter clause naming AppSec as a consulted party on a production code vulnerability doesn't stop AppSec from treating the finding as the SOC's problem because it surfaced in a SOC-owned alert queue instead of AppSec's own scanner output. The signed agreement is the starting position in a negotiation that recurs, in some form, most weeks the SOC operates — not a settled question that stops being asked once the ink is dry.

This part covers the eight standing counterpart relationships named in this book's scope — IT operations, engineering, legal, privacy, physical security, and three internal security sub-teams (AppSec, identity and access management, and vulnerability management) that outsiders, and sometimes a new SOC manager, assume share the SOC's incentives simply because they share the word "security" — plus the three friction points that recur across all eight regardless of which specific counterpart is involved: who owns remediation once a finding is confirmed, who gets paged first when an incident spans more than one team's territory, and whose tooling or scoring system wins when two teams disagree about the same finding. It closes with the "shadow SOC" problem: what happens when a frustrated business unit stops waiting for any of this to work and quietly builds its own monitoring instead.

This part does not re-derive the charter or RACI mechanics themselves — that groundwork is Part 4's, and every relationship below assumes it already exists or is actively being built. It does not cover the legal and HR procedural mechanics that follow once a finding is confirmed to need legal or disciplinary handling — evidence preservation, subpoena response, termination documentation — which is Part 27 — Legal, HR & Compliance Interfaces's job; this part covers the standing relationship and the recurring political friction with those two functions, not the procedure that runs once a specific finding triggers them. It does not cover the ticket-level mechanics of a good hand-off between any two parties, which is SOC Playbook Handbook, Part 27 — Escalation Quality's job regardless of which teams are handing off to each other, and it does not referee which team's detection logic is technically more accurate — that is a question of evidence, not politics, and belongs wherever the specific technical claim is made.

1. Eight standing relationships, not one

1.1 Why a uniform instinct fails

[CONCEPT] The RACI treats every counterpart relationship as the same shape: a table row, a letter, a name. In practice, each of the eight counterparts below optimizes for something structurally different, and a manager who applies one uniform "stakeholder management" instinct across all eight will misjudge most of them. IT operations is measured on uptime and change stability. Engineering is

measured on shipping velocity. Legal is measured on liability and privilege exposure avoided, not tickets closed. Privacy is measured on regulatory exposure and data-minimization discipline. Physical security frequently reports through an entirely separate chain — facilities or corporate security — with its own incident-classification vocabulary and almost no shared tooling with the SOC. AppSec, IAM, and vulnerability management usually report through the same CISO as the SOC and still generate real friction, because their mandates overlap with the SOC's instead of sitting cleanly outside it. The lever that works on one counterpart — naming a missed SLA in a report IT operations' own director reads — does nothing to a legal team whose performance metric has nothing to do with SLAs at all.

1.2 IT operations

[SENIOR MANAGER] IT operations usually owns actual remediation execution — Part 4's standing RACI names IT operations Responsible and Accountable for patching and root-cause remediation, and the Management Autopsy in that part's §2 shows exactly what happens when the escalation path for a missed SLA loops back through the same director who owns the change calendar. The everyday version of that friction is smaller and constant: an infrastructure team asking for "one more change window" before applying a fix, individually reasonable each time, cumulatively how a 5-day SLA on a critical finding becomes a 6-week backlog with nobody able to point to the single decision that let it happen. Paging follows a similar split: IT operations reasonably expects to be paged first for anything framed as an availability problem, because they own the infrastructure; the SOC needs to be paged first for anything with a live compromise indicator, regardless of whose infrastructure it's running on. The recurring dispute is the ambiguous middle case — a server that's both down and possibly compromised — and it's worth a written trigger rather than a judgment call made fresh every time: any finding carrying an active-compromise indicator pages the SOC first, full stop; everything else follows ownership as the RACI already states it.

1.3 Engineering

[SENIOR MANAGER] The friction with engineering has the same shape as IT operations — a counterpart optimizing for a metric (ship velocity) that a SOC finding directly threatens — but the SOC's actual leverage is usually weaker, because engineering guards deploy pipelines and production code access far more tightly than IT operations guards a server fleet. A SOC that can't get read access to an application's own logging can't make its case with evidence; it can only assert a concern and hope engineering agrees to look. That access gap is why AppSec functions, in most organizations, as the real day-to-day interface between the SOC and engineering (§1.7) rather than the SOC negotiating directly — a fact worth naming explicitly, because a SOC manager who tries to build a direct relationship with an engineering VP while ignoring AppSec is duplicating a channel that already exists and undermining the one team positioned to broker the technical conversation.

1.4 Legal

[SENIOR MANAGER] The standing tension with legal is speed against privilege. Legal's institutional habit is to control who talks, what gets written down, and when, because a poorly worded ticket note or an internal message thread can become discoverable evidence in litigation. The SOC's institutional habit is to document everything immediately and share it broadly, so the next shift or a different analyst can pick up an investigation without a briefing. Those two habits are in direct tension on any case that could

plausibly end up regulated or litigated — not only the ones that obviously will, because the ones that obviously will are rarely the ones that catch anyone off guard. The standing mechanism worth building before any of this is tested for real: a charter-documented trigger list (Part 4 §5.2's scope-boundary table is the right home for it) naming which finding categories generate an automatic, low-friction heads-up to legal within 1 hour, even before disposition is complete — so the first time legal hears about a case in progress isn't a surprise phone call 40 hours in. What legal actually does once looped in — evidence holds, outside counsel, notification timing — is Part 27's territory, not this one's.

1.5 Privacy

[SENIOR MANAGER] The friction with privacy is about monitoring scope. The SOC's job pushes toward seeing as much as possible — insider-threat indicators, data-loss-prevention signals, behavioral baselining across users. Privacy's job pushes toward making sure that visibility is proportionate, documented, and legally defensible, which varies sharply by jurisdiction — a works-council consultation requirement in one region has no equivalent in a more permissive at-will-monitoring jurisdiction, and a SOC operating across both needs privacy's read on which rules apply where far more than it needs privacy's permission in the abstract. The recurring dispute is a timing mismatch: the SOC wants a new telemetry source — full email content indexing, expanded endpoint monitoring — live now, because a specific investigation needs it; privacy wants a data-protection impact assessment first, which takes weeks, not hours. The fix isn't skipping the assessment under pressure; it's pre-negotiating the common categories of monitoring in the charter's scope section so most requests match an already-approved category, leaving the slower conversation for genuinely novel data sources rather than for every request. Where a monitoring finding turns into an insider-threat case requiring HR involvement, that hand-off is Part 27's procedure to run, not this section's.

1.6 Physical security

[SENIOR MANAGER] Physical security is the counterpart most likely to go a full year with almost no interaction with the SOC, because it frequently reports through a completely different chain — facilities or corporate security, sometimes a function staffed by former law enforcement with its own incident taxonomy — sharing no tooling and often no common manager below the executive layer. Part 4's RACI already draws the ownership boundary cleanly: physical security dispatches and manages the facility response, the SOC correlates digital signals against it when both exist. The actual friction isn't over who owns what; it's that nobody notices the other domain has a related event at all. A badge-cloning attempt correlated against an anomalous VPN logon from a geographically implausible location is only catchable if someone is looking at both streams together, and in most organizations nobody owns that correlation as a standing job. The fix is a short, jointly agreed trigger list — failed-badge-attempt bursts, after-hours badge use on a terminated employee's still-active credential — that routes automatically to both teams, rather than depending on either team remembering the other's alert stream exists.

1.7 The adjacent security sub-teams: AppSec, IAM, and vulnerability management

[SENIOR MANAGER] The least obvious friction is often the sharpest, because these three sub-teams share a reporting chain and a mission statement with the SOC and still optimize for different, sometimes conflicting, local metrics. AppSec's KPI is typically secure-development-lifecycle coverage — findings caught before code ships. The friction case: a production runtime detection that's really

evidence AppSec's pre-release scanning missed something, and a live dispute over whose miss that counts as, fought out in a retrospective instead of settled by a standing rule. IAM's KPI is typically a provisioning and access-review SLA, and the friction case is authority during an active investigation: does the SOC have unilateral authority to kill a suspicious active session, or must IAM execute every revocation through its own change process — a distinction that can add several minutes to a containment action during exactly the window when minutes matter most. Vulnerability management's KPI is typically patch-cycle throughput against a CVSS-driven backlog, and the friction case is the sharpest of the three: whose severity number is authoritative when the SOC's threat-intel-informed context and vulnerability management's CVSS baseline disagree about the same finding. §2.3 covers that dispute directly, with a worked case.

1.8 The standing-relationship matrix

[SENIOR MANAGER] The table below is a starting reference for where each of the eight relationships typically breaks and what standing mechanism actually resolves it, built to be reviewed and corrected against a specific organization's real behavior rather than adopted as written.

Counterpart	Primary Friction Point	Who Typically Owns Remediation	Recurring Dispute Trigger	Standing Mechanism
IT operations	Change-freeze delay vs. SLA	IT operations	"One more window" requests, ambiguous down-or-compromised cases	Written trigger: compromise indicator pages SOC first, else ownership as RACI states
Engineering	Telemetry/production access	Engineering	SOC can't evidence a finding without access engineering controls	AppSec brokers the technical conversation (§1.3, §1.7)
Legal	Speed vs. privilege	Legal (once looped in)	Case escalates before legal is aware	Charter-documented 1-hour heads-up trigger list (§1.4)
Privacy	Monitoring scope vs. proportionality	Privacy (approval), SOC (execution)	New telemetry needed faster than a fresh assessment allows	Pre-approved monitoring categories in the charter's scope section
Physical security	Nobody correlates the two domains	Physical security (facility response)	Badge/network correlation nobody owns as a job	Joint trigger list routed to both teams automatically
AppSec	Whose "miss" a runtime finding represents	AppSec (pre-prod), SOC (runtime)	Retrospective blame dispute after a production finding	Named ownership split by lifecycle stage, not by team identity
IAM	Authority to revoke mid-investigation	IAM (execution)	Minutes lost during active containment	SOC given a defined, limited kill-switch authority for active incidents
Vulnerability management	Whose severity score is authoritative	Vulnerability management (backlog)	CVSS baseline vs. contextual threat-intel severity disagree	Split authority: baseline for the general backlog, SOC override with corroborating evidence (§2.3)

Manager's Note Walk this table with each counterpart at least once a year and ask them, out loud, whether the "who typically owns remediation" column matches their own understanding — not whether they agree with it, just whether it matches what they'd have said unprompted. A gap between what the SOC believes the standing mechanism is and what the counterpart believes it is will surface here, cheaply, instead of during a live dispute where neither side has time to discover they were never aligned.

2. Three friction points that recur across every counterpart

2.1 Who owns remediation

[SENIOR MANAGER] Part 4's RACI settles ownership on paper. The live dispute is rarely about ownership at all — it's about whether the accountable party agrees the finding is real or urgent enough to act on now, which is a different question the RACI cell doesn't answer by itself. A team that doesn't want to prioritize a fix will usually argue the finding is wrong, overstated, or already mitigated some other way, rather than argue priority directly — because contesting the finding's validity is a stronger negotiating position than admitting the finding is real and simply asking for more time. Recognizing that pattern for what it is changes how a manager responds: the right question isn't "why won't you fix this," it's "do we actually agree this finding is real and correctly scoped," asked and settled first, separately from the timeline argument that follows.

Blind Spot A validity dispute that's really a priority dispute in disguise will consume a manager's time re-litigating technical detail that was never actually in question, while the real disagreement — is this urgent enough to disrupt this week's plan — never gets named or resolved. If a counterpart keeps finding new technical objections to a finding that survived the SOC's own validation, and each objection, once answered, is immediately replaced by another, that pattern itself is the signal: stop answering technical questions and ask directly whether the real issue is priority.

Cross-Book Pointer This part does not cover what a SOC should automate versus gate for human sign-off — that's SOC Playbook Handbook, Part 30 — Automation and SOAR. It matters here because an automated remediation action (auto-isolating a host, auto-disabling an account) that lands on IT operations' or engineering's system without their sign-off creates a political flashpoint distinct from any technical error in the action itself: the counterpart's objection is often "you didn't ask," not "you were wrong." Decide what to automate using that part's framework; treat the standing notification obligation to the owning team as this part's problem regardless of what that framework decides.

2.2 Who gets paged first

[SENIOR MANAGER]

The right default is to page whoever can act fastest to contain, not whoever eventually owns the fix. A laptop showing active command-and-control traffic pages the SOC first, because the SOC can isolate it at the endpoint level in minutes, even though IT asset management eventually owns the wipe and reissue. A badge-cloning attempt pages physical security first, even though the SOC eventually correlates it against a network anomaly. The common failure mode is defaulting to whoever the RACI names as Accountable for the eventual fix, which is frequently the slowest first responder for the specific job of stopping the incident from getting worse right now — accountability for the outcome and capability to act in the next five minutes are different things, and a paging policy built only around the first one pages the wrong team for the second.

[FRONTLINE MANAGER]

The shift lead living this in real time carries a burden the paging policy on paper doesn't show: coordinating two pages going to two on-call rotations that have rarely, if ever, actually run a joint drill together, while the clock on containment keeps running regardless of whether either rotation answers promptly. A paging matrix that looks clean in a document can still produce ten minutes of confusion at 3 a.m. because the physical-security on-call rotation and the SOC's own rotation have never once been tested together outside a real incident.

Manager's Note *After any incident that paged more than one team, check who actually got paged and how long it took before the right team was reached — not just before someone answered. A page that reaches the correct on-call person in 2 minutes but the wrong team looks fine on a response-time dashboard and is still a paging-policy failure; the dashboard just isn't measuring the failure that happened.*

2.3 Whose tooling wins a dispute

[SENIOR MANAGER]

Two tools regularly produce two authoritative-sounding numbers for the same finding: a vulnerability scanner's CVSS score against the SOC's own threat-intel-informed contextual severity, or an AppSec scanner's finding against a SOC-owned runtime alert for what turns out to be the same underlying weakness. SOC Playbook Handbook, Part 29 — Playbook Severity Model already owns the mechanics of how an individual alert's severity gets scored; this part's question is organizational, not mechanical — when two teams' scoring systems disagree about the same finding, whose number controls the remediation timeline.

The wrong fix is declaring one tool permanently authoritative across the board. That doesn't resolve the disagreement — it just relocates the failure mode into whatever category the losing tool would have caught correctly. The better fix splits authority by input type, consistent with the enforcement-rung logic Part 4 §4.1 already established for charter authority generally: a baseline system (CVSS-driven backlog prioritization, in the vulnerability-management case) governs the general patch cycle, and the SOC retains a named, narrow override authority to escalate any specific finding carrying corroborating active-exploitation evidence to a mandatory expedited SLA, regardless of what the baseline score says.

Management Autopsy — "make vulnerability management's CVSS score the single authoritative severity" (COMPOSITE CASE EXAMPLE, [CASE-2601](#))

The decision: A 12-person SOC and a separate 4-person vulnerability-management team kept opening duplicate tickets for the same findings with conflicting severities — the SOC's contextual score, informed by threat intelligence, and vulnerability management's CVSS base score. To end the duplicate-ticket noise, the organization declared vulnerability management's CVSS score the single authoritative severity system-wide; the SOC's contextual severity became informational only, visible on the ticket but no longer driving the remediation SLA.

Why it seemed reasonable: One number instead of two conflicting ones, less duplicate ticket traffic, and an easier prioritization conversation for IT operations, which had been the team stuck reconciling two severities on the same finding before acting on either.

How it failed: A specific CVE affecting internet-facing infrastructure scored 6.8 on the CVSS base scale — medium, scheduled into the normal patch cycle rather than expedited. The SOC's own threat-intelligence feed showed active exploitation attempts in the wild against that exact software version, on infrastructure matching the organization's own exposed footprint, within days of the CVE's publication — evidence CVSS's base score, by design, doesn't incorporate. Because the SOC's contextual severity no longer drove the SLA, the finding sat in the standard patch queue for 3 weeks. During that window, an exploitation attempt succeeded against one internet-facing host, triggering a real incident. Incident response, forensics, and expedited remediation cost roughly \$95,000 — a cost directly attributable to a finding that both teams had, in different ways, correctly flagged as dangerous well before the compromise.

The fix: Severity authority was split rather than centralized: vulnerability management's CVSS-driven backlog continued to govern the general patch cycle, but the SOC was given a documented, narrow override — any finding with corroborating active-exploitation evidence could be escalated to a mandatory expedited SLA regardless of its CVSS score, logged as an enforcement-rung-one clause in the charter (Part 4 §4.1) rather than left as an ad hoc argument re-fought on every disputed case.

Cross-Book Pointer This part does not referee whether a specific detection or scanner is technically more accurate than another — that's an evidentiary question about telemetry and detection logic, not an organizational-authority question, and it belongs to Detection Engineering Handbook V2, Part 41 — Detection Coverage and Part 43 — Detection Debt wherever a specific technical claim needs arbitrating. This part's job stops at who has standing authority to act once the technical question is settled, or while it's still genuinely unsettled.

3. The shadow SOC problem

3.1 Why a business unit builds its own

[CONCEPT] A shadow SOC is what happens when a business unit stops waiting for the standing relationships in §1 to produce a response it can live with and quietly stands up monitoring of its own — a SaaS alerting tool, a homegrown dashboard, a Slack bot wired directly to a cloud provider's native detection service. The trigger is rarely spite. It's usually a real, specific, repeated frustration: a central SOC that responds slowly because it doesn't understand the business unit's particular technology stack, a coverage gap for a domain the central SOC was never resourced or trained to monitor well (a specialized OT/ICS environment, a cloud-native product with its own telemetry conventions), or simply an engineer with security instincts, a company credit card, and an afternoon free. Treating a shadow SOC purely as a compliance violation to be shut down, without asking what specific service gap caused it, guarantees the same gap produces the same response again under a different tool name within roughly one budget cycle.

3.2 Recognizing it before an audit does

[SENIOR MANAGER] Most shadow SOC's are found by an outside party asking a direct question the central SOC never asked itself — an auditor's "list every tool with access to production data," a vendor-risk questionnaire, a new CISO's first inventory review. The table below names the signs worth checking for on a standing basis, before that outside question arrives.

Indicator	What It Looks Like	Why It's Easy To Miss
Unexplained security-tool spend	A SaaS security or monitoring line item in a business unit's own budget, not the SOC's	Budget reviews rarely cross-check line items against the SOC's own tool inventory
A dashboard the SOC didn't build	A business unit references "our security dashboard" in a meeting the SOC manager attends	Sounds like a healthy local interest in security, not a parallel monitoring system
Alerts that appear pre-actioned	An analyst notices a finding already has notes or a resolution from someone outside the SOC's own roster	Reads as a helpful colleague, not evidence of an unmanaged second response path
A "security" hire outside the SOC's pipeline	A business unit hires someone with a security-flavored title who was never sourced or interviewed through the SOC's own process	Feels like normal headcount growth elsewhere in the org
A vendor-risk or audit surprise	An external review asks for "all tools with access to production data" and surfaces one the SOC's asset inventory never listed	The question is asked rarely, and only from outside

3.3 What it actually costs

[SENIOR MANAGER] A shadow SOC is a governance risk even when the tool behind it works reasonably well technically, for reasons that have nothing to do with whether it happens to catch real threats. Two teams independently responding to the same incident can undo each other's containment step without either knowing the other already acted. A tool with no on-call rotation and no backup analyst has a bus factor of exactly one — the engineer who built it — and quietly stops being monitored the week that person leaves, with nobody outside the business unit aware coverage lapsed. A monitoring tool with access to regulated data that never went through data-governance or vendor-risk review is itself a finding, independent of anything it ever caught, the moment an auditor asks the right question. And the

risk runs in both directions: a central SOC that believes a domain is covered by "whatever the business unit built" may be under-resourcing that domain in its own coverage planning, or crediting itself with redundancy that quietly died months earlier along with the one person who maintained it.

Blind Spot *The central SOC's own coverage map is wrong the moment a shadow tool exists and nobody accounts for it in either direction — either a domain is silently uncovered because the SOC assumed the shadow tool had it, or a domain is double-covered in a way nobody is actually reconciling, producing two independent, uncoordinated responses to the same event. A coverage review that only asks "what does the SOC monitor" and never asks "what does anyone else monitor that the SOC doesn't know about" cannot see this gap by construction, no matter how carefully it's run.*

CASE-2602 — the GuardDuty Slack bot. *COMPOSITE CASE EXAMPLE — merges patterns from several cloud-team shadow-monitoring cases; figures below are illustrative, not sourced financial or audited data.*

A central SOC of 14 analysts served six business units, including a 35-engineer cloud product team whose infrastructure ran almost entirely on a single cloud provider. The SOC's average turnaround on cloud-specific alerts ran 4 to 6 hours, driven by unfamiliarity with that team's specific architecture rather than any staffing shortfall elsewhere. Roughly 8 months before discovery, two engineers on the cloud team wired the provider's native threat-detection service directly into a Slack channel they monitored themselves, paging each other on anything they judged urgent and bypassing the central SOC's queue entirely for their own environment.

CONCEPTUAL SAMPLE -- illustrative figures, not sourced benchmark or audited financial data.

Discovery: annual compliance audit request -- "list all tools with access to production data" -- surfaced the Slack-integrated detection feed; it was not on the SOC's asset inventory
Duration undetected: ~8 months
Alert history requiring retroactive review: ~14 months (the feed had been informally running even before the two engineers formalized the Slack routing)
Time to formally onboard the feed into the central SIEM and asset inventory: ~6 weeks
Contractor and analyst time to complete the onboarding and retroactive review: ~\$22,000
Audit outcome: formal finding requiring a documented corrective-action plan
One real event correctly caught and closed by the shadow tool during the 8-month window: a briefly public-facing storage bucket, misconfigured and exposed for under an hour before the two engineers closed it themselves

The tool worked, in the narrow sense that it caught a real exposure and the two engineers who built it closed it quickly. That is precisely why treating this as a story about incompetence misses the point: the underlying complaint — the central SOC's 4-to-6-hour turnaround on this team's environment — was accurate, and the fix that actually mattered was closing that gap, not merely retiring the Slack bot. Retiring it without addressing the turnaround problem would have removed the shadow tool and left the same engineers with the same incentive to rebuild it.

Manager's Note *The fastest way to find out whether a shadow SOC exists is to ask, directly, in a budget review: has anyone bought a monitoring, alerting, or logging tool this year that isn't on the SOC's own asset list? Most managers never ask this question in those exact words — and asking it plainly surfaces a yes far more often than a clean asset inventory would suggest.*

3.4 Three responses: absorb, federate, or shut down

[SENIOR MANAGER]

Once a shadow monitoring capability is found, there are three defensible responses, and the wrong choice is picking one without first answering whether the underlying complaint that caused it is real. Shutting it down with no fix to the underlying gap produces a shadow SOC again, under a different tool, within roughly one budget cycle — the complaint didn't go away because the tool did. Absorbing it — folding the data source and its access into the central SIEM and asset inventory, then retiring the standalone tool — works when the central SOC can take over without losing the business-unit-specific context that made the shadow tool useful in the first place. Federating it — giving the business unit's team a formally chartered, RACI-amended role feeding the central SOC rather than either side owning the domain alone — is the right long-term shape when that context is genuinely hard to replicate centrally, such as a specialized operational-technology environment the central SOC has neither the headcount nor the domain training to monitor well on its own.

flowchart TD

```
A["Shadow monitoring discovered"] --> B["Making real response\ndecisions with production\ndata access?"]
B -->|"No -- informal\ndashboard only"| C["Log it, monitor it,\nrevisit next audit cycle"]
B -->|"Yes"| D["Underlying complaint is a\nreal central-SOC service gap\n(speed, domain expertise,\ncoverage)?"]
D -->|"No -- mainly\nconvenience or control"| E["Absorb: onboard the tool\nand data source into\nthe\ncentral SIEM, retire the shadow path"]
D -->|"Yes -- real,\nverifiable gap"| F["Federate: charter a formal\nlocal role feeding the\ncentral\nSOC, amend the Part 4 RACI"]
E --> G["Update charter, RACI,\nand asset inventory"]
F --> G
G --> H["Shut down with no fix\nto the underlying gap"]
H -->|"recurs within\n~1 budget cycle"| A
```

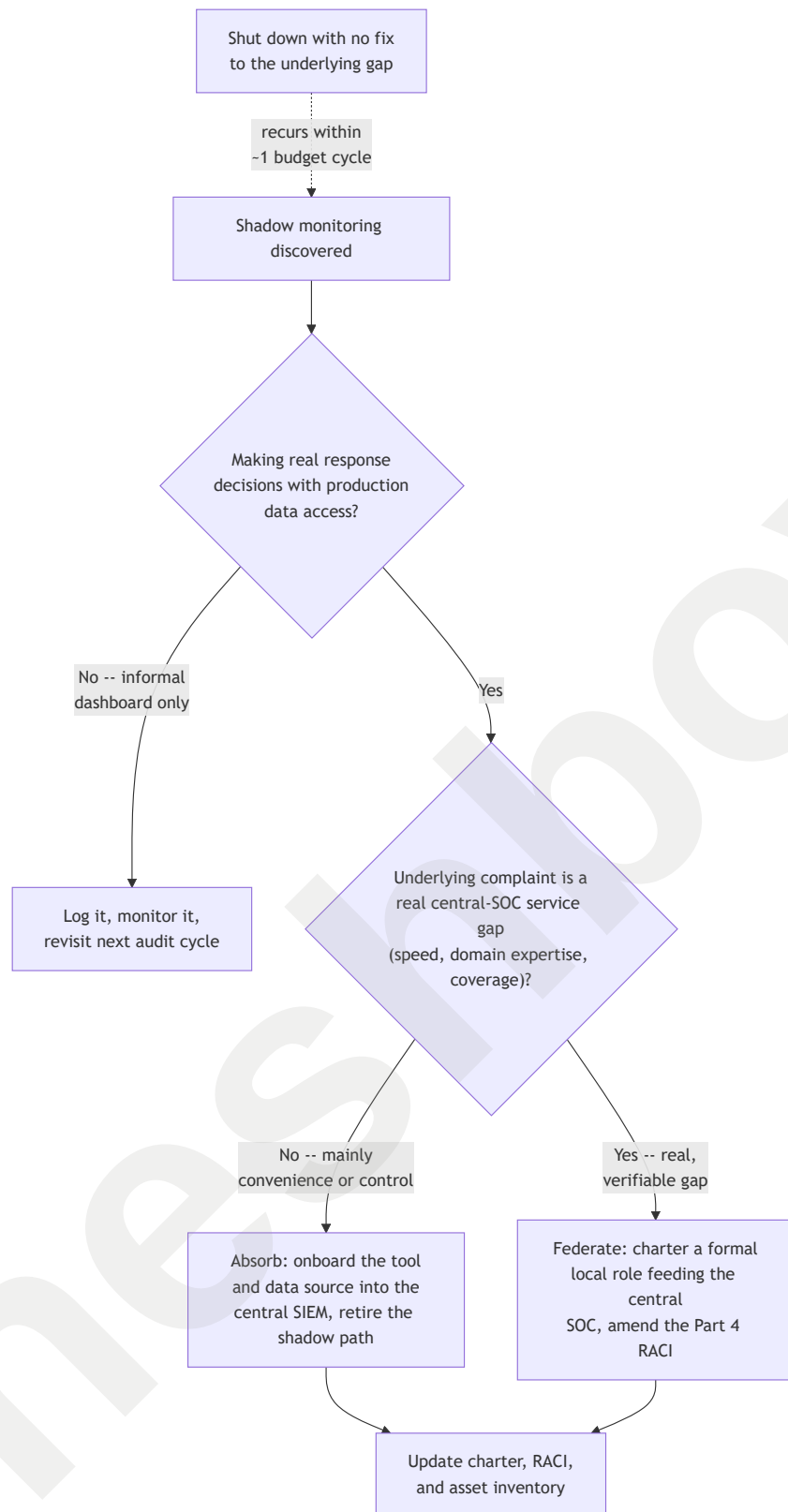


Figure 26.1 — Shadow-SOC resolution decision flow. *CONCEPTUAL.* Shows the decision path from discovering a shadow monitoring capability to one of three resolutions — absorb, federate, or a shutdown that skips fixing the underlying gap and predictably recurs. It supports the claim in §3.4 that the choice among the three responses depends on whether the underlying service gap is real, not on how the discovery was made. FIG-2601.

4. Building the standing alignment mechanism

4.1 A recurring cadence, not just an escalation path

[SENIOR MANAGER] Everything in §1 through §3 assumes the relationship is maintained between disputes, not only during them. A monthly or quarterly cross-team sync with each standing counterpart — combined into one session for the three security sub-teams where headcount doesn't justify eight separate meetings — with a fixed, short agenda (open disputes, any drift between the charter's RACI and what's actually happening, any new tool or data source either side introduced since the last session) catches most of §2's and §3's failure modes while they're still cheap to fix.

[FRONTLINE MANAGER] The SOC manager cannot be the sole point of contact for every judgment call that comes up at 2 a.m., which is exactly when a real cross-team dispute is likeliest to surface. Team leads need their own working relationship with their counterpart on-call rotations — a name and a number they've actually used before, not one they're meeting for the first time during a live incident — or the alignment this section describes exists only on the SOC manager's calendar and nowhere else the work actually happens.

4.2 When a conflict needs executive sponsorship

[EXECUTIVE] Some disputes are structural rather than personal, and no amount of relationship-building at the SOC-manager level resolves a structural placement problem — the independence problem Part 4 §1.2 names directly, where the SOC and the team it's disputing with share a boss who will predictably resolve the dispute in favor of whichever metric that boss is actually measured on. Recognizing that a dispute has become structural, rather than continuing to treat it as a relationship to be smoothed over indefinitely, is itself the judgment call Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty is built to support: whether to keep absorbing the recurring cost at the SOC-manager level or escalate the placement question itself to whoever can actually change it. Once that escalation decision is made, Part 24 — Executive & Board Reporting covers how to actually frame the ask for a board or executive audience with no visibility into the day-to-day friction that drove it.

5. Measuring whether alignment is actually working

[SENIOR MANAGER] The easiest things to measure about cross-team alignment are participation counts, and participation counts look healthy right up until a real dispute reveals they were never tracking the thing that actually matters. Pair each leading indicator below with the outcome check it's a weak proxy for.

Leading Indicator (Easy to Game)	What It's a Proxy For	Better Outcome Check
Number of cross-team sync meetings held	Relationships staying current	Time from a finding opening to cross-team agreement on ownership, tracked separately from time to remediation
Number of RACI updates logged	The RACI staying accurate	Share of this quarter's disputes that mapped cleanly to an existing RACI row versus needed an ad hoc decision
Zero shadow-monitoring findings in the latest audit	No shadow SOC problem exists	The trend across several audit cycles, not one clean reading — one audit missing something is not evidence it isn't there
Every counterpart signed the charter	Standing agreement is real	Whether a counterpart named in §1.8's matrix can describe, unprompted, the same ownership split the SOC believes is in place

Cross-references

This part assumes the charter and RACI groundwork from Part 4 — Organizational Placement & Charter, and points forward to Part 27 — Legal, HR & Compliance Interfaces for the procedural mechanics that follow once a legal-, privacy-, or HR-triggering finding is confirmed, and to Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty and Part 24 — Executive & Board Reporting for escalating a structurally unresolvable dispute past the SOC-manager level. Outside this book: SOC Playbook Handbook, Part 27 — Escalation Quality for the ticket-level hand-off mechanics this part's opening scope note points away from; SOC Playbook Handbook, Part 29 — Playbook Severity Model for the scoring mechanics behind the severity disputes in §2.3; SOC Playbook Handbook, Part 30 — Automation and SOAR for what to automate versus gate for human sign-off, referenced in §2.1; and Detection Engineering Handbook V2, Part 41 — Detection Coverage and Part 43 — Detection Debt for the technical arbitration question this part deliberately stops short of.

Part 27 — Legal, HR & Compliance Interfaces

Why this part exists

[CONCEPT] Part 4 built the SOC's standing RACI with legal, HR, and physical security as a set of rows in a matrix — who's Responsible, who's Accountable, for activities like "preserve evidence / apply legal hold" and "insider-threat finding hand-off to HR." A RACI row is a map reference, not a set of directions. This part writes the directions: what actually has to happen, in what order, documented how, the first time a litigation hold lands in a SOC manager's inbox, the first time an insider-threat finding is confirmed and someone has to decide what happens to the person behind it, and the first time an external examiner asks to see evidence rather than a policy document. None of this is technical work. It's procedural work with real legal and financial consequences attached to getting the procedure wrong, and it sits squarely in the manager's chair because no analyst, however senior, has the standing to make these calls alone.

Three standing interfaces make up this part's scope. Toward legal: recognizing when a preservation duty has attached, keeping a litigation hold from being scoped so narrowly it misses the evidence that matters or so broadly it paralyzes the team, responding to a subpoena or a law-enforcement request without either stonewalling or overcomplying, and understanding what changes procedurally once outside counsel is directing a breach investigation. Toward HR: what happens organizationally once an insider-threat finding is confirmed — the hand-off package, who decides discipline, and how a termination's paper trail either survives a legal challenge or hands the challenger their strongest exhibit. Toward audit and regulatory examiners: the standing cadence of internal audit, compliance-framework assessments, and regulatory exam cycles a SOC manager should expect to run into repeatedly, not as a one-time event.

[CONCEPT] The detection and response mechanics behind an insider-threat finding — the behavioral indicators, the correlation logic, the escalation criteria that gets a case confirmed in the first place — belong to SOC Playbook Handbook, Insider Threat Playbooks (Category 19), and this part does not re-walk any of it. This part starts exactly where that material ends: a finding is already confirmed, and now an organization has to decide what to do about a person. The same boundary holds for a live breach: Part 28 — The Manager's Role in a Major Incident owns the manager's in-the-moment decision to activate an IR retainer or bring in outside counsel during an active incident. This part owns the standing structure that decision activates — the engagement-letter terms, the privilege routing, the pre-negotiated retainer — so that decision isn't being improvised from scratch under pressure. And Part 16 — Performance Management & Coaching already built the coaching cadence, the diagnostic for what's actually causing a recurring performance problem, and the mechanics of a defensible Performance Improvement Plan; this part adds only the legal-review layer that plan needs once termination moves from a possibility to an actual decision.

1. Three interfaces, one shared discipline

[CONCEPT] Legal, HR, and audit look like three unrelated relationships with three different vocabularies, but a SOC manager's job across all three has the same shape: recognize the trigger early,

preserve and document rather than act unilaterally, route the decision to whoever actually owns it, and never let the team's normal operational instinct — contain the problem fast, fix it now — override a process that exists precisely because speed is not always the right answer once legal exposure, someone's employment, or a regulator's finding is on the table.

Interface	What triggers it	SOC's actual role	Who owns the decision
Legal — evidence & holds	Reasonably anticipated litigation, a subpoena, a regulatory inquiry, an insider-threat finding likely to end in termination	Preserve what's asked, flag what's about to expire, document chain of custody	In-house or outside counsel
Legal — subpoena / law-enforcement request	A document lands naming the organization or an employee, compelling or requesting data	Route immediately, do not respond or confirm anything independently	Legal, working the compulsion/timeline
HR — insider-threat aftermath	An insider-threat finding is confirmed under SOC Playbook Category 19's mechanics	Hand off evidence and technical scope, contain access on a coordinated timeline	HR, with legal, on discipline and termination
HR — termination documentation	A Performance Improvement Plan (Part 16) is failing or an insider-threat finding implicates a specific person	Supply the technical/behavioral record; do not draft the legal justification alone	HR and legal jointly
Audit / regulatory exam	A scheduled cycle: internal audit, a compliance-framework assessment, a regulator's exam	Produce evidence on request, on a single coordinated channel	The audit or exam program itself; the manager doesn't negotiate scope

[SENIOR MANAGER] The common failure across all five rows isn't malice or incompetence — it's a manager or analyst treating a legal/HR/audit trigger as an operational problem to be solved the way an incident is solved: fast, locally, by whoever's closest to it. A SOC that's excellent at incident response can still generate real legal and financial exposure by applying that same instinct to a subpoena or a termination, because the correct move in this domain is almost always to slow down, document, and route — the opposite of what a live security incident usually rewards.

2. Evidence preservation and legal holds

2.1 What triggers a hold, and who issues it

[SENIOR MANAGER] A duty to preserve evidence attaches the moment litigation, a regulatory investigation, or a comparable adversarial process is *reasonably anticipated* — not once a complaint is formally filed. That threshold is a legal judgment, and it belongs to counsel, not to the SOC manager. What belongs to the SOC manager is recognizing the situations that should prompt a call to legal before a hold notice ever arrives: a confirmed insider-threat finding likely to end in termination, a breach with regulated data in scope, an employee's discrimination or harassment complaint that touches systems the SOC has visibility into, or a subpoena or law-enforcement request that references specific data. A manager who waits for legal to proactively identify every one of these triggers is relying on legal having visibility the SOC actually has first — the SOC usually sees the technical fact pattern (an account was accessed, a file was exfiltrated, a login pattern is anomalous) well before legal knows there's anything to preserve.

2.2 The retention-clock problem

[SENIOR MANAGER] A litigation hold notice describes what to preserve going forward. It says nothing about data that already aged out of a retention tier before the notice existed, and this is where a SOC's own infrastructure becomes the biggest risk to a hold it never intended to create. A SIEM with a 90-day hot-tier retention window and an EDR platform with a 30-day telemetry retention window will auto-purge the exact evidence a hold exists to preserve, on schedule, with no malicious intent from anyone, unless someone on the SOC side flags the clock the moment a preservation trigger from §2.1 appears — not after legal formally issues a notice, which can take days to draft and route through the right approvals.

Blind Spot *A legal hold notice is written in legal language about custodians and date ranges; it is not written by someone who knows this organization's specific retention tiers, and it will not, by itself, stop a 30-day EDR telemetry window or a 90-day SIEM hot tier from purging data on schedule. The SOC manager is the only person in the chain who actually knows those numbers, and the gap this creates is silent — nothing errors out, nothing alerts, the data simply isn't there anymore when someone finally asks for it six weeks later. Flagging retention windows to legal the moment a preservation trigger appears, before the formal notice is even drafted, is the only thing that closes this gap.*

2.3 Scoping the hold, and what happens when it's wrong

[SENIOR MANAGER] A hold's scope — which custodians, which systems, which date range — is legal's call to make, but the SOC manager is usually the only person who can tell legal, accurately, what a given scope will actually capture: whether "email" also needs to include a specific chat platform, whether "endpoint" needs to include a personal device enrolled in BYOD MDM, whether the date range legal proposed predates or postdates the retention tier that actually holds the relevant data. Getting this conversation right the first time is materially cheaper than discovering the gap during document review, months later, with opposing counsel already aware something is missing.

CASE-2701 — the hold that named the wrong retention tier. *COMPOSITE CASE EXAMPLE — merges patterns from several regulated-industry litigation-hold failures into one illustrative narrative; figures below are illustrative, not sourced legal or audited data.*

CONCEPTUAL SAMPLE -- illustrative figures, not sourced legal or audited financial data

Organization: ~40-person SOC, regional financial-services firm

Day 1: Employment-discrimination complaint filed by a terminated analyst; legal issues a hold notice naming "email and personnel records" for the named custodians.

EDR and SIEM logs are not named -- nobody on the SOC side is asked, and nobody volunteers, that EDR telemetry for the custodian's former endpoint is on a 30-day rolling purge.

Day 34: EDR telemetry for the relevant endpoint has fully aged out and purged on schedule.

Day 52: Opposing counsel's document request specifically asks for endpoint activity logs to corroborate a claim about after-hours access patterns. The data no longer exists.

Outcome: Court gives an adverse-inference instruction (the jury may assume the missing data would have been unfavorable to the employer) based on a spoliation motion.

Outside counsel's own post-mortem estimate: the spoliation exposure alone added roughly \$150,000 to the eventual settlement value, on top of the underlying claim, specifically because the missing data could no longer be used to either confirm or rebut the access-pattern allegation.

[SENIOR MANAGER] The fix in CASE-2701 wasn't a smarter legal team — it was a SOC manager involved early enough to say "our EDR telemetry for that host is a 30-day rolling window; if this scope doesn't explicitly include it, it's gone before your notice is even drafted." That sentence takes 10 seconds to say and would have prevented the entire spoliation exposure. It never got said because nobody on the SOC side was in the room when the hold's scope was set, and legal had no way to know the question needed asking.

2.4 Chain of custody once evidence is actually pulled

[SENIOR MANAGER] Once a hold is in place and evidence is being collected, the documentation standard is the same one Part 16 assumes for a performance record and Part 4's RACI assumes for an insider-threat hand-off, applied to technical artifacts instead of conversation notes: who pulled the data, exactly when, from which system, with what identifying hash or export reference, stored where, and who has had access to it since. Four fields — who, when, what, and a verifiable integrity reference — turn a claim ("we pulled the logs") into evidence a court or an examiner can actually rely on. Skipping any one of the four doesn't make the evidence disappear; it makes the evidence contestable, which in a challenged termination or a regulatory finding is nearly as costly as not having it at all.

3. Subpoena and law-enforcement request response

3.1 The manager's role: router, not decider

[FRONTLINE MANAGER] The person who first sees a subpoena or a law-enforcement request is rarely the SOC manager — it's often a team lead who opens an email, or an analyst who's handed a document by a process server or a visiting agent. That person's job has exactly one correct move: route it to the named legal contact immediately, and say nothing substantive to the requester beyond confirming it was received. Answering a question, confirming or denying that a specific account exists, or promising a turnaround time — even something as innocuous as "sure, we can have that by Friday" — can commit the organization to a position legal hasn't reviewed and didn't authorize.

3.2 What kind of request this actually is

[SENIOR MANAGER] Not every document that arrives looks the same, and the correct response differs sharply by type. The table below is the fast triage a manager needs before legal gets involved — it doesn't replace legal's judgment, it makes sure the right artifact reaches legal fast enough for that judgment to matter.

Request type	Typical form	Compliance obligation	Manager's immediate action
Civil subpoena (litigation between two parties)	Subpoena duces tecum, produce documents by a set date	Legally compulsory on the timeline the issuing party and applicable court rules set	Route to legal now; do not produce anything or confirm scope independently
Grand jury subpoena	Criminal investigation, often with a secrecy expectation	Compulsory; may carry an informal or formal non-disclosure request	Route to legal; do not discuss its existence outside the named legal contact
Regulatory examiner information request	Written request during a scheduled exam (see §7)	Compulsory under the regulator's statutory authority	Route to compliance/legal; use the standing process in §7, not an ad hoc response
Informal law-enforcement preservation request	A letter or email asking the organization to preserve, not yet produce, specific data	Preserve only; producing content without a warrant or court order can itself create liability	Preserve the named data immediately; route to legal before producing anything
National security letter or statutory gag order	Rare; carries its own non-disclosure statute	Compulsory, with a legal prohibition on disclosing even that it was received	Route to the single legal contact named for this scenario; do not acknowledge its existence to anyone else

Field Test Setup: No prior drill has tested how a subpoena or law-enforcement request actually moves through the team once it lands. **Action:** With legal's sign-off, have someone outside the SOC "serve" a mock civil subpoena — a realistic-looking document with a fictitious case caption — on a randomly chosen Tier 1 analyst's inbox, timed for a normal shift.

Expected result: The analyst forwards it to the named legal contact within the same shift, without independently responding to the sender, confirming any account exists, or discussing its contents outside that routing step. If the analyst instead tries to answer it, sits on it past shift-end, or asks a teammate what to do because no one told them who the named contact even is, the routing path is aspirational, not operational — fix the on-call documentation before relying on it for a real one.

3.3 A worked response timeline

[SENIOR MANAGER] Many U.S. federal civil subpoenas give the recipient roughly 14 days to object under the applicable procedural rules and set a compliance date commonly 21 to 30 days out — enough time to do this correctly, but not enough time to do it correctly if the SOC's own collection step doesn't start until legal has already used up most of the window deciding how to respond. The table below is an illustrative decomposition of a 21-day window, not a universal timeline — actual deadlines

depend on jurisdiction, the specific rule under which the subpoena issued, and any negotiated extension.

CONCEPTUAL SAMPLE -- illustrative timeline, not a substitute for the specific deadline stated on an actual subpoena or counsel's own guidance

Day 0: Subpoena received, routed to legal same day (Sec 3.1).
Day 1-3: Legal reviews scope, decides whether to object, negotiate, or comply as written.
Day 3-4: Legal confirms the data scope with the SOC manager -- which systems, which date range, which custodians -- using the same scoping conversation Sec 2.3 describes for a litigation hold.
Day 4-14: SOC collects and hashes the requested data, logging chain of custody (Sec 2.4).
Day 14-18: Legal reviews the collected production for privilege, relevance, and scope before anything leaves the organization.
Day 18-21: Production delivered on or before the compliance date.

[SENIOR MANAGER] The collection step (days 4 through 14 above) is the one a SOC manager actually controls, and it's the one most often compressed by a slow start on days zero through three — every day legal spends deciding whether to object is a day not spent collecting, and the compliance date doesn't move to compensate. Starting the retention-tier check from §2.2 and the collection conversation from §2.3 the moment a subpoena is routed, in parallel with legal's own review rather than waiting for legal to finish first, is what keeps a 21-day window from becoming a 10-day scramble.

4. Outside counsel and privilege during a breach

4.1 Why the engagement letter matters as much as the forensics work

[SENIOR MANAGER] Who retains a forensics or incident-response firm during a breach — the SOC directly, or outside counsel on the organization's behalf — changes whether the resulting report is protected by attorney-client privilege or work-product doctrine in later litigation or a regulatory proceeding. A firm retained directly by IT or the SOC, for ordinary security services, produces a report that's discoverable by default; a firm retained by outside counsel, specifically for the purpose of advising counsel on litigation or regulatory risk arising from a specific incident, has a real (though not guaranteed) claim to privilege protection. This distinction became one of the most closely watched issues in cybersecurity law after a widely reported federal court ruling — *In re Capital One Consumer Data Security Breach Litigation* — where a court ordered production of a breach-investigation report specifically because the same forensics firm had a pre-existing, ordinary-course retainer with the organization before the breach occurred, undercutting the claim that the post-breach report was prepared primarily to help counsel litigate rather than to do the security work the firm was already doing anyway. The exact procedural history is worth verifying against current case law before relying on it in a real matter — this book is not a substitute for that legal research — but the operating lesson survives the specifics: a retainer signed *after* a breach, structured so counsel is the client and the report's primary purpose is legal advice, protects privilege in a way that retroactively re-labeling an existing vendor relationship does not.

[SENIOR MANAGER] The practical consequence for a SOC manager is upstream of any specific breach: the retainer terms, the engagement letter's language about who directs the work and for what purpose,

and the decision about whether the *existing* IR vendor relationship needs a separate, counsel-directed engagement letter the moment a real incident starts, are all standing decisions worth having settled before day one of an actual breach — which is exactly the gap Part 28's live-incident activation decision assumes has already been closed.

4.2 What changes once counsel is directing the investigation

[SENIOR MANAGER] Once outside counsel is formally directing a breach investigation, a few concrete things change in how the SOC operates day to day, not just on paper. Investigative findings often route to counsel first rather than up the SOC's normal management chain for that specific matter, with counsel deciding what's shared onward and to whom. Written communications about the investigation — reports, emails, chat messages — should carry an explicit note that they're prepared at counsel's direction for the purpose of legal advice, and informal speculation about root cause or blame needs to move out of ordinary team channels and into whatever narrow, counsel-sanctioned channel the engagement designates, because an offhand Slack message speculating about fault is exactly as discoverable as a formal report if it isn't covered by the same privilege structure.

People Risk Trap *A team's normal incident channel — Slack, Teams, a ticketing tool's comment thread — is where analysts naturally think out loud during a live breach: "looks like we missed patching this two months ago," "I think whoever owns this service dropped the ball." Once litigation or a regulatory inquiry is reasonably anticipated, that same speculation, written in an ungoverned channel with no privilege protection, becomes a document opposing counsel can request and quote directly — often more damaging than the underlying technical failure, because it reads as an admission rather than a working hypothesis under active investigation. The fix: the moment counsel is engaged, move investigative discussion to the specific channel or format the engagement designates, and treat anything written elsewhere about the incident's cause or fault as if it will be read aloud in a deposition, because it might be.*

4.3 Where this splits from Part 28

[SENIOR MANAGER] This part owns the standing structure — the retainer terms, the privilege routing, the pre-negotiated engagement letter language — that makes a fast, correct activation possible. Part 28 — The Manager's Role in a Major Incident owns the live decision of when to actually pull that trigger during an active incident, alongside the staffing surge and board-communication cadence that decision sits next to. A manager who's never read this part's material before a live incident starts is negotiating privilege structure for the first time under exactly the conditions least suited to getting it right; the standing work this part describes should already be done well before Part 28's decision point ever arrives.

5. Insider-threat findings: from confirmed finding to HR handoff

5.1 The scope boundary, stated plainly

[CONCEPT] Everything in this section assumes a finding already exists — the organizational-aftermath process below starts at confirmation, not at detection.

Cross-Book Pointer *This part does not cover how an insider-threat finding gets detected or confirmed — the behavioral indicators, the correlation logic across access and data-movement telemetry, and the escalation criteria that turn a suspicion into a confirmed finding are SOC Playbook Handbook, Insider Threat Playbooks (Category 19)'s territory, mechanically developed there. This part starts at the moment that category's own escalation criteria have already been satisfied and a finding is confirmed — what an organization does next, procedurally, with a real person's employment and possibly their legal exposure on the table.*

5.2 The handoff package: what HR and legal actually need

[HR/PEOPLE] A SOC that hands HR a raw case file — alert timestamps, query syntax, a list of accessed file paths — hands HR something it can't act on, because HR and legal need a plain-language narrative, not a technical export. The table below is the standard handoff content; a real handoff needs all six rows before HR or legal should be expected to make a decision.

Handoff element	What it must contain	Why it's non-negotiable
Plain-language summary	What happened, in one paragraph a non-technical reader can act on	HR and legal decide based on this, not on raw telemetry
Timeline	Dated sequence of the relevant events, cross-referenced to evidence	Establishes what happened when, which matters for both discipline and any legal exposure
Evidence list with chain of custody	Every artifact relied on, collected per §2.4's standard	Evidence without a documented chain is contestable in a later challenge
Scope of access implicated	Exactly what systems, data, or accounts the person could reach	Determines both containment urgency and the scope of any exfiltration risk
Data-exfiltration status	Confirmed yes/no/unable-to-determine — not omitted	Changes both urgency and whether a regulatory notification duty (Part 4's Legal/Privacy RACI row) is triggered
Recommended containment status	What access has already been revoked, what's pending, and why	Lets HR and legal coordinate timing instead of discovering containment already happened, or hasn't

5.3 Who decides, who acts, and why the sequence matters

[SENIOR MANAGER] The SOC's containment authority and HR/legal's discipline authority are two different decisions that often need to happen close together in time but not in the same sequence every time. A finding involving active, ongoing data exfiltration usually justifies revoking access before HR has finished its own review, because the technical risk of waiting outweighs the coordination cost. A finding involving something already contained — a completed, historical policy violation with no ongoing access risk — usually justifies waiting for HR and legal to weigh in on timing before any visible action happens, because an abrupt access revocation with no documented rationale can itself look punitive or retaliatory if the timeline is ever scrutinized later.

Management Autopsy — "revoke access the moment the finding is confirmed"
(COMPOSITE CASE EXAMPLE, CASE-2702)

The decision: A SOC analyst, on confirming an insider-threat finding involving a data-policy violation with no ongoing exfiltration risk, disabled the employee's badge and VPN access immediately, without looping in HR or legal first, and without documenting a rationale beyond the internal case ticket.

Why it seemed reasonable: Containing access fast is the correct instinct for almost every other kind of confirmed compromise this team handles, and nothing about this finding looked different from that pattern in the moment.

How it failed: The employee had filed an unrelated internal complaint the week before, and was separately partway through a documented Performance Improvement Plan for unrelated reasons. HR had no record of why access was revoked or when, and the employee's own manager wasn't told the reason. When the employee's attorney later challenged the termination, the undocumented, same-day access revocation — with no HR or legal record explaining it — became the strongest exhibit for a retaliation theory: it looked, on paper, exactly like a punitive response timed to the prior week's complaint, regardless of what actually motivated it. The case resolved through a negotiated settlement of roughly \$95,000 — a composite, illustrative figure, not drawn from one traceable matter — which outside counsel's own assessment attributed specifically to the absence of a documented, coordinated rationale for the access revocation's timing, not to any weakness in the underlying insider-threat finding itself, which was sound.

The fix: A pre-negotiated emergency protocol that lets the SOC revoke access immediately when the technical risk genuinely requires it, paired with a mandatory, near-simultaneous written notice to HR and legal — within an hour, not by end of day — stating what was revoked and why. The technical action doesn't wait; the documentation of why it couldn't wait has to happen just as fast, so the timeline never has to be reconstructed from memory months later.

6. Performance documentation that holds up under a challenged termination

6.1 What changes once termination is actually on the table

[HR/PEOPLE] Part 16 built the coaching cadence, the diagnostic for what's really causing a recurring miss, and the structural components of a defensible Performance Improvement Plan — problem statement, measurable criteria, committed support, timeline, stated consequence. None of that changes here. What changes, the moment termination moves from a stated possibility to an actual decision, is that the plan and its supporting record need a legal-review checkpoint before anything is delivered to the employee: a check for consistency against how similarly situated employees have been treated, a check for timing against any recent protected activity (a leave request, an accommodation request, a discrimination or harassment complaint), and a check that every criterion in the plan is measurable

enough to survive a pretext argument — a claim that the stated reason for termination wasn't the real one.

6.2 Common legal-review defects and their exposure

[HR/PEOPLE] The table below extends Part 16's PIP-quality checklist with the specific legal-exposure lens a review adds once termination is genuinely on the table — the same defects Part 16 flags as weak coaching practice tend to be exactly the defects that create the most legal exposure, which is not a coincidence.

Defect	Legal exposure it creates	Fix
No documented issue before the PIP	Weakens the "this was already a known, addressed problem" narrative against a retaliation or discrimination claim	Require a dated prior 1:1 record per Part 16 §2.3 before a PIP opens
Inconsistent treatment vs. a comparator	Disparate-treatment claim — why did this person get a PIP or termination for something a peer wasn't disciplined for	Run a comparator check (§6.3) before finalizing any action
Termination timing close to protected activity	Invites a retaliation inference regardless of the underlying performance record's own merits	Legal specifically reviews timing; may recommend delay or an independently documented justification
Vague, unmeasurable criteria	Supports a pretext argument — a vague standard can be claimed to mean anything after the fact	Hold to Part 16 §4.2's measurable-criteria standard, tied to an actual baseline
No coordinated access-offboarding record	Weak account of the organization's own process discipline if litigated; separate security exposure if access lingers	Pair the termination checklist with a documented, timestamped access-revocation step

6.3 The comparator check, concretely

[SENIOR MANAGER] Before finalizing any termination, pull the last 12 months of comparable actions — same alert type of performance issue, same tier, same rough tenure band — and ask whether this specific case is being handled the same way those were, or differently, and if differently, whether there's a real, documented reason. A 40-analyst SOC running two or three employment actions a year that could plausibly draw legal scrutiny doesn't need a formal analytics program to do this — it needs someone, usually HR with the SOC manager's input, actually looking at the short list before signing off, rather than assuming consistency because no one has complained yet.

[EXECUTIVE] When a termination is contested and litigation becomes a realistic outcome, the board-reporting register shifts — a personnel matter that would otherwise never reach executive attention can become a line item in a legal-exposure report the moment outside counsel is engaged. Part 24 — Executive & Board Reporting owns the cadence and format for that conversation; this part's only claim here is that legal's involvement changes what can safely be said in writing about the matter at all, well before it reaches a board deck.

7. Standing audit and regulatory-examiner interactions

7.1 What a SOC manager should expect, on a recurring basis

[SENIOR MANAGER]

Unlike a subpoena or an insider-threat finding, audit and examiner interactions are not rare events triggered by something going wrong — they're a standing, recurring part of running a SOC in almost any regulated or compliance-framework-bound organization, and treating each cycle as a fire drill rather than a routine is the single biggest driver of unnecessary audit stress.

Audit/exam type	Typical cadence	What they actually sample	Manager's standing prep obligation
Internal audit	Annual or continuous, per the internal audit function's own plan	Control design and operating effectiveness against the organization's own stated policies	Keep policy documents and actual practice in sync — internal audit tests the gap between them
Compliance-framework assessment (SOC 2 Type II, ISO 27001 surveillance)	Annual, with SOC 2 Type II covering a review period, commonly 6 or 12 months	A sample of tickets, access logs, or change records across the review period, not every instance	Maintain evidence continuously through the period, not just in the weeks before the assessor arrives
PCI DSS QSA assessment	Annual	A defined sample size against specific requirements (e.g., a sample of log-review evidence across the assessment period)	Log review and monitoring evidence needs to already exist in a QSA-legible form, not be reconstructed after the fact
Regulatory examiner (e.g., a financial or healthcare regulator's exam)	Set by the regulator's own exam cycle, often multi-year	Broader scope; can request specific incident records, policy documents, and interviews	A single named point of contact and a documented, coordinated response process (§7.2)
Cyber-insurance underwriting audit	At renewal, typically annual	Attestations plus, increasingly, evidence for specific controls (MFA coverage, backup testing, EDR deployment)	Keep the same evidence trail current; an underwriting audit that contradicts the policy attestation can affect coverage or premium

7.2 The manager's role during fieldwork

[SENIOR MANAGER]

An auditor or examiner asks for evidence, not assertions — "we have a 4-hour SLA for critical alerts" is a policy statement; the evidence is a sample of tickets showing that SLA was actually met, or a documented exception where it wasn't and why. A manager who lets every analyst answer auditor questions independently, in real time, risks inconsistent answers about the same control from different people who each have a slightly different mental model of how it actually works — designate a single point of contact for the engagement, brief that person on what's actually true (not what should be true), and route every request through them rather than letting fieldwork turn into a dozen informal hallway conversations with different answers.

Blind Spot An audit or examination only tests what it's scoped to sample, and a clean report — a passed SOC 2 Type II, a satisfactory exam letter — says nothing about the controls nobody thought to test this cycle. A manager who treats "we passed" as equivalent to "we're secure" is confusing an audit's specific, narrow scope with a general security posture claim, and that confusion tends to surface at the worst possible time: in a board deck (Part 24) that implies more assurance than the underlying assessment actually provides, or in an incident post-mortem where the exact gap that caused the incident turns out to be one the last three audit cycles never sampled.

Field Test Setup: No upcoming audit is scheduled, but the SOC claims a specific SLA or control operates consistently. **Action:** Pull a random sample of 10 tickets from the last quarter, cold, and ask whoever's on shift to produce evidence — not a description, actual evidence — that the stated SLA or control was met for each one, without going back to the analyst who originally worked the ticket. **Expected result:** For most or all of the 10, evidence should already exist and be retrievable within minutes. If retrieving evidence for a routine claim requires reconstructing it after the fact — asking the original analyst to remember, or rebuilding a timeline from scattered sources — the control isn't actually operating the way it's described, and a real audit sample would surface the same gap, just with less forgiving timing.

7.3 A finding that reveals a charter gap

[SENIOR MANAGER] An audit or exam finding that reveals a scope-boundary or RACI gap Part 4's charter never accounted for is itself worth adding to that charter's own out-of-cycle review triggers — not just fixed locally to satisfy this one finding. A finding that recurs across two consecutive audit cycles because the underlying charter gap was patched narrowly instead of revised is a sign the fix addressed the symptom, not the document that should have named the boundary in the first place.

8. Building the standing interface, not just reacting to one event

8.1 The first-action gap, named for each trigger

[SENIOR MANAGER] Part 4's own worked example showed that a RACI naming who's Accountable for a decision, without naming who's Responsible for the specific first act of notification, has a gap that looks filled on paper and is empty under real time pressure. The table below closes that gap for this part's four recurring triggers specifically — not a replacement for Part 4's general RACI, but the first-action detail it didn't carry.

Trigger	First action required	Who is Responsible for that first action	Target time
Preservation duty recognized (§2.1)	Flag the trigger and any at-risk retention clock to legal	SOC manager or the analyst who first recognizes the trigger	Same business day
Subpoena / LE request received (§3.1)	Route to the named legal contact; no independent response	Whoever physically receives it	Within the same shift
Insider-threat finding confirmed (§5.3)	Deliver the handoff package (§5.2) to HR and legal	SOC manager	Within 24 hours of confirmation, sooner if exfiltration is active
Audit/exam fieldwork request (§7.2)	Route to the single named point of contact	Whoever receives the request	Same business day

8.2 The decision flow, end to end

[CONCEPT] Figure 27.1 traces how a SOC manager should route any of this part's four trigger types once one appears — the shared discipline from §1 expressed as a single walkable path, rather than four separate flows that happen to look similar.

Figure 27.1 — Routing a legal, HR, or audit trigger to its owner. *CONCEPTUAL.* Illustrates the shared decision path across the four trigger types this part covers; it is a structural model of the routing decision, not a capture of any single organization's actual ticketing workflow.

flowchart TD

```

A["SOC becomes aware of a\npotential trigger event"] --> B{"Which kind?"}
B -->|"Preservation duty\n(Sec 2.1)"| C["Flag retention clocks to legal\nimmediately (Sec 2.2)"]
B -->|"Subpoena / LE request\n(Sec 3.1)"| D["Route to named legal contact;\nno independent response"]
B -->|"Insider-threat finding\nconfirmed (Sec 5.1)"| E{"Active exfiltration\nor ongoing access risk?"}
B -->|"Audit / exam request\n(Sec 7.1)"| F["Route to single named\npoint of contact"]
E -->|Yes| G["Contain access now;\nnotify HR/legal in writing\nwithin the hour (Sec 5.3)"]
E -->|No| H["Coordinate timing with\nHR/legal before any\nvisible action"]
C --> I["Document chain of custody\n(Sec 2.4) for anything collected"]
D --> I
G --> J["Deliver handoff package\n(Sec 5.2) to HR and legal"]
H --> J
F --> K["Produce evidence, not\nassertions (Sec 7.2)"]
I --> L["Decision owned by legal,\nHR, or the audit program --\nnot by the SOC alone"]
J --> L
K --> L

```

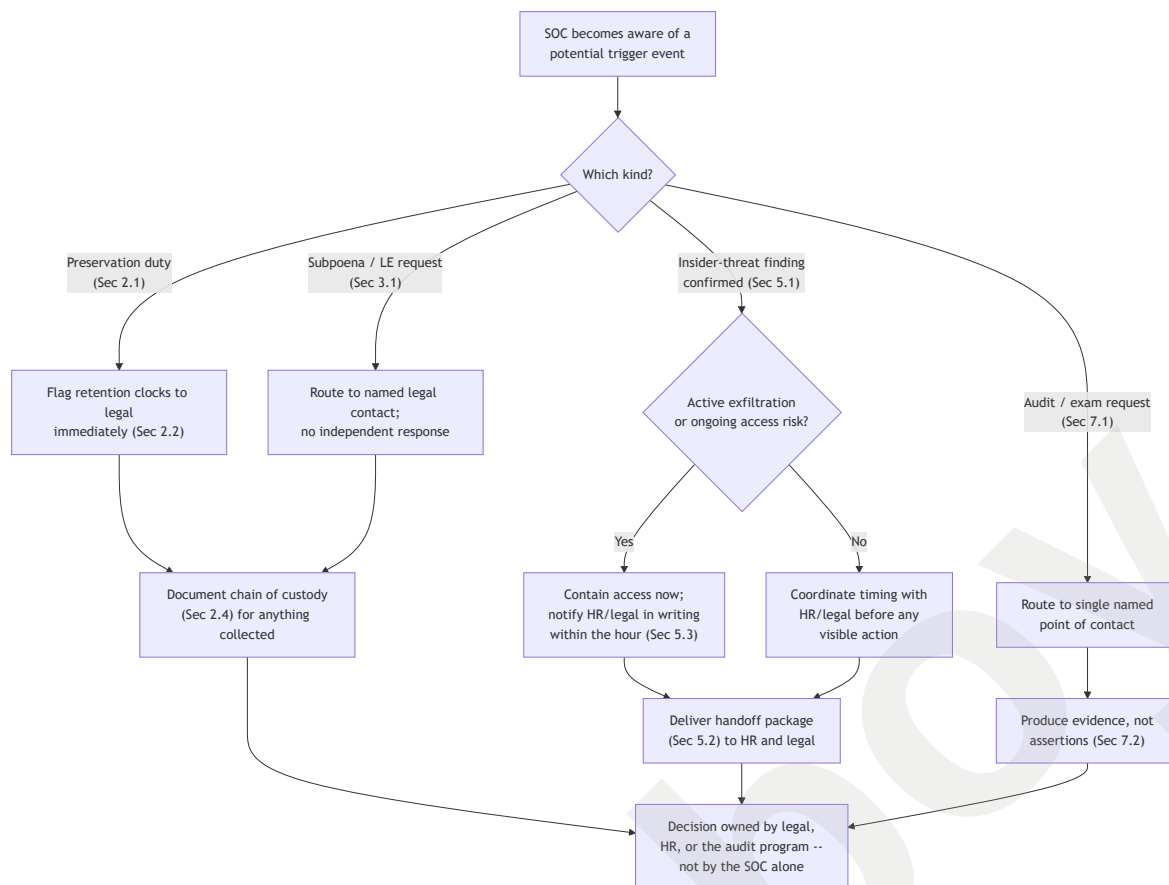


Figure 27.1 — Routing a legal, HR, or audit trigger to its owner. *CONCEPTUAL.* Renders the routing flowchart above as a static SVG. It supports this section's claim that all four trigger types in this part converge on the same shape — recognize, preserve/route/document, then hand off to the actual decision owner. FIG-2701.

8.3 Where the manager's job ends, on purpose

[SENIOR MANAGER] Across evidence preservation, subpoena response, insider-threat aftermath, termination documentation, and audit interaction, the manager's job has the same boundary every time: recognize the trigger fast enough that the organization's options aren't already foreclosed by a purged retention window or an undocumented unilateral action, preserve and document rather than improvise, and hand the actual decision to whoever legally and organizationally owns it. The manager who does all of that and still tries to personally decide whether to comply with a subpoena, whether an insider-threat finding warrants termination, or whether an audit finding is really a finding has stepped past the edge of the job this part describes — and the manager who recognizes none of these triggers until legal, HR, or an examiner brings it to their attention has stepped short of it. The competent middle is the same in all five cases: see it early, don't act alone on the decision itself, and make sure the paper trail would hold up if someone hostile ever read it back in a room the manager wasn't in.

Cross-references

Within this book, this part assumes the standing charter and RACI Part 4 — Organizational Placement & Charter already built — specifically the "preserve evidence / apply legal hold" and "insider-threat

finding hand-off to HR" rows this part fills in procedurally — and the coaching cadence and Performance Improvement Plan mechanics Part 16 — Performance Management & Coaching already built, adding only the legal-review layer those plans need once termination is genuinely on the table. It hands forward to Part 24 — Executive & Board Reporting (the board-reporting register shift once litigation or a regulatory matter is live) and assumes Part 28 — The Manager's Role in a Major Incident owns the live, in-the-moment decision to activate an IR retainer or outside counsel during an active breach, while this part owns the standing engagement-letter and privilege structure that decision plugs into. Outside this book, it cites SOC Playbook Handbook, Insider Threat Playbooks (Category 19) for the detection and escalation mechanics that confirm an insider-threat finding before this part's organizational-aftermath process ever begins.

Section G — Crisis Leadership & Major Incident Management

neshbooy

Part 28 — The Manager's Role in a Major Incident

Why this part exists

[CONCEPT] A declared major incident — a ransomware event, a confirmed large-scale data-exfiltration case, a business-email-compromise chain that's moved real money — puts two very different jobs in the same building at the same time, and the most common way that incident goes worse than it had to is that one person ends up doing both jobs badly instead of two people each doing one job well. SOC Playbook Handbook's Ransomware Master Playbook defines the incident commander role operationally: the person who runs the technical response, directs containment and eradication, and owns the tactical sequencing of the playbook itself. This book has no reason to re-derive any of that, and doesn't. This part is about the other chair in the room — the SOC manager's — and what a manager personally does for the duration of a live major incident that nobody else in the building is positioned to do instead.

[CONCEPT] Four things belong to that chair, and this part covers exactly those four: deciding how to surge staffing onto the incident and who backfills the standing queue while that surge is running; setting and running the cadence of communication upward to the CISO and the board, which is a different audience and a different rhythm than the client-facing communication SOC Playbook Handbook, Part 28 — SOC-to-Stakeholder Communication already owns; deciding when to activate a pre-negotiated incident-response retainer or bring in outside counsel; and drawing the decision-authority line clearly enough, in writing, before an incident happens, that the incident commander is never also the person fielding a board phone call at 2 a.m. That last item is this part's connective thread — the other three all exist, in part, to protect the boundary the fourth one draws.

[CONCEPT] Major-incident content is split three ways across this series on purpose, and none of the three duplicates another. The technical attack-stage analysis — how a specific compromise actually progresses through its stages — is Detection Engineering Handbook V2's applied compromise models (Parts 44–48). The analyst- and incident-commander-level operational playbook — what actually gets done, in what order, to contain and eradicate a specific attack — is SOC Playbook Handbook's Ransomware Master Playbook. This part owns only what's left: the layer neither of those two books has any reason to cover, because neither is written from the manager's chair. What follows deliberately does not build a severity model (that input is SOC Playbook Handbook, Part 29 — Playbook Severity Model, used here only as a trigger condition), does not re-teach the routine volume-surge toolkit already covered in Part 9 — Queue Health & Workload Management, and does not run the post-incident review — that's Part 29 — Post-Incident Organizational Review, once the incident is over and the manager's job shifts from live decision-making to structured retrospective.

1. Two chairs, one incident

[CONCEPT] An incident commander and a SOC manager answer to two different clocks. The incident commander's clock is the attack's: how fast can a compromised host be isolated, how fast can a forensic image be pulled before evidence degrades, how fast can eradication happen without tipping into a premature "all clear" that a lingering foothold makes false. The manager's clock is the organization's:

how fast does the CISO need an accurate answer before making a board update, how fast does a decision to activate a six-figure retainer need to happen before the delay itself becomes the more expensive choice, how fast can a surge staffing decision get made before the rest of the SOC's standing queue quietly goes unworked. Neither clock is optional, and neither person can run both at once without one of them slipping — usually the incident commander's, because a phone that rings with the CISO's name on it is very hard for anyone to ignore, however senior.

1.1 What the incident commander owns

[CONCEPT] The incident commander's job, as SOC Playbook Handbook's Ransomware Master Playbook defines it operationally, is the technical direction of the response: which host gets isolated first, what the containment sequence is, when eradication is safe to start, and when the technical evidence actually supports declaring the incident contained. That job requires sustained attention on a fast-moving technical picture, and it degrades measurably the moment it's interrupted by something that has nothing to do with the attack itself — a status call, a request to "just quickly explain where we are" to someone three organizational layers removed from the containment decision in front of them.

Cross-Book Pointer *This part does not define the incident commander role, the containment/eradication sequence, or when a ransomware incident is safe to declare resolved — that's SOC Playbook Handbook's Ransomware Master Playbook, and it's written at the operational depth this book has no reason to duplicate. Read that playbook for the mechanics of what the incident commander actually does; come back here for what the manager does at the same time, in a different chair, so the two jobs don't collapse into one overloaded person.*

1.2 What the manager owns

[SENIOR MANAGER] The manager's job during the same incident has almost nothing to do with the attack's technical shape and almost everything to do with the organization around it: deciding how many people to pull onto the incident and from where, without leaving the standing queue dangerously thin (§3); setting up and running the upward communication cadence so the CISO and board get accurate, regularly timed information without that information having to come from the person currently directing containment (§4); deciding when a retainer or outside counsel gets activated, and coordinating with Legal on how that engagement is structured (§5); and holding the line, out loud and in writing, on who decides what — so that a board member's reasonable-sounding request to "just get the technical lead on the phone" doesn't quietly become the thing that slows containment down.

1.3 Why collapsing the two chairs is the default failure, not the exception

[SENIOR MANAGER] Organizations rarely decide, on purpose, to have one person run both jobs. It happens by default, because the most senior, most technically credible person in the building is both the obvious choice to direct containment and the person a nervous executive most wants to hear from directly. **CASE-2801** below shows exactly how that default plays out, and what it costs when nobody catches it early enough to correct it mid-incident.

Management Autopsy — "let the incident commander brief the board directly" (CASE-2801, COMPOSITE CASE EXAMPLE)

The decision: During a confirmed ransomware incident at a mid-market SOC, the most senior analyst was designated incident commander per the organization's Ransomware Master Playbook implementation. The CISO, wanting direct technical credibility on every update, had that same person join each scheduled board update call personally rather than routing updates through the SOC manager.

Why it seemed reasonable: The incident commander genuinely was the person with the clearest, most current picture of the attack, and a relayed update always risks losing precision on the way through a second person. Nobody wanted the board hearing a watered-down or lagging version of the truth during a live crisis.

How it failed: Across the first 30 hours, the incident commander joined six board update calls averaging 25 minutes each — roughly 150 minutes of direct call time, plus preparation, pulled straight out of active containment work. The real cost showed up once, at hour 14: telemetry showed lateral movement to a third file server exactly as a scheduled call started. The isolation decision waited 47 minutes for the call to end, because the one person positioned to make it was on the phone with the board instead. During that window, the ransomware encrypted an additional set of files across that third server before isolation finally executed — a delay attributable, directly, to the call, not to any technical limitation. The incremental recovery and rebuild cost attributable to that one avoidable 47-minute window was on the order of \$85,000.

The fix: The SOC manager took over 100% of upward briefing from that point forward. The incident commander was given one standing rule for the remainder of the incident: never pulled off a live technical decision for a call, full stop. A designated scribe fed the manager real-time technical status between calls, and the manager translated that status into board-appropriate language personally — the same translation skill Part 24 — Executive & Board Reporting develops for routine reporting, applied here under crisis time pressure instead of a quarterly cadence.

(The dollar figure above is a **CONCEPTUAL SAMPLE** — illustrative arithmetic built to show the mechanism, not sourced benchmark data from a real organization's incident-recovery invoice.)

[SENIOR MANAGER] Nothing about **CASE-2801** required a bad incident commander or an unreasonable CISO. Both people made locally sensible choices — direct technical credibility on a call sounds like a good idea, and it is, right up until the same person's presence on that call is also the only thing standing between telemetry and an isolation decision. The fix isn't a smarter incident commander; it's a decision-authority boundary that exists on paper before the incident starts, so nobody has to improvise the right call under pressure at hour 14.

What Would Change My Mind This part treats separating the incident-commander chair from the manager's chair as one of the highest-leverage structural moves available during a live major incident. If organizations that kept one person in both chairs throughout a multi-day incident consistently showed no measurable difference in containment time, decision latency, or post-incident review findings compared to organizations that split the roles, that would undercut this chapter's central recommendation and argue for treating the split as situational rather than close to a default rule.

2. The decision-authority boundary, mapped

[SENIOR MANAGER] A boundary that exists only as a shared understanding evaporates under real pressure — a board member calling directly, a CISO asking "can't he just tell me himself," a well-meaning executive walking into the incident room. The boundary has to exist as a written, rehearsed answer to "who decides this" for the specific decisions a major incident actually generates, decided before the incident, not negotiated live during one.

[SENIOR MANAGER] The table below is a working reference for the decisions that recur across almost every declared major incident, mapped to who is accountable for each one, who gets consulted before it's made, and who is simply informed after the fact. It is deliberately narrower than a full incident playbook — it names the decision and its owner, not the technical criteria behind any single decision, which stays with whichever part or companion-book chapter already owns that mechanics.

The table below maps recurring incident-window decisions to who is accountable for each one, so the answer to "who decides this" is settled before an incident, not negotiated during one.

Decision During the Incident Window	Accountable	Consulted	Informed
Declare major-incident status (using the severity trigger from SOC Playbook Handbook, Part 29 — Playbook Severity Model)	SOC Manager	Incident Commander, CISO	Board
Direct technical containment and eradication steps	Incident Commander	SOC Manager (resourcing only)	—
Approve staffing surge and the backfill roster (§3)	SOC Manager	Incident Commander, team leads	CISO
Brief the CISO and the board (§4)	SOC Manager	CISO	Incident Commander (relay only — never joins the call directly)
Activate the IR retainer or engage outside counsel (§5)	SOC Manager, Legal	CISO	Incident Commander
Engage law enforcement	Legal, SOC Manager	CISO	Incident Commander, Board
Ransom-payment decision, where applicable	Board, CISO	Outside Counsel, SOC Manager	Incident Commander
Client or third-party stakeholder notification content and timing	Owner defined by SOC Playbook Handbook, Part 28 — SOC-to-Stakeholder Communication	SOC Manager	Incident Commander
Declare the incident resolved and stand down the surge	SOC Manager	Incident Commander	CISO, Board

[SENIOR MANAGER] Figure 28.1 below sequences the same boundary as a live decision flow rather than a static table — useful in the first hours of a declared incident, when several of these decisions surface in quick succession and the routing has to happen fast enough that nobody defaults to "whoever's already on the phone."

flowchart TD

```

A["A decision surfaces during\nthe incident window"] --> B{"Is it a\n\n technical\n\ncontainment/eradication\n\nstep -- what to isolate,\n\nwhen to eradicate?"}
B -->|Yes| C["Incident Commander decides.\n\nManager protects this person\n\nfrom interruption --\n\nSection 1."]
B -->|No| D{"Does it involve staffing,\n\nbudget, retainer/counsel,\n\nor upward communication?"}
D -->|Yes| E{"Does it require legal\n\nengagement -- privilege,\n\nlaw enforcement,\n\nransom\n\ndecision?"}
E -->|Yes| F["SOC Manager and Legal\n\ndecide jointly -- Section 5.\n\nCISO informed;\n\nBoard\n\naccountable on ransom calls."]
E -->|No| G["SOC Manager decides --\n\nSections 3 and 4.\n\nIncident Commander is\n\nconsulted, never\n\nthe one\n\nfielding the call."]
D -->|No| H["Check Figure 28.1's table\n\nabove -- if it isn't listed,\n\nescalate to Part 25's\n\nrisk-acceptance framework."]

```

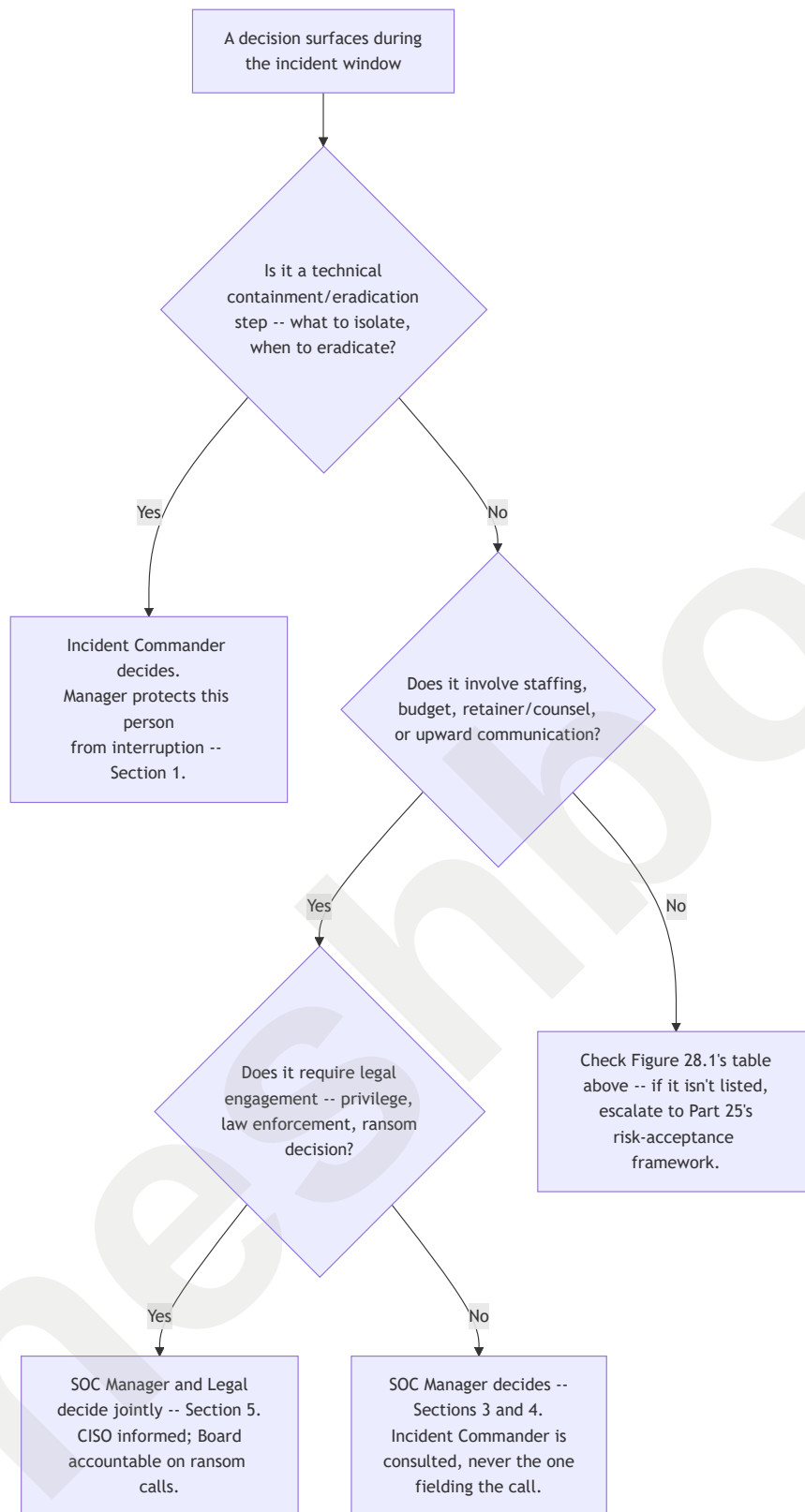


Figure 28.1 — Routing a live incident-window decision to its owner. *CONCEPTUAL.* FIG-2801.

Illustrates the routing logic behind the RACI-style table above, collapsed to three branching questions a manager can run in seconds during a fast-moving incident. It is a structural decision aid, not a capture of any single organization's actual incident-command structure.

Field Test Setup: A written decision-authority boundary exists — Figure 28.1's table, printed and posted where the incident bridge runs, naming the SOC manager as the sole owner of upward communication. **Action:** Run an unannounced tabletop in which a simulated CISO, mid-drill, demands the incident commander "get on the phone right now" while that person is in the middle of a simulated containment decision. **Expected result:** The manager redirects the request to themselves without pulling the incident commander off the technical decision, and the drill's own clock shows no measurable delay to that decision. If the incident commander gets pulled onto the call anyway, the boundary is aspirational, not operational — it needs a rehearsed script and a named owner willing to push back on a CISO in the moment, not just a diagram nobody's tested under pressure.

3. Staffing surge and backfill during a live major incident

[SENIOR MANAGER] Part 9 — Queue Health & Workload Management already covers the routine version of a volume surge: a phishing wave, a scan-and-triage wave off a widely reported vulnerability, a marketing-driven false-positive storm — spikes with an identifiable, usually short-lived cause and an end condition the manager can see coming. A declared major incident is a different kind of surge entirely. It has no natural end condition, it demands the SOC's most experienced people specifically rather than any available headcount, and it competes directly with the standing queue for exactly the analysts who are hardest to replace on short notice.

3.1 Declaring the surge

[SENIOR MANAGER] The decision to formally surge staff onto an incident sits with the manager, triggered by the same severity determination that declares major-incident status in the first place (Figure 28.1's first row). Declaring the surge is a separate act from declaring the incident: an incident can be declared major and still run with a small, dedicated response team for its first hour or two while the manager assesses scope, before a full surge — pulling analysts off other duties, activating a backfill roster, potentially engaging contract surge staff — is actually justified. Surging immediately and fully, before scope is even roughly known, routinely over-commits headcount to an incident that turns out to be smaller than the first hour suggested, at the direct cost of the standing queue described in §3.2.

3.2 Building the incident team without hollowing out the standing queue

[SENIOR MANAGER] The instinct under pressure is to pull the most senior, most capable analysts onto the incident bridge — which is usually correct for the incident itself and dangerous for everything else the SOC is still responsible for at the same time. [CASE-2802](#) shows the failure mode concretely.

Management Autopsy — "pull the best people, whatever it costs the standing queue"
(CASE-2802, COMPOSITE CASE EXAMPLE)

The decision: A 16-analyst SOC running three shifts, each normally staffed at five analysts after accounting for Part 5's shrinkage model, declared a major ransomware incident. The manager pulled nine of the 16 analysts onto the incident bridge across the first 24 hours — the most senior person from every shift, plus several others with relevant experience — leaving seven to cover the standing 24/7 queue across all three shifts combined.

Why it seemed reasonable: The incident was, by a wide margin, the highest-severity thing happening anywhere in the organization that day, and every one of the nine analysts pulled was a genuinely strong, defensible choice for the incident team specifically.

How it failed: Seven analysts spread across three shifts works out to roughly two per shift instead of five — well under half of normal standing coverage, with no adjustment made to intake volume or SLA targets for the rest of the queue during the surge. At roughly hour 30, a legitimate, unrelated business-email-compromise alert landed in the understaffed standing queue and sat untouched for 11 hours, because the two analysts on that shift were both buried in a backlog the reduced headcount had no way to clear. It was caught only because the targeted client called in, asking why an invoice-change request looked suspicious — not because anything in the SOC's own queue-health signals (Part 9) flagged it, since nobody was watching those signals closely with the team's attention entirely on the incident.

The fix: Cap the surge pull at a defined ceiling — no more than 40% of any single shift's standing headcount pulled onto an incident without a named backfill replacement for each person pulled — and build that backfill roster before an incident happens, not during one: identified backup analysts on a standing on-call list, or a pre-negotiated MSSP overflow-burst clause (Part 22 — MSSP & Managed-Service Contract Management owns negotiating that clause) that can absorb standing-queue volume specifically so the SOC's own remaining staff aren't asked to do five people's work with two.

[SENIOR MANAGER] The arithmetic behind the 40% ceiling is worth stating plainly, because it's the number a manager actually has to defend to a CISO who wants every capable person on the incident, right now: pulling more than that from a single shift routinely pushes standing-queue coverage below the level at which Part 9's queue-health signals can even be trusted, since the same understaffed shift that's failing to clear the backlog is also the shift least likely to have spare attention to notice and report that the backlog is failing. A surge that quietly disables its own early-warning system for a second, unrelated problem is not a surge that's actually under control — it's a surge that's traded one visible crisis for a second, invisible one.

3.3 Rotation, rest, and the multi-day incident

[HR/PEOPLE] A major incident that runs past a single shift's worth of hours — most do — turns rest into an operational decision, not a wellness afterthought. An incident commander or analyst running on 20 straight hours makes measurably worse containment decisions than one working a defined rotation,

and "the incident is too urgent to rotate people out" is exactly backward: the more urgent the incident, the more a fatigue-driven bad call costs. The manager's job here is narrow but concrete: name a rotation schedule for the incident team within the first several hours of a multi-day incident, not after someone's judgment has already visibly slipped, and hold to it even when the person being rotated out wants to stay.

[HR/PEOPLE] This part does not build the rest-and-recovery program itself — mandatory disconnected time after a major incident, the fatigue research behind rotation limits, and the broader wellbeing program this connects to are Part 17 — Burnout, Fatigue & Wellbeing's job. What belongs here is narrower: naming rotation as a live decision the manager has to make during the incident, not just after it, because a rotation plan that only exists in the post-incident wellbeing program arrives too late to protect the judgment calls made during hours 20 through 40.

4. Executive and board communication cadence during the incident

[EXECUTIVE] Part 24 — Executive & Board Reporting builds the standing cadence and format for translating operational metrics into a business-risk narrative a board can act on — a quarterly or monthly rhythm, built around metrics that are already stable and defined. A live major incident breaks that rhythm on purpose: the board needs information far more often, the information itself is changing hour to hour, and "we don't know yet" is frequently the honest, correct answer rather than a sign the reporting is failing. This section is the crisis-specific deviation from Part 24's baseline, not a second version of it — once the incident resolves, communication cadence steps back down to Part 24's standing rhythm, covered in §4.3.

4.1 Cadence by incident phase

[EXECUTIVE] Cadence should shift with the incident's phase, not stay fixed at one interval from declaration to resolution. The table below sets a starting cadence a manager can adapt, built around the reality that the first few hours need the fastest rhythm and the least formal format, while later phases can slow down and formalize.

The table below maps incident phase to communication cadence, primary audience, and what changes from Part 24's routine-reporting baseline.

Incident Phase	Typical Cadence	Primary Audience	What Changes From Routine Reporting (Part 24)
Hour 0–4 (detection and declaration)	Every 30–60 minutes, or on any milestone	CISO directly; board chair or audit-committee chair informed	Verbal/phone-first, not a deck; content is "what we know, what we don't, when the next update lands"
Hour 4–24 (active containment)	Every 2 hours, plus milestone-triggered updates	CISO, full board or a designated incident sub-committee	Short written updates supplement the verbal cadence; still no formal board-deck materials
Day 2–5 (containment holds, recovery begins)	Twice daily (for example, 09:00 and 17:00)	Full board; regulators or the cyber-insurance carrier often briefed in parallel	First board-deck-style materials appear, adapted from Part 24 and Appendix A7's templates rather than built from scratch mid-incident
Day 5+ (stabilization)	Daily, stepping down toward Part 24's standing cadence	Full board	Cadence returns to Part 24's routine rhythm once the incident commander formally stands the response down

Blind Spot A fixed-interval board cadence — an update every 2 hours, on the clock, regardless of what happened — still reports "no material change" at the 2-hour mark even when the actual milestone (a ransom note discovered, exfiltration confirmed) happened 20 minutes earlier and simply hadn't been packaged into an update yet. Calendar-driven cadence is predictable, which the board values, but it is not the same thing as milestone-driven cadence, and a manager who only watches the clock will occasionally deliver a materially stale "no change" update by accident. Pair the fixed cadence with a standing rule: any milestone on a defined trigger list — a ransom note, confirmed exfiltration, confirmed lateral movement to a new business-critical system — generates an immediate out-of-cadence update rather than waiting for the next scheduled slot.

4.2 What goes in a crisis briefing that doesn't belong in a routine one

[EXECUTIVE] Part 24 already covers translating a stable metric into a business-risk narrative. A crisis briefing carries three things a routine board report almost never has to: an explicit statement of what's still unknown, stated as unknown rather than papered over with a confident-sounding placeholder; a specific "next update" time, stated and kept, because a board that doesn't hear back when promised starts generating its own information requests through side channels the manager can't control; and a running decision log — what's been decided, by whom, and when — because a multi-day incident generates enough decisions that neither the board nor the manager can reliably reconstruct the sequence from memory once the post-incident review (Part 29) needs it.

4.3 Layering the audience

[EXECUTIVE] Not every crisis update goes to every audience at the same depth. The CISO typically needs the fullest technical picture the manager can translate, fast, because the CISO is the one making board-facing judgment calls in real time. The board itself usually needs less technical detail and more risk-and-decision framing — what's the exposure, what's the plan, what does the organization need

from the board right now (a ransom-payment decision, an insurer notification, nothing at all yet). Regulators and the cyber-insurance carrier, when they enter the picture, need a third layer again, shaped by specific notification-timing obligations that sit outside this part's scope entirely.

Cross-Book Pointer *This part covers upward communication cadence only — what the manager tells the CISO and the board, and how often. It does not cover client-facing or third-party stakeholder communication during the same incident — what gets told to an affected customer, a partner, or the public, and when. That's SOC Playbook Handbook, Part 28 — SOC-to-Stakeholder Communication, and the two communication tracks run in parallel during a real incident without being the same conversation: a board update can honestly say "we don't know the full scope yet" in a way a customer-facing notification usually can't afford to.*

5. Activating an IR retainer or outside counsel

[VENDOR/PROCUREMENT] A retainer or outside-counsel engagement that has to be negotiated from scratch during a live incident is, by definition, arriving too slowly to help with that incident. This section covers the manager's live decision — when to pull the trigger on an engagement that should already be pre-negotiated — not the contract-negotiation mechanics behind getting that engagement in place before an incident ever happens.

5.1 Trigger criteria

[SENIOR MANAGER] The decision to activate belongs to the manager, made jointly with Legal per Figure 28.1's routing table, and it should be triggered by a short, pre-agreed list of conditions rather than decided fresh under pressure each time: confirmed encryption or destructive activity across more than a small, pre-agreed number of hosts; confirmed or strongly suspected data exfiltration; the appearance of a ransom note or direct attacker communication; or a scope that has grown past what the in-house team can credibly contain and investigate on its own within the incident's operationally relevant timeframe. Waiting for full certainty on any of these before activating is the single most common way a manager turns a fast-response retainer into a slow one.

Manager's Note *Call the retainer firm's activation line the moment you have a credible major-incident signal, not once you're certain. A false start costs one phone call and, at most, a small activation fee; waiting for certainty costs the hours between "we think this is ransomware" and "we've confirmed it's ransomware" — hours a fast onsite-response SLA can't make up once it's already ticking against a later start time.*

5.2 Privilege, engagement order, and where this part stops

[SENIOR MANAGER] Legal typically needs to engage outside counsel first, ahead of the IR firm itself, so the IR engagement runs under attorney-client privilege from the start rather than being retrofitted onto an already-open investigation. That ordering decision — counsel first, then the IR firm operating under

counsel's direction — is the manager's to trigger but Legal's to actually structure. What that structure means procedurally once counsel is engaged — evidence-preservation holds, subpoena or regulator response, the specific mechanics of privileged versus non-privileged findings — is Part 27 — Legal, HR & Compliance Interfaces's territory, not this part's. This part's job stops at recognizing the trigger and making the call to activate; Part 27 picks up everything that follows from that point forward.

Operational Reality *A retainer contract's onsite-response SLA clock almost never starts the moment the manager places the call — it starts once the statement of work is signed and the engagement is formally opened, which in practice means routing through Legal first so outside counsel can direct the engagement under privilege. A 4-hour SLA quoted at signing routinely becomes 6 to 8 hours in practice once legal review, scoping calls, and the privilege letter are actually accounted for. Build that gap into the incident timeline the manager is mentally running, instead of discovering it live and treating it as the vendor missing its commitment.*

5.3 What not having one pre-negotiated actually costs

[VENDOR/PROCUREMENT] The cost difference between an activated retainer and a cold engagement is large enough to be the argument that gets a retainer funded in the first place, and it's worth stating in the manager's own numbers rather than a vendor's sales deck. A mid-market organization's pre-negotiated retainer, at roughly \$60,000 a year, secures priority access and a 4-hour initial-response SLA once the engagement opens, billed at a blended rate of roughly \$450 an hour for the responding team from that point. The same firm, engaged cold with no prior relationship or signed retainer, routinely quotes 48 to 72 hours to staff and mobilize an unfamiliar client, at a rate closer to \$650 an hour and with no SLA commitment at all — because a firm with no standing agreement has no obligation to prioritize a new client over its existing retainer clients, who called first and already have a contract. Forty-eight to 72 hours of unmanaged dwell time on an active ransomware or exfiltration event routinely costs an order of magnitude more than a year of retainer fees, which is the comparison worth having ready the next time this line item competes against something else in the budget conversation. Part 20 — Building & Defending the SOC Budget owns.

*(The dollar figures and lead-time ranges above are a **CONCEPTUAL SAMPLE** — illustrative arithmetic built to show the mechanism, not sourced pricing from any specific IR firm's current rate card.)*

[VENDOR/PROCUREMENT] This part covers only the manager's live decision to activate a retainer or engage outside counsel during an incident. It does not cover negotiating the retainer contract itself — the SLA terms, the right-to-audit clause, the pricing structure, or the burst-capacity provisions that make activation fast when the moment actually arrives. Part 22 — MSSP & Managed-Service Contract Management owns that negotiation; make sure it's already done, because §5.1's trigger criteria are only useful against a contract that already exists.

6. When the manager is also the incident commander

[SENIOR MANAGER] Every recommendation above assumes a SOC large enough to actually split the two chairs. Plenty of SOC's aren't. A four- or five-analyst team's most senior person is very often the

manager, the most technically credible incident responder, and the only plausible candidate for both roles at once, with no realistic second person to hand either job to during a live incident.

Operational Reality *A small SOC's honest answer to "who's the incident commander and who's the manager" is frequently "the same person, because there is no one else" — and no amount of process design changes that fact mid-incident. What a small-SOC manager can still do is protect the boundary's function even without a second person to fill it: designate whoever's next most senior (even a strong Tier 2 analyst without formal incident-command experience) as a technical scribe and communication buffer, so at least the timing problem from CASE-2801 — being pulled off a containment decision mid-call — is reduced, even though the decision-authority separation itself can't be fully achieved with one person wearing both hats. Naming this limitation honestly, in the incident plan itself, beats pretending a two-person structure exists when it doesn't.*

[SENIOR MANAGER] The honest fallback for a genuinely undersized team is external, not organizational: this is exactly the scenario a pre-negotiated retainer (§5) and an MSSP overflow arrangement (Part 22) exist to cover, by adding response capacity from outside the building rather than asking one person to be two people at once. A small SOC that has neither a retainer nor an overflow arrangement in place, and also has no way to split the incident-commander and manager roles internally, is carrying a structural gap that Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty would treat as a risk worth naming and formally accepting or escalating — not a gap to discover for the first time during the incident itself.

7. Where this goes next

[CONCEPT] Everything in this part assumes the incident eventually resolves, the surge stands down, and the organization moves from live decision-making into structured retrospective — that transition, and the organizational blameless-postmortem program built to make sure the same finding doesn't recur across future incidents, is Part 29 — Post-Incident Organizational Review's job. The decision-authority boundary this part builds in §2 is also worth revisiting outside of any live incident: Part 25's risk-acceptance framework and Part 4 — Organizational Placement & Charter's standing RACI both benefit from a written incident-authority boundary that's been rehearsed via the Field Test in §2, not just filed away until the next real incident tests it for the first time under pressure.

Cross-references

Within this book: Part 1 — SOC Manager Foundations & the Series Map (names crisis leadership as one of the four decision domains this part develops in full); Part 5 — Headcount & Capacity Modeling (the baseline shrinkage and staffing model that §3.2's surge-ceiling math is measured against); Part 9 — Queue Health & Workload Management (the routine volume-surge toolkit this part's declared-incident surge is explicitly distinct from); Part 17 — Burnout, Fatigue & Wellbeing (mandatory disconnected time and rotation research that §3.3 names as a live decision but does not build); Part 20 — Building &

Defending the SOC Budget (the business case a retainer's cost comparison in §5.3 feeds); Part 22 — MSSP & Managed-Service Contract Management (negotiating the retainer SLA, right-to-audit clause, and burst-capacity provisions that §5's activation decision depends on already existing); Part 24 — Executive & Board Reporting (the standing cadence and translation skill this part's crisis cadence in §4 deviates from and returns to); Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty (the framework for a structural gap like §6's small-SOC role collapse, and for any incident-window decision Figure 28.1's table doesn't resolve cleanly); Part 27 — Legal, HR & Compliance Interfaces (the procedural mechanics — privilege, evidence holds, subpoena response — that begin once §5.2's activation decision is made); Part 29 — Post-Incident Organizational Review (the retrospective this part's live decisions eventually feed).

SOC Playbook Handbook: the Ransomware Master Playbook (defines the incident commander role and the technical containment/eradication mechanics this part deliberately does not re-derive); Part 28 — SOC-to-Stakeholder Communication (client-facing and third-party communication during the same incident, run in parallel with but distinct from this part's upward-only cadence in §4); Part 29 — Playbook Severity Model (the severity trigger this part uses, in §2's table, to decide when major-incident status is declared).

Detection Engineering Handbook V2: the applied compromise models (Parts 44–48) own the technical attack-stage analysis behind any specific incident this part's manager is responding to — cited once here as the third leg of the series' deliberate three-way split of major-incident content, and not referenced again because none of that technical detail changes anything this part's manager actually decides.

Part 29 — Post-Incident Organizational Review

Why this part exists

[CONCEPT] Most SOC teams that have run for more than a year or two already hold a postmortem after a serious incident. Fewer of them can answer a harder question: has this specific finding — this exact root cause, worded slightly differently each time — shown up in a postmortem before? A single well-run postmortem can be excellent on its own terms: a clear timeline, an honest account of what went wrong, a list of fixes everyone in the room agrees to. None of that guarantees the fix survives past the meeting, and none of it tells anyone, six incidents and 14 months later, that the team is looking at the same underlying gap for the third time. This part is about the layer above any individual postmortem — the standing program that decides which incidents get reviewed, who sits in the room across dozens of reviews rather than just one, and the tracking system whose entire job is making a repeat finding visible before it becomes a fourth incident.

[CONCEPT] SOC Playbook Handbook, Part 35 — Playbook Failure Examples already owns the individual-breakdown lens: it walks specific playbooks that failed in specific incidents, the exact step that broke, and the mechanical fix to that playbook. This part does not re-walk that ground and does not build a second catalog of failed playbooks. Its unit of analysis is the pattern across many incidents' findings, not the mechanics of any one incident's breakdown — a program-level question about whether the organization is actually learning, not a technical question about what one playbook got wrong.

[CONCEPT] Two assumptions carry over from earlier in this book rather than getting rebuilt here. Part 19 — Team Culture & Psychological Safety establishes blameless postmortem culture as a precondition — the belief, earned through consistent practice, that naming a mistake honestly will not end a career. This part assumes that precondition exists, or is actively being built, and focuses on the program mechanics that sit on top of it; it does not re-argue why blamelessness matters. Part 28 — The Manager's Role in a Major Incident covers what the manager personally does while the incident is still live — staffing surge, board communication, decision authority. This part picks up exactly where that one ends: the incident is over, and now someone has to decide whether it gets reviewed, by whom, on what timeline, and what happens to what the review finds.

[CONCEPT] Four mechanics make up the program this part builds: trigger criteria that decide which incidents and near-misses earn a full review (§2); the cadence and participant structure across both the single-incident meeting and a standing review board that looks at many incidents together (§3–4); a corrective-action tracking system built specifically to catch a repeat finding before it becomes a third incident (§5); and the pattern-detection discipline that only exists at the organizational level, never inside any one incident's own review (§6). Appendix A7 carries the fillable post-incident organizational review template and corrective-action tracker this part builds toward.

1. From a good postmortem to a program that remembers

1.1 What changes when the unit of analysis is the program, not the incident

[CONCEPT] A single postmortem answers "what happened in this incident, and what should we do differently." A program answers a different question entirely: "across every incident we've reviewed this year, is the same underlying gap showing up more than once, and is anyone actually watching for that." Those two questions need different infrastructure. The first needs a good meeting, a skilled facilitator, and an honest room. The second needs a place all those meetings' findings actually land — a registry that persists after the meeting ends, survives the facilitator moving to a different role, and can be queried across 12 months of incidents at once rather than read one report at a time.

[SENIOR MANAGER] Most SOC's that think they have a mature postmortem practice actually have a mature postmortem *meeting* practice and nothing at the program layer. Every serious incident gets a well-attended review, a clear document, and a list of action items assigned to named owners — and every one of those documents lives in its own folder, named after its own incident, never rolled up anywhere a manager could see that "insufficient EDR console access during off-hours" has now appeared as a finding in three unrelated documents across the last year. The meeting practice is genuinely good. The program is missing, and it's the program's job — not any single meeting's job — to notice the third occurrence.

1.2 Blameless as an assumed precondition

[HR/PEOPLE] Everything this part builds — trigger criteria, participant lists, a tracking system — depends on the room being honest about what actually happened, and a room is only honest under that pressure if psychological safety already exists as a lived norm, not a slide in the kickoff deck. Part 19 develops what that actually requires: consistent practice over time, leadership that visibly protects someone who names their own mistake, and enough repetition that "blameless" stops being a word management says and becomes a thing analysts have personally tested and found true. A manager standing up the program below in an organization where that precondition doesn't yet hold should expect the first several reviews to under-report — people will talk around the real cause rather than name it — and should treat that as a signal to invest in Part 19's groundwork before assuming this part's mechanics are the missing piece.

Cross-Book Pointer *This part does not walk individual playbook breakdowns — SOC Playbook Handbook, Part 35 — Playbook Failure Examples already owns that ground in detail, with specific playbooks and the specific step that failed inside each one. Reach for Part 35 when the question is "what exactly broke in this one incident's response"; stay here when the question is "is this the second or third time something like this has broken across different incidents, and does anyone have a system that would even notice."*

2. Trigger criteria: which incidents earn a full review

2.1 Severity as the backbone trigger

[SENIOR MANAGER] A program that reviews every closed ticket, the way Part 15's QA program deliberately doesn't, drowns in review overhead before it produces anything useful; a program that only reviews the incidents that made the news gets an unrepresentative sample from the opposite direction.

The workable middle, and the one most mature programs converge on, ties the trigger to the severity classification SOC Playbook Handbook, Part 29 — Playbook Severity Model already defines: every incident scored at the top one or two severity tiers gets a mandatory full organizational review, full stop, with no manager discretion to skip it. This part does not redefine that severity scale — it consumes the tier boundary as an already-settled input, the same way Part 9 consumes it for queue-health diagnosis.

[SENIOR MANAGER] Below that mandatory line, most programs add a discretionary trigger: any incident commander, team lead, or the SOC manager can request a full review regardless of severity score, and a fixed, small percentage of lower-severity closures get pulled at random on the same logic Part 15's QA sampling uses — not because most of them need a full review, but because a severity score assigned in the heat of the incident is itself sometimes wrong, and a random floor catches the review-worthy incident that a rushed scoring call underrated.

2.2 The near-miss trigger most programs skip

[CONCEPT] Safety-engineering research going back to Heinrich's 1930s accident-ratio studies — since revised and criticized on its exact numbers, but not on its central claim — argues that near-misses vastly outnumber the incidents that actually cause damage, and that the near-misses carry most of the same causal information at a fraction of the cost of learning it. A SOC's version of a near-miss is a real one: an alert that should have fired and didn't, a playbook step that got skipped and happened not to matter this time, an analyst who noticed something felt wrong and escalated on instinct rather than because any tool told them to. None of these produce an incident. All of them can produce the same finding an incident eventually would, before the incident happens.

Blind Spot *A trigger list built only from realized incidents systematically excludes exactly the events most likely to contain a preventable finding before it costs anything — the near-miss where the team got lucky rather than got it right. A program that reviews every top-severity incident but has no path for an analyst to flag "this almost went badly and I don't think anyone else knows how close it was" is watching only the incidents where luck ran out, and missing every one where it happened to hold.*

[FRONTLINE MANAGER] The practical fix costs almost nothing: a standing, low-friction path — a single form field, a Slack command, a line item at shift handoff — for any analyst to flag a near-miss for review, with an explicit norm that flagging one is treated the same as reporting any other honest finding, never as an admission of fault. The volume this produces is usually modest — most programs report single-digit near-miss flags a month even in a busy SOC — which is exactly why it's cheap to run and easy to justify keeping even when budget pressure targets the review program first.

3. Cadence and participants: two layers, not one

3.1 The per-incident postmortem: timing and facilitation

[FRONTLINE MANAGER] The single-incident review has a timing window worth defending explicitly: run it too soon and the room is still in triage mode, relitigating disposition instead of reviewing process;

wait too long and memory degrades, people move to other incidents, and the review becomes an argument about what the logs say rather than a shared account of what happened. 5 to 10 business days after the incident closes is the range most programs converge on — long enough for the immediate adrenaline to clear and the full timeline to be assembled, short enough that everyone in the room still remembers the incident as something they lived through rather than something they're reconstructing from a ticket.

Manager's Note *Never schedule the review for the same day the incident closes, even when everyone's calendar is technically free that afternoon and the temptation to "just get it done" is real. The people who worked the incident are still running on the adrenaline that made triage decisions feel obvious in the moment; give it at least a couple of days before asking them to evaluate those same decisions with a clear head.*

[FRONTLINE MANAGER] Facilitation is a role, not a volunteer job that falls to whoever's in the room with the most seniority. The facilitator's only job is protecting the process — keeping the room on "what happened and why" rather than "whose fault was it," making sure the quietest person in the room gets asked a direct question before the loudest person gets to summarize for everyone, and closing the meeting with named owners and dates rather than a vague consensus that something should probably get fixed eventually. A facilitator who also worked the incident is compromised in exactly the way a reviewer scoring their own report is compromised in Part 15's QA program — rotate the role, or pull a neutral facilitator (a peer team lead from a different shift, an HR/People partner trained for it) from outside the incident's own chain.

3.2 The standing review board: the layer that watches the watchers

[SENIOR MANAGER] Nothing above this line requires anyone to look across incidents — every mechanic so far lives inside one incident's own review. The standing review board is the piece that exists purely to look sideways: a fixed group that meets on its own cadence, independent of any single incident's timing, and reads a rolling batch of recent postmortem summaries together specifically looking for the pattern no single incident's own room would ever be positioned to see. A monthly cadence is the common default for a SOC running enough volume to generate more than one or two full reviews a month; a quarterly cadence is reasonable for a smaller team where monthly would mean reviewing the same one or two incidents' findings repeatedly with nothing new to compare them against.

[SENIOR MANAGER] Board membership should be smaller and more stable than any single incident's review room: the SOC manager, one or two rotating team leads, a detection-engineering or threat-hunting representative when a finding trends technical, and — critically — someone who was not personally involved in most of the incidents under review that cycle, playing the same outside-perspective role the facilitator plays inside a single review. The board's output is not a new set of individual incident findings; it's a short, standing list of patterns — findings that have now shown up more than once, worded differently each time, that no single incident's review room had the visibility to notice on its own.

3.3 Who owns what, across both layers

[SENIOR MANAGER] The table below states accountability across the two layers this section builds — the per-incident review and the standing board — because the most common governance failure in this program is treating them as the same meeting with the same owner, which quietly starves the board layer of the independent attention it needs.

Table 29.1 — Post-incident review RACI, per-incident review vs. standing board. (*CONCEPTUAL SAMPLE.*) Use this table when review ownership is informal or when the standing board keeps getting cancelled in favor of "we already covered that in the incident's own review" — a sign the two layers have collapsed into one.

Activity	SOC Manager	Incident Commander / Facilitator	Team Leads	Analysts Involved	Standing Review Board
Decide if an incident meets the review trigger	A	C	I	I	I
Facilitate the per-incident review	I	R	C	C	—
Assign corrective-action owners from that review	A	R	C	I	—
Log findings to the corrective-action tracker	I	R	I	I	I
Review a rolling batch of findings for recurring patterns	C	I	C	I	R/A
Escalate a confirmed pattern to budget or staffing decisions	A	I	C	I	R
Report program health metrics upward	A	I	I	—	C

[SENIOR MANAGER] The row worth reading twice is pattern review: the standing board is both responsible and accountable for it, and nobody else on the table shares that row at more than a consulted level. That's deliberate — pattern detection is the one activity in this whole program that a per-incident review room structurally cannot do well, because it only ever sees one incident's findings at a time.

4. Keeping the review blameless when it counts, not just in the charter

4.1 Facilitation mechanics that actually protect the room

[HR/PEOPLE] Saying a review is blameless in the meeting invite protects nobody. What actually protects the room is a small set of enforced mechanics: the facilitator opens by naming the norm out loud every time, not just the first time the program ran; questions are phrased about the system, not the person ("what made it hard to see this in time" rather than "why didn't you catch this"); and the write-up that leaves the room describes actions and timeline, never attributing a mistake to a named individual in the document that circulates beyond the room itself. None of this is about hiding accountability — a manager still knows, and still handles, an individual performance issue through Part 16's process where one genuinely exists. It's about making sure the review's own document never becomes the mechanism that punishes someone for being honest in the room that produced it.

4.2 Management Autopsy — the review that named names

Management Autopsy — "publish the incident timeline with the analyst's name attached" (COMPOSITE CASE EXAMPLE, [CASE-2901](#))

The decision: A 25-analyst in-house SOC ran a full postmortem after a missed lateral-movement indicator sat unescalated for six hours during a real intrusion. The write-up, circulated to the SOC manager's own leadership chain and a handful of stakeholders outside the SOC, named the analyst who missed the escalation directly in the timeline: "Analyst Rivera reviewed the alert at 02:14 and closed it as a false positive without checking the associated process tree."

Why it seemed reasonable: The timeline needed to be specific and accurate to be useful, and the facilitator reasoned that naming who did what was simply factual — no different from naming which tool generated which alert. Leadership had also asked for "accountability" in the wake of the incident, and a document that read as vague about who did what risked looking evasive.

How it failed: Within two review cycles, the volume of near-miss flags and the honesty of what analysts volunteered in reviews both measurably dropped — one flag the following month against a running average of four to six a month in the prior half-year, and the reviews that did happen produced timelines noticeably thinner on the specific judgment calls analysts had made in the moment. Two senior analysts, in separate 1:1s unrelated to each other, told their team lead directly that they'd stopped volunteering details in reviews "in case it ends up in a document with my name on it going to people outside the team." The program had accidentally taught the floor that a postmortem's write-up was a risk to personal reputation, not a shared account of what happened — and once that lesson lands, it's expensive to unlearn even after the practice changes, because the first bad instance is the one people remember.

The fix: The write-up format changed permanently: the document that circulates beyond the review room itself never names an individual analyst against a specific action — "the reviewing analyst," "the on-call engineer," roles rather than names. A separate, narrower record — visible only to the SOC manager and, where relevant, HR/People — exists for the rare case where an individual performance issue genuinely needs to be tracked, kept entirely apart from the postmortem document and routed through Part 16's process instead of living inside the incident review's own paper trail.

4.3 Field Test — validating psychological safety in the room, not assuming it

Field Test Setup: A postmortem program has been running for at least two full review cycles, with a stated blameless norm but no direct measurement of whether the room actually believes it. **Action:** Immediately after a review meeting, send an anonymous two-question pulse survey to every attendee: "Did you say everything relevant in that room, or did you hold something back?" and "Would you flag a near-miss you personally caused, based on what you just saw happen in that meeting?" **Expected result:** A healthy program should see the large majority answer "said everything" and "yes, I would flag it." A meaningful share answering "held something back," especially concentrated among more junior or newer attendees, means the stated norm and the room's actual experience of the norm have diverged — treat that as a signal to revisit facilitation mechanics (§4.1) before running another cycle, not as noise to average away over time.

5. The corrective-action tracking system

5.1 What a tracked finding actually needs to contain

[SENIOR MANAGER] A finding that leaves a postmortem as a bullet point in a document and never becomes a structured record in a persistent system is, functionally, not tracked at all — it exists exactly as long as someone remembers to reread that specific document, which in practice means it exists for a few weeks. A tracked finding needs, at minimum: a plain-language statement of the gap (not the fix — the gap itself, stated so a reader six months later understands what was actually wrong); a category from a fixed taxonomy (§5.2); a named owner who is a person, not a team; a due date; a verification method stated in advance — what evidence would prove this is actually fixed, decided before anyone's incentive to declare victory early kicks in; and a link back to the incident or near-miss that produced it, so a later reviewer can trace a pattern to its origin incidents rather than working from the finding's text alone.

5.2 Taxonomy: routing a finding to where its real fix lives

[SENIOR MANAGER] The single most common way a corrective-action tracker turns into a graveyard is a finding assigned to "the SOC" with no category and no clear sense of which part of the organization's own toolkit actually owns fixing it. A fixed, small taxonomy solves most of that, because it forces the question "which kind of gap is this, actually" at the moment the finding is logged, not months later when nobody remembers the incident well enough to reconstruct it.

Table 29.2 — Finding taxonomy: category, what it looks like, where the real fix lives.

(CONCEPTUAL SAMPLE.) Use this table at the moment a finding is logged, not after — categorizing a finding correctly up front is what lets the pattern-detection work in §6 actually function across a full year of tracked findings.

Category	What It Looks Like in a Postmortem	Where the Real Fix Lives
Staffing / coverage gap	"Nobody with the right access was on shift when this needed a decision"	This book, Part 5 — Headcount & Capacity Modeling, Part 6 — Shift Pattern & Coverage Design
Training / competency gap	"The analyst didn't know this escalation path existed"	This book, Part 10 — Competency Models & Skills Matrices, Part 12 — Ongoing Training & Skill Development
Detection debt / rule gap	"The rule that should have caught this was never tuned after it started firing on the wrong thing"	Detection Engineering Handbook V2, Part 43 — Detection Debt
Playbook / process gap	"The playbook's escalation step assumed access this analyst didn't have"	SOC Playbook Handbook, Part 35 — Playbook Failure Examples
Tooling / vendor gap	"The EDR console was down for maintenance and nobody had a fallback"	This book, Part 21 — Tooling Procurement & Platform Strategy, Part 23 — Vendor Relationship & Renewal Management
Cross-team / access gap	"IT hadn't provisioned the access this role needed for six weeks"	This book, Part 26 — Cross-Team Politics & Stakeholder Alignment

Cross-Book Pointer A finding that traces back to a detection rule that should have caught something and didn't is a real, common outcome of a postmortem — and it is not this book's finding to fix. Detection Engineering Handbook V2, Part 43 — Detection Debt owns the technical framework for scoring and prioritizing exactly that gap. This part's job stops at correctly categorizing the finding and routing it there with enough context (the incident, the specific rule, the specific miss) that a detection engineer isn't starting from zero.

5.3 Verification, not a status field that says "closed"

[SENIOR MANAGER] A corrective action closed on the strength of the owner's own word that it's done is closed in the tracker and nowhere else. Verification means someone other than the owner checks the stated evidence against the stated method from §5.1 before the status changes — a second person confirming the access was actually granted and actually works, not a ticket number that shows a request was filed. This is the exact discipline Part 15's QA program applies to a ticket score: a self-report is cheap and mostly right, but the cases where it's wrong are exactly the ones a program most needs to catch, and a program that never independently checks will never find out which cases those were.

Operational Reality Most corrective-action trackers report a healthy closure rate and are, in practice, full of findings marked "closed" because the owner filed a request with another team and treated the filing itself as the fix. The tracker's own status field can't distinguish "the access was granted and verified working" from "a ticket exists somewhere in a different system that may or may not ever get actioned" unless verification is a separate, checked step — and if it isn't, the tracker's closure rate is measuring paperwork completion, not risk actually reduced.

People Risk Trap *Letting the same person who owns a corrective action also be the one who marks it verified concentrates exactly the same conflict Part 15 names for QA reviewers scoring their own reports — a busy owner under pressure to show progress has every incentive to call something done a little earlier than the evidence supports, not out of dishonesty but because "probably fine now" and "verified fine" feel like the same thing when you're the one who did the work. Route verification to someone other than the owner, even when that adds a short delay to closing the item — the delay is the cost of the check actually meaning something.*

5.4 Sizing the program: a worked cost example

CONCEPTUAL SAMPLE — illustrative program-sizing math for a 40-analyst SOC; not sourced benchmark data.

A 40-analyst in-house SOC runs, in a typical year, about 18 incidents that meet the mandatory top-tier severity trigger from §2.1, plus roughly 40 additional reviews pulled from the discretionary and random-sample triggers in §2.1–2.2 — 58 full reviews a year in total. Each review averages six attendees for 90 minutes (nine person-hours in the room) plus about three hours of facilitator time for scheduling, timeline assembly, and write-up — 12 person-hours per review before any corrective-action follow-up is counted. At 58 reviews, that's 696 person-hours a year just running the meetings.

Each review produces, on average, about three or four tracked findings; call it 3.4 across the mix of severities, or roughly 197 tracked findings a year across the whole program. Verified follow-up on a finding — an independent check against the stated evidence in §5.3, not just a status-field update — runs about one hour per finding on average, split across the owner's time and the verifier's time. That adds another 197 person-hours a year. Total program cost: roughly 893 person-hours a year, or about \$62,510 annually at a blended fully-loaded cost of \$70 an hour — split across a facilitator role (often the SOC manager or a rotating team lead), the standing review board's members, and whoever verifies closures, rather than one dedicated headcount line most SOC's this size would ever separately budget for.

[SENIOR MANAGER] That figure is worth saying out loud to whoever eventually asks why the review program needs protected time, because the alternative — the program running informally, on whatever time people find between other duties — doesn't make the cost disappear. It moves the cost into overtime, into reviews that get rushed or skipped when the queue is busy, or into the recurring-finding cost the next section quantifies, which is consistently larger than the cost of running the program properly in the first place.

6. Detecting patterns a single incident review can never see

6.1 Why the single-incident lens structurally can't do this

[CONCEPT] Ask the facilitator of any one incident's review whether this finding has happened before, and the honest answer is usually "I don't know" — not because the facilitator is careless, but because the question requires remembering or searching across every other review this year, which is exactly the

job nobody in a single-incident review room has time or mandate to do in the middle of running that meeting. SOC Playbook Handbook, Part 35 — Playbook Failure Examples has the same structural limit from the other direction: its catalog documents what broke in specific playbooks with real precision, but organizing by playbook, not by underlying cause, isn't built to answer "how many of the playbook failures documented there, across this SOC's own incidents, actually trace back to the same access gap." Pattern detection needs a different vantage point entirely — which is exactly what §3.2's standing board exists to provide.

6.2 The recurring-finding registry and its dedup logic

[SENIOR MANAGER] The mechanism that makes pattern detection possible rather than aspirational is a registry — the same tracker from §5, queried specifically for recurrence rather than for open-item status. Every time a new finding is logged, the review board's real work (not the per-incident facilitator's) is asking one question against the accumulated history: does this finding's plain-language gap statement (§5.1) match, in substance rather than exact wording, a finding already logged and supposedly closed within roughly the last 12 to 18 months? "The on-call engineer lacked read access to the EDR console" and "the responding analyst couldn't pull EDR telemetry without escalating for access" are different sentences describing the same gap, and a registry that only matches on exact text will miss that every time.

Figure 29.1 — Is this a new finding or a recurrence? Decision logic for the standing review board.

CONCEPTUAL. Illustrates the judgment sequence the board applies to a newly logged finding before deciding whether it represents fresh information or evidence that a previously "closed" corrective action didn't actually hold. It is a decision-support diagram for the board's own recurring review, not an automated matching algorithm. Diagram ID [FIG-2901](#).

flowchart TD

```
A["New finding logged\nfrom a review (§5.1)"] --> B{"Does the gap statement\nmatch a prior\nfinding\nin substance, not\nexact wording?"}
```

B -->|No close match| C["Log as a genuinely\nnew finding category"]

B -->|Yes -- matches a\nprior finding| D{"Was the prior\nfinding marked\nverified-closed\n(§5.3)?"}

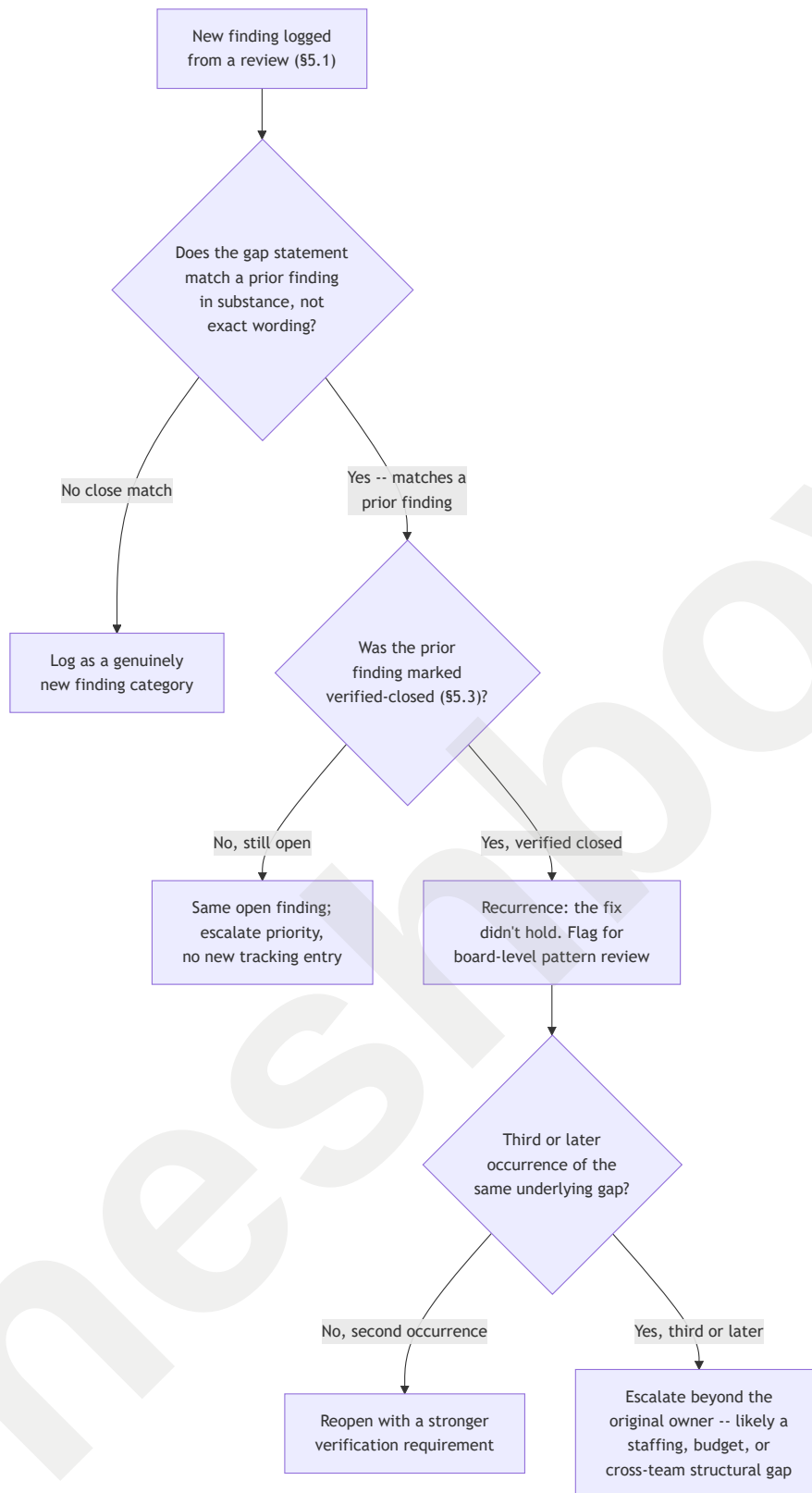
D -->|No, still open| E["Same open finding;\nescalate priority,\nno new tracking entry"]

D -->|Yes, verified closed| F["Recurrence: the fix\ndidn't hold. Flag for\nboard-level pattern\nreview"]

F --> G{"Third or later\noccurrence of the\nsame underlying gap?"}

G -->|No, second occurrence| H["Reopen with a stronger\nverification requirement"]

G -->|Yes, third or later| I["Escalate beyond the\noriginal owner -- likely a\nstaffing, budget,\nor\ncross-team structural gap"]



[SENIOR MANAGER]

The branch worth reading twice is the last one: a finding that recurs a third time after being verified closed twice is rarely still a single owner's problem to fix with a slightly better version of the same corrective action. Three verified-and-failed fixes at the same underlying gap is evidence the original owner didn't have the authority, budget, or standing to actually close it — which routes the finding up to whatever this book's Part 25 — Risk Acceptance & Manager Decision-Making

Under Uncertainty would treat as a resourcing or risk-acceptance call, not a fourth attempt at the same fix that already failed twice.

6.3 Management Autopsy — the finding that closed three times

Management Autopsy — "close the access finding and move on" (COMPOSITE CASE EXAMPLE, CASE-2902)

The decision: Across 14 months, a mid-sized MSSP SOC's postmortems produced the same underlying finding three separate times, worded differently each time: after a ransomware-precursor incident in month two ("on-call responder lacked EDR console access outside business hours"), after a credential-stuffing incident in month nine ("the analyst escalating overnight had to wait for a manager to grant temporary access before pulling telemetry"), and after a data-exfiltration near-miss in month 14 ("the on-call engineer's access request during the incident took 40 minutes to approve"). Each time, the corrective action — "provision standing off-hours EDR access for the on-call rotation" — was assigned, given a 30-day due date, and marked closed within that window.

Why it seemed reasonable: Each individual review had no visibility into the other two — different incident commanders, different months, no standing board yet cross-checking findings against each other. Each owner filed a real IT ticket requesting the access change, in good faith, and treated the filed ticket as sufficient evidence to close the postmortem's action item, which is exactly the failure §5.3's Operational Reality box describes.

How it failed: The underlying IT access-provisioning ticket, in a completely separate system from the postmortem tracker, sat unresolved all three times — deprioritized against IT's own competing work each time, with nobody in the postmortem tracker's chain positioned to see that the same request had already stalled twice before. No individual acted in bad faith at any point; the failure was structural — closing a postmortem action on "a ticket was filed" instead of "the access was verified working" (§5.3), with no registry (§6.2) checking whether this exact gap had already supposedly been fixed twice before.

The fix: The tracker adopted verified-closure as a hard requirement (§5.3), and the standing board began checking every new finding against the prior 18 months of history (§6.2) before a facilitator's proposed corrective action was accepted as adequate. The third occurrence, once correctly flagged as a repeat rather than logged as a fresh finding, got escalated past the original individual owner to a cross-team meeting between the SOC manager and IT leadership — the access provisioning finally happened as a standing, audited grant within two weeks of that escalation, after 14 months of the same gap surviving three separate "fixes."

6.4 What a good closure rate can still hide

What Would Change My Mind This part treats verified corrective-action closure and low recurrence as the two numbers that together indicate the program is actually working, with closure rate alone treated as insufficient (§5.3, [CASE-2902](#)). If a program with a consistently high closure rate and a consistently low measured recurrence rate still produced a steady stream of near-identical incidents that the tracker never once flagged as connected, that would undermine this part's confidence that the dedup logic in §6.2 is sufficient — it would suggest the taxonomy or the matching judgment itself is missing a category of connection between findings that plain-language matching can't catch, and the board's review method, not just its inputs, would need to change.

7. Program governance: the pipeline, the scorecard, and reporting it upward

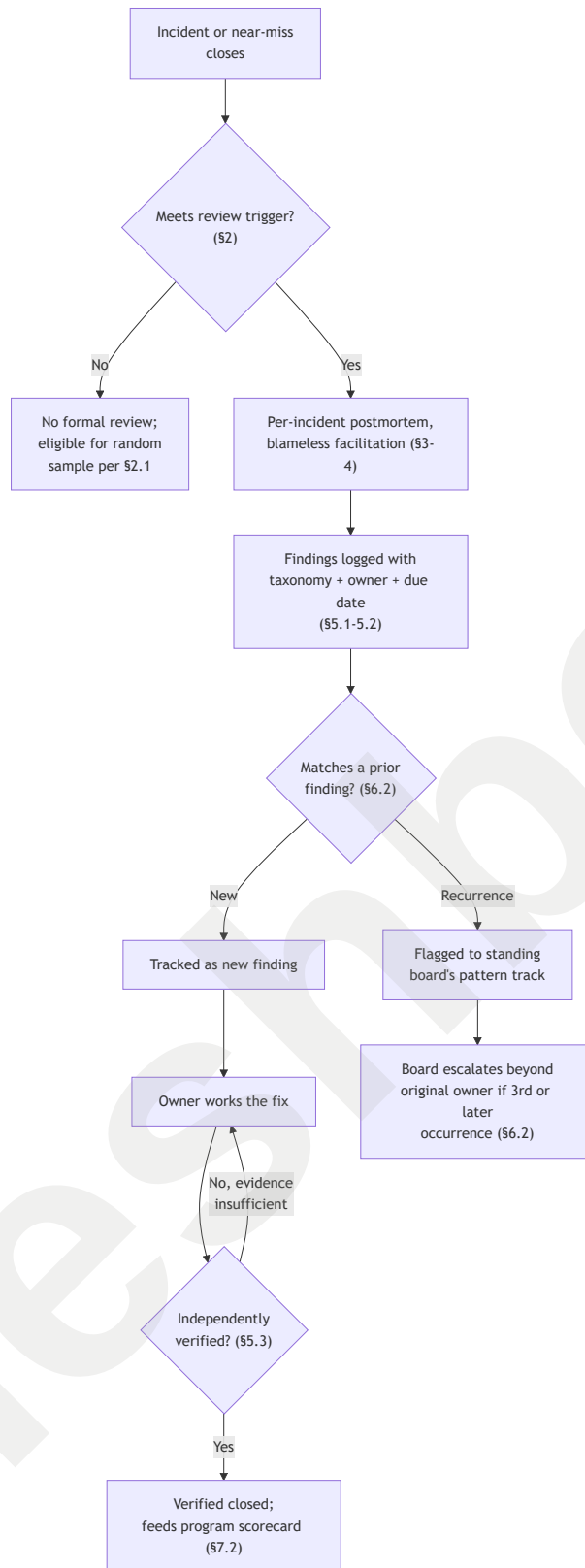
7.1 The pipeline, end to end

[CONCEPT] Figure 29.2 ties Sections 2 through 6 together into the single pipeline a finding travels from the moment an incident closes to the point it either verifies closed or surfaces as a pattern the standing board has to act on.

Figure 29.2 — The post-incident organizational review pipeline, incident close to verified closure or pattern escalation. *CONCEPTUAL.* Illustrates the sequence this part's sections build: trigger evaluation, per-incident review, taxonomy-tagged findings entering the tracker, verified closure, and the recurring-finding check that routes a repeat into the standing board's pattern-review track. It is a structural process diagram, not a capture of any specific ticketing tool's workflow. Diagram ID [FIG-2902](#).

flowchart TD

```
A["Incident or near-miss\ncloses"] --> B{"Meets review trigger?\n($2)"}
B -->|No| Z["No formal review;\neligible for random\nsample per §2.1"]
B -->|Yes| C["Per-incident postmortem,\nblameless facilitation (§3-4)"]
C --> D["Findings logged with\ntaxonomy + owner + due date\n($5.1-5.2)"]
D --> E{"Matches a prior\nfinding? (§6.2)"}
E -->|New| F["Tracked as new finding"]
E -->|Recurrence| G["Flagged to standing\nboard's pattern track"]
F --> H["Owner works the fix"]
H --> I{"Independently\nverified? (§5.3)"}
I -->|No, evidence\ninsufficient| H
I -->|Yes| J["Verified closed;\nfeeds program scorecard (§7.2)"]
G --> K["Board escalates beyond\noriginal owner if 3rd or later\noccurrence (§6.2)"]
```



7.2 A program-health scorecard

CONCEPTUAL SAMPLE — illustrative program-health figures for the same 40-analyst SOC modeled in §5.4; not sourced benchmark data.

Table 29.3 — Program-health scorecard, trailing 12 months. (CONCEPTUAL SAMPLE.) Use this table (or Appendix A7's fillable version) as a standing quarterly report to whoever the SOC manager answers to — it is the set of numbers that actually answers "is this program working," as distinct from "did the meetings happen."

Metric	Trailing-12-Month Figure	What It Actually Tells You
Trigger-eligible incidents reviewed within the 10-business-day target	19 of 22 (86%)	Whether the per-incident layer is running on schedule, not whether it's any good
Findings verified-closed within their stated due date	61%	Whether the tracker's closure discipline (§5.3) is holding, distinct from raw closure rate
Findings later flagged as a recurrence of a previously verified-closed finding	14% of closed findings	The number that most directly indicates whether fixes are actually holding — this book's central program metric
Near-miss flags logged per month (rolling average)	5	Whether the low-friction near-miss path (§2.2) is actually being used, not just documented
Findings escalated to a cross-team or budget decision after a 3rd-or-later occurrence	2	How often the program is catching a structural gap before a fourth incident, rather than after

[SENIOR MANAGER] The 14% recurrence figure is the one worth defending in a budget conversation over the 86% on-schedule figure, even though the schedule number looks more impressive on its own — a program that reviews everything on time but keeps re-closing the same three findings is running an expensive theater of diligence, and the recurrence number is what actually distinguishes that from a program that's making the SOC measurably harder to surprise twice the same way.

7.3 Field Test — proving the registry actually catches a repeat

Field Test Setup: A corrective-action tracker with at least 12 months of history and at least one previously verified-closed finding whose underlying gap is plausibly still present somewhere in the environment. **Action:** Deliberately draft a new, differently worded finding describing the same underlying gap as a prior verified-closed item — phrase it as if it came from a fresh, unrelated incident — and submit it into the standing board's next pattern-review cycle through the normal intake path, without flagging to the board in advance that it's a test. **Expected result:** The board's dedup check (§6.2) should catch the match and correctly identify it as a probable recurrence rather than logging it as a new, unrelated finding. If it doesn't — if the differently worded version sails through as brand new — the matching judgment is relying on exact wording rather than substance, and the registry is not actually doing the job this section claims it does.

7.4 Reporting the program upward

[EXECUTIVE] A board or CISO conversation about incident-response maturity usually gravitates toward response time and containment speed, both genuinely important and both owned by SOC Playbook Handbook's metrics and this book's own Part 24 — Executive & Board Reporting for how to frame them

upward. The recurrence rate from §7.2 belongs in the same conversation for a different reason: it's the single number that answers "does this organization actually learn from an incident, or does it just recover from one," and a board asking whether a repeat of last year's breach is more or less likely this year is really asking exactly that question, whether or not they'd phrase it that way.

8. Where this goes next

[CONCEPT] This part built the standing program as four connected mechanics: trigger criteria that decide which incidents and near-misses earn a review without drowning the program in overhead; a two-layer cadence — the per-incident postmortem and the standing board that watches across many of them — with ownership split cleanly between the two; a corrective-action tracker that treats a self-reported "closed" as worthless without independent verification; and a recurring-finding registry that gives the organization, for the first time, a way to notice its own third mistake before a fourth incident forces the question. Appendix A7 carries the fillable post-incident organizational review template and corrective-action tracker (TMPL-2901, TMPL-2902) built on the structure in §3 and §5. Nothing past this point should need to re-derive what a tracked, verified finding is for — it should assume the registry exists and query it.

Cross-references

Within this book: Assumes Part 19 — Team Culture & Psychological Safety for the blameless precondition this program's facilitation mechanics (§4) depend on rather than rebuild, and Part 28 — The Manager's Role in a Major Incident, whose close is where this part's trigger evaluation (§2) picks up. Cross-references Part 15 — Quality Assurance Programs for the calibration and independent-verification discipline this part's tracker (§5.3) reuses; Part 5 — Headcount & Capacity Modeling, Part 6 — Shift Pattern & Coverage Design, Part 10 — Competency Models & Skills Matrices, Part 12 — Ongoing Training & Skill Development, Part 21 — Tooling Procurement & Platform Strategy, Part 23 — Vendor Relationship & Renewal Management, and Part 26 — Cross-Team Politics & Stakeholder Alignment as the destination owners named in the finding taxonomy (Table 29.2); Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty for the resourcing call a third-or-later recurrence escalates into (§6.2); and Part 24 — Executive & Board Reporting for framing the program's recurrence-rate metric upward (§7.4). Points forward to Appendix A7 — Executive, Board & Risk-Acceptance Templates, which carries the fillable review and tracker templates this part's Section 3 and Section 5 build toward.

Other volumes: SOC Playbook Handbook, Part 35 — Playbook Failure Examples, for the individual-breakdown, single-playbook lens this part deliberately does not re-walk. SOC Playbook Handbook, Part 29 — Playbook Severity Model, for the severity tiers this part's trigger criteria (§2.1) consume as an already-settled input. Detection Engineering Handbook V2, Part 43 — Detection Debt, for the technical framework a detection-debt-category finding (Table 29.2) routes to rather than being fixed inside this program.

Section H — Maturity, Benchmarking & the Road Ahead

neshbooy

Part 30 — SOC Maturity Models

Why this part exists

[CONCEPT] A board member, a new CISO, or an auditor eventually asks some version of the same question: how mature is this SOC? The honest answer depends entirely on which of three completely different things the question is actually about, and a manager who answers with a single number has almost certainly picked the wrong one without realizing it. This part builds the maturity model this book owns outright: an organizational-design maturity model scored across four dimensions — staffing ratios, process maturity, training-program maturity, and tooling maturity — that describes whether the SOC has the people, the discipline, and the stack to execute reliably. It says nothing about whether what the SOC executes is any good, which is the entire reason two other maturity models exist in this series and are not re-derived here.

[CONCEPT] SOC Playbook Handbook's playbook-maturity model, built out in Appendix 38A, scores individual playbooks for structural quality — completeness, clarity, whether the steps actually produce the hand-off SOC Playbook Handbook, Part 27 — Escalation Quality requires. Detection Engineering Handbook V2's coverage and quality parts — Part 41 — Detection Coverage and Part 42 — Detection Quality — score whether a validated, tested detection exists for a given technique in the organization's threat profile, and whether that detection performs well once it exists. Neither model is re-explained in the pages that follow; both are cited, by name and part number, exactly where a manager needs to bring their output into this part's own model. This part's job is the fourth lens the other two don't cover — the organization running the playbooks and the detections — and, in §4, a synthesis table for reading all three lenses at once without letting a strong score on one stand in for the other two.

1. Why three maturity models, and why they don't collapse into one

[CONCEPT] A maturity model earns its keep by telling a manager where to invest next. That only works if the model's unit of analysis matches the thing the investment decision is actually about — headcount and process money buys a different outcome than detection-engineering money, which buys a different outcome again than a playbook-rewrite sprint. Collapse three models with three different units of analysis into one composite score and the number that survives the collapse can rise for reasons that have nothing to do with the risk a board actually cares about — a strong quarter of hiring and onboarding can lift a blended score even while the organization's actual detection coverage sits exactly where it did the quarter before. §6 works through a real version of that failure in detail.

[SENIOR MANAGER] The three models also have three different natural owners inside most organizations, which is a second, more practical reason not to merge them. A detection engineer or detection-engineering lead owns the inputs to DEH V2's coverage and quality scoring — telemetry availability, rule logic, test results. A senior analyst or team lead who writes and reviews playbooks owns the inputs to SOC Playbook Appendix 38A. The SOC manager — the reader of this book — owns the inputs to this part's model: the headcount number, the QA program, the training budget, the tooling stack. A single blended score has no natural owner, because no single role controls all the inputs that

move it, and a metric nobody owns tends to drift unmaintained until the day it's needed for something important.

Cross-Book Pointer *This part does not score playbook quality — whether a specific playbook's steps are complete, clear, and produce a hand-off that meets the standard SOC Playbook Handbook, Part 27 — Escalation Quality sets. That's SOC Playbook Handbook, Appendix 38A — the Playbook Maturity Model, which this part treats as a finished lens with its own output, not as raw material to re-derive. Go there for the mechanics of scoring an individual playbook; come back here once you have that score and want to read it alongside the organizational picture built in §2–§4.*

Cross-Book Pointer *This part does not score whether a validated detection exists for a given technique, or whether that detection performs well in production — that's Detection Engineering Handbook V2, Part 41 — Detection Coverage and Part 42 — Detection Quality, which build the technique-by-technique tiered coverage model and the precision/recall measures behind it. This part takes a coverage or quality figure from those parts as a finished input, the same way it takes a headcount ratio as its own output — neither model owes the other a rebuild.*

[CONCEPT] None of this means the three models are unrelated, only that they measure different things and need to stay legible on their own before anyone reads them together. §4 is where they actually get read together, on purpose, with a synthesis table built for exactly that job.

2. The four dimensions of organizational-design maturity

[CONCEPT] Each of the four dimensions below already has its own full treatment earlier in this book — this section's job is not to re-teach headcount modeling, QA program design, training strategy, or tooling procurement, but to define what "more mature" concretely looks like along each one, in a form that can be scored. Table 30.1 lays out five levels across all four dimensions; the subsections that follow explain the reasoning behind a handful of the more consequential jumps between levels.

2.1 Staffing-ratio maturity

[SENIOR MANAGER] Staffing-ratio maturity asks whether headcount and the ratios built on top of it — analyst-to-shift-lead, analyst-to-QA-reviewer, concurrent seats to volume — are set by a defensible, maintained model or by whatever the org chart happened to inherit. Part 5 — Headcount & Capacity Modeling builds the actual arithmetic (segmented volume and handle time, the shrinkage stack, coverage-hours reconciliation); this dimension asks whether that arithmetic, or something like it, is actually being run, on a cadence, against real data, rather than defended once at budget time and never revisited.

[SENIOR MANAGER] The jump from a merely documented ratio to a mature one is the same jump Part 5 §7 makes between a mean-based model and one stress-tested against a high-percentile volume day. A SOC that can state "we run a 1:8 analyst-to-lead ratio" has cleared a real bar over having no stated ratio at all — but a SOC that can additionally show that ratio held during a documented volume surge, with a named overflow lever that was actually exercised rather than only written down, has cleared a materially higher one. Table 30.1's Level 4 and Level 5 rows draw that line explicitly.

2.2 Process maturity

[SENIOR MANAGER] Process maturity asks whether the SOC's internal quality mechanics — QA sampling, calibration, onboarding structure, escalation criteria — exist as a documented, running program or as tribal knowledge held by whichever senior analyst has been there longest. Part 15 — Quality Assurance Programs is the fullest expression of one process-maturity input: a sampling methodology sized to a real reviewer-hour budget, a calibration practice with a measured inter-rater agreement threshold, and a score-to-coaching pipeline that reaches the analyst rather than sitting in a spreadsheet.

[SENIOR MANAGER] The distinction that matters most here is between a process that exists on paper and one that's been checked to actually run the way it's described. Part 15 §5.4's Field Test — asking analysts directly whether a low score actually produced a coaching conversation within about two weeks — is the concrete version of the question this dimension is really asking: not "is there a QA policy document," but "does the policy document describe what's actually happening." A SOC that has the document but has never run that check is, for maturity-scoring purposes, in a materially different place than one that has run it and passed.

2.3 Training-program maturity

[HR/PEOPLE] Training-program maturity asks whether onboarding, ongoing skill development, and career progression form a connected system or a set of disconnected activities — a new-hire packet here, a conference-attendance budget line there, a promotion decided informally when someone finally asks. Part 11 — Onboarding & Ramp-Up Programs, Part 12 — Ongoing Training & Skill Development, and Part 13 — Career Ladders & Promotion Criteria each own one stage of this dimension; Part 10 — Competency Models & Skills Matrices supplies the shared vocabulary — "ready for L2," "ready for promotion" defined in observable terms — that a mature training program uses consistently across all three stages rather than reinventing at each one.

[HR/PEOPLE] The clearest, cheapest signal of where a SOC sits on this dimension is whether ramp-to-independent-triage time is tracked at all, and whether it's trending in a direction anyone can point to. A SOC that can say "new analysts reach independent triage in roughly 90 days, down from around 130 two years ago, after we restructured onboarding around Part 11's shadowing sequence" is describing a managed system. A SOC that can only say "it takes a few months, depends on the person" has a training activity, not a training program, regardless of how much money is being spent on it.

2.4 Tooling maturity

[VENDOR/PROCUREMENT] Tooling maturity asks whether the SIEM/EDR/SOAR/TIP stack — and everything layered around it — reflects a rationalized set of deliberate procurement decisions with tracked total cost of ownership, or a history of whoever won the last vendor bake-off, with nothing formally retired since. Part 21 — Tooling Procurement & Platform Strategy owns the selection mechanics (PoC criteria, reference calls, TCO modeling that includes migration and retraining cost); Part 23 — Vendor Relationship & Renewal Management owns what happens after signing, including the tool-sprawl problem this dimension is most often failing on.

[VENDOR/PROCUREMENT] The sharpest test of tooling maturity is not how good the current stack looks in a vendor demo — it's whether the organization has ever actually retired a tool. Three case-management platforms doing overlapping work because none was ever formally sunset is a common, specific symptom, not a hypothetical one, and it's cheap to check for directly: ask how many licensed security tools are currently active, how many of them a new hire would need explained to them in onboarding, and how many have had zero material usage growth in the past year. A SOC that has never sunset a tool in its operating history, regardless of how modern its current top-line platform is, has not yet reached the higher levels of this dimension.

Table 30.1 — Organizational-design maturity, five levels across four dimensions. (*CONCEPTUAL SAMPLE — illustrative level definitions built from this book's own models in Parts 5, 10–15, and 21; not a sourced external framework.*) Use this table to place a SOC on each dimension independently before doing anything else with the result — §3 shows why scoring each row separately, rather than jumping straight to an average, is the entire point.

Level	Staffing-ratio maturity	Process maturity	Training-program maturity	Tooling maturity
1 — Ad hoc	Headcount set by budget headroom; no stated ratio; hiring purely reactive to resignations	No documented QA sampling; escalation criteria live in one person's head	No training budget line; certifications reimbursed case by case	Tool chosen by whoever had signing authority; overlapping tools never retired
2 — Developing	A headcount number exists from a single flat annual average, set once and unrevisited	A QA checklist exists; one reviewer uses it; no calibration ever run	A fixed onboarding checklist exists; ongoing training is opportunistic	A documented tool list exists; no TCO tracked; PoC criteria reinvented each purchase
3 — Defined	Headcount segmented by shift/time block (Part 5 §5–§6); a stated ratio target (e.g., 1:8) exists	A sampling methodology (Part 15 §2) and written rubric exist; calibration happens irregularly	A structured 90-day onboarding plan; a per-analyst training budget; a competency matrix (Part 10) exists but isn't consistently used	A rationalized core stack with documented ownership; renewal dates tracked centrally
4 — Managed	Headcount rebuilt on a fixed cadence against real volume and shrinkage (Part 5 §8); variance from the ratio target is explained to the budget owner	Calibration runs on a fixed cadence with a measured inter-rater agreement threshold (Part 15 §3.3); the QA-to-coaching pipeline has passed a Field Test	Career ladder with observable promotion criteria (Part 13); ramp-to-productivity time is tracked and trending down	TCO tracked per tool, including migration/retraining cost (Part 21); a documented vendor risk-scoring process exists
5 — Optimizing	Model includes a tested surge/overflow lever (Part 5 §7.2, Part 22); backfill lag (Part 18) is modeled into approved headcount	QA findings demonstrably change behavior (Part 15 §5.4 passes repeatedly); the process-maturity score itself is re-audited annually	Mentorship/succession plan removes single-person dependency (Part 14); competency data drives hiring-bar and headcount decisions before a gap appears	At least one redundant tool has actually been retired in the past cycle, not just flagged for retirement; renewal negotiation is informed by measured utilization

3. Scoring a SOC: a worked walkthrough

[SENIOR MANAGER] **CASE-3001** below is a composite, illustrative organization built to show a specific, common outcome: once a SOC is scored honestly across all four dimensions, the four scores are almost never the same, and the gap between them is the most useful piece of information the exercise produces.

COMPOSITE CASE EXAMPLE (**CASE-3001**) — illustrative organization and figures, constructed to demonstrate uneven dimension scoring; not drawn from a single traceable real SOC.

The environment: A 35-analyst regional MSSP SOC, three years past its initial buildout, scored against Table 30.1.

Staffing-ratio maturity — Level 3. Headcount is segmented by shift and rebuilt roughly once a year against the prior year's volume, with a documented 1:9 analyst-to-lead ratio. The model has never been stress-tested against a P90 volume day, and there is no named overflow lever — the ratio is real and defensible, but nobody has checked what happens to it on a bad week.

Process maturity — Level 2. A QA rubric exists and one senior team lead reviews a sample of tickets against it. No second reviewer has ever scored the same batch, and no calibration session has been run in the program's three-year history — the reviewer's judgment has never been checked against anyone else's.

Training-program maturity — Level 4. A structured onboarding sequence, a competency matrix tied to promotion criteria, and a tracked ramp curve are all in place; new analysts reach independent triage in roughly 97 days, down from about 130 days two years earlier, after onboarding was restructured around graduated shadowing.

Tooling maturity — Level 2. Three separate case-management platforms are active in production: a legacy tool from the SOC's original buildout, a second inherited when the MSSP acquired a competitor's client base, and the current nominal standard. No formal TCO comparison or consolidation plan exists for any of the three.

Table 30.2 — CASE-3001 scorecard. (COMPOSITE CASE EXAMPLE.) Use a scorecard like this one to record the evidence behind each dimension's score, not just the level number — the evidence is what a skeptical budget owner or board member will actually ask about.

Dimension	Level	Evidence	Gap to next level
Staffing-ratio maturity	3 — Defined	1:9 ratio documented; segmented headcount rebuilt annually	Rebuild on a fixed quarterly cadence; add a tested overflow lever
Process maturity	2 — Developing	Rubric exists; single reviewer, no calibration ever run	Add a second reviewer and run a calibration session per Part 15 §3.2
Training-program maturity	4 — Managed	Tracked ramp curve (130 to 97 days); competency-linked career ladder	Add a mentorship/succession layer per Part 14 to reach Level 5
Tooling maturity	2 — Developing	Three overlapping case-management platforms; no TCO tracked	Run a consolidation TCO analysis per Part 21; retire at least one platform

[SENIOR MANAGER] A manager who averages these four scores — $(3 + 2 + 4 + 2) \div 4 = 2.75$ — gets a number that rounds to "somewhere between Developing and Defined" and tells the budget owner almost nothing useful. The organization is not uniformly mediocre; it has a genuinely strong training program sitting next to a process-maturity gap serious enough that no QA score in this SOC's three-year history has ever been checked against a second reviewer's judgment. The averaged number obscures exactly the fact that matters most for deciding what to fund next quarter: process maturity, not training, is the dimension carrying the most unmanaged risk, and it's also the cheapest of the four to fix — a calibration session (Part 15 §3.2) costs a handful of reviewer-hours, not a hiring cycle or a platform migration. Report the four scores separately, every time; an average is a worse version of the same information this part exists to surface clearly.

What Would Change My Mind This part treats staffing ratios, process maturity, training-program maturity, and tooling maturity as the four dimensions that make up organizational-design maturity, on the claim that they're the right set to score independently. If a SOC scored at Level 4 or 5 on all four and still failed to execute reliably during a real major incident, for a reason not explained by any weakness in those four dimensions, that would mean a real fifth dimension is missing from this model — and it would need to be named and added on its own terms, not folded into whichever of the existing four seems closest.

4. Reading three lenses together, not one number

[SENIOR MANAGER] Table 30.1 and **CASE-3001** cover this book's own lens in isolation. A manager rarely gets to stop there — a board asks about overall SOC maturity, not just the organizational slice of it, and the honest answer requires bringing in the other two models this part deliberately doesn't re-derive. Table 30.3 lays out what each of the three lenses actually measures, what a high score on it does and doesn't tell a manager, and the specific conflation trap each one invites.

Table 30.3 — Three maturity lenses, read side by side. (CONCEPTUAL SAMPLE — synthesis built from this book's own model and the stated scope of the other two volumes' models; not a re-derivation of either.) Use this table before presenting any maturity figure to a board or CISO — it's the check for whether the figure being presented answers the question that was actually asked.

Lens	Owning model	Unit of analysis	A high score tells you	A high score does <i>not</i> tell you	Common conflation trap
Organizational-design maturity	This book, Part 30	The program: staffing, process, training, tooling	The SOC has the people, discipline, and stack to execute reliably at volume	Whether what it executes — the playbooks, the detections — is any good	Assuming a well-run team is automatically well-defended
Playbook-quality maturity	SOC Playbook Handbook, Appendix 38A	One playbook, scored for structural completeness and clarity	The response guidance for a given scenario is well-written, complete, and produces a proper hand-off	Whether staff are trained and resourced to run it under pressure, or whether the alert that triggers it exists at all	Mistaking well-documented playbooks for a rehearsed, resourced response capability
Detection coverage/quality maturity	Detection Engineering Handbook V2, Parts 41–42	One ATT&CK technique or analytic, scored for telemetry existence and validated detection tier	The org can technically see and reliably flag the specific threats in scope	Whether anyone is staffed and trained to respond once it fires, or whether the response guidance for that alert is any good	Assuming strong detection automatically means a strong overall security program

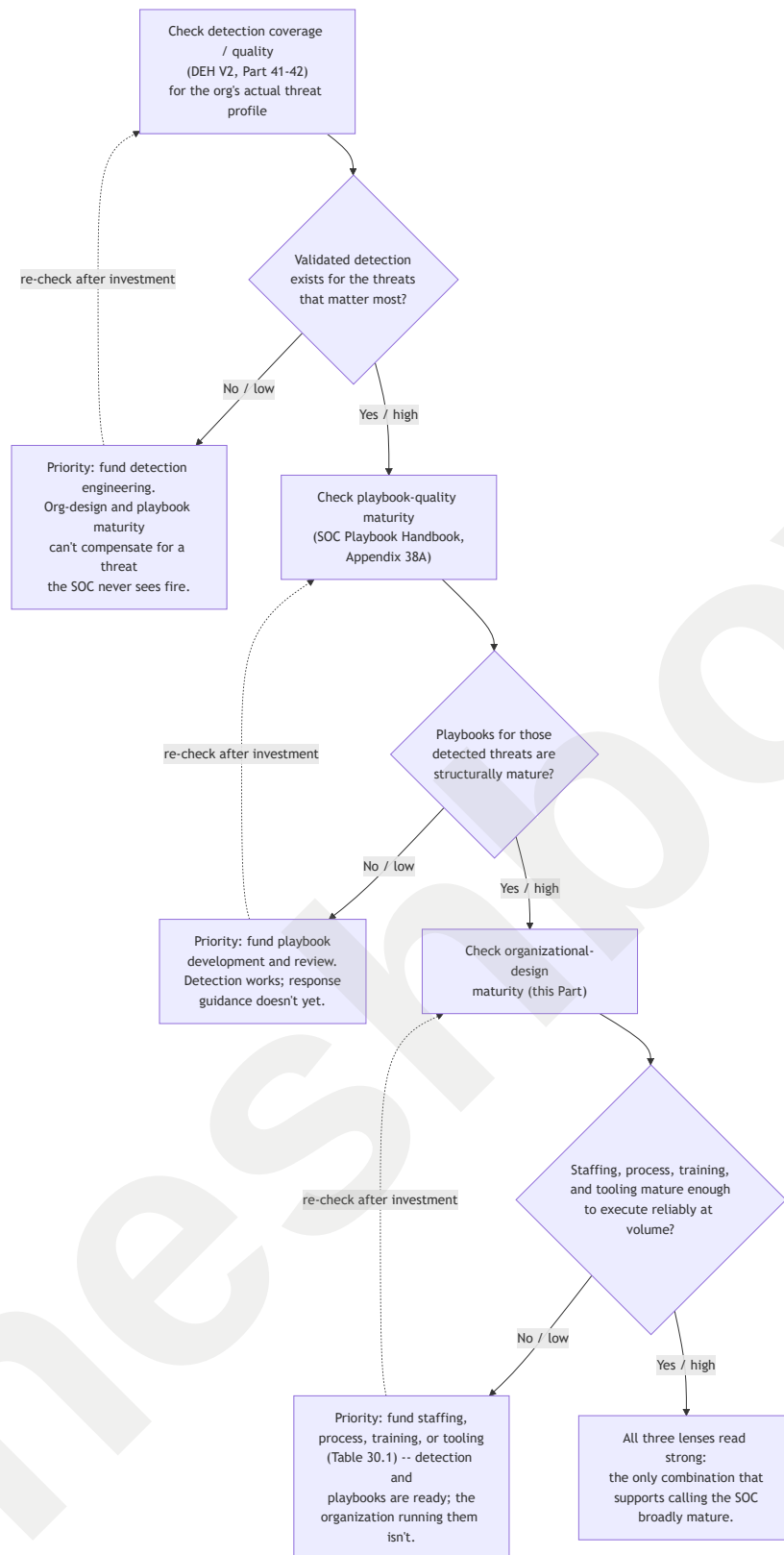
[SENIOR MANAGER] The three lenses have a natural reading order, not because one matters more than the others in the abstract, but because each one is a precondition for the next one mattering at all. A perfectly staffed, perfectly trained team running a beautifully written playbook against a technique with

no working detection never gets the call — organizational maturity and playbook maturity are both irrelevant to an attack the SOC never sees. A validated detection paired with no usable playbook produces an alert nobody knows how to act on correctly. Only once both of those hold does organizational-design maturity — the lens this part owns — determine whether the response actually happens reliably, at the volume and speed a real incident demands, rather than by heroics from whichever analyst happens to be sharpest that day.

Figure 30.1 — Reading the three maturity lenses in sequence. *CONCEPTUAL.* Illustrates a defensible default order for bringing the three models together when deciding what to fund next — detection coverage first, playbook quality second, organizational design third — rather than a claim that any one lens is intrinsically more important. It is a decision-sequencing diagram, not a scoring method for any of the three models it references. Diagram ID [FIG-3001](#).

flowchart TD

```
A["Check detection coverage / quality\n(DEH V2, Part 41-42)\nfor the org's actual threat profile"]
--> B{"Validated detection\nexists for the threats\nthat matter most?"}
B -->|"No / low"| C["Priority: fund detection engineering.\nOrg-design and playbook\nmaturity\ncan't compensate for a threat\nthe SOC never sees fire."]
B -->|"Yes / high"| D["Check playbook-quality maturity\n(SOC Playbook Handbook,\nAppendix 38A)"]
D --> E{"Playbooks for those\ndetected threats are\nstructurally mature?"}
E -->|"No / low"| F["Priority: fund playbook\ndevelopment and review.\nDetection works;\nresponse\nguidance doesn't yet."]
E -->|"Yes / high"| G["Check organizational-design\nmaturity (this Part)"]
G --> H{"Staffing, process, training,\nand tooling mature enough\nto execute reliably at volume?"}
H -->|"No / low"| I["Priority: fund staffing,\nprocess, training, or tooling\n(Table 30.1) --\ndetection and\nplaybooks are ready; the\norganization running them isn't."]
H -->|"Yes / high"| J["All three lenses read strong:\nthe only combination that\nsupports calling\nthe SOC\nbroadly mature."]
C -. ->|"re-check after investment"| A
F -. ->|"re-check after investment"| D
I -. ->|"re-check after investment"| G
```



[SENIOR MANAGER]

The sequence in Figure 30.1 is a default, not a rule that overrides judgment. §7 comes back to what happens when two or three lenses are simultaneously weak and only one investment can be funded this cycle — that's a resourcing tradeoff, and this book's own Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty owns the judgment call, not this part.

What Would Change My Mind This part defaults to reading the three lenses in the order detection coverage, then playbook quality, then organizational design, on the claim that each is a precondition for the next one mattering. If a well-documented case existed where a coverage gap was actually caused by an organizational-design failure — understaffed detection engineering, not a technical blind spot — and fixing the organizational problem directly closed the coverage gap faster or more cheaply than a detection-engineering-first investment would have, that would argue the sequencing in Figure 30.1 isn't universal, and the honest default should be "diagnose the root cause across all three lenses before assuming detection always goes first," not a fixed order.

5. Worked case: when the three lenses disagree

[SENIOR MANAGER] CASE-3002 below shows the three-lens synthesis doing its actual job — catching a combination of scores that looks fine on any single lens and is not fine once read together.

COMPOSITE CASE EXAMPLE (CASE-3002) — illustrative organization, figures, and incident narrative, constructed to demonstrate the risk of reading one maturity lens in isolation; not drawn from a single traceable real SOC or incident.

The environment: A 60-analyst enterprise in-house SOC, scored across all three lenses ahead of an annual security-committee review.

Organizational-design maturity (this part's model): Level 4 — Managed. Headcount is rebuilt quarterly against actual volume and shrinkage, holding a 1:7 analyst-to-lead ratio within a stated band. QA calibration runs quarterly with measured inter-rater agreement at 88%. A career ladder with observable promotion criteria is in place, and the tooling team retired one of two overlapping SOAR platforms in the past year after a documented TCO comparison.

Playbook-quality maturity (SOC Playbook Handbook, Appendix 38A): reported strong. Ninety percent of the SOC's Ransomware and Insider Threat playbook categories score at Appendix 38A's top quality tier, per the SOC Playbook Handbook's own scoring output — a genuinely strong result on that model's own terms, taken here as a finished input rather than re-checked.

Detection coverage/quality maturity (Detection Engineering Handbook V2, Parts 41–42): weak. Only 35% of the ATT&CK techniques in the organization's own documented threat profile sit at DEH V2's validated-detection tier; the remaining majority sit at a lower tier — telemetry exists, but no tested, tuned analytic has been built and confirmed against it.

[SENIOR MANAGER] Read on its own, the organizational-design score supports a genuinely positive report: a Level 4 program, staffed, trained, and processed to a real standard. Read alongside a strong playbook-quality figure, the combination looks even better — a well-run team with well-written response guidance. Neither fact changes what the detection-coverage figure is actually saying: for roughly two-thirds of the techniques this organization has told itself to worry about, no validated detection exists to trigger either the mature team or the mature playbook in the first place.

What happened next: A ransomware intrusion in this composite case used a lateral-movement technique that sat at DEH V2's lower coverage tier — telemetry existed, but the analytic had never been built and tested. No alert fired for that stage of the intrusion. The organization only engaged its Ransomware playbook once a later, unrelated detection tripped much further into the intrusion than the coverage model, had it been funded, would have allowed. The post-incident review — run per this book's own Part 29 — Post-Incident Organizational Review — found that once the playbook was engaged, the team executed it well: staffing surge worked as designed, the playbook's steps were followed correctly, and Appendix 38A's own quality score for that playbook held up under real use. The organizational-design and playbook-quality maturity this SOC had actually earned were both real, and neither one was the reason the intrusion ran as long as it did.

Blind Spot *Organizational-design maturity, scored the way this part scores it, has no visibility into whether the detections and playbooks it's executing are any good. A Level 5 staffing, process, training, and tooling program run against a detection portfolio that hasn't been meaningfully touched in years scores exactly as high on this model as the identical program run against a well-maintained one, because none of this part's four dimensions ever examine rule logic or playbook content — that's what the other two lenses are for, and CASE-3002 is what it costs to forget that.*

[EXECUTIVE] The report a manager should have given the security committee in CASE-3002, before the incident, is not "we are a mature SOC" — it's "our organization and our response playbooks are both strong; our detection coverage is not, and it's the lens that determines whether either of the other two ever gets used against a real intrusion." That framing is the one Part 24 — Executive & Board Reporting is built to help a manager deliver without either overstating confidence or burying the one number that actually mattered.

6. What happens when the three scores get averaged into one

[SENIOR MANAGER] CASE-3002 shows what it costs to read one lens without the others. The Management Autopsy below (CASE-3003) shows a more specific, more common version of the same mistake: reading all three, and then compressing them into a single figure anyway.

Management Autopsy — "one SOC Maturity number for the board" (COMPOSITE CASE EXAMPLE, CASE-3003)

The decision: Ahead of an annual security-committee review, a SOC manager combined the organizational-design maturity level, the playbook-quality score, and the detection-coverage percentage into a single blended figure — converted onto a common 100-point scale and averaged — presented as one number: "SOC Maturity: 78/100."

Why it seemed reasonable: The committee had explicitly asked for "one number we can track year over year" after a previous review ran long trying to reconcile three separately framed metrics in the same meeting. A single trend line looked easier to defend in a 20-minute agenda slot than three charts each needing its own context, and the manager reasoned that a rising number would still communicate "things are improving" even if the underlying mix shifted.

How it failed: The committee watched the blended score rise from 78 to 81 to 84 across three quarters and concluded that overall risk was decreasing. Underneath that trend, the detection-coverage figure — contributing roughly a third of the composite — had not moved at all across two of those three quarters; the entire visible increase came from organizational-design gains (a new analyst cohort completing onboarding, a documentation backlog clearing) and a modest playbook-quality improvement. When a ransomware intrusion later exploited a technique that had sat at the same low coverage tier the whole time, the committee's first question in the post-incident session was how a maturity score that had been rising for three straight quarters hadn't warned anyone. The honest answer was that the averaging math had buried a flat, high-risk-relevance number inside two rising, lower-risk-relevance ones, and nobody reading the single trend line had any way to see that from the number alone.

The fix: The three lenses are now reported as three separate trend lines on the same slide, each retaining its own unit and its own owner, never averaged into one figure. Where the committee wants a single glance across all three, the report uses a stoplight per lens — consistent with this book's own Part 24 board-reporting format — rather than a blended score a reader can't unpack. A rising organizational-design or playbook-quality trend next to a flat or declining detection-coverage trend is now visible on sight, instead of hidden inside arithmetic that treats three different kinds of risk as fungible with each other.

[SENIOR MANAGER] The lesson generalizes past this one case: any pressure to produce "one number" across the three models in this series should be met with a stoplight or small-multiples display, never an average. The three lenses are not different measurements of the same underlying quantity — they're measurements of three different things that all happen to matter to the same eventual outcome, and averaging quantities that aren't measuring the same thing produces a number with no defensible interpretation, however clean it looks on a slide.

7. Using maturity to sequence investment

[SENIOR MANAGER] The practical payoff of scoring all three lenses honestly, rather than reaching for a blended figure, is a defensible answer to the only question a maturity model actually needs to answer: what should get funded next. Figure 30.1's default sequence — detection coverage, then playbook quality, then organizational design — gives a starting order when a single investment has to be chosen and nothing else distinguishes the options. Table 30.1's per-dimension breakdown gives the same kind of answer inside this part's own lens once detection and playbook maturity are already accounted for: CASE-3001's process-maturity gap, fixable with a handful of reviewer-hours, is a materially cheaper and faster investment than CASE-3001's tooling-consolidation gap, which needs a TCO study and a migration plan before it moves at all.

[SENIOR MANAGER] Neither the sequencing default nor the per-dimension breakdown replaces judgment when budget is fixed and more than one lens is weak at the same time — that tradeoff, and how a manager defends it when the "wrong" lens gets funded first for a legitimate reason, is this book's own Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty, not a rule this part can encode. What this part hands forward is the honest input to that decision: four separately scored dimensions, read alongside two other models' outputs rather than blended with them, so whatever gets funded first is a choice made with the real picture in front of it. Part 33 — The Next Twelve Months: Building a SOC Roadmap picks the resulting priorities up and sequences them into an actual roadmap; this part's job ends at producing the maturity picture that roadmap gets built from.

8. Where this goes next

[CONCEPT] This part built one of three maturity lenses in the series — staffing ratios, process maturity, training-program maturity, and tooling maturity, scored independently rather than averaged — and a synthesis table and decision sequence for reading it alongside SOC Playbook Handbook's playbook-maturity model and Detection Engineering Handbook V2's detection-coverage and quality maturity without letting a strong score on any one stand in for the others. Nothing past this point in either this book or the other two volumes should need to re-derive what "SOC maturity" means across all three lenses at once — it should cite this part's synthesis and move directly to the funding decision.

Cross-references

Within this book: Assumes Part 5 — Headcount & Capacity Modeling for the staffing-ratio dimension's underlying arithmetic, Part 10 — Competency Models & Skills Matrices for the shared "ready for L2" vocabulary the training-program dimension reuses, Part 11 — Onboarding & Ramp-Up Programs for the ramp-structure stage that dimension scores, Part 12 — Ongoing Training & Skill Development for what a managed training program actually funds, Part 13 — Career Ladders & Promotion Criteria for the promotion-criteria stage of the same dimension, Part 14 — Mentorship & Knowledge Transfer for the succession-planning input Table 30.1 scores at Level 5, Part 15 — Quality Assurance Programs for the process-maturity dimension's sampling and calibration mechanics, and Part 21 — Tooling Procurement & Platform Strategy for the tooling-maturity dimension's TCO discipline. Points forward to Part 18 — Attrition & Retention (backfill lag as a Level 5 staffing input), Part 22 — MSSP & Managed-Service Contract Management and Part 23 — Vendor Relationship & Renewal Management (surge levers and tool-sprawl rationalization), Part 24 — Executive & Board Reporting (the stoplight format this

part's §6 recommends over a blended score), Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty (the funding tradeoff when multiple lenses are weak at once), Part 29 — Post-Incident Organizational Review (where CASE-3002's incident review happens), and Part 33 — The Next Twelve Months: Building a SOC Roadmap (sequencing this part's output into a funded plan).

Other volumes: SOC Playbook Handbook, Appendix 38A — Playbook Maturity Model, for the playbook-quality lens this part cites in §1 and §4–§6 rather than re-scores. SOC Playbook Handbook, Part 27 — Escalation Quality, for the hand-off standard Appendix 38A's own scoring checks playbooks against. SOC Playbook Handbook, Part 32 — Metrics, for the metric definitions this book's own Part 5 and Part 24 already treat as fixed inputs. Detection Engineering Handbook V2, Part 41 — Detection Coverage and Part 42 — Detection Quality, for the detection lens this part cites in §1 and §4–§6 rather than re-derives.

Part 31 — Benchmarking & Industry Comparison

Why this part exists

[CONCEPT] A SOC manager reaches for an outside number for one of three reasons: a board member just asked "is that normal," a budget request needs something more persuasive than "trust me," or a vendor's sales deck put a chart in front of the CISO with the manager's own ratio looking bad next to it. All three moments create real pressure to answer with a benchmark — a published staffing ratio, an SLA norm, a tooling-spend percentage — as if it were a settled external fact rather than a number somebody else produced, for their own purposes, from a population that may have nothing to do with the reader's organization. This part is about using that kind of data without being misled by it: what's actually comparable across organizations of different size, industry, and risk profile, what almost never is, and the single most common way a manager gets burned — treating a survey with an unknown, self-selected respondent pool as if it were a random, representative sample of "SOCs like mine."

[CONCEPT] This part does not define the metrics a benchmark compares — MTTR, handle time, false-positive rate, and the rest are fixed inputs here, defined once and precisely in SOC Playbook Handbook, Part 32 — Metrics, and every worked example below inherits those definitions rather than re-deriving them. It does not build the headcount model a staffing-ratio benchmark gets checked against — that arithmetic, including the honest failure modes of a naive model, is this book's own Part 5 — Headcount & Capacity Modeling. It does not build the budget-category structure a tooling-spend percentage is computed against — that's Part 20 — Building & Defending the SOC Budget. And it is not the same question as the internal SOC maturity model in Part 30 — SOC Maturity Models, even though the two get confused constantly: Part 30 scores your own organization against a staged internal rubric, using evidence only your own program can supply; this part compares your own numbers against other organizations' numbers, using evidence a third party collected under conditions you usually cannot fully audit. A SOC can be honestly assessed as maturity-stage three on Part 30's scale and still have no defensible answer to "are we appropriately staffed compared to peers," because those are different epistemic problems with different failure modes. What's left, and what this part actually owns, is the discipline of knowing when an external number is telling you something real about your own organization and when it's telling you something real only about whoever chose to answer a survey.

1. Why "are we normal" is the wrong first question

[CONCEPT] "Are we normal" is the question a benchmark gets asked to answer, and it's the wrong one, because normal is not the same thing as adequate. A SOC can sit exactly at the reported industry-median staffing ratio and still be dangerously thin for its own actual risk profile — a small SOC protecting a payment processor's core transaction environment has no business being staffed like the "median" respondent in a blended survey that's mostly retail and light manufacturing. The reverse also holds: a SOC can run well above every published ratio and still be correctly staffed, because its risk profile, regulatory exposure, or detection-quality debt genuinely requires more people than a peer with a cleaner environment and better-tuned detections. The number a benchmark gives back describes a population's central tendency, not a target, and not a floor.

[SENIOR MANAGER]

The question a benchmark can actually help answer is narrower and more useful: does our own number sit so far outside the range of comparable organizations that it's worth asking why, before anyone decides whether "why" points at a real gap or at a real difference. That framing changes what a manager does with the number in three concrete ways. First, an outlier reading becomes a prompt to investigate the organization's own volume, handle time, and shrinkage data — the inputs Part 5 already builds — rather than a number to defend or attack on its own terms. Second, a benchmark that roughly matches the organization's own number is not, by itself, evidence the staffing level is correct; it only means the organization isn't unusual relative to whatever population answered the survey, which is a much weaker claim. Third, and most important, a benchmark is never a substitute for the organization's own headcount model, SLA-setting process, or budget justification — it is, at best, a sanity check layered on top of work that has to happen anyway.

Operational Reality *In practice, a benchmark number gets used backward from how it should be: a manager who already wants a budget increase goes looking for a published ratio that supports the ask, and a manager who's being asked to defend headcount goes looking for one that shows the team is already lean. Both searches will succeed, because published benchmarks vary widely enough — across vendors, years, and definitions of "analyst" — that a motivated search can usually find a supporting number in either direction. The discipline this part argues for is choosing the benchmark's source and scope before knowing what number would be convenient, not after.*

2. Three kinds of benchmark data, and what each one actually measures

[CONCEPT]

This part covers three specific categories of external comparison data, because each one is asked to do a different job and fails in a different way. The table below states what each one is actually measuring, where it typically comes from, and the confounder most likely to make a naive comparison wrong — read it as an index for what's covered in detail in §5 through §7, not as a complete answer on its own.

The table below supports the comparability discussion in §4 and the worked examples in §5–§7 — use it to identify which confounder to check first for whichever kind of number is in front of you.

Benchmark Category	What It Actually Measures	Typical Source	Biggest Confounder
Staffing ratios (analyst-to-endpoint, analyst-to-lead, tier mix)	How many people a comparable organization assigned to a comparable function, at the moment they answered the survey	Vendor-sponsored "state of the SOC" surveys, industry association member polls, consultancy staffing studies	Denominator mismatch (endpoints vs. alerts vs. employees) and whether outsourced/MSSP headcount is counted at all
SLA and response-time norms	What response-time commitment a comparable organization has published, promised contractually, or set as an internal target	MSSP marketing materials, industry-association guidance, informal peer conversation at conferences	Conflating a contractual SLA commitment with an internal severity-driven response target — these are different instruments answering different questions, per SOC Playbook Handbook, Part 29 — Playbook Severity Model and this book's own Part 22 — MSSP & Managed-Service Contract Management
Tooling spend as a percentage of security budget	What share of a comparable organization's reported security spend went to SIEM/EDR/SOAR/TIP licensing and related platform costs	Analyst-firm benchmark reports, vendor total-cost-of-ownership studies, budget-consultancy surveys	The percentage's denominator is highly sensitive to headcount cost and organization size — see §7

[SENIOR MANAGER] Every category in that table shares one structural property worth naming before going further: none of them, on their own, disclose anything about whether the reporting organization's outcomes were good. A staffing ratio doesn't say whether the reporting SOC caught the incidents that mattered. An SLA norm doesn't say whether the reporting organization actually met its own published number under real incident conditions. A tooling-spend percentage doesn't say whether the reporting organization's tools produced tested, validated detection coverage or a shelf of unused modules. That gap is a blind spot every benchmark in this category shares, and it's worth returning to explicitly.

Blind Spot A benchmark's staffing or spend number has no visibility into detection quality or missed incidents at all — a lean ratio might mean an efficient, well-automated SOC, or it might mean a SOC that's quietly under-detecting and hasn't found out yet. Detection Engineering Handbook V2's coverage, quality, and debt material (Parts 41–43) is the only place that question gets a real technical answer; no staffing or spend benchmark in this part's three categories substitutes for checking it, and a manager who reads "we're leaner than the industry average" as automatic good news, without a coverage and quality check behind it, is reading a number that was never designed to tell them that.

3. The self-selection problem

[SENIOR MANAGER] Of every mistake a manager can make with benchmarking data, this is the one worth naming first, because it silently invalidates almost everything downstream if it's not caught: most published SOC benchmark reports are built from a self-selected respondent pool, and the report rarely says so in language a busy reader will notice.

3.1 How a benchmark survey's respondent pool gets built

[SENIOR MANAGER] A typical vendor-sponsored "state of the SOC" or "security operations benchmark" report is built by emailing a survey link to a mailing list — often the vendor's own customer base, a webinar-registrant list, or an industry-association membership roll — and reporting statistics on whoever chose to click through and finish it. Nobody is randomly sampled. Nobody is required to respond. The population that actually answers is systematically different from "all SOC's" in at least three predictable ways: organizations that bought the sponsoring vendor's product are over-represented relative to the broader market, because the list the survey went to was that vendor's own customer list; organizations with more mature programs are over-represented, because a security leader with a chaotic, understaffed SOC has neither the spare time nor, often, the appetite to fill out a twenty-minute survey admitting to it; and larger organizations are over-represented among voluntary respondents generally, because they're more likely to have a person whose job includes representing the security function in public forums, conference panels, and vendor research studies.

[SENIOR MANAGER] None of that makes the reported numbers fabricated — the individual responses are usually genuine. It makes the aggregate statistic a description of "organizations that use this vendor's product and had the time and inclination to answer a survey," not a description of "SOC's in general," and definitely not a description of "SOC's comparable to mine." A median ratio computed from that pool is real arithmetic performed on a real but skewed sample, and the skew almost always runs in the same direction: toward larger, more mature, better-resourced organizations than the median organization actually operating a SOC today.

What Would Change My Mind This section claims self-selected vendor and association surveys skew systematically toward larger, more mature respondents. If a benchmark report disclosed its full sampling methodology — a genuinely randomized or stratified sample drawn from a defined population, response rates by organization size band, and a comparison of respondent demographics against the target population's actual demographics — and that disclosure showed no meaningful skew, this section's caution wouldn't apply to that specific report. The claim is about the typical, undisclosed case, not about every benchmark ever published; a report that does the methodological work to rule the skew out earns a different level of trust, and the test is whether it actually published that work, not whether it claims to be rigorous.

3.2 Worked example: the ratio that wasn't there

[SENIOR MANAGER] **CASE-3101** below is a composite, illustrative mid-market logistics-company SOC — no real organization's actual figures — built to show how the self-selection problem and a quiet denominator mismatch can stack on top of each other inside the same benchmark citation.

COMPOSITE CASE EXAMPLE (**CASE-3101**) — illustrative organization and figures, constructed to demonstrate the self-selection and denominator-mismatch mechanisms together; not drawn from a single traceable real SOC.

The setup: A 900-employee regional logistics company runs a 26-analyst in-house day-shift SOC, with after-hours and weekend coverage handled under a co-managed MSSP contract per this book's own Part 2 — SOC Operating Models. The environment covers roughly 68,000 monitored endpoints. A vendor-published "State of the SOC" report, based on 412 self-selected respondents to an emailed survey, states a median ratio of one analyst per 1,400 monitored endpoints.

The comparison as first read. The organization's own raw ratio is 26 analysts against 68,000 endpoints, or one analyst per roughly 2,615 endpoints — a ratio that looks 87% thinner than the reported median. The SOC manager, preparing a headcount request, cites the benchmark directly to justify hiring 23 additional analysts — enough to bring the organization to roughly one analyst per 1,400 endpoints — and closing what the slide calls "the gap to industry median."

What the CFO's office found on review. Asked for the report's methodology section, the manager discovered two things the slide hadn't surfaced. First, the 412 respondents were drawn overwhelmingly from the sponsoring vendor's own enterprise EDR customer base — 78% of respondents identified as organizations with more than 5,000 employees running dedicated, fully in-house 24/7 SOCs, a materially different population from a 900-employee company running a single day shift with contracted overnight coverage. Second, the reported ratio's "analyst" count was self-reported by each respondent with no defined boundary around whether MSSP or managed-service analysts counted — meaning an unknown share of the reporting population's denominator likely included contracted headcount the way this organization's own 26-analyst figure explicitly did not, since the after-hours MSSP analysts were never counted in that number at all.

What actually happened next. The manager withdrew the benchmark-based request and rebuilt the headcount ask using Part 5's own segmented volume-and-shrinkage model instead, checking the organization's real ticket volume, handle time, and coverage-hours requirement against its current 26-analyst roster. That model showed a genuine, defensible gap of four analysts driven by a 22% year-over-year alert-volume increase following a recent acquisition — a smaller, better-supported number than the benchmark-driven ask, and one the CFO approved within a single review cycle because it was arithmetic the CFO's own office could check line by line, not a borrowed statistic from an undisclosed population.

[SENIOR MANAGER] Nothing about the benchmark report in [CASE-3101](#) was dishonest — the vendor genuinely surveyed 412 real security leaders and genuinely reported a real median from that pool. The failure was entirely on the reading side: treating an unaudited, self-selected, larger-and-more-mature-skewed sample's median as if it were a neutral fact about "SOCs like ours," and treating two differently defined denominators — endpoints counted one way at the reporting organizations, headcount counted one way here — as if they were the same measurement. The fix wasn't finding a better benchmark. It was going back to the organization's own model, which is where the defensible number was sitting the entire time.

4. What's actually comparable across organizations

[SENIOR MANAGER] Before any benchmark number crosses from "interesting" to "actionable," it has to survive a check against the dimensions that most often make two organizations' numbers genuinely

incomparable, even when the report's own methodology is disclosed honestly. Four dimensions matter most.

4.1 Size

[SENIOR MANAGER] Organization size drives staffing and spend ratios nonlinearly, not proportionally, for a reason that has nothing to do with efficiency: fixed costs. A SIEM platform's base licensing tier, a SOAR platform's minimum seat count, a threat-intelligence feed's floor subscription price, and a shift's minimum coverage requirement (per Part 5's coverage-hours floor) all exist regardless of whether the organization has 5,000 endpoints or 50,000. A small organization pays close to the same fixed floor as a mid-size one and spreads it over a much smaller base, producing a materially higher ratio or percentage with zero difference in actual efficiency. Comparing a 5,000-endpoint organization's tooling-spend percentage against a 50,000-endpoint organization's, with no size-band adjustment, compares two different cost structures and calls the difference a management problem.

4.2 Industry and regulatory risk profile

[EXECUTIVE] A healthcare organization bound by breach-notification timelines, a payment processor under contractual card-network obligations, and a light-manufacturing company with no comparable regulatory clock are not answering the same underlying question when they set an SLA or staff a shift, even if a blended cross-industry survey reports one median number for all three. The "right" response-time target in each case is driven by the organization's own contractual and regulatory exposure — covered in this book's Part 22 — MSSP & Managed-Service Contract Management for the contract-instrument side, and by SOC Playbook Handbook, Part 29 — Playbook Severity Model for the internal severity-driven target — not by what a peer in a different regulatory posture happens to have published.

4.3 Coverage model and tiering

[SENIOR MANAGER] CASE-3101 already showed the mechanism: a benchmark's headcount figure is only comparable to another organization's if both count the same population of people doing the same work. An organization running the fully in-house model has every analyst on its own books; one running the co-managed or MSSP-overflow model per this book's own Part 2 — SOC Operating Models has a real chunk of coverage delivered by people who never appear in its internal headcount at all. Comparing raw analyst counts across those two models without adjusting for who's actually excluded is comparing two different definitions of "staffed," not two different levels of staffing. Tiering structure compounds the same problem: a four-tier SOC with a dedicated threat-hunting function and a three-tier SOC without one will report different "analyst" counts for reasons that have nothing to do with how well either is staffed for its own tier structure, a design question this book's own Part 3 — Tiering Models: L1/L2/L3 and Beyond owns.

4.4 Automation and detection maturity

[SENIOR MANAGER] Two organizations with identical raw alert volume can have wildly different real workloads if one has invested heavily in auto-closure, auto-enrichment, and SOAR-driven remediation and the other hasn't — the filtering ratio Part 5 §2 already names as a direct multiplier on every

downstream headcount calculation. A staffing benchmark that doesn't disclose or control for the respondent population's automation maturity is blending organizations that need very different headcounts to handle the same raw volume, and reading the blended median as a target for either one is a mistake regardless of how carefully every other comparability dimension was checked.

Cross-Book Pointer *This part does not cover how to decide what should be automated versus gated for human review in the first place — that design decision, and the workload reduction it produces, is SOC Playbook Handbook, Part 30 — Automation and SOAR. Read that part before trusting any staffing benchmark's implied "normal" workload per analyst; a benchmark's respondent pool almost never discloses its automation maturity, and two organizations with the same raw alert volume can need very different headcounts depending entirely on what that part's decisions already filtered out before a human ever saw the queue.*

[SENIOR MANAGER] The table below turns §4.1 through §4.4 into a working checklist — a comparability scorecard to run against any benchmark number before using it in a real conversation, not a complete taxonomy of every way two SOC's can differ.

Comparability Dimension	Why It Moves the Raw Number	Where This Book Covers the Adjustment
Organization size (headcount, revenue, endpoint/asset count)	Fixed licensing and coverage-floor costs spread over a smaller base inflate ratios and percentages with no efficiency difference	Part 5 (headcount), Part 20 (budget categories)
Industry vertical and regulatory obligation	Contractual and regulatory response-time clocks differ by industry regardless of what a blended survey reports	Part 22 (contract SLAs); SOC Playbook Handbook, Part 29 — Playbook Severity Model (internal severity targets)
Coverage/operating model (in-house, MSSP, co-managed, hybrid)	Headcount and spend figures may or may not include outsourced staff and services, changing what's actually being counted	Part 2 — SOC Operating Models
Tiering structure	Different tier designs produce different "analyst" headcounts for the same underlying workload	Part 3 — Tiering Models
Automation/SOAR maturity	The same raw alert volume can require very different headcounts depending on how much is filtered before a human sees it	SOC Playbook Handbook, Part 30 — Automation and SOAR
Detection quality and coverage maturity	A lean ratio may reflect efficiency or under-detection, and a staffing/spend benchmark alone cannot distinguish the two	Detection Engineering Handbook V2, Parts 41–43

5. Staffing ratio benchmarks in practice

[SENIOR MANAGER] With §3 and §4's cautions in place, staffing ratios still have a legitimate, narrow use: as one input among several when sanity-checking a headcount model that Part 5 has already built from the organization's own volume, handle time, and shrinkage data — never as the model itself. A Tier 1-to-Tier 2 ratio somewhere around three or four to one is a common pattern in a mature three-tier SOC,

per this book's own Part 3, and a manager whose own ratio sits far outside that range — a Tier 1-to-Tier 2 ratio of ten to one, say — has a legitimate prompt to ask why, not a mandate to hire down to a specific published number.

[SENIOR MANAGER] The honest use of a staffing ratio benchmark is directional and comparative to the organization's own trend line over time, not absolute and comparative to an outside population. A ratio that's drifted from four Tier 1 analysts per Tier 2 analyst to nine over eighteen months, with no corresponding change in the tiering model or automation maturity, is a real internal signal worth investigating regardless of what any external survey reports — the drift itself is the evidence, and the benchmark is only useful here as a rough gut-check that the organization hasn't drifted so far from common patterns that something structural has clearly changed. Used that way, a benchmark earns a supporting role in a staffing conversation. Used as the target itself, it reintroduces every problem §3 and §4 already named.

Manager's Note *When a benchmark ratio is going to appear anywhere in a budget deck, write the comparability caveats into the deck itself, not into a footnote nobody reads — "this figure comes from a vendor survey of 412 self-selected respondents, skewed toward larger in-house SOCs; our own segmented model in Part 5 is the basis for this request, and the benchmark is included only as directional context" takes one sentence and survives a skeptical follow-up question far better than a bare chart. If you wouldn't want to defend the source out loud in the room, don't put the number on the slide.*

6. SLA and response-time norms across risk profiles

[SENIOR MANAGER] SLA benchmarking runs into a comparability problem that's structural rather than statistical: "SLA" is used to mean at least two different instruments, and a benchmark report rarely says which one it's reporting. A contractual SLA is a commitment written into an MSSP or managed-detection agreement, with financial or termination consequences for missing it — this book's own Part 22 — MSSP & Managed-Service Contract Management owns that instrument. An internal severity-driven response target is a triage discipline, scored per alert against SOC Playbook Handbook, Part 29 — Playbook Severity Model's severity model, with no external party enforcing it. A published "industry-standard" fifteen-minute response time for critical alerts might describe either one, and the two carry very different consequences for missing them.

[EXECUTIVE] The table below illustrates why the same published SLA figure can be simultaneously too aggressive and not aggressive enough depending on the reader's own risk profile — a normalization exercise, not a comparison across real organizations (CONCEPTUAL SAMPLE).

Sector / Risk Profile	Illustrative Published "Industry Norm" for Critical-Severity Response	Why the Norm May Not Transfer
Payment processing, card-network-bound	15 minutes	The published norm may already be looser than the organization's own contractual card-network obligations; adopting it as a ceiling instead of a floor risks a real compliance breach
Healthcare, breach-notification-bound	15 minutes	Breach-notification timelines are driven by discovery-to-notification clocks measured in days, not minutes; a 15-minute triage target may be far stricter than the regulatory clock actually requires, diverting resourcing from where it's genuinely needed
Light manufacturing, low regulatory exposure	15 minutes	With no comparable external clock at all, the "right" number is whatever the organization's own risk appetite and Part 29 severity model set — the published norm supplies no information about what that appetite should be

[SENIOR MANAGER] The practical rule this table argues for: use a published SLA norm, if at all, only as a prompt to check the organization's own contractual and regulatory obligations against its current internal target — never as a substitute for reading the actual contract language or the actual regulatory clock. A norm that happens to match an organization's current target is not evidence the target is correct; it only means the target isn't unusual, which, as in §1, is a much weaker claim than "correct for this organization's own risk."

7. Tooling spend as a percentage of security budget

[VENDOR/PROCUREMENT] Tooling-spend percentages are the benchmark category most likely to produce a number that looks alarming on a board slide for reasons that have nothing to do with overspending, because the percentage is only as meaningful as its denominator, and the denominator moves for structural reasons a percentage alone hides completely.

7.1 Why the percentage moves with the denominator

[VENDOR/PROCUREMENT] A "percentage of security budget" figure has a denominator — total security budget — that is dominated, in most organizations, by headcount cost, per the budget-category structure this book's own Part 20 — Building & Defending the SOC Budget builds. Tooling costs (SIEM, EDR, SOAR, TIP licensing) scale roughly with data volume, endpoint count, or monitored-asset count, not with analyst headcount in anything close to a one-to-one relationship — the same core platform stack can cost a broadly similar absolute dollar amount whether an organization runs eight analysts or twenty-five against a comparable environment. An organization with a smaller headcount, whether by design (heavy automation, a co-managed model) or by circumstance (a leaner team stretched thin), has a smaller denominator, and the same absolute tooling bill produces a mechanically higher percentage with zero connection to whether the tooling itself is overpriced, underused, or badly negotiated.

7.2 Worked example: the tooling line that looked twice the industry average

[VENDOR/PROCUREMENT] CASE-3102 below is a composite, illustrative example — no real organization's actual figures — built to show the denominator effect described in §7.1 with concrete numbers.

COMPOSITE CASE EXAMPLE (CASE-3102) — illustrative organization and figures, constructed to demonstrate the denominator-sensitivity mechanism behind tooling-spend percentages; not drawn from a single traceable real SOC.

The board's question. An analyst-firm benchmark report, cited in a board briefing, states that organizations typically spend around 25% of total security budget on SOC tooling licensing. The organization's own total security budget is \$4.2 million; its tooling line (SIEM, EDR, SOAR, and threat-intel licensing combined) is \$1.85 million — 44% of the total. A board member asks, reasonably on its face, why the organization is spending nearly double what's apparently normal.

The breakdown behind the number. The organization's \$4.2 million budget splits into \$1.65 million in fully loaded headcount cost for a 14-analyst team plus two team leads, \$1.85 million in tooling licensing, and roughly \$700,000 in training, facilities, and MSSP retainer costs. The reported 25% benchmark figure, on inspection of the report's own respondent breakdown, comes disproportionately from larger in-house SOCs — a comparably sized illustrative organization in that population running 60 analysts against a \$9 million total budget, with \$6 million of that in headcount and \$2.25 million in tooling, produces the same 25% figure the report highlights.

What the comparison actually shows. The two organizations' absolute tooling spend — \$1.85 million versus \$2.25 million — is much closer to each other than either organization's total budget or headcount, because both are monitoring a broadly comparable volume of endpoints and log sources and paying broadly comparable platform licensing for it. The percentage looks nearly double only because the smaller organization's denominator — driven by a much smaller headcount cost — is proportionally smaller, not because its tooling is overpriced or its procurement was undisciplined. Reframing the comparison around cost per monitored endpoint, rather than percentage of total budget, showed the two organizations within roughly 12% of each other — a genuinely comparable figure, unlike the original 44%-versus-25% framing.

[EXECUTIVE] The board conversation that followed used cost per endpoint and cost per analyst instead of percentage of total budget, disclosed the benchmark's own size-band skew per \$3, and closed with a scoped, defensible follow-up: a one-time review of contract terms against two named comparable-size peers' publicly disclosed pricing, rather than an open-ended mandate to "get the percentage down to 25%" — a target that would have meant either firing tooling the SOC genuinely needed or growing headcount for reasons unrelated to actual staffing need, purely to inflate the denominator.

What Would Change My Mind *This section argues that tooling-spend percentages are usually more sensitive to headcount-driven denominator differences than to real differences in tooling efficiency. If a benchmark report normalized its percentage figure against a fixed unit — cost per monitored endpoint, cost per analyst, cost per terabyte of ingested log volume — instead of percentage of total budget, and organizations of very different sizes still showed materially different normalized figures, that would be real evidence of an efficiency difference rather than a denominator artifact, and this section's caution about the percentage framing specifically would not apply to that better-normalized figure.*

8. A decision framework: the benchmark comparability check

[SENIOR MANAGER] §3 through §7 build toward one repeatable check: a short sequence of questions to run against any external number before it appears in a staffing, SLA, or budget conversation. Figure 31.1 below lays that sequence out as a decision path.

Figure 31.1 — The benchmark comparability check. *CONCEPTUAL.* Illustrates the decision sequence built across §3–§7 for evaluating whether an external benchmark number is usable, usable only with normalization, or not usable at all in a given conversation; it is a process model for this part's own reasoning, not a capture of any specific team's review workflow. Diagram ID **FIG-3101**.

flowchart TD

```
A["External benchmark number\nin hand"] --> B["Is the respondent pool\nand sample size disclosed?"]
B -->|No| Z1["Treat as directional/marketing\nonly -- do not cite as a target"]
B -->|Yes| C["Is the sample self-selected\n(opt-in survey, vendor's\nown customer list)?"]
C -->|Yes, undisclosed skew| D["Assume upward skew toward\nlarger, more mature orgs (S3)"]
C -->|No, structured/random sample| E["Proceed to comparability check"]
D --> E
E --> F["Same size band,\nindustry, and risk profile\nas our org? (S4.1-S4.2)"]
F -->|No| Z2["Do not use directly --\nnormalize per size/industry\nor discard"]
F -->|Yes| G["Same coverage model\nand tiering structure? (S4.3)"]
G -->|No| Z2
G -->|Yes| H["Same denominator\n(endpoints, headcount,\nrevenue, alerts/day)? (S5-S7)"]
H -->|No| Z2
H -->|Yes| I["Automation/detection\nmaturity comparable? (S4.4)"]
I -->|No/unknown| J["Usable only as a rough\ndirectional check, caveated"]
I -->|Yes| K["Usable as one supporting\ndata point alongside the\norg's own model (S1, S5)"]
```

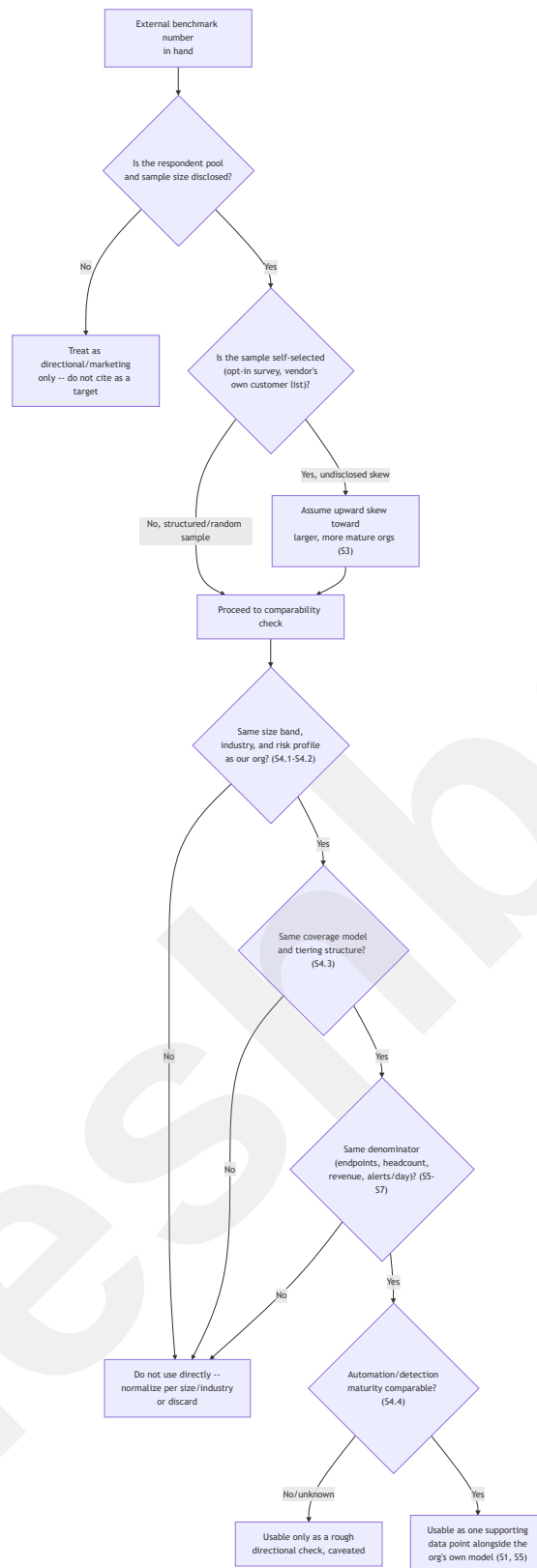


Figure 31.1 — The benchmark comparability check. *CONCEPTUAL.* Renders the decision path above as a static diagram. It supports the comparability check argued for across §3–§8 and gives a manager a repeatable sequence to run before citing any external number in a real conversation. **FIG-3101.**

[SENIOR MANAGER] Two outcomes at the bottom of that path matter more than the others. A benchmark that survives every check still lands as "one supporting data point," never as the model itself

— the organization's own Part 5 headcount model, Part 20 budget structure, or SOC Playbook Handbook Part 29 severity-driven SLA target remains the actual basis for the decision. A benchmark that fails the respondent-pool disclosure check at the very first node should be treated as marketing collateral, however polished the report looks, and citing it anywhere a skeptical follow-up question might land is a real professional risk to whoever cites it.

[SENIOR MANAGER] The table below turns Figure 31.1 into a standing checklist a manager can run against any specific report before it appears in a deck — a reusable credibility scorecard, not a one-time exercise for a single controversial number.

Question to Ask of the Benchmark	Why It Matters	If You Can't Answer It
Who published this, and what do they sell?	A vendor surveying its own customer base structurally over-represents organizations that already bought that vendor's product	Treat the figure as marketing collateral, not an independent benchmark
How many respondents, and is that number disclosed?	A median computed across 40 respondents carries far more sampling noise than one computed across 4,000	Do not cite a figure whose sample size you cannot state in the same sentence
How was the sample recruited?	Opt-in survey links systematically favor larger, more resourced organizations whose staff have time to answer a lengthy survey	Assume an upward skew toward size and maturity unless the report proves otherwise
What size, industry, and geography band is the figure sliced to?	A blended median across very different organization types describes neither type well	Only use figures sliced to your own size/industry band, or don't use the figure
What's the exact denominator?	Ratios and percentages with different denominators aren't comparable even when the numerator looks identical	Recompute your own figure using the same denominator before comparing anything
Does the figure include or exclude outsourced/MSSP headcount and spend?	A co-managed SOC's internal headcount excludes real work happening at the MSSP; an undisclosed convention compares different populations	Normalize for coverage model (Part 2) before treating the numbers as comparable
Does the report disclose anything about outcome quality — missed incidents, dwell time, tested coverage?	A lean ratio might reflect efficiency or under-detection, and most benchmarks can't tell you which	Treat the figure as directional only, never as evidence of good outcomes

9. Using a benchmark honestly in a board or budget conversation

[EXECUTIVE] A benchmark's most legitimate use is exactly the use §1 already named: a sanity check that earns a place in the conversation only after the organization's own model — Part 5's headcount arithmetic, Part 20's budget structure, SOC Playbook Handbook Part 29's severity-driven SLA target — has already produced the actual number being defended. Presented that way, a benchmark strengthens a request: "our own model, built from our actual volume and shrinkage data, calls for four additional analysts; that would bring our ratio roughly in line with the range reported across comparably sized organizations in [disclosed source], though we're treating that as supporting context, not the basis for the ask." Presented the other way — leading with the benchmark and backfilling the organization's own

justification afterward — invites exactly the kind of scrutiny [CASE-3101](#) describes, and rarely survives it intact.

Operational Reality *The version of this that shows up in real board decks is benchmark theater: a single chart, no disclosed methodology, positioned as the headline justification for a request that was actually decided on other grounds. It works until someone on the board — increasingly often, someone who has seen the same vendor's survey cited by a peer company for the opposite conclusion — asks where the number came from. Building the deck around the organization's own model from the start, with the benchmark as a one-line footnote, costs no more effort and survives that question the first time it's asked instead of the second.*

[EXECUTIVE] Part 24 — Executive & Board Reporting owns the broader discipline of translating any operational number, benchmarked or not, into a risk narrative a non-technical board member can act on and a skeptical follow-up question won't unravel; this part's job stops at making sure the specific external numbers that narrative might lean on have actually earned their place in it.

10. Where this goes next

[CONCEPT] This part is a caution layered on top of work built elsewhere in this book, not a standalone model. Part 5's headcount arithmetic, Part 20's budget-category structure, and SOC Playbook Handbook Part 29's severity-driven SLA targets remain the actual basis for a staffing, budget, or response-time decision; this part's comparability check exists to keep an outside number from quietly displacing that work rather than supporting it. Part 30 — SOC Maturity Models picks up the adjacent, frequently conflated question of scoring the organization's own program against a staged internal rubric, which needs no external population at all and fails in almost the opposite way this part's material does — an internal maturity score can be gamed by whoever's scoring it, where a benchmark's failure mode is an unaudited outside population. Part 33 — The Next Twelve Months: Building a SOC Roadmap is the place a defensible number, benchmarked or not, ultimately has to survive contact with a real sequencing and budget plan.

Cross-references. This book: Part 2 (operating models — what a benchmark's headcount figure does or doesn't count), Part 3 (tiering structure behind a staffing ratio), Part 5 (the headcount model a benchmark checks against, never replaces), Part 20 (the budget-category structure behind a tooling-spend percentage), Part 22 (contractual SLA commitments versus internal severity targets), Part 24 (translating a benchmark-supported number into a board-ready risk narrative), Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty (the judgment call when a defensible internal number and a comparison figure genuinely disagree), Part 30 (the adjacent but distinct internal-maturity lens), and Part 33 (where a benchmarked number has to survive a real roadmap). Other volumes: SOC Playbook Handbook, Part 29 — Playbook Severity Model (internal severity-driven response targets, as distinct from a contractual or published SLA norm) and Part 32 — Metrics (the MTTR, handle time, and volume definitions every ratio in this part treats as a fixed input); SOC Playbook Handbook, Part 30 — Automation and SOAR (the workload-filtering decisions behind any staffing benchmark's implied "normal" per-analyst load); Detection Engineering Handbook V2, Parts 41–43

(Detection Coverage, Detection Quality, Detection Debt — the outcome-quality question no staffing or spend benchmark in this part can answer on its own).

neshboy

Part 32 — AI, Automation, and the Future Shape of the SOC

Why this part exists

[CONCEPT] Two different questions get asked in the same sentence more often than any other topic in this book: "how do we use an AI copilot in triage" and "what does an AI copilot do to my headcount plan." They sound like the same question, and they aren't. The first is a workflow-mechanics question — which tool, which prompt pattern, which human-review gate, how a copilot's suggested disposition gets verified before a ticket closes — and SOC Playbook Handbook, Part 31 — AI-Assisted SOC Operations already owns that ground. The second is a staffing and organizational-design question with no home in either companion volume, because neither one is written from the chair that decides how many Tier 1 analysts a SOC needs once a real share of what those analysts used to do by hand gets dispositioned before a human ever opens the ticket. This part is that second question, and only that second question.

The distinction is load-bearing, not academic. A manager who reads SOC Playbook Part 31, learns how to gate an AI-suggested disposition behind a confidence threshold, and stops there has learned how to run this month's queue more efficiently. Nothing in that material tells the same manager whether a headcount plan built for a three-tier model six months ago is still the right plan, what a competency matrix should certify once a copilot does the technical legwork a Tier 1 anchor used to test for directly, or what a manager actually owes the analyst whose job just got smaller. Those are the three questions this part answers, in the same name-the-number register the rest of this book uses for every other staffing decision — not "AI is transforming the SOC," but which specific tasks are gone, which headcount number moves as a result, and what happens to the specific people who used to do those tasks.

This part assumes Part 3's tiering vocabulary and Part 10's four-axis competency framework already exist as artifacts a reader can point to — it revisits both directly, in §4 and §5, rather than rebuilding either from scratch. It also assumes the headcount-modeling mechanics from Part 5 — Headcount & Capacity Modeling and the cross-training program from Part 12 — Ongoing Training & Skill Development as tools this part routes decisions into rather than re-derives.

1. The displacement question: what AI actually removes from a Tier 1 queue, task by task

[CONCEPT] "AI is automating Tier 1" is not a claim a manager can act on, because Tier 1 is not one task. Part 3 defines Tier 1 as intake, known-pattern triage, and escalation of what doesn't fit — but that single sentence covers at least seven distinguishable activities, and an AI copilot or agentic workflow doesn't touch all seven the same way. Unbundling them is the first, most concrete thing this part does, because a headcount or competency decision built on the unbundled version and one built on the collapsed version produce different, and sometimes opposite, answers.

[SENIOR MANAGER] The table below scores each Tier 1 activity against how exposed it actually is to displacement as of this writing, not how exposed a vendor pitch deck claims it is. "High" exposure means a well-tuned agentic workflow can complete the task end to end for the majority of instances

without a human touching the ticket first; "low" means the task still requires a human judgment call that current tooling cannot reliably make on its own (CONCEPTUAL SAMPLE — illustrative exposure ratings based on the current state of commercially available SOAR-plus-LLM triage tooling, not a validated measurement study).

Tier 1 task	Automation exposure	What's left for the analyst
Known-pattern intake and auto-closure of confirmed benign or duplicate alerts	High	Auditing a sampled share of autonomous closures (§4); handling the residual low-confidence tail the model routes back
Initial context enrichment — asset owner, identity history, prior dispositions, threat-intel lookups	High	Judging when the assembled context is wrong, stale, or missing something the tool had no reason to check
Executing a documented, fully deterministic playbook step sequence	Medium-high	Playbook branches with an undocumented judgment call still route to a human; writing that branch down at all is SOC Playbook Handbook's territory, not this part's
Drafting an escalation note from raw ticket data	Medium	Verifying the draft is accurate before it meets the hand-off bar SOC Playbook Handbook, Part 27 — Escalation Quality defines
Ambiguous severity judgment with no clean playbook match	Low	This is the judgment axis Part 10 §3.4 defines directly; still squarely human work
Recognizing a genuinely novel technique the existing analytics weren't built to catch	Low	A copilot pattern-matches against what it's seen; a technique outside that distribution is invisible to it precisely because nothing flagged it as worth flagging
Hostile, urgent, or emotionally loaded stakeholder communication	Very low	The communication axis (Part 10 §3.3); a distressed stakeholder on a live call does not want to be routed to a bot, and routing them to one is a trust cost this book returns to in §6

[SENIOR MANAGER]

Read down that table and a pattern falls out that the single word "automation" hides: the top three rows are volume — the things Tier 1 does the most often, at the least depth per instance. The bottom three rows are judgment — the things Tier 1 does least often, at the most depth per instance. High-volume, low-depth work is exactly what current AI-assisted triage displaces well, and low-volume, high-depth work is exactly what it doesn't. That's not a coincidence; it's the same mechanism that made tiering itself a good idea in Part 3 — a cheap filter absorbing the routine majority so an expensive resource spends time on the minority that needs it. AI-assisted triage is a second filter stacked in front of the first one, not a different kind of thing.

Cross-Book Pointer This part does not cover how to build or gate the automation described in the table above — the confidence-threshold design, the human-review checkpoint, the specific prompt and tool-chain architecture that makes an agentic disposition trustworthy enough to run unattended. SOC Playbook Handbook, Part 31 — AI-Assisted SOC Operations owns that in full, and SOC Playbook Handbook, Part 30 — Automation and SOAR owns the broader automate-versus-gate decision this table's exposure ratings assume is already being made well. Read those for how to build the thing; this part only asks what having built it does to your org chart.

[SENIOR MANAGER] One number is worth naming even though it can't be sourced to a single validated study: SOCs running a mature AI-assisted triage layer against a high-volume, high-duplication alert stream report absorbing somewhere in the range of 40% to 65% of what previously required a human touch on the top three rows of the table above, measured in ticket-count terms, within 12 to 18 months of a serious rollout (CONCEPTUAL SAMPLE — illustrative range synthesized from the pattern this book's case examples below draw on, not a sourced industry benchmark). That range is wide on purpose. It depends heavily on how duplicative and low-context the alert stream already was — a SOC with high detection debt and a noisy rule set (the same variable Part 3 §6.1 names as a driver of tiering fit in the first place) has more of exactly the kind of alert an automated triage layer is good at absorbing, and a SOC that already tuned its detections well has less of that fat to cut.

2. The volume cliff: what a real drop in Tier 1 volume does to a tiering model built for the old volume

[SENIOR MANAGER] Part 3's tiering-fit scorecard treats alert volume as one of four variables that decides whether tiering earns its cost. A serious automation rollout doesn't change the SOC's total risk exposure — it changes that one variable, sharply, inside a single budget cycle, while the other three (team size, hiring-pool skill floor, detection debt) move much more slowly if at all. A tier boundary drawn for 4,000 alerts a day reaching a human doesn't automatically stay the right boundary once that number drops to 1,800, even though nothing else about the SOC's risk profile changed.

[SENIOR MANAGER] Four things can happen to the capacity a volume drop frees up, and a manager who doesn't choose deliberately among them will get the fourth one by default, which is usually the worst option:

- **Shrink Tier 1 headcount to match the new volume.** The straightforward option, and the right one when the freed capacity has nowhere useful to go and the SOC genuinely was overstaffed relative to its new, lower ticket count.
- **Redeploy freed capacity upward, into Tier 2 or a specialist bench-depth gap.** The right option when Part 12's cross-training matrix already shows a gap — recall the bench-depth matrix in Part 12 §3.2, where several specialist functions sat one person short of target. A shrinking Tier 1 queue is a scheduling opportunity to close exactly that gap, using people who already know the environment.
- **Convert the surviving Tier 1 seats into an oversight function auditing the automation itself.** Covered in full in §4 — someone has to check that the automated disposition layer is actually right, and that job is a genuinely new kind of Tier 1 work, not the old kind at lower volume.
- **Do nothing yet, because the volume drop isn't confirmed to be stable.** The default outcome, and the dangerous one if it isn't a deliberate choice: a manager who hasn't decided watches headcount sit idle against a queue that shrank months ago, until someone above them notices the gap and makes the call unilaterally, usually as a layoff rather than a redeployment.

Operational Reality *A vendor's pilot numbers almost always overstate the volume reduction a production rollout will actually sustain, because a pilot runs against a curated slice of the queue chosen to make the tool look good, and a 90-day pilot window rarely survives a full seasonal cycle of alert-volume variance. Treat a vendor's claimed absorption percentage as a ceiling to test against your own ticket data over a full quarter, not a number to build next year's headcount plan around on day one.*

3. Two SOC's, the same volume drop, opposite outcomes

[SENIOR MANAGER] The choice among those four options is easiest to see concretely against two SOC's that started from a similar position and picked differently. Both are composite, illustrative constructions, not traceable real organizations.

CASE-3201 — the redeployment path. *COMPOSITE CASE EXAMPLE — merges patterns from several mid-sized in-house SOC's that rolled out AI-assisted triage in 2025–2026; no single organization's staffing or figures are reproduced.* A 30-analyst in-house SOC — 18 Tier 1, 8 Tier 2, 4 specialist — deployed an AI-assisted disposition layer against its highest-duplication alert categories. Over 12 months, measured ticket-count absorption on those categories reached 55%, freeing roughly the equivalent of six full-time Tier 1 seats' worth of capacity. The SOC made no involuntary reductions. It let attrition — assumed here at an illustrative 20%-a-year baseline (CONCEPTUAL SAMPLE; Part 18 documents the tenure-at-departure timing behind burnout-driven exits but doesn't itself publish an aggregate annual rate) — absorb two of the six freed seats over the year without backfilling them, redeployed three analysts into a Part 12 cross-training rotation targeting the two specialist functions already sitting a person short of target bench depth, and created one new "disposition audit" seat (\$4) sampling and re-reviewing the AI layer's autonomous closures. Fully loaded Tier 1 cost held roughly flat at around \$1.1 million a year (16 seats at a blended \$70,000 fully loaded cost, CONCEPTUAL SAMPLE), but the SOC exited the year with two additional specialist-capable analysts it hadn't budgeted a new hire for, and a documented, growing false-negative catch rate from the audit seat that the SOC could show a CISO as evidence the automation was actually safe to keep expanding.

CASE-3202 — the layoff path. *COMPOSITE CASE EXAMPLE — merges patterns from several publicized and privately reported AI-driven SOC headcount reductions; no single organization's figures are reproduced.* A comparably sized SOC deployed a similar tool against a similar alert mix, measured a similar absorption rate within the first two quarters, and reduced Tier 1 headcount from 18 to 11 in a single reduction-in-force inside a 60-day window, timed to capture the projected savings in the same fiscal year the tool was purchased. No cross-training rotation, no audit seat, no communicated plan for the remaining analysts beyond "the tool will handle more of the volume now." Within two quarters, voluntary attrition among the surviving 11 Tier 1 analysts ran at roughly double the SOC's prior baseline — exit interviews (Part 18's structured methodology) cited "no confidence in job security" and "unclear what's expected of me now" as the two most common themes. Separately, and more expensively: the automation layer's absorption had been measured on ticket count, not on investigative depth, and the tickets that still reached a human were disproportionately the harder, more time-consuming ones the tool had never touched — average per-ticket handle time for the surviving Tier 1 group rose by roughly

35% within the same two quarters, largely erasing the headcount savings the reduction was built to capture. The SOC re-hired three Tier 1 seats within nine months, at a market rate roughly 12% above what the departed analysts had been paid, plus a full ramp cycle (Part 11) for each replacement.

[SENIOR MANAGER] Both SOC's measured the same absorption number. The difference in outcome wasn't the tool — it was whether the freed capacity was routed to a deliberate destination or left to default into a layoff sized to whatever the volume number implied on paper. §6 develops the retraining and communication obligation this comparison turns on; the decision tree below is the routing logic CASE-3201 followed and CASE-3202 skipped.

flowchart TD

A["AI-assisted disposition layer\nabsorbs a measured share of\nTier 1 known-pattern volume"] -->

B{"Absorption confirmed stable\nacross a full quarter,\nnot just a pilot window?"}

B -->|No| C["Hold headcount; keep measuring.\nDo not backfill attrition yet,\nbut do not reduce either."]

B -->|Yes| D{"Does Part 12's cross-training\nmatrix show an existing\nbench-depth gap?"}

D -->|Yes| E["Redeploy freed capacity into\nthe gap via a paired rotation\n(Part 12 §3)"]

D -->|No| F{"Is autonomous-disposition\naccuracy independently\naudited yet (§4)?"}

F -->|No| G["Convert one or more freed\nseats into a disposition-audit\nrole before reducing anything"]

F -->|Yes, and no gap exists| H{"Is the volume drop large\nenough that even audited,\nredeployed capacity exceeds\nrealistic near-term need?"}

H -->|No| C

H -->|Yes| I["Reduce headcount only through\nattrition and a communicated\nredeployment offer (§6)\n—\never a same-quarter RIF\ntimed to the tool's ROI case"]

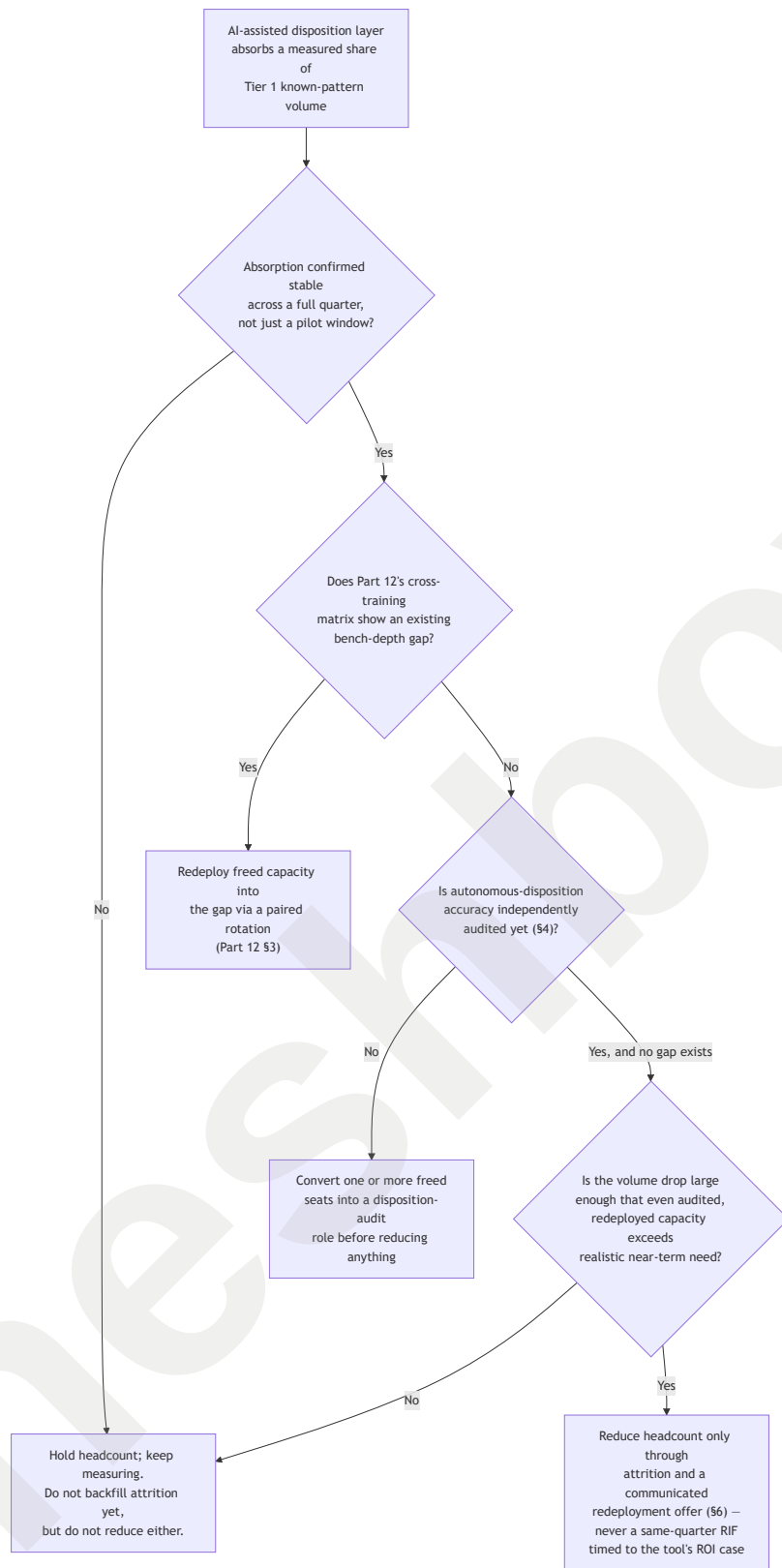


Figure 32.1 — Routing freed Tier 1 capacity after a confirmed automation-driven volume drop. *CONCEPTUAL*. Diagram ID **FIG-3201**. Illustrates the sequence of checks a manager should run before treating a measured absorption percentage as a headcount-reduction number — confirming stability, checking for an existing bench-depth gap, and confirming audit coverage exists — before attrition-

driven reduction is ever the answer. This is a decision-support diagram, not a capture of either case example's actual internal process.

Figure 32.1 — Routing freed Tier 1 capacity after a confirmed automation-driven volume drop. *CONCEPTUAL.* Static decision-tree diagram rendered from the Mermaid source directly above. It supports the freed-capacity routing logic §3's case comparison and §6's sequencing obligations both depend on. Diagram ID [FIG-3201](#).

4. Extending the hand-off model: where the AI sits relative to Tier 1, not above or below it

[SENIOR MANAGER] Part 3 §2.4 builds a RACI matrix across the three tiers and the specialist functions. An AI-assisted disposition layer is tempting to add to that chart as a new row sitting below Tier 1 — "Tier 0" — and that instinct is worth resisting for the same reason Part 3 §3.1 rejects treating threat hunting as "L4": a tier is an org-chart position with a person accountable for it, and an automated disposition layer has no accountability of its own to assign. The accountability for what it decides still belongs to a named human, every time, even when no human looked at a specific ticket before it closed.

[SENIOR MANAGER] The practical fix is a confidence-threshold boundary sitting inside Tier 1's existing scope, not a new tier alongside it: dispositions the model completes above a defined confidence bar close autonomously, subject to audit (below); dispositions below that bar route to a human exactly the way an ambiguous ticket always has. The table below extends Part 3's hand-off-trigger table (§4 of that part) with the one new trigger this shift adds — it does not replace or restate the rest of that table, which still governs every Tier 1 → Tier 2 and Tier 2 → Tier 3 boundary unchanged.

Trigger	From → To	Basis	Example Threshold
Autonomous-disposition confidence	Alert → closed, no human touch	Model's own calibrated confidence score against a validated benign/duplicate pattern	Confidence at or above a validated threshold, re-validated per §4.1 below
Below-threshold confidence	Alert → Tier 1 human review	Same score, below the validated bar	Any score below the validated threshold
Sampled autonomous closure	Closed ticket → disposition-audit review	Random sample, independent of confidence score	A fixed percentage of all autonomous closures, reviewed after the fact

4.1 Why the threshold itself needs a named owner and a re-validation cadence

[SENIOR MANAGER] A confidence threshold is a number someone picked, usually during a vendor's initial tuning pass, and it drifts out of date the same way every other tuned threshold in this book does the moment the underlying alert mix shifts — a new detection ships, an upstream data source changes format, a threat actor starts using an infrastructure pattern the model has never seen labeled correctly. Detection Engineering Handbook V2, Part 43 — Detection Debt already names the mechanism by which an unmaintained detection threshold quietly stops matching the environment it was tuned against; the same mechanism applies to a disposition-confidence threshold, and it needs the same owner and the same re-validation discipline, not a one-time vendor configuration a manager never revisits.

[SENIOR MANAGER] The disposition-audit sample from the table above is what makes re-validation possible at all — without it, a drifting threshold has no feedback loop, and the first sign of trouble is a missed incident rather than a rising audit-flagged error rate. Sizing that sample is a QA-program decision in every sense Part 15 already covers: random, not risk-selected (an analyst who only samples the tickets that already look suspicious will never catch a confidently-wrong autonomous closure that looked routine), and large enough to catch a meaningful drift within a quarter rather than a year. A sample covering 5% to 10% of autonomous closures, reviewed by a human against the same disposition standard a live ticket would get, is a reasonable starting point for most volumes (CONCEPTUAL SAMPLE — illustrative sampling rate, calibrate against your own false-negative tolerance and audit capacity).

5. Rewriting the competency matrix for an AI-augmented Tier 1

[CONCEPT] Part 10 built a four-axis competency matrix — technical skill, tool proficiency, communication, judgment — anchored to what a Tier 1 analyst has to demonstrate unaided. Part 10 §7 flagged, without answering, exactly the question this section now answers: what happens to the technical-skill and tool-proficiency anchors once a copilot absorbs a meaningful share of the work those anchors used to test.

[SENIOR MANAGER] The honest answer is not "those axes stop mattering." It's that the specific behavior each axis certifies has to shift from *doing the task* to *supervising the tool doing the task*, and those are different skills that don't reliably co-occur — an analyst can be excellent at manually building a cross-tool query and mediocre at spotting when an AI-assembled context summary quietly dropped a relevant data source, because the second skill has never been tested until now. The table below reworks Part 10's L1 anchors for each axis; it assumes Part 10's four-axis structure and gate-per-axis rule (Part 10 §4.1) unchanged, and only rewrites what each anchor certifies (CONCEPTUAL SAMPLE — illustrative revised anchors, calibrate against your own tool's actual failure modes before adopting).

Axis	Old L1 anchor (Part 10 §4.2)	Revised L1 anchor for an AI-augmented queue
Technical skill	Maps an alert to the correct playbook step unaided; explains why a match is or isn't a true positive	Independently verifies whether an AI-assembled disposition or context summary is correct before acting on it — can reconstruct the same conclusion manually, on demand, without the tool
Tool proficiency	Pulls the exact fields a documented playbook calls for, within SLA, unaided	Directs the AI tool effectively (prompting, scoping, escalating its output) and recognizes the specific query or data-gap patterns where the tool reliably gets it wrong
Communication	Escalation notes meet the SOC Playbook Handbook, Part 27 — Escalation Quality bar, with no rewrite needed	Edits an AI-drafted escalation note to the same bar and can identify, specifically, what the draft got wrong or omitted — not just accepts a fluent-sounding draft unread
Judgment	Applies the severity model consistently on clear-cut cases; escalates genuinely ambiguous ones instead of guessing	Same anchor, unchanged — and now tested more often, because the tickets that still reach a human are disproportionately the ambiguous ones the tool routed up

[SENIOR MANAGER] Notice what the judgment row does and doesn't do: it doesn't change. That's the point, not an oversight. The judgment axis was always the hardest one to automate, per §1's task table,

and an AI-augmented queue doesn't reduce how much judgment a surviving Tier 1 seat needs — if anything, it concentrates judgment demand, because the tickets that still reach a human are the ones the model's confidence threshold explicitly flagged as not confidently routine. A manager who assumes AI-augmentation lowers the overall skill bar for Tier 1 has the direction backward: it raises the average difficulty of what's left in the human-facing queue, even as it lowers the total volume.

What Would Change My Mind This section claims the judgment axis doesn't get easier, and may get effectively harder per ticket, once AI absorbs the routine majority of Tier 1 volume. If a SOC running a mature AI-assisted triage layer showed its Tier 1 judgment-axis QA scores (Part 15) holding flat or improving with no change in hiring bar or training investment — not because the remaining tickets got easier, but because the model's confidence threshold was somehow pre-filtering out judgment difficulty along with volume — that would undercut this section's central claim and suggest the concentration effect described above is smaller in practice than this part assumes.

[HR/PEOPLE] One consequence worth naming directly for hiring and promotion: the revised technical-skill and tool-proficiency anchors above are, in a specific sense, harder to certify than the ones they replace, because "can reconstruct the AI's conclusion manually on demand" requires the underlying manual skill to still exist, not to have atrophied from disuse. A hiring pipeline or promotion bar that lets a candidate lean entirely on the tool during an assessment, never demonstrating the manual reconstruction Part 8's assessment methodology should now explicitly test for, is quietly certifying tool-operation skill and calling it analyst competency — a substitution risk with the same shape as the tenure-substitutes-for-competency failure Part 10 §2 already warns against, just with a different proxy standing in for the real thing.

6. The retraining and redeployment obligation

[CONCEPT] Everything in §§1–5 is a technical and organizational-design question a manager can answer with a table, a threshold, and a decision tree. This section is not that kind of question. It's a judgment call about what a manager owes a specific analyst whose role is changing under them, and the honest answer has real limits a manager cannot promise past, alongside real obligations that don't disappear just because the tool vendor's contract doesn't mention them.

6.1 What can be promised, and what can't

[SENIOR MANAGER] A manager cannot honestly promise that no Tier 1 role will ever shrink because of automation — CASE-3201's own SOC still let two seats go through natural attrition rather than backfilling them, and pretending otherwise to a nervous team erodes exactly the trust it's meant to protect, the moment the first seat quietly doesn't get refilled. What a manager can promise, and should say out loud rather than leave implied, is threefold: that a reduction in Tier 1 headcount driven by automation will be sequenced through attrition and redeployment first, not a same-quarter reduction timed to a vendor's ROI case; that every analyst affected gets a real, named retraining path — not a vague "opportunities may arise" — before any involuntary reduction is even considered; and that the

manager will say clearly, on a specific timeline, what's actually changing about the job, rather than let rumor fill the silence CASE-3202's SOC left open.

[HR/PEOPLE] "A real, named retraining path" has a specific meaning here, not a motivational one: a documented destination (Tier 2, a specialist bench-depth gap per Part 12 §3.2, the disposition-audit role from §4, or an adjacent function entirely), a defined format and duration drawn from Part 12's cross-training toolkit, and an explicit, observable bar — the same discipline Part 10's competency matrix already applies to a tier transition — for what "ready" looks like at the end of it. An analyst told "we're retraining you" with no destination, no format, and no bar has been told nothing actionable; it's the identical failure Part 10 §2 diagnoses in "I'd know it if I saw it" competency judgments, now applied to a redeployment promise instead of a promotion one.

Management Autopsy — "automate first, explain later" (CASE-3202, COMPOSITE CASE EXAMPLE)

The decision: Facing budget pressure and a vendor's projected first-year savings, a SOC manager approved an AI-assisted triage rollout and a same-quarter Tier 1 reduction sized to the vendor's absorption estimate, communicated to the affected team only after both decisions were finalized.

Why it seemed reasonable: The vendor's pilot numbers were real and reproducible in the SOC's own data; the budget case was genuinely strong; and moving fast on both the rollout and the reduction looked like decisive leadership rather than the drawn-out, uncertain process a phased redeployment plan would have required explaining to finance in the interim.

How it failed: The reduction was sized to a ticket-count absorption number that said nothing about the investigative-depth shift described in §3 — the tickets that stayed human-facing got harder on average, not just fewer, and per-ticket handle time rose enough to erode most of the projected savings within two quarters. Separately, and more durably damaging: the surviving team learned about both the tool and the layoffs at the same time, with no stated plan for what came next for them specifically, and voluntary attrition among the survivors roughly doubled the SOC's prior baseline inside two quarters — the exact people the reduction was meant to retain around a leaner, tool-assisted queue left it fastest.

The fix: Separate the rollout decision from the headcount decision by at least one full quarter of measured, stable absorption data (Figure 32.1's first gate), and never let a vendor's ROI timeline set the pace of a reduction affecting real people. Communicate the automation rollout and the redeployment plan together, before either the tool or any reduction goes live — not the tool now and the plan for people later, which is the sequencing CASE-3202 got backward.

6.2 A worked sequencing that held

[SENIOR MANAGER] CASE-3201's SOC, revisited: the redeployment offer given to the three analysts moved into cross-training wasn't generic. Each was offered a specific, named track — two into the specialist bench-depth gaps Part 12's own matrix had already flagged as a person short of target, one

into the new disposition-audit seat — with a defined 90-day rotation window drawn directly from Part 12's paired-rotation format, an explicit sign-off bar borrowed from Part 10's competency-matrix discipline, and a stated fallback: if the rotation didn't clear the bar inside the window, the analyst returned to a Tier 1 seat at unchanged pay and standing, not a smaller one. That fallback clause mattered more than the opportunity itself for morale — it converted "we're retraining you" from a euphemism for "we're evaluating whether to keep you" into an actual offer with a floor under it, which is precisely the distinction analysts in CASE-3202's SOC never got the benefit of.

People Risk Trap *Announcing an automation rollout and a headcount reduction in the same breath — even an honestly sized, well-sequenced reduction — teaches every remaining analyst that the next automation announcement is also a layoff announcement, regardless of what a manager actually intends this time. Once that association forms, it doesn't reset with the next rollout's better handling; it compounds, and a manager loses the ability to communicate a genuine efficiency gain without triggering a flight response in exactly the analysts most worth retaining. Announce the retraining and redeployment plan first, on its own, before or alongside the tool — never the reduction number first with a retraining promise attached as an afterthought.*

[HR/PEOPLE] None of this removes the real possibility that a genuine, non-punitive reduction is still the right call once redeployment and attrition have both been tried and the volume drop is confirmed durable. §6.1's promise was never "nobody's headcount ever shrinks" — it was that the sequence runs attrition and redeployment first, on a timeline measured in quarters against Figure 32.1's gates, not compressed into the same budget cycle as the tool's purchase order.

7. New roles this shift creates, and where they sit on the ladder

[SENIOR MANAGER] §4's disposition-audit function and §6's redeployment tracks both point at the same underlying fact: an AI-augmented SOC doesn't just shrink Tier 1, it creates work that didn't exist before, and that work needs a name, a pay band, and a place on the career ladder Part 13 governs, or it ends up as an unrecognized, uncompensated addition to whoever happens to be closest to it.

[SENIOR MANAGER] The disposition-audit role is the clearest new seat: someone whose job is specifically sampling and re-checking autonomous closures against the standard Part 15's QA program already applies to human-closed tickets, feeding threshold-drift findings back to whoever owns the confidence bar per §4.1. This is closer to a QA specialization than a triage role, and it should be leveled and compensated that way — Part 13's ladder should treat it as a named branch alongside detection engineer and threat hunter, not an unpaid addition to an existing Tier 1 job description.

[SENIOR MANAGER] A second, less obvious new role sits closer to detection engineering than to triage: someone who tunes the automation layer's confidence thresholds and reviews its failure patterns the way a detection engineer tunes a rule's false-positive rate. Detection Engineering Handbook V2, Part 22 — Detection as Code already defines the review-pipeline discipline a role like this should borrow — version control, peer review, and regression testing applied to a threshold change instead of a rule change. This part's job is narrower than that pipeline's mechanics: naming that the role exists, that it

draws a genuinely different skill set than either classic Tier 1 triage or classic detection-rule authoring, and that Part 13's branch-bar discipline (a real work sample, not a self-declared interest) should gate entry into it the same way it gates entry into the existing specialist branches.

[HR/PEOPLE] Neither new role should be filled by simply relabeling an existing Tier 1 seat without a corresponding pay-band and competency-matrix update. Doing that reproduces the exact leveling-drift failure Part 13 §5 names in detail — a title changes, scope changes, and pay does not — just triggered by a tool rollout instead of a retention negotiation. Route both new roles through the same evidence-packet discipline Part 13 §4.2 already requires for any other branch promotion.

8. Headcount and budget implications, briefly

[SENIOR MANAGER] This part does not own headcount-modeling mechanics — Part 5 does — or the budget case for the automation tooling itself — Part 20 and Part 21 do. What's worth stating plainly here, because it's easy to get backward under budget pressure, is that an AI-assisted triage layer changes the volume input to Part 5's model; it does not remove the shrinkage math (PTO, sick leave, training time, per Part 5 §3) that model already accounts for, and it adds a genuinely new cost line — tooling license cost, the audit and threshold-tuning roles from §4 and §7, and the retraining investment from §6 — that a naive "AI saves headcount" budget pitch tends to leave out entirely.

[VENDOR/PROCUREMENT] The tooling-selection process for an AI-assisted triage platform is, structurally, the same PoC-evaluation and total-cost-of-ownership exercise Part 21 — Tooling Procurement & Platform Strategy already builds for a SIEM, EDR, or SOAR selection — reference-customer calls, a real PoC against your own alert data rather than the vendor's curated demo set, and a TCO model that includes the retraining and audit-staffing cost this part names rather than only the license fee. This part adds one item that vendor's TCO conversation rarely raises on its own: ask specifically what the vendor's own pilot customers did with the headcount their tool freed up, and whether any of them can be reached directly rather than through a vendor-selected reference call — a vendor with a genuinely durable case will have a customer willing to describe a redeployment outcome like CASE-3201's in their own words, not just an absorption percentage.

Operational Reality A board or CFO hearing "AI will cut our SOC headcount cost" almost never hears the second half of that sentence — "and add a smaller, different cost for oversight, tuning, and retraining" — unless a manager says it explicitly and puts a number next to it. Left unsaid, the gap between the promised savings and the actual, smaller net savings shows up a year later as a credibility problem for the manager who made the original case, not as evidence the technology failed.

9. How much of this is real: a falsifiability check

[CONCEPT] Every claim in this part rests on one underlying premise: that a meaningful, durable share of Tier 1 volume is genuinely displaceable now, not merely a vendor narrative ahead of the actual

capability. That premise deserves the same scrutiny this book applies to every other confidently stated claim.

[SENIOR MANAGER] The honest evidence for the premise, as of this writing, is mixed in a specific and checkable way: the high-volume, low-context tasks in §1's top three rows — known-pattern closure, context enrichment, deterministic playbook execution — have solid, reproducible absorption evidence across multiple independent SOC deployments, which is why this part treats displacement in that band as real rather than speculative. The bottom three rows — ambiguous judgment, novel-technique recognition, hostile stakeholder communication — have essentially no comparable evidence of genuine displacement, and vendor claims that stretch into that territory should be treated with the same skepticism this book applies to a self-selected benchmarking survey in Part 31 — Benchmarking & Industry Comparison.

What Would Change My Mind This part treats Tier 1's high-volume, low-context tasks as genuinely, durably displaceable by current AI-assisted triage tooling, and its judgment-heavy tasks as not. If a SOC could show — across more than one incident, not a single anecdote — an AI-assisted workflow correctly recognizing a genuinely novel technique with no prior labeled pattern to match against, and doing so reliably rather than as a lucky one-off, that would move the judgment row of §1's table from "low exposure" toward "medium," and this part's redeployment guidance in §§3 and 6 would need to shift correspondingly toward assuming a smaller, not larger, surviving judgment-heavy Tier 1 function.

[SENIOR MANAGER] Absent that evidence, the operating assumption this part recommends is the one every case example above already used: plan the volume-driven headcount and competency changes in §§2–5 with real confidence, because the evidence supports them, and plan the judgment-heavy core of Tier 1 as a stable, durable function that a manager should keep investing in — hiring for, training for, and building a ladder toward — rather than one to quietly shrink on the strength of a vendor roadmap slide describing capability that doesn't exist yet.

Cross-references

Within this book: this part assumes Part 3 — Tiering Models: L1/L2/L3 and Beyond (the tiering vocabulary revisited in §§2–4) and Part 10 — Competency Models & Skills Matrices (the four-axis framework rewritten in §5), and consumes Part 5 — Headcount & Capacity Modeling and Part 12 — Ongoing Training & Skill Development as tools it routes decisions into rather than re-derives. It draws on Part 9 — Queue Health & Workload Management for distinguishing a volume drop from a detection-quality fix, Part 13 — Career Ladders & Promotion Criteria for where the new roles in §7 sit on the ladder, Part 15 — Quality Assurance Programs for the audit-sampling discipline in §4, Part 18 — Attrition & Retention for the baseline attrition figures the worked examples in §3 and §6 compare against, and Part 20 — Building & Defending the SOC Budget and Part 21 — Tooling Procurement & Platform Strategy for the budget and vendor-evaluation mechanics §8 points to rather than repeats. It feeds forward into Part 33 — The Next Twelve Months: Building a SOC Roadmap, where an automation-driven redeployment plan becomes one input into a broader sequencing exercise.

Other volumes: SOC Playbook Handbook, Part 31 — AI-Assisted SOC Operations, for the operational mechanics this part deliberately does not re-cover — tool selection inside a workflow, confidence-threshold implementation, and human-review gating at the ticket level. SOC Playbook Handbook, Part 30 — Automation and SOAR, for the broader automate-versus-gate decision this part's displacement analysis assumes. Detection Engineering Handbook V2, Part 22 — Detection as Code, for the review-pipeline discipline §7's threshold-tuning role should borrow, and Part 43 — Detection Debt, for the drift mechanism §4.1 applies to a disposition-confidence threshold instead of a detection rule.

neshbooy

Part 33 — The Next Twelve Months: Building a SOC Roadmap

Why this part exists

[CONCEPT] Every earlier part in this book hands a manager one lever and tells them how to pull it well: how to size headcount (Part 5), how to design a shift pattern (Part 6), how to hire (Part 7), how to build a competency matrix (Part 10), how to run QA (Part 15), how to defend a budget (Part 20), how to score maturity (Part 30). None of them answer a question every SOC manager still has to answer once all of that exists on paper: in what order, over the next 12 months, given one budget cycle and one hiring pipeline that can't move any faster than it actually moves. A roadmap is not a longer to-do list. It is a sequencing decision under three hard constraints — money, hiring throughput, and the finite number of hours a manager's own senior people can spend building new programs before those programs get a shallow version instead of a real one.

[CONCEPT] This part does not re-derive the arithmetic behind any single lever. It does not rebuild the headcount formula — that's Part 5's job, and this part takes a finished headcount number as an input the same way Part 5 itself takes handle time and volume as inputs from SOC Playbook Handbook, Part 32 — Metrics. It does not redesign the budget's category structure — Part 20 — Building & Defending the SOC Budget owns headcount, tooling, training, facilities, and vendor-services as spending categories, and this part treats those categories as the lanes a roadmap's line items travel through, not as something to reinvent. And it does not build the maturity-scoring rubric itself — Part 30 — SOC Maturity Models owns the four organizational-design axes (staffing, process, training-program, and tooling maturity) this part uses only as an already-scored starting point and a stated target. What's left, and what this part actually owns, is the sequencing problem: given a maturity gap and a budget, what happens in month one, what has to wait until month seven because it depends on something that hasn't shipped yet, and what a manager tells the board when only three of the year's four target-maturity axes actually move.

[CONCEPT] The horizon is deliberately capped at 12 months, and that boundary is a design choice, not a limitation this part apologizes for. A budget cycle, a hiring pipeline's realistic annual throughput, and most vendor contract terms all run on roughly annual clocks — a roadmap built to that clock can be defended against real money and real reqs. Past 12 months, the compounding uncertainty (a new detection doubling queue volume, an acquisition adding a business unit, an unplanned resignation wave) makes a bespoke, organization-specific plan more honest than a templated multi-year one, and this part stops at the edge where that's true rather than pretending a longer horizon is just this same model run twice.

1. Sequencing is a different problem than prioritization

[CONCEPT] Prioritization asks which gap matters most. Sequencing asks a harder question: given that three or four gaps all matter, which one has to close first because something else depends on it, which ones can run in parallel without competing for the same scarce resource, and which one has to wait

even though it's important, because starting it now would only produce a rushed version. A manager who prioritizes well and sequences badly still fails — a maturity assessment that correctly identifies "we need a competency matrix and we need to migrate our SIEM" as the two biggest gaps says nothing about whether launching both in the same quarter is a plan or a collision. §6 works through exactly that collision with real numbers.

1.1 Three scarce resources a roadmap actually arbitrates

[SENIOR MANAGER] A roadmap is really arbitrating three resources, and only one of them shows up on a budget spreadsheet. The first is money, tracked against Part 20's categories and the most visible constraint, because it has a dollar figure and an approval process attached to it. The second is hiring throughput — not money, but calendar time: a requisition approved in January and a requisition approved in October compete for the same finite number of qualified candidates the sourcing channels in Part 7 can actually produce inside a year, and doubling the budget for recruiting doesn't double the rate qualified people come through the pipeline. The third, and the one most roadmaps never name explicitly, is the bandwidth of the specific senior people — team leads, the SOC manager, a handful of tenured analysts — whose hours get consumed every time a new program launches: building a competency matrix, running the first QA calibration sessions, mentoring a hiring cohort through onboarding, sitting on a vendor's proof-of-concept calls. Money and hiring throughput are constraints on what the roadmap can afford and how fast it can staff up. The third constraint is what actually breaks when two initiatives get scheduled in the same quarter because nothing on the budget spreadsheet flagged that they'd draw on the same three people's time.

Manager's Note *Before scheduling anything, write down the names — not roles, actual names — of the three or four people whose hours every major initiative on the draft roadmap will consume. If the same two or three names show up next to more than one initiative in the same quarter, that's the collision waiting to happen, and it's visible on paper weeks before it shows up as a rushed onboarding cohort or a PoC nobody had time to evaluate properly.*

1.2 What this part assumes is already sitting on the table

[SENIOR MANAGER] This part assumes two artifacts already exist before roadmap-building starts: a maturity assessment scored against Part 30's four axes, naming a current level and a target level per axis, and a budget model built or refreshed per Part 5 and Part 20, giving a real headcount number and a real dollar figure to work from. A manager without either of those isn't ready to sequence yet — sequencing an unscored gap or an unbudgeted number just produces a plan that looks organized and isn't defensible the first time someone asks where a figure came from.

2. The four levers and their sequencing constraints

[CONCEPT] Every investment a SOC roadmap sequences falls into one of four levers — hiring, tooling, training, and process — and each has a different lead time, a different dependency chain, and a different natural forcing function: the thing that makes the initiative actually happen on schedule rather than quietly sliding, unremarked, into next quarter.

The table below compares the four levers on lead time, what has to exist before each one can start, and whether anything besides the roadmap itself forces it to happen on time.

Lever	Typical lead time to real impact	What it depends on first	Natural forcing function
Hiring	60–150 days time-to-fill (Part 5, Part 7) plus a 60–90 day ramp (Part 11) before full productivity	A defensible headcount number (Part 5) and a job description that isn't inherited unedited from last year (Part 7)	Strong — an open requisition is visible, and it either fills or it doesn't
Tooling	Weeks for a PoC, months for a full migration and retraining cycle (Part 21)	A clear enough process to know what the new tool needs to encode, not just what it needs to replace (§2.4)	Strong — a purchase order and a contract renewal date force a decision point
Training	Days to design a module; ongoing to deliver a program	A competency matrix (Part 10) defining what "ready" actually looks like — training with no defined bar teaches to nothing	Weak — nothing forces a training investment to happen on schedule besides the roadmap itself
Process	Weeks of workshop and calibration time; near-zero hard dollar cost	Willingness to spend senior analysts' hours, which competes directly with queue time	Weakest — no PO, no contract date, no visible open req; process work is the first thing quietly dropped under pressure

[SENIOR MANAGER] The bottom two rows are the ones a roadmap most often gets wrong, and for the same underlying reason: hiring and tooling both come with a forcing function baked in — an open req looks unfinished until it's filled, and a signed contract has a renewal date attached whether anyone revisits it or not. Training and process have no such mechanism. A training budget line and a QA-calibration line item can both sit unspent at year-end with nobody having to explain why, precisely because neither one has a visible, external trigger demanding action. That asymmetry is the single biggest reason process investments are the ones most often postponed indefinitely under queue pressure — not because managers don't value them, but because nothing forces the question the way an unfilled seat does.

2.1 Hiring: the slowest-moving lever, so it goes first

[HR/PEOPLE] Because time-to-fill plus ramp time can run five to eight months end to end for a single hire, any hiring decision meant to change capacity within the roadmap's 12-month window has to be locked in during the first quarter. A req approved in month nine of the roadmap doesn't meaningfully change this roadmap's capacity picture at all — it changes next year's, and treating it as this year's fix is the single most common way a roadmap's headcount line looks resolved on paper and stays unresolved on the floor.

2.2 Tooling: the lever with the longest tail risk

[VENDOR/PROCUREMENT] Part 21 — Tooling Procurement & Platform Strategy owns the PoC evaluation process, the reference-customer calls, and the total-cost-of-ownership modeling a platform decision needs; this part's only job is where that process lands on the calendar relative to everything else competing for the same evaluators' time. The technical effort behind an actual platform migration — what it costs in engineering hours to re-normalize and re-validate detections against a new backend —

is Detection Engineering Handbook V2, Part 6 — Normalisation's territory, cited there rather than re-estimated here; this part only needs to know that a migration's tail (the months after signature, not the PoC itself) is long enough that starting it late in a 12-month roadmap all but guarantees it finishes inside next year's, not this one.

2.3 Training: the fastest lever to start, with a hidden ceiling

[HR/PEOPLE] Training is the cheapest and quickest lever to greenlight — a curriculum can be drafted in days — which makes it tempting to schedule generously across every quarter. The ceiling that actually limits it isn't budget; it's the same senior-analyst bandwidth named in §1.1. A structured onboarding cohort (Part 11), a cross-training rotation (Part 12), and a coaching cadence tied to performance management (Part 16) all draw on a small, overlapping pool of people qualified to mentor, and stacking all three in the same quarter competes for that pool regardless of how much money is behind each line item.

2.4 Process: the lever with no natural forcing function

[SENIOR MANAGER] A competency matrix (Part 10), a QA sampling and calibration program (Part 15), and a tiering redesign (Part 3) are, in dollar terms, the cheapest items on most roadmaps — mostly workshop time and calibration sessions, not headcount or a purchase order. They are also the items most likely to get quietly dropped the first time the queue runs hot, precisely because nothing forces them the way an open req or a contract renewal date does. The practical fix is naming a hard date for each process deliverable in the roadmap itself and treating a missed process-track date with the same seriousness as a missed hiring or tooling deadline — because absent that discipline, the roadmap itself is the only forcing function process investments will ever have.

3. Reading a maturity gap into a dated backlog

3.1 From Part 30's four axes to this part's four levers

[CONCEPT] Part 30's four maturity axes and this part's four levers line up closely but not perfectly, and the gap between them is worth naming before scheduling anything against it.

The table below maps each Part 30 maturity axis to the lever that typically closes its gap, and flags where the mapping isn't as clean as it looks.

Part 30 maturity axis	Primary matching lever	Where the mapping breaks
Staffing maturity	Hiring	Close to 1:1
Process maturity	Process	Close to 1:1
Tooling maturity	Tooling	Retraining cost on a new platform lands partly on the training lever, not tooling alone
Training-program maturity	Training	A training-program gap is sometimes actually a process gap in disguise — there's no competency matrix to train against yet, so no curriculum can define "ready"

[SENIOR MANAGER] That last row is worth dwelling on, because it's the same "looks like X, is actually Y" diagnostic error this book returns to under different names in different parts — Part 9's staffing-problem-versus-detection-quality-problem diagnosis for a growing queue, Part 16's skill-gap-versus-process-gap diagnosis for a recurring performance miss. A training-program maturity score stuck at Level 2 often isn't a training-delivery problem at all; it's that nobody has defined, in observable terms, what a trained analyst is supposed to be able to do — which is Part 10's competency matrix, not Part 12's training-delivery mechanics. Scheduling a bigger training budget against that gap without first scheduling the competency matrix that should precede it spends money curing the wrong axis.

3.2 Sizing the gap before scheduling it

[SENIOR MANAGER] Once the current and target level are known per axis, the roadmap's real job starts: translating each axis's gap into named initiatives with a lever, a lead time, a dependency, and a dollar figure attached — not a restated version of the maturity model's own language. "Improve staffing maturity from Level 1 to Level 3" is a scoring outcome, not a plan; "re-run the headcount model against segmented volume, rewrite the job descriptions, and open three specific requisitions by end of Q1" is a plan a hiring pipeline can actually execute against. §5 works a full worked example of that translation end to end.

4. Tying the roadmap to the budget model

4.1 Budget categories as spending lanes, not a wish list

[EXECUTIVE] A roadmap that isn't mapped to Part 20's actual budget categories — headcount, tooling, training, facilities, vendor services — is a wish list, not a plan a CFO will engage with. Every initiative named in §3.2 needs a category, a dollar figure, and a quarter it lands in before it's a defensible line in next year's budget defense, not just a good idea a manager believes in. The discipline that makes a roadmap survive a budget review is the same discipline Part 20 already teaches for a single number: state the claim, then the figure, then which category absorbs it.

4.2 Cash-flow timing: why a Q1 dollar and a Q4 dollar aren't the same dollar

[EXECUTIVE] A three-headcount addition approved in January and staffed by March costs roughly nine months of that year's budget; the same three heads approved in September cost roughly three months of this year's budget and the full annualized figure next year. A roadmap that reports only the annualized run-rate cost of its hiring line, without separating this-year's actual cash impact from next year's, will either look more expensive than it actually is this cycle or blow past budget the following cycle when the full run-rate finally lands — either error erodes the credibility of the next roadmap's numbers.

The table below shows how one composite roadmap's four spending lanes actually land across a fiscal year's four quarters, illustrating the timing gap between an annualized figure and the cash a given quarter actually spends.

CONCEPTUAL SAMPLE — illustrative figures built for this part's worked example in §5; not sourced benchmark data.

Budget category (Part 20)	Q1 spend	Q2 spend	Q3 spend	Q4 spend	Note
Headcount	\$0	\$40,000	\$95,000	\$95,000	Reqs open in Q1–Q2; cash impact starts only once hires land, per \$2.1
Tooling/licensing	\$0	\$0	\$10,000	\$25,000	PoC and evaluation costs land in Q3–Q4; migration execution deferred past the window
Training	\$8,000	\$14,000	\$16,000	\$8,000	Curriculum design and rotation stipends, staggered against the same senior-analyst bandwidth as process
Vendor/MSSP governance	\$5,000	\$5,000	\$5,000	\$5,000	Standing QBR and audit-clause cadence (Part 22), flat across the year, not new spend

[EXECUTIVE] The pattern worth pointing out to a budget owner directly: the headcount row totals \$230,000 in actual cash for the year, not the \$405,000 annualized run-rate the three new heads will carry once fully staffed (\$5.2) — Q1 shows zero cash impact because nobody has started yet, and even Q3 and Q4 only reflect partial-year salary for hires who landed mid-year. Reporting the \$405,000 annualized figure alone, without this quarter-by-quarter breakdown, overstates this year's actual ask by roughly \$175,000 and sets up next year's budget conversation to look like an unexplained jump when the same three heads finally show up at their full annualized cost. Part 24 — Executive & Board Reporting owns how to frame that distinction for a board audience; this part's job is making sure the underlying numbers are already split correctly before they reach that conversation.

5. Worked example: a reactive, understaffed SOC's 12-month roadmap

COMPOSITE CASE EXAMPLE CASE-3301 — merges patterns from more than one small in-house SOC's first structured roadmap cycle; no single organization is identifiable, and every figure below is illustrative.

5.1 The starting point

[SENIOR MANAGER] An 8-analyst, 1-manager in-house SOC runs 24/7 coverage across three shifts with no team-lead layer and no enforced distinction between its nominal L1 and L2 titles — everyone works everything. Three of the eight seats turned over in the trailing 12 months, a 37% attrition rate that keeps hiring permanently reactive: requisitions open only after someone resigns, average time-to-fill runs 120 days end to end, and nobody has run a structured onboarding program because there's rarely a quiet enough month to build one. There is no competency matrix, no QA sampling program, and no written promotion criteria. The SIEM has been in place for six years with no SOAR layer; a burst-capacity clause exists in the SOC's MSSP overflow contract, but nobody has run the governance cadence Part 22 describes since it was signed. Total annual SOC budget is \$1.8 million: roughly \$1.3 million headcount, \$310,000 tooling and licensing, \$140,000 MSSP overflow, and \$12,000 training.

[SENIOR MANAGER] A Part 30-style maturity assessment scores the SOC at Level 1 (Reactive) on staffing maturity, Level 1 on process maturity, Level 2 (Developing) on training-program maturity, and Level 2 on tooling maturity. The reactive-hiring, no-competency-matrix, no-QA-program picture is not four independent gaps; it's a single mechanism feeding itself — no process maturity means no defensible way to tell who's ready for more responsibility, which means no real career progression, which is a

documented driver of the poaching pattern Part 18 covers, which keeps attrition high, which keeps hiring reactive, which leaves no quiet quarter to build the process work that would break the cycle. A roadmap for this SOC has to name that cycle explicitly, not just list four unrelated maturity scores.

5.2 The roadmap, quarter by quarter

[SENIOR MANAGER] The roadmap sequences four tracks against the constraint named in §1.1: the same three senior analysts are the only people currently qualified to build the competency matrix, facilitate QA calibration, and mentor a hiring cohort, so those three obligations are staggered rather than launched together, even though hiring and tooling — funded from different budget lines — are allowed to run on their own separate timeline without waiting on the process track.

Figure 33.1 lays out the sequence below as a 12-month timeline. Every task names the part of this book it draws its method from, so the roadmap reads as an execution plan against material the manager already has, not a list of good intentions.

```
gantt
    title CASE-3301 -- 12-month roadmap, reactive toward target maturity (illustrative)
    dateFormat YYYY-MM-DD
    axisFormat %b
    section Process
        Build 4-axis competency matrix (Part 10)           :p1, 2027-01-04, 40d
        Redesign tiering & escalation criteria (Part 3)      :p2, after p1, 30d
        Stand up QA sampling & calibration (Part 15)        :p3, after p2, 70d
    section Hiring
        Re-run headcount model on segmented volume (Part 5) :h1, 2027-01-04, 25d
        Rewrite L1/L2/team-lead job descriptions (Part 7)   :h2, after h1, 20d
        Open 3 reqs -- 2 analyst, 1 team lead              :h3, after h2, 90d
        Structured onboarding cohort begins (Part 11)       :h4, after h3, 90d
    section Training
        Design onboarding curriculum against matrix (Part 12) :t1, after p1, 30d
        First cross-training rotation cohort                :t2, after p3, 45d
    section Tooling
        SIEM/SOAR PoC evaluation (Part 21)                 :o1, 2027-09-01, 75d
        Vendor selection & contract negotiation             :o2, after o1, 45d
        Migration execution begins, carries into next year's roadmap :o3, after o2, 10d
```

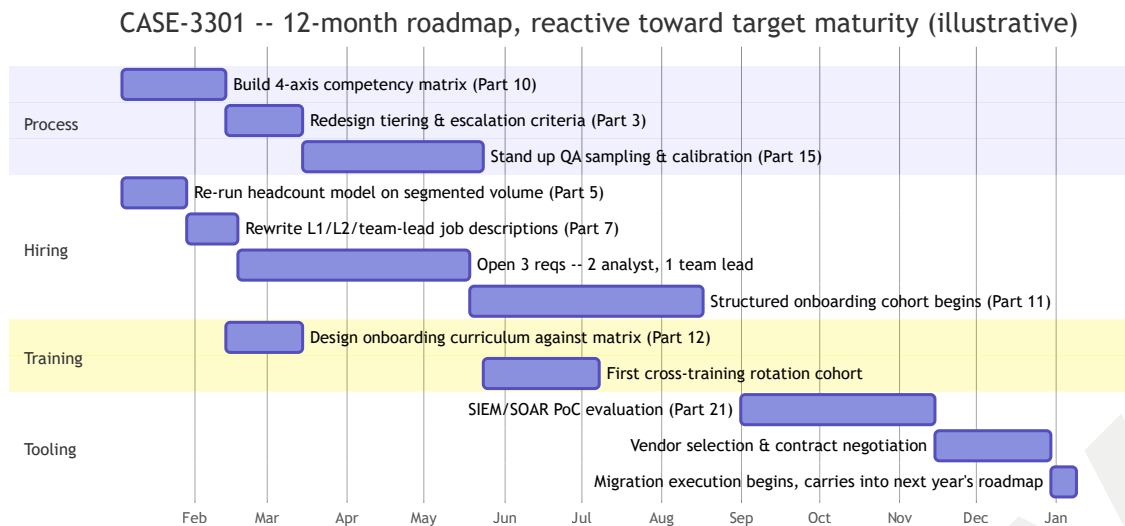


Figure 33.1 — CASE-3301's 12-month roadmap, sequenced across process, hiring, training, and tooling. CONCEPTUAL. FIG-3301. Illustrates the sequencing logic in §5.2 — the process and hiring tracks run in parallel from month one because neither draws on the same constrained senior-analyst hours the other needs at that point in the sequence, while the tooling PoC is deliberately held until month nine, after the onboarding cohort's mentorship demand has passed, to avoid the exact resource collision CASE-3302 in §6.1 illustrates. It is a structural model of one composite roadmap, not a capture of any single real organization's actual planning artifact. Rendered as a static SVG Gantt-style timeline from the Mermaid source above; supports §5.2's claim that deliberately staggering the tooling PoC behind the hiring wave's heaviest mentorship demand — rather than running both in the same quarter because they draw on different budget lines — is what keeps this composite roadmap from repeating CASE-3302's collision.

[SENIOR MANAGER] Total incremental spend against the composite's existing \$1.8 million budget runs to roughly \$490,000 in annualized run-rate cost — about \$405,000 for the three new heads once fully staffed, \$46,000 in added training spend, and roughly \$35,000 for the tooling PoC itself, with full migration cost deferred to next year's roadmap. Per §4.2's cash-timing logic, actual year-one cash impact runs closer to \$310,000, because the hires land mid-year and the tooling spend in this window is limited to evaluation, not migration.

5.3 What actually moved, and what didn't

[SENIOR MANAGER] By month 12, staffing maturity moves from Level 1 to Level 3: the headcount model is rebuilt against real segmented volume, the two analyst reqs and the team-lead req all filled (the team-lead req took until month nine to close, later than planned, and the onboarding cohort accordingly ran later than the Figure 33.1 timeline's optimistic case), and time-to-fill for the year's later reqs drops from 120 days to 68 once the rewritten job descriptions and a real pipeline are in place — real progress, short of the roughly 55-day figure a fully mature pipeline can eventually reach, and worth naming as such rather than rounding up. Process maturity also moves to Level 3: the competency matrix is live, QA calibration runs monthly, and the tiering redesign has a real escalation criterion behind it instead of an informal norm. Training-program maturity reaches Level 3 on the strength of the structured onboarding curriculum and the first completed cross-training rotation. Attrition over the 12

months drops from 37% to 21% — still above a healthy target, but a real, attributable improvement, not noise.

[SENIOR MANAGER] Tooling maturity is the honest exception: the PoC and vendor selection complete on schedule, but full migration and retraining — the resource-intensive part — only begins in the roadmap's final weeks and explicitly carries into next year's cycle, landing the axis around Level 2 rather than the Level 3 the original assessment targeted. A roadmap that reported this as a completed target-maturity axis would be lying with a maturity score; a roadmap that names the slip and schedules the remainder against next year's plan is doing exactly what a living document is supposed to do.

Field Test Setup: A draft roadmap exists with initiatives assigned to named quarters, before it's presented for budget approval. **Action:** For every initiative on the draft, write down which specific named person's hours it draws on — not a role, an actual name — and total those hours per person per quarter against that person's realistic available time after Part 5's shrinkage deductions and their existing queue or team-lead duties. **Expected result:** Any quarter where one named person's total claimed hours exceed roughly 70–80% of their genuinely available time flags for staggering before the roadmap ships — not after a hiring wave and a tooling PoC collide on the same three senior analysts, the way [CASE-3302](#) in §6.1 shows happening in practice.

6. Common sequencing failures

6.1 Overloading one quarter with two initiatives that need the same people

[SENIOR MANAGER] Two budget lines funding two different levers can still collide on the same people, and that collision is invisible until it's already happened, unless someone names the constrained resource before scheduling starts.

COMPOSITE CASE EXAMPLE [CASE-3302](#) — merges patterns from more than one mid-market SOC's first attempt at a formal roadmap; no single organization is identifiable, and all figures are illustrative.

Management Autopsy — "fund the SIEM PoC and the year's biggest hiring wave from different budget lines, so schedule them together"

The decision: A 14-analyst SOC's maturity assessment names tooling maturity (Level 1) and staffing maturity (Level 2) as the year's two biggest gaps. Leadership approves launching the SIEM/SOAR PoC and opening five new requisitions in the same quarter, reasoning that tooling spend comes from a capital budget line and headcount comes from an operating budget line, so nothing on the budget spreadsheet suggested a conflict.

Why it seemed reasonable: Both were the two largest, most visible gaps from the assessment, and showing visible movement on both in the same quarter looked like a strong opening quarter to report upward — nobody at budget approval asked whose hours either initiative would actually consume.

How it failed: The three senior analysts qualified to mentor new hires through Part 11's structured onboarding are the same three the vendor's PoC needs for daily 90-minute evaluation calls across six weeks. Onboarding reverts to informal "shadow whoever has a free minute," and the PoC evaluation itself gets compressed into fewer, rushed sessions because the same three people are stretched across both obligations. Of the five new hires, two require a formal Performance Improvement Plan within six months — a 40% rate. Part 8's own case data ties a well-run structured interview-and-assessment process to a six-month PIP rate closer to 15%; a 40% rate here means the collapsed onboarding is doing real additional damage on top of whatever the interview loop already screened for, not a wash against it. The SIEM/SOAR vendor selected under the rushed evaluation requires \$165,000 of unplanned professional-services reconfiguration four months after signature, once gaps the compressed PoC missed surface in production.

The fix: Name the constrained resource before scheduling anything — not "capital budget" versus "operating budget," but the specific senior-analyst hours both initiatives actually draw on — and stagger any two initiatives that share those named people by at least one full quarter, regardless of which budget line funds each one. CASE-3301's §5.2 sequencing deliberately holds its own tooling PoC until month nine, after the onboarding cohort's heaviest mentorship demand has passed, for exactly this reason.

6.2 Sequencing hiring ahead of the process work it silently depends on

[SENIOR MANAGER] A subtler version of §6.1's mistake: opening reqs and building a competency matrix in parallel looks efficient, but a hiring cohort that ramps up before the matrix exists has nothing observable to ramp toward — Part 10's own point about competency assessment collapsing into impression without dedicated evidence per axis applies just as much to a brand-new hire's ramp plan as it does to a promotion nomination. CASE-3301's roadmap avoids this by starting the competency matrix (p1) in the same month as the headcount remodel (h1), so the matrix exists before the onboarding cohort (h4) that needs it actually begins.

6.3 Letting the roadmap go stale after the planning offsite ends

[SENIOR MANAGER] A roadmap built well once and never re-checked against its own resource assumptions doesn't stay accurate by default — it drifts silently toward whichever track already has a forcing function, per §2's table, and nobody decides that drift on purpose.

COMPOSITE CASE EXAMPLE **CASE-3303** — merges patterns from more than one SOC whose first roadmap wasn't revisited between annual planning cycles; no single organization is identifiable, and all figures are illustrative.

Management Autopsy — "build the roadmap once at the annual offsite, review it again at year-end"

The decision: An 18-analyst SOC builds a genuinely well-sequenced 12-month roadmap in January, following roughly the structure this part recommends, and schedules no formal review of it before the following January's planning offsite.

Why it seemed reasonable: The roadmap was carefully built, sequencing looked sound, and a mid-year review felt like it would just repeat the same conversation the offsite had already had — a reasonable-sounding way to avoid what looked like redundant process overhead.

How it failed: In month five, a regional MSSP poaching wave — the same 15% to 20% pay-premium pattern Part 18 documents — pulls three senior analysts in six weeks. The roadmap's Q3 hiring plan (two reqs earmarked for growth) quietly gets repurposed to backfill instead, which nobody formally revises in the roadmap document, because there's no scheduled moment that asks the question. Those same three senior analysts were also the people the roadmap's Q3 process track — the QA calibration rollout — was built around; the rollout slips to Q4, then silently to "next year," without anyone deciding that on purpose at any point. By the December review, 90% of the roadmap's hiring-track items have shipped (because open reqs are visible and get reactive attention by default) against only 40% of its process-track items — a gap nobody noticed the size of at any single checkpoint, because no checkpoint ever asked the process-track question directly.

The fix: A written quarterly checkpoint (§7.1) that explicitly re-asks whether the resource assumptions behind the next quarter's items still hold, not a status report on what already shipped. A roadmap with no scheduled moment to notice that its own assumptions broke will drift exactly the way **CASE-3303**'s did, and it will drift silently, in the direction of whatever track already has a visible forcing function — which per §2's table is never the process track.

7. Governing the roadmap after it ships

7.1 The quarterly re-sequencing checkpoint

[SENIOR MANAGER] A roadmap's quarterly review should not be a status report on what shipped — that's a useful input, but it's not the point of the meeting. The point is re-asking, explicitly and in

writing, whether the resource assumptions behind the next quarter's remaining items still hold: has a named hire fallen through, has a vendor's timeline slipped, has an attrition spike pulled the same senior people the process track depends on, the way it did in [CASE-3303](#). A checkpoint that only asks "what's done" will faithfully report a roadmap quietly repurposing itself out of existence, one unremarked substitution at a time, exactly as §6.3 describes.

Field Test Setup: A roadmap with quarterly milestones exists and has completed at least one full quarter. **Action:** At the quarterly review, before discussing what shipped, ask each track owner one specific question in writing: "has anything changed about who was supposed to do next quarter's work, or what they were supposed to have finished first?" Require a yes/no answer with a name attached, not a general status update. **Expected result:** If the answer is yes for any track, the roadmap gets a visible, dated re-sequencing decision before the meeting ends — not a note to "keep an eye on it." A checkpoint that produces zero re-sequencing decisions across an entire year is either evidence of an unusually stable year or evidence that the checkpoint isn't actually asking the question — worth checking which, rather than assuming the former.

7.2 Who owns re-sequencing: a governance RACI

[SENIOR MANAGER] Re-sequencing fails for the same reason any standing cadence fails, per this book's own recurring pattern from Part 22's contract-governance cadence: a task with no named owner quietly stops happening the moment the person who used to do it informally changes roles or gets pulled onto something more urgent.

The table below assigns Responsible, Accountable, Consulted, or Informed status for each roadmap-governance activity across the roles typically involved (CONCEPTUAL SAMPLE — illustrative role assignment; substitute your own organization's actual titles).

Governance activity	SOC manager	Team leads	HR/talent partner	Finance/budget owner
Quarterly re-sequencing checkpoint (§7.1)	R/A	R	C	I
Resource-collision stress test before shipping (Field Test, §5.3)	R/A	R	C	—
Mid-year budget-line reforecast (Part 20)	C	I	—	R/A
Escalating a slipped axis to a full re-scope (§7.3)	R/A	C	C	C

[SENIOR MANAGER] Finance/budget owner is Accountable for exactly one row — the mid-year reforecast — and Informed on the rest, which is deliberate: re-sequencing which quarter an initiative lands in is the SOC manager's call, not the budget owner's, as long as the total annual figure Part 20 defends doesn't change. The moment a re-sequencing decision does change that total, it stops being this table's §7.1 conversation and becomes a §4.2-style reforecast conversation instead.

7.3 Escalation triggers: when a slip needs a re-scope, not just a note

[SENIOR MANAGER]

Most quarterly slips are absorbable with a note and a revised date — a req that takes three extra weeks to fill doesn't need the whole roadmap rebuilt. Two conditions justify a full re-scope rather than a routine adjustment: a slip on the hiring or tooling track that pushes an item past the 12-month window entirely, the way tooling did in CASE-3301's §5.3, which needs to be explicitly handed to next year's roadmap rather than left as an open item on this one; and a slip that reveals the original resource-collision assumption in §1.1 was wrong for reasons beyond one bad quarter — for instance, discovering the same three senior analysts are load-bearing for four different tracks, not two, which is a structural finding about the team, not a scheduling one, and belongs in the next maturity assessment's staffing-axis conversation as much as in this roadmap.

8. Where this leaves the reader

[CONCEPT]

This part closes the book's arc, and the honest summary of what it adds is narrow on purpose: every other part in this book taught how to build one program well. This part is the discipline of not building four of them badly at once by scheduling them as if they didn't share the same finite calendar, the same finite budget, and the same handful of senior people's hours. CASE-3301's roadmap doesn't hit every target-maturity axis inside 12 months, and that's the honest case for this part's method, not against it — a roadmap that claims to close every gap on schedule either got lucky or isn't telling the whole story, and a manager who reports a slipped axis with a dated plan to close it next cycle is doing the job this part actually asks for.

What Would Change My Mind This part treats staggering initiatives that share the same named senior people's hours as the single highest-value sequencing rule in a 12-month roadmap — ahead of strict maturity-gap prioritization or strict budget-category ordering. If a meaningful share of SOC's running formal roadmaps found that resource collisions of the kind CASE-3302 describes were rare or easily absorbed without the deliberate staggering this part recommends — because, say, a deep enough bench of qualified mentors and evaluators already existed — that would weaken the case for treating this as the roadmap's central constraint rather than one factor among several, and would shift the emphasis back toward straightforward maturity-gap prioritization for organizations with that kind of depth.

Cross-references

Within this book: Assumes a completed maturity assessment from Part 30 — SOC Maturity Models and a headcount/budget model from Part 5 — Headcount & Capacity Modeling and Part 20 — Building & Defending the SOC Budget, treating all three as inputs rather than re-deriving them. Sequences the hiring lever using Part 7 — Hiring & Sourcing Analysts and Part 11 — Onboarding & Ramp-Up Programs, the tooling lever using Part 21 — Tooling Procurement & Platform Strategy, the training lever using Part 10 — Competency Models & Skills Matrices and Part 12 — Ongoing Training & Skill Development, and the process lever using Part 3 — Tiering Models & Beyond and Part 15 — Quality Assurance Programs. Draws its attrition and poaching figures from Part 18 — Attrition & Retention, its resource-collision fix from the same governance-cadence discipline built in Part 22 — MSSP &

Managed-Service Contract Management, and its budget-timing framing from Part 24 — Executive & Board Reporting.

SOC Playbook Handbook: Part 32 — Metrics (the handle-time, MTTR, and volume definitions this part's own budget and headcount inputs treat as already fixed, per Part 5's own citation of the same part); Part 30 — Automation and SOAR (the automation/human-review boundary a tooling-track SOAR rollout in a roadmap should already have resolved before scheduling it).

Detection Engineering Handbook V2: Part 6 — Normalisation (the technical engineering effort behind a platform migration's tail, cited rather than re-estimated in §2.2).

neshbooy

APPENDICES

Appendix A1 — Staffing, Shift-Design & Capacity-Model Templates

Why this appendix exists

[CONCEPT] This appendix holds the fillable worksheets that Part 5 — Headcount & Capacity Modeling and Part 6 — Shift Pattern & Coverage Design build formulas for but don't reproduce as blank, reusable artifacts. Both parts cite these templates by permanent ID rather than re-deriving their arithmetic — this is the file those citations resolve to. Four templates are collected here: the headcount calculator worksheet ([TMPL-0001](#)), the shift-pattern templates covering fixed, rotating, follow-the-sun, and on-call/bridge coverage ([TMPL-0601](#)), the shift-handoff content checklist ([TMPL-0602](#)), and the on-call rotation calculator ([TMPL-0603](#)).

Each template is a companion to a specific section of Part 5 or Part 6, and each assumes the reader has already read the reasoning behind it — this appendix does not re-argue why shrinkage matters, why a flat daily average misstates concurrency, or why rotation direction affects error rate. It only gives the reader something to fill in. Where a worksheet needs an example value to show the arithmetic working, that value is drawn from Part 5's [CASE-0501](#) (a composite, illustrative regional-bank SOC) or is marked as a conceptual sample — neither is a benchmark to target, only a demonstration of the math.

Treat every template below as a working document, not a form filled in once and filed away. Part 5 §8 makes this point explicitly for the headcount worksheet — a model built once at budget time and never re-run is already stale by the next quarter — and the same discipline applies to the shift-pattern and on-call templates: a rotation calendar built for a 12-person team doesn't survive three new hires and one resignation without being rebuilt, not just relabeled.

1. Headcount calculator worksheet ([TMPL-0001](#))

[SENIOR MANAGER] A SOC manager building or defending a headcount number at budget time, or re-running the model on Part 5 §8's quarterly cadence, uses this worksheet. It walks the same seven steps Part 5 §3–§7 develops in prose: a coverage-hours floor, volume and handle time segmented by time block, a shrinkage stack, and a stress-test check against a high-percentile day — fill it in top to bottom with your own data.

TEMPLATE – Headcount Calculator Worksheet, permanent ID TMPL-0001

STEP 1 – Coverage-hours floor (Part 5 §3.3)

Coverage target	Weekly hours covered	Coverage-floor seats
Business hours only (5x10)	50	1
Extended hours (2 shifts x 8 hrs)	80	2
24/7, single-seat minimum	168	3 (1 per shift)
24/7, two-person overnight minimum	168	varies – see Part 5 §7.1
Follow-the-sun (regional handoff)	168	varies by region

Your coverage target: _____ Coverage-floor seats: _____

STEP 2 – Volume & handle time by time block (Part 5 §3.1-§3.2; split at minimum by business/overnight)

Time block	Hrs/day in block	Days/yr	Alert vol./block (post-automation)	Avg handle time (min)	Raw analyst-min (vol x time)	Raw analyst-hrs	Concurrent analysts (raw = hrs / block hrs)	Concurrent analysts (round up)
------------	------------------	---------	------------------------------------	-----------------------	------------------------------	-----------------	---	--------------------------------

_____	_____	_____	_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____	_____	_____	_____
_____	_____	_____	_____	_____	_____	_____	_____	_____

Example (CASE-0501, COMPOSITE CASE EXAMPLE): Business hours – 12 hrs, 365 days, 420 tickets, 15 min, 6,300 min, 105.0 hrs, 8.75, 9. Overnight – 12 hrs, 365 days, 180 tickets, 15 min, 2,700 min, 45.0 hrs, 3.75, 4.

STEP 3 – Seat-hours per year

Seat-hrs/yr = SUM over every time block of (Concurrent analysts, rounded up) x (Hrs/day) x (Days/yr)

Your total: _____

Example: $(9 \times 12 \times 365) + (4 \times 12 \times 365) = 39,420 + 17,520 = 56,940$ seat-hrs/yr

STEP 4 – Shrinkage stack (Part 5 §4.1)

Category	Hours/yr	% of 2,080 nominal
PTO & vacation	_____	_____
Sick leave	_____	_____
Company holidays	_____	_____
Training & certification	_____	_____
Coaching, 1:1s, QA calibration	_____	_____
Team meetings & admin overhead	_____	_____
Paid breaks & shift-change overlap	_____	_____
Fleet-wide new-hire ramp deduction	_____	_____
Total shrinkage	_____	_____

Reference point only, not a target: Part 5 §4.1 works a ~32% stack from these same eight categories.

STEP 5 – Productive hours per FTE

Productive hrs/FTE = $2,080 \times (1 - \text{Total shrinkage \%})$

Your value: _____ Example at 32%: $2,080 \times 0.68 = 1,414.4$

STEP 6 – Draft headcount

Headcount = Seat-hrs/yr (Step 3) / Productive hrs/FTE (Step 5), rounded up

Your value: _____ Example: $56,940 / 1,414 = 40.28 \rightarrow 41$ FTE

STEP 7 – Stress-test check before this number ships (Part 5 §7)

[] Re-run Step 2's binding time block at its P90 daily volume, not the mean – does concurrency change?

[] Does any resulting shift staff at exactly 1 analyst (queue-of-one, Part 5 §7.1)? If yes, name the backup path here: _____

[] Is a surge/overflow lever documented for a spike beyond P90 (MSSP overflow – Part 22; pre-approved overtime; cross-trained pool)? _____

[] Re-run date scheduled (quarterly default, Part 5 §8): _____

[SENIOR MANAGER]

This worksheet automates the arithmetic; it does not automate the two judgment calls Part 5 §7 flags as the model's real limits — what percentile of volume to size standing concurrency against, and whether an organization is willing to accept a queue-of-one shift's zero reserve capacity rather than fund a second seat. Both are risk-tolerance decisions a spreadsheet can't make for the manager signing off on it.

2. Shift-pattern templates (TMPL-0601)

[FRONTLINE MANAGER]

A team lead or SOC manager building the actual calendar that delivers Step 6's headcount number uses these four fillable calendars, one per coverage model from Part 6 Table 6.1. Pick the model Part 6 §8's decision framework points to before filling in the corresponding table — these aren't four options to blend, they're four different answers to the same coverage question.

TEMPLATE – Fixed Shift Calendar, permanent ID TMPL-0601 (Part A: fixed, Part 6 §2)

Crew	Shift block	Local start-end	Days-on / days-off pattern	Night differential?
Crew A	Day	07:00-15:00	5 on / 2 off	No
Crew B	Evening	15:00-23:00	5 on / 2 off	Partial
Crew C	Night	23:00-07:00	5 on / 2 off	Yes

CONCEPTUAL SAMPLE – illustrative start/end times; set to your own shift boundaries and Step 1's coverage-floor seat count per crew.

For a two-shift 12-hour model instead (Part 6 §2.2), replace the three rows above with Day (07:00-19:00) and Night (19:00-07:00), each crew on a 2-2-3 pattern across a 14-day cycle.

TEMPLATE – Rotating Shift Ladder, permanent ID TMPL-0601 (Part B: rotating, Part 6 §3)

One crew's path through a slow, forward rotation (day -> evening -> night, per Part 6 §3.2's direction guidance),

with a rest-day buffer inserted at each direction change:

Rotation step	Shift block	Length (days)	Buffer before next block
1	Day	5	–
2	Evening	5	1 rest day (forward: D->E)
3	Night	5	1 rest day (forward: E->N)
4	Off / rest	4	returns to step 1

CONCEPTUAL SAMPLE – offset each additional crew's start point by one step to keep every shift covered every day; do not compress the buffer to save calendar days (Part 6's `CASE-0601` shows what removing it costs in QA-scored error rate).

TEMPLATE – Follow-the-Sun Coverage Map, permanent ID TMPL-0601 (Part C: follow-the-sun, Part 6 §4)

Region	Local business hours	Coverage window (UTC)	Handoff to next region (UTC)	Analysts on shift
----- ----- ----- ----- -----				
Americas	08:00-17:00 local	___-___	___	___
EMEA	08:00-17:00 local	___-___	___	___
APAC	08:00-17:00 local	___-___	___	___

CONCEPTUAL SAMPLE – the 08:00-17:00 local business-hours window is an illustrative placeholder for all three regions; set each region's actual local business hours before deriving its UTC coverage window and handoff time.

Set analysts-on-shift to at least the coverage-floor seats from Step 1 per region; price the regional-overhead line (management, duplicated tooling, cross-region calibration) per Part 6's `CASE-0602` before comparing total cost to a single-site model.

TEMPLATE – On-Call / Bridge Coverage Calendar, permanent ID TMPL-0601 (Part D: on-call, Part 6 §5)

Week starting	Primary on-call	Secondary on-call	Bridge escalation contact(s)
----- ----- ----- -----			
___	___	___	___

See `TMPL-0603` for sizing how many analysts need to be in this rotation and how often each one carries it.

[FRONTLINE MANAGER]

None of these four calendars decides which model to run — that choice is Part 6 §8's decision framework, applied before this template is opened. What this template doesn't automate is the exception handling a real calendar always needs: a named backup when a crew member on a fixed or rotating schedule is out, and a documented rule for who absorbs a shift nobody volunteers for during a holiday window (Part 6 §6).

3. Shift-handoff content checklist (TMPL-0602)

[FRONTLINE MANAGER]

The outgoing analyst on any shift fills this in during the shift's final 30 minutes, before the incoming analyst arrives — not narrated from memory during the overlap itself. The incoming analyst reads it, asks clarifying questions during the overlap window, and signs the acknowledgment field before the outgoing analyst is released. This is a shift-level transfer, distinct from the per-ticket escalation quality owned by SOC Playbook Handbook, Part 27 — Escalation Quality; if an item below needs its own ticket-level escalation, route it there instead of trying to close it out inside this checklist.

TEMPLATE – Shift-Handoff Content Checklist, permanent ID TMPL-0602

Shift date: _____ Outgoing analyst: _____ Incoming analyst: _____
Outgoing shift block: _____ Handoff record completed at (time): _____
Incoming acknowledgment (initials + time): _____

Category	What to record
Detail	Confirmed read by incoming analyst?
-----	-----
-----	-----
Open work in progress	Case/ticket IDs, current status, what's already ruled out, next-action owner
____ [] Y [] N	
Watch items	Anything informally monitored that isn't yet a ticket, and why it's being watched
____ [] Y [] N	
System / tooling status	Known outages, ingestion delays, maintenance windows, workarounds in use
____ [] Y [] N	
What to expect next	Scheduled changes, planned noisy events, known upcoming activity
____ [] Y [] N	

If any row above needs its own escalation, log it on a ticket via the standard escalation path (SOC Playbook Handbook, Part 27 – Escalation Quality) rather than leaving it only in this record.

[FRONTLINE MANAGER]

This checklist's main limitation is the same one Part 6 §7.2 names directly: it only works if the outgoing analyst gets protected time to fill it in before the overlap starts, and if the incoming analyst's acknowledgment is a real, checked step rather than an assumed formality. A checklist filled in after the outgoing analyst has already logged off isn't a handoff — it's a note nobody was required to read.

4. On-call rotation calculator (TMPL-0603)

[SENIOR MANAGER]

A SOC manager sizing a primary/secondary on-call rotation — for the on-call/bridge coverage model itself, or as the surge lever named in Step 7 of TMPL-0001 — uses this calculator to check whether the eligible pool is large enough to keep on-call from becoming a chronic burden rather than an occasional one.

TEMPLATE – On-Call Rotation Calculator, permanent ID TMPL-0603

INPUTS

Input	Your value
Analysts eligible for on-call rotation	_____
Rotation length (days per on-call turn)	_____ (7 recommended – Part 6 §5)
Primary ack window (minutes)	_____ (10-15 typical – Part 6 §5)
Secondary ack window after a missed primary page (min)	_____
Avg. off-hours pages/week (trailing 90 days)	_____
On-call stipend per week (\$)	_____

CALCULATION

On-call weeks per analyst per year = 52 / (Analysts eligible for on-call rotation)

Your value: _____

CONCEPTUAL SAMPLE: 8 eligible analysts -> 52 / 8 = 6.5 weeks/analyst/year (about one week every 8 weeks)

SUSTAINABILITY CHECK

[] Is on-call weeks/analyst/year at or below roughly one week in 6 (a rough starting heuristic for this worksheet, not a figure Part 17 specifies or a validated threshold – calibrate against your own team's actual complaint/attrition pattern)?

[] If below that line, is a stipend (not just a per-page bonus) budgeted to compensate the lost personal time (Part 18)?

[] Does the secondary's ack window plus the primary's ack window still land inside the response-time target the alert's severity requires (SOC Playbook Handbook, Part 29 – Playbook Severity Model)?

ROSTER

Week starting	Primary on-call	Secondary on-call	Stipend paid (Y/N)	Bridge contacts if a major incident is declared
_____	_____	_____	_____	_____
_____	_____	_____	_____	_____

[SENIOR MANAGER]

The sustainability check's threshold is a starting heuristic, not a validated number this book has tested against its own data — a team with a genuinely low off-hours page volume may tolerate a tighter rotation than a team fielding frequent 2 a.m. pages on the same cadence. What this calculator doesn't decide is whether the pain of a tight rotation is better solved by hiring into the on-call pool or by reducing off-hours page volume at the source — that's the same staffing-versus-detection-quality judgment call Part 9 walks through for the standing queue.

Cross-references. This book: Part 5 — Headcount & Capacity Modeling (source of [TMPL-0001](#)'s formulas and [CASE-0501](#)'s worked figures); Part 6 — Shift Pattern & Coverage Design (source of [TMPL-0601](#)'s four coverage models, [TMPL-0602](#)'s handoff categories, and [TMPL-0603](#)'s on-call structure); Part 9 — Queue Health & Workload Management (the staffing-vs.-detection-quality judgment call referenced in §4); Part 17 — Burnout, Fatigue & Wellbeing (rotation-direction and on-call-frequency fatigue evidence behind §2 and §4's checks); Part 18 — Attrition & Retention (on-call compensation as a retention lever); Part 22 — MSSP & Managed-Service Contract Management (the surge/overflow lever in [TMPL-0001](#) Step 7 and [TMPL-0601](#) Part C's follow-the-sun cost comparison). Other volumes: SOC Playbook Handbook, Part 27 —

Escalation Quality (per-ticket hand-off, distinct from TMPL-0602's shift-level transfer) and Part 29 — Playbook Severity Model (per-alert response timing, distinct from TMPL-0603's ack-window fields).

neshboy

Appendix A2 — Hiring, Interviewing & Competency Templates

Why this appendix exists

[CONCEPT] This appendix is the fillable companion to three chapters: Part 7 — Hiring & Sourcing Analysts, Part 8 — Interviewing & Technical Assessment Design, and Part 10 — Competency Models & Skills Matrices. Those chapters make the case for why a leveled job description, a calibrated interview loop, and a four-axis competency matrix each beat their informal alternative — tenure-only promotion, an unstructured "just have a conversation" interview, a kitchen-sink requirements list. This appendix does not repeat that argument. It carries the actual artifact each chapter promised: the leveled job-description skeleton ([TMPL-0701](#)), the structured-interview scorecard and practical-assessment rubrics ([TMPL-0801](#)), and the tiered competency-matrix worksheet ([TMPL-1001](#)).

[HR/PEOPLE] Use this appendix the way the chapters intend: fill in one skeleton, not three unrelated documents, when a req opens against a tier Part 3 already defines; run the scorecard and rubrics live during an actual interview loop, one per candidate per stage; and revisit the matrix worksheet at nomination for a tier change or roughly annually, never as a stand-in for a monthly performance review. None of the three templates decides the judgment call it sits on top of — a hiring manager still has to draw the must-have/strongly-preferred line, a panel still has to write its own pass-bar language, and a reviewer still has to write the actual anchor text for each tier. Each template below states its specific unautomated judgment call where it applies.

[SENIOR MANAGER] Sample values inside any block below are marked CONCEPTUAL SAMPLE — illustrative, not sourced benchmark data or a real requisition — per this book's evidence-classification standard. Substitute your own tier definitions, rubric language, and pass bars before using a template on a real candidate or a real nomination.

1. Leveled job-description template ([TMPL-0701](#))

[HR/PEOPLE] A hiring manager or team lead fills this in once per requisition, against a tier Part 3's model already defines, before a req is posted to any sourcing channel from Part 7. It is one skeleton with a level parameter, not a separate document per tier — the level line changes; the structure underneath it doesn't.

TEMPLATE – Leveled analyst job description skeleton, permanent ID TMPL-0701

ROLE: SOC Analyst – [Tier: L1 / L2 / L3]

SHIFT COMMITMENT: [State the actual pattern from Part 6 – do not write "flexible hours" if the role is a fixed night rotation]

SCOPE OF WORK (what this person owns, not what the team owns):

- <!-- state the specific alert categories, ticket types, or investigation depth this level handles unsupervised, not "monitors security events" -->

DECISION AUTHORITY:

- <!-- what this level can close/escalate/contain without sign-off, and what always requires a hand-off -->

MUST HAVE (screens candidates out – keep this list to three items or fewer):

- <!-- the two or three things a person genuinely cannot do the job without, stated as a capability, not a certification: "can read raw firewall logs and reconstruct a session," not "CCNA required" -->

STRONGLY PREFERRED (does not screen out; used to break ties):

- <!-- everything else that would be nice -->

EQUIVALENT-EXPERIENCE CLAUSE:

- <!-- name the concrete non-traditional background that satisfies a must-have item, per Part 7 §6 – "or two years in a role requiring structured investigation under time pressure," not a bare "equivalent experience considered" -->

GROWTH PATH: What "ready for the next tier" looks like, one sentence, pointing to Part 13's ladder rather than restating it.

ON-CALL / ESCALATION EXPECTATION: <!-- the actual frequency, stated as a number ("one week in six"), not "occasional on-call support may be required" -->

[HR/PEOPLE] Filled example (CONCEPTUAL SAMPLE — illustrative values, not a real requisition):

ROLE: SOC Analyst – L2

SHIFT COMMITMENT: Rotating 12-hour day shift, one weekend in three

MUST HAVE:

- Forms and tests a hypothesis across multiple log sources with no playbook
- Decides when to escalate vs. close an ambiguous severity call, unsupervised
- Comfortable building an ad hoc cross-tool query with no saved-search template

EQUIVALENT-EXPERIENCE CLAUSE:

- Two years of internal audit or compliance investigation work reconstructing an anomaly from disparate logs satisfies the first must-have item above

ON-CALL: One week in six, secondary rotation

[HR/PEOPLE] The template does not decide where the must-have/strongly-preferred line sits for a given tier — that judgment belongs to whoever owns the competency matrix in [TMPL-1001](#) (§3 below), and a copy filled in without that input just relocates the kitchen-sink problem from "must have" to "strongly preferred" instead of fixing it.

2. Structured-interview scorecard and practical-assessment rubrics ([TMPL-0801](#))

[CONCEPT]

Part 8's three-stage loop — screen, practical assessment, panel — needs a scorecard for the panel stage and a dedicated rubric for each of the two practical exercises it describes: the log-triage exercise and the mock-escalation exercise. All three share one design rule from Part 8 §4.1: every line has to name what an interviewer would actually see or hear, not a restated impression. All three also feed the same four axes [TMPL-1001](#) uses later, so a candidate's assessment record and an analyst's ongoing competency record are scored on the same vocabulary from day one.

2.1 Panel scorecard

[HR/PEOPLE]

One interviewer fills in exactly one copy per candidate per interview, independently, before any debrief conversation starts — the fix Part 8 §2.3 names for debrief-order and recency bias.

TEMPLATE – Structured-interview panel scorecard, permanent ID TMPL-0801 (Part A)

CANDIDATE ID: _____ REQ / TARGET TIER: [L1 / L2 / L3] STAGE: Panel & behavioral loop
INTERVIEWER: _____ DATE: _____

For each dimension, score 1-5 and state the specific observed behavior that earned the score – a score with no observed-behavior line is not valid input to the debrief.

Dimension	Score (1-5)	Observed behavior (what was seen/heard)
Technical skill		<!-- e.g., correctly named the missing field without prompting -->
Tool proficiency		<!-- e.g., built the cross-tool query within the time budget -->
Communication		<!-- e.g., explained the disposition in under two minutes, unprompted -->
Judgment		<!-- e.g., named the specific enrichment needed to resolve the ambiguity -->

OVERALL RECOMMENDATION: [Advance / Hold / Reject] SUBMITTED BEFORE DEBRIEF: [Y/N]

[HR/PEOPLE]

This scorecard does not set the pass-bar language for a given tier — that comes from the calibration exercise Part 8 §3.3 describes, run before the first real candidate, and belongs in the "observed behavior" column as a written standard the interviewer checks against, not invented fresh per candidate.

2.2 Log-triage exercise rubric

[FRONTLINE MANAGER]

The assessment designer or SOC manager who built the exercise fills in the scenario-specific detail; the interviewer proctoring it fills in the score, per Part 8 §3.1's rule that the reasoning path is scored, not the final disposition label alone.

TEMPLATE – Log-triage exercise rubric, permanent ID TMPL-0801 (Part B)

CANDIDATE ID: ____ TARGET TIER: ____ TIME USED / BUDGET: ____ / ____ min

Rubric line	Score	Evidence
Reached a defensible disposition		<!-- reasoning stated, not just the label -->
Correctly identified missing/ambiguous evidence		<!-- named the specific gap, not "something's off" -->
If "unable to determine": named the exact enrichment needed		<!-- specific query/source, not "more info" -->
Reasoning reconstructable by a second reviewer		<!-- could a reviewer retrace this without the candidate present -->

PASS BAR FOR THIS TIER (from Part 8 §3.3's calibration table): ____

RESULT: [Clears bar / Does not clear bar]

[FRONTLINE MANAGER]

This rubric does not supply the underlying alert or log excerpt — Part 8 §3.1 requires that material be sanitized real detection content, not an invented puzzle, and sourcing it is a per-SOC task this template assumes is already done.

2.3 Mock-escalation exercise rubric

[FRONTLINE MANAGER]

A team lead or senior analyst playing the receiving Tier 2 role scores this rubric live, against the same standard a real escalation is judged against — SOC Playbook Handbook, Part 27 — Escalation Quality — rather than a hiring-specific standard invented for the exercise.

TEMPLATE – Mock-escalation exercise rubric, permanent ID TMPL-0801 (Part C)

CANDIDATE ID: ____ TIME USED / BUDGET: ____ / 10-15 min

Rubric line (per SOC Playbook Part 27)	Met? (Y/N)	Note
Context included (what the receiver needs to pick this up cold)		
Severity justified, not just asserted		
Concrete next action named		

RESULT: [Clears bar / Does not clear bar]

[FRONTLINE MANAGER]

This rubric does not define what a mechanically good hand-off contains — that standard lives in SOC Playbook Part 27, and this exercise borrows it directly so a strong performer here already escalates the way the SOC wants escalations to look on day one.

Manager's Note Time-box the mock escalation the same way for every candidate, and don't let how much of the window a candidate used factor into the score — only the rubric content should. It's the same completion-speed bias Part 8 §5.3 names for unpaid take-home exercises, showing up here as "sounded confident and finished in eight minutes" outscoring a candidate who used the full 15 to reason carefully.

3. Competency-matrix worksheet (TMPL-1001)

[SENIOR MANAGER] A team lead or SOC manager fills this in per analyst at nomination for a tier change, and roughly annually otherwise — never on the same monthly cadence as a performance review, per Part 10 §1's hard split between competency and performance. The four axis rows are fixed; the tier-floor and evidence columns are what a reviewer writes in, calibrated against their own tier definitions from Part 3.

TEMPLATE — Tiered competency-matrix worksheet, permanent ID TMPL-1001

ANALYST: ____ CURRENT TIER: ____ TARGET TIER: ____ REVIEW DATE: ____
REVIEWER(S): ____ (calibration requires a second independent reviewer on any disputed axis, per Part 10 §6)

Axis	Target-tier floor (write the specific observable behavior)	Evidence source
Technical skill	<!-- e.g., forms and tests a hypothesis across multiple log sources unaided -->	[Met / Not yet met / Not yet tested]
Tool proficiency	<!-- e.g., builds an ad hoc cross-tool query with no documentation -->	[Met / Not yet met / Not yet tested]
Communication	<!-- e.g., briefs a stakeholder outside the SOC, live, unscripted -->	[Met / Not yet met / Not yet tested]
Judgment	<!-- e.g., resolves an ambiguous severity call without escalating by default -->	[Met / Not yet met / Not yet tested]

GATE CHECK (no averaging — Part 10 §4.1):

- ☐ All four axes independently show "Met." If yes: forward to Part 13's promotion process.
- ☐ Any axis "Not yet met": route to a named hold with a specific remediation plan and a reassessment date — per axis, not blended.
- ☐ Any axis "Not yet tested": log the evidence gap and assign a way to gather it before the next review; do not score it as "Not yet met" (Part 10 §8's distinction between an unmeasured skill and an unmet one).

[SENIOR MANAGER] This worksheet does not write the anchor language itself — the specific behavior that counts as "meets the L2 floor" for technical skill or judgment is a per-SOC calibration exercise, worked through in full with a filled example at the L1-to-L2 boundary in Part 10 §4.2. A copy of this worksheet filled in without that calibration step just moves the "I'd know it if I saw it" problem from an unstructured promotion conversation into a structured-looking document that still means nothing.

Cross-references

This appendix is the template companion to Part 7 — Hiring & Sourcing Analysts (TMPL-0701), Part 8 — Interviewing & Technical Assessment Design (TMPL-0801), and Part 10 — Competency Models & Skills Matrices (TMPL-1001); it does not restate those chapters' arguments and assumes a reader has already read the relevant part before filling in a template here. The mock-escalation rubric in §2.3 cites SOC Playbook Handbook, Part 27 — Escalation Quality for its scoring standard. See Appendix A3 for what happens to a hire after the offer (onboarding and the career ladder) and Appendix A4 for the QA calibration mechanism this appendix's scorecards and matrix both reuse.

Appendix A3 — Onboarding, Training & Career-Ladder Templates

Why this appendix exists

[CONCEPT] This appendix carries the three fillable artifacts Parts 11 through 13 each promise a reader rather than re-derive inline: a 90-day onboarding plan and gate sign-off tracker, a training-budget and bench-depth worksheet, and a career-ladder and promotion-evidence-packet template. None of the three makes a judgment call for you — they exist to hold the judgment calls those chapters describe in a form a team lead, an HR partner, or a promotion committee can actually fill in, defend, and revisit later.

[CONCEPT] Part 11 — Onboarding & Ramp-Up Programs defines the four-phase ramp structure, the gate logic between phases, and the pacing adjustments a lateral hire or career-changer needs; **TMPL-1101** below is the fillable phase/gate document that part's §7 and §10 both point to. Part 12 — Ongoing Training & Skill Development defines what a continuing-education budget actually has to cover and how a bench-depth gap gets tracked to closure; **TMPL-1201** is the worksheet its §1.1 and §5.2 both cite. Part 13 — Career Ladders & Promotion Criteria defines the rungs, the branch bars into detection engineer, threat hunter, or team lead, and the evidence a promotion committee should expect before a name is even discussed; **TMPL-1301** is the fillable version of that ladder and its evidence packet. Read the relevant part before filling in its template — this appendix states what goes in each field, not why the bar is set where it is.

[CONCEPT] Every blank field below is intentionally undated and unscored — a manager filling this in for a real analyst enters real dates, real names, and real evidence, and every cell that would otherwise carry an invented number is marked **CONCEPTUAL SAMPLE** or **COMPOSITE CASE EXAMPLE** per the book's evidence-classification standard so a reader never mistakes an illustrative fill-in for a benchmark to copy.

1. The 90-day onboarding plan and gate sign-off tracker (**TMPL-1101**)

[HR/PEOPLE] The team lead and the new hire open this document on day one and both add to it through day 90; HR references the same document for the parallel probationary-period record rather than keeping a separate, silently drifting copy. It gives a phase/gate structure a specific place to record planned versus actual dates, the exit-criteria evidence a gate actually checked, and who signed off — the fillable counterpart to Part 11's Table 11.1, with the pacing-adjustment note Part 11 §10 requires attached to the same record rather than a separate memo.

TEMPLATE – 90-day onboarding plan and gate sign-off tracker, permanent ID TMPL-1101, home appendix A3

ANALYST: _____ START DATE: _____ TEAM LEAD: _____

BACKGROUND: ☐ New to SOC work (standard pacing, no Table 11.4 adjustment)

Or, per Part 11 Table 11.4: ☐ Help-desk/internal transfer

☐ Career-changer / apprenticeship pipeline ☐ Experienced lateral hire ☐ MSSP transfer

PACING ADJUSTMENT APPLIED: _____

REASON (one line, citing the Table 11.4 row that applied): _____

Phase result	Planned dates Sign-off name / date	Actual dates	Exit criteria met? (evidence)	Gate
-----	-----	-----	-----	-----
1. Shadowing escalation paths -->	☐ Advance ☐ Hold ☐ Remediate		<!-- names the 5 seeded alerts and	
2. Supervised triage dispositions -->	☐ Advance ☐ Hold ☐ Remediate		<!-- 3 consecutive shifts, 0 overturned	
3. Graduated independence consecutive weeks -->	☐ Advance ☐ Hold ☐ Remediate		<!-- spot-check pass rate, 2	
4. Simulation certification Part 11 Table 11.3 -->	☐ Pass ☐ Fail – remediation plan attached		<!-- link to the certification report,	

REMEDIATION LOG (one row per failed gate; a gate can appear more than once):

Gate failed	Date	Specific gap named	Remediation plan (2-week default)	Retest date	Retest result
---	---	---	---	---	---

This tracker does not define what counts as a passed gate — that bar comes from Part 10's competency matrix and Part 11's Table 11.1 — and it is not a substitute for Table 11.3's certification report, which stays a separate document a promotion or audit trail can point to on its own.

2. The training-budget and bench-depth worksheet (TMPL-1201)

[SENIOR MANAGER] A SOC manager fills in Part A once during annual budget planning and updates the utilization column monthly, per Part 12's Manager's Note that a training line sitting unspent by October is a scheduling failure, not evidence the team doesn't want training. Part B is a per-request decision log a manager fills in each time a certification funding request arrives, and Part C is the standing cross-training tracker Part 12 §3.2 and §5.2 both assume exists, rebuilt on whatever cadence the manager chooses — quarterly is Part 12's own default.

TEMPLATE – Training-budget and bench-depth worksheet, permanent ID TMPL-1201, home appendix A3

PART A – Per-analyst annual allocation, by category (Part 12 §1.1)

| Analyst | Paid study/lab hours | Certification/exam fees | Internal lab/range access | Content licensing | Conference/other | Total allocated | Spent to date | Utilization |

|---|---|---|---|---|---|---|---|---|

| CONCEPTUAL SAMPLE – J. Alvarez | 32 hrs | \$1,600 | \$300 | \$200 | \$0 | \$2,100 + 32 hrs | \$1,900 + 18 hrs | 90% \$ / 56% hrs |

PART B – Certification funding request log (Part 12 §2.1, §2.5)

| Analyst | Certification requested | Exam format (hands-on / multiple-choice) | Cost | Internal role/track this targets | Service commitment attached? | Decision | Decision date |

|---|---|---|---|---|---|---|---|

| | | | <!-- if blank, see Part 12 §2.5 before approving spend above ~\$1,000 --> | | |

PART C – Bench-depth gap tracker (Part 12 §3.2, §5.2)

| Specialist function | Analysts unsupervised-capable | Target bench depth | Gap | Closing format | Planned closure date | Actual closure date | Status |

|---|---|---|---|---|---|---|---|

| CONCEPTUAL SAMPLE – SOAR playbook administration | 1 | 2 | 1 | Paired rotation, 60 days | 2026-11-15 |
| – | In progress |

The worksheet tracks allocation, utilization, and gap-closure dates; it does not decide whether a specific certification predicts on-the-job capability — that question belongs to Part 12 §2.2's certification scorecard, applied before Part B's decision column ever gets filled in.

Manager's Note Review Part A's utilization column at the same meeting where you review queue-health metrics, not on a separate HR calendar — a training line that's technically "on track" against the annual total but never actually protected against backlog pull-backs (Part 12 §1.2) will still show 90% spent by October with almost none of it converted into a closed bench-depth gap in Part C.

3. The career-ladder and promotion-evidence-packet template (TMPL-1301)

[HR/PEOPLE] An HR partner and the SOC manager fill in Part A once, when the ladder is first published or formally revised — not per candidate, per Part 13 §1's warning against treating the ladder as a title generator. A nominating team lead fills in Part B per candidate, per promotion cycle, and the SOC manager or HR partner runs Part C on the audit cadence Part 13 §5.3 sets.

TEMPLATE – Career ladder and promotion evidence packet, permanent ID TMPL-1301, home appendix A3

PART A – Ladder rung and branch definitions (Part 13 §2-§3; fill in once, revise rarely)

	Rung / branch	Title	Minimum bar (matrix rows, QA score, tenure floor, work sample)	Pay band	Sign-off authority
	1	Analyst I	- - -		
	2	Analyst II / Senior Analyst	- - -		
	3a	Detection Engineer	<!-- work-sample count from merged detections, Part 13 §3.1 -->	- -	
	3b	Threat Hunter	<!-- completed-hunt count and hypothesis-discipline check, §3.2 -->	- -	
	3c	Team Lead	<!-- coaching-aptitude assessment + shadow-lead cycle, §3.3 -->	- -	
	4	Principal / Staff Analyst	<!-- the fourth road, §3.4 – do not leave this row blank -->	- -	

PART B – Promotion evidence packet (Part 13 §4.2; one packet per candidate, per cycle)

Evidence item	Source	Minimum bar	Met?	Evidence attached / link	Reviewer
Competency matrix rating	Part 10 matrix	"Meets" on every next-level row	[]		
QA calibration score	Part 15 QA program	≥85%, sustained 2 quarters	[]		
Handle time and independence	Ticket system	Within 15% of median; <20% escalation	[]		
Open PIP or coaching plan	Part 16 records	None open, trailing 6 months	[]		
Peer input	Peer-nomination form	≥2 submissions, no unaddressed concern	[]		
Branch-specific work sample	PRs / hunt records / shadow-lead log	Meets the §3 branch bar	[]		

PART C – Leveling-audit snapshot (Part 13 §5.3; run on a fixed schedule)

Level	Target % of headcount	Target headcount	Actual headcount	Gap	Corrective action
					<!-- correct forward only – never a demotion, per §5.3 -->

Filling in every field does not make a promotion decision for the committee, and it cannot resolve a split vote — that judgment stays with the committee process Part 13 §4 defines. What it produces is the packet a candidate, a peer, or an auditor can be shown a year later as the actual basis for the call.

Cross-references

Companion to Part 11 — Onboarding & Ramp-Up Programs, Part 12 — Ongoing Training & Skill Development, and Part 13 — Career Ladders & Promotion Criteria. Also draws on Part 10 — Competency Models & Skills Matrices (the bar every gate and rung checks against), Part 15 — Quality Assurance Programs and Part 16 — Performance Management & Coaching (the QA score and PIP fields in TMPL-1301's evidence packet), and Part 20 — Building & Defending the SOC Budget (where TMPL-1201's allocation line sits inside the wider budget).

Appendix A4 — QA, Performance & Retention Templates

Why this appendix exists

[CONCEPT] This appendix carries the four fillable artifacts that Part 15 — Quality Assurance Programs, Part 16 — Performance Management & Coaching, and Part 18 — Attrition & Retention each build toward but stop short of publishing in full: a QA ticket scorecard (TMPL-1501), a calibration-session guide (TMPL-1502), a performance-improvement-plan template (TMPL-1601), and an exit-interview question set (TMPL-1801). None of the four is new content — each one is the working paper version of a structure those three parts already justify, section by section, on the reasoning that a rubric, a session format, a plan, or a question set needs to exist *before* the judgment calls those parts spend most of their pages on can happen at all.

[CONCEPT] The four templates are also a deliberate chain, not four unrelated forms. A QA scorecard (§1) produces the score that Part 15's sampling and calibration program is built to trust; a calibration-session guide (§2) is how a team keeps more than one reviewer's scorecard meaning the same thing; a performance-improvement plan (§3) is where a *sustained, calibrated* pattern of low scores ends up, per Part 16's three-stage pipeline, and only after the diagnostic in that part's §3 has ruled out a training gap or a broken rule; and an exit-interview question set (§4) is run on the analyst a manager couldn't keep, coded against the same attrition taxonomy Part 18 builds its retention levers from. Read together, they cover the full path from "did this analyst do good work on this ticket" to "why did we lose this analyst," which is not a coincidence — Parts 15, 16, and 18 were written as one continuous accountability chain for exactly that reason.

[CONCEPT] None of the four templates automates the judgment call the companion part spends its pages developing. A scorecard doesn't decide whether a rubric category is well-calibrated for a given ticket type; a PIP form doesn't decide whether a recurring miss is a skill gap or a broken detection rule; a question set doesn't make a departing analyst candid. Each template's limitation is stated explicitly where it appears below, per this book's own convention that a bare template with no stated limitation is treated as incomplete, not helpful.

1. QA ticket scorecard (TMPL-1501)

[SENIOR MANAGER] A team lead or QA specialist fills one of these out per sampled ticket immediately after reviewing it, rather than reconstructing a score from memory later — the same independent-judgment discipline Part 15 §3.2 requires of calibration-session scoring, applied here to routine, single-reviewer sampling rather than a shared batch. The category weights below are the illustrative starting rubric from Part 15's Table 15.2 (CONCEPTUAL SAMPLE — a reasonable default weighting, not a fixed standard); a program should revisit its own weights annually against what its own incident history says actually predicts a missed threat, per Part 15 §4.2.

TEMPLATE – QA ticket scorecard, permanent ID TMPL-1501

Ticket ID: _____ Analyst reviewed: _____
Reviewer: _____ Review date: _____
Sampling reason: ☐ Risk-weighted (P1/P2) ☐ Random baseline ☐ Targeted (complaint/reopen)
<!-- Sampling reason feeds the ratio Part 15 §2.3 tracks -- do not leave blank. -->

Category	Weight	Score (1-5)	Weighted	Reviewer note (cite the specific evidence the score is based on)
-----	-----	-----	-----	-----
Technical accuracy	30%			
Escalation judgment	25%			
Documentation completeness	20%			
Process adherence	15%			
Communication and tone	10%			
TOTAL WEIGHTED SCORE	100%			

Below program's coaching threshold? ☐ Yes ☐ No
<!-- State your program's own numeric cutoff here once set -- Part 15 does not fix one for you. -->
Coaching conversation scheduled for (date): _____
<!-- Part 15 SS5.4's own Field Test checks this field gets filled, not left as an intention. -->

Manager's Note Fill the reviewer-note cell for every category, not only the ones that scored low. A 5 with no note is indistinguishable from a rushed review, and the note is what makes a disputed score defensible weeks later, when neither reviewer remembers the ticket without it.

[SENIOR MANAGER] This scorecard does not decide, by itself, whether a low score is noise, a coaching moment, or evidence for Part 16's formal track — that routing decision is Part 15's Figure 15.2, and this form only supplies the calibrated input that decision needs.

2. Calibration-session guide (TMPL-1502)

[FRONTLINE MANAGER] A facilitator — usually the SOC manager or a senior reviewer — runs this guide at least once a quarter, the cadence Part 15 §6.2 sets, following the session mechanics Part 15 §3.2 lays out. Every reviewer scores the shared batch independently before Step 2 opens; comparing scores before independent scoring is complete defeats the session's purpose.

TEMPLATE – Calibration-session guide, permanent ID TMPL-1502

Session date: _____ Facilitator: _____
Reviewers present: _____
Rubric version reviewed: _____
Batch (5-10 already-closed, unscored tickets): _____

STEP 1 -- Independent scoring (complete before Step 2)

Ticket ID	Category	Reviewer A	Reviewer B	Reviewer C	Reviewer D	Spread
-----	-----	-----	-----	-----	-----	-----

STEP 2 -- Agreement calculation

% of category-scores within 1 point of that ticket's median across reviewers: _____%
<!-- Part 15 SS3.3 treats <80% as not yet trustworthy for coaching or PIP input. -->

STEP 3 -- Divergent-category log (discuss widest-spread categories only; skip categories already in agreement)

Category	Ticket ID	Spread	What each reviewer weighed differently	Written anchor example agreed
-----	-----	-----	-----	-----

STEP 4 -- Next step

Blind rescore date (same batch, ~2 weeks out): _____
Agreement target for rescore: _____% (program's own working threshold)

[FRONTLINE MANAGER]

This guide structures the conversation; it doesn't resolve a disagreement for the room. If Step 3's discussion keeps tracing back to "what does a good hand-off even require" rather than a scoring-consistency question, the gap is in the team's shared understanding of SOC Playbook Handbook, Part 27 — Escalation Quality, not in this rubric — see Part 15 §3.3's Cross-Book Pointer before assuming another anchor example will fix it.

3. Performance-improvement-plan template (TMPL-1601)

[HR/PEOPLE]

A manager completes this only after Part 16 §3's three-hypothesis diagnostic has ruled out a training gap and a broken process, and only after the specific gap has already appeared in a dated, prior 1:1 note — the two preconditions Part 16 §4.1 sets before a PIP is opened at all. Route the completed draft through Part 27's legal-review step before delivery in any jurisdiction with meaningful wrongful-termination exposure.

TEMPLATE – Performance improvement plan, permanent ID TMPL-1601

Analyst: _____ Manager: _____
Start date: _____ Plan length: ☐ 30 days ☐ 45 days ☐ 60 days
Prior documented coaching this gap traces to (1:1 date): _____
<!-- Must predate this plan by more than a few days -- see the People Risk Trap below. -->
HR partner reviewed: ☐ Yes, on (date) _____ ☐ Not yet

PROBLEM STATEMENT

<!-- The specific, observable gap, tied to the 1:1 date above or a QA score (TMPL-1501) --
not a general impression. -->

SUCCESS CRITERIA

<!-- A measurable target tied to an existing baseline (team QA average, handle-time norm,
error rate on a named alert type) -- not a subjective bar. -->

SUPPORT COMMITTED

<!-- Specific coaching time, shadowing, or resources the manager is actually scheduling --
not "will be available if needed." -->

CHECKPOINT LOG

Checkpoint date	Progress against success criteria	Support delivered this period	Adjustment (if any)
----- ----- ----- -----			

CONSEQUENCE OF NON-IMPROVEMENT (state plainly)

CLOSING OUTCOME (complete at plan end):

☐ Criteria met -- plan closed ☐ Partial, bounded extension granted (new end date: ____)
☐ Criteria not met -- route to Part 27's termination-documentation process

People Risk Trap A "prior documented coaching" field filled in retroactively, after the plan is already drafted, defeats the precondition it exists to prove — it launders a plan that was never actually preceded by real coaching into looking like one that was. Only complete that field by pointing to a 1:1 note dated and written before this plan was drafted, never backfilled the same week the plan opens.

[HR/PEOPLE] This template does not automate the diagnostic call in Part 16 §3 — no template can — and a completed form with a vague problem statement or no interim checkpoint is the exact failure pattern Part 16's **CASE-1602** shows collapsing under a wrongful-termination challenge.

4. Exit-interview question set (TMPL-1801)

[HR/PEOPLE] An HR partner, or a manager one level removed from the departing analyst's own chain, runs this — never the departing analyst's direct manager, who is both an interested party and often not trusted with a fully candid answer, per Part 18 §3.1. Code every answer against Table 18.1's taxonomy before filing; a set of uncoded narrative answers analyzes badly no matter how thorough the conversation felt.

TEMPLATE — Exit-interview question set, permanent ID TMPL-1801

Departing analyst (role / tenure band, not name, for aggregation): _____

Interview date: _____ Interviewer: _____

Voluntary departure? ☐ Yes ☐ No (route to Part 16 -- not this program's territory)

FIXED QUESTIONS (ask in this order; do not paraphrase into a single open prompt)

1. What is the primary reason you're leaving?
2. Did compensation play a role? If so, how does the new role's pay compare to this one?
3. In the last 6 months, did workload, shift pattern, or on-call burden affect this decision?
4. Did you feel there was a clear path for growth here? What would that path have needed to include?
5. Was this decision tied to a specific team, shift, or manager, rather than the role overall?
6. What would have needed to change in the last 3 months for you to stay?
7. Is there anything you raised before now that, if addressed, would have changed this decision?

CODING (interviewer completes after the conversation, against Table 18.1)

☐ Burnout-driven exit ☐ Competitor poaching ☐ Plateau / no-growth exit

☐ Shift-burden exit ☐ Performance-managed-out ☐ Other: _____

Named competitor/destination (if poaching): _____

Corroborated against a leading indicator (Part 18 SS2.1) or stay-interview data (SS2.3)?

☐ Yes ☐ No ☐ Not checked

COMPOSITE CASE EXAMPLE — illustrative coded response, not a real interview: A mid-tenure L2 analyst's coded answers named a specific competing MSSP and a stated pay gap in Question 2, with no workload or growth complaints in Questions 3–6 — coded as **Competitor poaching**, consistent with Part 18's **CASE-1801** pattern, and routed to a pay-band review rather than a culture-side fix.

[HR/PEOPLE] This question set cannot make a departing employee candid who has already decided candor isn't worth the effort on the way out — Part 18 §2.2's Blind Spot applies here directly. Corroborate its coded themes against leading indicators and stay-interview data before treating any single exit interview's coding as a confirmed organizational pattern.

Cross-references

Within this book: Companion to Part 15 — Quality Assurance Programs (**TMPL-1501**, **TMPL-1502**), Part 16 — Performance Management & Coaching (**TMPL-1601**), and Part 18 — Attrition & Retention (**TMPL-1801**). See those parts for the sampling methodology, the diagnostic judgment call, and the retention-lever taxonomy each template assumes rather than restates here.

Appendix A5 — Vendor, Contract & Procurement Templates

What this appendix contains

[CONCEPT] This appendix is the filing cabinet for the three standing instruments Part 21 — Tooling Procurement & Platform Strategy, Part 22 — MSSP & Managed-Service Contract Management, and Part 23 — Vendor Relationship & Renewal Management each build a piece of, but never fully house inline: a weighted RFP/PoC evaluation scorecard, a library of contract-clause language for the terms that most often go wrong after signing, and a standing vendor risk-scoring worksheet. None of the three is new content invented for this appendix — each is the fillable, standalone version of a template those three chapters already describe and cite by ID, assembled here so a negotiator or a procurement lead can copy the actual worksheet instead of reconstructing it from prose scattered across three chapters.

[CONCEPT] The RFP/PoC scorecard (**TMPL-2101**) is Part 21's instrument for scoring SIEM, EDR, SOAR, or TIP finalists on functional fit, total cost of ownership, and exit-cost exposure together, rather than letting a feature bake-off decide the purchase alone. The SLA contract-clause library (**TMPL-2201**) turns the clause-by-clause negotiation guidance in Part 22 — clock-start definitions, coverage-hours minimums, service-credit scaling, right-to-audit scope, and transition-out terms — plus the lock-in mitigation terms Part 21, §4.1 argues belong in the same contract, into draftable contract language a negotiator can hand to legal counsel as a starting point, rather than drafting each clause from a blank page under time pressure. The vendor risk-scoring worksheet (**TMPL-2301**) is Part 23's standing, six-dimension instrument for catching a vendor's financial, ownership, support, or security drift on a fixed cadence, long before it shows up as a bad renewal quote.

[CONCEPT] None of the three templates automates the judgment call it exists to support. The scorecard still requires someone to have actually run the proof-of-concept and the off-list reference call Part 21 describes before its rows mean anything; the clause library still requires review by qualified legal counsel before any language is put in front of a vendor, because contract enforceability is jurisdiction-specific in ways this appendix cannot generalize across; and the risk-scoring worksheet still requires a named reviewer willing to write down an honest score and a real reason, not a number chosen to avoid an uncomfortable conversation. Treat every value in this appendix's worked rows marked CONCEPTUAL SAMPLE as illustrative, not sourced benchmark data.

1. RFP/PoC weighted evaluation scorecard (**TMPL-2101**)

[VENDOR/PROCUREMENT] A SOC manager or the designated procurement lead uses this scorecard once per finalist platform, filled in only after the proof-of-concept and reference calls described in Part 21 — Tooling Procurement & Platform Strategy, §2.3–§2.4 are both complete — never filled in from a vendor's own pitch deck or a gut read of the demo. Score each row against evidence the buying organization gathered itself; a row with no evidence behind it should stay blank rather than get a guessed number.

The legend below defines what each 1–5 score means, since a shared scale across finalists and reviewers is what keeps the total comparable.

Score	Meaning
1	Fails to meet the requirement; a disqualifying gap
2	Meets the requirement only with significant workaround or added cost
3	Meets the requirement adequately, no notable strength or weakness
4	Meets the requirement well, with a specific advantage over the alternative(s)
5	Exceeds the requirement in a way that materially changes the decision

TEMPLATE — Platform RFP/PoC weighted evaluation scorecard, permanent ID [TMPL-2101](#)

VENDOR/FINALIST: _____		EVALUATOR: _____	DATE: _____
CRITERION	WEIGHT	SCORE (1-5)	WEIGHTED
Functional fit against the use cases named in the RFP <!-- score only against the use cases the RFP actually named, not the vendor's full feature list -->	25%	—	—
Data ingest/normalization fit, worst-case log source <!-- score from the PoC's messiest, most poorly-parsed log source, never from the vendor demo's cleanest one -->	15%	—	—
Integration surface (SOAR, ticketing, IdP, EDR)	10%	—	—
Three-year total cost of ownership (license + migration + retraining) <!-- pull this number from a completed TCO worksheet, never the vendor's list price alone -->	25%	—	—
Exit-cost / lock-in exposure <!-- pull this number from a completed exit-cost estimate: egress fees, content re-migration, and any early-termination penalty -->	15%	—	—
Vendor viability & support model	5%	—	—
Reference-customer signal, including one off-list call	5%	—	—

TOTAL	100%		—

[VENDOR/PROCUREMENT] This scorecard's default weighting deliberately puts three-year TCO and exit-cost exposure at 40% combined, above functional fit's 25%, because a standard feature bake-off already over-weights functional fit on its own. Adjust the weights for a genuinely different risk profile — a heavily regulated environment might push exit-cost exposure higher still — but do the adjusting before scoring starts, not after a preferred finalist's total comes in low.

2. SLA contract-clause library ([TMPL-2201](#))

[VENDOR/PROCUREMENT] A contract negotiator preparing for or reviewing an MSSP, MDR, or managed-service statement of work uses this library as starting language for six clause families: the five Part 22 — MSSP & Managed-Service Contract Management identifies as the ones most likely to be either missing or too vague to enforce — the SLA clock-start definition, the coverage-hours staffing minimum, service-credit scaling, right-to-audit scope, and the transition-out clause — plus the lock-in mitigation set Part 21 — Tooling Procurement & Platform Strategy, §4.1 argues is cheapest to negotiate into the same contract before signing rather than left for a separate conversation later. Each block below is drafted to be filled in with the buying organization's own negotiated numbers, then handed to qualified legal

counsel for jurisdiction-specific review — none of this language is a substitute for that review, and none of it should go to a vendor unreviewed.

TEMPLATE — SLA contract-clause library, permanent ID [TMPL-2201](#)

neshbooy

CLAUSE 1 – NOTIFICATION CLOCK-START DEFINITION (Part 22 §1.2)

Vendor shall notify Client of any Critical-severity security event within [__] minutes of the Event Timestamp. "Event Timestamp" means the earlier of: (a) the timestamp recorded by Vendor's detection platform at first alert generation, or (b) the timestamp of the underlying log event that triggered the alert, as recorded in a system Client can independently verify. Vendor shall additionally report, on a monthly basis and regardless of whether the primary SLA in this Section was met, the secondary metric "Total Elapsed Time from Alert Generation to Client Notification" for all Critical-severity events during the reporting period.

<!-- if Vendor will not agree to (b), the secondary metric is the fallback
Part 22 §1.2 recommends -- it carries no remedy but keeps the gap visible -->

CLAUSE 2 – COVERAGE-HOURS STAFFING MINIMUM (Part 22 §1.3)

Vendor shall maintain no fewer than [__] dedicated, named security-operations analysts staffed continuously on every shift, 24 hours per day, 7 days per week, covering no more than [__] client accounts per analyst per shift. Client may verify this commitment under the right-to-audit provisions of Section [__] without advance notice exceeding [__] hours.

<!-- "24/7 monitored" alone commits to nothing; name the analyst count and
the per-analyst account load explicitly -->

CLAUSE 3 – SERVICE-CREDIT SCHEDULE, SCALED BY SEVERITY (Part 22 §1.4)

Severity tier scored under Section [__]	Credit per missed instance	Cap per billing period
Critical	[__]% of monthly fee	[__]% of monthly fee
High	[__]% of monthly fee	[__]% of monthly fee
Medium	[__]% of monthly fee	[__]% of monthly fee

A missed Critical-severity SLA occurring during a Client-confirmed security incident shall be escalated for a documented root-cause review within [__] business days, independent of whether a credit was issued.

<!-- a flat, low percentage cap regardless of severity is the trap Part 22
§1.4 names -- scale the cap, don't just scale the credit -->

CLAUSE 4 – RIGHT-TO-AUDIT (Part 22 §2.1-§2.2)

Client shall have the right to conduct, no more than [__] time(s) per contract year without cause, a [] documentation and access review [] technical control assessment [] on-site assessment of Vendor's security controls relevant to the delivery of the Services, upon no fewer than [__] and no more than [__] days' written notice. [Vendor / Client / the parties in proportion to scope] shall bear the cost of any such review. Vendor shall remediate any confirmed finding within [__] days of written notice, with escalation to [named contact/role] if the deadline is missed. This right extends to any subcontractor, offshore delivery tier, or fourth-party platform Vendor utilizes to deliver any portion of the Services, and Vendor shall disclose the existence and role of any such subcontractor upon request.

<!-- the subcontractor/fourth-party sentence is the clause most often left
out entirely -- see Part 22 §2.2's fourth-party blind spot -->

CLAUSE 5 – TRANSITION-OUT / OFFBOARDING (Part 22 §3.2)

Upon termination or expiration of this Agreement, Vendor shall: (a) return all Client historical alert, case, and log data in [named portable format, e.g. CEF/JSON export] within [__] days of the termination date; (b) retain a copy of such data for [__] days post-termination for reconciliation purposes only, accessible to Client on request, and provide written certification of deletion upon expiration of that period; (c) provide a documented, human-readable statement of the logic and tuning rationale behind every

custom detection rule, playbook, or enrichment routine built for Client's environment during the term, not solely the underlying rule syntax; (d) make personnel reasonably available to Client and Client's incoming vendor or internal team for a Transition-Assistance Period of [__] days following termination; and (e) charge no fee for the activities in (a)-(d) beyond [capped amount / rate], regardless of the reason for termination.

<!-- item (e)'s cap is the highest-priority, cheapest-to-negotiate line in this entire library per Part 22 §5's negotiation-priority table -- push for it even when nothing else in this clause moves -->

CLAUSE 6 – LOCK-IN MITIGATION SET (Part 21 §4.1)

- (a) Export format: Vendor shall provide, on request and at no additional charge, an export of all Client-authored detection/correlation logic in [a documented, non-proprietary format] sufficient to permit re-authoring on an alternative platform.
- (b) Egress allowance: Vendor shall include [__] TB per year of historical-data bulk export within the base subscription fee, with any excess billed at a disclosed, capped rate of no more than \$[__] per additional TB.
- (c) Price-increase ceiling: Any renewal price increase shall not exceed [__]% year-over-year absent a material, individually itemized change in scope.

[VENDOR/PROCUREMENT]

This library gives a negotiator draftable starting language for the clause families that are cheapest to negotiate before signing and most expensive to renegotiate after; it does not tell a negotiator which of the six clauses to prioritize when a vendor is only willing to concede on some of them. Part 22, §5's negotiation-priority table answers that ordering question — use it alongside this library, not instead of it.

3. Vendor risk-scoring template (TMPL-2301)

[VENDOR/PROCUREMENT]

A vendor-governance owner uses this worksheet on a fixed cadence — quarterly for a Tier 1 vendor, at minimum annually for everyone else, per the contract-tier calendar in Part 23 — Vendor Relationship & Renewal Management, §2.1 — to catch drift in a vendor's financial health, ownership, support quality, roadmap, security posture, or concentration risk before that drift shows up as a bad renewal quote or a support failure during a live incident. File a completed copy alongside the renewal-calendar entry for the same vendor each cycle, so the trend line across quarters, not just the current score, is visible at the next review.

TEMPLATE — Standing vendor risk scorecard, permanent ID [TMPL-2301](#)

VENDOR: _____ REVIEW PERIOD: Q__ 20__ REVIEWER: _____

Dimension	Score (1-5)	Evidence / reason for this score	Change vs. last review
Financial stability	—	<!-- cite the specific signal, not a gut feel -->	
Ownership / M&A exposure	—		
Support quality trend	—	<!-- compare to a specific prior-period number, e.g. median response time -->	
Product / roadmap health	—		
Security posture	—		
Concentration risk	—	<!-- name the specific functions that depend on this vendor with no fallback -->	

OVERALL FLAG: ☐ Green (no action) ☐ Yellow (monitor, revisit next cycle)
☐ Red (any single dimension at 4-5, or 2+ dimensions worsened since last review -- build a contingency plan before the next renewal date, regardless of how far out it is)

CONCEPTUAL SAMPLE — illustrative scoring, not sourced benchmark data.

VENDOR: [EDR platform, Tier 1] REVIEW PERIOD: Q3 2026 REVIEWER: [name/role]

Dimension	Score	Evidence / reason for this score	Change
Financial stability	2	No distress signals; steady renewal pricing	
Ownership / M&A exposure	4	Acquired by PE-backed consolidator 14 months ago; no integration roadmap published yet	+2
Support quality trend	4	Median ticket response rose from ~4 business hours to ~3 business days over two quarters; named TAM left, unreplaced	+2
Product / roadmap health	3	No material feature release addressing our use case in 10 months	+1
Security posture	1	SOC 2 Type II current; no disclosed incidents	
Concentration risk	3	Sole source for endpoint detection; no fallback tooling staged	

OVERALL FLAG: ☐ Green ☐ Yellow ☒ Red

Note: two dimensions (ownership/M&A exposure, support quality trend) are already at 4, and three dimensions have worsened since the last review -- this trips Red under both criteria. Build the contingency plan now rather than waiting for the next quarterly cycle.

[VENDOR/PROCUREMENT]

This worksheet catches gradual, visible drift reasonably well; it does not catch a sudden, non-public event between review cycles — an acquisition announced the week after a review just ran, a breach disclosed on a delay. Pair the fixed cadence with a standing habit of watching public M&A and security-disclosure news for every Tier 1 vendor between formal reviews, per Part 23, §3.3's Blind Spot.

Cross-references

This appendix is the companion filing for Part 21 — Tooling Procurement & Platform Strategy ([TMPL-2101](#), §2.2), Part 22 — MSSP & Managed-Service Contract Management ([TMPL-2201](#), §1–§3, §5), and Part 23 — Vendor Relationship & Renewal Management ([TMPL-2301](#), §3.2). It also assumes the TCO and exit-cost worksheets described in Part 21, §3–§4 as direct inputs to [TMPL-2101](#)'s two highest-weighted rows, and the negotiation-priority ordering in Part 22, §5 as the sequencing guide for [TMPL-2201](#). See Appendix A6 — Budget & Business-Case Templates for the TCO calculator itself, and Appendix A8 — Cross-Series Quick Reference for where MSSP severity terminology diverges from the internal severity model owned by SOC Playbook Handbook, Part 29 — Playbook Severity Model.

Appendix A6 — Budget & Business-Case Templates

Why this appendix exists

[CONCEPT] This appendix holds the three fillable artifacts Part 2 — SOC Operating Models and Part 20 — Building & Defending the SOC Budget both build toward but deliberately don't reprint in full: a budget worksheet organized by Part 20 §1's five categories ([TMPL-2001](#)), the weighted build-vs-buy-vs-outsource decision matrix Part 20 §6 walks through inline for one composite organization ([TMPL-2002](#)), and a three-year total-cost-of-ownership calculator that extends a single year's budget into the multi-year comparison Part 20 §5 argues is the minimum defensible horizon ([TMPL-2003](#)). Every field in every template below matches a section those two parts already justify — this appendix doesn't introduce new budget logic, it makes the logic fillable.

Part 2 makes the build/buy/blend operating-model decision on cost, control, retention, telemetry sensitivity, and coverage-hours economics; Part 20 turns that decision into dollars a CFO can hold onto and a business case that survives more than a Year 1 comparison. Both chapters use one composite 4,000-endpoint organization as a running worked example, and this appendix reuses the same organization's numbers throughout so a reader can trace a figure from a chapter straight into the worksheet that produced it, rather than reconciling two unrelated sets of illustrative numbers.

None of the three templates automates the judgment calls Part 2 and Part 20 both flag as unresolved by dollars alone — how much a weight in the decision matrix should favor control over cost, whether a rising cost-per-alert is a real efficiency gain or a quality problem in disguise, whether a model's detection coverage holds up once the cost comparison is settled. Fill these out as the documented starting point for that conversation, not as a substitute for having it.

1. Budget worksheet by category

[SENIOR MANAGER] A SOC manager or budget owner fills this out when building the annual budget request, or on the quarterly refresh cadence Part 20 §8 recommends, using the five-category structure Part 20 §1 defines: headcount, tooling, training, facilities, and vendor services. Complete the Basis/Driver column for every line — a category with no stated driver is exactly the "single number, no explanation" budget Part 20 §1 says doesn't survive a finance reviewer's first question.

TEMPLATE – Budget worksheet by category, permanent ID TMPL-2001, home appendix A6

Category	Line Item	Basis / Driver	Prior-Year Actual	Current-Year Budget	Variance (\$)	Variance (%)	Notes
Headcount	Tier 1/2/3 analysts (loaded cost x headcount)	Coverage-hours target, Part 5 headcount model	-	-	-	-	<!-- state the shrinkage assumption feeding this year's headcount number, not "based on last year" -->
Headcount	Team leads	Span-of-control ratio, Part 13	-	-	-	-	
Headcount	SOC manager allocation	Org chart, Part 4	-	-	-	-	
Tooling	SIEM licensing	Data volume / seat count	-	-	-	-	<!-- note pricing structure: per-endpoint, per-GB-ingested, or per-user, per Part 2 Sec. 2.1 -->
Tooling	EDR licensing	Endpoint count	-	-	-	-	
Tooling	SOAR / TIP licensing	Feature-tier, seat count	-	-	-	-	
Training	Certifications	Headcount growth, skill-gap plan (Part 12)	-	-	-	-	
Training	Conferences	Headcount x per-head allowance	-	-	-	-	
Training	Internal knowledge-sharing time	Opportunity cost, no line-item price (Part 12)	-	-	-	-	<!-- state the estimation method used, not "n/a" -->
Facilities	Secure floor / badge access	On-site headcount	-	-	-	-	
Facilities	Redundant power / network circuits	Coverage-hours model	-	-	-	-	
Facilities	Remote-work provisioning (stipends, hardened endpoints)	Remote/on-site mix	-	-	-	-	
Vendor services	MSSP / MDR contract	Operating-model choice, Part 2	-	-	-	-	<!-- name the pricing structure and renewal date -->
Vendor services	Standing IR retainer	Part 28 activation policy	-	-	-	-	
Vendor services	Threat-intel feed subscriptions	Coverage scope	-	-	-	-	
Vendor services	Pentest / red-team engagements	Annual test cadence	-	-	-	-	
TOTAL			-	-	-	-	

The worked block below shows the worksheet filled in for the fully in-house model Part 20 \$2 prices for its composite 4,000-endpoint organization, so the shape of a completed row is visible before a reader fills in their own numbers.

CONCEPTUAL SAMPLE -- extends Part 20 Sec. 2's composite fully-in-house organization, not sourced benchmark data.

Headcount	11 Tier 1/2/3 analysts x \$95,000	Coverage-hours target	\$1,045,000
Headcount	2 team leads x \$130,000	Span-of-control ratio	\$260,000
Headcount	1 SOC manager	Org chart	\$160,000
Tooling	SIEM + EDR + SOAR licensing	Data volume/seat count	\$450,000
Training	\$3,000/analyst x 11, \$2,500/lead x 3	Headcount growth	\$40,500
Facilities	Secure floor, redundant network, badge	Coverage-hours model	\$60,000
Vendor svcs	Threat-intel feed + standing IR retainer	Part 28 policy	\$90,000
		Total =	\$2,105,500/year

This worksheet totals dollars correctly, but it doesn't decide which category *should* be growing this year — that diagnostic call belongs to Part 20 \$1, and a worksheet with every Basis/Driver cell filled in with "same as last year" produces a number a finance reviewer can add up but not defend.

2. Build-vs-buy-vs-outsource decision matrix

[SENIOR MANAGER]

Fill this out when defending or reconsidering Part 2's operating-model decision, after completing a [TMPL-2001](#) worksheet for each candidate model and the TCO calculator ([TMPL-2003](#), §3 below) — Part 20 §6 walks through the identical matrix inline, scored for one composite organization, and this is the reusable blank version of that same instrument. Score each model 1 through 5 (5 is best) per dimension, then multiply by the dimension's weight and sum for a weighted total.

TEMPLATE — Build-vs-buy-vs-outsource decision matrix, permanent ID TMPL-2002, home appendix A6

Dimension	Weight (%)	Fully In-House (1-5)	Fully Outsourced (1-5)	Hybrid + Embedded DE (1-5)	Co-Managed (1-5)
3-year TCO (lower cost scores higher)		-	-	-	-
Cost-per-alert trend (Part 20 §4/§5.2)		-	-	-	-
Tuning-authority latency / control (Part 2 §3)		-	-	-	-
Telemetry-sensitivity fit (Part 2 §4)		-	-	-	-
Retention lever for senior talent (Part 2 §5)		-	-	-	-
Coverage-hours cost efficiency, thin shifts (Part 2 §6)		-	-	-	-
WEIGHTED TOTAL (weight x score, summed)	100%	-	-	-	-

<!-- state why each weight was set where it was -- "cost-constrained, low telemetry sensitivity, no near-term retention crisis" -- not just the number, per Part 20 Sec. 7's Operational Reality on documenting weighting rationale -->

The block below is Part 20 §6's own worked scoring, reproduced here as the filled-in reference case rather than re-derived:

CONCEPTUAL SAMPLE -- illustrative weights and scores for the composite organization used throughout Part 2 and Part 20, not a universal scoring or weighting key.

Weights: TCO 30%, cost-per-alert 15%, control 15%, sensitivity fit 15%, retention 15%, coverage-hours efficiency 10%

Dimension	In-House	Outsourced	Hybrid	Co-Managed
3-year TCO	1	5	4	2
Cost-per-alert trend	1	5	4	2
Tuning-authority latency/control	5	2	4	3
Telemetry-sensitivity fit	5	1	3	4
Retention lever	4	1	4	4
Coverage-hours efficiency	1	5	4	4
Weighted total:	2.65	3.35	3.85	2.95

A weighted total is only as honest as the weights behind it, and the matrix has no way to check whether the winning model's actual detection coverage holds up — Part 20 §5.3's Cross-Book Pointer sends that specific check to Detection Engineering Handbook V2, Parts 41–43 before treating a weighted win as the final answer.

3. Three-year TCO calculator

[SENIOR MANAGER]

Use this once each candidate model has a completed [TMPL-2001](#) worksheet, to extend a single year's steady-state budget into the multi-year comparison Part 20 §5.2 argues every

build-vs-buy case needs — a Year 1 comparison alone hides the one-time hiring or vendor-onboarding cost every model carries and, per Part 20 §5.2's Management Autopsy, can make the wrong model look cheapest.

TEMPLATE — Three-year TCO calculator, permanent ID TMPL-2003, home appendix A6

```
| Model | Year 1 Steady-State Budget (TMPL-2001 total) | One-Time Ramp/Transition Cost | Year 2 Budget
| Year 3 Budget | 3-Year TCO | 3-Yr Alerts Triaged | Blended Cost-per-Alert | | | |
|---|---|---|---|---|---|---|
| Fully in-house | - | - | - | - | - | - |
| Fully outsourced | - | - | - | - | - | - |
| Hybrid + embedded DE | - | - | - | - | - | - |
| Co-managed | - | - | - | - | - | - |

<!-- Year 2/3 budgets default to the Year 1 steady-state figure carried flat unless a known
      driver change (renewal, headcount growth, a new data source) is documented in a footnote -->
```

The worked block below reproduces Part 20 §5.2's own four-model comparison, so the calculator's output is checkable against the chapter that specifies it:

CONCEPTUAL SAMPLE -- extends Part 20 Sec. 5.2's composite organization, not sourced benchmark data. Assumes flat alert volume across 3 years (219,000 alerts/year, 657,000 over the window) -- a simplifying assumption, not a forecast; see Part 20 Sec. 4's What Would Change My Mind before trusting a flat-volume 3-year projection for your own SOC.

Model	Y1 Budget	Ramp Cost	3-Yr TCO	Blended \$/Alert
Fully in-house	\$2,105,500	\$342,000	\$6,658,500	\$10.13
Fully outsourced	\$1,157,000	\$65,000	\$3,536,000	\$5.38
Hybrid + embedded DE	\$1,229,000	\$62,000	\$3,749,000	\$5.71
Co-managed	\$1,738,000	\$88,000	\$5,302,000	\$8.07

This calculator assumes Year 2 and Year 3 hold the Year 1 driver assumptions flat unless a footnote says otherwise, and it says nothing about detection quality — pair its output with the decision matrix in §2 above rather than treating the cheapest 3-year number as the recommendation on its own, per Part 20 §5.3.

Cross-references: Companion to Part 2 — SOC Operating Models: In-House, MSSP, Co-Managed, Hybrid (the operating-model decision [TMPL-2002](#) scores) and Part 20 — Building & Defending the SOC Budget (the five-category structure behind [TMPL-2001](#), the TCO methodology behind [TMPL-2003](#), and the cost-per-analyst/cost-per-alert framings both templates report against). See Part 20 §5.3 and Detection Engineering Handbook V2, Parts 41–43, for the detection-coverage/quality check none of these three templates performs.

Appendix A7 — Executive, Board & Risk-Acceptance Templates

Why this appendix exists

[EXECUTIVE] This appendix carries the four fillable artifacts that Part 24 — Executive & Board Reporting, Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty, and Part 29 — Post-Incident Organizational Review each build toward but stop short of formatting as a standalone, copyable document: the board deck skeleton ([TMPL-2401](#)), the risk-acceptance memo ([TMPL-2501](#)), the post-incident organizational review record ([TMPL-2901](#)), and its companion corrective-action tracker ([TMPL-2902](#)). All four are grouped here because they share the same sourcing logic — each one is the durable, dated written record of a judgment call a SOC manager made under some amount of uncertainty, and each one is the artifact a skeptical board member, a future reviewer, or the manager's own successor eventually goes looking for.

[SENIOR MANAGER] None of the four templates below teaches the judgment they record. The board deck skeleton assumes the translation recipe built in Part 24 §3 and the metric-selection scorecard in Part 24 §6 have already run; it only formats what those sections produce. The risk-acceptance memo assumes the four-dimension scoring in Part 25's Table 25.1 has already been worked through honestly; it only forces that scoring to be dated and written down before the decision gets made or defaulted into by inaction. The post-incident review record and the corrective-action tracker both assume the blameless-facilitation mechanics in Part 29 §4 and the taxonomy in Part 29's Table 29.2 are already in use; they only give those mechanics somewhere permanent to live. Fill in any of these forms without doing that upstream work first, and the result is a well-formatted document that records a decision nobody actually made carefully.

[SENIOR MANAGER] Use this appendix by copying the relevant fenced block, filling every field, and keeping the result somewhere that survives past the meeting that produced it — a shared risk register, a tracked ticket, a versioned document store. A template filled out once and left in a slide deck's speaker notes is functionally the same as not having filled it out at all, which is the exact failure both [CASE-2501](#) (Part 25) and [CASE-2902](#) (Part 29) trace back to.

1. Board deck template — [TMPL-2401](#)

[EXECUTIVE] The SOC manager or the CISO uses this skeleton to build the quarterly (or per-bylaws) board or audit-committee deck described in Part 24 §5. Start from all five sections every cycle and drop any section with nothing real to report — a slide added only to fill the slot is the same failure mode as a metrics dashboard nobody prunes. This template does not decide which metrics belong in the deck or how to translate a raw number into a risk narrative; that's Part 24 §3's translation recipe and §6's scorecard, applied before a single field below gets filled in.

TEMPLATE – Board deck template (five-section skeleton), permanent ID TMPL-2401

SECTION 1 -- RISK POSTURE SUMMARY (1 slide, 3-5 lines)

<!-- One line per metric that scored 6+ on Part 24 S6's selection scorecard. -->

Metric	This Quarter	Last Quarter	Trend	Risk Translation (one sentence)	What This Should Prompt the Board To Do
---	---	---	---	---	---

SECTION 2 -- INCIDENTS OF NOTE (1 slide per incident; at most 2-3 incidents)

<!-- Source this from the incident's own post-incident organizational review (TMPL-2901), not from memory. -->

Incident ID	Severity Tier	What Happened (2-3 sentences)	What the Response Demonstrated	Source Review
---	---	---	---	---

SECTION 3 -- TREND METRICS (1 slide)

<!-- Reuse Section 1's row format for any metric tracked but not urgent enough to lead with. -->

Metric	Trend (4-quarter view)	One-Sentence Mechanism
---	---	---

SECTION 4 -- RESOURCING / SPEND ASK (only when there's an actual ask -- omit otherwise)

<!-- State the ask as a decision, not a status update: what the board approves, not what's happening. -->

The Ask (dollars or headcount)	Risk It Reduces	Consequence If Not Funded	Owner
---	---	---	---

SECTION 5 -- FORWARD RISK ITEMS (1 slide)

<!-- Pull directly from the open risk-acceptance register (TMPL-2501 entries) -- this is not a new list. -->

Risk Item	Current Acceptance Level	Expected Escalation/Review Date	Source Memo
---	---	---	---

SKEPTICAL-QUESTION PREP (not presented; rehearsed beforehand per Part 24 S7)

<!-- Fill this in for every question you can anticipate, not just the comfortable ones. -->

Anticipated Question	Headline Answer	Evidence Ready	Honest Bound
---	---	---	---

Manager's Note Fill the skeptical-question-prep table before you finalize a single slide, not after the deck is done. If you can't write a headline, evidence, and an honest bound for the question you're most afraid of getting, the deck itself isn't ready yet, no matter how polished the other five sections look.

2. Risk-acceptance memo template — TMPL-2501

[SENIOR MANAGER]

The SOC manager, or whoever the escalation path in Part 25 §2.1 names, fills this out the moment a known, standing risk is first identified — not months into living with it — to force Table 25.1's four-dimension scoring into writing before the risk gets formally accepted, escalated, or defaulted into by nobody stopping the process carrying it. The memo does not calculate the dollar-exposure figure or verify the compensating control on its own; both come from the manager's own analysis, informed by Part 5's and Part 9's models where relevant. What the memo forces is that the reasoning gets dated, written down, and given a calendar date to be checked again — the exact discipline CASE-2501 shows going missing for six months.

TEMPLATE — Risk-acceptance memo, permanent ID TMPL-2501

Risk ID / short title:

Date identified:

Description of the gap (plain language, not the fix -- state what is actually wrong):

TABLE 25.1 SCORING

<!-- A risk landing in the right-hand column on any single row is a strong candidate for escalation even if the other three rows look manager-level. -->

| Dimension | Manager-Level Signal Present? (Y/N + evidence) | Board/CISO-Level Signal Present? (Y/N + evidence) |

|---|---|---|

| Dollar exposure if it materializes | | |

| Reversibility | | |

| Blast radius (validated, not assumed) | | |

| Duration / standing nature | | |

Decision: Accepted at [SOC manager / CISO / Board] level

Reasoning (written now, not reconstructed later):

Compensating control relied upon, and how/when it was last verified:

<!-- Name the specific control and the date someone last actually re-checked it -- not the date it was designed or first approved. -->

Re-check date (mandatory -- re-verifies the fact this acceptance depends on, not just that the memo still exists on file):

What specific, checkable evidence would change this decision, and by when could it arrive:

Sign-off: Name / Title / Date

COMPOSITE CASE EXAMPLE — illustrative filled memo, based on CASE-2501 (Part 25 §2.3). Risk ID: legacy file-cluster coverage gap. Dollar exposure: worst case roughly \$1.2 million, above the organization's \$250,000 manager-authority threshold — a board/CISO-level signal on that row alone. Reversibility and duration: contained within a named 14-month decommission window — manager-level signal. Blast radius: confirmed bounded by a verified firewall-segmentation rule at the time of signing — manager-level signal. Decision: accepted at the SOC-manager level, reasoning that the verified segmentation boundary made the realistic exposure far below the unmitigated worst case. Re-check date: the field this real case was missing, which is exactly what let a later firewall exception move the control's boundary for two months with nobody watching.

3. Post-incident organizational review template — TMPL-2901

[SENIOR MANAGER] The facilitator named for a given incident — never someone who worked the incident directly, per Part 29 §3.1 — fills this out for the per-incident review, scheduled 5 to 10 business days after the incident closes. The template's fixed field for a role-based timeline exists because a document that circulates outside the review room and still names an individual analyst against a specific action teaches the floor to stop volunteering honest detail, the exact failure CASE-2901 walks through. This template does not, by itself, make a review blameless — that depends on the facilitation mechanics in Part 29 §4.1 and the psychological-safety precondition Part 19 builds; a form filled out by a room that doesn't feel safe yet is still an incomplete record, however cleanly the fields are completed.

TEMPLATE – Post-incident organizational review, permanent ID TMPL-2901

Incident ID / near-miss flag ID:

Trigger reason: [mandatory severity tier / discretionary request / random sample / near-miss flag]

Review date: Incident closed on:

Facilitator (must be independent of this incident):

Attendees (by role, e.g. "on-call analyst," "detection engineer," not required to be a fixed roster):

TIMELINE (role-based language only -- "the reviewing analyst," "the on-call engineer," never a named individual, in any version of this document that leaves the room)

<!-- Ask what made the gap hard to see in time, not who missed it. -->

WHAT HAPPENED (blameless account -- system and process focus)

FINDINGS

<!-- Category taken from Table 29.2; log every finding here even if its fix is trivial -- a finding that never reaches the tracker (TMPL-2902) cannot be checked for recurrence later. -->

Finding (plain-language gap)	Taxonomy Category	Owner	Due Date	Routed To (part/other book)
---	---	---	---	---

Facilitator sign-off: SOC manager sign-off:

4. Corrective-action tracker — TMPL-2902

[SENIOR MANAGER] This is a standing register, not a per-incident form — every finding logged from every filled TMPL-2901 lands here, and the standing review board (Part 29 §3.2) queries this register on its own monthly or quarterly cadence, independent of any single incident's timing. The tracker does not run the dedup matching described in Part 29 §6.2 automatically; deciding whether a new finding matches a prior one "in substance, not exact wording" is a human judgment call the standing board makes each cycle, not a keyword search this register performs on its own.

TEMPLATE – Corrective-action tracker, permanent ID TMPL-2902

Field	Entry
---	---
Finding ID	
Gap statement (plain language)	
Taxonomy category (Table 29.2)	
Source incident / near-miss ID	
Owner (a named person, not a team)	
Due date	
Verification method (stated in advance -- what evidence proves this is fixed)	
Status: open / owner-reported done / independently verified closed	
Verifier (must not be the owner)	
Prior-match check: new finding / matches Finding ID ____	
Occurrence count (this exact gap, trailing 18 months)	
Escalation trigger: 3rd-or-later occurrence -> escalate beyond original owner	

People Risk Trap Letting a finding's own owner also mark it verified closed concentrates the same conflict Part 15's QA program names for a reviewer scoring their own report — a busy owner under pressure to show progress calls "a ticket was filed" the same thing as "the fix was verified," which is exactly the pattern CASE-2902 shows the same access gap surviving three separate times. Route the verifier field to someone other than the owner every time, even when it adds a short delay to closing the item.

Cross-references. Companion to Part 24 — Executive & Board Reporting (TMPL-2401, §5), Part 25 — Risk Acceptance & Manager Decision-Making Under Uncertainty (TMPL-2501, §2.4), and Part 29 — Post-Incident Organizational Review (TMPL-2901 and TMPL-2902, §3 and §5). See those parts for the translation recipe, the risk-acceptance scorecard, and the facilitation and taxonomy discipline this appendix's templates format but do not teach. CASE-2501, CASE-2901, and CASE-2902 (CASE-INVENTORY.md) are the worked examples this appendix's filled sample draws on.

Appendix A8 — Cross-Series Quick Reference: Where to Find It

Why this appendix exists

[CONCEPT] Part 1 — SOC Manager Foundations & the Series Map previews the three-book split in miniature: a short table mapping a handful of representative questions to whichever volume actually owns the answer, plus a "which book do I open" decision tree collapsed to three yes/no questions. Part 1 says explicitly that its own table is "a working summary, not the authoritative index" and points forward to this appendix for the full version. This is that full version: every distinct cross-reference this book's 33 parts actually make into *SIGNAL TO ACTION: The Complete SOC Playbook Handbook* and *The Detection Engineering Handbook V2*, compiled once, grouped by topic, and checked against each companion volume's own current `BOOK-INDEX.md` rather than trusted from memory.

[CONCEPT] This is the single place a reader checks before assuming a topic is missing from this book. If a paragraph anywhere in Parts 1–33 cites a companion-volume part by number and name, that citation is supposed to resolve to a row below — per this book's own production model, a build check fails if it doesn't. Nothing below is a template or a worksheet; unlike Appendices A1–A7, this appendix has no fillable artifact, because its job is navigation, not documentation. Read Part 1 first for the thesis behind the three-book split (a SOC manager's job, a detection engineer's job, and a client-facing playbook's job are three different units of analysis, not three difficulty tiers of the same subject); read this appendix when you already know roughly what you're looking for and need the specific part number.

[SENIOR MANAGER] Maintenance discipline matters more for this appendix than for any other, because a stale row here is worse than no row at all — it sends a reader to a part number a companion volume has since renumbered. Every row below was checked against *SOC Playbook Handbook*'s and *Detection Engineering Handbook V2*'s own `BOOK-INDEX.md` files as of this appendix's `last_validated` date, not reconstructed from memory. Whoever adds a new cross-reference anywhere in this book adds the matching row here in the same pull request — that is the build-check contract `BOOK-INDEX.md` describes, not a suggestion.

1. How to read this table

[CONCEPT] Each section below groups a topic area and lists, for every distinct question this book's chapters actually route elsewhere, which companion part owns the answer and why a reader would go there. The **Cited From** column names which part(s) of *this* book carry that specific cross-reference today — useful for auditing a citation against its actual source paragraph, and for spotting a topic this book touches only glancingly. An em dash (–) means no cross-reference exists for that cell, not that the topic doesn't matter. Where a part number could be confused with a same-numbered or similarly-titled part in the *other* companion volume, that collision is called out explicitly in the section it falls under — §5 carries the sharpest one.

2. Staffing, Shift Design & Operating Model

[SENIOR MANAGER] The table below covers operating-model, tiering, and staffing questions that this book owns outright but that brush up against a companion volume's mechanics at a specific, narrow point.

Topic / Question	Book & Part	What You'll Find There	Cited From (this book)
Mechanics of a good Tier 1 → Tier 2 hand-off, on one ticket	SOC Playbook Handbook, Part 27 — Escalation Quality	What context has to travel, how to phrase an escalation note, how a manager audits hand-off quality at scale	Parts 1, 2, 3, 4, 6, 8, 9, 10, 11, 12, 15, 20, 26, 30, 32
How an individual alert's severity is scored	SOC Playbook Handbook, Part 29 — Playbook Severity Model	The per-ticket scoring mechanics this book's staffing and risk-acceptance decisions treat as an already-settled input	Parts 1, 3, 6, 9, 10, 22, 25, 26, 28, 29, 31
What to automate vs. gate for human review	SOC Playbook Handbook, Part 30 — Automation and SOAR	The automation/human-review boundary decision itself, distinct from this book's staffing consequences of that decision (Part 21, Part 32)	Parts 1, 5, 21, 26, 31, 32
Client-facing and third-party stakeholder communication content and timing	SOC Playbook Handbook, Part 28 — SOC-to-Stakeholder Communication	Who owns what gets told to a customer, partner, or the public, and when — run in parallel with, never confused with, this book's upward-only cadence in Part 28	Parts 1, 28
Detection rule logic, telemetry, and query-language mechanics behind any tool a SOC operates or migrates	Detection Engineering Handbook V2, Parts 3–29	The technical content this book cites by name rather than re-derives whenever a staffing, hiring, or tooling decision touches a rule's internals	Parts 1, 2, 3, 10
SIEM/EDR/SOAR platform-migration engineering cost (field mapping, parity testing, dual-running)	Detection Engineering Handbook V2, Part 6 — Normalisation	The component cost model this book's TCO worksheets (Part 21) and budget business case (Part 20) pull directly rather than re-price	Parts 1, 2, 21, 33
Detection-as-code discipline: git-based rule review, ownership metadata, version control	Detection Engineering Handbook V2, Part 22 — Detection as Code	The review-pipeline standard a cross-trained detection engineer (Part 12), a promoted branch candidate (Part 13), a departing engineer's rule-ownership metadata (Part 14), or a new threshold-tuning role (Part 32) is held to	Parts 2, 12, 13, 14, 32

3. Escalation, Severity & Stakeholder Communication

[CONCEPT] This section repeats the two most heavily cited SOC Playbook Handbook parts on their own, because "escalation" and "severity" are the two topics this book's chapters reach for a companion citation on most often — worth its own quick lookup rather than only appearing folded into §2's broader table.

Topic / Question	Book & Part	What You'll Find There	Cited From (this book)
Escalation-quality mechanics for a single ticket	SOC Playbook Handbook, Part 27 — Escalation Quality	See §2 above — repeated here because it is this book's single most frequently cited companion part	Parts 1, 2, 3, 4, 6, 8, 9, 10, 11, 12, 15, 20, 26, 30, 32
Playbook severity scoring for a single alert	SOC Playbook Handbook, Part 29 — Playbook Severity Model	See §2 above	Parts 1, 3, 6, 9, 10, 22, 25, 26, 28, 29, 31
Contractual SLA response-time commitment vs. an internal severity-driven triage target	SOC Playbook Handbook, Part 29 — Playbook Severity Model	The internal scoring instrument this book's Part 22 explicitly distinguishes from a negotiated, remedy-backed MSSP contract clause	Part 22
Individual playbook breakdowns: which step failed, in which incident	SOC Playbook Handbook, Part 35 — Playbook Failure Examples	The single-playbook, single-incident lens Part 29 of this book deliberately does not re-walk — Part 29 owns the pattern across many incidents instead	Parts 1, 29

4. Metrics

[SENIOR MANAGER] One structural rule governs every row in this section: a metric's *definition* lives with whichever volume actually owns it — the companion Playbook Handbook for MTTR, MTTA, and false-positive rate, or this book itself for handle time, backlog age, and MTDD, none of which the Playbook defines. Metric *use* — headcount modeling, board narratives, benchmarking caution, QA scoring — stays in this book either way. Redefining a metric that belongs to the other volume is treated as a defect, not a stylistic choice.

Topic / Question	Book & Part	What You'll Find There	Cited From (this book)
What MTTR and false-positive rate precisely mean, and how they're instrumented consistently	SOC Playbook Handbook, Part 32 — Metrics	The fixed definitions every headcount model (Part 5), queue-health signal (Part 9), onboarding metric (Part 11), training review (Part 12), QA rubric (Part 15), attrition leading indicator (Part 18), budget cost-per-alert figure (Part 20), board translation (Part 24), and benchmark comparison (Part 31) treats as an already-settled input	Parts 1, 5, 9, 11, 12, 15, 18, 20, 24, 30, 31, 33
What this book means by "handle time," "backlog age," or "MTDD"	This book's own operational definitions — Part 5 §2 (handle time), Part 9 §1 (backlog age), Part 24 §3 (MTDD)	Terms the Playbook Handbook never defines; owned locally so headcount modeling (Part 5), queue-health signal (Part 9), and board translation (Part 24) have one audited definition instead of three local ones	Parts 1, 5, 9, 24

5. Detection Engineering & Telemetry

[CONCEPT] This is where a manager's decision most often brushes against technical territory this book has no business re-deriving — a QA review touches escalation quality, a performance conversation

touches false-positive engineering, a rationalization audit touches detection coverage. Every row below is a case where this book's own governing test (`STYLE-GUIDE.md` §1, Part 1 §2.1) routed the technical claim to a citation instead of a rewrite.

Topic / Question	Book & Part	What You'll Find There	Cited From (this book)
Vocabulary for triage judgment (event, signal, alert, disposition) and what collapsing "the rule matched" into "the activity was malicious" gets wrong	Detection Engineering Handbook V2, Part 1 — Detection Engineering Foundations	The foundational distinctions this book's hiring-assessment rubric (Part 8) and competency matrix (Part 10) borrow rather than reinvent	Part 8
Query-language mechanics across Sigma, KQL, SPL, AQL, YARA-L, and Elastic	Detection Engineering Handbook V2, Parts 24–29	The six query-syntax parts, carried through a canonical detection so a reader can compare semantics across languages rather than learning six disconnected ones	Part 1
Correlation engineering, baselining, threat intelligence as a scoring input, and risk-based detection	Detection Engineering Handbook V2, Parts 30–33	The analytic-technique dependency chain (correlation → baselining → threat intel → risk scoring) this book's series map previews but never develops	Part 1
False-positive tuning mechanics for a noisy detection rule	Detection Engineering Handbook V2, Part 38 — False Positive Engineering	The technical fix behind every case this book routes to a rule rather than a roster — a queue-health decomposition (Part 9), a performance diagnostic (Part 16), an alert-fatigue rotation decision (Part 17), a tooling migration (Part 21), a board translation (Part 24), or a risk-acceptance dual-track holding pattern (Part 25)	Parts 1, 3, 9, 16, 17, 24, 25

Blind Spot Both companion volumes have a chapter called "False Positive Engineering," at two different part numbers, covering two different things — a naming collision this book's own citations navigate correctly but a reader checking a companion index cold could easily miss. SOC Playbook Handbook, Part 25 — False Positive Engineering covers the client-facing tuning workflow and correlation-thinking side of the problem, filed under that book's Operations & Governance section alongside SIEM queries and client-approval workflow. Detection Engineering Handbook V2, Part 38 — False Positive Engineering covers the technical taxonomy, tuning mechanics, and exception-list-as-debt framing, filed under that book's Testing & Quality Engineering section. Every citation inside this book's 33 parts that reads "Part 38 — False Positive Engineering" means the Detection Engineering Handbook V2 chapter specifically — none of this book's chapters currently cite SOC Playbook Handbook's Part 25 version. If a reader's own question is closer to "how do we get a client to sign off on a tuning change" than "how do we measure and fix a rule's false-positive rate," SOC Playbook Part 25, not DEH Part 38, is the one to open.

6. Threat Hunting

[CONCEPT] Threat hunting shows up in this book only as an organizational-design and career-track question — where hunting sits relative to the tiers, how a hunter branch is staffed and protected, what a hunter candidate has to demonstrate to earn the title. The methodology itself lives entirely in one companion volume.

Topic / Question	Book & Part	What You'll Find There	Cited From (this book)
Threat-hunting methodology broadly: hypothesis formation, scoping, pivoting, converting a hunt into a detection candidate	Detection Engineering Handbook V2, Parts 34–36 — Threat Hunting Methodology	The technical content this book's specialist-tier design (Part 3), cross-training rotation (Part 12), and branch-promotion bar (Part 13) all cite rather than re-teach	Parts 1, 3, 12
Hypothesis formation and the specific discipline of stating, before touching the data, what a negative result would look like	Detection Engineering Handbook V2, Part 34 — Threat Hunting Fundamentals	The work-sample standard a threat-hunter branch candidate's completed-hunt record is measured against	Part 13
The range of hunt types (IOC-based, TTP-based, anomaly-based, intel-led, incident-led, gap-driven, retrospective)	Detection Engineering Handbook V2, Part 35 — Hunt Types	The taxonomy behind "at least two structured hunts" in a threat-hunter branch's entrance bar	Part 13

7. Detection Coverage, Quality & Debt

[SENIOR MANAGER] These three parts are the technical measurement layer underneath a large share of this book's judgment-call chapters — a queue-health decomposition, a tool-rationalization audit, a board translation, and the maturity synthesis in §11 below all consume this trio as a finished input rather than re-scoring anything themselves.

Topic / Question	Book & Part	What You'll Find There	Cited From (this book)
Testing whether a detection fires correctly, for the right reason, and survives a schema change, clock skew, or high volume	Detection Engineering Handbook V2, Part 37 — Detection Testing	The test-case patterns this book's ramp-certification exercises (Part 11) pull sanitized scenarios from	Part 11
Whether a validated, tested detection exists for a given technique in the organization's own threat profile	Detection Engineering Handbook V2, Part 41 — Detection Coverage	The six-tier coverage model this book's tool-rationalization audit (Part 23), cross-team severity-authority dispute (Part 26), and maturity synthesis (Part 30) all cite as the technical arbitration this book itself never performs	Parts 1, 20, 23, 24, 26, 30, 31
Precision/recall in SOC terms, alert-to-incident ratio, analyst handling time, and suppression tracking as measures of an existing detection's own quality	Detection Engineering Handbook V2, Part 42 — Detection Quality	The measurement this book's queue-health decision tree (Part 9) and board-facing translation of a false-positive-rate figure (Part 24) both route to before treating a lean ratio as good news	Parts 1, 9, 17, 20, 23, 24, 30, 31
Detection debt, telemetry debt, parser debt, and tuning debt as an inventory-and-payoff ledger	Detection Engineering Handbook V2, Part 43 — Detection Debt	The framework this book routes to whenever a recurring miss, a departing detection engineer's undocumented rule, or a third-time-recurring postmortem finding turns out to be a rule problem, not a people problem	Parts 1, 3, 10, 14, 16, 18, 19, 20, 25, 29, 31, 32
Whether a rationalization candidate is the sole source of coverage for a technique before it's retired	Detection Engineering Handbook V2, Parts 41–43	The combined coverage/quality/debt check a tool-sprawl retirement decision (Part 23) or a build-vs-buy business case (Part 20) cannot substitute for on its own	Parts 20, 23

8. AI Security

[CONCEPT] This book's AI content is scoped narrowly and deliberately: the staffing and organizational-design consequences of AI copilots and agentic automation, not the workflow mechanics of using one or the technical detection content an AI system itself generates. The gap between what this book covers and what a reader might actually be looking for under "AI security" is real enough to state plainly rather than paper over.

Topic / Question	Book & Part	What You'll Find There	Cited From (this book)
Which Tier 1 work is genuinely displaced by AI-assisted triage, and what that does to headcount, competency matrices, and career tracks	This book, Part 32 — AI, Automation, and the Future Shape of the SOC	This book's own territory, owned outright	—
Tool selection inside a triage workflow, confidence-threshold implementation, and human-review gating at the ticket level	SOC Playbook Handbook, Part 31 — AI-Assisted SOC Operations	The operational mechanics this book's Part 32 explicitly does not re-cover	Parts 1, 32
Prompt injection, RAG/knowledge-base poisoning, agent and MCP-tool abuse, AI API compromise — the technical detection content for an AI system itself	Detection Engineering Handbook V2, Part 20 — AI Systems Telemetry & Audit-Trail Engineering, Part 21 — AI Security Detection Engineering	The telemetry and analytic layers for AI-specific attacks — not currently cross-referenced from any part of this book ; a reader landing here from this book's index will not find a pointer to it anywhere else in these 33 parts	—
Per-alert playbooks for AI Security incidents (prompt injection, malicious file upload, agent abuse) and the multi-stage AI malicious-file-upload master playbook	SOC Playbook Handbook, AI Security playbook category (Category 18)	The client-facing, per-alert response playbooks for the same attack surface DEH V2 Parts 20–21 detect — also not currently cross-referenced from this book	—
The full attack-stage model for an AI system compromise (prompt/file → model → RAG → agent → tool → sensitive system → external destination)	Detection Engineering Handbook V2, Part 48 — AI Compromise Model	The applied compromise model for this domain, alongside the ransomware, identity, and web compromise models Part 28 of this book cites collectively as "Parts 44–48" without singling Part 48 out by name	—

9. Major Incidents & Post-Incident Review

[SENIOR MANAGER] Major-incident content is split three ways across the series on purpose, and this section's three rows are that split made literal: the technical attack-stage analysis, the analyst/incident-commander-level operational playbook, and the manager's-chair layer neither of the other two books has a reason to cover.

Topic / Question	Book & Part	What You'll Find There	Cited From (this book)
The incident-commander role and the technical containment/eradication sequence for a ransomware event	SOC Playbook Handbook, Ransomware Master Playbook	The operational playbook this book's Part 28 explicitly does not re-derive — the manager's-chair complement, not a replacement	Parts 1, 28
The technical attack-stage analysis behind a specific compromise (ransomware, identity, web, AI)	Detection Engineering Handbook V2, Parts 44–48 — Applied Compromise Models	The third leg of the series' deliberate three-way split of major-incident content — cited once by this book's Part 28 and not referenced again, because none of that technical detail changes what a manager decides	Part 28
Insider-threat detection and escalation mechanics: the behavioral indicators and correlation logic that confirm a finding	SOC Playbook Handbook, Insider Threat Playbooks (Category 19)	The detection-and-confirmation mechanics this book's Part 27 explicitly starts after — Part 27 owns the organizational aftermath of an already-confirmed finding, not detecting one	Part 27
Individual playbook breakdowns inside one incident's own response	SOC Playbook Handbook, Part 35 — Playbook Failure Examples	See §3 above — the single-incident lens Part 29's organizational review deliberately does not duplicate	Parts 1, 29
Client-facing communication cadence during a live major incident	SOC Playbook Handbook, Part 28 — SOC-to-Stakeholder Communication	See §2 and §3 above — runs in parallel with, and is never the same conversation as, this book's upward-only board cadence	Parts 1, 28

10. Playbook Mechanics, Testing & Governance

[CONCEPT] A small cluster of citations exists purely to keep this book from re-authoring playbook structure or test methodology anywhere a manager's chapter happens to touch a playbook in passing.

Topic / Question	Book & Part	What You'll Find There	Cited From (this book)
How to write or structure a specific playbook, from a bare detection rule to a finished document	SOC Playbook Handbook, Foundations / Playbook Library	Playbook mechanics across the whole series — the destination for tribal knowledge this book's Part 14 identifies as needing to be written down, once it's ready to become a playbook	Parts 1, 14
Simulation and lab-based methodology for producing realistic test telemetry	SOC Playbook Handbook, Part 34 — Playbook Testing	The methodology this book's ramp-certification program (Part 11) and playbook-quality maturity model (Part 30, via Appendix 38A) both cite rather than re-derive	Part 11

11. Maturity Models

[SENIOR MANAGER]

Three maturity models exist across the series, deliberately kept from collapsing into one blended score — Part 30 of this book is the synthesis chapter that reads all three together without letting a strong score on one stand in for the others.

Topic / Question	Book & Part	What You'll Find There	Cited From (this book)
Organizational-design maturity: staffing ratios, process maturity, training-program maturity, tooling maturity	This book, Part 30 — SOC Maturity Models	This book's own model, owned outright	—
Playbook-quality maturity: whether a specific playbook is complete, clear, and produces a proper hand-off	SOC Playbook Handbook, Appendix 38A — Playbook Maturity Model	The structural-quality lens this book's Part 30 cites and never re-scores	Part 30
Detection coverage/quality maturity: whether a validated, tested detection exists per technique, and performs well	Detection Engineering Handbook V2, Part 41 — Detection Coverage, Part 42 — Detection Quality	The technical-content lens Part 30's three-lens synthesis reads alongside the other two, in that order, when sequencing an investment decision	Part 30

Cross-references

[CONCEPT]

This appendix is the standing, build-checked expansion of Part 1 — SOC Manager Foundations & the Series Map §2's series-map table; every cross-book citation added anywhere in Parts 2–33 must resolve to a row above.